

TidesDB 9

Unified Manual

<https://tidesdb.com>

TidesDB Corp.

Table of Contents

1. What is TidesDB	3
2. How TidesDB Works	6
3. Building & Benchmarking	45
4. C Reference	80
5. C++ Reference	175
6. C# Reference	231
7. Go Reference	287
8. Python Reference	351
9. Java Reference	394
10. Rust Reference	440
11. TypeScript Reference	511
12. Lua Reference	570
13. TideSQL Reference	625
14. Kafka Streams Plugin	701
15. Admintool Reference	733
16. Tuning Reference	770

What is TidesDB?

TidesDB is a fast, light-weight and efficient embeddable key-value storage engine library written in C. Built on a log-structured merge-tree (LSM-tree) architecture, it provides a foundational library for building database systems or can be used directly as a standalone key-value or column store.

Key Characteristics

TidesDB is designed for direct integration into applications. It handles keys and values as raw byte sequences without size restrictions (up to system memory limits), giving you full control over serialization. Optimized for write-heavy workloads, it maintains efficient reads through bloom filters, caching, block indices, and compaction.

Core Features

- ACID transactions with MVCC supporting 5 isolation levels: `READ_UNCOMMITTED`, `READ_COMMITTED`, `REPEATABLE_READ`, `SNAPSHOT`, and `SERIALIZABLE`. Serializable isolation uses SSI (Serializable Snapshot Isolation) with read-write conflict detection to prevent all anomalies including write-skew. Transactions support savepoints for partial rollback and read-your-own-writes semantics.
- Multi-column family transactions with atomic all-or-nothing semantics. Each CF's WAL receives transaction operations with the same global commit sequence number, ensuring atomicity through sequence-based ordering.
- Column families provide isolated key-value stores with independent configuration (memtable size, compression, bloom filters, compaction parameters). Runtime configuration updates without restart.
- Bidirectional iterators with heap-based merge-sort across memtables and SSTables. Snapshot isolation ensures consistent iteration during concurrent writes and compactions. Seeking positions at the first key $\geq$ the target, enabling prefix-style range scans.
- Hybrid and adaptive compaction with three modes: full preemptive merge (minimize space amplification), dividing merge (create partition boundaries), and partitioned merge (minimize write amplification). Dynamic Capacity Adaptation (DCA) automatically scales level capacities based on data size. Dynamic level management adds/removes levels on demand.
- Automatic crash recovery reconstructs memtables from write-ahead log (WAL) files on startup. WAL entries are replayed into skip lists and queued for flush to disk.
- Optional bloom filters (configurable false positive rate) reduce disk reads for absent keys. Built during SSTable creation and persisted in metadata.

- Key-value separation (WiscKey-style) with configurable threshold. Small values stored inline in klog, large values in vlog with offset reference. Reduces write amplification during compaction.
- TTL support for automatic key-value expiration. Expired entries skipped during reads and removed during compaction.
- Custom comparators with 6 built-in comparators: memcmp, lexicographic, uint64, int64, reverse, case_insensitive. Used consistently across skip lists, SSTables, and compaction.
- Lock-free block manager using `pread/pwrite` for concurrent I/O. Reference-counted blocks with atomic operations. xxHash32 checksums for integrity. Supports up to 4GB blocks with partial reads.
- Two-tier caching
 - File handle cache with LRU eviction (default 512 open SSTables). Background reaper closes oldest unused files.
 - NUMA-aware block cache using partitioned CLOCK eviction. Caches raw klog block bytes with zero-copy API and reference bit protection.
- Background thread pools for flush and compaction (default 2 threads each). Work queues distribute tasks. Compaction auto-triggers when Level 1 reaches (default 4) files.
- Three sync modes · `TDB_SYNC_NONE` (OS-managed), `TDB_SYNC_FULL` (fsync every write), `TDB_SYNC_INTERVAL` (periodic sync). Structural operations always enforce durability.
- Compression support · LZ4, LZ4-FAST, Zstd, Snappy (configurable per column family). Applied to klog and vlog blocks, not WAL.
- Block indexes for fast seeks. Sample every Nth block (configurable ratio, default 1 = every block) storing prefix boundaries and file positions for binary search.
- Cross-platform · Linux, macOS, Windows, BSD variants, Solaris/Illumos on x86, ARM, RISC-V, PowerPC (32-bit and 64-bit). Comprehensive platform abstraction layer.
- File portability · Little-endian serialization throughout. Database files work across any platform/architecture without conversion.
- Clean C API · Returns `TDB_SUCCESS` (0) on success, negative error codes on failure (`TDB_ERR_MEMORY`, `TDB_ERR_INVALID_ARGS`, `TDB_ERR_NOT_FOUND`, `TDB_ERR_IO`, `TDB_ERR_CORRUPTION`, `TDB_ERR_CONFLICT`, etc.). Configurable debug logging with 6 levels: `TDB_LOG_DEBUG` (most verbose), `TDB_LOG_INFO`, `TDB_LOG_WARN`, `TDB_LOG_ERROR`, `TDB_LOG_FATAL`, and `TDB_LOG_NONE` (disable). Log level set via `tidesdb_config_t` at database open. Timestamped log output to stderr with file/line information.
- Optional `mimalloc`, `tcMalloc`, and `jemalloc` support
- Optional hybrid B-tree + LSM-tree which can be configured per column family
- Ability to clone column families
- Easy pluggable custom allocator support
- Object store mode with S3-compatible storage, local file caching, and node-failure recovery

- Primary/replica replication over the object store · one primary and many read-only replicas share a bucket; replicas poll and lazily fetch, a replica promotes to primary on failure with a catch-up sync, and single-writer fencing (a lease epoch claimed via conditional writes) stops a superseded-but-alive primary from causing split-brain. Optional synchronous WAL upload for RPO=0 failover.

Community

Join the [TidesDB Discord Community](#) to ask questions, work on development, and discuss the future of TidesDB.

How does TidesDB work?

v9.3.11

Introduction

This document explains how TidesDB works, from the idea it is built on down to the bytes on disk. It is organized so that understanding accumulates: each section relies only on what came before it. We start with the whole system in miniature, then name its parts, and only at the end open up the on-disk formats, the modules that implement everything and testing mechanics.

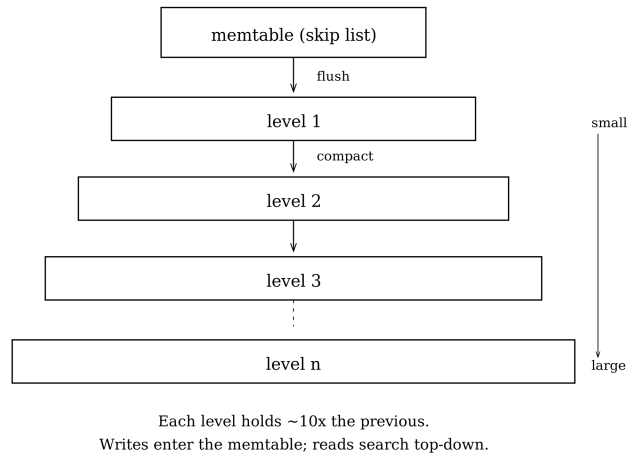
TidesDB is an embeddable key-value storage engine built on the log-structured merge tree, or LSM tree. The LSM tree rests on one old and well-understood bargain. Random writes scattered across a large sorted structure on disk are slow, because each one forces a seek and a rewrite. So instead of writing in place, an LSM tree batches writes in memory and then flushes them to disk all at once, as a sorted run. Writes become fast and sequential.

Nothing is free, and the bargain has two costs. The first is write amplification: the same data is rewritten several times as those sorted runs are later merged together. The second is read amplification: a single lookup may have to search several sorted runs before it finds the key, because the newest copy could be in any of them. Much of what follows is machinery for keeping both costs small.

On top of that foundation, TidesDB adds what a production engine needs: ACID transactions with five isolation levels, and data organized as a hierarchy of sorted files called sorted string tables, or SSTables. The hierarchy is arranged in levels, each holding roughly N times more data than the level above it. A background process called compaction continually merges SSTables from adjacent levels, throwing away obsolete entries and reclaiming space as it goes.

Here is the whole lifecycle of a piece of data in one paragraph (Figure 1). A write first lands in a structure called a memtable, which comprises a sorted data structure called a skip-list and a block-managed WAL, known as a write-ahead log. The write is appended to the WAL and then the skip-list, so it survives a crash. When the memtable grows past a configured write buffer size, it is frozen and made immutable, and a background worker writes it to disk as an SSTable at the top level of the hierarchy. SSTables accumulate there, and compaction merges them downward into larger, deeper levels. A read walks this same structure from newest to oldest and stops at the first copy of the key it finds. Everything else in this document is a refinement of that loop.

Fig. 1. Data flow through the LSM tree.



(TidesDB builds its memtable from a skip list rather than a balanced tree. The reasons are practical: a skip list is simpler to implement correctly, and its structure lends itself naturally to lock-free concurrent access. The [Skip List](#) section returns to this once the rest of the system is in view.)

The Data Model

Before following a write or a read through the system, we need names for the parts. This section introduces them at the level of *what each thing is and why it exists*. The exact byte layouts are deferred to [On-Disk Format](#), once the parts have been seen in motion.

Column Families

A TidesDB database is divided into column families. A column family is an independent key-value namespace: it has its own configuration, its own memtables and write-ahead logs, and its own levels of SSTables on disk. The isolation between them is complete, which is what lets two column families in the same database use entirely different compression algorithms, key orderings, and tuning parameters.

In the default mode, each column family owns one active memtable that receives new writes, a queue of frozen memtables waiting to be flushed, a write-ahead log paired with each memtable, up to 32 levels of SSTables on disk, and a manifest that records which SSTables belong to which level. There is also an alternative arrangement, the unified memtable, in which all column families share a single memtable and a single log; it solves a specific write-amplification problem and is covered with the write path, under [Unified Memtable](#).

The Memtable

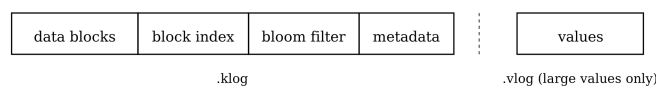
The memtable is where writes live with a write ahead log before a sorted run creates an SSTable pair.

Sorted String Tables

An SSTable is an immutable sorted run of key-value pairs on disk. Once written, it is never modified; it is only ever read, or eventually replaced by compaction. Immutability is what makes the whole system safe to read concurrently without locks.

Three properties of an SSTable matter for everything that follows (Figure 2). First, its entries are sorted by key, so the engine can binary-search within it. Second, it carries a small bloom filter — a compact probabilistic structure that can say “this key is definitely not here” without touching the data, which lets a lookup skip most SSTables outright. Third, it carries a block index that maps key ranges to positions in the file, so a lookup can jump near the right place instead of scanning from the start.

Fig. 2. SSTable file layout.



An SSTable is physically two files: a key log holding keys and small values, and a value log holding large values off to the side. Small values are stored inline with their keys; values at or above a threshold (512 bytes by default) are written to the value log, and the key log keeps only an offset pointing to them. This split keeps the key log compact and fast to scan during lookups and iteration, while still accommodating values of any size. The key log is divided into compressed blocks of about 64KB, and the bloom filter and block index sit at the end of it. (The exact framing of all this — entry layout, block structure, how the bloom and index are serialized — is in [On-Disk Format](#).)

A column family may optionally store its key log as a B+tree instead of the default block layout. The trade is more work at flush time in exchange for faster point lookups and faster seeks into the middle of a table, which suits workloads dominated by random reads. The two formats are interchangeable from the rest of the engine’s point of view; the B+tree’s structure and serialization are described in [On-Disk Format](#).

Levels

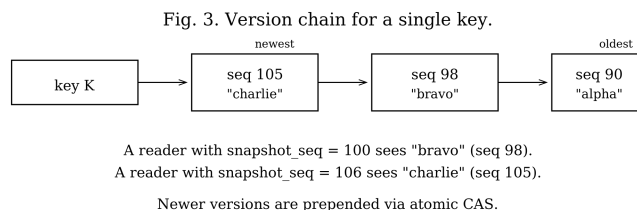
SSTables are organized into a hierarchy of levels, numbered from the top. A flush always produces an SSTable at level 1. Compaction merges level 1 into level 2, level 2 into level 3, and so on, with each level holding roughly N times more data than the one above it (N is configurable, default 10). The deeper a level, the larger and older its data. This shape is the source of the read-amplification cost from the introduction: a key could live in any level, so a lookup may have to check several of them. The bloom filters and block indexes are what keep that check cheap, and compaction is what keeps the number of levels small. How the levels are searched is the subject of the [Read Path](#); how they are kept in balance is the subject of [Compaction](#).

Transactions and Visibility

Many threads read and write a TidesDB database at once, and a reader must see a consistent picture even while writers are changing it. This section explains how. It comes before the write and read paths because both of them constantly refer to the machinery built here: snapshots, sequence numbers, and version chains. TidesDB is fully ACID and offers five isolation levels, from Read Uncommitted to Serializable, all built on a single underlying mechanism — multi-version concurrency control.

Multi-Version Concurrency Control

The core idea is that the engine never overwrites a value in place. Instead, each write creates a new *version* of the key, stamped with a sequence number, and the versions for a key form a chain ordered newest-first (Figure 3). A reader does not see “the value”; it sees the newest version whose sequence number is at or below its own snapshot. Writers add versions while readers walk the chain, and neither blocks the other.



Every transaction is given a snapshot sequence number when it begins, and that number defines what it can see. The rule for choosing it depends on the isolation level: Read Uncommitted uses the maximum possible value so it sees everything, including uncommitted writes; Read Committed refreshes its snapshot on every read so it always sees the latest committed data; and Repeatable Read, Snapshot, and Serializable take `global_seq - 1`, freezing their view at everything committed before they started.

A commit assigns a fresh commit sequence number from a single global atomic counter, writes the operations to the WAL, applies them to the active memtable under that sequence, and then marks the sequence committed in a fixed-size circular buffer (65536 entries). Readers consult that buffer and skip any version whose sequence is not yet marked committed, which is how a half-finished commit stays invisible. The buffer wraps, and the visibility check accounts for the wrap rather than risking a wrong answer: a sequence that has fallen more than the buffer size behind the high-water mark has had its slot recycled by a newer sequence, so the check treats it as committed. That is sound because a version only becomes visible to a reader after its commit applied it, and a sequence that far back cannot still be in flight — the same assumption recovery makes when it backfills only the trailing window. Without it, an old committed version would read its recycled slot as the newer sequence’s in-progress state, and a snapshot read would fall through to a stale older value.

Two refinements keep this correct under real workloads.

The first protects long-running readers. Compaction would like to throw away superseded versions, but a slow reader holding an old snapshot may still need one. So the engine tracks the *minimum snapshot sequence held by any live reader* and treats it as a floor: at every point where compaction or flush would drop a superseded version, it keeps any version whose sequence is at or above that floor. Without the floor, a long scan could watch a value vanish mid-iteration because a background merge decided the newer version was enough. The floor is recomputed at the start of every compaction and flush, so once a reader finishes — by commit, rollback, or being freed — the versions it was pinning become collectable by the next merge. Freeing a transaction also removes it from the active list as part of teardown, so a caller who frees without committing or rolling back cannot strand a dangling snapshot that compaction would later try to honor.

The second refinement extends the same guarantee to iterators. When an iterator is created, it captures its sources up front: the active memtable, the frozen memtables, and — in unified mode — every entry still sitting in the unified immutable queue. That last source matters because a flush worker may have frozen the shared memtable but not yet written its contents to SSTables; without snapshotting the immutable queue, the iterator would find those entries in neither memory nor on disk. With it, the iterator sees every committed write visible to its snapshot, no matter where on the flush pipeline that write currently sits.

The Five Isolation Levels

The five levels differ in how much they check at commit time, and that is the whole of the difference.

Read Uncommitted sees every version, committed or not. Its snapshot is the maximum sequence value.

Read Committed does no validation and refreshes its snapshot on each read, so it always reflects the most recently committed data.

Repeatable Read remembers every key it read, along with the version it saw. At commit it checks whether any of those keys gained a newer version in the meantime, and aborts if so.

Snapshot Isolation detects write-write conflicts only, with first-committer-wins. Two concurrent commits to the same key cannot both win: at commit, before anything is written, each write key claims a slot in a fixed reservation table, stamping it with a fingerprint of the key alongside the committing sequence. The slot is shared by every key that hashes to it, so a committer that finds it already holding a sequence past the version it read checks the stored fingerprint — a match is a genuine same-key conflict and the committer aborts having written nothing, while a mismatch is an unrelated key colliding into the same slot and is ignored, unless that colliding sequence is recent enough that another writer of the other key could still depend on the slot, where it stays conservative. The compare-and-swap lets exactly one of two racing same-key committers take the slot, and the decision needs only the slot and the oldest live snapshot, never another committer's data. The reservation validates each write against the version the transaction actually read, not merely its snapshot start — so Snapshot does

record its point reads, purely for that check — which closes the window where a writer becomes visible between the read and the commit. It does no read-write conflict detection, and it deliberately allows write skew — two transactions reading overlapping data and writing disjoint keys — because that matches the textbook definition, under which snapshot isolation requires only write-write conflict detection.

Serializable adds read-write conflict tracking on top of snapshot isolation, implementing serializable snapshot isolation (SSI). Repeatable Read and Serializable allocate a full read set for that tracking — Snapshot keeps only the lighter record of its point reads described above — and once a read set passes 64 entries it is backed by an xxHash table for $O(1)$ conflict checks. At commit the engine examines other concurrent serializable transactions: if transaction T read a key that another transaction T' wrote, it marks an outgoing conflict on T and an incoming conflict on T'. A transaction carrying both an incoming and an outgoing conflict is a pivot in a “dangerous structure,” and its commit aborts. This is a deliberately simplified SSI: it detects pivots but builds no precedence graph and does no cycle detection, so it can occasionally abort a transaction that was in fact serializable.

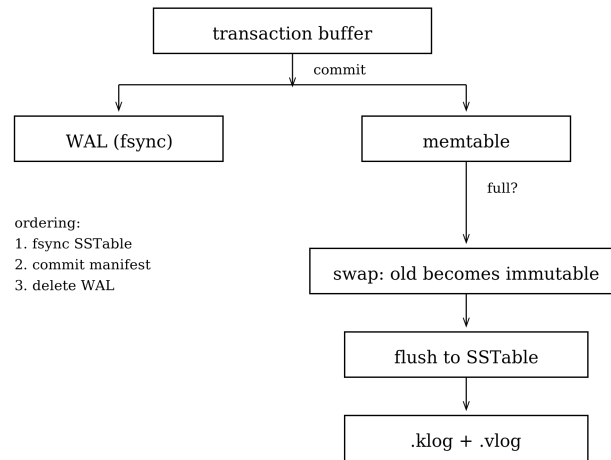
Transactions Across Column Families

A single transaction can span several column families, and TidesDB makes that atomic without the cost of two-phase commit. The transaction holds an array of every column family it touched, and at commit it stamps the operations in all of them with the *same* sequence number drawn once from the global counter. That shared number is the entire coordination mechanism: each column family's WAL records its own operations under it, and because visibility is decided by sequence number, the writes across all the column families become visible together, in one atomic step.

The Write Path

We can now follow a write from the application all the way to durable storage (Figure 4). The path has three parts: committing the transaction into the active memtable, flushing a full memtable to an SSTable, and the flow control that keeps writes from outrunning the flushers.

Fig. 4. The write path.



Commit

A transaction buffers its operations in memory and does nothing durable until commit. At commit the engine validates according to the isolation level, draws a commit sequence number from the global counter, reserves its write keys for first-committer-wins (Snapshot Isolation and above), writes the operations to each column family's WAL, applies them to the active memtable under that sequence, checks whether any memtable has grown past its flush threshold, and finally marks the sequence committed — which is the step that makes its writes visible to other readers.

For Snapshot Isolation and above, validation must check each key in the write set for conflicts against the active memtable, the immutable memtables, and the SSTables at every level. Done naively this would be expensive, so two optimizations cut the cost. The first skips whole SSTables: any SSTable whose maximum sequence number predates the transaction's snapshot cannot possibly contain a conflicting version, so it is skipped entirely — no bloom check, no block read. In a typical workload most SSTables are older than any live transaction, which removes the bulk of the conflict-detection I/O. The second optimization makes the remaining probes cheap: when an SSTable must be checked, the search runs in a sequence-only mode that finds the key through the bloom filter and block index but returns just its sequence number, allocating nothing and never touching the value log.

That scan, however, sees only writers that have already applied to the memtable, so two transactions committing the same key concurrently could each pass it without seeing the other. The reservation closes that race: drawn between the sequence number and the WAL write, each write key claims its slot — stamped with a fingerprint of the key, so a hash collision with an unrelated key is told apart from a real conflict — before anything is written, so a racing committer sees the claim and exactly one of them wins. The reservation table lives only in memory and is empty after recovery, so it changes nothing on disk, and because the loser aborts before its WAL write, an aborted transaction still leaves no trace.

A transaction also collapses its own redundant writes, keeping only the final operation per key. The deduplication table is built lazily, only once a transaction accumulates eight operations within a single column family, and uses a fast non-cryptographic hash; a hash collision simply writes both operations (the memtable handles duplicates correctly), so collisions are harmless. This shrinks the memtable when a transaction rewrites the same key many times.

Flushing a Memtable

When a memtable crosses its flush threshold, the engine hands the old one to a flush worker and swaps in a fresh empty memtable: it enqueues the frozen memtable, publishes the updated immutable snapshot, and then republishes the active pointer with a single atomic store and a memory fence for visibility. The worker then walks the frozen memtable in sorted order and builds an SSTable: it writes entries into 64KB blocks, sends large values to the value log, optionally compresses each block, and appends the block index, bloom filter, and metadata.

The threshold itself is not fixed. It adapts to how far behind the flushers are, trading write batching against memory pressure. When the flush queue is empty the threshold rises to 150% of the configured write-buffer size, letting the memtable accumulate more before flushing, which batches better and helps throughput. Under moderate pressure it falls to 125%. Once the queue is half-full or global memory is tight it drops to exactly the write-buffer size, with no slack, forcing an immediate flush. With the default 64MB buffer the effective threshold ranges from 64MB under pressure to 96MB when idle.

The final steps of a flush run in a specific order, and the order is load-bearing: fsync both files, add the SSTable to level 1, commit the manifest, then delete the WAL. Fsync before the manifest commit ensures the SSTable is durable before it becomes discoverable. The manifest commit before the WAL deletion ensures that if the machine dies in between, recovery can still find the data in the WAL.

That ordering produces exactly two crash windows, both benign. A crash *after the fsync but before the manifest commit* leaves an SSTable on disk that the manifest does not mention; recovery sees it is unreferenced and deletes it. A crash *after the manifest commit but before the WAL is deleted* leaves both the committed SSTable and the WAL; recovery replays the WAL again and produces a second SSTable, with a new ID, holding the same data. The duplicate is harmless, because every version carries its original commit sequence — reads still return the correct newest version, and a later compaction merges the copies away.

WAL files and SSTables are validated differently on read, for the same reason the flush order exists. A WAL is validated permissively: if its last block is torn or incomplete, the engine walks back to the last intact block and truncates there, because a crash mid-write is a normal thing for a log to suffer. An SSTable is validated strictly: any corruption in its last block rejects the whole table, because an SSTable is permanent and the manifest already promised it was complete.

Unified Memtable

The default arrangement gives each column family its own memtable and WAL, which means a transaction touching N column families performs N separate WAL writes. For some workloads that is wasteful. Consider a database plugin that models one logical table as a data column family plus several secondary-index column families: every transaction touches all of them, and WAL I/O grows linearly with the number of indexes.

Unified memtable mode removes that multiplier. All column families share one memtable and one WAL at the database level, so a transaction writes its WAL once regardless of how many column families it spans. It is enabled with `unified_memtable = 1`, and companion settings tune the shared buffer size, skip-list parameters, and WAL sync behavior.

The obvious difficulty is keeping column families from seeing each other's keys in a shared structure. The solution is to prefix every key. Each column family gets a unique 4-byte index at creation, and every key in the shared memtable is stored with that index, big-endian, in front of the user key. The prefix does two jobs at once. It groups each column family's keys into a contiguous sorted range (all of family 0, then all of family 1, and so on), and it makes every lookup an exact match on the prefixed key, so other families' keys are simply never seen. Because the shared structure has one sort order, all families in it must use the default byte-wise comparator; a custom comparator is rejected at creation. Byte-wise ordering sorts the big-endian prefix numerically and then the user keys in byte order, which is exactly what is wanted.

That 4-byte index has to survive restarts, and the consequence of getting it wrong is severe: if a column family's index changed between runs, the entire WAL would replay under the wrong families — silent data loss, not a crash. So the name-to-index map is persisted in a file (`UNIMAP`) at the database root, rewritten atomically whenever it changes, and keyed on column family *name*. Name is the only identity that outlives a crash; directory order, manifest order, and the atomic counter all reset, but the name written into the config does not. `UNIMAP` is loaded before any WAL is replayed, so each family re-registers under the same index it held before, the counter is advanced past the maximum loaded index so new families never collide, and in object-store mode the file is uploaded with the manifest so a cold replica reconstructs the primary's indexes rather than inventing its own.

The rest of unified mode follows from the prefix.

Writing serializes all of a transaction's operations, across all its column families, into one WAL batch and applies them to the shared memtable with their prefixed keys — one WAL write where per-CF mode would have done several.

Reading a key from a given family builds the prefixed key and searches the shared active memtable, then the shared immutable memtables newest-first, then falls back to that family's own SSTable levels. Immutable memtables already flushed are skipped via a flag, so a lookup never returns data that is already durable in an SSTable.

Flushing is where the shared structure has to fan back out into per-family SSTables, and it is done in two phases so the work parallelizes. Phase one is a single cheap pass over the frozen memtable: because keys are sorted by family index first, each family's entries form one contiguous run, and the pass simply records, for each family, how many entries its run holds. Phase two dispatches one flush task per run onto the shared worker queue. Any worker can pick up any task; each task streams its family's run straight into an SSTable at that family's level 1, stripping the 4-byte prefix as it writes. There is no intermediate copy, because the run is already in the family's sort order. The task that finishes last closes the WAL segment, optionally uploads it, and marks the memtable flushed. With N non-empty families and M workers, the per-family writes run in about N/M waves instead of N . If a task's allocation fails the dispatcher writes that run inline so nothing is lost, and a failure in one task is recorded and logged while its siblings keep going.

Rotating the shared memtable mirrors the per-CF case but must be airtight, because one memtable now backs the whole database. A compare-and-swap admission gate lets only one thread rotate at a time. Three invariants make the handoff safe: the public flush entry point takes that gate *before* rotating, so two racing callers cannot rotate twice and enqueue the same memtable twice; a writer captures the active pointer, takes a reference, then rechecks that the pointer is still active and retries on the new memtable if a rotation slipped in, so it can never write into a memtable that has already been frozen; and the flush worker waits for the writer reference count to drain before closing the WAL, so an in-flight write always finishes before the file descriptor it is using is torn down.

Two smaller points complete the picture. Backpressure is still applied per column family even though the memtable is shared, so a single family that is falling behind on flush or compaction still throttles its own writes. And each unified WAL has a strict one-to-one relationship with the immutable memtable it backs — no reference counting — so once a flush has demuxed all entries into SSTables, the WAL segment is uploaded if replication is on and then deleted.

Backpressure and Flow Control

When writes arrive faster than the flush workers can drain memtables, frozen memtables pile up and, without intervention, memory grows without bound. TidesDB applies *graduated* backpressure rather than a single on/off stall, and that gradualness is most of why its write throughput degrades smoothly under load instead of collapsing.

Backpressure watches three signals: the depth of the flush queue (call it $L0$; default stall threshold 10), the number of SSTables at level 1 (default trigger 4), and the size of the active memtable. Based on the pressure these indicate, it adds a small delay to each commit, applied once per column family. At moderate pressure — half the stall threshold, or three times the L1 trigger — it inserts a 0.5ms sleep, barely noticeable but enough to hand the flushers some CPU. At high pressure — 80% of the stall threshold, or four times the L1 trigger — it inserts 2ms; the jump is deliberately non-linear, taking a sharper bite out of throughput so flush and compaction can catch up. At the stall threshold and above, writes block entirely until the queue drains.

The stall is *progress-based*, not a fixed timeout, and this distinction is the heart of the design. A blocked writer keeps waiting as long as the flush engine is making headway, which the engine detects two ways: the L0 queue shrinking, or a global flush heartbeat advancing (flush workers bump the heartbeat as they iterate, and the dispatcher bumps it as it picks up work). So a flush that is merely slow — a big memtable on slow disk, or many column families contending for the pool — paces the writer rather than failing it. Only after a sustained run of polls in which *neither* signal moves does the engine conclude the flush path is genuinely wedged and return `TDB_ERR_BUSY`, a transient “overloaded, retry” error kept distinct from `TDB_ERR_IO` for a real I/O failure. A true wedge of this kind usually means a full disk, a failing disk, or a deadlock; because the error is transient, a caller can simply retry. The engine polls every 10ms, trading a little responsiveness for low syscall overhead.

The L0 stall bounds the queue of frozen memtables, but not the active memtable that writers are still filling. With many writers, if rotation is deferred because the concurrency limit on in-flight flushes is saturated, the active memtable can balloon past its buffer size before anything stops it. A hard ceiling closes that gap: at twice the write-buffer size, a committing writer stalls with the same progress-based logic as the L0 stall, kicking rotation on the way in so a deferred rotate runs the moment a slot frees. The same branch handles unified mode through the same rotation path, rather than introducing a second rotator.

Level 1 is watched alongside L0 because a high L1 count means compaction is falling behind, and a compaction backlog eventually starves flushing too (flushers wait on compaction to free space). Throttling on L1 therefore acts as a leading indicator, applying pressure before L0 becomes critical and heading off a cascade.

The per-column-family signals above cannot, by themselves, prevent an out-of-memory condition when many column families fill up at once. So a separate global guard runs in the reaper thread every 100ms. It sums all the memory the database is using — active and immutable memtables, in-flight transaction buffers, compaction scratch space, bloom filters, block indexes, and caches — and divides by a resolved limit (`max_memory_usage`, default 75% of system RAM, never less than 5%). The resulting pressure level is graduated by fraction of that limit: normal below 60%, elevated from 60%, high from 75%, and critical from 95%. The write path reads this level with one atomic load per commit, so it costs nothing at normal pressure. As pressure climbs, the response escalates: at elevated, the flush threshold tightens and the current family is flushed proactively; at high, the current family is force-flushed, the reaper force-flushes the largest non-flushing family, and it aggressively compacts the family with the most SSTables; at critical, writes block entirely until the reaper brings pressure down (timing out after 10 seconds with `TDB_ERR_BUSY`), while the reaper force-flushes every non-flushing family. In unified mode, where one memtable is shared, the reaper rotates that single memtable instead of iterating empty per-CF ones. As a last line of defense, an OS-level check polls real free memory every few seconds and forces the level to critical if free RAM drops below 5%, catching consumption that TidesDB’s own accounting cannot see.

The point of the whole scheme is smooth degradation. Increasing the write-buffer size trades flush frequency against memory used during stalls; raising the stall threshold trades memory for burst tolerance; adding flush workers drains the queue faster; and `max_memory_usage` caps the whole envelope. The right settings depend on the write pattern, the available memory, and the disk — but in every case the system slows down gradually as it approaches its limits, rather than swinging between full speed and a dead stop.

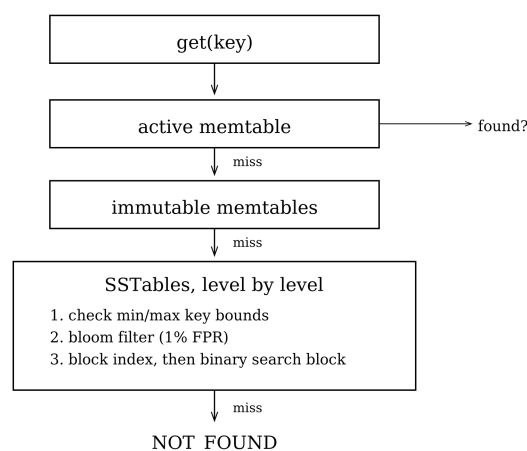
The Read Path

A read has to find the newest version of a key that is visible to its snapshot, and that key could be anywhere in the lifecycle — in the active memtable, in a frozen one, or in an SSTable at any level. The read path is the discipline that searches those places in the right order and makes each step cheap.

Search Order

The search runs from newest data to oldest (Figure 5): the active memtable first, then the immutable memtables newest to oldest, then SSTables in level 1, then level 2, and so on. It stops at the first match. Because newer data always sits earlier in this order, the first match is the newest version, which is exactly what a reader wants.

Fig. 5. The read path for a point lookup.



Looking Inside an SSTable

For each SSTable the search considers, the work is layered cheapest-first. It first checks the key against the table's minimum and maximum bounds. If a bloom filter is present, it checks that next — a negative answer means the key is definitely absent, and the entire table is skipped with no I/O. If a block index is present, it identifies which block could hold the key, and the search opens a cursor there rather than at the start.

For the candidate block, the engine consults the block cache. On a hit it pins the cached block and reads directly from it, with no copy and no decompression, because the cache holds blocks already decompressed. On a miss it reads the block from disk, decompresses it, caches it, and then reads. Either way it binary-searches the block for the key, and if the matching entry points into the value log, it fetches the value from there.

One subtlety makes the block search more than a single comparison. A key can have more than one retained version in the same block, because the reader-retention floor from the previous section may be keeping an old version alive past a newer write. So the search cannot stop at the first match: it finds the key, then scans the short run of entries that share it, and returns the one with the highest sequence number at or below a ceiling supplied by the caller. A snapshot reader passes its snapshot sequence and gets the version it is entitled to; conflict detection passes the maximum value and gets the newest committed version regardless of what readers are pinning. This is what gives a precise answer for a key that holds, say, a put at sequence 5 and a delete at sequence 9 in the same block.

The bloom filter and block index are optional, per-column-family optimizations, and they are worth understanding together because a bloom *false positive* is what makes them matter. A false positive costs a bloom check, a block-index lookup that is probably a cache miss and therefore a disk read, a block read and decompress that is probably a second miss and a second disk read, and a binary search — roughly two disk reads spent looking for a key that does not exist. At the default 1% false-positive rate and a high query volume, that adds up, which is why the rate is tunable.

The block cache evicts with a CLOCK policy. Each entry carries a reference bit; the clock hand evicts an entry whose bit is clear and clears the bit of an entry it finds set, giving that entry a second chance. Readers set the bit on access, which both records recent use and protects an entry from eviction while it is in use, and multiple readers share a cached block without copying it.

The Block Index

The block index turns a linear scan of an SSTable's blocks into a binary search. It stores, for each indexed block, the first and last key of that block (as fixed-length prefixes) and the block's position in the file. The sampling ratio is configurable and defaults to every block: indexing every block costs the most memory and gives the most precision, while indexing every tenth block saves memory at the cost of occasionally scanning a few blocks per lookup.

The index is *lossy*, because it keeps only a prefix of each boundary key, and the lookup is built around that lossiness. Two keys with the same prefix but different suffixes can land in different blocks the index cannot tell apart, so a naive “rightmost block whose start is at or below the search key” would sometimes overshoot the block that actually holds the key. The lookup is therefore done in two steps. The first binary-searches for the *leftmost* block whose end-prefix is at or above the search prefix — the first block that could hold the key or one sorting after it, so the search never begins past the target. The second counts how many consecutive blocks from there share a start-prefix at or below the search prefix; that run of prefix-colliding blocks must be scanned in full before declaring the key absent,

since the index cannot say which of them holds it. For unique prefixes the run is one block and the lookup reads exactly one. For shared prefixes it is short, rarely more than two or three, and still far cheaper than scanning every block. When the index covers every block, the scan is also conclusive: if the key is not in the colliding run, it is not in the table, and the search stops early.

That early stop is valuable on its own. Once the index points at a block, a key absent from that block cannot exist in a later one, because the blocks are sorted — so a negative lookup ends immediately instead of scanning to the end of the table. The same index drives iterator seeks: a seek finds the candidate block and colliding run, jumps the cursor there, and scans forward (or backward, for a reverse seek). Without it, seeking into the middle of a large table would mean scanning every block from the beginning; with it, a seek is logarithmic in the index plus a short linear scan.

One invariant matters when compression is on. The cursor advances by the *on-disk* (compressed) size of the block it just read, so the cached size must always be the compressed size, never the decompressed one. On a cache miss the iterator records the on-disk size; on a cache hit, where the cached bytes are already decompressed, it instead invalidates the cached size so the next advance re-reads it from the size header on disk. Writing the decompressed size back would advance the cursor into the middle of the next block — precisely the failure the block manager’s size guard exists to catch.

Iterator Fast Paths

Iteration and repeated seeks are common enough that several specific costs on their hot path have been engineered away. Each optimization targets one.

Block reuse handles seeks that stay nearby. When an SSTable source already holds a deserialized block and the seek target falls within that block’s key range, the seek searches the block in place, skipping the release, cache lookup, and deserialization cycle entirely. It handles targets before the current block (return its first entry) and after it (advance to the next block without re-running the index search). For workloads with high seek locality this drops deserialization from dominating the CPU profile to a small fraction of it, and the sequential-advance case is what makes ordinary forward iteration fast, since the next key is usually in the next block.

Boundary prefetch hides cross-block I/O. When the iterator reaches the last entry of a block, it issues an OS readahead hint on the next block, so the kernel begins loading it before the iterator asks.

Cache-aware advance closes a gap where seeks consulted the block cache but sequential advance did not. Now the advance checks the cache first; a hit pins the data with no I/O, a miss reads from disk and populates the cache. Because point lookups and iterators share the cache, one thread’s reads warm blocks that another thread’s scan then gets for free.

Incremental indexed advance avoids re-deserializing a whole block to read one entry. When a cached block carries a pre-built offset index, each step parses only the single next entry directly from the raw bytes, with key and value pointers referencing the pinned buffer with no copy. Only non-indexed blocks fall back to full deserialization.

Cached sources and zero-copy merge eliminate per-seek allocation. The iterator caches its memtable sources at creation rather than rebuilding them on every seek, pinning the active memtable with a reference so a concurrent rotation cannot free it mid-setup, and snapshotting the immutables through the same lock-free mechanism. SSTable sources expose their current entry as a borrowed pointer into pinned block data instead of allocating a fresh pair each step. Together these make the hot seek path perform zero allocations. The borrowed pointers stay valid because the iterator pins its sources for its whole lifetime; they are only copied into a stable buffer when the caller actually keeps the entry, so entries discarded while skipping over tombstones cost nothing.

Compaction

Compaction is the process that keeps the LSM tree healthy. Flushing alone would pile up an ever-growing number of SSTables at level 1, and read amplification would climb without bound. Compaction merges SSTables together, discards data that has been superseded or deleted, and moves data down into the level hierarchy so that each level stays within its size budget. It is, in effect, the engine paying down the write- and read-amplification debt it took on by writing fast in the first place.

How Compaction Decides What to Do

TidesDB does not expose the usual menu of named policies (leveled, tiered, and so on). Instead it runs three merge strategies and chooses among them automatically from the current shape of the tree, following the principles of the “Spooky” compaction algorithm and working alongside a second mechanism, Dynamic Capacity Adaptation, described below. A single controller drives the whole process.

Compaction fires under any of three conditions. The first is file count: when level 1 accumulates a threshold number of SSTables (default 4), flushed memtables are piling up and must be merged downward. The second is level capacity: when level 1’s total size exceeds its budget, data must move down to preserve the rule that each level is roughly N times the one above it. The fire condition watches level 1 specifically; once a compaction runs, the dividing-level logic below decides how far down the tree to rebalance, so a deeper level over its budget is resolved by the merge that gets scheduled rather than by firing a trigger of its own. The third is tombstone density, which deserves its own explanation.

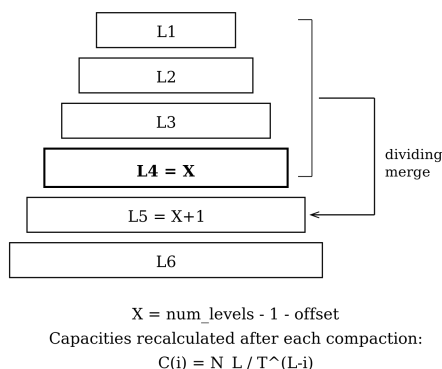
A deletion in an LSM tree is not an erasure; it is a *tombstone*, a marker that shadows older copies of the key in deeper levels. A tombstone can only be physically dropped once it reaches the largest level, where there is nothing beneath it left to shadow. Until then, every range scan over a deleted region

pays to skip past the accumulated tombstones. A delete-heavy column family that rarely triggers a structural compaction can therefore build up enough tombstones to badly hurt scan latency. The density trigger watches for this. Each SSTable records its tombstone count next to its entry count, persisted behind a flag bit so older binaries stay compatible. After each flush, a witness pass asks whether any single SSTable's ratio of tombstones to entries exceeds a configured trigger while holding at least a minimum number of entries (so a tiny all-tombstone table does not fire it). One dense table is enough; on a hit, the engine steers a compaction toward the largest level, where the tombstones can finally be reclaimed. The trigger is off by default (ratio 0.0) and arms when set above zero, and it runs independently of the structural triggers, so a workload that never trips file-count or capacity can still get tombstone relief. The same witness is wired into both per-CF and unified flushes, and the engine exposes the underlying counts (total tombstones, per-level breakdown, worst single table) so operators can see how close they are to the trigger and tune it.

The Dividing Level

When compaction does run, it first computes a dividing level, written X , which is its primary target (Figure 6). X is a real level in the tree, not an abstract reference point:

Fig. 6. Compaction and the dividing level.



```
X = num_levels - 1 - dividing_level_offset
```

The offset (default 1) tunes aggressiveness: a smaller offset compacts more often into deeper levels, a larger one defers the work. With 7 active levels and the default offset, X is 5, so level 5 is the primary merge destination. Given X and the level that triggered the compaction, the engine picks one of three strategies. If the triggering level is X , it runs a dividing merge, consolidating levels 1 through X back into X — the common case under steady writes. If some level shallower than X can absorb the combined data of levels 1 through itself, it runs a full preemptive merge into that level, handling intermediate levels that fill faster than expected. And if X is still over capacity after the initial merge, it runs a partitioned merge as a secondary cleanup.

The Three Merge Modes

Each mode suits a different point in the tree's life.

The full preemptive merge is the simplest: it combines all SSTables across a range of adjacent levels into the target level. It opens every source, builds a min-heap of them, and repeatedly emits the smallest surviving key, keeping only the newest version of each, dropping expired entries, and dropping tombstones only when the target is the largest level and the merge reaches it from the top. It is effective for small merges but, because it does not partition the key space, it can produce large output files. It also serves as the fallback for the other two modes when they cannot work out partition boundaries.

The dividing merge is the workhorse of steady-state writes, taken when the target equals X . It consolidates levels 1 through X and writes the result back into X . If X is already the largest level, the engine first adds a new level beneath it so the tree always has room to grow downward. Crucially, it partitions intelligently: it reads the key ranges of the SSTables one level below and uses them as boundaries, merging in chunks so that each output SSTable covers a single range and is pre-aligned with the level it will later be pushed into. This keeps output files small and evenly spread across X instead of producing one monolith. If no boundaries can be found — the level below is empty — it falls back to a full preemptive merge.

The partitioned merge is a focused secondary cleanup, used when X is still over capacity after the initial merge. Rather than merging from level 1, it works on a narrow range starting at X , again using the largest level's key ranges as partition boundaries and producing small single-range outputs. It relieves pressure in one part of the tree without triggering a tree-wide compaction, which keeps compaction from falling behind during bursty writes. When it targets the largest level, its output files are capped at the dividing level's capacity (per Algorithm 2 of the Spooky paper), splitting at block boundaries if a partition would exceed that. This bounds transient space amplification regardless of how skewed the partition's keys are.

Running Merges in Parallel

A compaction *round* is serialized per column family, to preserve level invariants and manifest ordering, but the work *inside* a round is not single-threaded. All three modes split their output into independent key-range partitions and run those partitions concurrently. An executor is handed N partitions and borrows up to $N-1$ helper threads from a database-wide budget, with the calling worker pitching in as well. Because the budget is shared across all column families, several rounds on different families can never oversubscribe the pool. Partitions are claimed through an atomic work-stealing cursor — each worker grabs the next one and repeats until they run out — and because the calling thread always works, progress is guaranteed even if the budget is zero or a thread fails to spawn; unclaimed partitions are simply picked up by whoever is free. Each partition owns its merge heap and output and writes independently; only the commit step (adding the output to its level and committing the manifest) is serialized, on a per-family lock. The three modes differ only in how they

derive the partition boundaries — sampled from input keys for the full preemptive merge, taken from the level below for the other two — and when no boundaries exist, the merge runs as a single partition on the calling thread, exactly the old serial behavior.

Inside a Merge

All three modes share one execution path. It opens every source SSTable across the level range, builds a min-heap ordered by current key under the column family's comparator, and repeatedly pops the smallest source, advances its cursor, and sifts it back in under its new key.

Filtering happens as entries are popped. When several sources hold the same key, only the highest sequence survives — unless an older version sits at or above the reader-retention floor, in which case it is kept because a live snapshot may still need it. Expired entries are dropped. A tombstone is dropped only when the merge both writes into the largest level and started at the top of the tree, so that nothing beneath it remains to shadow and no shallower level is left holding an older put that could resurface — a partial merge into the largest level from a middle level keeps the tombstone. Even then the drop waits until the tombstone's own sequence has fallen to or below the reader-retention floor and below the oldest write still unflushed in a memtable, so neither a long-running reader nor a not-yet-flushed older version can watch the deleted key reappear. Merging into a shallower level keeps the tombstone so it goes on shadowing deeper versions. Survivors are written into 64KB blocks with large values going to the value log, blocks optionally compressed, followed by the block index, bloom filter, and metadata, and both files are fsynced.

The commit order protects concurrent readers, mirroring the flush rule. The merged output is added to its target level *before* any input is removed, so a reader always sees at least one copy of every key. The manifest commit is atomic — temp file, fsync, rename. The inputs are then removed from the in-memory levels in a single atomic swap per level, processed deepest level first; because a point read stops at the first level holding the key, removing deepest-first guarantees a reader sees either all of a level's merged inputs or none, never an old put stranded without the tombstone that shadowed it. The actual file deletions can be deferred to the reaper so they never block the worker.

Skipping tombstones efficiently is its own small art. The merge heap only copies a popped entry into the iterator's buffer when the caller actually keeps it, so tombstones discarded during a skip cost nothing. The skip loop — shared between forward and backward iteration — copies the tombstone's key to a stable stack buffer and then advances every other source whose current entry matches, each at the cost of one cursor step and no copies. The key is copied to the stack first precisely so that later pops inside the loop cannot overwrite the pointer the comparator still depends on.

If a source hits corruption while advancing, the heap catches it through a checksum failure, hands the bad SSTable back to be deleted, drops it from the heap, and finishes the merge with the rest — so one damaged table cannot stop compaction. And large values flow *through* the merge rather than being copied verbatim: each is read from the source value log, recompressed under the current configuration, and written to the destination, which lets compression settings change over time without a full rebuild.

Single-Delete: Dropping Tombstones Early

The tombstone rule above is conservative for a reason, but the conservatism has a cost worth optimizing away in a specific case. A regular delete must be carried through every compaction until the largest level, because some deeper level might still hold an older put it needs to shadow. Workloads that put a key once and delete it once therefore pay a read tax until a bottom-level compaction finally reaps the tombstone.

`tidesdb_txn_single_delete` lets a caller opt out of that conservatism, under a contract: between any two single-deletes of a key (and before the first), the key was put at most once. Under that promise the engine may drop a put and its matching single-delete together the first time a merge sees both, at any level. Reads treat a single-delete like any other tombstone; the difference lives entirely in the merge. The subtype is a second flag bit carried beside the tombstone bit, already part of the persisted byte, so the format is unchanged and older binaries simply see a tombstone. The bit rides the whole write path — WAL, skip list, flush, merge sources — into compaction.

The cancellation happens during the merge's emit phase. Because the heap delivers a key's versions newest-first, the first entry popped for a key is the newest survivor; the emit loop holds it pending and resolves it when the next distinct key arrives. If that pending entry is a single-delete and the next older version of the same key is a live put, the pair is cancelled: the single-delete is dropped outright and the put with it. A pending regular tombstone that found no pair follows the ordinary rules. The same lookahead runs at every emit site, with the partitioned merge using a slightly narrower peek variant to fit around its mid-loop file splitting; the effect is identical for the dominant case where put and single-delete arrive adjacent. Violating the contract — putting a key more than once between single-deletes — cannot be detected by the engine; the result is that only the most recent put is masked while older puts stay visible, so a caller who cannot guarantee the contract must use a regular delete.

Compacting a Range on Demand

The density trigger reclaims space automatically, but operators sometimes need to reclaim a known key range *now* — after a large range delete, on tenant eviction, or for a sliding-window expiry that does not fit TTL. `tidesdb_compact_range` does this synchronously: it snapshots each level, selects only the SSTables overlapping the range, and merges them on the calling thread, leaving everything outside the range untouched. It runs inline rather than queuing, so if a background compaction is already in flight on the family it returns `TDB_ERR_LOCKED` at once rather than waiting behind it. The merge uses the same emit logic as the automatic ones — tombstone reclamation, single-delete cancellation, deduplication, recompression — so the output matches what a background compaction would have produced. It shares the heap, block writer, and manifest machinery with the other modes and blocks until it commits or fails. The witness-driven path from the density trigger is the asynchronous sibling of this call: it enqueues a steer-to-bottom work item instead of running inline, but applies the same rules. Together they capture most of the value of a true range-tombstone primitive without its cost: a range delete still writes per-key tombstones, but the operator can reclaim the affected tables in one pass immediately afterward. The plain `tidesdb_compact` call similarly blocks

until its work item has been serviced, but enqueues onto the global queue rather than running inline, so calls from different threads can run in parallel across column families; it is never coalesced against an in-flight compaction, so each call captures everything committed up to the moment it was made.

Dynamic Capacity Adaptation

The three merge modes decide *how* to merge. Dynamic Capacity Adaptation (DCA) is a separate mechanism that decides *when to add or remove levels* and continuously retunes each level's capacity to match the data that actually exists. It does not run continuously; it fires after events that change the tree's shape — at the end of a compaction cycle, and after a level is removed.

Its core is a recalculation of every level's capacity from the measured size of the largest level:

$$C_i = N_L / T^{(L-i)}$$

Here C_i is the new capacity for level i , N_L is the actual byte size of the largest level L (the ground truth of how much data exists), T is the level-size ratio (default 10), and L is the number of active levels. Anchoring capacities to the real data at the bottom of the tree is what keeps compaction timing optimal as the database grows or shrinks: it avoids both over-provisioned capacities, which raise read amplification, and under-provisioned ones, which cause excessive compaction and write amplification.

DCA *adds* a level when a dividing merge needs to write below the current largest level, or when a level overflows and needs a destination that does not yet exist. It creates the empty level, increments the active count, and lets ordinary compaction move data into it — no data is migrated during the addition itself, which avoids a whole class of key-loss bugs. It *removes* a level only under strict conditions: the largest level is completely empty after a compaction, the active count is above the configured minimum, the level was not just added this cycle, and no flushes are pending. The minimum-level check prevents thrashing — adding and removing the same level repeatedly. On startup, a column family begins with at least its configured minimum number of levels, or more if recovery finds SSTables deep in the tree, and each level's capacity is set directly from the configured base size and ratio. DCA itself does not run during open; it takes over once the tree's shape next changes — at the end of a compaction cycle, or when a level is removed — re-deriving the capacities from the data that actually exists.

Summary

Compaction is three merge modes, chosen automatically, working with DCA to scale the tree up and down with the data. A dividing merge is the steady-state workhorse, taken when the target equals the dividing level X ; a full preemptive merge handles a shallower level that fills early; a partitioned merge is the secondary cleanup when X stays full. The dividing level is a one-line formula rather than the output of any complex analysis. Tombstones are reclaimed conservatively by default and eagerly under

the single-delete contract, automatically by the density witness and on demand through range compaction. The result handles everything from steady writes to sudden bursts while protecting both read and write performance through deliberate placement and scheduling.

Recovery

Recovery is the inverse of the write path: it reconstructs the in-memory state from what survived on disk, so the database resumes exactly the logical state it had before a crash. On startup the engine reads the database-root state, scans each column family's directory, and rebuilds its structures in a careful order.

The database-root state loads first. In unified mode this means reading `UNIMAP` before any column family opens, so each family re-registers under the index it held before the crash and replayed WAL entries land in the right group — the ordering is load-bearing, since getting it wrong would replay the log under the wrong families. Each column family's manifest then loads. On a normal local restart the engine recovers level membership by scanning the directory (the level is encoded in each SSTable's filename) and uses the manifest as a completeness check, confirming each SSTable finished its commit handshake; the manifest drives membership directly only on object-store cold start, where no local files exist.

WAL replay happens in place, and that detail is what makes recovery deterministic. The engine adopts the highest-numbered WAL segment and opens it where it sits rather than discarding it, validates it permissively (trimming only a preallocation tail or torn final block), and replays every committed entry directly into the column family's active memtable. The memtable a fresh open hands back to the application is the very one the WAL replayed into; ordinary flush pressure promotes it to an SSTable later. Older, lower-numbered segments left by a crash are replayed into separate memtables and flushed asynchronously. The unified WAL recovers the same way — adopt the highest generation, validate, replay into the shared memtable — with each entry's prefix resolving correctly because `UNIMAP` already loaded.

The sequence counter is recovered to match. The engine scans the maximum sequence across the recovered SSTables, the immutable memtables, and the active memtable the WAL just filled, and advances the global counter past it. This is why a transaction that reached the WAL but never an SSTable still counts: its sequence is seen during the scan and stays visible to readers in the new run.

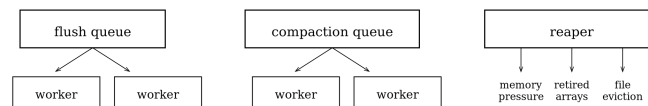
SSTables are validated strictly, the mirror of the WAL's permissiveness. A log is transient and append-only and must tolerate torn tails because recovery routinely meets them; an SSTable is permanent, so the manifest is the authority on which ones finished their commit handshake. An SSTable on disk that the manifest does not list is an incomplete write — a crash after the fsync but before the manifest commit — and recovery deletes it. An SSTable the manifest *does* reference but that fails to load is kept

on disk rather than condemned, since a load failure can come from a write-side bug as readily as from media corruption; it is skipped for this open and logged loudly, so a later repair stays possible instead of the data being lost outright.

Background Workers

The work that does not happen on the caller’s thread happens here (Figure 7). Flush and compaction workers are configurable thread pools; the sync worker and the reaper are single threads. Object-store mode adds an upload pool, and a read-only replica adds a sync thread of its own.

Fig. 7. Background worker pools.



Multiple workers may process different column families in parallel.
Within a column family: one flush and one compaction at a time.

Flush workers (default auto, min of CPU count and 4) take frozen memtables off the queue and write them to SSTables, in parallel across column families. Compaction workers (default 2) merge SSTables across levels, in parallel across families, and fan out within a single round through sub-compaction. The sync worker (1 thread, started only if any WAL uses interval sync) periodically fsyncs the WALs configured for it; it finds the smallest configured interval, sleeps that long, and syncs each due WAL. Column families on interval sync also force an explicit fsync at structural boundaries — when a memtable rotates, and during every sorted-run creation and merge — which preserves durability while still batching ordinary writes.

The reaper (1 thread) runs a maintenance loop every 100ms and is the system’s general groundskeeper. Each cycle it sweeps the deferred-free list, retries flushes that were deferred under the concurrency cap, redelivers object-store uploads that exhausted their fast retries (in object-store mode), services any compaction triggers that arrived while a compaction was already running, recomputes global memory pressure and acts on it, and evicts idle SSTable file handles when too many are open. The memory-pressure response was described with [Backpressure](#); the two pieces of bookkeeping unique to the reaper are worth a word each.

Retired-array reclamation solves a lifetime problem. When a flush or compaction swaps a level’s SSTable array, the old array cannot be freed at once, because a reader may still be walking it. Each level keeps a counter of active readers. Rather than spinning until it hits zero — which would stall the worker and cascade under mixed load — the worker spins briefly, and if readers remain, pushes the retired array onto a lock-free deferred-free list. The reaper sweeps that list each cycle, freeing what has gone quiet and re-queuing the rest, and a final drain at shutdown clears whatever is left.

File-handle eviction bounds open descriptors. Each SSTable uses two (key log and value log). When the open count reaches its budget — the configured maximum less a reserve held back for the write path — the reaper closes the least recently used quarter of the idle, unreferenced tables, reclaiming down to that budget rather than to the maximum, so the reserve stays available as burst headroom. The default maximum is 256 SSTables (512 descriptors), lower on platforms with tighter limits. If the working set is persistently larger than the budget, the reaper evicts continuously, and descriptors thrash.

How Work Is Distributed

The database keeps two global queues, one for flushes and one for compactions, each work item naming its column family. A memtable crossing its threshold enqueues a flush; a level over capacity enqueues a compaction. Workers block until work arrives, and different workers serve different families at once.

Two atomic flags coordinate per family, and a subtlety in how they are used is worth stating because it surfaces in the public API. The `is_flushing` flag is now only a brief gate over the memtable swap and the enqueue — it no longer means “a flush is in progress.” Concurrency on the actual SSTable writes is bounded instead by a database-wide counter capped at the flush-thread count, behaving like a semaphore; a hot family can thus have several memtables flushing at once and drain its queue at pool speed, while the global cap also bounds total transient memory across many families. Compaction rounds stay one-per-family via `is_compacting` to preserve invariants, with a trigger that arrives mid-round setting an “armed” flag the worker honors afterward rather than dropping. Because `is_flushing` is just a gate, the real “is anything being written” signal is a per-family pending-flush counter, and the completion wait polls *that*, so a caller asking for its data to reach disk does not get a false all-clear the instant the rotate gate clears. The public `tidesdb_is_flushing` and `tidesdb_is_compacting` check both the flag and the counter and report true if either shows pending work; operators reading these externally should read `is_flushing` as “a rotation is in flight” and the counter as “queued or running flushes exist.” For forced maintenance, `tidesdb_purge_cf` synchronously flushes and aggressively compacts one family (rotating the shared memtable first in unified mode), and `tidesdb_purge` does the same across all families and drains both global queues.

The parallelism that results is straightforward to reason about. Flush and compaction workers run different families in parallel; a single family can flush multiple memtables at once up to the cap, so a hot family drains at pool speed; and compaction stays serialized per family but fans out within a round. For sizing, set flush threads toward the number of concurrently hot families and compaction threads toward the number of families; with N families and M flush workers, flush latency on a quiet family is roughly (N/M) times the flush time. The main thread always enqueues and returns immediately, which is what sustains the write throughput.

Error Handling

Functions return an integer: zero for success, a negative code for a specific failure.

Code	Value	Meaning
<code>TDB_ERR_MEMORY</code>	-1	allocation failure
<code>TDB_ERR_INVALID_ARGS</code>	-2	invalid parameters
<code>TDB_ERR_NOT_FOUND</code>	-3	key not found
<code>TDB_ERR_IO</code>	-4	I/O error
<code>TDB_ERR_CORRUPTION</code>	-5	data corruption detected
<code>TDB_ERR_EXISTS</code>	-6	resource already exists
<code>TDB_ERR_CONFLICT</code>	-7	transaction conflict
<code>TDB_ERR_TOO_LARGE</code>	-8	key or value too large
<code>TDB_ERR_MEMORY_LIMIT</code>	-9	memory limit exceeded
<code>TDB_ERR_INVALID_DB</code>	-10	invalid database handle
<code>TDB_ERR_UNKNOWN</code>	-11	unknown error
<code>TDB_ERR_LOCKED</code>	-12	database locked by another process
<code>TDB_ERR_READ_ONLY</code>	-13	write attempted on a read-only replica
<code>TDB_ERR_BUSY</code>	-14	transient overload; a backpressure stall hit its no-progress budget (retry; matches POSIX <code>EBUSY</code>)
<code>TDB_ERR_PRECONDITION</code>	-15	a conditional object-store write failed its precondition (HTTP 412); raised by single-writer fencing when a superseded primary tries to publish

More codes appear in the [C reference](#).

The guiding distinction is between transient errors (out of disk, out of memory) and permanent ones (corruption, invalid arguments). Durability rests on fsync, every disk read is checksum-validated by the block manager, and magic numbers catch corruption at the SSTable level. A few scenarios are worth making explicit. A failed pre-flush disk-space check skips the flush and leaves the memtable in memory; a write that fails partway discards the partial SSTable and re-enqueues the work for retry — in both cases writes keep flowing into the active memtable until backpressure stalls them, and the failure is logged rather than surfaced per write. Corruption found on read returns `TDB_ERR_CORRUPTION` without condemning the whole SSTable, since a later read may hit an intact block; corruption found during compaction drops the bad table and continues; an allocation failure during compaction aborts it and leaves the old SSTables intact for the next attempt. One configuration error the engine cannot detect is a changed comparator between restarts: keys end up out of order, lookups miss, iterators return them wrongly, and compaction produces mis-sorted output — the logical structure is corrupted with no physical corruption at all. Bloom false positives cost two needless disk reads but raise no error.

Design Rationale

The defaults are not arbitrary; each is a trade-off struck at a particular point. This section records the reasoning, so the defaults can be moved with eyes open.

Block size trades compression efficiency against random-access granularity. Larger blocks compress better, because they give the compressor more context, but a point lookup must read a whole block to find one value; smaller blocks cut that waste but compress poorly and inflate the block index. The 64KB target matches common SSD page sizes and compresses text 2–3×.

The level-size ratio, how much larger each level is than the one above, sets write amplification: each level holds $N\times$ the previous one. A smaller ratio ($5\times$) lowers write amplification but adds levels and hurts reads; a larger ratio ($20\times$) removes levels but raises write amplification. Default $10\times$, per column family.

Write amplification follows from that ratio. In leveled compaction an entry is rewritten about once per level it passes, and the average works out to roughly $R\cdot L/2$ rather than $R\cdot L$ because shallow levels are rewritten more often than deep ones. For a 1TB database with a 64MB level 1 and ratio 10, the tree settles at about 6 levels, so write amplification is around $30\times$ — not the $60\times$ a naive estimate suggests. Updates amplify less than pure inserts.

Read amplification, in the worst case, is one SSTable read per level — 6 reads at 6 levels without bloom filters. At a 1% false-positive rate, the expected reads for an absent key fall to about 1.06, since the bloom rejects almost every level with no I/O. For a key that is present, bloom filters do not help; the block still has to be read.

The value-log threshold keeps the key log compact for scanning: values at or above 512 bytes go to the value log. A smaller threshold causes many value-log seeks; a larger one bloats the key log. The default is roughly the point where the cost of the indirection becomes cheaper than scanning a large inline value during iteration.

The bloom false-positive rate, 1% by default, balances memory against effectiveness: dropping to 0.1% needs ten times the bits per key to cut false positives tenfold, while 5% saves memory at the cost of more pointless reads. At 1% a filter is about 10 bits per key, so a million keys cost roughly 1.25MB — small enough to stay resident.

Memtable size trades flush frequency against recovery time and memory. Larger memtables flush less often but lengthen recovery and use more memory; smaller ones flush more (more SSTables, more compaction) but recover faster. The 64MB default holds about a million small pairs and flushes every few seconds under moderate load. Doubling it halves flush frequency but raises level-1-to-level-2 amplification, since each flush produces a larger table that takes longer to merge.

Worker counts default to auto flush threads (the CPU count, capped at 4) and two compaction threads, which give cross-family parallelism at modest cost. More threads help with many active families but cost memory (each buffers 64KB blocks) and descriptors (two per table in flight). The device dominates the choice: on a spinning disk, several concurrent compactors cause head seeks that destroy throughput; on NVMe, more workers help. So 1–2 workers for HDD, 4–8 for NVMe.

Operational Considerations

A TidesDB instance is safe for many threads in one process but exclusive to a single process: only one process may open a database directory at a time. Exclusivity is a non-blocking file lock taken during open — if another process holds it, open returns `TDB_ERR_LOCKED` at once rather than waiting. The locking primitive is chosen per platform for correct semantics: `fcntl` locks on macOS and BSD (which, unlike `flock`, are not inherited across `fork`, with the owning PID written to the lock file so a same-process double-open is caught), OFD locks on modern Linux, and `LockFileEx` on Windows, with retries on signal interruption so a stray signal cannot spuriously fail the lock.

Memory use per family comes from a few structures: the active memtable is configurable (default 64MB) and the immutable queue is that size times its depth (usually 1–2); the block cache is shared across families (default 64MB total); bloom filters cost about 10 bits per key and block indexes about 32 bytes per block. A family with 10M keys across 100 SSTables therefore runs around 150MB plus its share of the cache. The `max_memory_usage` cap (default auto, resolving to 75% of system RAM, never clamped below 5%) bounds the aggregate across all families, which is what prevents an out-of-memory condition in many-family deployments where per-family limits cannot.

Three operational limits interact at the margins. When writes outpace compaction, backpressure stalls them once the flush queue passes its threshold, trading occasional latency spikes for bounded memory. Because SSTables are immutable, space is reclaimed only after a compaction finishes and deletes its inputs, so a compaction can briefly need double the space of the level it rewrites; the engine checks free space before starting one. And because each SSTable holds two descriptors open, a working set larger than the open-file budget makes the reaper thrash; an operator who wants a bigger resident set can raise the process's descriptor ceiling before opening the database, after which the engine sizes its budget to fit. The raise is opt-in and a partial failure is non-fatal.

On-Disk Format

Up to here, the formats have been described by what they accomplish. This section gives the bytes. It is placed last among the engine internals on purpose: by now every structure has been seen in use, so the layouts read as the concrete realization of ideas already understood rather than as detail to memorize up front.

The Key-Log Entry

Every entry in a key log — and, since they share the format, every entry in a per-CF write-ahead log — is laid out as:

```
flags (1 byte)
key_size (varint)
value_size (varint)
seq (varint)
ttl (8 bytes, if HAS_TTL flag set)
vlog_offset (varint, if HAS_VLOG flag set)
key (key_size bytes)
value (value_size bytes, if inline)
```

The flags byte marks a tombstone (`0x01`), the presence of a TTL (`0x02`), value-log indirection (`0x04`), delta sequence encoding (`0x08`), and the single-delete tombstone subtype (`0x10`, always set alongside the tombstone bit). The remaining high bits are in-memory ownership bookkeeping and are masked off before any write, so they never reach disk. The integers are varints, so the common case is small: a value under 128 takes one byte, and the full 64-bit range takes at most ten.

The unified WAL uses a slightly different batch framing, since one log now carries entries for many column families: a 2-byte magic (`0x55AA`) opens the batch, and each entry is prefixed with its 4-byte big-endian column family index ahead of the standard fields. Recovery recognizes a unified WAL by its filename prefix and magic, replays each entry into the shared memtable with its prefix intact, and flushes it through the normal unified path.

Auxiliary Blocks and the Chunking Edge Case

The block index, bloom filter, and metadata at the tail of a key log are each written as ordinary blocks, and a single block is framed by a 4-byte size field that caps it just under 4GB. In practice this ceiling never binds: a bloom filter is bounded by its bit count, itself a 32-bit quantity, so even a bottom-level SSTable with hundreds of millions of keys serializes its bloom to under a gigabyte, and a block index is far smaller. Both fit in one block.

The format is nonetheless built to be correct at *any* size, not merely large enough. An auxiliary blob that would overflow the single-block limit is split across consecutive blocks instead of failing the write; the metadata records each chunked blob's offset and total size behind a flag bit, and the reader reassembles the chunks before deserializing. A blob that fits in one block — every bloom filter and block index produced in practice — sets no flag and is laid out byte-for-byte as older binaries expect, so the format stays backward-compatible. Chunking thus stays dormant for real data and engages only for a blob that genuinely cannot fit one block, reachable only if the bloom's bit count were ever widened past 32 bits. One footer-writing routine is shared by the flush path and every merge writer, so chunking applies uniformly. On read, the block manager never trusts a size field blindly — it rejects any size whose block would run past the file, and refuses any size beyond a memory-safety budget — and a column family that cannot load an oversized bloom simply runs without it. That size guard is detailed under [Block Manager](#).

The Metadata Footer

The last block of a key log is its metadata footer, the table's self-description: a magic number, the entry and block counts, the data-end offset and file sizes, the minimum and maximum keys, the maximum sequence number, the compression algorithm, a flags word, and an xxHash64 checksum over everything preceding it. Versioning lives in the flags word rather than a version integer, and it is additive — a newer build sets a flag and appends a section, while a build that does not understand the flag would reject the size, and a newer build reading an older footer that lacks the flag simply treats that section as absent. Four flags are defined: one marks the B+tree format, one an eight-byte tombstone count, one a chunked-auxiliary descriptor for a bloom or index blob too large to fit a single block, and one an eight-byte distinct-key count. The reader computes the exact expected size from the flags and refuses any footer whose size does not match, then verifies the checksum before trusting a byte, so a truncated or reordered footer never deserializes.

The tombstone count and the distinct-key count are summary statistics tallied for free during the sorted-run write and read back without a scan. The tombstone count feeds the density trigger from [Compaction](#). The distinct-key count records how many unique keys the table holds rather than how many versioned entries — the two diverge whenever a snapshot kept older versions alive through a flush or merge — and it is counted with a single comparison against the previous key as the already-sorted run is written, so it needs no hashing and no sketch. `tidesdb_cf_estimate_cardinality` sums it across a column family's tables, plus the in-memory entries, to estimate the family's key cardinality without touching the data; because a key living in several tables is counted once per table,

the estimate is an upper bound, which suits selectivity and row-count planning. A table written before either count existed reports it as unknown and the reader falls back to the entry count, so the format stays compatible in both directions.

The B+tree Format

A column family may store its key log as a B+tree rather than the default block layout, enabled with `use_btree=1`. All key-value entries live in the leaves; internal nodes hold only separator keys and child pointers. Leaves are doubly linked through previous- and next-offset pointers, giving $O(1)$ traversal either way. The tree is immutable once built: it is bulk-loaded from sorted memtable data at flush and never changed.

Construction is bottom-up. The builder fills a leaf to the target node size (64KB by default), serializes it, and writes it out; once all leaves exist it builds internal levels from the first key of each child; a final backpatching pass fills in the leaf links. For compressed nodes the links live in an uncompressed header, so backpatching rewrites them without decompressing the node. A point lookup descends root to leaf, binary-searching at each internal node and again within the leaf — $O(\log N)$ in nodes, against the potential multi-block scan of the block format. Range scans use a cursor that walks a leaf's entries and follows the link to the next (or previous) leaf.

As in a key-log block, a leaf can hold several retained versions of one key, so the leaf scan cannot stop at the first match: the lookup lower-bounds to the key, scans the contiguous run that shares it, and returns the highest sequence at or below the caller's ceiling. The builder cooperates by keeping a key's whole version run inside one leaf, so the scan never crosses a boundary, and the ceiling is threaded from the caller's snapshot exactly as in the block format — a put-then-delete returns the delete to a fresh reader and the put to a snapshot that predates the delete, with no race between leaves.

The B+tree wins on point lookups and on seeking range scans, and most on workloads with many small-to-medium SSTables, where each table's hot nodes stay cached independently; the block format remains better for sequential full-table scans and write-heavy workloads, where the B+tree's flush-time overhead does not pay off. When a block cache is configured the B+tree gets a dedicated clock cache of fully deserialized nodes, keyed by a hash of the key-log path plus the node offset, with each node's memory drawn from a single arena so eviction frees it in one step and an SSTable's nodes invalidated by prefix when it closes. Large values still go to the value log with the leaf holding only the offset; nodes compress independently; bloom filters are checked before any traversal; and five extra metadata fields (root offset, first and last leaf offsets, node count, height) are persisted and restored on reopen.

Several techniques shrink each serialized node. *Varint encoding* of metadata fields (counts, sizes, offsets) typically saves 50–70% over fixed-width integers. *Prefix compression* stores each key as a shared-prefix length plus its suffix, reconstructing it from its predecessor, which cuts sorted keys with common prefixes by 60–80%. A *key-offset table* at the head of each leaf gives $O(1)$ access to any key

during binary search without walking the variable-length keys. *Delta sequence encoding* stores each entry's sequence as a small signed delta from the node's minimum, and TTLs use zigzag encoding. *Internal child offsets* are likewise stored as small signed deltas from a base. The two node layouts are:

Leaf node format

```
[type:1][num_entries:varint][prev_offset:8][next_offset:8]
[key_offsets_table: num_entries × 2 bytes]
[base_seq:varint]
[entries: prefix_len:varint, suffix_len:varint, value_size:varint,
        vlog_offset:varint, seq_delta:signed_varint, ttl:signed_varint,
 flags:1]
[keys: prefix-compressed suffixes]
[values: inline values only]
```

Internal node format

```
[type:1][num_keys:varint][base_offset:8]
[child_offset_deltas: signed_varint × (num_keys + 1)]
[key_sizes: varint × num_keys]
[separator_keys: raw key bytes]
```

Internal Components

The engine is assembled from reusable, separately tested modules, each solving one problem behind a clean interface. Having seen how they are used, we can now describe what they are.

Block Manager

The block manager is the lock-free, append-only file abstraction beneath all persistent storage — WALs, key logs, and value logs alike. Each file opens with an 8-byte header (3-byte magic “TDB”, a version, padding). Each block is a header (4-byte size, 4-byte xxHash32 checksum), the data, and a footer (the size again, plus a 4-byte magic “BTDB” for fast backward validation).

Writers use positioned `pread/pwrite`, so reads and writes proceed without locks, and a block write combines its header, data, and footer into one scatter-gather syscall, cutting three syscalls to one and lifting write throughput 2–2.5×. These calls are abstracted through `compat.h`; file size is tracked atomically in memory to avoid a syscall; and blocks are reference-counted, freed at zero.

A second, subtler contention point lives inside the kernel: filesystems take a per-inode write lock on any operation that advances the file's end (ext4's `i_rwsem`, APFS's vnode lock, NTFS's extension lock), which serializes many threads appending to one file no matter how cleanly userland hands out offsets. To keep the kernel out of the way, the block manager preallocates the file in 64MB chunks ahead of writes, through a cross-platform helper, so ordinary writes land inside the already-extended end and take only the cheaper read path on the inode lock. The extension is itself lock-free — a compare-and-swap claims the right to extend, and a racing claimant at worst issues a redundant, idempotent preallocation — and a clean close truncates the file back to its real data, so the trailing zeros exist only while it is open. After a crash, validation tells a legitimate preallocation tail (an all-zero suffix) from real corruption (non-zero garbage past the data) by combining a forward scan with a trailing-zero check. This buys 1.6–2× higher multi-writer throughput on the small-block WAL workload, at the cost of one preallocation chunk of space per open file (zero at rest).

A read is two `preads` — one for the 8-byte header, one for the payload into a reusable per-thread buffer — and the checksum is verified in that buffer *before* any final allocation, so a block that fails verification never costs a heap allocation. A fused read-and-advance avoids a redundant header read, and cursors cache block sizes to skip the size lookup when valid. That cached size has a strict contract: it is always the on-disk (compressed) size, because the cursor advances by header plus that size plus footer; a caller working with decompressed bytes must either set it from the on-disk block or invalidate it, never write the decompressed size back, on pain of advancing into the middle of the next block.

The read path never trusts the 4-byte size field blindly. Before allocating, it computes the block's full extent and rejects any size whose block would run past the file's data — the signature of a stale offset, a torn write, or corruption — returning a structured diagnostic rather than allocating from garbage. For unusually large blocks it additionally refuses any size beyond a memory-safety budget derived from the database memory limit; the check is a single relaxed atomic load, so small blocks pay nothing and the hot path makes no syscall. This turns a cursor landing at a bad offset from a crash (or, on a 32-bit host, an out-of-memory) into a graceful read failure. (An earlier fixed 256MB ceiling wrongly rejected the legitimately large bloom blocks of very large bottom-level SSTables, silently disabling their filters on reopen; the extent- and budget-relative checks accept any block that fits both the file and the budget while still catching absurd sizes.)

The layered corruption defenses are the reason the engine can trust what it reads. A per-block xxHash32 checksum is recomputed on every read and fails the read on mismatch. The size is stored twice, so a single-bit flip in one copy is caught by cross-validation. The footer magic is a high-entropy sentinel whose absence reliably marks a torn write. Permissive validation uses these to walk forward through a WAL and truncate at the first inconsistency, distinguishing a partial write (no footer magic) from genuine corruption (footer present, checksum fails). SSTables inherit all of this directly, and WAL entries add bounds-checked deserialization on top, so even a corrupt entry that somehow passed the block checksum fails to load rather than poisoning the memtable.

Bloom Filter

The bloom filter is a packed bitset queried by several hash functions, sized from the target false-positive rate and expected key count by the standard formulas. It serializes with varint headers and *sparse* encoding — storing only the non-zero words with their indices — which saves 70–90% at the low fill rates typical in practice. Hashing uses Kirsch-Mitzenmacher double hashing: two base hashes are computed and finalized with a murmur3 avalanche step, and the i -th bit position is derived as $h1 + i \cdot h2$, giving many independent positions from two computations, each mapped to a bit with Lemire’s fast range reduction. The hash is versioned: a filter records which hash built it and is always queried with that same one, so tables written before the avalanche step was added keep using the original hash and never produce a false negative, while new tables get the lower false-positive rate — and the improvement spreads naturally as compaction rewrites old files into new ones. One filter is built per SSTable, written after the data blocks, checked before the block index on reads, and kept resident once loaded.

Clock Cache

The clock cache is a partitioned, lock-free cache pairing a hash index with CLOCK eviction. It divides into many partitions (default 4 per core), each with its own clock hand and index, so contention falls with the partition count. On multi-CCX processors it reads the L3 topology and routes a key to a partition local to the calling thread’s cache group, caching the group in thread-local storage and re-probing periodically to catch thread migration; on a single die or a non-Linux platform this degrades to a plain hash. The partition struct is laid out across cache lines to avoid false sharing — read-only fields on one line, writer atomics on another, reader counters on a third. Entries use atomic state machines, and a single field doubles as the CLOCK recently-used bit (its low bit) and an active-reader count (its upper bits). Gets are fully lock-free, with prefetching to overlap memory latency; puts claim a slot through the clock sweep from a thread-local start position. Eviction gives a recently-used entry a second chance and checks the reader count twice — before clearing pointers and again before freeing — reverting if a reader appeared in between. A zero-copy get returns a pointer into the cached bytes, protected from eviction by the reader count until released.

When a block cache is configured, the engine builds two independent clock caches: one for raw (decompressed but not deserialized) key-log block bytes, keyed by family, file, and offset; and one for fully deserialized B+tree nodes. Caching raw bytes makes block entries about 20× smaller than caching deserialized ones, which sharply raises the hit rate (87% versus under 4% in one 64MB-cache-over-80MB-dataset measurement). A block-cache hit pins the decompressed bytes and lets the caller deserialize from them; a B+tree hit returns the node directly with no I/O; and closing an SSTable invalidates its nodes by prefix.

Skip List

The skip list is the lock-free, multi-versioned ordered map behind every memtable. Each key holds a chain of versions, newest first, each carrying a sequence number, value, TTL, and tombstone flag; probabilistic leveling gives $O(\log n)$ search. Inserts are optimistic — traverse, build the node, then compare-and-swap the forward pointers, retrying from the start on a lost race — and the hot path uses stack-allocated scratch for any practical level count. A skip list can be backed by a lock-free bump arena that hands out nodes from contiguous blocks via a non-atomic per-thread pointer bump, making individual frees no-ops and reclaiming everything in bulk when the arena is destroyed; this packs nodes sequentially for locality and is ideal for a memtable that is filled, flushed, and freed whole. New writes prepend a version by compare-and-swap; readers walk the chain for the version matching their snapshot; a tombstone is just a version with the deleted flag. Nodes store forward and backward pointers in one array, giving $O(1)$ access to the last key through the tail sentinel. A skip list can also be given a pointer to an externally cached clock value to avoid repeated time syscalls during TTL checks, and it supports pluggable comparators — which is exactly what lets different column families order keys differently.

Queue

The work queues are thread-safe FIFOs with separate head and tail locks (so enqueue and dequeue rarely contend), an atomic size for lock-free size checks, and a small free list of reusable nodes to cut allocation churn under load. A blocking dequeue lets idle workers sleep on a condition variable rather than spin, and a shutdown flag wakes them to exit cleanly. The flush and compaction pools are driven by exactly these queues.

Manifest

The manifest is the per-column-family record of which SSTables exist at which level, kept as simple text under a reader-writer lock — one line per SSTable (`level,id,num_entries,size_bytes`), behind a version header and a global sequence. It holds an in-memory array that doubles as it grows; lookups are linear, which is fine because the count per family is rarely over a thousand and manifest writes happen only at flush and compaction. A commit writes a temp file, fsyncs, and renames over the original, so the manifest is always wholly old or wholly new, never partial. During recovery it is the authority on durability: a local restart deletes any SSTable the manifest does not list as an incomplete write, and an object-store cold start rebuilds level membership directly from it.

Platform Compatibility

`compat.h` isolates every platform difference so the core never has to. Positioned I/O and `fdatasync` map to their Windows equivalents; atomics use C11 or the older `Interlocked` intrinsics; threading is pthreads (native or shimmed); directory walking, semaphores, and type and format-specifier differences are all hidden behind it; and performance hints compile to the right intrinsics where they

exist. Every source file includes it first, and it has zero runtime overhead — every macro and inline resolves to a native call. It is what lets one codebase run on Windows, macOS, Linux, the BSDs, and Solaris/Illumos unchanged.

Object Store Mode

Everything so far assumed local disk. Object store mode is an optional layer that keeps the durable copy of the data in a remote object store — S3, MinIO, GCS, or any S3-compatible service — and treats local disk as a cache in front of it. This is what enables cloud-native deployments that separate compute from storage: a node can be replaced, and a fresh one rebuilds its working set from the store. Object store mode runs on top of unified memtable mode (enabling it automatically if it is not already on) and is activated by setting a connector on the configuration.

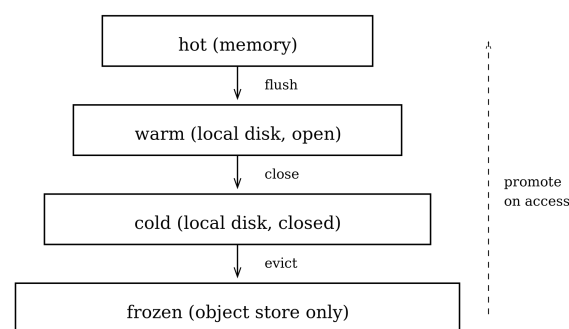
The Connector

Access to the store sits behind a small connector interface: nine operations (put, get, range-get, delete, exists, list, a conditional put and a metadata head added for single-writer fencing, and destroy) and an opaque context for credentials. Connectors must be thread-safe, and the backend is identified by an enum rather than a string so an invalid configuration fails at compile time. Two ship with the engine: a filesystem connector that writes objects as files under a directory (useful for testing and local replication), and an S3 connector that signs requests with SigV4 and works against AWS, MinIO, and compatible endpoints. Object keys are path-like strings derived from each file’s path relative to the database directory.

Two Physical Tiers, Four Access States

Files live across two *physical* tiers (Figure 8). The local tier is the database directory on disk, where all writes, flushes, and compactions produce files exactly as in non-object-store mode. The remote tier is the object store, to which files are uploaded after creation and from which they are fetched when absent locally.

Fig. 8. Storage tiers in object store mode.



Data ages downward. Accessed data promotes upward.

A key-value pair moves through these tiers in order. It starts in the shared memtable, backed by the unified WAL on local disk, which the reaper periodically syncs to the store by write volume. When the memtable freezes, a flush worker demuxes it into per-CF SSTables on local disk — identical to non-object-store mode — and then *synchronously* uploads each key log and value log, so the store has a copy before local eviction could ever delete one (the manifest is published separately, under the single-writer fence described below). The files then live on as cache entries, promoted to the head of an LRU list on every access. When a local-cache size limit is set and new files push past it, the least-recently-used files are evicted from disk — a key log and its value log always together — while the copies in the store remain. A later read of an evicted table fetches what it needs from the store; a compaction downloads its evicted inputs, writes and uploads the merged output, and deletes the inputs from both tiers.

Layered over those two physical tiers is a finer *access* hierarchy that classifies data by how fast a read of it will be. Hot data is the active memtable and the block cache — served from memory with no I/O. Warm data is SSTables whose files are local and whose block managers are open — served at page-cache or NVMe speed. Cold data is local files whose block managers the reaper has closed to bound descriptors — a read reopens them, a local operation with no network. Frozen data has been evicted from disk but remains in the store — a point lookup fetches just the one block it needs over a single HTTP range request and caches it straight into the hot tier, while an iterator prefetches whole files in parallel and lands them warm. Data flows downward through these states as it ages, and the system is self-tuning: frequently read tables stay promoted automatically through LRU touches and cache hits, while the store provides an effectively unlimited capacity floor.

How the Paths Change

The *write* path is unchanged in memory and in the WAL; only the flush adds remote work — a synchronous, retried upload of each SSTable after it is written locally, a fenced manifest publish (synchronous when fencing is active, so it can re-assert the primary lease; see below), and (when WAL replication is on) an upload of each closed WAL segment before it is deleted. An upload that exhausts its fast retries is not dropped: it is parked into a deferred backlog that the reaper re-attempts on a slow, capped backoff, so even a multi-minute store outage still delivers once it clears. Until its upload confirms, an SSTable stays untracked and therefore unevictable, so a failed upload can never let a file be reclaimed off local disk before it has landed remotely. Rotated WAL segments and manifests defer the same way; the active-WAL snapshot is the exception, since its periodic resync re-ships it anyway and parking a stale snapshot could overwrite a newer one. The S3 connector keeps its memory bounded on large files by switching from a single streaming PUT to a multipart upload above 64MB, streaming the file in fixed 8MB parts and aborting cleanly on any failure so no orphaned parts linger; the connector never holds more than one part in memory, and the multipart path lifts the single-object ceiling well past S3's single-PUT limit.

The *read* path diverges only for a frozen SSTable. A point lookup runs its bloom and block-index checks on the always-resident in-memory metadata, rejecting most tables with no I/O, and on a definitive index hit for a non-local file it fetches exactly one block (~64KB) over one range request, verifies and decompresses it, and caches it so later lookups in that block are pure memory hits; a value in the value log costs two more range requests, one for its header and one for the data. An iterator instead prefetches every non-local file it will need in parallel, bounded by a download-concurrency limit, so prefetch time is set by the largest file plus network latency rather than the sum of all files. If a range request fails or the index is not precise enough, the read falls back to downloading the whole file with retries, checking existence first so a fresh local SSTable whose object does not exist remotely yet is not chased pointlessly.

Compaction operates on local files with remote work only at the edges: it prefetches evicted inputs in parallel (by default), writes and uploads the merged output, and commits and uploads the manifest *before* deleting the old inputs, so replicas and cold-start nodes always see the new table before the old ones vanish. A lazy-compaction option doubles the level-1 trigger to trade read amplification for less remote I/O.

Cold Start, WAL Sync, and Replication

A node starting with an empty local directory but a configured store recovers by first pulling the root `UNIMAP` (so column-family indexes are right before anything opens), then listing the store, identifying families by their manifests, and downloading each family's config and manifest in parallel. It reconstructs the SSTable inventory and metadata in memory and leaves the data in the store until queries ask for it, so cold-start time tracks network latency, not data volume.

The reaper syncs the active WAL to the store by *write volume*, not wall clock: each cycle it checks how much the WAL has grown and, past a threshold (`wal_sync_threshold_bytes`, default 1MB), enqueues an upload. No syncs happen while idle; they fire often during bursts. Since the WAL is append-only, uploading a snapshot mid-write is safe, and recovery replays it normally. A WAL upload ships only the log's logical length, not the 64MB the block manager preallocates ahead of it, so even a per-commit sync is a small write rather than a full-file transfer. This bounds the crash data-loss window to the configured byte threshold rather than a whole flush cycle. Separately, when WAL replication is enabled (the default), closed segments are uploaded after flush so another node can recover uncommitted data by replaying them, with a flag choosing whether the upload blocks the flush path for stronger durability or runs in the background for lower latency.

Replicas, Promotion, and Failure

A read-only replica follows a primary through the store. It is enabled with a flag; in replica mode writes return `TDB_ERR_READONLY` immediately while reads work normally, using the same isolation levels, iterators, and caches as the primary. Before each sync the replica re-downloads `UNIMAP` so it picks up any family the primary added, dropped, or renamed — and, because it loads first, the replica's new families adopt the primary's indexes, the only way WAL prefixes can resolve to the right families.

It then polls each family's remote manifest (default every 5 seconds), diffs it against the local one, and adds or drops SSTables to match, fetching data on demand rather than during the sync. With WAL replay on (the default), it also discovers and replays new WAL segments in generation order, idempotently — entries below the current maximum are skipped, and the skip list rejects any duplicate key-sequence pair — so the replica sees writes the primary committed but has not yet flushed, even across several WAL generations. Its memtable is ephemeral, rebuilt from the primary's segments each cycle. For tighter lag the primary can upload the WAL synchronously on every commit (`wal_sync_on_commit`), at the cost of one HTTP round-trip per commit, bounding the replica's lag to roughly the sync interval.

Promotion turns a replica into a primary, and its first act is to claim the bucket. It verifies the store still enforces conditional writes, then acquires the primary lease by advancing its epoch with a conditional, compare-and-swap write; if another node has already taken the lease the acquire fails and promotion is refused with `TDB_ERR_PRECONDITION`, leaving the node a working replica for the orchestrator to retry. Holding the lease, it stops and joins the sync thread, runs one final manifest sync and WAL replay — gated on the *outgoing* primary's epoch, so that primary's last writes are absorbed rather than skipped as stale against the new epoch — creates a local WAL for new writes, and only then atomically clears replica mode. Because the replica already holds the manifest and metadata in memory, the catch-up is a final WAL download and replay plus a file creation, milliseconds of work; the decision of *when* to promote is still left to an external orchestration layer, but the lease is what makes that decision safe to act on even when the old primary is slow to die.

The concurrency model is unchanged within a node: many threads commit concurrently with full MVCC, flush and compaction run in parallel, and the upload pipeline is itself multi-threaded. Single-writer is a *bucket-level* invariant — two nodes writing one bucket would otherwise produce conflicting manifests, colliding SSTable IDs, and racing deletes — and the engine enforces it directly rather than trusting the orchestrator to. The primary holds a lease object in the bucket whose epoch it advances with a conditional compare-and-swap write, and every manifest publish re-asserts that lease before it writes. If a superseded primary — one that lost the role but is still alive and can still reach the store, the case an external lease or STONITH cannot fully rule out — tries to publish, its lease re-assertion fails the precondition and it self-demotes to a replica instead of writing. Manifests and WAL segments carry the epoch that wrote them, and every reader (replica sync, cold start, WAL replay) ignores any object whose epoch is below the current lease, so even an object a fenced node uploads before it notices is never honored. The protection is contingent on the store actually enforcing conditional writes: a probe at startup checks this, and if the backend does not, fencing is disabled and logged, falling back to the orchestrator-honored invariant. On a clean shutdown the engine flushes everything and drains all uploads before exiting, so there is no loss. On a crash the loss window is exactly the WAL-sync configuration: zero with sync-on-commit, the byte threshold with periodic sync, a full flush cycle if periodic sync is disabled. Recovery without a replica follows the cold-start path; recovery with a warm replica is a promotion. Upload failures are surfaced through a stats counter and the shutdown

log and retried from the deferred backlog until they land, so a transient outage costs latency rather than data; download failures after all retries propagate as read errors; and at shutdown the upload pipeline broadcasts a wake to every worker so threads terminate promptly on every platform.

Testing and Quality Assurance

TidesDB is tested with CI automation across more than 15 platform and architecture combinations. Every internal module has its own test file — block manager, skip list, bloom filter, and the rest — with unit tests, integration tests, and benchmarks, and a main suite exercises the whole database lifecycle: basic operations, transactions at every isolation level, persistence, WAL recovery, every compaction strategy, iterators, TTL, compression, bloom filters, block indexes, concurrency, edge cases, and stress. The test harness takes an optional substring filter, so any binary can run a named subset.

The build adapts itself to Linux, macOS, Windows (both MSVC and MinGW under MSYS2), the BSDs, and Solaris/Illumos, detecting the platform and, on macOS, Intel versus Apple Silicon. The CI exercises an architecture matrix of x64, x86, PowerPC, and RISC-V, supplying the cross-compilation toolchains; it manages dependencies per platform, enables sanitizers on Unix, and registers everything with CTest. The installed library carries a soname and follows each OS's install conventions, so packagers need no per-distro patches.

A few details show the breadth of coverage. PowerPC is built and verified on both its targets, but how far the tests run depends on the target: the macOS PowerPC build stops at build verification, because Mach-O PowerPC binaries have no execution path on the Linux CI hosts, while the Linux PowerPC build additionally runs its tests best-effort under QEMU, where emulating PowerPC pthreads and atomics is unreliable enough that a failure is logged rather than failing the build. MinGW runs one code path for both x64 and x86, building Snappy from source because the prebuilts were dropped, and resolving zstd and lz4 through the package files MSYS2 does ship. Most tellingly, a portability test creates a database on Linux x64, ships it as an artifact, and reads every key back correctly on seven different platforms — direct proof that the on-disk format is portable across architectures and endianness.

Object storage and replication carry their own workflows. The store itself is exercised against MinIO, and failover is tested at two fidelities. A fast, dependency-free workflow runs real node processes that share a filesystem bucket on the CI runner — the filesystem connector takes a `flock`, so the single-writer fence holds across processes — and drives clean failover with promotion catch-up, the fencing of a still-alive old primary that self-demotes the moment it next tries to publish, and zero-RPO recovery of writes that were acknowledged but never flushed. A heavier cloud simulation stands up a Kubernetes cluster with `kind`, a real MinIO bucket, and the node running one per pod, so the fence is exercised over genuine S3 conditional writes rather than the filesystem shortcut: it covers fan-out replication to several replicas, a primary failure that the surviving replicas re-follow onto the newly

promoted node, the fencing of a primary that is network-partitioned from the bucket and self-demotes when it reconnects to find it has been superseded, and cascading failovers in which the lease epoch advances monotonically across each promotion.

Together this infrastructure is what keeps the engine correct, fast, and portable across every platform it claims to support.

Building & Benchmarking TidesDB

v9.3.11

Latest Version

Check the [latest release](#) on GitHub for the current version.

Supported Platforms

TidesDB is tested and supported on the following platforms:

Operating Systems

- **Linux** (x86, x64, PowerPC 32-bit, RISC-V 64-bit)
- **macOS** (x64, ARM64/Apple Silicon)
- **Windows** (x86, x64 with MSVC or MinGW)
- **FreeBSD** 14.0+ (x64)
- **OpenBSD** 7.4+ (x64)
- **NetBSD** (x64)
- **DragonFlyBSD** (x64)
- **Illumos/OmniOS** (x64)
- **Solaris** (x64)

Architectures

- **x86** (32-bit Intel/AMD)
- **x64** (64-bit Intel/AMD)
- **ARM64** (Apple Silicon, ARM v8)
- **PowerPC** (32-bit, cross-compiled)
- **RISC-V** (64-bit, cross-compiled)

Compilers

- **GCC** 7.0+ (Linux, MinGW, cross-compilation)
- **Clang** 6.0+ (macOS, Linux, BSD)
- **MSVC** 2019 16.8+ (Windows with `/experimental:c11atomics`)

Platform-Specific Notes

- **SunOS/Illumos** · Snappy compression is not available; TidesDB automatically disables it
- **PowerPC** · Requires `libatomic` for 64-bit atomic operations
- **32-bit architectures** · Require `libatomic` and multilib support
- **Windows** · Both MSVC and MinGW are fully supported with vcpkg for dependencies

Prerequisites

Build Tools

- **CMake** 3.25 or higher
- **C Compiler** with C11 support:
 - GCC 7.0+ (Linux/MinGW)
 - Clang 6.0+ (macOS/Linux)
 - MSVC 2019 16.8+ (Windows with `/experimental:c11atomics`)

Dependencies

The only hard requirements are a C11 toolchain, CMake, and threading support:

- **pthread** · POSIX threads (Linux/macOS/BSD: built-in, Windows: PThreads4W via vcpkg)
- **libatomic** · Atomic operations library (required for 32-bit architectures like PowerPC)

Compression backends (optional, enabled by default)

Each of the three compression libraries is optional. All three are built in by default; drop any with `-DTIDESDB_WITH_<NAME>=OFF`. A build with all three disabled has no compression dependencies at all and supports only `TDB_COMPRESS_NONE`, so TidesDB can be installed with nothing beyond libc and a thread library. See [Compression Backends](#) under Build Options.

- **Snappy** · Fast compression/decompression (`TIDESDB_WITH_SNAPPY`; not packaged on SunOS/Illumos, so disabled there by default)
- **LZ4** · Extremely fast compression (`TIDESDB_WITH_LZ4`)
- **Zstandard** · High compression ratio (`TIDESDB_WITH_ZSTD`)
- **C++ standard library** · Pulled in only when Snappy is enabled (Snappy's implementation is C++); a build without Snappy carries no libstdc++ dependency

Installing Dependencies

Linux (Debian/Ubuntu)



```
# Install all dependencies
sudo apt update
sudo apt install -y cmake build-essential \
    libzstd-dev liblz4-dev libsnappy-dev
```

Other Linux Distributions



```
# Fedora/RHEL/CentOS
sudo dnf install cmake gcc libzstd-devel lz4-devel snappy-devel

# Arch Linux
sudo pacman -S cmake gcc zstd lz4 snappy

# 32-bit builds (e.g., x86, PowerPC)
sudo apt install -y gcc-multilib g++-multilib libatomic1
```

macOS

TidesDB supports multiple package managers on macOS. By default, CMake will auto-detect Homebrew, but you can use MacPorts, pkgsrc, or Fink instead.

Option 1 · Homebrew (Default)



```
# Install Homebrew if not already installed
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Install dependencies
brew install cmake zstd lz4 snappy

# Build (Homebrew is auto-detected)
cmake -S . -B build
cmake --build build --clean-first --verbose
```

Option 2 · MacPorts



```
# Install MacPorts from https://www.macports.org/install.php

# Install dependencies
sudo port install cmake zstd lz4 snappy

# Build with MacPorts prefix
cmake -S . -B build -DMACOS_DEPENDENCY_PREFIX=/opt/local -DUSE_HOMEBREW=OFF
cmake --build build --clean-first --verbose
```


Option 3 · pkgsrc



```
# Install pkgsrc for macOS

# Install dependencies
pkgin install cmake zstd lz4 snappy

# Build with pkgsrc prefix
cmake -S . -B build -DMACOS_DEPENDENCY_PREFIX=/usr/pkg -DUSE_HOMEBREW=OFF
cmake --build build --clean-first --verbose
```

Option 4 · Fink



```
# Install Fink from https://www.finkproject.org/

# Install dependencies
fink install cmake zstd lz4 snappy

# Build with Fink prefix
cmake -S . -B build -DMACOS_DEPENDENCY_PREFIX=/sw -DUSE_HOMEBREW=OFF
cmake --build build --clean-first --verbose
```

Custom Paths

If you have dependencies installed in a custom location, use:



```
cmake -S . -B build -DMACOS_DEPENDENCY_PREFIX=/your/custom/path -
DUSE_HOMEBREW=OFF
```

Or set `CMAKE_PREFIX_PATH` environment variable:



```
export CMAKE_PREFIX_PATH=/your/custom/path
cmake -S . -B build -DUSE_HOMEBREW=OFF
```

Windows

Using vcpkg (Recommended)



```
# Install vcpkg if not already installed
git clone https://github.com/Microsoft/vcpkg.git
cd vcpkg
.\bootstrap-vcpkg.bat

# Install dependencies
.\vcpkg install zstd:x64-windows lz4:x64-windows snappy:x64-windows
pthreadsx64-windows

# For 32-bit builds
.\vcpkg install zstd:x86-windows lz4:x86-windows snappy:x86-windows
pthreadsx86-windows
```

BSD Variants

TidesDB supports FreeBSD, OpenBSD, NetBSD, and DragonFlyBSD with platform-specific package paths.

FreeBSD



```
# Install dependencies (packages in /usr/local)
sudo pkg install cmake pkgconf liblz4 zstd snappy

# Build
cmake -S . -B build
cmake --build build --clean-first --verbose
```

OpenBSD



```
# Install dependencies (packages in /usr/local)
sudo pkg_add cmake gmake lz4 zstd snappy pkgconf

# Build
cmake -S . -B build
cmake --build build --clean-first --verbose
```

NetBSD



```
# Install dependencies (packages in /usr/pkg)
sudo pkgin install cmake lz4 zstd snappy

# Build
cmake -S . -B build
cmake --build build --clean-first --verbose
```

DragonFlyBSD



```
# Install dependencies (packages in /usr/local)
sudo pkg install cmake lz4 zstd snappy

# Build
cmake -S . -B build
cmake --build build --clean-first --verbose
```

Illumos/OmniOS/Solaris

Note

Snappy is not available in OmniOS repositories. TidesDB automatically disables Snappy compression on SunOS builds.



```
# Install dependencies (packages in /opt/ooce)
sudo pkg install cmake lz4 zstd

# Build
cmake -S . -B build
cmake --build build --clean-first --verbose
```

PowerPC (32-bit)

Cross-compilation for PowerPC requires building dependencies from source.



```
# Install cross-compilation toolchain
sudo apt install -y crossbuild-essential-powerpc \
    gcc-powerpc-linux-gnu g++-powerpc-linux-gnu \
    qemu-user-static

# Build dependencies from source (LZ4, Zstandard, Snappy)
# See .github/workflows/build_and_test_tidesdb.yml for complete build
script

# Configure with PowerPC toolchain
cmake -S . -B build \
    -DCMAKE_C_COMPILER=powerpc-linux-gnu-gcc \
    -DCMAKE_CXX_COMPILER=powerpc-linux-gnu-g++ \
    -DCMAKE_SYSTEM_NAME=Linux \
    -DCMAKE_SYSTEM_PROCESSOR=powerpc

# Build
cmake --build build --clean-first --verbose
```

RISC-V (64-bit)

Cross-compilation for RISC-V 64-bit requires building dependencies from source.



```
# Install cross-compilation toolchain
sudo apt install -y crossbuild-essential-riscv64 \
    gcc-riscv64-linux-gnu g++-riscv64-linux-gnu \
    qemu-user-static

# Build dependencies from source (LZ4, Zstandard, Snappy)
# See .github/workflows/build_and_test_tidesdb.yml for complete build
script

# Configure with RISC-V toolchain
cmake -S . -B build \
    -DCMAKE_C_COMPILER=riscv64-linux-gnu-gcc \
    -DCMAKE_CXX_COMPILER=riscv64-linux-gnu-g++ \
    -DCMAKE_SYSTEM_NAME=Linux \
    -DCMAKE_SYSTEM_PROCESSOR=riscv64

# Build
cmake --build build --clean-first --verbose
```

Building

Once you have the dependencies installed, you can build using the commands below.

Unix (Linux/macOS)

Standard Build (Production)

A release build enables optimizations and disables debugging features:



```
rm -rf build && cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --clean-first --verbose
cmake --install build

# On linux run ldconfig to update the shared library cache
ldconfig
```

Development Build

For development, enable sanitizers and profiling:



```
rm -rf build && cmake -S . -B build \  
  -DCMAKE_BUILD_TYPE=Debug \  
  -DTIDESDB_WITH_SANITIZER=ON \  
  -DENABLE_READ_PROFILING=ON  
cmake --build build --clean-first --verbose
```

Minimal Build (No Tests)

For embedded systems or minimal deployments:



```
rm -rf build && cmake -S . -B build \  
  -DCMAKE_BUILD_TYPE=Release \  
  -DTIDESDB_BUILD_TESTS=OFF \  
  -DBUILD_SHARED_LIBS=OFF  
cmake --build build --clean-first --verbose  
cmake --install build
```

Static archives are position independent

Even with `BUILD_SHARED_LIBS=OFF`, all object files compile with `-fPIC` (via `CMAKE_POSITION_INDEPENDENT_CODE ON`). This means the static archive can link directly into a shared module like the MariaDB engine plugin without a recompile-with-fPIC step. The flag has negligible runtime cost on modern x86_64 and ARM64 toolchains.

Windows

BUILD_SHARED_LIBS on Windows

By default, TidesDB builds as a **static library** on Windows (`BUILD_SHARED_LIBS=OFF`) to avoid DLL export/import complexity. If you need a shared library (DLL), explicitly set `BUILD_SHARED_LIBS=ON` during configuration. Note that shared builds require proper symbol exports and may need additional configuration.

MinGW-w64

MinGW-w64 provides a GCC-based toolchain with better C11 support and POSIX compatibility.

Prerequisites

- Install [MinGW-w64](#)
- Install [CMake](#)
- Install [vcpkg](#) for dependencies

Build Steps



```
# Clean previous build
Remove-Item -Recurse -Force build -ErrorAction SilentlyContinue

# Configure with MinGW
cmake -S . -B build -G "MinGW Makefiles" -DCMAKE_C_COMPILER=gcc -
DCMAKE_TOOLCHAIN_FILE=C:\vcpkg\scripts\buildsystems\vcpkg.cmake

# Build (Debug or Release)
cmake --build build --clean-first --verbose --config Release

# Run tests
cd build
ctest --verbose # or use --output-on-failure to only show failures
```

MSVC (Visual Studio)

Prerequisites

- Install [Visual Studio 2019 or later](#) with C++ development tools
- Install [CMake](#)

- Install vcpkg for dependencies

Build Steps



```
# Clean previous build
Remove-Item -Recurse -Force build -ErrorAction SilentlyContinue

# Configure with MSVC
cmake -S . -B build -
DCMAKE_TOOLCHAIN_FILE=C:\vcpkg\scripts\buildsystems\vcpkg.cmake

# Build (Debug or Release)
cmake --build build --clean-first --verbose --config Release

# Run tests
cd build

ctest -C Debug --verbose
# or
ctest -C Release --verbose
```

Note

MSVC requires Visual Studio 2019 16.8 or later for C11 atomics support (`/experimental:c11atomics`). Both Debug and Release builds are fully supported.

Testing

After building, run the test suite to verify everything works



```
cd build
ctest --output-on-failure
```

Or run tests directly



```
./build/tidesdb_tests
```

Running Individual Tests

All test binaries support an optional filter argument to run only tests whose names contain a given substring. This works for every component test binary (`tidesdb_tests`, `block_manager_tests`, `skip_list_tests`, etc.).



```
# Run only checkpoint-related tests
./build/tidesdb_tests checkpoint

# Run only compaction tests
./build/tidesdb_tests compaction

# Run only bloom filter serialization tests
./build/bloom_filter_tests serialize

# Run a single test by exact name
./build/tidesdb_tests test_purge_cf_basic
```

The filter uses substring matching - any test whose function name contains the filter string will run. Tests that don't match are skipped. The summary output shows passed, failed, and skipped counts along with the active filter.

Consolidated Include

After building, TidesDB creates a consolidated include directory in the build tree for easy consumption. This allows you to use:

```
#include <tidesdb/tidesdb.h>
```

With the compiler flag:



```
-I<build_dir>/include
```

All public headers (from `src/` and `external/`) plus the generated `tidesdb_version.h` are copied to `<build_dir>/include/tidesdb/`.

Build Options

TidesDB provides several CMake options to customize the build:

Option	Description	Default
<code>TIDESDB_WITH_SANITIZER</code>	Enable address/undefined behavior sanitizers	<code>OFF</code>
<code>TIDESDB_BUILD_TESTSUITE</code>	Build test suite	<code>ON</code>
<code>BUILD_SHARED_LIBS</code>	Build shared libraries instead of static	<code>ON</code> (Unix), <code>OFF</code> (Windows)
<code>ENABLE_READ_PROFILING</code>	Enable read profiling instrumentation	<code>OFF</code>
<code>TIDESDB_WARN_MAYBE_UNINIT</code>	Enable <code>-Wmaybe-uninitialized</code> (GCC only; requires an optimized build)	<code>OFF</code>
<code>TIDESDB_WITH_SNAPPY</code>	Build with Snappy compression support	<code>ON</code> (<code>OFF</code> on SunOS)
<code>TIDESDB_WITH_LZ4</code>	Build with LZ4 compression support	<code>ON</code>
<code>TIDESDB_WITH_ZSTD</code>	Build with Zstandard compression support	<code>ON</code>
<code>TIDESDB_SNAPPY_TARGET</code>	External Snappy link target/item to use instead of autodetect	(empty)
<code>TIDESDB_LZ4_TARGET</code>	External LZ4 link target/item to use instead of autodetect	(empty)
<code>TIDESDB_ZSTD_TARGET</code>	External Zstandard link target/item to use instead of autodetect	(empty)
<code>TIDESDB_WITH_S3</code>	Build S3-compatible object store connector (requires libcurl only)	<code>OFF</code>
<code>TIDESDB_WITH_MIMALLOC</code>	Use mimalloc as the memory allocator	<code>OFF</code>
<code>TIDESDB_WITH_TCMALLOC</code>	Use tcmalloc as the memory allocator	<code>OFF</code>
<code>TIDESDB_WITH_JEMALLOC</code>	Use jemalloc as the memory allocator	<code>OFF</code>
<code>CMAKE_BUILD_TYPE</code>	Build type (Debug/Release/RelWithDebInfo)	(unset)

Compression Backends

The Snappy, LZ4, and Zstandard backends are independently optional and all enabled by default. Disable any you do not need:



```
# build without Zstandard
cmake -S . -B build -DTIDESDB_WITH_ZSTD=OFF

# zero-dependency build -- no compression libraries linked,
TDB_COMPRESS_NONE only
cmake -S . -B build -DTIDESDB_WITH_SNAPPY=OFF -DTIDESDB_WITH_LZ4=OFF -
DTIDESDB_WITH_ZSTD=OFF
```

The on-disk compression algorithm IDs are stable regardless of which backends are compiled in, so an SSTable written with an algorithm a given build lacks is reported with a clear error rather than misread. The default column family compression resolves to the best available backend at runtime (LZ4, then Zstandard, then Snappy, then none), so a default configuration always works against whatever was built.

Consumers need no compression headers

The third-party compression headers are included only inside the library, so an application that links `libtidesdb` and includes `<tidesdb/tidesdb.h>` does not need the Snappy/LZ4/Zstandard development headers itself.

Linking against a vendored compression library

By default each backend links a known CMake target if one exists in the build (for example one provided by a parent project via `FetchContent` or `add_subdirectory`), and otherwise falls back to a system `-l<name>`. If you vendor a compression library under a target name the autodetect cannot guess, point TidesDB at it explicitly:



```
# an exact CMake target from your dependency
cmake -S . -B build -DTIDESDB_ZSTD_TARGET=zstd::libzstd_static

# or any target/link item your build produces
cmake -S . -B build -DTIDESDB_ZSTD_TARGET=my_zstd_target
```

This avoids the forced system `-l` link that otherwise fails when the library is only available as an in-tree target rather than installed system-wide. The same applies to `TIDESDB_SNAPPY_TARGET` and `TIDESDB_LZ4_TARGET`.

Mimalloc Memory Allocator

TidesDB optionally supports [mimalloc](#), Microsoft's high-performance memory allocator. When enabled, mimalloc replaces the standard malloc/free at link time, providing improved allocation performance for memory-intensive workloads.

Enabling mimalloc



```
# Linux/macOS with vcpkg
cmake -S . -B build -DTIDESDB_WITH_MIMALLOC=ON

# Windows with vcpkg
cmake -S . -B build -DTIDESDB_WITH_MIMALLOC=ON -
DCMAKE_TOOLCHAIN_FILE=C:\vcpkg\scripts\buildsystems\vcpkg.cmake
```

Requirements

- mimalloc is installed via vcpkg as an optional feature
- On Windows, vcpkg manifest mode automatically installs mimalloc when the option is enabled
- No code changes required — mimalloc overrides standard allocators at link time

When to use mimalloc

- High-throughput workloads with frequent allocations
- Multi-threaded applications (mimalloc has excellent thread-local caching)
- Memory-constrained environments (mimalloc has lower fragmentation)

Performance impact

- Typical improvement - 5-15% reduction in allocation overhead
- Best results with many small allocations (skip list nodes, block buffers)
- Minimal overhead when disabled (default)

TCMalloc Memory Allocator

TidesDB optionally supports [tcmalloc](#) (Thread-Caching Malloc), Google's high-performance memory allocator from gperftools. Like mimalloc, tcmalloc replaces the standard malloc/free at link time, providing improved allocation performance especially for multi-threaded workloads.

Enabling tcmalloc



```
# Linux/macOS
cmake -S . -B build -DTIDESDB_WITH_TCMALLOC=ON

# Windows with vcpkg
cmake -S . -B build -DTIDESDB_WITH_TCMALLOC=ON -
DCMAKE_TOOLCHAIN_FILE=C:\vcpkg\scripts\buildsystems\vcpkg.cmake
```

Installing tcmalloc



```
# Linux (Debian/Ubuntu)
sudo apt install libgoogle-perftools-dev

# macOS (Homebrew)
brew install gperftools

# Windows (vcpkg)
vcpkg install gperftools:x64-windows
```

When to use tcmalloc

- High-throughput multi-threaded workloads
- Applications with many small allocations

- When you need detailed heap profiling (gperftools includes profiling tools)

Performance impact

- Typical improvement - 5-20% reduction in allocation overhead for multi-threaded code
- Excellent thread-local caching reduces lock contention
- Minimal overhead when disabled (default)

Jemalloc Memory Allocator

TidesDB optionally supports jemalloc, Facebook's high-performance memory allocator. Like mimalloc and tcmalloc, jemalloc replaces the standard malloc/free at link time, providing improved allocation performance especially for multi-threaded workloads.

Enabling jemalloc



```
# Linux/macOS
cmake -S . -B build -DTIDESDB_WITH_JEMALLOC=ON

# Windows with vcpkg
cmake -S . -B build -DTIDESDB_WITH_JEMALLOC=ON -
DCMAKE_TOOLCHAIN_FILE=C:\vcpkg\scripts\buildsystems\vcpkg.cmake
```

Installing jemalloc



```
# Linux (Debian/Ubuntu)
sudo apt install libjemalloc-dev

# macOS (Homebrew)
brew install jemalloc

# Windows (vcpkg)
vcpkg install jemalloc:x64-windows
```

When to use jemalloc

- High-throughput multi-threaded workloads

- Applications with many small allocations
- When you need detailed heap profiling (jemalloc includes profiling tools)

Performance impact

- Typical improvement - 5-25% reduction in allocation overhead for multi-threaded code
- Excellent thread-local caching reduces lock contention
- Minimal overhead when disabled (default)

Note

You cannot enable both mimalloc, tcmalloc, and jemalloc simultaneously. Choose one based on your workload characteristics.

S3 Object Store Connector

TidesDB optionally supports an S3-compatible object store connector for storing SSTables in remote storage (AWS S3, MinIO, Google Cloud Storage, or any S3-compatible endpoint). When enabled, TidesDB uses local disk as a cache and uploads SSTables to the object store for durable, replicated storage. The filesystem connector is always available without additional dependencies.

Enabling the S3 connector



```
# Linux
sudo apt install -y libcurl4-openssl-dev

cmake -S . -B build -DTIDESDB_WITH_S3=ON
cmake --build build --clean-first --verbose
```



```
# macOS
brew install curl

cmake -S . -B build -DTIDESDB_WITH_S3=ON
cmake --build build --clean-first --verbose
```

Requirements

- [libcurl](#) for HTTP requests to S3 endpoints

AWS SigV4 request signing (SHA-256 and HMAC-SHA256) is implemented inside TidesDB, so libcurl is the only external dependency — no separate crypto library is needed.

When to use object store mode

- Cloud-native deployments with separated compute and storage
- Workloads that benefit from virtually unlimited storage capacity
- Multi-node setups where cold start recovery from remote storage replaces traditional replication
- Environments where local disk is ephemeral (containers, serverless)

Filesystem connector (no extra dependencies)

The filesystem connector stores objects as files under a root directory using the same path-like key structure. It is always available and requires no build flags. Use it for testing object store mode locally or for replication to a network-mounted filesystem.

Read Profiling

TidesDB includes optional read profiling instrumentation to analyze read performance and cache effectiveness. When enabled, it tracks detailed statistics about where reads are served from (memtable, immutable, SSTable), cache hit rates, bloom filter effectiveness, and I/O patterns.

Read profiling is disabled by default to avoid overhead in production. Enable it for development or performance analysis:



```
# Development build with profiling
cmake -B build -DENABLE_READ_PROFILING=ON -DCMAKE_BUILD_TYPE=Debug
cmake --build build --clean-first --verbose
```

For production builds, profiling is automatically disabled:



```
cmake -B build # ENABLE_READ_PROFILING=OFF by default
```

Using Read Profiling

Once enabled, use the profiling API in your code:

```
#include <tidesdb.h>

int main() {
    tidesdb_t *db;
    tidesdb_config_t config = { /* ... */ };
    tidesdb_open(&config, &db);

    /* Perform operations */
    tidesdb_txn_t *txn;
    tidesdb_txn_begin(db, &txn);

    /* ... do reads ... */

    tidesdb_txn_commit(txn);

    /* Print profiling statistics */
    tidesdb_print_read_stats(db);

    /* Or get stats programmatically */
    tidesdb_read_stats_t stats;
    tidesdb_get_read_stats(db, &stats);
    printf("Total reads: %lu\n", stats.total_reads);
    printf("Cache hit rate: %.1f%%\n",
           100.0 * stats.cache_block_hits /
           (stats.cache_block_hits + stats.cache_block_misses));

    tidesdb_close(db);
    return 0;
}
```

Statistics Collected

When profiling is enabled, TidesDB tracks:

Read Hit Location

- `total_reads` · Total number of read operations
- `memtable_hits` · Reads served from active memtable (fastest)
- `immutable_hits` · Reads served from immutable memtables

- `sstable_hits` · Reads served from SSTables on disk

SSTable Search Efficiency

- `levels_searched` · Total LSM levels searched across all reads
- `sstables_checked` · Total SSTables examined (bloom filter + range checks)
- `bloom_checks` · Number of bloom filter checks performed
- `bloom_hits` · Bloom filter checks that indicated key might exist

Block-Level Cache Performance

- `cache_block_hits` · Blocks served from cache (zero I/O)
- `cache_block_misses` · Blocks read from disk
- `disk_reads` · Total disk read operations
- `blocks_read` · Total blocks read from disk

Example Output

```
*=== TidesDB Read Profiling Stats ===*
Total Reads:          10000

Read Hit Location:
  Memtable hits:      2500 (25.0%)
  Immutable hits:     1500 (15.0%)
  SSTable hits:       6000 (60.0%)

SSTable Search:
  Levels searched:    18000 (avg: 1.80 per read)
  SSTables checked:   24000 (avg: 2.40 per read)
  Bloom checks:       24000
  Bloom hits:         6500 (27.1%)

Block-Level Cache:
  Cache hits:         15000
  Cache misses:       3000
  Cache hit rate:     83.3%
  Blocks read:        3000 (avg: 0.30 per read)
  Disk reads:         3000
```

Interpreting Results

Good Performance Indicators

- High memtable/immutable hit rate (>20%) · Recent data is being accessed
- High cache hit rate (>80%) · Block cache is sized appropriately
- High bloom hit rate (>90%) · Bloom filters are effective at filtering non-existent keys
- Low avg levels per read (<2.0) · LSM tree is well-compacted
- Low avg blocks per read (<1.0) · Good data locality

Performance Issues

- Low cache hit rate (<50%) · Consider increasing `block_cache_size`
- High avg levels per read (>3.0) · May need more aggressive compaction
- Low bloom hit rate (<80%) · Increase bloom filter size or reduce false positive rate
- High avg SSTables per read (>5.0) · Compaction may be falling behind

Performance Impact

When **enabled**

- Adds atomic increment operations on read path (~1-2% overhead)
- Minimal memory overhead (~64 bytes for stats structure)
- No impact on write performance

When **disabled** (default)

- Zero overhead - profiling code is compiled out via `#ifdef`
- No runtime cost

Use Cases

Development & Testing



```
# Enable profiling and sanitizers for development
cmake -B build \
  -DENABLE_READ_PROFILING=ON \
  -DIDESDB_WITH_SANITIZER=ON \
  -DCMAKE_BUILD_TYPE=Debug
```

Performance Tuning



```
# Profile with production-like config (no sanitizers)
cmake -B build \
  -DENABLE_READ_PROFILING=ON \
  -DCMAKE_BUILD_TYPE=Release
```

Production Deployment



```
# Default: profiling and sanitizers disabled for zero overhead
cmake -B build -DCMAKE_BUILD_TYPE=Release
```

Notes

- Statistics are cumulative from database open to close
- Thread-safe using atomic operations
- Can be queried multiple times during database lifetime
- Useful for identifying cache sizing issues and compaction tuning needs
- Benchmark suite automatically uses profiling when available

Benchmarking

TidesDB includes a comprehensive benchmark suite with fully configurable parameters.



Build benchmarks in Release

Configure with `-DCMAKE_BUILD_TYPE=Release`. CMake sets no build type by default, which compiles at `-O0`, and an unoptimized binary produces numbers that do not represent real performance.

Quick Start



```
# Build and run with defaults
cmake -B build
cmake --build build --clean-first --verbose
./build/tidesdb_bench
```

Benchmark Operations

The benchmark tests the following operations

- Concurrent writes across multiple threads
- Concurrent reads across multiple threads
- Concurrent deletes across multiple threads
- Full scan from first to last key
- Full scan from last to first key
- Random seeks to specific keys

Configuration Options

All benchmark parameters can be customized at build time using CMake variables

Operation Parameters

Variable	Description	Default
BENCH_NUM_OPERATIONS	Number of put/get/delete operations	10000000
BENCH_NUM_SEEK_OPS	Number of iterator seek operations	100
BENCH_KEY_SIZE	Key size in bytes	16
BENCH_VALUE_SIZE	Value size in bytes	100
BENCH_NUM_THREADS	Number of concurrent threads	8
BENCH_KEY_PATTERN	Key distribution pattern	"random"
BENCH_CF_NAME	Column family name	"benchmark_cf"
BENCH_DB_PATH	Directory path	"benchmark_db"
BENCH_DB_LOG_LEVEL	Database log level	TDB_LOG_DEBUG
BENCH_DB_MAX_MEMORY	Global memory limit in bytes (0 = auto, 75% of system RAM)	0
BENCH_DB_FLUSH_POOL_THREADS	Flush thread pool size	2
BENCH_DB_COMPACTION_POOL_THREADS	Compaction thread pool size	2

Column Family Configuration

Variable	Description	Default
<code>BENCH_WRITE_BUFFER_SIZE</code>	Memtable flush threshold (bytes)	67108864 (64MB)
<code>BENCH_LEVEL_RATIO</code>	Level size multiplier	10
<code>BENCH_DIVIDING_LEVEL_OFFSET</code>	Compaction dividing level offset	2
<code>BENCH_MIN_LEVELS</code>	Minimum LSM levels, TidesDB can scale up levels if need be.	5
<code>BENCH_SKIP_LIST_MAX_LEVEL</code>	Skip list max level	12
<code>BENCH_SKIP_LIST_PROBABILITY</code>	Skip list probability	0.25
<code>BENCH_ENABLE_COMPRESSION</code>	Enable compression (0=off, 1=on)	1
<code>BENCH_COMPRESSION_ALGORITHM</code>	Compression algorithm	TDB_COMPRESS_LZ4
<code>BENCH_ENABLE_BLOOM_FILTER</code>	Enable bloom filter (0=off, 1=on)	1
<code>BENCH_BLOOM_FILTER_FP_RATE</code>	Bloom filter false positive rate	0.01
<code>BENCH_ENABLE_BLOCK_INDEXES</code>	Enable block indexes	1
<code>BENCH_BLOCK_INDEX_SAMPLING_COUNT</code>	Block index sampling ratio (every Nth block)	1
<code>BENCH_BLOCK_INDEX_PREFIX_LENGTH</code>	Prefix length for block index keys (bytes)	16
<code>BENCH_SYNC_MODE</code>	Sync mode	TDB_SYNC_NONE

Variable	Description	Default
<code>BENCH_SYNC_INTERVAL_US</code>	Sync interval in microseconds (for <code>TDB_SYNC_INTERVAL</code>)	128000 (128ms)
<code>BENCH_COMPARATOR_NAME</code>	Key comparator	"memcmp"
<code>BENCH_BLOCK_CACHE_SIZE</code>	Global block cache size (bytes)	67108864 (64MB)
<code>BENCH_ISOLATION_LEVEL</code>	Transaction isolation level	<code>TDB_ISOLATION_READ_COMMITTED</code>
<code>BENCH_KLOG_VALUE_THRESHOLD</code>	Key-log value separation threshold (bytes)	512
<code>BENCH_MIN_DISK_SPACE</code>	Minimum free disk space required (bytes)	104857600 (100MB)
<code>BENCH_L1_FILE_COUNT_TRIGGER</code>	L1 file count trigger for compaction	4
<code>BENCH_L0_QUEUE_STALL_THRESHOLD</code>	L0 queue stall threshold for backpressure	20
<code>BENCH_MAX_OPEN_SSTABLES</code>	Maximum number of open SSTables	256
<code>BENCH_USE_BTREE</code>	Use B+tree format for klog instead of block-based (0=off, 1=on)	0

Key Distribution Patterns

The benchmark supports three key distribution patterns to simulate different workloads:

Sequential

Keys are generated in order: `key_000000000000000000`, `key_000000000000000001`, etc.

- Use case · Testing sequential write/read performance
- Best for · Measuring raw throughput with optimal cache behavior



```
cmake -B build -DBENCH_KEY_PATTERN=sequential
```

Random (Default)

Keys are randomly generated alphanumeric strings.

- Use case · Testing random access patterns
- Best for · Simulating unpredictable workloads with poor cache locality



```
cmake -B build -DBENCH_KEY_PATTERN=random
```

Zipfian

Keys follow a Zipfian distribution (80/20 rule) where 80% of accesses go to 20% of keys.

- Use case · Simulating real-world workloads with “hot” keys
- Best for · Testing cache effectiveness and realistic access patterns



```
cmake -B build -DBENCH_KEY_PATTERN=zipfian
```

Example Configurations

Small Workload (Default)



```
cmake -B build
cmake --build build --clean-first --verbose
./build/tidesdb_bench
```

Medium Workload



```
cmake -B build \  
  -DBENCH_NUM_OPERATIONS=1000 \  
  -DBENCH_NUM_SEEK_OPS=100 \  
  -DBENCH_NUM_THREADS=4 \  
  -DBENCH_KEY_PATTERN=random  
  
cmake --build build --clean-first --verbose  
./build/tidesdb_bench
```

Large Workload with Zipfian Distribution



```
cmake -B build \  
  -DBENCH_NUM_OPERATIONS=100000 \  
  -DBENCH_NUM_SEEK_OPS=10000 \  
  -DBENCH_KEY_SIZE=32 \  
  -DBENCH_VALUE_SIZE=512 \  
  -DBENCH_NUM_THREADS=8 \  
  -DBENCH_KEY_PATTERN=zipfian  
  
cmake --build build --clean-first --verbose  
./build/tidesdb_bench
```

Stress Test with No Compression



```
cmake -B build \  
  -DBENCH_NUM_OPERATIONS=10000000 \  
  -DBENCH_KEY_SIZE=128 \  
  -DBENCH_VALUE_SIZE=4096 \  
  -DBENCH_NUM_THREADS=16 \  
  -DBENCH_ENABLE_COMPRESSION=0 \  
  -DBENCH_ENABLE_BLOOM_FILTER=0 \  
  -DBENCH_KEY_PATTERN=sequential  
  
cmake --build build --clean-first --verbose  
./build/tidesdb_bench
```

B+tree Format Benchmark



```
# Enable B+tree format for klog (faster point lookups, O(log N) tree  
traversal)  
cmake -B build \  
  -DBENCH_USE_BTREE=1 \  
  -DBENCH_NUM_OPERATIONS=100000 \  
  -DBENCH_KEY_PATTERN=random  
  
cmake --build build --clean-first --verbose  
./build/tidesdb_bench
```

Testing Different Compression Algorithms



```
# LZ4 (default, good balance)
cmake -B build -DBENCH_COMPRESSION_ALGORITHM=TDB_COMPRESS_LZ4

# LZ4 Fast (fastest compression)
cmake -B build -DBENCH_COMPRESSION_ALGORITHM=TDB_COMPRESS_LZ4_FAST

# Snappy (balanced)
cmake -B build -DBENCH_COMPRESSION_ALGORITHM=TDB_COMPRESS_SNAPPY

# Zstandard (best compression ratio)
cmake -B build -DBENCH_COMPRESSION_ALGORITHM=TDB_COMPRESS_ZSTD
```

Benchmark Output

The benchmark displays

- Configuration summary (operations, threads, key pattern, features enabled)
- Operations per second for each operation type
- Total time taken for each operation
- Debug logs (if `enable_debug_logging` is enabled in config)

Notes

- The benchmark creates a temporary storage directory that is cleaned up before each run
- All operations use transactions for consistency
- Results may vary based on system resources, disk speed, and configuration
- For accurate results, run multiple times and average the results
- Disable debug logging for production-like performance measurements

Getting Help

If you encounter issues not covered here:

1. Check the [GitHub Issues](#) for similar problems
2. Review the [CI build logs](#) for working configurations
3. Open a new issue with
 - Your OS and version

- Compiler and version
- Full build output
- CMake configuration command used

TidesDB C API Reference

v9.3.11

Overview

TidesDB is designed to provide a simple and intuitive C API for all your embedded storage needs. This document is complete reference for the C API covering database operations, transactions, column families, iterators and more.

Include

Note

You can use other components of TidesDB such as skip list, bloom filter etc. under `tidesdb/` - this also prevents collisions.

Choosing Between `tidesdb.h` and `db.h`

TidesDB provides two header files with different purposes:

tidesdb.h

Full C implementation header. Mainly use this for native C/C++ applications.

```
#include <tidesdb/tidesdb.h>
```

db.h

db.h is mainly an FFI/Language binding interface with minimal dependencies and simpler ABI.

```
#include <tidesdb/db.h>
```




When to Use Each

- C/C++ applications → Use `tidesdb.h` for full access to the API
- Language bindings (jextract, rust-bindgen, ctypes, cgo, etc.) → Use `db.h` for a stable FFI interface
- Library developers → Use `db.h` to avoid exposing internal implementation details

Error Codes

TidesDB provides detailed error codes for production use.

Code	Value	Description
TDB_SUCCE SS	0	Operation completed successfully
TDB_ERR_M EMORY	-1	Memory allocation failed
TDB_ERR_I NVALID_AR GS	-2	Invalid arguments passed to function (NULL pointers, invalid sizes, etc.)
TDB_ERR_N OT_FOUND	-3	Key not found in column family
TDB_ERR_I O	-4	I/O operation failed (file read/write error)
TDB_ERR_C ORRUPTION	-5	Data corruption detected (checksum failure, invalid format version, truncated data)
TDB_ERR_E XISTS	-6	Resource already exists (e.g., column family name collision)
TDB_ERR_C ONFLICT	-7	Transaction conflict detected (read-write conflict in REPEATABLE_READ, write-write conflict in SNAPSHOT, full SSI in SERIALIZABLE)
TDB_ERR_T OO_LARGE	-8	Key or value size exceeds maximum allowed size
TDB_ERR_M EMORY_LIM IT	-9	Operation would exceed memory limits (safety check to prevent OOM)
TDB_ERR_I NVALID_DB	-10	Database handle is invalid (e.g., after close)
TDB_ERR_U NKNOWN	-11	Unknown or unspecified error
TDB_ERR_L OCKED	-12	Database is locked by another process
TDB_ERR_R EADONLY	-13	Database is in read-only replica mode (writes rejected)
TDB_ERR_B USY	-14	System is at capacity; the backpressure stall hit its no-progress budget. Transient — the caller should retry. Matches POSIX EBUSY semantics

Code	Value	Description
<code>TDB_ERR_P RECONDITI ON</code>	<code>-15</code>	A conditional object-store write failed its precondition (HTTP 412). Raised by single-writer fencing — e.g. <code>tidesdb_promote_to_primary</code> when another node already holds the primary lease

Error categories

- `TDB_ERR_CORRUPTION` indicates data integrity issues requiring immediate attention
- `TDB_ERR_CONFLICT`, `TDB_ERR_BUSY` indicate transient conditions (retry may succeed)
- `TDB_ERR_MEMORY`, `TDB_ERR_MEMORY_LIMIT`, `TDB_ERR_TOO_LARGE` indicate resource constraints
- `TDB_ERR_NOT_FOUND`, `TDB_ERR_EXISTS` are normal operational conditions, not failures

Example Error Handling

```
int result = tidesdb_txn_put(txn, cf, key, key_size, value,
value_size, -1);
if (result != TDB_SUCCESS)
{
    switch (result)
    {
        case TDB_ERR_MEMORY:
            fprintf(stderr, "out of memory\n");
            break;
        case TDB_ERR_INVALID_ARGS:
            fprintf(stderr, "invalid arguments\n");
            break;
        case TDB_ERR_CONFLICT:
            fprintf(stderr, "transaction conflict detected\n");
            break;
        default:
            fprintf(stderr, "operation failed with error code: %d\n",
result);
            break;
    }
    return -1;
}
```

Initialization

TidesDB supports **optional** custom memory allocators for integration with custom memory managers (e.g., a Redis module allocator, jemalloc, etc). You don't need to do this, this is completely optional.

tidesdb_init

Initializes TidesDB with *optional* custom memory allocation functions. When in use, this must be called exactly once before any other TidesDB function.

```
int tidesdb_init(tidesdb_malloc_fn malloc_fn, tidesdb_calloc_fn
                calloc_fn,
                tidesdb_realloc_fn realloc_fn, tidesdb_free_fn free_fn);
```

Parameters

Name	Type	Description
<code>malloc_fn</code>	<code>tidesdb_malloc_fn</code>	Custom malloc function (or <code>NULL</code> for system malloc)
<code>calloc_fn</code>	<code>tidesdb_calloc_fn</code>	Custom calloc function (or <code>NULL</code> for system calloc)
<code>realloc_fn</code>	<code>tidesdb_realloc_fn</code>	Custom realloc function (or <code>NULL</code> for system realloc)
<code>free_fn</code>	<code>tidesdb_free_fn</code>	Custom free function (or <code>NULL</code> for system free)

Returns

- `0` on success
- `-1` if already initialized

Example (system allocator)

```
tidesdb_init(NULL, NULL, NULL, NULL);
```

Example (custom allocator)

```
tidesdb_init(RedisModule_Alloc, RedisModule_Calloc,
             RedisModule_Realloc, RedisModule_Free);
```

Auto-initialization

If `tidesdb_init()` is not called, TidesDB will auto-initialize with the system allocator on the first call to `tidesdb_open()`.

tidesdb_finalize

Finalizes TidesDB and resets the allocator. Should be called after all TidesDB operations are complete. After calling this, `tidesdb_init()` can be called again.

```
void tidesdb_finalize(void);
```

Example

```
tidesdb_init(NULL, NULL, NULL, NULL);

/* ... use TidesDB ... */

tidesdb_close(db);
tidesdb_finalize();
```

Storage Engine Operations

Opening TidesDB

```
tidesdb_config_t config = {
    .db_path = "./mydb",
    .num_flush_threads = 2,                                /* Flush thread pool size
(default: 0 = auto, min(cpu_count, 4)) */
    .num_compaction_threads = 2,                            /* Compaction thread pool
size (default: 2) */
    .log_level = TDB_LOG_INFO,                               /* Log level:
TDB_LOG_DEBUG, TDB_LOG_INFO, TDB_LOG_WARN, TDB_LOG_ERROR, TDB_LOG_FATAL,
TDB_LOG_NONE */
    .block_cache_size = 64 * 1024 * 1024,                  /* 64MB global block
cache (default: 64MB) */
    .max_open_sstables = 256,                               /* Max cached SSTable
structures (default: 256 on Linux/macOS/most BSDs, 64 on OpenBSD which has
lower default fd limits) / 0 for AUTO */
    .max_memory_usage = 0,                                  /* Global memory limit in
bytes (default: 0 = auto, 75% of system RAM; minimum: 5% of system RAM) */
    .log_to_file = 0,                                       /* Write logs to file
instead of stderr (default: 0) */
    .log_truncation_at = 24 * (1024*1024),                 /* Log file truncation
size (default: 24MB), 0 = no truncation */
    .unified_memtable = 0,                                  /* Enable unified
memtable mode (default: 0 = per-CF memtables) */
    .unified_memtable_write_buffer_size = 0,               /* Unified memtable write
buffer size (default: 0 = use TDB_DEFAULT_WRITE_BUFFER_SIZE, 64MB) */
    .unified_memtable_skip_list_max_level = 0,             /* Skip list max level
for unified memtable (default: 0 = 12) */
    .unified_memtable_skip_list_probability = 0,           /* Skip list probability
for unified memtable (default: 0 = 0.25) */
    .unified_memtable_sync_mode = 0,                       /* Sync mode for unified
WAL (default: 0 = TDB_SYNC_NONE) */
    .unified_memtable_sync_interval_us = 0,               /* Sync interval for
unified WAL in microseconds (default: 0) */
    .object_store = NULL,                                   /* Object store connector
(default: NULL = local disk). Set via tidesdb_objstore_fs_create /
tidesdb_objstore_s3_create; see Object Store Mode. Enabling it turns on
unified memtable mode automatically */
    .object_store_config = NULL,                           /* Object store behavior
config (default: NULL = defaults from tidesdb_objstore_default_config()) */
    .max_concurrent_flushes = 0,                           /* Pinned 1:1 to
num_flush_threads at open. 0 = match the pool size automatically. Any other
```

```

value is normalized to match num_flush_threads with a TDB_LOG_WARN */
    .finish_compactions_on_close = 0,          /* 0 (default) = cancel
in-flight compactions at their next checkpoint for a fast shutdown
(lossless; inputs are preserved and the merge is redone later). 1 = let in-
flight compactions run to completion before close returns */
};

tidesdb_t *db = NULL;
if (tidesdb_open(&config, &db) != 0)
{
    return -1;
}

if (tidesdb_close(db) != 0)
{
    return -1;
}

```

Using default configuration

Use `tidesdb_default_config()` to get a configuration with sensible defaults, then override specific fields as needed:

```

tidesdb_config_t config = tidesdb_default_config();
config.db_path = "./mydb";
config.log_level = TDB_LOG_WARN; /* Override: only warnings and errors */

tidesdb_t *db = NULL;
if (tidesdb_open(&config, &db) != 0)
{
    return -1;
}

```

Multiple DBs Allowed

Multiple TidesDB instances can be opened in the same process, each with its own configuration and data directory.

Raising the Open File Limit

Each open SSTable uses two file descriptors (klog and vlog), so a database with many SSTables can run into the process open-file ceiling. The engine sizes `max_open_sstables` to fit that ceiling at open time and never raises it on its own.

```
long tidesdb_raise_open_file_limit(long desired);
```

Call it **before** `tidesdb_open()` to raise the process ceiling toward `desired` descriptors so a larger working set of SSTables can stay open. On POSIX it raises the `RLIMIT_NOFILE` soft limit toward the hard limit; on Windows it raises the CRT stdio cap. It returns the open-file ceiling in effect after the attempt. The call is opt-in, a failed or partial raise is non-fatal (the prior ceiling stands), and a `desired` at or below the current ceiling simply reports it.

```
long ceiling = tidesdb_raise_open_file_limit(65536);
printf("open-file ceiling is now %ld\n", ceiling);
/* then open the database */
```

Logging

TidesDB provides structured logging with multiple severity levels.

Log Levels

- `TDB_LOG_DEBUG` · Detailed diagnostic information
- `TDB_LOG_INFO` · General informational messages (default)
- `TDB_LOG_WARN` · Warning messages for potential issues
- `TDB_LOG_ERROR` · Error messages for failures
- `TDB_LOG_FATAL` · Critical errors that may cause shutdown
- `TDB_LOG_NONE` · Disable all logging

Configure at startup


```
tidesdb_config_t config = {
    .db_path = "./mydb",
    .log_level = TDB_LOG_DEBUG /* Enable debug logging */
};

tidesdb_t *db = NULL;
tidesdb_open(&config, &db);
```

Production configuration

```
tidesdb_config_t config = {
    .db_path = "./mydb",
    .log_level = TDB_LOG_WARN /* Only warnings and errors */
};
```

Output format Logs are written to **stderr** by default with timestamps:

```
[HH:MM:SS.mmm] [LEVEL] filename:line: message
```

Example output

```
[22:58:00.454] [INFO] tidesdb.c:9322: Opening TidesDB with path=./mydb
[22:58:00.456] [INFO] tidesdb.c:9478: Block clock cache created with
max_bytes=64.00 MB
```

Log to file

Enable `log_to_file` to write logs to a `LOG` file in the database directory instead of stderr:

```

tidesdb_config_t config = {
    .db_path = "./mydb",
    .log_level = TDB_LOG_DEBUG,
    .log_to_file = 1 /* Write to ./mydb/LOG instead of stderr */
};

tidesdb_t *db = NULL;
tidesdb_open(&config, &db);
/* Logs are now written to ./mydb/LOG */

```

The log file is opened in append mode and uses line buffering for real-time logging. If the log file cannot be opened, logging falls back to default.

Writing your own log lines

Applications can emit lines into the same log stream through the engine's logger. `tidesdb_log_write(int level, const char *file, int line, const char *fmt, ...)` is the underlying primitive; it honors the configured `log_level` and respects the file-versus-stderr destination, so your messages are filtered and formatted exactly like the engine's own.

```

tidesdb_log_write(TDB_LOG_INFO, __FILE__, __LINE__, "loaded %d
tenants", n);

```

Redirect stderr to file (alternative)



```

./your_program 2> tidesdb.log # Redirect std output to file

```

Backup

`tidesdb_backup` creates an on-disk snapshot of an open database without blocking normal reads/writes.

```

int tidesdb_backup(tidesdb_t *db, char *dir);

```

Usage

```
tidesdb_t *db = NULL;
tidesdb_open(&config, &db);

if (tidesdb_backup(db, "./mydb_backup") != 0)
{
    fprintf(stderr, "Backup failed\n");
}
```

Behavior

- Requires `dir` to be a non-existent directory or an empty directory; returns `TDB_ERR_EXISTS` if not empty.
- Does not copy the `LOCK` file, so the backup can be opened normally.
- Two-phase copy approach:
 - Copies immutable files first (SSTables listed in the manifest plus metadata/config files) and skips WALs.
 - Forces memtable flushes, waits for flush/compaction queues to drain, then copies any remaining files (including WALs and updated manifests). Existing SSTables already copied are not recopied.
- Database stays open and usable during backup; no exclusive lock is taken on the source directory.

Notes

- The backup represents the database state after the final flush/compaction drain.
- If you need a quiesced backup window, you can pause writes at the application level before calling this API.

Checkpoint

`tidesdb_checkpoint` creates a lightweight, near-instant snapshot of an open database using hard links instead of copying SStable data.

```
int tidesdb_checkpoint(tidesdb_t *db, const char *checkpoint_dir);
```

Usage

```

tidesdb_t *db = NULL;
tidesdb_open(&config, &db);

if (tidesdb_checkpoint(db, "./mydb_checkpoint") != 0)
{
    fprintf(stderr, "Checkpoint failed\n");
}

```

Behavior

- Requires `checkpoint_dir` to be a non-existent directory or an empty directory; returns `TDB_ERR_EXISTS` if not empty.
- For each column family:
 - Flushes the active memtable so all data is in SSTables.
 - Halts compactions to ensure a consistent view of live SSTable files.
 - Hard links all SSTable files (`.klog` and `.vlog`) into the checkpoint directory, preserving the level subdirectory structure.
 - Copies small metadata files (manifest, config) into the checkpoint directory.
 - Resumes compactions.
- Falls back to file copy if hard linking fails (e.g., cross-filesystem).
- Database stays open and usable during checkpoint; no exclusive lock is taken on the source directory.

Checkpoint vs Backup

	<code>tidesdb_backup</code>	<code>tidesdb_checkpoint</code>
Speed	Copies every SSTable byte-by-byte	Near-instant (hard links, O(1) per file)
Disk usage	Full independent copy	No extra disk until compaction removes old SSTables
Portability	Can be moved to another filesystem or machine	Same filesystem only (hard link requirement)
Use case	Archival, disaster recovery, remote shipping	Fast local snapshots, point-in-time reads, streaming backups

Notes

- The checkpoint represents the database state at the point all memtables are flushed and compactions are halted.

- Hard-linked files share storage with the live database. Deleting the original database does not affect the checkpoint (hard link semantics).
- The checkpoint can be opened as a normal TidesDB database with `tidesdb_open`.

Column Family Operations

Creating a Column Family

Column families are isolated key-value stores. Use the config struct for customization or use defaults.

```
/* Create with default configuration */
tidesdb_column_family_config_t cf_config =
tidesdb_default_column_family_config();

if (tidesdb_create_column_family(db, "my_cf", &cf_config) != 0)
{
    return -1;
}
```

Custom configuration example

```

tidesdb_column_family_config_t cf_config = {
    .write_buffer_size = 128 * 1024 * 1024,          /* 128MB
memtable flush threshold */
    .level_size_ratio = 10,                          /* Level size
multiplier (default: 10) */
    .min_levels = 1,                                  /* Minimum
LSM levels (default: 1) */
    .dividing_level_offset = 1,                      /* Spooky
dividing level  $X = \text{num\_levels} - 1 - \text{offset}$ , clamped to  $\geq 1$ . offset=0 is
2L-Spooky (transient space-amp bounded by  $1/T$ , highest write-amp); offset=1
is the paper's recommended generalized tuning (default); higher offsets
reduce write-amp further but multiply the number of open files per Spooky
Eq 12 */
    .skip_list_max_level = 12,                      /* Skip list
max level */
    .skip_list_probability = 0.25f,                 /* Skip list
probability */
    .compression_algorithm = TDB_COMPRESS_LZ4,      /*
TDB_COMPRESS_LZ4, TDB_COMPRESS_LZ4_FAST, TDB_COMPRESS_ZSTD,
TDB_COMPRESS_SNAPPY, or TDB_COMPRESS_NONE */
    .enable_bloom_filter = 1,                      /* Enable
bloom filters */
    .bloom_fpr = 0.01,                             /* 1% false
positive rate */
    .enable_block_indexes = 1,                      /* Enable
compact block indexes */
    .index_sample_ratio = 1,                        /* Sample
every block for index (default: 1) */
    .block_index_prefix_len = 16,                   /* Block
index prefix length (default: 16) */
    .sync_mode = TDB_SYNC_FULL,                     /*
TDB_SYNC_NONE, TDB_SYNC_INTERVAL, or TDB_SYNC_FULL */
    .sync_interval_us = 1000000,                   /* Sync
interval in microseconds (1 second, only for TDB_SYNC_INTERVAL) */
    .comparator_name = {0},                         /* Empty =
use default "memcmp" */
    .comparator_ctx_str = {0},                     /* Optional
persisted string context for a parameterized comparator, so it is re-
created with the same parameters after a restart (empty = none) */
    .klog_value_threshold = 512,                    /* Values  $\geq$ 
512 bytes go to vlog (default: 512) */
    .min_disk_space = 100 * 1024 * 1024,           /* Minimum
disk space required (default: 100MB) */

```

```

        .default_isolation_level = TDB_ISOLATION_READ_COMMITTED, /* Default
transaction isolation */
        .l1_file_count_trigger = 4,                               /* L1 file
count trigger for compaction (default: 4) */
        .l0_queue_stall_threshold = 10,                          /* L0 queue
stall threshold (default: 10) */
        .tombstone_density_trigger = 0.0,                        /* Tombstone
ratio above which compaction escalates (default: 0.0 = disabled, range 0.0
to 1.0) */
        .tombstone_density_min_entries = 1024,                  /* Minimum
entry count for an SSTable to be considered by the density trigger
(default: 1024) */
        .use_btree = 0                                           /* Use B+tree
format for klog (default: 0 = block-based) */
};

if (tidesdb_create_column_family(db, "my_cf", &cf_config) != 0)
{
    return -1;
}

```

Using custom comparator

```

/* Register comparator after opening database but before creating CF
*/
tidesdb_register_comparator(db, "reverse", my_reverse_compare, NULL, NULL);

tidesdb_column_family_config_t cf_config =
tidesdb_default_column_family_config();
strncpy(cf_config.comparator_name, "reverse", TDB_MAX_COMPARATOR_NAME - 1);
cf_config.comparator_name[TDB_MAX_COMPARATOR_NAME - 1] = '\0';

if (tidesdb_create_column_family(db, "sorted_cf", &cf_config) != 0)
{
    return -1;
}

```

Dropping a Column Family

By name (looks up column family internally):

```
if (tidesdb_drop_column_family(db, "my_cf") != 0)
{
    return -1;
}
```

By pointer (skips name lookup when you already have the pointer):

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

if (tidesdb_delete_column_family(db, cf) != 0)
{
    return -1;
}
```

Which to use

- `tidesdb_drop_column_family(db, name)` · convenient when you only have the name
- `tidesdb_delete_column_family(db, cf)` · faster when you already hold a `tidesdb_column_family_t*`, avoids a redundant linear scan

Renaming a Column Family

Atomically rename a column family and its underlying directory. The operation drains all in-flight writes and waits for any in-progress flush or compaction to complete before renaming.

```
if (tidesdb_rename_column_family(db, "old_name", "new_name") != 0)
{
    return -1;
}
```

Behavior

- Marks the column family to reject new writes during the rename
- Force-flushes the active memtable to rotate the WAL and drain in-flight transactions
- Waits unbounded for any in-progress flush or compaction to complete

- Atomically renames the column family directory on disk
- Updates all internal paths (SSTables, manifest, config)
- Clears the write rejection mark on completion or error
- Thread-safe with proper locking

Return values

- `TDB_SUCCESS` · Rename completed successfully
- `TDB_ERR_INVALID_ARGS` · `db`, `old_name`, or `new_name` is NULL
- `TDB_ERR_NOT_FOUND` · Column family with `old_name` doesn't exist
- `TDB_ERR_EXISTS` · Column family with `new_name` already exists
- `TDB_ERR_IO` · Failed to rename directory on disk

Cloning a Column Family

Create a complete copy of an existing column family with a new name. The clone contains all the data from the source at the time of cloning.

```
if (tidesdb_clone_column_family(db, "source_cf", "cloned_cf") != 0)
{
    return -1;
}

/* Both column families now exist independently */
tidesdb_column_family_t *original = tidesdb_get_column_family(db,
"source_cf");
tidesdb_column_family_t *clone = tidesdb_get_column_family(db,
"cloned_cf");
```

Behavior

- Flushes the source column family's memtable to ensure all data is on disk
- Waits for any in-progress flush or compaction to complete
- Copies all SSTable files (`.klog` and `.vlog`) to the new directory
- Copies manifest and configuration files
- Creates a new column family structure and loads the copied SSTables
- The clone is completely independent - modifications to one do not affect the other

Use cases

- Testing · Create a copy of production data for testing without affecting the original
- Branching · Create a snapshot of data before making experimental changes
- Migration · Clone data before schema or configuration changes
- Backup verification · Clone and verify data integrity without modifying the source

Return values

- `TDB_SUCCESS` · Clone completed successfully
- `TDB_ERR_NOT_FOUND` · Source column family doesn't exist
- `TDB_ERR_EXISTS` · Destination column family already exists
- `TDB_ERR_INVALID_ARGS` · Invalid arguments (NULL pointers or same source/destination name)
- `TDB_ERR_IO` · Failed to copy files or create directory

Clone vs Backup

`tidesdb_clone_column_family` creates a new column family within the same database instance. For creating an external backup of the entire database, use `tidesdb_backup` instead.

Getting a Column Family

Retrieve a column family pointer to use in operations.

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (cf == NULL)
{
    /* Column family not found */
    return -1;
}
```

Listing Column Families

Get all column family names on the TidesDB instance.

```
char **names = NULL;
int count = 0;

if (tidesdb_list_column_families(db, &names, &count) == 0)
{
    printf("Found %d column families:\n", count);
    for (int i = 0; i < count; i++)
    {
        printf("  - %s\n", names[i]);
        free(names[i]);
    }
    free(names);
}
```

Column Family Statistics

Get detailed statistics about a column family.

```

tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_stats_t *stats = NULL;
if (tidesdb_get_stats(cf, &stats) == 0)
{
    printf("Memtable Size: %zu bytes\n", stats->memtable_size);
    printf("Number of Levels: %d\n", stats->num_levels);
    printf("Total Keys: %" PRIu64 "\n", stats->total_keys);
    printf("Total Data Size: %" PRIu64 " bytes\n", stats->total_data_size);
    printf("Avg Key Size: %.1f bytes\n", stats->avg_key_size);
    printf("Avg Value Size: %.1f bytes\n", stats->avg_value_size);
    printf("Read Amplification: %.2f\n", stats->read_amp);
    printf("Cache Hit Rate: %.1f%%\n", stats->hit_rate * 100.0);

    /* Column family name is available via config */
    printf("Column Family: %s\n", stats->config->name);

    for (int i = 0; i < stats->num_levels; i++)
    {
        printf("Level %d: %d SSTables, %zu bytes, %" PRIu64 " keys\n",
            i + 1, stats->level_num_sstables[i], stats->level_sizes[i],
            stats->level_key_counts[i]);
    }

    /* B+tree stats (only populated if use_btree=1) */
    if (stats->use_btree)
    {
        printf("B+tree Total Nodes: %" PRIu64 "\n", stats->btree_total_nodes);
        printf("B+tree Max Height: %u\n", stats->btree_max_height);
        printf("B+tree Avg Height: %.2f\n", stats->btree_avg_height);
    }

    /* Tombstone density observability */
    printf("Total Tombstones: %" PRIu64 "\n", stats->total_tombstones);
    printf("Tombstone Ratio: %.2f%%\n", stats->tombstone_ratio * 100.0);
    printf("Worst SSTable Density: %.2f%% at level %d\n",
        stats->max_sst_density * 100.0, stats->max_sst_density_level +
1);
    for (int i = 0; i < stats->num_levels; i++)
    {
        printf("Level %d tombstones: %" PRIu64 "\n", i + 1, stats->level_tombstone_counts[i]);
    }
}

```

```

    }

    /* Write amplification counters (lifetime since open, on-disk framed
    bytes) */
    printf("WAL Bytes Written: %" PRIu64 "\n", stats->wal_bytes_written);
    printf("Flush Bytes Written: %" PRIu64 "\n", stats->flush_bytes_written);
    printf("Compaction Bytes Written: %" PRIu64 "\n", stats->compaction_bytes_written);
    printf("Compaction Bytes Read: %" PRIu64 "\n", stats->compaction_bytes_read);
    printf("User Bytes Written: %" PRIu64 "\n", stats->user_bytes_written);
    printf("Flushes: %" PRIu64 ", Compaction outputs: %" PRIu64 "\n",
           stats->flush_count, stats->compaction_count);
    if (stats->user_bytes_written > 0)
    {
        double wa = (double)(stats->wal_bytes_written + stats->flush_bytes_written +
                               stats->compaction_bytes_written) /
                    (double)stats->user_bytes_written;
        printf("Write Amplification: %.2f\n", wa);
    }

    /* Access configuration */
    printf("Write Buffer Size: %zu\n", stats->config->write_buffer_size);
    printf("Compression: %d\n", stats->config->compression_algorithm);
    printf("Bloom Filter: %s\n", stats->config->enable_bloom_filter ?
           "enabled" : "disabled");

    tidesdb_free_stats(stats);
}

```

Statistics include

Field	Type	Description
<code>num_levels</code>	<code>int</code>	Number of LSM levels
<code>memtable_size</code>	<code>size_t</code>	Current memtable size in bytes
<code>level_sizes</code>	<code>size_t*</code>	Array of per-level total sizes
<code>level_num_stables</code>	<code>int*</code>	Array of per-level SSTable counts
<code>level_key_counts</code>	<code>uint64_t*</code>	Array of per-level key counts
<code>config</code>	<code>tidesdb_column_family_config_t*</code>	Full column family configuration. This includes column family name if you need it!
<code>total_keys</code>	<code>uint64_t</code>	Total keys across memtable and all SSTables
<code>total_data_size</code>	<code>uint64_t</code>	Total data size (klog + vlog) in bytes
<code>avg_key_size</code>	<code>double</code>	Estimated average key size in bytes
<code>avg_value_size</code>	<code>double</code>	Estimated average value size in bytes
<code>read_amp</code>	<code>double</code>	Read amplification factor (point lookup cost)
<code>hit_rate</code>	<code>double</code>	Block cache hit rate (0.0 to 1.0)
<code>use_btree</code>	<code>int</code>	Whether column family uses B+tree format
<code>btree_total_nodes</code>	<code>uint64_t</code>	Total B+tree nodes across all SSTables
<code>btree_max_height</code>	<code>uint32_t</code>	Maximum tree height across all SSTables
<code>btree_avg_height</code>	<code>double</code>	Average tree height across all SSTables
<code>total_tombstones</code>	<code>uint64_t</code>	Total tombstones across all SSTables
<code>tombstone_ratio</code>	<code>double</code>	Database-wide tombstone count divided by entry count (0.0 to 1.0)

Field	Type	Description
<code>level_tombstone_counts</code>	<code>uint64_t*</code>	Per-level tombstone counts
<code>max_sst_density</code>	<code>double</code>	Worst single-SSTable tombstone density seen
<code>max_sst_density_level</code>	<code>int</code>	Level index where the worst SSTable lives
<code>wal_bytes_written</code>	<code>uint64_t</code>	Framed bytes appended to this CF's WAL. Stays 0 in unified mode, where the shared WAL volume is reported db-wide as <code>uwal_bytes_written</code>
<code>flush_bytes_written</code>	<code>uint64_t</code>	On-disk bytes (klog + vlog, including framing, index and bloom blobs) this CF's flushes wrote to L0
<code>compaction_bytes_written</code>	<code>uint64_t</code>	On-disk bytes this CF's compactions wrote
<code>compaction_bytes_read</code>	<code>uint64_t</code>	On-disk bytes this CF's compactions read as input. <code>compaction_bytes_read - compaction_bytes_written</code> is the space reclaimed by merges
<code>user_bytes_written</code>	<code>uint64_t</code>	Logical key + value bytes committed to this CF, the write-amplification denominator. Counted on commit and re-established on WAL replay
<code>flush_count</code>	<code>uint64_t</code>	Flushed L0 SSTables produced by this CF
<code>compaction_count</code>	<code>uint64_t</code>	Compaction output SSTables produced by this CF (a single triggered compaction can produce several)

Write Amplification

The write-amplification counters are lifetime totals since the database was opened, measured in on-disk framed bytes. Compute this CF's write amplification as $(\text{wal_bytes_written} + \text{flush_bytes_written} + \text{compaction_bytes_written}) / \text{user_bytes_written}$. For windowed analysis, snapshot the counters twice and subtract. They are not persisted, so they reset to zero on reopen (WAL replay re-establishes `user_bytes_written` for data that was not yet flushed).

B+tree Statistics

The B+tree stats (`btree_total_nodes`, `btree_max_height`, `btree_avg_height`) are only populated when `use_btree=1` in the column family configuration. These provide insight into the index structure overhead and lookup depth.

Estimating Distinct Key Cardinality

`tidesdb_cf_estimate_cardinality` estimates the number of distinct keys in a column family without scanning it. Every SSTable records, at write time, how many unique keys it holds — tallied for free during the sorted-run write, so counting it costs nothing at query time. The estimate sums those per-SSTable counts and adds the in-memory memtable entries.

The result is an upper bound. A key that lives in several SSTables, or in both memory and on disk, is counted once per place it lives, because reconciling keys across SSTables would require the merge the call is designed to avoid. This makes it well suited to optimizer selectivity and row-count estimates rather than an exact count. An SSTable written before the distinct-key summary existed falls back to its entry count, so mixed-vintage databases still produce a usable number.

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

uint64_t estimate = 0;
if (tidesdb_cf_estimate_cardinality(cf, &estimate) == TDB_SUCCESS)
{
    printf("estimated distinct keys: %llu\n", (unsigned long
long)estimate);
}
```

Signature:

```
int tidesdb_cf_estimate_cardinality(tidesdb_column_family_t *cf,
                                   uint64_t *out_estimate);
```

Returns `TDB_SUCCESS` and writes the estimate to `out_estimate`, or `TDB_ERR_INVALID_ARGS` if `cf` or `out_estimate` is `NULL`.

Database-Level Statistics

Get aggregate statistics across the entire database instance.

```
tidesdb_db_stats_t db_stats;
if (tidesdb_get_db_stats(db, &db_stats) == 0)
{
    printf("Column families: %d\n", db_stats.num_column_families);
    printf("Total memory: %" PRIu64 " bytes\n", db_stats.total_memory);
    printf("Resolved memory limit: %zu bytes\n",
db_stats.resolved_memory_limit);
    printf("Memory pressure level: %d\n", db_stats.memory_pressure_level);
    printf("Global sequence: %" PRIu64 "\n", db_stats.global_seq);
    printf("Flush queue: %zu pending\n", db_stats.flush_queue_size);
    printf("Compaction queue: %zu pending\n",
db_stats.compaction_queue_size);
    printf("Total SSTables: %d\n", db_stats.total_sstable_count);
    printf("Total data size: %" PRIu64 " bytes\n",
db_stats.total_data_size_bytes);
    printf("Open SSTable handles: %d\n", db_stats.num_open_sstables);
    printf("In-flight txn memory: %" PRId64 " bytes\n",
db_stats.txn_memory_bytes);
    printf("Immutable memtables: %d\n", db_stats.total_immutable_count);
    printf("Memtable bytes: %" PRId64 "\n", db_stats.total_memtable_bytes);

    /* Write amplification (uwal is db-scoped, the rest are summed across
CFs) */
    printf("Unified WAL bytes: %" PRIu64 "\n",
db_stats.uwal_bytes_written);
    printf("Per-CF WAL bytes: %" PRIu64 "\n", db_stats.wal_bytes_written);
    printf("Flush bytes: %" PRIu64 "\n", db_stats.flush_bytes_written);
    printf("Compaction bytes written: %" PRIu64 "\n",
db_stats.compaction_bytes_written);
    printf("Compaction bytes read: %" PRIu64 "\n",
db_stats.compaction_bytes_read);
    printf("User bytes written: %" PRIu64 "\n",
db_stats.user_bytes_written);
}
```

Database statistics include

Field	Type	Description
<code>num_column_families</code>	<code>int</code>	Number of column families
<code>total_memory</code>	<code>uint64_t</code>	System total memory
<code>available_memory</code>	<code>uint64_t</code>	System available memory at open time
<code>resolved_memory_limit</code>	<code>size_t</code>	Resolved memory limit (auto or configured)
<code>memory_pressure_level</code>	<code>int</code>	Current memory pressure (0=normal, 1=elevated, 2=high, 3=critical)
<code>flush_pending_count</code>	<code>int</code>	Number of pending flush operations (queued + in-flight)
<code>total_memtable_bytes</code>	<code>int64_t</code>	Total bytes in active plus unflushed immutable memtables across all CFs and the unified memtable. Active-only readings require pinning <code>cf->active_memtable</code> directly via <code>tidesdb_get_stats</code> and reading the per-CF <code>memtable_size</code> , which itself folds in the per-CF immutable queue
<code>total_immutable_count</code>	<code>int</code>	Total per-CF immutable memtables across all CFs
<code>total_sstable_count</code>	<code>int</code>	Total SSTables across all CFs and levels
<code>total_data_size_bytes</code>	<code>uint64_t</code>	Total data size (klog + vlog) across all CFs
<code>num_open_sstables</code>	<code>int</code>	Number of currently open SSTable file handles
<code>global_seq</code>	<code>uint64_t</code>	Current global sequence number

Field	Type	Description
<code>txn_memo ry_bytes</code>	<code>int64 _t</code>	Bytes held by in-flight transactions
<code>compacti on_queue _size</code>	<code>size_ t</code>	Number of pending compaction tasks
<code>flush_qu eue_size</code>	<code>size_ t</code>	Number of pending flush tasks in queue
<code>unified_ memtable _enabled</code>	<code>int</code>	1 if the database is in unified memtable mode
<code>unified_ memtable _bytes</code>	<code>int64 _t</code>	Bytes in the unified active memtable (unified mode only)
<code>unified_ immutable _count</code>	<code>int</code>	Number of unified immutable memtables awaiting flush
<code>unified_ is_flush ing</code>	<code>int</code>	1 if a unified flush is currently in progress
<code>unified_ next_cf_ index</code>	<code>uint3 2_t</code>	Next CF prefix index to assign in unified mode
<code>unified_ wal_gene ration</code>	<code>uint6 4_t</code>	Current generation number of the unified WAL
<code>object_s tore_ena bled</code>	<code>int</code>	1 if an object store connector is attached
<code>object_s tore_con nector</code>	<code>const char*</code>	Name of the object store connector (“fs”, “s3”, “unknown”, or NULL when no store is attached)
<code>local_ca che_byte s_used</code>	<code>size_ t</code>	Bytes currently used by the local SSTable cache (object store mode)

Field	Type	Description
local_cache_bytes_max	size_t	Local cache capacity in bytes (object store mode)
local_cache_num_files	int	Number of SSTable files resident in the local cache
last_uploaded_generation	uint64_t	Generation of the most recently uploaded WAL (object store mode)
upload_queue_depth	size_t	Pending upload work items in the object store upload queue
total_uploads	uint64_t	Cumulative successful uploads since open
total_upload_failures	uint64_t	Cumulative upload failures since open
replica_mode	int	1 if the database is in read-only replica mode
primary_epoch	uint64_t	Single-writer fencing: the lease epoch this primary currently holds (0 when not a primary / no lease). Advances on a successful promotion
seen_epoch	uint64_t	Single-writer fencing: the highest lease epoch this replica has observed in the store
uwal_bytes_written	uint64_t	Framed bytes appended to the shared unified WAL. 0 when unified mode is off
wal_bytes_written	uint64_t	Per-CF WAL bytes summed across all column families. 0 in unified mode
flush_bytes_written	uint64_t	Flush output bytes summed across all column families
compaction_bytes_written	uint64_t	Compaction output bytes summed across all column families

Field	Type	Description
<code>compaction_bytes_read</code>	<code>uint64_t</code>	Compaction input bytes summed across all column families
<code>user_bytes_written</code>	<code>uint64_t</code>	Logical committed bytes summed across all column families
<code>flush_count</code>	<code>uint64_t</code>	Flushed SSTables summed across all column families
<code>compaction_count</code>	<code>uint64_t</code>	Compaction output SSTables summed across all column families

Database-Wide Write Amplification

`uwal_bytes_written` is db-scoped because the unified WAL is shared; the remaining write-amplification fields are folded across every column family. Total WAL volume is `uwal_bytes_written + wal_bytes_written` (one term is always 0 depending on mode), and db-wide write amplification is `(uwal_bytes_written + wal_bytes_written + flush_bytes_written + compaction_bytes_written) / user_bytes_written`.

Stack Allocated

Unlike `tidesdb_get_stats` (which heap-allocates), `tidesdb_get_db_stats` fills a caller-provided struct on the stack. No free is needed.

Block Cache Statistics

Get statistics for the global block cache (shared across all column families).

```

tidesdb_cache_stats_t cache_stats;
if (tidesdb_get_cache_stats(db, &cache_stats) == 0)
{
    if (cache_stats.enabled)
    {
        printf("Cache enabled: yes\n");
        printf("Total entries: %zu\n", cache_stats.total_entries);
        printf("Total bytes: %.2f MB\n", cache_stats.total_bytes / (1024.0
* 1024.0));
        printf("Hits: %" PRIu64 "\n", cache_stats.hits);
        printf("Misses: %" PRIu64 "\n", cache_stats.misses);
        printf("Hit rate: %.1f%%\n", cache_stats.hit_rate * 100.0);
        printf("Partitions: %zu\n", cache_stats.num_partitions);
    }
    else
    {
        printf("Cache enabled: no (block_cache_size = 0)\n");
    }
}

```

Cache statistics include

- `enabled` · Whether block cache is active (0 if `block_cache_size` was set to 0)
- `total_entries` · Number of cached blocks
- `total_bytes` · Total memory used by cached blocks
- `hits` · Number of cache hits (blocks served from memory)
- `misses` · Number of cache misses (blocks read from disk)
- `hit_rate` · Hit rate as a decimal (0.0 to 1.0)
- `num_partitions` · Number of cache partitions (scales with CPU cores)

Block Cache

The block cache is a database-level resource shared across all column families. It caches raw klog block bytes (decompressed but not deserialized) to avoid repeated disk I/O. On cache hit, the block is deserialized on the fly. This yields much smaller cache entries compared to caching deserialized blocks, dramatically improving hit rates for a given cache size. Configure cache size via `config.block_cache_size` when opening the database. Set to 0 to disable caching.

Read Statistics

Database-wide read counters track where reads are served from and how much work each read does. They are useful for tuning bloom filters, block indexes, and cache size.

Compile-time opt-in

The read-profiling API and the `tidesdb_read_stats_t` struct are gated behind the `TDB_ENABLE_READ_PROFILING` build macro. Build with `-DTDB_ENABLE_READ_PROFILING` to enable them. Without that flag the counters are not collected (the per-read instrumentation compiles to nothing) and these three functions are not declared, so leave this section out of the picture in a default build.

```
int tidesdb_get_read_stats(tidesdb_t *db, tidesdb_read_stats_t
*stats);
void tidesdb_print_read_stats(tidesdb_t *db);
void tidesdb_reset_read_stats(tidesdb_t *db);
```

`tidesdb_get_read_stats` copies the current counters into a caller-provided `tidesdb_read_stats_t`. `tidesdb_print_read_stats` writes a formatted summary to the log. `tidesdb_reset_read_stats` zeroes the counters, which is handy before measuring a specific workload.

`tidesdb_read_stats_t` fields

Field	Description
<code>total_reads</code>	Total point reads served
<code>memtable_hits</code>	Reads satisfied by the active memtable
<code>immutable_hits</code>	Reads satisfied by an immutable memtable
<code>sstable_hits</code>	Reads satisfied by an SSTable
<code>levels_searched</code>	Total levels visited across all reads
<code>sstables_checked</code>	Total SSTables probed across all reads
<code>bloom_checks</code>	Bloom filter lookups performed
<code>bloom_hits</code>	Bloom lookups that passed (key may be present)
<code>blocks_read</code>	klog blocks read
<code>cache_block_hits</code>	Block reads served from the block cache
<code>cache_block_misses</code>	Block reads that missed the cache and hit disk
<code>disk_reads</code>	Block reads that went to disk

```

tidesdb_read_stats_t rs;
tidesdb_get_read_stats(db, &rs);
printf("reads=%llu bloom %llu/%llu cache %llu/%llu\n",
      (unsigned long long)rs.total_reads,
      (unsigned long long)rs.bloom_hits, (unsigned long
long)rs.bloom_checks,
      (unsigned long long)rs.cache_block_hits, (unsigned long
long)rs.cache_block_misses);

```

Range Cost Estimation

`tidesdb_range_cost` estimates the computational cost of iterating between two keys in a column family. The returned value is an opaque double - meaningful only for comparison with other values from the same function. It uses only in-memory metadata and performs no disk I/O.

```

int tidesdb_range_cost(tidesdb_column_family_t *cf,
                      const uint8_t *key_a, size_t key_a_size,
                      const uint8_t *key_b, size_t key_b_size,
                      double *cost);

```


Parameters

Name	Type	Description
<code>cf</code>	<code>tidesdb_column_family_t*</code>	Column family to estimate cost for
<code>key_a</code>	<code>const uint8_t*</code>	First key (bound of range)
<code>key_a_size</code>	<code>size_t</code>	Size of first key
<code>key_b</code>	<code>const uint8_t*</code>	Second key (bound of range)
<code>key_b_size</code>	<code>size_t</code>	Size of second key
<code>cost</code>	<code>double*</code>	Output: estimated traversal cost (higher = more expensive)

Returns

- `TDB_SUCCESS` on success
- `TDB_ERR_INVALID_ARGS` on bad input (NULL pointers, zero-length keys)

Example

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

double cost_a = 0.0, cost_b = 0.0;

tidesdb_range_cost(cf, (uint8_t *)"user:0000", 9,
                  (uint8_t *)"user:0999", 9, &cost_a);

tidesdb_range_cost(cf, (uint8_t *)"user:1000", 9,
                  (uint8_t *)"user:1099", 9, &cost_b);

if (cost_a < cost_b)
{
    printf("Range A is cheaper to iterate\n");
}
```

How it works

The function walks all SSTable levels and uses in-memory metadata to estimate how many blocks and entries fall within the given key range:

- With block indexes enabled · Uses $O(\log B)$ binary search per overlapping SSTable to find the block slots containing each key bound. The block span between slots, scaled by `index_sample_ratio`, gives the estimated block count.
- Without block indexes · Falls back to byte-level key interpolation. The leading 8 bytes of each key are converted to a numeric position within the SSTable's min/max key range to estimate the fraction of blocks covered.
- B+tree SSTables (`use_btree=1`) · Uses the same key interpolation against tree node counts, plus tree height as a seek cost. Only applies to column families configured with B+tree klog format.
- Compression · Compressed SSTables receive a $1.5\times$ weight multiplier to account for decompression overhead.
- Merge overhead · Each overlapping SSTable adds a small fixed cost for merge-heap operations.
- Memtable · The active memtable's entry count contributes a small in-memory cost.

Key order does not matter - the function normalizes the range so `key_a > key_b` produces the same result as `key_b > key_a`.

Use cases

- Query planning · Compare candidate key ranges to find the cheapest one to scan
- Load balancing · Distribute range scan work across threads by estimating per-range cost
- Adaptive prefetching · Decide how aggressively to prefetch based on range size
- Monitoring · Track how data distribution changes across key ranges over time

Cost Values

The returned cost is not an absolute measure (it does not represent milliseconds, bytes, or entry counts). It is a relative scalar - only meaningful when compared with other `tidesdb_range_cost` results. A cost of 0.0 means no overlapping SSTables or memtable entries were found for the range.

Compression Algorithms

TidesDB supports multiple compression algorithms to reduce storage footprint and I/O bandwidth. Compression is applied to both klog (key-log) and vlog (value-log) blocks before writing to disk.

Available Algorithms

- `TDB_COMPRESS_NONE` · No compression (value: 0)

- Raw data written directly to disk
- **Use case** · Pre-compressed data, maximum write throughput, CPU-constrained environments
- `TDB_COMPRESS_LZ4` · LZ4 standard compression (value: 2; the **effective default** on a standard build — see note below)
 - Fast compression and decompression with good compression ratios
 - **Use case** · General purpose, balanced performance and compression
 - **Performance** · ~500 MB/s compression, ~2000 MB/s decompression (typical)
- `TDB_COMPRESS_LZ4_FAST` · LZ4 fast mode (value: 4)
 - Faster compression than standard LZ4 with slightly lower compression ratio
 - Uses acceleration factor of 2
 - **Use case** · Write-heavy workloads prioritizing speed over compression ratio
 - **Performance** · Higher compression throughput than standard LZ4
- `TDB_COMPRESS_ZSTD` · Zstandard compression (value: 3)
 - Best compression ratio with moderate speed (compression level 1)
 - **Use case** · Storage-constrained environments, archival data, read-heavy workloads
 - **Performance** · ~400 MB/s compression, ~1000 MB/s decompression (typical)
- `TDB_COMPRESS_SNAPPY` · Snappy compression (value: 1)
 - Fast compression with moderate compression ratios
 - **Availability** · Not available on SunOS/Illumos/OmniOS platforms
 - **Use case** · Legacy compatibility, platforms where Snappy is preferred

Default resolution

`tidesdb_default_column_family_config()` leaves `compression_algorithm` as `TDB_COMPRESS_NONE`; at open the engine resolves an unset/none value to the best *available* backend — preferring LZ4, then ZSTD, then Snappy, and falling back to no compression if none was built in. So LZ4 is the effective default on a standard build, but the resolved choice depends on which libraries were compiled in.

Configuration Example

```

tidesdb_column_family_config_t cf_config =
tidesdb_default_column_family_config();

/* Use LZ4 explicitly (also the resolved default on a standard build) */
cf_config.compression_algorithm = TDB_COMPRESS_LZ4;

/* Use Zstandard for better compression ratio */
cf_config.compression_algorithm = TDB_COMPRESS_ZSTD;

/* Use LZ4 fast mode for maximum write throughput */
cf_config.compression_algorithm = TDB_COMPRESS_LZ4_FAST;

/* Disable compression */
cf_config.compression_algorithm = TDB_COMPRESS_NONE;

tidesdb_create_column_family(db, "my_cf", &cf_config);

```

Important

Compression algorithm can be changed at runtime via `tidesdb_cf_update_runtime_config`, but the change only affects **new** SSTables. Existing SSTables retain their original compression and are decompressed correctly during reads. Compression is applied at the block level (both klog and vlog blocks).

- Decompression happens automatically during reads
- Block cache stores **raw bytes** (decompressed but not deserialized) to maximize cache capacity and hit rates
- Different column families can use different compression algorithms

B+tree KLog Format (Optional)

Column families can optionally use a B+tree structure for the key log instead of the default block-based format. The B+tree format offers faster point lookups through $O(\log N)$ tree traversal rather than linear block scanning.

Enabling B+tree format

```
tidesdb_column_family_config_t cf_config =
tidesdb_default_column_family_config();
cf_config.use_btree = 1; /* Enable B+tree format */

tidesdb_create_column_family(db, "btree_cf", &cf_config);
```

Characteristics

- Point lookups · $O(\log N)$ tree traversal with binary search at each node, compared to potentially scanning multiple 64KB blocks in block-based format
- Range scans · Doubly-linked leaf nodes enable efficient bidirectional iteration
- Immutable · Tree is bulk-loaded from sorted memtable data during flush and never modified afterward
- Compression · Nodes compress independently using the same algorithms (LZ4, LZ4-FAST, Zstd, Snappy)
- Large values · Values meeting or exceeding `klog_value_threshold` are stored in vlog, same as block-based format
- Bloom filter · Works identically — checked before tree traversal to skip lookups for absent keys

When to use B+tree format

- Read-heavy workloads with frequent point lookups
- Workloads where read latency is more important than write throughput
- Large SSTables where block scanning becomes expensive

Tradeoffs

- Slightly higher write amplification during flush (building tree structure)
- Larger metadata overhead per node compared to block-based format
- Block-based format may be faster for sequential scans of entire SSTables

Important

`use_btree` **cannot be changed** after column family creation. Different column families can use different formats (some B+tree, some block-based). Both formats support the same compression algorithms and bloom filters.

Choosing a Compression Algorithm

Workload	Recommended Algorithm	Rationale
General purpose	TDB_COMPRESS_LZ4	Best balance of speed and compression
Write-heavy	TDB_COMPRESS_LZ4_FAST	Minimize CPU overhead on writes
Storage-constrained	TDB_COMPRESS_ZSTD	Maximum compression ratio
Read-heavy	TDB_COMPRESS_ZSTD	Reduce I/O bandwidth, decompression is fast
Pre-compressed data	TDB_COMPRESS_NONE	Avoid double compression overhead
CPU-constrained	TDB_COMPRESS_NONE or TDB_COMPRESS_LZ4_FAST	Minimize CPU usage

Updating Column Family Configuration

Update runtime-safe configuration settings. Configuration changes are applied to new operations only.

```

tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_column_family_config_t new_config =
tidesdb_default_column_family_config();
new_config.write_buffer_size = 256 * 1024 * 1024;
new_config.skip_list_max_level = 16;
new_config.skip_list_probability = 0.25f;
new_config.bloom_fpr = 0.001;          /* 0.1% false positive rate */
new_config.index_sample_ratio = 8; /* sample 1 in 8 blocks */

int persist_to_disk = 1;          /* save to config.ini */
if (tidesdb_cf_update_runtime_config(cf, &new_config, persist_to_disk) ==
0)
{
    printf("Configuration updated successfully\n");
}

```

Updatable settings (all applied by `tidesdb_cf_update_runtime_config`):

- `write_buffer_size` · Memtable flush threshold
- `skip_list_max_level` · Skip list level for **new** memtables

- `skip_list_probability` · Skip list probability for **new** memtables
- `bloom_fpr` · False positive rate for **new** SSTables
- `enable_bloom_filter` · Enable/disable bloom filters for **new** SSTables
- `enable_block_indexes` · Enable/disable block indexes for **new** SSTables
- `block_index_prefix_len` · Block index prefix length for **new** SSTables
- `index_sample_ratio` · Index sampling ratio for **new** SSTables
- `compression_algorithm` · Compression for **new** SSTables (existing SSTables retain their original compression)
- `klog_value_threshold` · Value log threshold for **new** writes
- `sync_mode` · Durability mode (TDB_SYNC_NONE, TDB_SYNC_INTERVAL, or TDB_SYNC_FULL). Also updates the active WAL's sync mode immediately.
- `sync_interval_us` · Sync interval in microseconds (only used when sync_mode is TDB_SYNC_INTERVAL)
- `level_size_ratio` · LSM level sizing (DCA recalculates capacities dynamically)
- `min_levels` · Minimum LSM levels
- `dividing_level_offset` · Compaction dividing level offset
- `l1_file_count_trigger` · L1 file count compaction trigger
- `l0_queue_stall_threshold` · Backpressure stall threshold
- `default_isolation_level` · Default transaction isolation level
- `min_disk_space` · Minimum disk space required
- `commit_hook_fn` / `commit_hook_ctx` · Commit hook callback and context

Non-updatable settings (not modified by this function):

- `comparator_name` · Cannot change sort order after creation (would corrupt key ordering in existing SSTables)
- `use_btree` · Cannot change klog format after creation (existing SSTables use the original format)

Backpressure Defaults

The default `l0_queue_stall_threshold` is 10. The default `l1_file_count_trigger` is 4. Both align with the Spooky paper's stop and trigger thresholds.

Configuration persistence

If `persist_to_disk = 1`, changes are saved to `config.ini` in the column family directory. On restart, the configuration is loaded from this file.

```

/* Save configuration to custom INI file */
tidesdb_cf_config_save_to_ini("custom_config.ini", "my_cf", &new_config);

/* Load configuration from INI file */
tidesdb_column_family_config_t loaded_config;
if (tidesdb_cf_config_load_from_ini("custom_config.ini", "my_cf",
&loaded_config) == 0)
{
    printf("Configuration loaded successfully\n");
}

```

Important Notes

Changes apply immediately to new operations. Existing SSTables and memtables retain their original settings. The update operation is thread-safe.

Commit Hook (Change Data Capture)

`tidesdb_cf_set_commit_hook` registers an optional callback that fires synchronously after every transaction commit on a column family. The hook receives the full batch of committed operations atomically, enabling real-time change data capture without WAL parsing or external log consumers.

```

int tidesdb_cf_set_commit_hook(tidesdb_column_family_t *cf,
                              tidesdb_commit_hook_fn fn,
                              void *ctx);

```

Parameters

Name	Type	Description
<code>cf</code>	<code>tidesdb_column_family_t*</code>	Column family handle
<code>fn</code>	<code>tidesdb_commit_hook_fn</code>	Commit hook callback (or <code>NULL</code> to disable)
<code>ctx</code>	<code>void*</code>	User-provided context passed to the callback

Returns

- `TDB_SUCCESS` on success
- `TDB_ERR_INVALID_ARGS` if `cf` is `NULL`

Callback signature

```
typedef int (*tidesdb_commit_hook_fn)(const tidesdb_commit_op_t *ops,  
int num_ops,  
uint64_t commit_seq, void *ctx);
```

Parameter	Type	Description
<code>ops</code>	<code>const tidesdb_commit_op_t*</code>	Array of committed operations (valid only during callback)
<code>num_ops</code>	<code>int</code>	Number of operations in the array
<code>commit_seq</code>	<code>uint64_t</code>	Monotonic commit sequence number
<code>ctx</code>	<code>void*</code>	User-provided context pointer

The callback returns `0` on success. A non-zero return is logged as a warning but does not roll back the commit.

Operation struct

```
typedef struct tidesdb_commit_op_t  
{  
    const uint8_t *key;  
    size_t key_size;  
    const uint8_t *value;    /* NULL for deletes */  
    size_t value_size;      /* 0 for deletes */  
    time_t ttl;  
    int is_delete;          /* 1 for delete, 0 for put */  
} tidesdb_commit_op_t;
```

Example (replication sink)

```

typedef struct
{
    int socket_fd;
    uint8_t *send_buf;
    size_t buf_size;
} replication_ctx_t;

static int replication_hook(const tidesdb_commit_op_t *ops, int num_ops,
                           uint64_t commit_seq, void *ctx)
{
    replication_ctx_t *rctx = (replication_ctx_t *)ctx;
    for (int i = 0; i < num_ops; i++)
    {
        /* Serialize and send each op to replica */
        send_to_replica(rctx->socket_fd, commit_seq, &ops[i]);
    }
    return 0;
}

/* Attach hook at runtime */
replication_ctx_t rctx = { .socket_fd = replica_fd };
tidesdb_cf_set_commit_hook(cf, replication_hook, &rctx);

/* Normal writes now trigger the hook automatically */
tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);
tidesdb_txn_put(txn, cf, key, key_size, value, value_size, -1);
tidesdb_txn_commit(txn); /* replication_hook fires here */
tidesdb_txn_free(txn);

/* Detach hook */
tidesdb_cf_set_commit_hook(cf, NULL, NULL);

```

Setting hook via config at creation time

```

tidesdb_column_family_config_t cf_config =
tidesdb_default_column_family_config();
cf_config.commit_hook_fn = replication_hook;
cf_config.commit_hook_ctx = &rctx;

tidesdb_create_column_family(db, "replicated_cf", &cf_config);

```

Behavior

- The hook fires after WAL write, memtable apply, and commit status marking are complete - the data is fully durable before the callback runs
- Hook failure (non-zero return) is logged but does not affect the commit result
- Each column family has its own independent hook; a multi-CF transaction fires the hook once per CF with only that CF's operations
- `commit_seq` is monotonically increasing across commits and can be used as a replication cursor
- Pointers in `tidesdb_commit_op_t` are valid only during the callback invocation - copy any data you need to retain
- The hook executes synchronously on the committing thread; keep the callback fast to avoid stalling writers
- Setting the hook to `NULL` disables it immediately with no restart required

Use cases

- Replication · Ship committed batches to replicas in commit order
- Event streaming · Publish mutations to Kafka, NATS, or any message broker
- Secondary indexing · Maintain a reverse index or materialized view
- Audit logging · Record every mutation with key, value, TTL, and sequence number
- Debugging · Attach a temporary hook in production to inspect live writes

Runtime-Only

The `commit_hook_fn` and `commit_hook_ctx` fields are not persisted to `config.ini`. After a database restart, hooks must be re-registered by the application. This is by design - function pointers cannot be serialized.

Transactions

All operations in TidesDB are done through transactions for ACID guarantees per column family.

Basic Transaction

```
/* Get column family pointer first */
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;
if (tidesdb_txn_begin(db, &txn) != 0)
{
    return -1;
}

const uint8_t *key = (uint8_t *)"mykey";
const uint8_t *value = (uint8_t *)"myvalue";

if (tidesdb_txn_put(txn, cf, key, 5, value, 7, -1) != 0)
{
    tidesdb_txn_free(txn);
    return -1;
}

if (tidesdb_txn_commit(txn) != 0)
{
    tidesdb_txn_free(txn);
    return -1;
}

tidesdb_txn_free(txn);
```

With TTL (Time-to-Live)

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

const uint8_t *key = (uint8_t *)"temp_key";
const uint8_t *value = (uint8_t *)"temp_value";

/* TTL is Unix timestamp (seconds since epoch) -- absolute expiration time
*/
time_t ttl = time(NULL) + 60; /* Expires 60 seconds from now */

/* Use -1 for no expiration */
tidesdb_txn_put(txn, cf, key, 8, value, 10, ttl);
tidesdb_txn_commit(txn);
tidesdb_txn_free(txn);
```

TTL Examples

```
time_t ttl = -1;

time_t ttl = time(NULL) + (5 * 60);

time_t ttl = time(NULL) + (60 * 60);

time_t ttl = 1730592000;
```

Getting a Key-Value Pair

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

const uint8_t *key = (uint8_t *)"mykey";
uint8_t *value = NULL;
size_t value_size = 0;

if (tidesdb_txn_get(txn, cf, key, 5, &value, &value_size) == 0)
{
    /* Use value */
    printf("Value: %.*s\n", (int)value_size, value);
    free(value);
}

tidesdb_txn_free(txn);
```

Non-Tracking Reads and Existence Checks

`tidesdb_txn_get` records every point read into the transaction's read set so serializable validation and the write-write reservation can detect conflicts against the version actually read. That tracking is exactly wrong for an existence probe — a primary-key uniqueness check before an insert, for instance — because the probe read would pollute the writing transaction's conflict footprint and can cause it to abort against a key it only looked at.

`tidesdb_txn_get_notrack` resolves the value at the transaction's snapshot just like `tidesdb_txn_get`, but does not record the read. It does not participate in read-write conflict detection and does not seed the write-write reservation base. Uniqueness stays enforced by the subsequent write's own reservation, so dropping the read tracking is safe for that use.

`tidesdb_txn_contains` is an existence check built on the same non-tracking read. It never materializes the value to the caller — there is nothing to free — and returns `TDB_SUCCESS` when the key exists, `TDB_ERR_NOT_FOUND` when it does not.

```

tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

const uint8_t *key = (uint8_t *)"user:1000";

/* existence check without tracking the read */
if (tidesdb_txn_contains(txn, cf, key, 9) == TDB_SUCCESS)
{
    /* key already present -- reject the insert */
}

/* or read the value without adding it to the conflict footprint */
uint8_t *value = NULL;
size_t value_size = 0;
if (tidesdb_txn_get_notrack(txn, cf, key, 9, &value, &value_size) ==
TDB_SUCCESS)
{
    free(value);
}

tidesdb_txn_free(txn);

```

Signatures:

```

int tidesdb_txn_get_notrack(tidesdb_txn_t *txn,
                           tidesdb_column_family_t *cf,
                           const uint8_t *key,
                           size_t key_size,
                           uint8_t **value,
                           size_t *value_size);

int tidesdb_txn_contains(tidesdb_txn_t *txn,
                        tidesdb_column_family_t *cf,
                        const uint8_t *key,
                        size_t key_size);

```

`tidesdb_txn_get_notrack` returns `TDB_SUCCESS` and sets `value/value_size` (the caller frees `value`) on a hit, or a negative error code otherwise. `tidesdb_txn_contains` returns `TDB_SUCCESS` if the key exists, `TDB_ERR_NOT_FOUND` if it does not, or a negative error code on failure.

Deleting a Key-Value Pair

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

const uint8_t *key = (uint8_t *)"mykey";
tidesdb_txn_delete(txn, cf, key, 5);

tidesdb_txn_commit(txn);
tidesdb_txn_free(txn);
```

Single-Delete

`tidesdb_txn_single_delete` writes a tombstone with the same read semantics as `tidesdb_txn_delete`, but carries a caller-provided promise that lets compaction drop the put and the tombstone together as soon as both appear in the same merge input, rather than carrying the tombstone forward until it reaches the largest active level.

Between any two single-deletes on the same key, and between the start of the key's history and its first single-delete, the key has been put **at most once**. The engine does not and cannot verify this at runtime; violating the contract can leave older puts visible after the single-delete and is a bug in the caller.

This is the right choice for workloads that insert each key exactly once and then delete it exactly once (classic insert-benchmark patterns, secondary-index entries on columns that are never updated, log-style tables with scheduled purges). It is **not** safe for tables that issue repeated updates to the same key.


```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

const uint8_t *key = (uint8_t *)"mykey";
tidesdb_txn_single_delete(txn, cf, key, 5);

tidesdb_txn_commit(txn);
tidesdb_txn_free(txn);
```

Signature:

```
int tidesdb_txn_single_delete(tidesdb_txn_t *txn,
                             tidesdb_column_family_t *cf,
                             const uint8_t *key,
                             size_t key_size);
```

Returns `TDB_SUCCESS` on success or a negative error code on failure. When in doubt, prefer `tidesdb_txn_delete`.

Multi-Operation Transaction

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

tidesdb_txn_put(txn, cf, (uint8_t *)"key1", 4, (uint8_t *)"value1", 6, -1);
tidesdb_txn_put(txn, cf, (uint8_t *)"key2", 4, (uint8_t *)"value2", 6, -1);
tidesdb_txn_delete(txn, cf, (uint8_t *)"old_key", 7);

/* Commit atomically -- all or nothing */
if (tidesdb_txn_commit(txn) != 0)
{
    tidesdb_txn_free(txn);
    return -1;
}

tidesdb_txn_free(txn);
```

Transaction Rollback

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

tidesdb_txn_put(txn, cf, (uint8_t *)"key", 3, (uint8_t *)"value", 5, -1);

tidesdb_txn_rollback(txn);
tidesdb_txn_free(txn);
```

Transaction Reset

`tidesdb_txn_reset` resets a committed or aborted transaction for reuse with a new isolation level. This avoids the overhead of freeing and reallocating transaction resources in hot loops.

```
int tidesdb_txn_reset(tidesdb_txn_t *txn, tidesdb_isolation_level_t isolation);
```

Parameters

Name	Type	Description
<code>txn</code>	<code>tidesdb_txn_t*</code>	Transaction handle (must be committed or aborted)
<code>isolation</code>	<code>tidesdb_isolation_level_t</code>	New isolation level for the reset transaction

Returns

- `TDB_SUCCESS` on success
- `TDB_ERR_INVALID_ARGS` if `txn` is `NULL`, still active (not committed/aborted), or isolation level is invalid

Example

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

tidesdb_txn_put(txn, cf, (uint8_t *)"key1", 4, (uint8_t *)"value1", 6, -1);
tidesdb_txn_commit(txn);

tidesdb_txn_reset(txn, TDB_ISOLATION_READ_COMMITTED);

tidesdb_txn_put(txn, cf, (uint8_t *)"key2", 4, (uint8_t *)"value2", 6, -1);
tidesdb_txn_commit(txn);

tidesdb_txn_free(txn);
```

Behavior

- The transaction must be committed or aborted before reset; resetting an active transaction returns `TDB_ERR_INVALID_ARGS`
- Internal buffers (ops array, read set arrays, arena pointer array, column family array, savepoints array) are retained to avoid reallocation

- Per-operation key/value data, arena buffers, hash tables, and savepoint children are freed
- A fresh `txn_id` and `snapshot_seq` are assigned based on the new isolation level
- The isolation level can be changed on each reset (e.g., `READ_COMMITTED` -> `REPEATABLE_READ`)
- If switching to an isolation level that requires read tracking (`REPEATABLE_READ` or `SERIALIZABLE`), read set arrays are allocated automatically
- `SERIALIZABLE` transactions are correctly unregistered from and re-registered to the active transaction list

When to use

- Batch processing · Reuse a single transaction across many commit cycles in a loop
- Connection pooling · Reset a transaction for a new request without reallocation
- High-throughput ingestion · Reduce malloc/free overhead in tight write loops



Reset vs Free + Begin

For a single transaction, `tidesdb_txn_reset` is functionally equivalent to calling `tidesdb_txn_free` followed by `tidesdb_txn_begin_with_isolation`. The difference is performance, reset retains allocated buffers and avoids repeated allocation overhead. This matters most in loops that commit and restart thousands of transactions.

Savepoints

Savepoints allow partial rollback within a transaction. You can create named savepoints and rollback to them without aborting the entire transaction.

```

tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

tidesdb_txn_put(txn, cf, (uint8_t *)"key1", 4, (uint8_t *)"value1", 6, -1);

if (tidesdb_txn_savepoint(txn, "sp1") != 0)
{
    tidesdb_txn_rollback(txn);
    tidesdb_txn_free(txn);
    return -1;
}

tidesdb_txn_put(txn, cf, (uint8_t *)"key2", 4, (uint8_t *)"value2", 6, -1);

if (tidesdb_txn_rollback_to_savepoint(txn, "sp1") != 0)
{
    tidesdb_txn_rollback(txn);
    tidesdb_txn_free(txn);
    return -1;
}

tidesdb_txn_put(txn, cf, (uint8_t *)"key3", 4, (uint8_t *)"value3", 6, -1);

if (tidesdb_txn_commit(txn) != 0)
{
    tidesdb_txn_free(txn);
    return -1;
}

tidesdb_txn_free(txn);

```

Savepoint API

- `tidesdb_txn_savepoint(txn, "name")` · Create a savepoint
- `tidesdb_txn_rollback_to_savepoint(txn, "name")` · Rollback to savepoint
- `tidesdb_txn_release_savepoint(txn, "name")` · Release savepoint without rolling back

Savepoint behavior

- Savepoints capture the transaction state at a specific point

- Rolling back to a savepoint discards all operations after that savepoint
- Releasing a savepoint frees its resources without rolling back
- Multiple savepoints can be created with different names
- Creating a savepoint with an existing name updates that savepoint
- Savepoints are automatically freed when the transaction commits or rolls back
- Returns `TDB_ERR_NOT_FOUND` if the savepoint name doesn't exist

Multi-Column-Family Transactions

TidesDB supports atomic transactions across multiple column families with true all-or-nothing semantics.

```
tidesdb_column_family_t *users_cf = tidesdb_get_column_family(db,
"users");
tidesdb_column_family_t *orders_cf = tidesdb_get_column_family(db,
"orders");
if (!users_cf || !orders_cf) return -1;

tidesdb_txn_t *txn = NULL;
if (tidesdb_txn_begin(db, &txn) != 0)
{
    return -1;
}

tidesdb_txn_put(txn, users_cf, (uint8_t *)"user:1000", 9,
                (uint8_t *)"John Doe", 8, -1);

tidesdb_txn_put(txn, orders_cf, (uint8_t *)"order:5000", 10,
                (uint8_t *)"user:1000|product:A", 19, -1);

if (tidesdb_txn_commit(txn) != 0)
{
    tidesdb_txn_free(txn);
    return -1;
}

tidesdb_txn_free(txn);
```

Multi-CF guarantees

- Either all CFs commit or none do (atomic)
- Automatically detected when operations span multiple CFs

- Uses global sequence numbers for atomic ordering
- Each CF's WAL receives operations with the same commit sequence number
- No two-phase commit or coordinator overhead

Isolation Levels

TidesDB supports five MVCC isolation levels for fine-grained concurrency control.

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;

tidesdb_txn_begin_with_isolation(db, TDB_ISOLATION_READ_UNCOMMITTED, &txn);
tidesdb_txn_begin_with_isolation(db, TDB_ISOLATION_READ_COMMITTED, &txn);
tidesdb_txn_begin_with_isolation(db, TDB_ISOLATION_REPEATABLE_READ, &txn);
tidesdb_txn_begin_with_isolation(db, TDB_ISOLATION_SNAPSHOT, &txn);
tidesdb_txn_begin_with_isolation(db, TDB_ISOLATION_SERIALIZABLE, &txn);

tidesdb_txn_put(txn, cf, (uint8_t *)"key", 3, (uint8_t *)"value", 5, -1);

int result = tidesdb_txn_commit(txn);
if (result == TDB_ERR_CONFLICT)
{
    tidesdb_txn_free(txn);
    return -1;
}

tidesdb_txn_free(txn);
```

Isolation level characteristics

- READ UNCOMMITTED · Maximum concurrency, minimal consistency
- READ COMMITTED · Balanced for OLTP workloads (default)
- REPEATABLE READ · Strong point read consistency
- SNAPSHOT · First-committer-wins with write-write conflict detection only (allows write skew)
- SERIALIZABLE · Strongest guarantees with full SSI (read-write + write-write + dangerous structure detection), higher abort rates

Iterators

Iterators provide efficient forward and backward traversal over key-value pairs.

Memory Ownership

The key and value pointers returned by `tidesdb_iter_key()` and `tidesdb_iter_value()` are **internal pointers owned by the iterator**. Do **NOT** free them. They remain valid until the next `tidesdb_iter_next()`, `tidesdb_iter_prev()`, seek operation, or `tidesdb_iter_free()` call. If you need to retain the data beyond the current iteration step, copy it to your own buffer.

Forward Iteration

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

tidesdb_iter_t *iter = NULL;
if (tidesdb_iter_new(txn, cf, &iter) != 0)
{
    tidesdb_txn_free(txn);
    return -1;
}

tidesdb_iter_seek_to_first(iter);

while (tidesdb_iter_valid(iter))
{
    uint8_t *key = NULL;
    size_t key_size = 0;
    uint8_t *value = NULL;
    size_t value_size = 0;

    if (tidesdb_iter_key(iter, &key, &key_size) == 0 &&
        tidesdb_iter_value(iter, &value, &value_size) == 0)
    {
        printf("Key: %.*s, Value: %.*s\n",
               (int)key_size, key, (int)value_size, value);
    }

    tidesdb_iter_next(iter);
}

tidesdb_iter_free(iter);
tidesdb_txn_free(txn);
```

Backward Iteration

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

tidesdb_iter_t *iter = NULL;
tidesdb_iter_new(txn, cf, &iter);

tidesdb_iter_seek_to_last(iter);

while (tidesdb_iter_valid(iter))
{
    tidesdb_iter_prev(iter);
}

tidesdb_iter_free(iter);
tidesdb_txn_free(txn);
```

Iterator Seek Operations

TidesDB provides seek operations that allow you to position an iterator at a specific key or key range without scanning from the beginning.

How Seek Works

With Block Indexes Enabled (`enable_block_indexes = 1`):

- Uses compact block index with parallel arrays (min/max key prefixes and file positions)
- Binary search through sampled keys at configurable ratio (default 1:1 via `index_sample_ratio`, meaning every block is indexed)
- Jumps directly to the target block using the file position
- Scans forward from that block to find the exact key
- **Performance** · $O(\log n)$ binary search + $O(k)$ entries per block scan

Block indexes provide dramatic speedup for large SSTables at the cost of ~2-5% storage overhead for the compact index structure (parallel arrays with delta-encoded file positions).

Seek to Specific Key

`tidesdb_iter_seek(iter, key, key_size)` · Positions iterator at the first key $\geq$ target key

```

tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

tidesdb_iter_t *iter = NULL;
tidesdb_iter_new(txn, cf, &iter);

/* Seek to specific key */
const char *target = "user:1000";
if (tidesdb_iter_seek(iter, (uint8_t *)target, strlen(target)) == 0)
{
    /* Iterator is now positioned at "user:1000" or the next key after it
    */
    if (tidesdb_iter_valid(iter))
    {
        uint8_t *key = NULL;
        size_t key_size = 0;
        tidesdb_iter_key(iter, &key, &key_size);
        printf("Found: %.*s\n", (int)key_size, key);
    }
}

tidesdb_iter_free(iter);
tidesdb_txn_free(txn);

```

`tidesdb_iter_seek_for_prev(iter, key, key_size)` · Positions iterator at the last key $\leq$ target key

```

/* Seek for reverse iteration */
const char *target = "user:2000";
if (tidesdb_iter_seek_for_prev(iter, (uint8_t *)target, strlen(target)) ==
0)
{
    /* Iterator is now positioned at "user:2000" or the previous key before
it */
    while (tidesdb_iter_valid(iter))
    {
        /* Iterate backwards from this point */
        tidesdb_iter_prev(iter);
    }
}

```

Prefix Seeking

Since `tidesdb_iter_seek` positions the iterator at the first key $\geq$ target, you can use a prefix as the seek target to efficiently scan all keys sharing that prefix:

```

/* Seek to prefix and iterate all matching keys */
const char *prefix = "user:";
if (tidesdb_iter_seek(iter, (uint8_t *)prefix, strlen(prefix) + 1) == 0)
{
    while (tidesdb_iter_valid(iter))
    {
        uint8_t *key = NULL;
        size_t key_size = 0;
        tidesdb_iter_key(iter, &key, &key_size);

        /* Stop when keys no longer match prefix */
        if (strncmp((char *)key, prefix, strlen(prefix)) != 0) break;

        /* Process key */
        printf("Found: %.*s\n", (int)key_size, key);

        if (tidesdb_iter_next(iter) != TDB_SUCCESS) break;
    }
}

```

This pattern works across both memtables and SSTables. When block indexes are enabled, the seek operation uses binary search to jump directly to the relevant block, making prefix scans efficient even on large datasets.

Reading Key and Value Together

`tidesdb_iter_key_value` returns the current entry's key and value in a single call, instead of calling `tidesdb_iter_key` and `tidesdb_iter_value` separately.

```
int tidesdb_iter_key_value(tidesdb_iter_t *iter, uint8_t **key, size_t
*key_size,
                        uint8_t **value, size_t *value_size);
```

The returned key and value pointers are internal pointers owned by the iterator, with the same lifetime rules as `tidesdb_iter_key` and `tidesdb_iter_value`: do not free them, and copy them out if you need to retain the data past the next iterator step.

```
uint8_t *key = NULL, *value = NULL;
size_t key_size = 0, value_size = 0;
while (tidesdb_iter_valid(iter))
{
    if (tidesdb_iter_key_value(iter, &key, &key_size, &value, &value_size)
== TDB_SUCCESS)
        printf("%.s = %.s\n", (int)key_size, key, (int)value_size,
value);
    if (tidesdb_iter_next(iter) != TDB_SUCCESS) break;
}
```

Custom Comparators

TidesDB uses comparators to determine the sort order of keys throughout the entire system, memtables, SSTables, block indexes, and iterators all use the same comparison logic. Once a comparator is set for a column family, it **cannot be changed** without corrupting data.

Built-in Comparators

TidesDB provides six built-in comparators that are automatically registered on database open:

`"memcmp"` (default) · Binary byte-by-byte comparison

- Compares `min(key1_size, key2_size)` bytes using `memcmp()`
- If bytes are equal, shorter key sorts first
- **Use case** · Binary keys, raw byte data, general purpose

"lexicographic" · Null-terminated string comparison

- Uses `strcmp()` for lexicographic ordering
- Ignores `key_size` parameters (assumes null-terminated)
- **Use case** · C strings, text keys
- **Warning** · Keys must be null-terminated or behavior is undefined

"uint64" · Unsigned 64-bit integer comparison

- Interprets 8-byte keys as `uint64_t` values
- Falls back to `memcmp` if `key_size != 8`
- **Use case** · Numeric IDs, timestamps, counters
- **Example** ·

```
uint64_t id = 1000; tidesdb_txn_put(txn, cf, (uint8_t*)&id, 8, ...)
```

"int64" · Signed 64-bit integer comparison

- Interprets 8-byte keys as `int64_t` values
- Falls back to `memcmp` if `key_size != 8`
- **Use case** · Signed numeric keys, relative timestamps
- **Example** ·

```
int64_t offset = -500; tidesdb_txn_put(txn, cf, (uint8_t*)&offset, 8, ...)
```

"reverse" · Reverse binary comparison

- Negates the result of `memcmp` comparator
- Sorts keys in descending order
- **Use case** · Reverse chronological order, descending IDs

"case_insensitive" · Case-insensitive ASCII comparison

- Converts A-Z to a-z during comparison
- Compares `min(key1_size, key2_size)` bytes
- If bytes are equal (ignoring case), shorter key sorts first
- **Use case** · Case-insensitive text keys, usernames, email addresses

Each built-in is reachable both by the registered name above and as a public function, in case you want to register it under a different name or call it directly. Every one has the standard comparator signature

```
int cmp(const uint8_t *key1, size_t key1_size, const uint8_t *key2, size_t
```

key2_size, void *ctx):

Name	Function
"memcmp"	tidesdb_comparator_memcmp
"lexicographic"	tidesdb_comparator_lexicographic
"uint64"	tidesdb_comparator_uint64
"int64"	tidesdb_comparator_int64
"reverse"	tidesdb_comparator_reverse_memcmp
"case_insensitive"	tidesdb_comparator_case_insensitive

Custom Comparator Registration

```
/* Define your comparison function */
int my_timestamp_compare(const uint8_t *key1, size_t key1_size,
                        const uint8_t *key2, size_t key2_size, void *ctx)
{
    (void)ctx; /* unused */

    if (key1_size != 8 || key2_size != 8)
    {
        /* fallback for invalid sizes */
        return memcmp(key1, key2, key1_size < key2_size ? key1_size :
key2_size);
    }

    uint64_t ts1, ts2;
    memcpy(&ts1, key1, 8);
    memcpy(&ts2, key2, 8);

    /* reverse order for newest-first */
    if (ts1 > ts2) return -1;
    if (ts1 < ts2) return 1;
    return 0;
}

/* Register before creating column families */
tidesdb_register_comparator(db, "timestamp_desc", my_timestamp_compare,
NULL, NULL);

/* Use in column family */
tidesdb_column_family_config_t cf_config =
tidesdb_default_column_family_config();
strncpy(cf_config.comparator_name, "timestamp_desc",
TDB_MAX_COMPARATOR_NAME - 1);
cf_config.comparator_name[TDB_MAX_COMPARATOR_NAME - 1] = '\0';
tidesdb_create_column_family(db, "events", &cf_config);
```

The signature is `tidesdb_register_comparator(db, name, fn, ctx_str, ctx)`. The last two arguments are for parameterized comparators that need configuration. `ctx` is an in-memory context pointer passed to the comparator on every call. `ctx_str` is an optional string form of that context; setting it (or the matching `comparator_ctx_str` field on the column family config) persists the context so the comparator can be re-created with the same parameters after a restart. Pass `NULL` for both when the comparator is stateless, as in the example above.

Retrieving a Registered Comparator

Use `tidesdb_get_comparator` to retrieve a previously registered comparator by name:

```
tidesdb_comparator_fn fn = NULL;
void *ctx = NULL;

if (tidesdb_get_comparator(db, "timestamp_desc", &fn, &ctx) == 0)
{
    /* Comparator found - fn and ctx are now populated */
    printf("Comparator 'timestamp_desc' is registered\n");
}
else
{
    /* Comparator not found */
    printf("Comparator not registered\n");
}
```

Use cases

- Validation · Check if a comparator is registered before creating a column family
- Debugging · Verify comparator registration during development
- Dynamic configuration · Query available comparators at runtime

Comparator function signature

```
int (*comparator_fn)(const uint8_t *key1, size_t key1_size,
                    const uint8_t *key2, size_t key2_size,
                    void *ctx);
```

Return values

- `< 0` if `key1 < key2`
- `0` if `key1 == key2`
- `> 0` if `key1 > key2`

Important

Comparators must be **registered before** creating column families that use them. Once set, a comparator **cannot be changed** for a column family. The same comparator is used across memtables, SSTables, block indexes, and iterators.

- Custom comparators can use the `ctx` parameter for runtime configuration

Sync Modes

Control durability vs performance tradeoff with three sync modes.

```
tidesdb_column_family_config_t cf_config =
tidesdb_default_column_family_config();

/* TDB_SYNC_NONE      -- Fastest, least durable (OS handles flushing) */
cf_config.sync_mode = TDB_SYNC_NONE;

/* TDB_SYNC_INTERVAL -- Balanced performance with periodic background
syncing */
cf_config.sync_mode = TDB_SYNC_INTERVAL;
cf_config.sync_interval_us = 128000; /* Sync every 128ms (default) */

/* TDB_SYNC_FULL      -- Most durable (fsync on every write) */
cf_config.sync_mode = TDB_SYNC_FULL;

tidesdb_create_column_family(db, "my_cf", &cf_config);
```

Sync Mode Options

- **TDB_SYNC_NONE** · No explicit sync, relies on OS page cache (fastest, least durable)
 - Best for · Maximum throughput, acceptable data loss on crash
 - Use case · Caches, temporary data, reproducible workloads
- **TDB_SYNC_INTERVAL** · Periodic background syncing at configurable intervals (balanced)
 - Best for · Production workloads requiring good performance with bounded data loss
 - Use case · Most applications, configurable durability window
 - Features:
 - Single background sync thread monitors all column families using interval mode
 - Configurable sync interval via `sync_interval_us` (microseconds)
 - Structural operations (flush, compaction, WAL rotation) always enforce durability
 - At most `sync_interval_us` worth of data at risk on crash
 - Mid-durability correctness - When memtables rotate, the WAL receives an escalated fsync before entering the flush queue. Sorted run creation and merge operations always propagate full sync to block managers regardless of sync mode.
- **TDB_SYNC_FULL** · Fsync on every write operation (slowest, most durable)
 - Best for · Critical data requiring maximum durability
 - Use case · Financial transactions, audit logs, critical metadata
 - Structural operations always use fsync regardless of sync mode :::

Sync Interval Examples

```
/* Sync every 100ms (good for low-latency requirements) */
cf_config.sync_mode = TDB_SYNC_INTERVAL;
cf_config.sync_interval_us = 100000;

/* Sync every 128ms (default) */
cf_config.sync_mode = TDB_SYNC_INTERVAL;
cf_config.sync_interval_us = 128000;

/* Sync every 1 second (higher throughput, more data at risk) */
cf_config.sync_mode = TDB_SYNC_INTERVAL;
cf_config.sync_interval_us = 1000000;
```

Manual WAL Sync

`tidesdb_sync_wal` forces an immediate `fdatasync` of the active write-ahead log for a column family. This is useful for explicit durability control when using `TDB_SYNC_NONE` or `TDB_SYNC_INTERVAL` modes.

```
int tidesdb_sync_wal(tidesdb_column_family_t *cf);
```

Parameters

Name	Type	Description
<code>cf</code>	<code>tidesdb_column_family_t*</code>	Column family handle

Returns

- `TDB_SUCCESS` on success
- `TDB_ERR_INVALID_ARGS` if `cf` is NULL or has no associated database
- `TDB_ERR_IO` if the fsync operation fails

Example

```

tidesdb_column_family_t *cf = tidesdb_get_column_family(db,
"my_cf");
if (!cf) return -1;

/* Force WAL durability after a batch of writes */
if (tidesdb_sync_wal(cf) != 0)
{
    fprintf(stderr, "WAL sync failed\n");
}

```

When to use

- Application-controlled durability · Sync the WAL at specific points (e.g., after a batch of related writes) when using `TDB_SYNC_NONE` or `TDB_SYNC_INTERVAL`
- Pre-checkpoint · Ensure all buffered WAL data is on disk before taking a checkpoint
- Graceful shutdown · Flush WAL buffers before closing the database
- Critical writes · Force durability for specific high-value writes without using `TDB_SYNC_FULL` for all writes

Behavior

- Acquires a reference to the active memtable to safely access its WAL
- Calls `fdatsync` on the WAL file descriptor
- Thread-safe - can be called concurrently from multiple threads
- If the memtable rotates during the call, retries with the new active memtable

Structural Operations

Regardless of sync mode, TidesDB **always** enforces durability for structural operations:

- Memtable flush to SSTable
- SSTable compaction and merging
- WAL rotation
- Column family metadata updates

This ensures the database structure remains consistent even if user data syncing is delayed.

Compaction

TidesDB performs automatic background compaction when L1 reaches the configured file count trigger (default: 4 SSTables). However, you can manually trigger compaction for specific scenarios.

Checking Flush/Compaction Status

Check if a column family currently has flush or compaction operations in progress.

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

/* Check if flushing is in progress */
if (tidesdb_is_flushing(cf))
{
    printf("Flush in progress\n");
}

/* Check if compaction is in progress */
if (tidesdb_is_compacting(cf))
{
    printf("Compaction in progress\n");
}
```

Use cases

- Graceful shutdown · Wait for background operations to complete before closing
- Maintenance windows · Check if operations are running before triggering manual compaction
- Monitoring · Track background operation status for observability
- Testing · Verify flush/compaction behavior in unit tests

Return values

- 1 · Operation is in progress
- 0 · No operation in progress (or invalid column family)

Manual Flush

Manually flush a column family's memtable to disk. This creates a new SSTable (sorted run) in level 1.

```

tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

/* Trigger flush manually */
if (tidesdb_flush_memtable(cf) != 0)
{
    fprintf(stderr, "Failed to trigger flush\n");
    return -1;
}

```

When to use manual flush

- Before backup · Ensure all in-memory data is persisted before taking a backup
- Memory pressure · Force data to disk when memory usage is high
- Testing · Verify SSTable creation and compaction behavior
- Graceful shutdown · Flush pending data before closing the database

Behavior

- Rotates the column family's active memtable and enqueues the rotated memtable for flush regardless of its current size (no write-buffer threshold gate)
- In unified-memtable mode the shared memtable is rotated through the unified flush path, so the call behaves the same whether the database is in per-CF or unified-memtable mode
- Returns immediately (non-blocking) — flush runs asynchronously in background threads
- If flush is already running for the column family, the call succeeds but doesn't queue duplicate work
- Thread-safe — can be called concurrently from multiple threads

Manual Compaction

```

tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

/* Trigger compaction manually */
if (tidesdb_compact(cf) != 0)
{
    fprintf(stderr, "Failed to trigger compaction\n");
    return -1;
}

```

When to use manual compaction

- After bulk deletes · Reclaim disk space by removing tombstones and obsolete versions
- After bulk updates · Consolidate multiple versions of keys into single entries
- Before read-heavy workloads · Optimize read performance by reducing the number of levels to search
- During maintenance windows · Proactively compact during low-traffic periods to avoid compaction during peak load
- After TTL expiration · Remove expired entries to reclaim storage
- Space optimization · Force compaction to reduce space amplification when storage is constrained

Behavior

- Enqueues compaction work in the global compaction thread pool
- **Blocks until the worker has finished servicing the request**, including any compaction already in flight on this column family. The call routes through an internal blocking entry point that attaches a per-call completion signal to the work item; the worker fires that signal on every exit path that consumes the item
- The request is never coalesced — even if another compaction is mid-flight, the caller's request runs as its own work item
- Compaction merges SSTables across levels, removes tombstones, expired TTL entries, and obsolete versions
- Thread-safe — can be called concurrently from multiple threads

Performance considerations

- Manual compaction uses the same thread pool as automatic background compaction
- Configure thread pool size via `config.num_compaction_threads` (default: 2)
- Compaction is I/O intensive — avoid triggering during peak write workloads
- Multiple column families can compact in parallel up to the thread pool limit
- Because the call blocks, application code that needs fire-and-forget semantics should run `tidesdb_compact` from a dedicated thread

See [How does TidesDB work?](#) for details on compaction algorithms, merge strategies, and parallel compaction.

Targeted Range Compaction

`tidesdb_compact_range` runs a synchronous compaction over a specific key range. Only SSTables whose minimum and maximum keys overlap the requested range participate in the merge, so the work and I/O are bounded to the affected portion of the LSM tree rather than the whole column family.

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

const uint8_t start[] = "tenant_42: ";
const uint8_t end[]   = "tenant_42: ";

if (tidesdb_compact_range(cf, start, sizeof(start) - 1, end, sizeof(end) -
1) != 0)
{
    fprintf(stderr, "Targeted compaction failed\n");
    return -1;
}
```

When to use

- Bulk reclaim after a large range delete, where waiting for natural compaction would leave tombstones and obsolete versions on disk
- Tenant eviction or sliding-window expiration that does not fit TTL semantics
- Post-import cleanup of a known key range loaded with `tidesdb_txn_put` followed by `tidesdb_txn_delete`
- Operational counterpart to the automatic tombstone density trigger when an operator wants reclaim now rather than at the next natural threshold crossing

Behavior

- Synchronous, blocks the caller until the merge commits or fails
- Does not enqueue work onto the compaction thread pool, the calling thread does the work
- Selects only SSTables whose key range overlaps the requested range using the column family's comparator, SSTables outside the range are not touched
- Applies the same emit-loop logic as background compactions (tombstone reclamation rules, single-delete pair cancellation, sequence-based deduplication, value recompression)
- Output SSTables are committed to the manifest atomically and old inputs are marked for deletion

Return values

- `TDB_SUCCESS` on success
- `TDB_ERR_INVALID_ARGS` if `cf` is NULL, if both `start_key` and `end_key` are NULL, or if a non-NULL key has size zero (a single NULL key is allowed and means an unbounded bound on that side)
- `TDB_ERR_LOCKED` if another compaction is already running on the column family
- Standard I/O and memory error codes if the merge cannot complete

Purge Column Family

`tidesdb_purge_cf` forces a synchronous flush and aggressive compaction for a single column family. Unlike `tidesdb_flush_memtable` (which is non-blocking and returns once flush work is enqueued), purge blocks until all flush and compaction I/O is complete; `tidesdb_compact` also blocks now but only on a single compaction round, whereas purge runs the full flush-wait-compact-drain pipeline.

```
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");
if (!cf) return -1;

if (tidesdb_purge_cf(cf) != 0)
{
    fprintf(stderr, "Purge failed\n");
    return -1;
}
/* All data is now flushed to SSTables and compacted */
```

Behavior

1. If unified memtable mode is enabled, rotates the unified memtable and waits for the flush to complete so that entries belonging to this CF are moved to SSTables
2. Waits for any in-progress flush to complete
3. Force-flushes the active per-CF memtable (even if below threshold)
4. Waits for flush I/O to fully complete
5. Waits for any in-progress compaction to complete
6. Triggers synchronous compaction inline (bypasses the compaction queue)
7. Waits for any queued compaction to drain

When to use

- Before backup or checkpoint · Ensure all data is on disk and compacted
- After bulk deletes · Reclaim space immediately by compacting away tombstones
- Manual maintenance · Force a clean state during a maintenance window

- Pre-shutdown · Ensure all pending work is complete before closing

Return values

- `TDB_SUCCESS` on success
- `TDB_ERR_INVALID_ARGS` if `cf` is NULL or has no associated database

Purge Database

`tidesdb_purge` forces a synchronous flush and aggressive compaction for **all** column families, then drains both the global flush and compaction queues.

```
if (tidesdb_purge(db) != 0)
{
    fprintf(stderr, "Database purge failed\n");
}
/* All CFs flushed and compacted, all queues drained */
```

Behavior

1. If unified memtable mode is enabled, rotates and flushes the unified memtable then waits for the flush to complete
2. Calls `tidesdb_purge_cf` on each column family
3. Drains the global flush queue (waits for queue size and pending count to reach 0)
4. Drains the global compaction queue (waits for queue size to reach 0)

Return values

- `TDB_SUCCESS` if all column families purged successfully
- First non-zero error code if any CF fails (continues processing remaining CFs)
- `TDB_ERR_INVALID_ARGS` if `db` is NULL

Purge vs Manual Flush + Compact

`tidesdb_flush_memtable` is non-blocking — it enqueues flush work and returns immediately. `tidesdb_compact`, `tidesdb_purge_cf`, and `tidesdb_purge` all block until the work they request has been serviced. Use `purge` when you need the additional flush + retry pipeline plus compaction in one call; use plain `tidesdb_compact` when you only need to wait for a compaction.

Cancelling Background Work (Fast Shutdown)

`tidesdb_cancel_background_work` cancels background compaction database-wide so that `tidesdb_close` does not have to wait for a large compaction backlog to drain. In-flight merges bail at their next checkpoint and queued compaction work is skipped. Flushes are left untouched, so durability is preserved.

```
/* Fast shutdown: abandon pending compaction, then close immediately
*/
if (tidesdb_cancel_background_work(db) != 0)
{
    fprintf(stderr, "Failed to cancel background work\n");
}

if (tidesdb_close(db) != 0)
{
    fprintf(stderr, "Failed to close database\n");
}
```

Behavior

- Signals every in-flight merge to bail at its next checkpoint; uncommitted output is discarded and the input SSTables are left intact, so the on-disk state stays recovery-safe
- Skips any compaction work still sitting in the queue
- Leaves flushes unaffected — already-flushed data and durability are preserved
- Blocks (bounded) until compaction has gone idle before returning
- The cancellation is **sticky** for the current database session and is reset on the next `tidesdb_open`

When to use

- Fast shutdown · Call immediately before `tidesdb_close` when a large compaction backlog would otherwise make close take seconds to minutes
- Bounded close latency · Cap how long teardown can block in latency-sensitive shutdown paths

Return values

- `TDB_SUCCESS` on success
- `TDB_ERR_INVALID_ARGS` if `db` is NULL

Caution

This trades compaction progress for shutdown speed: read amplification may be higher on the next open because the abandoned compactions did not reduce the SSTable count. The skipped work is picked up again by normal background compaction after the next `tidesdb_open`. Use `tidesdb_purge` instead when you want pending work *completed* before close rather than abandoned.

Thread Pools

TidesDB uses separate thread pools for flush and compaction operations. Understanding the parallelism model is important for optimal configuration.

Parallelism semantics

- Cross-CF parallelism · Multiple flush/compaction workers CAN process different column families in parallel
- Intra-CF flush parallelism · A single column family can have multiple memtables flushing concurrently up to the global `max_concurrent_flushes` cap, so a hot CF drains its immutable queue at the speed of the worker pool rather than serially
- Within-CF compaction serialization · Compaction is still one-per-CF (enforced by the atomic `is_compacting` flag) so level invariants and manifest ordering stay consistent
- Global flush semaphore · `max_concurrent_flushes` is pinned 1:1 to `num_flush_threads` at open. A cap above the pool size would be meaningless because the pool size is the hard upper bound, a cap below the pool size would leave workers idle, so the open path normalizes any mismatch to `num_flush_threads` and emits a `TDB_LOG_WARN`

Thread pool sizing guidance

- Tune `num_flush_threads` to the parallelism you want for in-flight flushes; `max_concurrent_flushes` will follow it
- Single column family with sustained write pressure · Increase `num_flush_threads` so the immutable queue drains faster, since multiple flushes per CF are allowed
- Multiple column families · Set flush thread count up to roughly the number of concurrently hot CFs, and compaction thread count up to the number of column families for maximum cross-CF parallelism

Configuration

```

tidesdb_config_t config = {
    .db_path = "./mydb",
    .num_flush_threads = 4,                /* Flush thread pool size
(default: 0 = auto, min(cpu_count, 4)) */
    .num_compaction_threads = 4,          /* Compaction thread pool size
(default: 2) */
    .max_concurrent_flushes = 0,          /* 0 = auto-match
num_flush_threads (recommended) */
    .log_level = TDB_LOG_INFO,
    .block_cache_size = 64 * 1024 * 1024, /* 64MB global block cache
(default: 64MB) */
    .max_open_sstables = 256,             /* LRU cache for SSTable objects
(default: 256, 64 on OpenBSD; each has 2 FDs) */
};

tidesdb_t *db = NULL;
tidesdb_open(&config, &db);

```

Note

`max_open_sstables` is a **storage-engine-level** configuration, not a column family configuration. It controls the LRU cache size for SSTable structures. Each SSTable uses 2 file descriptors (klog + vlog), so 256 SSTables = 512 file descriptors.

See [How does TidesDB work?](#) for details on thread pool architecture and work distribution.

Unified Memtable

By default, each column family maintains its own skip list and WAL. A transaction touching N column families performs N WAL writes. Unified memtable mode replaces all per-CF skip lists and WALs with a single shared skip list and a single WAL at the database level, reducing N WAL writes per transaction to exactly one.

Enabling Unified Memtable

```
tidesdb_config_t config = tidesdb_default_config();
config.db_path = "./mydb";
config.unified_memtable = 1; /* Enable unified memtable mode */

tidesdb_t *db = NULL;
if (tidesdb_open(&config, &db) != 0)
{
    return -1;
}
```

With custom configuration

```
tidesdb_config_t config = tidesdb_default_config();
config.db_path = "./mydb";
config.unified_memtable = 1;
config.unified_memtable_write_buffer_size = 128 * 1024 * 1024; /* 128MB
unified write buffer */
config.unified_memtable_skip_list_max_level = 16; /* Higher
max level for larger datasets */
config.unified_memtable_skip_list_probability = 0.25f; /* Default
probability */
config.unified_memtable_sync_mode = TDB_SYNC_FULL; /* Fsync on
every WAL write */

tidesdb_t *db = NULL;
if (tidesdb_open(&config, &db) != 0)
{
    return -1;
}
```

Configuration Fields

Field	Type	Default	Description
<code>unified_memtable</code>	<code>int</code>	<code>0</code>	Enable unified memtable mode (0 = per-CF, 1 = unified)
<code>unified_memtable_write_buffer_size</code>	<code>size_t</code>	<code>0</code> (auto)	Write buffer size for unified memtable. 0 = <code>TDB_DEFAULT_WRITE_BUFFER_SIZE</code> (64MB)
<code>unified_memtable_skip_list_max_level</code>	<code>int</code>	<code>0</code> (auto)	Skip list max level. 0 = default 12
<code>unified_memtable_skip_list_probability</code>	<code>float</code>	<code>0</code> (auto)	Skip list probability. 0 = default 0.25
<code>unified_memtable_sync_mode</code>	<code>int</code>	<code>0</code>	WAL sync mode (<code>TDB_SYNC_NONE</code> , <code>TDB_SYNC_FULL</code> , <code>TDB_SYNC_INTERVAL</code>)
<code>unified_memtable_sync_interval_us</code>	<code>uint64_t</code>	<code>0</code>	Sync interval in microseconds (only for <code>TDB_SYNC_INTERVAL</code>)

How It Works

All column families write into a single shared skip list and a single WAL file. Keys in the unified skip list are prefixed with a 4-byte big-endian column family index to maintain isolation between column families.

Write path · At transaction commit, all operations across all column families are serialized into a single WAL batch and written as one block. Operations are then applied to the unified skip list with prefixed keys. A transaction touching 5 column families results in 1 WAL write instead of 5.

Read path · Reads construct the prefixed key (4-byte CF index + user key) and search the unified active skip list, then unified immutable memtables (newest to oldest), then fall back to per-CF SSTables on disk. Keys from other column families are never visible.

Flush · When the unified memtable exceeds the write buffer, a flush worker iterates it in sorted order, demuxing entries by CF prefix into per-CF SSTables. The on-disk format is identical to per-CF mode - SSTables contain user keys without the prefix.

Usage

All existing transaction, iterator, and column family APIs work identically in unified mode. No application code changes are needed beyond setting `unified_memtable = 1`:


```

/* Normal transaction -- works identically in both modes */
tidesdb_column_family_t *users = tidesdb_get_column_family(db, "users");
tidesdb_column_family_t *orders = tidesdb_get_column_family(db, "orders");

tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

/* In unified mode, both ops write to the same skip list + same WAL (1 WAL
write total) */
/* In per-CF mode, each op writes to its own skip list + own WAL (2 WAL
writes total) */
tidesdb_txn_put(txn, users, (uint8_t *)"user:1", 6, (uint8_t *)"Alice", 5,
-1);
tidesdb_txn_put(txn, orders, (uint8_t *)"order:1", 7, (uint8_t *)"item_A",
6, -1);

tidesdb_txn_commit(txn);
tidesdb_txn_free(txn);

```

When to Use Unified Memtable

- **Many column families per transaction** · Workloads where each transaction touches multiple CFs (e.g., a table with secondary indexes where each index is a separate CF)
- **Write-heavy multi-CF workloads** · Reduces WAL I/O from N writes to 1 per transaction
- **Plugin/binding scenarios** · Storage engine plugins (e.g., MariaDB) that map SQL tables to column families with automatic secondary index CFs

Constraints

- All column families sharing the unified memtable must use the default `memcmp` comparator; a custom comparator is rejected at CF creation with `TDB_ERR_INVALID_ARGS`, because the shared structure has a single sort order
- The unified skip list always uses `memcmp` as its comparator for the prefixed keys
- Per-CF backpressure still applies - each CF's L0 queue depth and L1 file count are checked independently at commit time

SSTables Are Per-CF

Even in unified mode, SSTables on disk remain per-column-family. The unified memtable only affects the in-memory write path and WAL. Flush, compaction, reads from disk, iterators over SSTables, bloom filters, and block indexes all work identically to per-CF mode.

See [How does TidesDB work?](#) for the full design rationale, flush demuxing details, and WAL format.

Object Store Mode

Object store mode allows TidesDB to store SSTables in a remote object store (S3, MinIO, GCS, or any S3-compatible service) while using local disk as a cache. This separates compute from storage and enables cold start recovery from the remote store. Object store mode requires unified memtable mode and is automatically enforced when a connector is set.

Enabling Object Store Mode (Filesystem Connector)

```
#include <tidesdb/tidesdb.h>

/* create a filesystem connector (for testing and local replication) */
tidesdb_objstore_t *store = tidesdb_objstore_fs_create("/mnt/nfs/tidesdb-objects");

tidesdb_objstore_config_t os_cfg = tidesdb_objstore_default_config();

tidesdb_config_t config = tidesdb_default_config();
config.db_path = "./mydb";
config.object_store = store;
config.object_store_config = &os_cfg;

tidesdb_t *db = NULL;
if (tidesdb_open(&config, &db) != 0)
{
    return -1;
}

/* use the database normally -- SSTables are uploaded after flush */

tidesdb_close(db);
```

Enabling Object Store Mode (S3/MinIO Connector)

Build with `-DTIDESDB_WITH_S3=ON` to enable the S3 connector. This requires libcurl and OpenSSL.

```
#include <tidesdb/tidesdb.h>

tidesdb_objstore_t *s3 = tidesdb_objstore_s3_create(
    "s3.amazonaws.com",           /* endpoint */
    "my-tidesdb-bucket",          /* bucket */
    "production/db1/",            /* key prefix (or NULL) */
    "AKIAIOSFODNN7EXAMPLE",       /* access key */
    "wJalrXUtnFEMI/K7MDENG/bPxrFcYEXAMPLEKEY", /* secret key */
    "us-east-1",                  /* region */
    1,                             /* use_ssl (HTTPS) */
    0                               /* use_path_style (0 for
AWS, 1 for MinIO) */
);

tidesdb_objstore_config_t os_cfg = tidesdb_objstore_default_config();
os_cfg.local_cache_max_bytes = 512 * 1024 * 1024; /* 512MB local cache */
os_cfg.max_concurrent_uploads = 8;

tidesdb_config_t config = tidesdb_default_config();
config.db_path = "./mydb";
config.object_store = s3;
config.object_store_config = &os_cfg;

tidesdb_t *db = NULL;
tidesdb_open(&config, &db);
```

MinIO Example

```
tidesdb_objstore_t *minio = tidesdb_objstore_s3_create(
    "localhost:9000", /* MinIO endpoint */
    "tidesdb-bucket", /* bucket */
    NULL,             /* no key prefix */
    "minioadmin",     /* access key */
    "minioadmin",     /* secret key */
    NULL,             /* region (NULL for MinIO) */
    0,                /* no SSL for local dev */
    1                 /* path-style URLs (required for MinIO) */
);
```

Advanced S3 Configuration

`tidesdb_objstore_s3_create` is a thin wrapper with default settings. For options it cannot express (a key prefix, custom TLS, or multipart tuning), build a `tidesdb_objstore_s3_config_t`, zero-initialize it, set the fields you need, and pass it to `tidesdb_objstore_s3_create_config`.

```
tidesdb_objstore_t *tidesdb_objstore_s3_create_config(const
tidesdb_objstore_s3_config_t *config);
```

`tidesdb_objstore_s3_config_t` fields

Field	Description
<code>endpoint</code>	S3 endpoint host and port
<code>bucket</code>	Bucket name
<code>prefix</code>	Optional key prefix prepended to every object key (<code>NULL</code> for none)
<code>access_key</code> / <code>secret_key</code>	Credentials
<code>region</code>	AWS region (<code>NULL</code> for MinIO and compatible servers)
<code>use_ssl</code>	1 for HTTPS, 0 for plain HTTP
<code>use_path_style</code>	1 for path-style URLs (MinIO), 0 for virtual-hosted (AWS)
<code>tls_ca_path</code>	Path to a CA bundle for TLS verification (<code>NULL</code> for system default)
<code>tls_insecure_skip_verify</code>	1 to skip TLS certificate verification (testing only)
<code>multipart_threshold</code>	File size at or above which multipart upload is used (0 uses the connector default of 64MB)
<code>multipart_part_size</code>	Size of each multipart part (0 uses the connector default of 8MB)

```

tidesdb_objstore_s3_config_t s3 = {0};
s3.endpoint = "s3.amazonaws.com";
s3.bucket = "my-bucket";
s3.prefix = "tenant-42/";
s3.access_key = "AKIA...";
s3.secret_key = "wJal...";
s3.region = "us-east-1";
s3.use_ssl = 1;
s3.multipart_part_size = 16 * 1024 * 1024; /* 16MB parts */

tidesdb_objstore_t *store = tidesdb_objstore_s3_create_config(&s3);

```

Object Store Configuration

Use `tidesdb_objstore_default_config()` for sensible defaults, then override fields as needed.

Field	Type	Default	Description
<code>local_cache_path</code>	<code>const char *</code>	<code>NULL</code> (uses <code>db_path</code>)	Local directory for cached SSTable files
<code>local_cache_maximum_bytes</code>	<code>size_t</code>	<code>0</code> (unlimited)	Maximum local cache size in bytes
<code>cache_on_read</code>	<code>int</code>	<code>1</code>	Cache downloaded files locally
<code>cache_on_write</code>	<code>int</code>	<code>1</code>	Keep local copy after upload
<code>max_current_uploads</code>	<code>int</code>	<code>4</code>	Number of parallel upload threads
<code>max_current_downloads</code>	<code>int</code>	<code>8</code>	Number of parallel download threads
<code>multipart_threshold</code>	<code>size_t</code>	<code>67108864</code> (64MB)	Reserved; not currently wired on this DB-level config. The S3 connector uploads files at or above a fixed 64MB threshold (<code>TDB_S3_MULTIPART_THRESHOLD</code>) as a multipart upload; the per-connector <code>tidesdb_objstore_s3_config_t.multipart_threshold</code> can override it
<code>multipart_part_size</code>	<code>size_t</code>	<code>8388608</code> (8MB)	Reserved; not currently wired on this DB-level config. The S3 connector streams multipart uploads in a fixed 8MB part size (<code>TDB_S3_MULTIPART_PART_SIZE</code>); the per-connector <code>tidesdb_objstore_s3_config_t.multipart_part_size</code> can override it
<code>sync_manifest_to_object</code>	<code>int</code>	<code>1</code>	Upload MANIFEST after each compaction
<code>replicate_wal</code>	<code>int</code>	<code>1</code>	Upload closed WAL segments for replication

Field	Type	Default	Description
<code>wal_upload_sync</code>	<code>int</code>	<code>0</code>	0 for background WAL upload, 1 to block flush
<code>wal_sync_c_threshold_bytes</code>	<code>size_t</code>	<code>1048576</code> (1MB)	Sync active WAL to object store when it grows by this many bytes since last sync (0 to disable)
<code>wal_sync_on_commit</code>	<code>int</code>	<code>0</code>	Upload WAL after every txn commit for RPO=0 replication
<code>replica_mode</code>	<code>int</code>	<code>0</code>	Enable read-only replica mode (writes return TDB_ERR_READONLY)
<code>replica_sync_interval_us</code>	<code>uint64_t</code>	<code>5000000</code> (5s)	MANIFEST poll interval for replica sync in microseconds
<code>replica_replay_wal</code>	<code>int</code>	<code>1</code>	Replay WAL from object store for near-real-time reads on replicas

Per-CF Object Store Tuning

Column family configurations include object store tuning fields. These are persisted to config.ini and survive restarts.

Field	Type	Default	Description
<code>object_target_file_size</code>	<code>size_t</code>	<code>0</code>	Reserved for API compatibility. Not used; output SSTable sizing during partitioned merge is derived automatically from level geometry (<code>file_max = C_X</code>) per the Spooky algorithm
<code>object_lazy_compaction</code>	<code>int</code>	<code>0</code>	1 to double the L1 file count compaction trigger, reducing compaction frequency and remote I/O at the cost of higher read amplification
<code>object_prefetch_compaction</code>	<code>int</code>	<code>1</code>	1 to download all input SSTables in parallel before the compaction merge begins (uses <code>max_concurrent_downloads</code> threads). 0 to download on demand during merge source creation

Object Store Statistics

`tidesdb_get_db_stats` includes object store fields when a connector is active.

```
tidesdb_db_stats_t stats;
tidesdb_get_db_stats(db, &stats);

if (stats.object_store_enabled)
{
    printf("connector: %s\n", stats.object_store_connector);
    printf("total uploads: %" PRIu64 "\n", stats.total_uploads);
    printf("upload failures: %" PRIu64 "\n", stats.total_upload_failures);
    printf("upload queue depth: %" PRIu64 "\n", stats.upload_queue_depth);
    printf("local cache: %zu / %zu bytes\n",
           stats.local_cache_bytes_used, stats.local_cache_bytes_max);
    /* single-writer fencing: lease epoch held / highest observed */
    printf("primary epoch: %" PRIu64 " seen epoch: %" PRIu64 "\n",
           stats.primary_epoch, stats.seen_epoch);
}
```

Cold Start Recovery

When the local database directory is empty but a connector is configured, TidesDB automatically discovers column families from the object store during recovery. It downloads MANIFEST and config files in parallel (one thread per CF), reconstructs the SSTable inventory, and fetches SSTable data on demand as queries arrive.


```

/* delete all local state */
remove_directory("./mydb");

/* reopen with the same connector -- cold start recovery */
tidesdb_config_t config = tidesdb_default_config();
config.db_path = "./mydb";
config.object_store = s3;
config.object_store_config = &os_cfg;

tidesdb_t *db = NULL;
tidesdb_open(&config, &db);

/* all data is available -- SSTables are fetched from the object store on
demand */
tidesdb_column_family_t *cf = tidesdb_get_column_family(db, "my_cf");

```

How It Works

- Object store mode requires unified memtable mode. Setting `object_store` on the config automatically enables `unified_memtable`
- After each flush, SSTables are uploaded synchronously to the object store before being tracked in the local cache, ensuring the object store has a copy before cache eviction can delete the local file. Uploads use retry (3 attempts with exponential backoff) and post-upload verification (size check)
- Point lookups on frozen SSTables (not present locally) fetch just the single needed klog block (~64KB) via one HTTP range request using `range_get`, bypassing the full file download. The block is cached in the clock cache so subsequent reads are pure memory hits. If the value is in the vlog, a second range request fetches just that vlog block
- Iterators prefetch all needed SStable files in parallel at creation time using bounded threads (`max_concurrent_downloads`, default 8), so sequential reads proceed at local disk speed. Prefetch runs at both initial iterator creation and after seek invalidation
- A hash-indexed LRU local file cache manages disk usage, evicting least-recently-used SStable file pairs (klog + vlog together) when `local_cache_max_bytes` is set
- The MANIFEST is published under the single-writer fence: the primary re-asserts its lease and tags the MANIFEST with the lease epoch before uploading it (synchronously when fencing is active; asynchronously only on a legacy connector without conditional-write support). Cold-start recovery and replicas read the MANIFEST to reconstruct the SStable inventory
- During compaction, the MANIFEST is uploaded after the new merged SStable is committed, before old input SSTables are cleaned up. This ensures replicas and cold-start nodes see the new SStable before old inputs are removed

- Compaction prefetches input SSTables in parallel when `object_prefetch_compaction` is enabled (default). Output SSTables are uploaded synchronously after the merge. Old input SSTables are deleted from the object store with retry and exponential backoff when their reference count reaches zero
- The reaper thread periodically syncs the active WAL to the object store based on write volume (`wal_sync_threshold_bytes`, default 1MB). This reads the WAL's atomic file size lock-free and uploads when the delta since last sync exceeds the threshold, bounding the data loss window to write volume rather than wall clock time
- Upload failures are tracked in `total_upload_failures` on `tidesdb_db_stats_t` for operator monitoring

Build Requirement for S3

The S3 connector requires libcurl and OpenSSL. Enable it with `-DTIDESDB_WITH_S3=ON` during CMake configuration. The filesystem connector is always available without additional dependencies.

Replica Mode

Replica mode enables read-only nodes that follow a primary through the object store. The primary handles all writes while replicas poll for MANIFEST updates and replay WAL segments for near-real-time reads.

Enabling Replica Mode

```
tidesdb_objstore_config_t os_cfg = tidesdb_objstore_default_config();
os_cfg.replica_mode = 1;
os_cfg.replica_sync_interval_us = 1000000; /* 1 second sync interval */
os_cfg.replica_replay_wal = 1;             /* replay WAL for fresh reads */

tidesdb_config_t config = tidesdb_default_config();
config.db_path = "./mydb_replica";
config.object_store = s3; /* same bucket as the primary */
config.object_store_config = &os_cfg;

tidesdb_t *db = NULL;
tidesdb_open(&config, &db);

/* reads work normally */
tidesdb_txn_t *txn = NULL;
tidesdb_txn_begin(db, &txn);

uint8_t *val = NULL;
size_t val_size = 0;
tidesdb_txn_get(txn, cf, key, key_size, &val, &val_size);

/* writes are rejected */
int rc = tidesdb_txn_put(txn, cf, key, key_size, val, val_size, 0);
/* rc == TDB_ERR_READONLY */
```

Sync-on-Commit WAL (Primary Side)

For tighter replication lag, enable sync-on-commit on the primary so every committed write is uploaded to the object store immediately.

```
tidesdb_objstore_config_t os_cfg = tidesdb_objstore_default_config();
os_cfg.wal_sync_on_commit = 1; /* RPO = 0, every commit is durable in S3 */

/* replica sees committed data within one replica_sync_interval_us */
```

Promoting a Replica to Primary

When the primary fails, promote a replica to accept writes.

```

/* external health check detects primary is down */

/* promote this replica */
int rc = tidesdb_promote_to_primary(db);
if (rc == TDB_SUCCESS) {
    /* now writes are accepted */
    tidesdb_txn_t *txn = NULL;
    tidesdb_txn_begin(db, &txn);
    tidesdb_txn_put(txn, cf, key, key_size, val, val_size, 0); /* succeeds
*/
    tidesdb_txn_commit(txn);
} else if (rc == TDB_ERR_PRECONDITION) {
    /* another node already holds the primary lease -- stay a replica and
retry */
}

```

`tidesdb_promote_to_primary` first claims the bucket: it verifies the store enforces conditional writes, then acquires the primary lease by advancing its epoch with a conditional compare-and-swap write. If another node already holds the lease the acquire fails and promotion is **refused with `TDB_ERR_PRECONDITION`**, leaving this node a working replica for an orchestrator to retry (other lease-acquire failures return `TDB_ERR_IO`). Holding the lease, it stops and joins the dedicated replica sync thread (draining any in-progress sync cycle, which also prevents lock contention with the first post-promotion query), runs a final MANIFEST sync and WAL replay — gated on the *outgoing* primary's epoch so that primary's last writes are absorbed rather than skipped as stale — creates a local WAL for crash recovery, and atomically switches to primary mode. It returns `TDB_ERR_INVALID_ARGS` if the node is already a primary.

Single-Writer Fencing

Only one node may write to a bucket at a time — two nodes writing one bucket would otherwise produce conflicting manifests, colliding SSTable IDs, and racing deletes. Rather than leaving that invariant to the orchestrator, the engine enforces it at the store. The primary holds a lease object in the bucket whose epoch it advances with a conditional (compare-and-swap) write, and every MANIFEST publish re-asserts the lease before writing. A superseded primary — one that lost the role but is still alive and can still reach the store — fails its lease re-assertion with `TDB_ERR_PRECONDITION` and self-demotes to a replica instead of writing. MANIFEST and WAL objects carry the epoch that wrote them, and every reader (replica sync, cold start, WAL replay) ignores any object whose epoch is below the current lease, so even an object a fenced node uploads before it notices is never honored.

The guarantee depends on the object store actually enforcing conditional writes (S3 `If-Match` / `If-None-Match`, or the equivalent on MinIO and GCS). A probe at open checks this; if the backend does not enforce them, fencing is disabled and logged, falling back to the orchestrator-honored single-writer invariant. The current lease epoch is observable through `primary_epoch` (held by this node) and `seen_epoch` (highest observed) on `tidesdb_db_stats_t`.

How Replica Sync Works

- Before each MANIFEST sync cycle, the dedicated replica sync thread discovers new column families in the object store that do not exist locally and creates them automatically by downloading their config and MANIFEST
- The replica sync thread polls the remote MANIFEST for each CF every `replica_sync_interval_us`
- New SSTables from the primary's flushes and compactions are added to the replica's levels
- SSTables compacted away on the primary are removed from the replica's levels
- When `replica_replay_wal` is enabled, all available WAL segments are discovered from the object store via listing and replayed in generation order into the memtable for near-real-time reads
- WAL replay is idempotent using sequence numbers so entries already present are skipped
- SSTable data is not downloaded during sync. It is fetched on demand via `range_get` for point lookups or `prefetch` for iterators

Utility Functions

`tidesdb_free`

Use `tidesdb_free` to free memory allocated by TidesDB. This is particularly useful for FFI/language bindings where the caller needs to free memory returned by TidesDB functions (e.g., values from `tidesdb_txn_get`).

```
uint8_t *value = NULL;
size_t value_size = 0;

if (tidesdb_txn_get(txn, cf, key, key_size, &value, &value_size) == 0)
{
    /* Use value */
    printf("Value: %.*s\n", (int)value_size, value);

    /* Free using tidesdb_free (or standard free) */
    tidesdb_free(value);
}
```

① When to use `tidesdb_free`

- FFI bindings · Language bindings (Java, Rust, Go, Python) should use `tidesdb_free` to ensure memory is freed by the same allocator that allocated it
- Cross-platform · Ensures correct deallocation on all platforms
- Consistency · Provides a uniform API for memory management

For native C/C++ applications, `free()` works identically since TidesDB uses the standard allocator.

TidesDB C++ API Reference

Getting Started

Prerequisites

You **must** have the TidesDB shared C library installed on your system. You can find the installation instructions [here](#).

Installation



```
git clone https://github.com/tidesdb/tidesdb-cpp.git
cd tidesdb-cpp
cmake -S . -B build
cmake --build build
sudo cmake --install build
```

Custom Installation Paths

If you installed TidesDB to a non-standard location, you can specify custom paths:



```
cmake -S . -B build -DCMAKE_PREFIX_PATH=/custom/path
cmake --build build
```

Usage

Opening and Closing a Database

```
#include <tidesdb/tidesdb.hpp>
#include <iostream>

int main() {
    tidesdb::Config config;
    config.dbPath = "./mydb";
    config.numFlushThreads = 2; // Flush
thread pool size (default: 2)
    config.numCompactionThreads = 2; // Compaction
thread pool size (default: 2)
    config.logLevel = tidesdb::LogLevel::Info; // Log level
(default: Info)
    config.blockCacheSize = 64 * 1024 * 1024; // 64MB
global block cache (default: 64MB)
    config.maxOpenSSTables = 256; // Max cached
SSTable structures (default: 256)
    config.maxMemoryUsage = 0; // Global
memory limit in bytes (default: 0 = auto, 50% of system RAM)
    config.logToFile = false; // Write logs
to file instead of stderr (default: false)
    config.logTruncationAt = 24 * 1024 * 1024; // Log file
truncation size (default: 24MB), 0 = no truncation
    config.unifiedMemtable = false; // Enable
unified memtable mode (default: false = per-CF memtables)
    config.unifiedMemtableWriteBufferSize = 0; // Unified
memtable write buffer size (0 = auto)
    config.unifiedMemtableSkipListMaxLevel = 0; // Skip list
max level for unified memtable (0 = default 12)
    config.unifiedMemtableSkipListProbability = 0; // Skip list
probability (0 = default 0.25)
    config.unifiedMemtableSyncMode = tidesdb::SyncMode::None; // Sync mode
for unified WAL
    config.unifiedMemtableSyncIntervalUs = 0; // Sync
interval for unified WAL in microseconds
    config.maxConcurrentFlushes = 0; // Cap on in-
flight memtable flushes across all CFs (0 = library default)
    config.finishCompactionsOnClose = false; // false =
cancel in-flight compactions at next checkpoint for fast shutdown
(default); true = let them finish before close returns
```



```

    config.objectStore = nullptr; // Pluggable
    object store connector (nullptr = local only)
    config.objectStoreConfig = std::nullopt; // Object
    store behavior config (nullopt = defaults)

    try {
        tidesdb::TidesDB db(config);
        std::cout << "Database opened successfully" << std::endl;
    } catch (const tidesdb::Exception& e) {
        std::cerr << "Failed to open database: " << e.what() << std::endl;
        return 1;
    }

    return 0;
}

```

Using default configuration

Use `TidesDB::defaultConfig()` to get a configuration with sensible defaults, then override specific fields as needed. The defaults are pulled from the underlying C library, so values such as `maxConcurrentFlushes` and the unified-memtable settings track the engine's defaults automatically — if you previously hardcoded these values when overriding fields, re-test after upgrading.

```

auto config = tidesdb::TidesDB::defaultConfig();
config.dbPath = "./mydb";
config.logLevel = tidesdb::LogLevel::Warn; // Override: only warnings and
errors

tidesdb::TidesDB db(config);

```

maxConcurrentFlushes

`maxConcurrentFlushes` is a global semaphore on the number of in-flight memtable flushes across all column families. It bounds total transient memory and work-queue depth when many column families flush at once. Leave it at `0` to inherit the library default (recommended), or lower it on memory-constrained hosts to throttle peak flush concurrency.

Initialization and Custom Allocators

Calling `tidesdb::init()` is **optional** — the first database open lazily initializes the library with the system allocator. Call `init()` explicitly only when you need to install custom memory allocation functions (for example, when embedding TidesDB inside a host that owns memory, such as a Redis module). It must be called once, before opening any database.

```
// Optional: install custom allocators before opening any database
bool initialized = tidesdb::init(myMalloc, myCalloc, myRealloc, myFree);
// initialized == true if this call performed initialization,
// false if TidesDB was already initialized (allocators left unchanged)

// ... open databases, do work ...

tidesdb::finalize(); // reset the allocator after all databases are closed
```

Pass `nullptr` (the default for each parameter) to keep the corresponding system allocator. After `finalize()` returns, `init()` may be called again with different allocators.

Function signatures

```
bool init(tidesdb_malloc_fn mallocFn = nullptr, tidesdb_calloc_fn
callocFn = nullptr,
          tidesdb_realloc_fn reallocFn = nullptr, tidesdb_free_fn freeFn =
nullptr);
void finalize();
```

Raising the Open-File Limit

`tidesdb::raiseOpenFileLimit()` raises the process's open-file ceiling so a database can keep more SSTables open. Call it **before** opening the database — the engine sizes `maxOpenSSTables` to fit the ceiling at open time. This is an explicit, opt-in operator action; TidesDB never raises the limit itself. A failed or partial raise is non-fatal.

```
// Report the current ceiling (desired <= 0 just reports)
long current = tidesdb::raiseOpenFileLimit(0);

// Request a higher ceiling before opening the database
long effective = tidesdb::raiseOpenFileLimit(8192);
std::cout << "Open-file ceiling now: " << effective << std::endl;
```

On POSIX systems this raises the `RLIMIT_NOFILE` soft limit toward the hard limit; on Windows it raises the CRT stdio cap (max 8192). The return value is the ceiling in effect after the attempt.

Logging

TidesDB provides structured logging with multiple severity levels.

Log Levels

- `LogLevel::Debug` · Detailed diagnostic information
- `LogLevel::Info` · General informational messages (default)
- `LogLevel::Warn` · Warning messages for potential issues
- `LogLevel::Error` · Error messages for failures
- `LogLevel::Fatal` · Critical errors that may cause shutdown
- `LogLevel::None` · Disable all logging

Configure at startup

```
tidesdb::Config config;
config.dbPath = "./mydb";
config.logLevel = tidesdb::LogLevel::Debug; // Enable debug logging

tidesdb::TidesDB db(config);
```

Production configuration

```
tidesdb::Config config;
config.dbPath = "./mydb";
config.logLevel = tidesdb::LogLevel::Warn; // Only warnings and errors

tidesdb::TidesDB db(config);
```

Output format Logs are written to **stderr** by default with timestamps:

```
[HH:MM:SS.mmm] [LEVEL] filename:line: message
```

Log to file

Enable `logToFile` to write logs to a `LOG` file in the database directory instead of stderr:

```
tidesdb::Config config;
config.dbPath = "./mydb";
config.logLevel = tidesdb::LogLevel::Debug;
config.logToFile = true; // Write to ./mydb/LOG instead of stderr

tidesdb::TidesDB db(config);
// Logs are now written to ./mydb/LOG
```

The log file is opened in append mode and uses line buffering for real-time logging. If the log file cannot be opened, logging falls back to default.

Creating and Dropping Column Families

Column families are isolated key-value stores with independent configuration.

```

auto cfConfig = tidesdb::ColumnFamilyConfig::defaultConfig();
db.createColumnFamily("my_cf", cfConfig);

auto cfConfig = tidesdb::ColumnFamilyConfig::defaultConfig();
cfConfig.writeBufferSize = 128 * 1024 * 1024;
cfConfig.levelSizeRatio = 10;
cfConfig.minLevels = 5;
cfConfig.compressionAlgorithm = tidesdb::CompressionAlgorithm::LZ4;
cfConfig.enableBloomFilter = true;
cfConfig.bloomFPR = 0.01;
cfConfig.enableBlockIndexes = true;
cfConfig.syncMode = tidesdb::SyncMode::Interval;
cfConfig.syncIntervalUs = 128000;
cfConfig.defaultIsolationLevel = tidesdb::IsolationLevel::ReadCommitted;
cfConfig.tombstoneDensityTrigger = 0.0; // Per-SSTable tombstone
density above which compaction escalates (0.0 = disabled, range 0.0 to 1.0)
cfConfig.tombstoneDensityMinEntries = 1024; // Minimum entry count for an
SSTable to be considered by the density trigger
cfConfig.useBtree = false; // Use block-based format (default), set true
for B+tree klog format

db.createColumnFamily("my_cf", cfConfig);

db.dropColumnFamily("my_cf");

// Delete by handle (faster when you already hold a handle, avoids name
lookup)
auto cf2 = db.getColumnFamily("another_cf");
db.deleteColumnFamily(cf2);

```

CRUD Operations

All operations in TidesDB are performed through transactions for ACID guarantees.

Writing Data

```

auto cf = db.getColumnFamily("my_cf");

auto txn = db.beginTransaction();
txn.put(cf, "key", "value", -1);
txn.commit();

```

Writing with TTL

```
#include <ctime>

auto cf = db.getColumnFamily("my_cf");

auto txn = db.beginTransaction();

auto ttl = std::time(nullptr) + 10;

txn.put(cf, "temp_key", "temp_value", ttl);
txn.commit();
```

TTL Examples

```
auto ttl = static_cast<std::time_t>(-1);

auto ttl = std::time(nullptr) + (5 * 60);

auto ttl = std::time(nullptr) + (60 * 60);

auto ttl = static_cast<std::time_t>(1735689599);
```

Reading Data

```
auto cf = db.getColumnFamily("my_cf");

auto txn = db.beginTransaction();
auto value = txn.get(cf, "key");
std::string valueStr(value.begin(), value.end());
std::cout << "Value: " << valueStr << std::endl;
```

Deleting Data

```
auto cf = db.getColumnFamily("my_cf");

auto txn = db.beginTransaction();
txn.del(cf, "key");
txn.commit();
```

Single Delete

`singleDelete` emits a single-delete tombstone. When the tombstone meets exactly one prior put for the same key during compaction, both records are dropped, so the tombstone does not persist past its matching put. Use it only when the caller guarantees at most one put precedes the delete; otherwise prefer `del`.

```
auto cf = db.getColumnFamily("my_cf");

auto txn = db.beginTransaction();
txn.singleDelete(cf, "key");
txn.commit();
```

Multi-Operation Transactions

```
auto cf = db.getColumnFamily("my_cf");

auto txn = db.beginTransaction();

txn.put(cf, "key1", "value1", -1);
txn.put(cf, "key2", "value2", -1);
txn.del(cf, "old_key");

txn.commit();
```

Transaction Rollback

```
auto cf = db.getColumnFamily("my_cf");

auto txn = db.beginTransaction();
txn.put(cf, "key", "value", -1);

// Discard all operations
txn.rollback();
```

Iterating Over Data

Iterators provide efficient bidirectional traversal over key-value pairs.

Forward Iteration

```
auto cf = db.getColumnFamily("my_cf");

auto txn = db.beginTransaction();
auto iter = txn.newIterator(cf);

iter.seekToFirst();

while (iter.valid()) {
    auto key = iter.key();
    auto value = iter.value();

    std::string keyStr(key.begin(), key.end());
    std::string valueStr(value.begin(), value.end());

    std::cout << "Key: " << keyStr << ", Value: " << valueStr << std::endl;

    iter.next();
}
```


Backward Iteration

```
auto cf = db.getColumnFamily("my_cf");

auto txn = db.beginTransaction();
auto iter = txn.newIterator(cf);

iter.seekToLast();

while (iter.valid()) {
    auto key = iter.key();
    auto value = iter.value();
    iter.prev();
}
```

Seeking

How Seek Works

With Block Indexes Enabled (`enableBlockIndexes = true`):

- Uses compact block index with parallel arrays (min/max key prefixes and file positions)
- Binary search through sampled keys at configurable ratio (default 1:1 via `indexSampleRatio`, meaning every block is indexed)
- Jumps directly to the target block using the file position
- Scans forward from that block to find the exact key
- **Performance** · $O(\log n)$ binary search + $O(k)$ entries per block scan

Block indexes provide dramatic speedup for large SSTables at the cost of ~2-5% storage overhead for the compact index structure.

`seek(key)` · Positions iterator at the first key $\geq$ target key

```

auto iter = txn.newIterator(cf);

iter.seek("user:1000");

// Iterator is now positioned at "user:1000" or the next key after it
if (iter.valid()) {
    auto key = iter.key();
    std::string keyStr(key.begin(), key.end());
    std::cout << "Found: " << keyStr << std::endl;
}

```

seekForPrev(key) · Positions iterator at the last key $\leq$ target key

```

auto iter = txn.newIterator(cf);

iter.seekForPrev("user:2000");

// Iterator is now positioned at "user:2000" or the previous key before it
while (iter.valid()) {
    // Iterate backwards from this point
    iter.prev();
}

```

Prefix Seeking

Since `seek` positions the iterator at the first key $\geq$ target, you can use a prefix as the seek target to efficiently scan all keys sharing that prefix:

```

auto iter = txn.newIterator(cf);

std::string prefix = "user:";
iter.seek(prefix);

while (iter.valid()) {
    auto key = iter.key();
    std::string keyStr(key.begin(), key.end());

    // Stop when keys no longer match prefix
    if (keyStr.substr(0, prefix.size()) != prefix) break;

    auto value = iter.value();
    std::string valueStr(value.begin(), value.end());
    std::cout << "Found: " << keyStr << " = " << valueStr << std::endl;

    iter.next();
}

```

This pattern works across both memtables and SSTables. When block indexes are enabled, the seek operation uses binary search to jump directly to the relevant block, making prefix scans efficient even on large datasets.

Combined Key-Value Fetch

`keyValue()` retrieves both key and value in a single call, which can be more efficient than separate `key()` and `value()` calls.

```

auto iter = txn.newIterator(cf);
iter.seekToFirst();

while (iter.valid()) {
    auto [key, value] = iter.keyValue();
    std::string keyStr(key.begin(), key.end());
    std::string valueStr(value.begin(), value.end());

    std::cout << keyStr << " = " << valueStr << std::endl;
    iter.next();
}

```

Getting Column Family Statistics

Retrieve detailed statistics about a column family.

```

auto cf = db.getColumnFamily("my_cf");

auto stats = cf.getStats();

std::cout << "Number of Levels: " << stats.numLevels << std::endl;
std::cout << "Memtable Size: " << stats.memtableSize << " bytes" <<
std::endl;
std::cout << "Total Keys: " << stats.totalKeys << std::endl;
std::cout << "Total Data Size: " << stats.totalDataSize << " bytes" <<
std::endl;
std::cout << "Average Key Size: " << stats.avgKeySize << " bytes" <<
std::endl;
std::cout << "Average Value Size: " << stats.avgValueSize << " bytes" <<
std::endl;
std::cout << "Read Amplification: " << stats.readAmp << std::endl;
std::cout << "Cache Hit Rate: " << stats.hitRate << std::endl;

// B+tree stats (only populated if useBtree=true)
if (stats.useBtree) {
    std::cout << "B+tree Total Nodes: " << stats.btreeTotalNodes <<
std::endl;
    std::cout << "B+tree Max Height: " << stats.btreeMaxHeight <<
std::endl;
    std::cout << "B+tree Avg Height: " << stats.btreeAvgHeight <<
std::endl;
}

// Tombstone density observability
std::cout << "Total Tombstones: " << stats.totalTombstones << std::endl;
std::cout << "Tombstone Ratio: " << (stats.tombstoneRatio * 100.0) << "%"
<< std::endl;
std::cout << "Max SSTable Density: " << stats.maxSstDensity << std::endl;
if (stats.maxSstDensityLevel != 0) {
    std::cout << "Max SSTable Density Level: " << stats.maxSstDensityLevel
<< std::endl;
}

// Per-level statistics
for (int i = 0; i < stats.numLevels; ++i) {
    std::cout << "Level " << (i + 1) << ": "
        << stats.levelNumSSTables[i] << " SSTables, "
        << stats.levelSizes[i] << " bytes, "
        << stats.levelKeyCounts[i] << " keys, "

```

```

        << stats.levelTombstoneCounts[i] << " tombstones" <<
std::endl;
}

if (stats.config.has_value()) {
    std::cout << "Write Buffer Size: " << stats.config->writeBufferSize <<
std::endl;
    std::cout << "Compression: " << static_cast<int>(stats.config-
>compressionAlgorithm) << std::endl;
    std::cout << "Bloom Filter: " << (stats.config->enableBloomFilter ?
"enabled" : "disabled") << std::endl;
}

```

Statistics Fields

- `numLevels` · Number of LSM levels
- `memtableSize` · Current memtable size in bytes
- `levelSizes` · Total bytes per level
- `levelNumSSTables` · Number of SSTables per level
- `totalKeys` · Total number of keys across memtable and all SSTables
- `totalDataSize` · Total data size (klog + vlog) across all SSTables
- `avgKeySize` · Average key size in bytes
- `avgValueSize` · Average value size in bytes
- `levelKeyCounts` · Number of keys per level
- `readAmp` · Read amplification (point lookup cost multiplier)
- `hitRate` · Cache hit rate (0.0 if cache disabled)
- `useBtree` · Whether column family uses B+tree klog format
- `btreeTotalNodes` · Total B+tree nodes across all SSTables (only if `useBtree=true`)
- `btreeMaxHeight` · Maximum tree height across all SSTables (only if `useBtree=true`)
- `btreeAvgHeight` · Average tree height across all SSTables (only if `useBtree=true`)
- `totalTombstones` · Sum of tombstone counts across all SSTables in the column family
- `tombstoneRatio` · `totalTombstones / totalKeys` (0.0 to 1.0, 0.0 if `totalKeys` is 0)
- `levelTombstoneCounts` · Per-level tombstone counts (parallels `levelKeyCounts`)
- `maxSstDensity` · Worst per-SSTable tombstone density observed in the CF (0.0 to 1.0)
- `maxSstDensityLevel` · 1-based level index where `maxSstDensity` was observed (0 if none)
- `walBytesWritten` · Framed bytes appended to this CF's WAL (0 in unified memtable mode)
- `flushBytesWritten` · On-disk bytes this CF's flushes wrote to L0 SSTables
- `compactionBytesWritten` · On-disk bytes this CF's compactions wrote

- `compactionBytesRead` · On-disk bytes this CF's compactions read as input
- `userBytesWritten` · Logical key+value bytes committed to this CF (write-amplification denominator)
- `flushCount` · Flushed SSTables produced by this CF
- `compactionCount` · Compaction output SSTables produced by this CF
- `config` · Column family configuration (optional)

The write-amplification counters are lifetime totals since the database was opened. A column family's write amplification is $(\text{walBytesWritten} + \text{flushBytesWritten} + \text{compactionBytesWritten}) / \text{userBytesWritten}$. In unified memtable mode `walBytesWritten` is 0 — the shared WAL volume is reported db-wide as `DbStats::uwalBytesWritten`.

```
if (stats.userBytesWritten > 0) {
    double wa = static_cast<double>(stats.walBytesWritten +
    stats.flushBytesWritten +
                                stats.compactionBytesWritten) /
    static_cast<double>(stats.userBytesWritten);
    std::cout << "Write amplification: " << wa << "x" << std::endl;
}
```

Database-Level Statistics

Get aggregate statistics across the entire database instance.

```

auto dbStats = db.getDbStats();

std::cout << "Column families: " << dbStats.numColumnFamilies << std::endl;
std::cout << "Total memory: " << dbStats.totalMemory << " bytes" <<
std::endl;
std::cout << "Available memory: " << dbStats.availableMemory << " bytes" <<
std::endl;
std::cout << "Resolved memory limit: " << dbStats.resolvedMemoryLimit << "
bytes" << std::endl;
std::cout << "Memory pressure level: " << dbStats.memoryPressureLevel <<
std::endl;
std::cout << "Global sequence: " << dbStats.globalSeq << std::endl;
std::cout << "Flush queue: " << dbStats.flushQueueSize << " pending" <<
std::endl;
std::cout << "Compaction queue: " << dbStats.compactionQueueSize << "
pending" << std::endl;
std::cout << "Total SSTables: " << dbStats.totalSstableCount << std::endl;
std::cout << "Total data size: " << dbStats.totalDataSizeBytes << " bytes"
<< std::endl;
std::cout << "Open SSTable handles: " << dbStats.numOpenSstables <<
std::endl;
std::cout << "In-flight txn memory: " << dbStats.txnMemoryBytes << " bytes"
<< std::endl;
std::cout << "Immutable memtables: " << dbStats.totalImmutableCount <<
std::endl;
std::cout << "Memtable bytes: " << dbStats.totalMemtableBytes << std::endl;

// Unified memtable fields
std::cout << "Unified memtable enabled: " << dbStats.unifiedMemtableEnabled
<< std::endl;
std::cout << "Unified memtable bytes: " << dbStats.unifiedMemtableBytes <<
std::endl;
std::cout << "Unified immutable count: " << dbStats.unifiedImmutableCount
<< std::endl;
std::cout << "Unified is flushing: " << dbStats.unifiedIsFlushing <<
std::endl;
std::cout << "Unified WAL generation: " << dbStats.unifiedWalGeneration <<
std::endl;

// Object store fields
std::cout << "Object store enabled: " << dbStats.objectStoreEnabled <<
std::endl;
if (dbStats.objectStoreEnabled) {

```



```

        std::cout << "Object store connector: " << dbStats.objectStoreConnector
<< std::endl;
        std::cout << "Local cache bytes used: " << dbStats.localCacheBytesUsed
<< std::endl;
        std::cout << "Upload queue depth: " << dbStats.uploadQueueDepth <<
std::endl;
        std::cout << "Total uploads: " << dbStats.totalUploads << std::endl;
    }
    std::cout << "Replica mode: " << dbStats.replicaMode << std::endl;

```

Database statistics fields

- `numColumnFamilies` · Number of column families
- `totalMemory` · System total memory
- `availableMemory` · System available memory at open time
- `resolvedMemoryLimit` · Resolved memory limit (auto or configured)
- `memoryPressureLevel` · Current memory pressure (0=normal, 1=elevated, 2=high, 3=critical)
- `flushPendingCount` · Number of pending flush operations (queued + in-flight)
- `totalMemtableBytes` · Total bytes in active memtables across all CFs
- `totalImmutableCount` · Total immutable memtables across all CFs
- `totalSstableCount` · Total SSTables across all CFs and levels
- `totalDataSizeBytes` · Total data size (klog + vlog) across all CFs
- `numOpenSstables` · Number of currently open SSTable file handles
- `globalSeq` · Current global sequence number
- `txnMemoryBytes` · Bytes held by in-flight transactions
- `compactionQueueSize` · Number of pending compaction tasks
- `flushQueueSize` · Number of pending flush tasks in queue
- `unifiedMemtableEnabled` · Whether unified memtable mode is active
- `unifiedMemtableBytes` · Bytes in unified active memtable
- `unifiedImmutableCount` · Number of unified immutable memtables
- `unifiedIsFlushing` · Whether unified memtable is currently flushing/rotating
- `unifiedNextCfIndex` · Next CF index to be assigned in unified mode
- `unifiedWalGeneration` · Current unified WAL generation counter
- `objectStoreEnabled` · Whether object store mode is active
- `objectStoreConnector` · Connector name (“s3”, “gcs”, “fs”, etc.)
- `localCacheBytesUsed` · Current local file cache usage in bytes
- `localCacheBytesMax` · Configured maximum local cache size in bytes

- `localCacheNumFiles` · Number of files tracked in local cache
- `lastUploadedGeneration` · Highest WAL generation confirmed uploaded
- `uploadQueueDepth` · Number of pending upload jobs in the queue
- `totalUploads` · Lifetime count of objects uploaded to object store
- `totalUploadFailures` · Lifetime count of permanently failed uploads
- `replicaMode` · Whether running in read-only replica mode
- `primaryEpoch` · Single-writer fencing: lease epoch this primary currently holds (0 when not a primary / no lease)
- `seenEpoch` · Single-writer fencing: highest lease epoch a replica has observed
- `uwalBytesWritten` · Framed bytes appended to the shared unified WAL (0 when unified mode is off)
- `walBytesWritten` · Per-CF WAL bytes summed across all column families
- `flushBytesWritten` · Flush output bytes summed across all column families
- `compactionBytesWritten` · Compaction output bytes summed across all column families
- `compactionBytesRead` · Compaction input bytes summed across all column families
- `userBytesWritten` · Logical committed bytes summed across all column families
- `flushCount` · Flushed SSTables summed across all column families
- `compactionCount` · Compaction output SSTables summed across all column families

The write-amplification counters are lifetime totals since the database was opened. db-wide write amplification is $\frac{(\text{uwalBytesWritten} + \text{walBytesWritten} + \text{flushBytesWritten} + \text{compactionBytesWritten})}{\text{userBytesWritten}}$. `uwalBytesWritten` is the shared unified WAL volume (0 when unified mode is off); the remaining counters are summed across all column families.

Unlike `getStats()` (which heap-allocates internally), `getDbStats()` fills a stack-allocated struct. No free is needed.

Listing Column Families

```
auto cfList = db.listColumnFamilies();

std::cout << "Available column families:" << std::endl;
for (const auto& name : cfList) {
    std::cout << "  - " << name << std::endl;
}
```

Renaming a Column Family

Atomically rename a column family and its underlying directory.

```
db.renameColumnFamily("old_name", "new_name");
```

Behavior

- Waits for any in-progress flush or compaction to complete
- Atomically renames the column family directory on disk
- Updates all internal paths (SSTables, manifest, config)
- Thread-safe with proper locking

Cloning a Column Family

Create a complete copy of an existing column family with a new name. The clone contains all the data from the source at the time of cloning.

```
db.cloneColumnFamily("source_cf", "cloned_cf");

// Both column families now exist independently
auto original = db.getColumnFamily("source_cf");
auto clone = db.getColumnFamily("cloned_cf");
```

Behavior

- Flushes the source column family's memtable to ensure all data is on disk
- Waits for any in-progress flush or compaction to complete
- Copies all SSTable files (`.klog` and `.vlog`) to the new directory
- Copies manifest and configuration files
- The clone is completely independent — modifications to one do not affect the other

Use cases

- Testing · Create a copy of production data for testing without affecting the original
- Branching · Create a snapshot of data before making experimental changes
- Migration · Clone data before schema or configuration changes
- Backup verification · Clone and verify data integrity without modifying the source

Compaction

Manual Compaction

```
auto cf = db.getColumnFamily("my_cf");

cf.compact();
```

Targeted Range Compaction

`compactRange` runs a synchronous compaction over a specific key range. Only SSTables whose minimum and maximum keys overlap the requested range participate in the merge, so the work and I/O are bounded to the affected portion of the LSM tree rather than the whole column family.

```
auto cf = db.getColumnFamily("my_cf");

std::vector<std::uint8_t> start{'t','e','n','a','n','t','_','4','2',':'};
std::vector<std::uint8_t> end{'t','e','n','a','n','t','_','4','2',';'};

cf.compactRange(start, end);
```

Pass `std::nullopt` for either endpoint to leave that side unbounded:

```
using OptKey = std::optional<std::vector<std::uint8_t>>;

// Compact everything from `start` upward
cf.compactRange(OptKey{start}, OptKey{});

// Compact everything below `end`
cf.compactRange(OptKey{}, OptKey{end});
```

A `string_view` overload is also provided for the common case of textual keys:

```
using OptSV = std::optional<std::string_view>;

cf.compactRange(OptSV{std::string_view{"tenant_42:"}},
                OptSV{std::string_view{"tenant_42;"}});
```

When to use

- Bulk reclaim after a large range delete, where waiting for natural compaction would leave tombstones and obsolete versions on disk
- Tenant eviction or sliding-window expiration that does not fit TTL semantics
- Post-import cleanup of a known key range loaded with `put` followed by `del`
- Operational counterpart to the automatic tombstone density trigger when an operator wants reclaim now rather than at the next natural threshold crossing

Behavior

- Synchronous, blocks the caller until the merge commits or fails
- Does not enqueue work onto the compaction thread pool, the calling thread does the work
- Selects only SSTables whose key range overlaps the requested range using the column family's comparator, SSTables outside the range are not touched
- Applies the same emit-loop logic as background compactions (tombstone reclamation rules, single-delete pair cancellation, sequence-based deduplication, value recompression)
- Output SSTables are committed to the manifest atomically and old inputs are marked for deletion

Return values

- Returns normally on success
- Throws `tidesdb::Exception` with `ErrorCode::InvalidArgs` if both endpoints are `std::nullopt` (use `compact()` for full CF compaction)
- Throws `tidesdb::Exception` with `ErrorCode::Locked` if another compaction is running for the column family
- Throws `tidesdb::Exception` with standard I/O and memory error codes if the merge cannot complete

Manual Memtable Flush

```
auto cf = db.getColumnFamily("my_cf");

cf.flushMemtable();
```

Checking Flush/Compaction Status

Check if a column family currently has flush or compaction operations in progress.

```

auto cf = db.getColumnFamily("my_cf");

if (cf.isFlushing()) {
    std::cout << "Flush in progress" << std::endl;
}

if (cf.isCompacting()) {
    std::cout << "Compaction in progress" << std::endl;
}

```

Use cases

- Graceful shutdown · Wait for background operations to complete before closing
- Maintenance windows · Check if operations are running before triggering manual compaction
- Monitoring · Track background operation status for observability

Purge Column Family

`purge()` forces a synchronous flush and aggressive compaction for a single column family. Unlike `flushMemtable()` and `compact()` (which are non-blocking), `purge` blocks until all flush and compaction I/O is complete.

```

auto cf = db.getColumnFamily("my_cf");

cf.purge();
// All data is now flushed to SSTables and compacted

```

Behavior

1. Waits for any in-progress flush to complete
2. Force-flushes the active memtable (even if below threshold)
3. Waits for flush I/O to fully complete
4. Waits for any in-progress compaction to complete
5. Triggers synchronous compaction inline (bypasses the compaction queue)
6. Waits for any queued compaction to drain

When to use

- Before backup or checkpoint · Ensure all data is on disk and compacted

- After bulk deletes · Reclaim space immediately by compacting away tombstones
- Manual maintenance · Force a clean state during a maintenance window
- Pre-shutdown · Ensure all pending work is complete before closing

Purge Database

`purge()` on the database forces a synchronous flush and aggressive compaction for **all** column families, then drains both the global flush and compaction queues.

```
db.purge();
// All CFs flushed and compacted, all queues drained
```

Behavior

1. Calls `purge()` on each column family
2. Drains the global flush queue (waits for queue size and pending count to reach 0)
3. Drains the global compaction queue (waits for queue size to reach 0)

Purge vs Manual Flush + Compact

`flushMemtable()` and `compact()` are non-blocking, they enqueue work and return immediately. `ColumnFamily::purge()` and `TidesDB::purge()` are synchronous, they block until all work is complete. Use purge when you need a guarantee that all data is on disk and compacted before proceeding.

Cancel Background Work (Fast Shutdown)

`cancelBackgroundWork()` cancels background compaction db-wide so the database can close quickly. In-flight merges bail out safely at their next checkpoint and queued compaction is skipped; flushes are left untouched, so durability is preserved. It blocks (bounded) until compaction is idle.

```
// Just before closing for a fast shutdown
db.cancelBackgroundWork();
// db destructor / close now returns quickly without waiting on long merges
```

Behavior

- Cancels compaction only — flushes still run, so no committed data is lost

- In-flight merges discard their uncommitted output and leave inputs intact (recovery handles the mid-merge state the same way)
- The cancellation is sticky for the session and reset on the next open
- Intended to be called right before closing the database

Close behavior: `finishCompactionsOnClose`

`cancelBackgroundWork()` is the explicit, on-demand way to make a single close fast. `Config::finishCompactionsOnClose` is the standing policy applied to every close of that database:

- `false` (default) — close cancels in-flight compactions at their next checkpoint for a fast shutdown. The merge discards its uncommitted output and leaves inputs intact, so no committed data is lost (recovery handles a mid-merge state the same way).
- `true` — close lets in-flight compactions run to completion before returning, trading shutdown latency for a fully compacted on-disk state.

Promote Replica to Primary

Switch a read-only replica database to primary (read-write) mode.

```
db.promoteToPrimary();
```

Behavior

- Only valid when the database was opened in replica mode (via object store configuration)
- Transitions the database from read-only to read-write mode
- Throws `ErrorCode::InvalidArgs` if the database is not a replica

Unified Memtable Mode

Enable a single shared memtable across all column families for reduced memory overhead and simplified WAL management.


```

tidesdb::Config config;
config.dbPath = "./mydb";
config.unifiedMemtable = true;
config.unifiedMemtableWriteBufferSize = 64 * 1024 * 1024; // 64MB unified
buffer
config.unifiedMemtableSkipListMaxLevel = 12;
config.unifiedMemtableSkipListProbability = 0.25f;
config.unifiedMemtableSyncMode = tidesdb::SyncMode::Interval;
config.unifiedMemtableSyncIntervalUs = 128000;

tidesdb::TidesDB db(config);

```

Configuration fields

- `unifiedMemtable` · Enable unified memtable mode (default: false = per-CF memtables)
- `unifiedMemtableWriteBufferSize` · Write buffer size for unified memtable (0 = auto)
- `unifiedMemtableSkipListMaxLevel` · Skip list max level (0 = default 12)
- `unifiedMemtableSkipListProbability` · Skip list probability (0 = default 0.25)
- `unifiedMemtableSyncMode` · Sync mode for unified WAL (default: `SyncMode::None`)
- `unifiedMemtableSyncIntervalUs` · Sync interval for unified WAL in microseconds (default: 0)

Updating Runtime Configuration

Update runtime-safe configuration settings without restarting the database.

```

auto cf = db.getColumnFamily("my_cf");

auto newConfig = tidesdb::ColumnFamilyConfig::defaultConfig();
newConfig.writeBufferSize = 256 * 1024 * 1024;
newConfig.skipListMaxLevel = 16;
newConfig.bloomFPR = 0.001; // 0.1% false positive rate

bool persistToDisk = true; // Save to config.ini
cf.updateRuntimeConfig(newConfig, persistToDisk);

```

Updatable settings (all applied by `updateRuntimeConfig`):

- `writeBufferSize` · Memtable flush threshold

- `skipListMaxLevel` · Skip list level for **new** memtables
- `skipListProbability` · Skip list probability for **new** memtables
- `bloomFPR` · False positive rate for **new** SSTables
- `enableBloomFilter` · Enable/disable bloom filters for **new** SSTables
- `enableBlockIndexes` · Enable/disable block indexes for **new** SSTables
- `blockIndexPrefixLen` · Block index prefix length for **new** SSTables
- `indexSampleRatio` · Index sampling ratio for **new** SSTables
- `compressionAlgorithm` · Compression for **new** SSTables (existing SSTables retain their original compression)
- `klogValueThreshold` · Value log threshold for **new** writes
- `syncMode` · Durability mode. Also updates the active WAL's sync mode immediately
- `syncIntervalUs` · Sync interval in microseconds (only used when syncMode is `SyncMode::Interval`)
- `levelSizeRatio` · LSM level sizing (DCA recalculates capacities dynamically)
- `minLevels` · Minimum LSM levels
- `dividingLevelOffset` · Compaction dividing level offset
- `l1FileCountTrigger` · L1 file count compaction trigger
- `l0QueueStallThreshold` · Backpressure stall threshold
- `defaultIsolationLevel` · Default transaction isolation level
- `minDiskSpace` · Minimum disk space required
- `commitHookFn` / `commitHookCtx` · Commit hook callback and context

Non-updatable settings (not modified by this function):

- `comparatorName` · Cannot change sort order after creation (would corrupt key ordering in existing SSTables)
- `useBtree` · Cannot change klog format after creation (existing SSTables use the original format)

Changes apply immediately to new operations. Existing SSTables and memtables retain their original settings. The update operation is thread-safe.

Commit Hook (Change Data Capture)

`ColumnFamily::setCommitHook` registers a callback that fires synchronously after every transaction commit on a column family. The hook receives the full batch of committed operations atomically, enabling real-time change data capture without WAL parsing or external log consumers.

```

auto cf = db.getColumnFamily("my_cf");

// Define a commit hook
auto myHook = [](const tidesdb_commit_op_t* ops, int num_ops,
                 uint64_t commit_seq, void* ctx) -> int {
    for (int i = 0; i < num_ops; ++i) {
        std::string key(reinterpret_cast<const char*>(ops[i].key),
ops[i].key_size);
        if (ops[i].is_delete) {
            std::cout << "[" << commit_seq << "] DELETE " << key <<
std::endl;
        } else {
            std::string value(reinterpret_cast<const char*>(ops[i].value),
ops[i].value_size);
            std::cout << "[" << commit_seq << "] PUT " << key << " = " <<
value << std::endl;
        }
    }
    return 0;
};

// Attach hook at runtime
cf.setCommitHook(myHook, nullptr);

// Normal writes now trigger the hook automatically
auto txn = db.beginTransaction();
txn.put(cf, "user:1", "Alice", -1);
txn.commit(); // myHook fires here

// Detach hook
cf.clearCommitHook();

```

Setting hook via config at creation time

```

auto cfConfig = tidesdb::ColumnFamilyConfig::defaultConfig();
cfConfig.commitHookFn = myHook;
cfConfig.commitHookCtx = nullptr;

db.createColumnFamily("replicated_cf", cfConfig);

```

Callback signature

```
int (*tidesdb_commit_hook_fn)(const tidesdb_commit_op_t* ops, int
num_ops,
                                uint64_t commit_seq, void* ctx);
```

The callback returns `0` on success. A non-zero return is logged as a warning but does not roll back the commit.

Operation struct fields (`tidesdb_commit_op_t`)

- `key` / `key_size` · Key bytes (valid only during callback)
- `value` / `value_size` · Value bytes (`NULL` / `0` for deletes, valid only during callback)
- `tll` · Time-to-live for the entry
- `is_delete` · `1` for delete, `0` for put

Behavior

- The hook fires after WAL write, memtable apply, and commit status marking are complete - data is fully durable before the callback runs
- Hook failure (non-zero return) is logged but does not affect the commit result
- Each column family has its own independent hook; a multi-CF transaction fires the hook once per CF with only that CF's operations
- `commit_seq` is monotonically increasing across commits and can be used as a replication cursor
- Pointers in `tidesdb_commit_op_t` are valid only during the callback invocation - copy any data you need to retain
- The hook executes synchronously on the committing thread; keep the callback fast to avoid stalling writers
- Setting the hook to `NULL` via `clearCommitHook()` disables it immediately with no restart required

Use cases

- Replication · Ship committed batches to replicas in commit order
- Event streaming · Publish mutations to Kafka, NATS, or any message broker
- Secondary indexing · Maintain a reverse index or materialized view
- Audit logging · Record every mutation with key, value, TTL, and sequence number
- Debugging · Attach a temporary hook in production to inspect live writes

The `commitHookFn` and `commitHookCtx` config fields are not persisted to `config.ini`. After a database restart, hooks must be re-registered by the application. This is by design - function pointers cannot be serialized.

Backup

Create an on-disk snapshot of an open database without blocking normal reads/writes.

```
db.backup("./mydb_backup");
```

Behavior

- Requires `dir` to be a non-existent directory or an empty directory
- Does not copy the `LOCK` file, so the backup can be opened normally
- Two-phase copy approach:
 - Copies immutable files first (SSTables listed in the manifest plus metadata/config files)
 - Forces memtable flushes, waits for flush/compaction queues to drain, then copies remaining files
- Database stays open and usable during backup

Checkpoint

Create a lightweight, near-instant snapshot of an open database using hard links instead of copying SSTable data.

```
db.checkpoint("./mydb_checkpoint");
```

Behavior

- Requires `dir` to be a non-existent directory or an empty directory
- For each column family:
 - Flushes the active memtable so all data is in SSTables
 - Halts compactions to ensure a consistent view of live SSTable files
 - Hard links all SSTable files (`.klog` and `.vlog`) into the checkpoint directory
 - Copies small metadata files (manifest, config) into the checkpoint directory
 - Resumes compactions
- Falls back to file copy if hard linking fails (e.g., cross-filesystem)

- Database stays open and usable during checkpoint

Checkpoint vs Backup

	backup	checkpoint
Speed	Copies every SSTable byte-by-byte	Near-instant (hard links, O(1) per file)
Disk usage	Full independent copy	No extra disk until compaction removes old SSTables
Portability	Can be moved to another filesystem or machine	Same filesystem only (hard link requirement)
Use case	Archival, disaster recovery, remote shipping	Fast local snapshots, point-in-time reads, streaming backups

Notes

- The checkpoint can be opened as a normal TidesDB database with `TidesDB(config)`
- Hard-linked files share storage with the live database. Deleting the original database does not affect the checkpoint (hard link semantics)

Block Cache Statistics

Get statistics for the global block cache (shared across all column families).

```
auto cacheStats = db.getCacheStats();

if (cacheStats.enabled) {
    std::cout << "Cache enabled: yes" << std::endl;
    std::cout << "Total entries: " << cacheStats.totalEntries << std::endl;
    std::cout << "Total bytes: " << cacheStats.totalBytes << std::endl;
    std::cout << "Hits: " << cacheStats.hits << std::endl;
    std::cout << "Misses: " << cacheStats.misses << std::endl;
    std::cout << "Hit rate: " << (cacheStats.hitRate * 100.0) << "%" <<
std::endl;
    std::cout << "Partitions: " << cacheStats.numPartitions << std::endl;
} else {
    std::cout << "Cache disabled (block_cache_size = 0)" << std::endl;
}
```

Cache statistics fields

- `enabled` · Whether block cache is active

- `totalEntries` · Number of cached blocks
- `totalBytes` · Total memory used by cached blocks
- `hits` · Number of cache hits
- `misses` · Number of cache misses
- `hitRate` · Hit rate as a decimal (0.0 to 1.0)
- `numPartitions` · Number of cache partitions

Range Cost Estimation

Estimate the computational cost of iterating between two keys in a column family. The returned value is an opaque double - meaningful only for comparison with other values from the same function. It uses only in-memory metadata and performs no disk I/O.

```
auto cf = db.getColumnFamily("my_cf");

double costA = cf.rangeCost("user:0000", "user:0999");
double costB = cf.rangeCost("user:1000", "user:1099");

if (costA < costB) {
    std::cout << "Range A is cheaper to iterate" << std::endl;
}
```

Key order does not matter - the function normalizes the range so `keyA > keyB` produces the same result as `keyB > keyA`.

How it works

- With block indexes enabled · Uses $O(\log B)$ binary search per overlapping SSTable to estimate block span
- Without block indexes · Falls back to byte-level key interpolation against SSTable min/max keys
- B+tree SSTables (`useBtree=true`) · Uses key interpolation against tree node counts, plus tree height as a seek cost
- Compressed SSTables receive a $1.5\times$ weight multiplier for decompression overhead
- Each overlapping SSTable adds a small fixed cost for merge-heap operations
- The active memtable's entry count contributes a small in-memory cost

Use cases

- Query planning · Compare candidate key ranges to find the cheapest one to scan

- Load balancing · Distribute range scan work across threads by estimating per-range cost
- Adaptive prefetching · Decide how aggressively to prefetch based on range size
- Monitoring · Track how data distribution changes across key ranges over time

A cost of 0.0 means no overlapping SSTables or memtable entries were found for the range.

Sync Modes

Control the durability vs performance tradeoff with three sync modes.

```
auto cfConfig = tidesdb::ColumnFamilyConfig::defaultConfig();

// Fastest, least durable (OS handles flushing)
cfConfig.syncMode = tidesdb::SyncMode::None;

// Balanced performance with periodic background syncing
cfConfig.syncMode = tidesdb::SyncMode::Interval;
cfConfig.syncIntervalUs = 128000; // Sync every 128ms (default)

// Most durable (fsync on every write)
cfConfig.syncMode = tidesdb::SyncMode::Full;

db.createColumnFamily("my_cf", cfConfig);
```

Sync Mode Details

- **SyncMode::None** · No explicit sync, relies on OS page cache (fastest, least durable)
 - Best for · Maximum throughput, acceptable data loss on crash
 - Use case · Caches, temporary data, reproducible workloads
- **SyncMode::Interval** · Periodic background syncing at configurable intervals (balanced)
 - Best for · Production workloads requiring good performance with bounded data loss
 - Use case · Most applications, configurable durability window
 - Single background sync thread monitors all column families using interval mode
 - Structural operations (flush, compaction, WAL rotation) always enforce durability
 - At most `syncIntervalUs` worth of data at risk on crash
- **SyncMode::Full** · Fsync on every write operation (slowest, most durable)
 - Best for · Critical data requiring maximum durability

- Use case · Financial transactions, audit logs, critical metadata

Sync Interval Examples

```
// Sync every 100ms (good for low-latency requirements)
cfConfig.syncMode = tidesdb::SyncMode::Interval;
cfConfig.syncIntervalUs = 100000;

// Sync every 128ms (default)
cfConfig.syncMode = tidesdb::SyncMode::Interval;
cfConfig.syncIntervalUs = 128000;

// Sync every 1 second (higher throughput, more data at risk)
cfConfig.syncMode = tidesdb::SyncMode::Interval;
cfConfig.syncIntervalUs = 1000000;
```

Regardless of sync mode, TidesDB **always** enforces durability for structural operations: memtable flush to SSTable, SSTable compaction and merging, WAL rotation, and column family metadata updates.

Manual WAL Sync

`syncWal()` forces an immediate fsync of the active write-ahead log for a column family. This is useful for explicit durability control when using `SyncMode::None` or `SyncMode::Interval`.

```
auto cf = db.getColumnFamily("my_cf");

// Force WAL durability after a batch of writes
cf.syncWal();
```

When to use

- Application-controlled durability · Sync the WAL at specific points (e.g., after a batch of related writes) when using `SyncMode::None` or `SyncMode::Interval`
- Pre-checkpoint · Ensure all buffered WAL data is on disk before taking a checkpoint
- Graceful shutdown · Flush WAL buffers before closing the database
- Critical writes · Force durability for specific high-value writes without using `SyncMode::Full` for all writes

Behavior

- Acquires a reference to the active memtable to safely access its WAL
- Calls `fdatsync` on the WAL file descriptor
- Thread-safe — can be called concurrently from multiple threads
- If the memtable rotates during the call, retries with the new active memtable

Object Store Mode

Object store mode allows TidesDB to store SSTables in a remote object store (S3, MinIO, GCS, or any S3-compatible service) while using local disk as a cache. This separates compute from storage and enables cold start recovery from the remote store. Object store mode requires unified memtable mode and is automatically enforced when a connector is set.

Enabling Object Store Mode (Filesystem Connector)

```
#include <tidesdb/tidesdb.hpp>

// create a filesystem connector (for testing and local replication)
tidesdb_objstore_t* store =
tidesdb::objstore::filesystem("/mnt/nfs/tidesdb-objects");

auto osCfg = tidesdb::ObjectStoreConfig::defaultConfig();

tidesdb::Config config;
config.dbPath = "./mydb";
config.objectStore = store;
config.objectStoreConfig = osCfg;

tidesdb::TidesDB db(config);

// use the database normally -- SSTables are uploaded after flush
```

Connector ownership

A connector returned by `tidesdb::objstore::filesystem`, `tidesdb::objstore::s3`, or the raw C factories is owned by the database once it opens successfully — the database frees it on close, so do not destroy it yourself. Build the connector immediately before opening: if `TidesDB` construction throws, the connector is leaked.

Enabling Object Store Mode (S3/MinIO Connector)

Build with `-DTIDESDB_WITH_S3=ON` to enable the S3 connector. This requires libcurl and OpenSSL.

```
#include <tidesdb/tidesdb.hpp>

tidesdb_objstore_t* s3 = tidesdb::objstore::s3(
    "s3.amazonaws.com",           // endpoint
    "my-tidesdb-bucket",          // bucket
    "production/db1/",            // key prefix (or "" for
none)
    "AKIAIOSFODNN7EXAMPLE",       // access key
    "wJalrXUtnFEMI/K7MDENG/bPxrFcYEXAMPLEKEY", // secret key
    "us-east-1",                  // region ("" for MinIO)
    true,                          // use SSL (HTTPS)
    false                          // path-style URLs (false
for AWS, true for MinIO)
);

auto osCfg = tidesdb::ObjectStoreConfig::defaultConfig();
osCfg.localCacheMaxBytes = 512 * 1024 * 1024; // 512MB local cache
osCfg.maxConcurrentUploads = 8;

tidesdb::Config config;
config.dbPath = "./mydb";
config.objectStore = s3;
config.objectStoreConfig = osCfg;

tidesdb::TidesDB db(config);
```

MinIO Example

```
tidesdb_objstore_t* minio = tidesdb::objstore::s3(
    "localhost:9000",    // MinIO endpoint
    "tidesdb-bucket",   // bucket
    "",                 // no key prefix
    "minioadmin",       // access key
    "minioadmin",       // secret key
    "",                 // region (" " for MinIO)
    false,              // no SSL for local dev
    true                // path-style URLs (required for MinIO)
);
```

Advanced S3 Configuration (TLS and Multipart Tuning)

The positional `objstore::s3` overload covers the common case. For custom TLS settings or multipart tuning, pass a `tidesdb::objstore::S3Config` instead. Zero/empty fields fall back to secure defaults (TLS verification on, the system CA bundle, and the library's built-in multipart sizes).

```
tidesdb::objstore::S3Config cfg;
cfg.endpoint = "minio.internal:9000";
cfg.bucket = "tidesdb-bucket";
cfg.accessKey = "minioadmin";
cfg.secretKey = "minioadmin";
cfg.usePathStyle = true;
cfg.tlsCaPath = "/etc/ssl/certs/internal-ca.pem"; // custom CA bundle
cfg.multipartThreshold = 128 * 1024 * 1024;      // 128MB multipart
threshold
cfg.multipartPartSize = 16 * 1024 * 1024;        // 16MB chunks

tidesdb_objstore_t* s3 = tidesdb::objstore::s3(cfg);
```

`S3Config` fields

- `endpoint` · S3 endpoint (required)
- `bucket` · Bucket name (required)
- `prefix` · Key prefix, or empty for none
- `accessKey` · AWS access key ID (required)
- `secretKey` · AWS secret access key (required)
- `region` · AWS region, or empty for the default

- `useSsl` · true for HTTPS, false for HTTP (default: true)
- `usePathStyle` · true for path-style URLs / MinIO (default: false)
- `tlsCaPath` · Custom CA bundle file path, or empty for the system bundle
- `tlsInsecureSkipVerify` · true disables TLS verification (test only, insecure; default: false)
- `multipartThreshold` · Object size at/above which multipart upload is used (0 = library default)
- `multipartPartSize` · Multipart chunk size in bytes (0 = library default)

Both `objstore::s3` overloads require a `libtidesdb` built with `-DTIDESDB_WITH_S3=ON`; otherwise the connector symbols are unresolved at link time. Because the factories are header-inline, a build that never calls them links fine against an S3-less library.

Object Store Configuration

Use `ObjectStoreConfig::defaultConfig()` for sensible defaults, then override fields as needed.

Configuration fields

- `localCachePath` · Local directory for cached SSTable files (empty = use `db_path`)
- `localCacheMaxBytes` · Maximum local cache size in bytes (0 = unlimited)
- `cacheOnRead` · Cache downloaded files locally (default: true)
- `cacheOnWrite` · Keep local copy after upload (default: true)
- `maxConcurrentUploads` · Number of parallel upload threads (default: 4)
- `maxConcurrentDownloads` · Number of parallel download threads (default: 8)
- `multipartThreshold` · Use multipart upload above this size (default: 64MB)
- `multipartPartSize` · Chunk size for multipart uploads (default: 8MB)
- `syncManifestToObject` · Upload MANIFEST after each compaction (default: true)
- `replicateWal` · Upload closed WAL segments for replication (default: true)
- `walUploadSync` · false = background WAL upload, true = block flush until uploaded (default: false)
- `walSyncThresholdBytes` · Sync active WAL to object store when it grows by this many bytes (default: 1MB, 0 = off)
- `walSyncOnCommit` · Upload WAL after every txn commit for RPO=0 replication (default: false)
- `replicaMode` · Enable read-only replica mode (default: false)
- `replicaSyncIntervalUs` · MANIFEST poll interval for replica sync in microseconds (default: 5s)
- `replicaReplayWal` · Replay WAL from object store for near-real-time reads on replicas (default: true)

Per-CF Object Store Tuning

Column family configurations include two object store tuning fields.

- `objectLazyCompaction` · 1 to compact less aggressively for remote storage (default: 0)
- `objectPrefetchCompaction` · 1 to download all inputs before compaction merge (default: 1)

Object Store Statistics

`getDbStats()` includes object store fields when a connector is active.

```
auto dbStats = db.getDbStats();

if (dbStats.objectStoreEnabled) {
    std::cout << "Connector: " << dbStats.objectStoreConnector <<
std::endl;
    std::cout << "Total uploads: " << dbStats.totalUploads << std::endl;
    std::cout << "Upload failures: " << dbStats.totalUploadFailures <<
std::endl;
    std::cout << "Upload queue depth: " << dbStats.uploadQueueDepth <<
std::endl;
    std::cout << "Local cache: " << dbStats.localCacheBytesUsed
        << " / " << dbStats.localCacheBytesMax << " bytes" <<
std::endl;
}
```

Cold Start Recovery

When the local database directory is empty but a connector is configured, TidesDB automatically discovers column families from the object store during recovery. It downloads MANIFEST and config files in parallel (one thread per CF), reconstructs the SSTable inventory, and fetches SSTable data on demand as queries arrive.

```
// delete all local state, then reopen with the same connector
tidesdb::Config config;
config.dbPath = "./mydb";
config.objectStore = s3;
config.objectStoreConfig = osCfg;

tidesdb::TidesDB db(config);

// all data is available -- SSTables are fetched from the object store on
demand
auto cf = db.getColumnFamily("my_cf");
```

How It Works

- Object store mode requires unified memtable mode. Setting `objectStore` on the config automatically enables `unifiedMemtable`
- After each flush, SSTables are uploaded via an asynchronous upload pipeline with retry (3 attempts with exponential backoff) and post-upload verification
- Point lookups on remote SSTables fetch just the single needed klog block (~64KB) via one HTTP range request, cached in the clock cache for subsequent reads
- Iterators prefetch all needed SSTable files in parallel at creation time using bounded threads (`maxConcurrentDownloads`, default 8)
- A hash-indexed LRU local file cache manages disk usage, evicting least-recently-used SSTable file pairs when `localCacheMaxBytes` is set
- The MANIFEST is uploaded asynchronously after each flush and compaction for cold start recovery
- The reaper thread periodically syncs the active WAL based on write volume (`walSyncThresholdBytes`, default 1MB)

The S3 connector requires libcurl and OpenSSL. Enable it with `-DTIDESDB_WITH_S3=ON` during CMake configuration. The filesystem connector is always available.

Replica Mode

Replica mode enables read-only nodes that follow a primary through the object store. The primary handles all writes while replicas poll for MANIFEST updates and replay WAL segments for near-real-time reads.

Enabling Replica Mode

```
auto osCfg = tidesdb::ObjectStoreConfig::defaultConfig();
osCfg.replicaMode = true;
osCfg.replicaSyncIntervalUs = 1000000; // 1 second sync interval
osCfg.replicaReplayWal = true;          // replay WAL for fresh reads

tidesdb::Config config;
config.dbPath = "./mydb_replica";
config.objectStore = s3; // same bucket as the primary
config.objectStoreConfig = osCfg;

tidesdb::TidesDB db(config);

// reads work normally
auto txn = db.beginTransaction();
auto cf = db.getColumnFamily("my_cf");
auto value = txn.get(cf, "key");

// writes are rejected with ErrorCode::Readonly
```

Sync-on-Commit WAL (Primary Side)

For tighter replication lag, enable sync-on-commit on the primary so every committed write is uploaded to the object store immediately.

```
auto osCfg = tidesdb::ObjectStoreConfig::defaultConfig();
osCfg.walSyncOnCommit = true; // RPO = 0, every commit is durable in S3

// replica sees committed data within one replicaSyncIntervalUs
```

Promoting a Replica to Primary

When the primary fails, promote a replica to accept writes.


```
// external health check detects primary is down
db.promoteToPrimary();

// now writes are accepted
auto txn = db.beginTransaction();
auto cf = db.getColumnFamily("my_cf");
txn.put(cf, "key", "value", -1);
txn.commit();
```

`promoteToPrimary()` performs a final MANIFEST sync and WAL replay, creates a local WAL for crash recovery, and atomically switches to primary mode. Throws `ErrorCode::InvalidArgs` if the node is already a primary.

i Monitoring single-writer fencing

`getDbStats()` exposes the lease epochs used to fence stale primaries. `DbStats::primaryEpoch` is the lease epoch the local node holds as primary (0 when it is not a primary or holds no lease), and `DbStats::seenEpoch` is the highest lease epoch a replica has observed. A promotion that takes the lease bumps `primaryEpoch`; a primary that has been fenced by a newer promotion sees `replicaMode` flip back to `true`. Compare these across nodes to detect split-brain or a fenced primary.

How Replica Sync Works

- The reaper thread polls the remote MANIFEST for each CF every `replicaSyncIntervalUs`
- New SSTables from the primary's flushes and compactions are added to the replica's levels
- SSTables compacted away on the primary are removed from the replica's levels
- When `replicaReplayWal` is enabled, the latest WAL is downloaded and replayed into the memtable for near-real-time reads
- WAL replay is idempotent using sequence numbers so entries already present are skipped
- SSTable data is not downloaded during sync — it is fetched on demand via `range_get` for point lookups or prefetch for iterators

Compression Algorithms

TidesDB supports multiple compression algorithms:

```

auto cfConfig = tidesdb::ColumnFamilyConfig::defaultConfig();

cfConfig.compressionAlgorithm = tidesdb::CompressionAlgorithm::None;    //
No compression
cfConfig.compressionAlgorithm = tidesdb::CompressionAlgorithm::LZ4;      //
LZ4 standard (default)
cfConfig.compressionAlgorithm = tidesdb::CompressionAlgorithm::LZ4Fast;  //
LZ4 fast mode
cfConfig.compressionAlgorithm = tidesdb::CompressionAlgorithm::Zstd;     //
Zstandard
#ifdef __sun
cfConfig.compressionAlgorithm = tidesdb::CompressionAlgorithm::Snappy;    //
Snappy (not available on SunOS)
#endif

db.createColumnFamily("my_cf", cfConfig);

```

Choosing a Compression Algorithm

Workload	Recommended Algorithm	Rationale
General purpose	<code>CompressionAlgorithm::LZ4</code>	Best balance of speed and compression
Write-heavy	<code>CompressionAlgorithm::LZ4Fast</code>	Minimize CPU overhead on writes
Storage-constrained	<code>CompressionAlgorithm::Zstd</code>	Maximum compression ratio
Read-heavy	<code>CompressionAlgorithm::Zstd</code>	Reduce I/O bandwidth, decompression is fast
Pre-compressed data	<code>CompressionAlgorithm::None</code>	Avoid double compression overhead
CPU-constrained	<code>CompressionAlgorithm::None</code> or <code>LZ4Fast</code>	Minimize CPU usage

Compression algorithm can be changed at runtime via `updateRuntimeConfig`, but the change only affects **new** SSTables. Existing SSTables retain their original compression and are decompressed correctly during reads. Different column families can use different compression algorithms.

B+tree KLog Format (Optional)

Column families can optionally use a B+tree structure for the key log instead of the default block-based format. The B+tree klog format offers faster point lookups through $O(\log N)$ tree traversal rather than linear block scanning.

```
auto cfConfig = tidesdb::ColumnFamilyConfig::defaultConfig();
cfConfig.useBtree = true; // Enable B+tree klog format

db.createColumnFamily("btree_cf", cfConfig);
```

Characteristics

- Point lookups · $O(\log N)$ tree traversal with binary search at each node
- Range scans · Doubly-linked leaf nodes enable efficient bidirectional iteration
- Immutable · Tree is bulk-loaded from sorted memtable data during flush
- Compression · Nodes compress independently using the same algorithms (LZ4, LZ4-FAST, Zstd)
- Large values · Values exceeding `klogValueThreshold` are stored in vlog, same as block-based format
- Bloom filter · Works identically - checked before tree traversal

When to use B+tree klog format

- Read-heavy workloads with frequent point lookups
- Workloads where read latency is more important than write throughput
- Large SSTables where block scanning becomes expensive

Tradeoffs

- Slightly higher write amplification during flush (building tree structure)
- Larger metadata overhead per node compared to block-based format
- Block-based format may be faster for sequential scans of entire SSTables

Important · `useBtree` **cannot be changed** after column family creation. Different column families can use different formats.

Error Handling

The C++ wrapper uses exceptions for error handling. All errors throw `tidesdb::Exception`.

```

try {
    auto cf = db.getColumnFamily("my_cf");

    auto txn = db.beginTransaction();
    txn.put(cf, "key", "value", -1);
    txn.commit();
} catch (const tidesdb::Exception& e) {
    std::cerr << "Error: " << e.what() << std::endl;
    std::cerr << "Code: " << static_cast<int>(e.code()) << std::endl;
}

```

Error Codes

- `ErrorCode::Success` (0) · Operation successful
- `ErrorCode::Memory` (-1) · Memory allocation failed
- `ErrorCode::InvalidArgs` (-2) · Invalid arguments
- `ErrorCode::NotFound` (-3) · Key not found
- `ErrorCode::IO` (-4) · I/O error
- `ErrorCode::Corruption` (-5) · Data corruption
- `ErrorCode::Exists` (-6) · Resource already exists
- `ErrorCode::Conflict` (-7) · Transaction conflict
- `ErrorCode::TooLarge` (-8) · Key or value too large
- `ErrorCode::MemoryLimit` (-9) · Memory limit exceeded
- `ErrorCode::InvalidDB` (-10) · Invalid database handle
- `ErrorCode::Unknown` (-11) · Unknown error
- `ErrorCode::Locked` (-12) · Database is locked
- `ErrorCode::ReadOnly` (-13) · Database is read-only
- `ErrorCode::Busy` (-14) · Resource is temporarily busy
- `ErrorCode::Precondition` (-15) · A required precondition was not met

Error categories

- `ErrorCode::Corruption` indicates data integrity issues requiring immediate attention
- `ErrorCode::Conflict` indicates transaction conflicts (retry may succeed)
- `ErrorCode::Memory`, `ErrorCode::MemoryLimit`, `ErrorCode::TooLarge` indicate resource constraints
- `ErrorCode::NotFound`, `ErrorCode::Exists` are normal operational conditions, not failures

Complete Example

```
#include <tidesdb/tidesdb.hpp>
#include <iostream>
#include <ctime>

int main() {
    try {
        tidesdb::Config config;
        config.dbPath = "./example_db";
        config.numFlushThreads = 1;
        config.numCompactionThreads = 1;
        config.logLevel = tidesdb::LogLevel::Info;
        config.blockCacheSize = 64 * 1024 * 1024;
        config.maxOpenSSTables = 256;

        tidesdb::TidesDB db(config);

        auto cfConfig = tidesdb::ColumnFamilyConfig::defaultConfig();
        cfConfig.writeBufferSize = 64 * 1024 * 1024;
        cfConfig.compressionAlgorithm = tidesdb::CompressionAlgorithm::LZ4;
        cfConfig.enableBloomFilter = true;
        cfConfig.bloomFPR = 0.01;
        cfConfig.syncMode = tidesdb::SyncMode::Interval;
        cfConfig.syncIntervalUs = 128000;

        db.createColumnFamily("users", cfConfig);

        auto cf = db.getColumnFamily("users");

        {
            auto txn = db.beginTransaction();
            txn.put(cf, "user:1", "Alice", -1);
            txn.put(cf, "user:2", "Bob", -1);

            auto ttl = std::time(nullptr) + 30;
            txn.put(cf, "session:abc", "temp_data", ttl);

            txn.commit();
        }

        {
            auto txn = db.beginTransaction();
            auto value = txn.get(cf, "user:1");
```

```

        std::string valueStr(value.begin(), value.end());
        std::cout << "user:1 = " << valueStr << std::endl;
    }

    {
        auto txn = db.beginTransaction();
        auto iter = txn.newIterator(cf);

        std::cout << "\nAll entries:" << std::endl;
        iter.seekToFirst();
        while (iter.valid()) {
            auto key = iter.key();
            auto value = iter.value();
            std::string keyStr(key.begin(), key.end());
            std::string valueStr(value.begin(), value.end());
            std::cout << "  " << keyStr << " = " << valueStr <<
std::endl;
            iter.next();
        }
    }

    auto stats = cf.getStats();
    std::cout << "\nColumn Family Statistics:" << std::endl;
    std::cout << "  Number of Levels: " << stats.numLevels <<
std::endl;
    std::cout << "  Memtable Size: " << stats.memtableSize << " bytes"
<< std::endl;
    std::cout << "  Uses B+tree: " << (stats.useBtree ? "yes" : "no")
<< std::endl;

    db.dropColumnFamily("users");

} catch (const tidesdb::Exception& e) {
    std::cerr << "Error: " << e.what() << std::endl;
    return 1;
}

return 0;
}

```

Isolation Levels

TidesDB supports five MVCC isolation levels:

```
auto txn =  
db.beginTransaction(tidesdb::IsolationLevel::ReadCommitted);  
txn.commit();
```

Available Isolation Levels

- `IsolationLevel::ReadUncommitted` · Sees all data including uncommitted changes
- `IsolationLevel::ReadCommitted` · Sees only committed data (default)
- `IsolationLevel::RepeatableRead` · Consistent snapshot, phantom reads possible
- `IsolationLevel::Snapshot` · Write-write conflict detection
- `IsolationLevel::Serializable` · Full read-write conflict detection (SSI)

Savepoints

Savepoints allow partial rollback within a transaction:

```
auto txn = db.beginTransaction();  
  
txn.put(cf, "key1", "value1", -1);  
  
txn.savepoint("sp1");  
txn.put(cf, "key2", "value2", -1);  
  
txn.rollbackToSavepoint("sp1");  
  
txn.releaseSavepoint("sp1");  
  
txn.commit();
```

Savepoint API

- `savepoint(name)` · Create a savepoint
- `rollbackToSavepoint(name)` · Rollback to savepoint
- `releaseSavepoint(name)` · Release savepoint without rolling back

Savepoint behavior

- Savepoints capture the transaction state at a specific point

- Rolling back to a savepoint discards all operations after that savepoint
- Releasing a savepoint frees its resources without rolling back
- Multiple savepoints can be created with different names
- Creating a savepoint with an existing name updates that savepoint
- Savepoints are automatically freed when the transaction commits or rolls back

Transaction Reset

`Transaction::reset` resets a committed or aborted transaction for reuse with a new isolation level. This avoids the overhead of freeing and reallocating transaction resources in hot loops.

```
auto cf = db.getColumnFamily("my_cf");

auto txn = db.beginTransaction();

txn.put(cf, "key1", "value1", -1);
txn.commit();

txn.reset(tidesdb::IsolationLevel::ReadCommitted);

txn.put(cf, "key2", "value2", -1);
txn.commit();
```

Behavior

- The transaction must be committed or aborted before reset; resetting an active transaction throws an exception
- Internal buffers are retained to avoid reallocation
- A fresh transaction ID and snapshot sequence are assigned based on the new isolation level
- The isolation level can be changed on each reset (e.g., `ReadCommitted` to `RepeatableRead`)

When to use

- Batch processing · Reuse a single transaction across many commit cycles in a loop
- Connection pooling · Reset a transaction for a new request without reallocation
- High-throughput ingestion · Reduce malloc/free overhead in tight write loops

Reset after rollback


```
auto txn = db.beginTransaction();

txn.put(cf, "key", "value", -1);
txn.rollback();

txn.reset(tidesdb::IsolationLevel::ReadCommitted);
txn.put(cf, "new_key", "new_value", -1);
txn.commit();
```

Multi-Column-Family Transactions

TidesDB supports atomic transactions across multiple column families with true all-or-nothing semantics.

```
auto usersCf = db.getColumnFamily("users");
auto ordersCf = db.getColumnFamily("orders");

auto txn = db.beginTransaction();

txn.put(usersCf, "user:1000", "John Doe", -1);

txn.put(ordersCf, "order:5000", "user:1000|product:A", -1);

txn.commit();
```

Multi-CF guarantees

- Either all CFs commit or none do (atomic)
- Automatically detected when operations span multiple CFs
- Uses global sequence numbers for atomic ordering

Default Configuration

Get default configurations for database and column families.

```
auto defaultDbConfig = tidesdb::TidesDB::defaultConfig();
defaultDbConfig.dbPath = "./mydb";
tidesdb::TidesDB db(defaultDbConfig);

auto defaultCfConfig = tidesdb::ColumnFamilyConfig::defaultConfig();
db.createColumnFamily("my_cf", defaultCfConfig);
```

`defaultConfig()` (both DB and CF variants) now mirrors the underlying C library defaults rather than duplicating constants in the binding. Fields like `maxConcurrentFlushes`, `tombstoneDensityMinEntries`, and the unified-memtable settings track the engine automatically. If you previously relied on hardcoded defaults when overriding only some fields, re-test after upgrading.

Configuration Persistence

Load and save column family configuration from/to INI files.

```
auto config = tidesdb::ColumnFamilyConfig::loadFromIni("config.ini",
"my_cf");
db.createColumnFamily("my_cf", config);

auto cfConfig = tidesdb::ColumnFamilyConfig::defaultConfig();
cfConfig.writeBufferSize = 128 * 1024 * 1024;
tidesdb::ColumnFamilyConfig::saveToIni("config.ini", "my_cf", cfConfig);
```

Custom Comparators

Register custom comparators for controlling key sort order.

```

int myReverseCompare(const uint8_t* key1, size_t key1_size,
                    const uint8_t* key2, size_t key2_size, void* ctx) {
    (void)ctx;
    int result = memcmp(key1, key2, std::min(key1_size, key2_size));
    if (result == 0) {
        return (key1_size < key2_size) ? 1 : (key1_size > key2_size) ? -1 :
0;
    }
    return -result; // Reverse order
}

db.registerComparator("reverse", myReverseCompare);

auto cfConfig = tidesdb::ColumnFamilyConfig::defaultConfig();
cfConfig.comparatorName = "reverse";
db.createColumnFamily("reverse_cf", cfConfig);

```

Built-in comparators

- `"memcmp"` (default) · Binary byte-by-byte comparison
- `"lexicographic"` · Null-terminated string comparison
- `"uint64"` · Unsigned 64-bit integer comparison
- `"int64"` · Signed 64-bit integer comparison
- `"reverse"` · Reverse binary comparison
- `"case_insensitive"` · Case-insensitive ASCII comparison

Select a built-in comparator for a column family by setting its registered name:

```

auto cfConfig = tidesdb::ColumnFamilyConfig::defaultConfig();
cfConfig.comparatorName = "uint64"; // 8-byte big-endian integer keys
db.createColumnFamily("metrics", cfConfig);

```

The built-in comparator function pointers are also exposed directly under `tidesdb::comparators`, which is handy for re-registering one under a custom name or referencing it in your own code:

- `tidesdb::comparators::memcmp`
- `tidesdb::comparators::lexicographic`
- `tidesdb::comparators::uint64`
- `tidesdb::comparators::int64`

- `tidesdb::comparators::reverseMemcmp` (registered name `"reverse"`)
- `tidesdb::comparators::caseInsensitive` (registered name `"case_insensitive"`)

```
// Re-register a built-in under your own name
db.registerComparator("desc", tidesdb::comparators::reverseMemcmp);
```

Comparator context string

`ColumnFamilyConfig::comparatorCtxStr` carries an optional context string that is persisted to `config.ini` alongside `comparatorName`. Use it to parameterize a comparator (for example, an endianness or collation hint) so the setting survives a restart and a `loadFromIni` round-trip.

```
auto cfConfig = tidesdb::ColumnFamilyConfig::defaultConfig();
cfConfig.comparatorName = "uint64";
cfConfig.comparatorCtxStr = "endian=big";
db.createColumnFamily("ts_cf", cfConfig);
```

Important

Once a comparator is set for a column family, it **cannot be changed** without corrupting data.

Retrieving a Registered Comparator

Use `getComparator` to retrieve a previously registered comparator by name:

```
tidesdb_comparator_fn fn = nullptr;
void* ctx = nullptr;

db.getComparator("timestamp_desc", &fn, &ctx);
// fn and ctx are now populated
```

Use cases

- Validation · Check if a comparator is registered before creating a column family
- Debugging · Verify comparator registration during development
- Dynamic configuration · Query available comparators at runtime

Thread Pools

TidesDB uses separate thread pools for flush and compaction operations. Understanding the parallelism model is important for optimal configuration.

Parallelism semantics

- Cross-CF parallelism · Multiple flush/compaction workers CAN process different column families in parallel
- Within-CF serialization · A single column family can only have one flush and one compaction running at any time (enforced by atomic `isFlushing` and `isCompacting` flags)
- No intra-CF memtable parallelism · Even if a CF has multiple immutable memtables queued, they are flushed sequentially

Thread pool sizing guidance

- Single column family · Set `numFlushThreads = 1` and `numCompactionThreads = 1`. Additional threads provide no benefit since only one operation per CF can run at a time
- Multiple column families · Set thread counts up to the number of column families for maximum parallelism. With N column families and M workers (where $M \leq N$), throughput scales linearly

Configuration

```
tidesdb::Config config;
config.dbPath = "./mydb";
config.numFlushThreads = 2;           // Flush thread pool size
                                        (default: 2)
config.numCompactionThreads = 2;      // Compaction thread pool
                                        size (default: 2)
config.blockCacheSize = 64 * 1024 * 1024; // 64MB global block cache
                                        (default: 64MB)
config.maxOpenSSTables = 256;         // LRU cache for SSTable
                                        objects (default: 256, each has 2 FDs)

tidesdb::TidesDB db(config);
```

`maxOpenSSTables` is a **storage-engine-level** configuration, not a column family configuration. It controls the LRU cache size for SSTable structures. Each SSTable uses 2 file descriptors (klog + vlog), so 256 SSTables = 512 file descriptors.

Testing



```
cmake -S . -B build -DTIDESDB_CPP_BUILD_TESTS=ON
cmake --build build

cd build
ctest --output-on-failure
```

TidesDB C# API Reference

Getting Started

Prerequisites

You **must** have the TidesDB shared C library installed on your system. You can find the installation instructions [here](#).

Installation

Install the package from NuGet:



```
dotnet add package TidesDB
```

Or build from source:



```
git clone https://github.com/tidesdb/tidesdb-cs.git
cd tidesdb-cs
dotnet build
```

Custom Installation Paths

If you installed TidesDB to a non-standard location, you can specify custom paths using environment variables:



```
# Linux
export LD_LIBRARY_PATH="/custom/path/lib:$LD_LIBRARY_PATH"

# macOS
export DYLD_LIBRARY_PATH="/custom/path/lib:$DYLD_LIBRARY_PATH"

# Windows (add to PATH)
set PATH=C:\custom\path\bin;%PATH%
```

Custom prefix installation



```
# Install TidesDB to custom location
cd tidesdb
cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/opt/tidesdb
cmake --build build
sudo cmake --install build

# Configure environment to use custom location
export LD_LIBRARY_PATH="/opt/tidesdb/lib:$LD_LIBRARY_PATH" # Linux
# or
export DYLD_LIBRARY_PATH="/opt/tidesdb/lib:$DYLD_LIBRARY_PATH" # macOS
```


Usage

Opening and Closing a Database

```
using TidesDB;

var config = new Config
{
    DbPath = "./mydb",
    NumFlushThreads = 2,
    NumCompactionThreads = 2,
    LogLevel = LogLevel.Info,
    BlockCacheSize = 64 * 1024 * 1024,
    MaxOpenSstables = 256,
    MaxMemoryUsage = 0, // 0 = auto (50% of system RAM)
    LogToFile = false,
    LogTruncationAt = 0,
    MaxConcurrentFlushes = 0, // 0 = library default (set by
tidesdb_default_config)
};

using var db = TidesDb.Open(config);
Console.WriteLine("Database opened successfully");
```

MaxConcurrentFlushes

`MaxConcurrentFlushes` is a global semaphore on the number of in-flight memtable flushes across all column families. It bounds total transient memory and work-queue depth when many column families flush at once. Leave it at the default (sourced from `tidesdb_default_config`) to inherit the library's value, or lower it on memory-constrained hosts to throttle peak flush concurrency. `Config.Default(path)` and the parameterless `Config { ... }` initializer both pull `MaxConcurrentFlushes` from the C library, so anyone overriding the previous hardcoded values should re-test.

Opening with Unified Memtable

Unified memtable mode uses a single shared memtable across all column families instead of per-CF memtables.

```
using TidesDB;

var config = new Config
{
    DbPath = "./mydb",
    NumFlushThreads = 2,
    NumCompactionThreads = 2,
    LogLevel = LogLevel.Info,
    BlockCacheSize = 64 * 1024 * 1024,
    MaxOpenSstables = 256,
    UnifiedMemtable = true,
    UnifiedMemtableWriteBufferSize = 128 * 1024 * 1024,
    UnifiedMemtableSkipListMaxLevel = 12,
    UnifiedMemtableSkipListProbability = 0.25f,
    UnifiedMemtableSyncMode = SyncMode.None,
    UnifiedMemtableSyncIntervalUs = 0,
};

using var db = TidesDb.Open(config);
```

Opening with Object Store

Object store mode stores SSTables in a remote object store while using local disk as a cache. It automatically enables unified memtable mode.

Filesystem connector (testing/local replication)

```

using TidesDB;

var config = new Config
{
    DbPath = "./mydb",
    NumFlushThreads = 2,
    NumCompactionThreads = 2,
    LogLevel = LogLevel.Info,
    BlockCacheSize = 64 * 1024 * 1024,
    MaxOpenSstables = 256,
    ObjectStoreConfig = new ObjectStoreConfig
    {
        ConnectorType = ObjectStoreConnectorType.Filesystem,
        FsRootDir = "/mnt/nfs/tidesdb-objects",
        LocalCacheMaxBytes = 512 * 1024 * 1024, // 512MB local cache
        MaxConcurrentUploads = 8,
        MaxConcurrentDownloads = 16,
    },
};

using var db = TidesDb.Open(config);
// SSTables are uploaded after flush

```

Replica mode

```

using TidesDB;

var config = new Config
{
    DbPath = "./mydb_replica",
    ObjectStoreConfig = new ObjectStoreConfig
    {
        ConnectorType = ObjectStoreConnectorType.Filesystem,
        FsRootDir = "/mnt/nfs/tidesdb-objects", // same store as primary
        ReplicaMode = true,
        ReplicaSyncIntervalUs = 1_000_000, // 1 second sync interval
        ReplicaReplayWal = true,
    },
};

using var db = TidesDb.Open(config);
// reads work normally, writes return TDB_ERR_READONLY

```

Promoting a replica to primary

```

// External health check detects primary is down
db.PromoteToPrimary();
// Now writes are accepted

```

When the object store enforces conditional writes, promotion acquires the primary lease by advancing its epoch with a compare-and-swap. If another node already holds the lease the call is refused with `ErrorCode.Precondition` (the node stays a working replica for an orchestrator to retry); it throws `ErrorCode.InvalidArgs` if the node is already a primary. The active lease epoch is observable through `DbStats.PrimaryEpoch` (held by this node) and `DbStats.SeenEpoch` (highest observed).

```
try
{
    db.PromoteToPrimary();
}
catch (TidesDBException ex) when (ex.ErrorCode == ErrorCode.Precondition)
{
    // Another node won the promotion race; remain a replica and let the
    orchestrator retry.
}
```

Object Store Configuration Options

Property	Type	Default	Description
ConnectorType	ObjectStoreConnectorType	Filesystem	Connector type (Filesystem or S3)
FsRootDir	string?	null	Root directory for filesystem connector
LocalCachePath	string?	null	Local directory for cached SSTable files (null = use db_path)
LocalCacheMaxBytes	ulong	0	Maximum local cache size in bytes (0 = unlimited)
CacheOnRead	bool	true	Cache downloaded files locally
CacheOnWrite	bool	true	Keep local copy after upload
MaxConcurrentUploads	int	4	Number of parallel upload threads
MaxConcurrentDownloads	int	8	Number of parallel download threads
MultipartThreshold	ulong	64MB	Use multipart upload above this size
MultipartPartSize	ulong	8MB	Chunk size for multipart uploads
SyncManifestToObject	bool	true	Upload MANIFEST after each compaction
ReplicateWal	bool	true	Upload closed WAL segments for replication
WalUploadSync	bool	false	Block flush until WAL is uploaded
WalSyncThresholdBytes	ulong	1MB	Sync active WAL when it grows by this many bytes (0 = off)
WalSyncOnCommit	bool	false	Upload WAL after every txn commit for RPO=0
ReplicaMode	bool	false	Enable read-only replica mode
ReplicaSyncIntervalUs	ulong	5000000	MANIFEST poll interval for replica sync in microseconds
ReplicaReplayWal	bool	true	Replay WAL from object store for near-real-time reads

Object Store Mode

Object store mode automatically enables unified memtable mode. The S3 connector requires TidesDB built with `-DTIDESDB_WITH_S3=ON`. The filesystem connector is always available.

S3-compatible connector (AWS S3, MinIO, ...)

```
using TidesDB;

var config = new Config
{
    DbPath = "./mydb",
    ObjectStoreConfig = new ObjectStoreConfig
    {
        ConnectorType = ObjectStoreConnectorType.S3,
        S3Endpoint = "s3.amazonaws.com",    // or "localhost:9000" for MinIO
        S3Bucket = "my-tidesdb",
        S3KeyPrefix = "production/db1/",    // optional
        S3AccessKey = "AKIA...",
        S3SecretKey = "...",
        S3Region = "us-east-1",             // null for MinIO
        S3UseSsl = true,
        S3UsePathStyle = false,             // true for MinIO
        LocalCacheMaxBytes = 512 * 1024 * 1024,
    },
};

using var db = TidesDb.Open(config);
```

S3 Connector Options

Property	Type	Default	Description
<code>S3Endpoint</code>	string?	null	S3 endpoint (required for S3). e.g. <code>s3.amazonaws.com</code> or <code>localhost:9000</code>
<code>S3Bucket</code>	string?	null	Bucket name (required for S3)
<code>S3KeyPrefix</code>	string?	null	Key prefix (optional), e.g. <code>production/db1/</code>
<code>S3AccessKey</code>	string?	null	AWS access key ID (required for S3)
<code>S3SecretKey</code>	string?	null	AWS secret access key (required for S3)
<code>S3Region</code>	string?	null	AWS region, e.g. <code>us-east-1</code> (null for MinIO)
<code>S3UseSsl</code>	bool	true	Use HTTPS
<code>S3UsePathStyle</code>	bool	false	Path-style URLs (set true for MinIO)
<code>S3TlsCaPath</code>	string?	null	Custom CA bundle file path (null = system bundle)
<code>S3TlsInsecureSkipVerify</code>	bool	false	Disable TLS peer/host verification (test only, insecure)
<code>S3MultipartThreshold</code>	ulong	0	Connector multipart threshold in bytes (0 = library default)
<code>S3MultipartPartSize</code>	ulong	0	Connector multipart chunk size in bytes (0 = library default)

Setting any of `S3TlsCaPath`, `S3TlsInsecureSkipVerify`, `S3MultipartThreshold`, or `S3MultipartPartSize` routes connector creation through the full S3 config factory (`tidesdb_objstore_s3_create_config`); otherwise the positional factory (`tidesdb_objstore_s3_create`) is used.

TLS verification

`S3TlsInsecureSkipVerify = true` disables certificate verification and is for local testing only — never use it against a production endpoint.

Creating and Dropping Column Families

Column families are isolated key-value stores with independent configuration.


```

using TidesDB;

db.CreateColumnFamily("my_cf");

db.CreateColumnFamily("my_cf", new ColumnFamilyConfig
{
    WriteBufferSize = 128 * 1024 * 1024,
    LevelSizeRatio = 10,
    MinLevels = 5,
    CompressionAlgorithm = CompressionAlgorithm.Lz4,
    EnableBloomFilter = true,
    BloomFpr = 0.01,
    EnableBlockIndexes = true,
    SyncMode = SyncMode.Interval,
    SyncIntervalUs = 128000,
    DefaultIsolationLevel = IsolationLevel.ReadCommitted,
    TombstoneDensityTrigger = 0.0,          // Per-SSTable tombstone density
above which compaction priority escalates (0.0 = disabled, range 0.0 to
1.0)
    TombstoneDensityMinEntries = 1024,      // Minimum entry count for an
SSTable to be considered by the density trigger
    UseBtree = false, // Use B+tree format for klog (default: false =
block-based)
});

db.DropColumnFamily("my_cf");

// Or delete by pointer (faster when you already hold a ColumnFamily
handle)
var cf = db.GetColumnFamily("my_cf")!;
db.DeleteColumnFamily(cf);

```

Cloning a Column Family

Create a complete copy of an existing column family with a new name. The clone contains all the data from the source at the time of cloning.

```
using TidesDB;

db.CreateColumnFamily("source_cf");

// Clone the column family
db.CloneColumnFamily("source_cf", "cloned_cf");

var original = db.GetColumnFamily("source_cf");
var clone = db.GetColumnFamily("cloned_cf");
```

Use cases

- Testing · Create a copy of production data for testing without affecting the original
- Branching · Create a snapshot of data before making experimental changes
- Migration · Clone data before schema or configuration changes

Renaming a Column Family

Atomically rename a column family and its underlying directory.

```
using TidesDB;

db.CreateColumnFamily("old_name");

db.RenameColumnFamily("old_name", "new_name");

var cf = db.GetColumnFamily("new_name");
```

CRUD Operations

All operations in TidesDB are performed through transactions for ACID guarantees.

Writing Data

```
var cf = db.GetColumnFamily("my_cf");

using var txn = db.BeginTransaction();

txn.Put(cf, Encoding.UTF8.GetBytes("key"), Encoding.UTF8.GetBytes("value"),
-1);

txn.Commit();
```

Writing with TTL

```
var cf = db.GetColumnFamily("my_cf");

using var txn = db.BeginTransaction();

var ttl = DateTimeOffset.UtcNow.ToUnixTimeSeconds() + 10;

txn.Put(cf, Encoding.UTF8.GetBytes("temp_key"),
Encoding.UTF8.GetBytes("temp_value"), ttl);

txn.Commit();
```

TTL Examples

```
long ttl = -1;

long ttl = DateTimeOffset.UtcNow.ToUnixTimeSeconds() + 5 * 60;

long ttl = DateTimeOffset.UtcNow.ToUnixTimeSeconds() + 60 * 60;

long ttl = new DateTimeOffset(2026, 12, 31, 23, 59, 59,
TimeSpan.Zero).ToUnixTimeSeconds();
```

Reading Data

```
var cf = db.GetColumnFamily("my_cf");

using var txn = db.BeginTransaction();

var value = txn.Get(cf, Encoding.UTF8.GetBytes("key"));
if (value != null)
{
    Console.WriteLine($"Value: {Encoding.UTF8.GetString(value)}");
}
```

Deleting Data

```
var cf = db.GetColumnFamily("my_cf");

using var txn = db.BeginTransaction();

txn.Delete(cf, Encoding.UTF8.GetBytes("key"));

txn.Commit();
```

Single-Delete

`Transaction.SingleDelete` writes a tombstone with the same read semantics as `Delete`, but carries a caller-provided promise that lets compaction drop the put and the tombstone together as soon as both appear in the same merge input, rather than carrying the tombstone forward until it reaches the largest active level.

Between any two single-deletes on the same key, and between the start of the key's history and its first single-delete, the key has been put **at most once**. The engine does not and cannot verify this at runtime; violating the contract can leave older puts visible after the single-delete and is a bug in the caller.

This is the right choice for workloads that insert each key exactly once and then delete it exactly once (classic insert-benchmark patterns, secondary-index entries on columns that are never updated, log-style tables with scheduled purges). It is **not** safe for tables that issue repeated updates to the same key.

```
var cf = db.GetColumnFamily("my_cf");

using var txn = db.BeginTransaction();

txn.SingleDelete(cf, Encoding.UTF8.GetBytes("key"));

txn.Commit();
```

When in doubt, prefer `Transaction.Delete`.

Multi-Operation Transactions

```
var cf = db.GetColumnFamily("my_cf");

using var txn = db.BeginTransaction();

try
{
    txn.Put(cf, Encoding.UTF8.GetBytes("key1"),
Encoding.UTF8.GetBytes("value1"), -1);
    txn.Put(cf, Encoding.UTF8.GetBytes("key2"),
Encoding.UTF8.GetBytes("value2"), -1);
    txn.Delete(cf, Encoding.UTF8.GetBytes("old_key"));

    txn.Commit();
}
catch
{
    txn.Rollback();
    throw;
}
```

Iterating Over Data

Iterators provide efficient bidirectional traversal over key-value pairs.

Forward Iteration

```
var cf = db.GetColumnFamily("my_cf")!;

using var txn = db.BeginTransaction();
using var iter = txn.NewIterator(cf);

iter.SeekToFirst();

while (iter.Valid())
{
    var key = iter.Key();
    var value = iter.Value();

    Console.WriteLine($"Key: {Encoding.UTF8.GetString(key)}, Value: {Encoding.UTF8.GetString(value)}");

    iter.Next();
}
```

Backward Iteration

```
var cf = db.GetColumnFamily("my_cf")!;

using var txn = db.BeginTransaction();
using var iter = txn.NewIterator(cf);

iter.SeekToLast();

while (iter.Valid())
{
    var key = iter.Key();
    var value = iter.Value();

    Console.WriteLine($"Key: {Encoding.UTF8.GetString(key)}, Value: {Encoding.UTF8.GetString(value)}");

    iter.Prev();
}
```

Seek to Specific Key

```
var cf = db.GetColumnFamily("my_cf");

using var txn = db.BeginTransaction();
using var iter = txn.NewIterator(cf);

// Seek to first key >= target
iter.Seek(Encoding.UTF8.GetBytes("user:1000"));

if (iter.Valid())
{
    Console.WriteLine($"Found: {Encoding.UTF8.GetString(iter.Key())}");
}
```

Seek for Previous

```
var cf = db.GetColumnFamily("my_cf");

using var txn = db.BeginTransaction();
using var iter = txn.NewIterator(cf);

// Seek to last key <= target
iter.SeekForPrev(Encoding.UTF8.GetBytes("user:2000"));

while (iter.Valid())
{
    Console.WriteLine($"Key: {Encoding.UTF8.GetString(iter.Key())}");
    iter.Prev();
}
```

Combined Key-Value Retrieval

Retrieve both key and value in a single native call, more efficient than calling `Key()` and `Value()` separately.

```

var cf = db.GetColumnFamily("my_cf")!;

using var txn = db.BeginTransaction();
using var iter = txn.NewIterator(cf);

iter.SeekToFirst();

while (iter.Valid())
{
    var (key, value) = iter.KeyValue();

    Console.WriteLine($"Key: {Encoding.UTF8.GetString(key)}, Value:
{Encoding.UTF8.GetString(value)}");

    iter.Next();
}

```

Prefix Scanning

```

var cf = db.GetColumnFamily("my_cf")!;

using var txn = db.BeginTransaction();
using var iter = txn.NewIterator(cf);

var prefix = "user:";
iter.Seek(Encoding.UTF8.GetBytes(prefix));

while (iter.Valid())
{
    var key = Encoding.UTF8.GetString(iter.Key());

    if (!key.StartsWith(prefix)) break;

    Console.WriteLine($"Found: {key}");
    iter.Next();
}

```

Getting Column Family Statistics

Retrieve detailed statistics about a column family.


```

var cf = db.GetColumnFamily("my_cf")!;

var stats = cf.GetStats();

Console.WriteLine($"Number of Levels: {stats.NumLevels}");
Console.WriteLine($"Memtable Size: {stats.MemtableSize} bytes");
Console.WriteLine($"Total Keys: {stats.TotalKeys}");
Console.WriteLine($"Total Data Size: {stats.TotalDataSize} bytes");
Console.WriteLine($"Avg Key Size: {stats.AvgKeySize:F1} bytes");
Console.WriteLine($"Avg Value Size: {stats.AvgValueSize:F1} bytes");
Console.WriteLine($"Read Amplification: {stats.ReadAmp:F2}");
Console.WriteLine($"Cache Hit Rate: {stats.HitRate * 100:F1}%");

for (int i = 0; i < stats.NumLevels; i++)
{
    Console.WriteLine($"Level {i + 1}: {stats.LevelNumSstables[i]}
SSTables, {stats.LevelSizes[i]} bytes, {stats.LevelKeyCounts[i]} keys");
}

if (stats.UseBtree)
{
    Console.WriteLine($"B+tree Total Nodes: {stats.BtreeTotalNodes}");
    Console.WriteLine($"B+tree Max Height: {stats.BtreeMaxHeight}");
    Console.WriteLine($"B+tree Avg Height: {stats.BtreeAvgHeight:F2}");
}

// Tombstone density observability
Console.WriteLine($"Total Tombstones: {stats.TotalTombstones}");
Console.WriteLine($"Tombstone Ratio: {stats.TombstoneRatio * 100:F2}%");
Console.WriteLine($"Worst SSTable Density: {stats.MaxSstDensity * 100:F2}%
at level {stats.MaxSstDensityLevel}");
for (int i = 0; i < stats.NumLevels; i++)
{
    Console.WriteLine($"Level {i + 1} tombstones:
{stats.LevelTombstoneCounts[i]}");
}

if (stats.Config != null)
{
    Console.WriteLine($"Tombstone Density Trigger:
{stats.Config.TombstoneDensityTrigger}");
    Console.WriteLine($"Tombstone Density Min Entries:
{stats.Config.TombstoneDensityMinEntries}");
}

```

Listing Column Families

```
var cfList = db.ListColumnFamilies();

Console.WriteLine("Available column families:");
foreach (var name in cfList)
{
    Console.WriteLine($"  - {name}");
}
```

Updating Column Family Configuration

Update runtime-safe configuration settings for a column family. Changes apply to new operations only.

```
var cf = db.GetColumnFamily("my_cf")!;

cf.UpdateRuntimeConfig(new ColumnFamilyConfig
{
    WriteBufferSize = 256 * 1024 * 1024,
    SkipListMaxLevel = 16,
    SkipListProbability = 0.25f,
    BloomFpr = 0.001,
    IndexSampleRatio = 8,
});
```

Without persisting to disk

```
cf.UpdateRuntimeConfig(new ColumnFamilyConfig
{
    WriteBufferSize = 256 * 1024 * 1024,
}, persistToDisk: false);
```

Updatable settings

- `WriteBufferSize` · Memtable flush threshold
- `SkipListMaxLevel` · Skip list level for **new** memtables
- `SkipListProbability` · Skip list probability for **new** memtables

- `BloomFpr` · False positive rate for **new** SSTables
- `EnableBloomFilter` · Enable/disable bloom filters for **new** SSTables
- `EnableBlockIndexes` · Enable/disable block indexes for **new** SSTables
- `BlockIndexPrefixLen` · Block index prefix length for **new** SSTables
- `IndexSampleRatio` · Index sampling ratio for **new** SSTables
- `CompressionAlgorithm` · Compression for **new** SSTables (existing SSTables retain their original compression)
- `KlogValueThreshold` · Value log threshold for **new** writes
- `SyncMode` · Durability mode. Also updates the active WAL's sync mode immediately
- `SyncIntervalUs` · Sync interval in microseconds (only used when SyncMode is Interval)
- `LevelSizeRatio` · LSM level sizing
- `MinLevels` · Minimum LSM levels
- `DividingLevelOffset` · Compaction dividing level offset
- `L1FileCountTrigger` · L1 file count compaction trigger
- `L0QueueStallThreshold` · Backpressure stall threshold
- `DefaultIsolationLevel` · Default transaction isolation level
- `MinDiskSpace` · Minimum disk space required

Non-updatable settings

- `ComparatorName` · Cannot change sort order after creation
- `UseBtree` · Cannot change klog format after creation

Important Notes

Changes apply immediately to new operations. Existing SSTables and memtables retain their original settings. The update operation is thread-safe.

Saving and Loading Configuration (INI)

A `ColumnFamilyConfig` can be persisted to a named section of an INI file and reloaded later. Keys not present in the file fall back to the library defaults, so partial files are valid.

```
var cfg = new ColumnFamilyConfig
{
    WriteBufferSize = 32 * 1024 * 1024,
    CompressionAlgorithm = CompressionAlgorithm.Zstd,
    BloomFpr = 0.005,
    UseBtree = true,
};

// Write to (or update) a section in the INI file
cfg.SaveToIni("cf-config.ini", "metrics");

// Read it back and use it to create a column family
var loaded = ColumnFamilyConfig.LoadFromIni("cf-config.ini", "metrics");
db.CreateColumnFamily("metrics", loaded);
```

Compaction

Manual Compaction

```
var cf = db.GetColumnFamily("my_cf");

cf.Compact();
```

Targeted Range Compaction

`CompactRange` runs a synchronous compaction over a specific key range. Only SSTables whose minimum and maximum keys overlap the requested range participate in the merge, so the work and I/O are bounded to the affected portion of the LSM tree rather than the whole column family.

```

var cf = db.GetColumnFamily("my_cf")!;

var start = Encoding.UTF8.GetBytes("tenant_42:");
var end   = Encoding.UTF8.GetBytes("tenant_42;");

cf.CompactRange(start, end);

// Either endpoint may be empty/default to indicate "unbounded on that
side":
cf.CompactRange(default, end); // compact everything up to "tenant_42;"
cf.CompactRange(start, default); // compact everything from "tenant_42:"
forward

```

When to use

- Bulk reclaim after a large range delete, where waiting for natural compaction would leave tombstones and obsolete versions on disk
- Tenant eviction or sliding-window expiration that does not fit TTL semantics
- Post-import cleanup of a known key range loaded with `Put` followed by `Delete`
- Operational counterpart to the automatic tombstone density trigger (`TombstoneDensityTrigger`) when an operator wants reclaim now rather than at the next natural threshold crossing

Behavior

- Synchronous; blocks the calling thread until the merge commits or fails
- Does not enqueue work onto the compaction thread pool, the calling thread does the work
- Selects only SSTables whose key range overlaps the requested range using the column family's comparator; SSTables outside the range are not touched
- Applies the same emit-loop logic as background compactions (tombstone reclamation rules, single-delete pair cancellation, sequence-based deduplication, value recompression)
- Output SSTables are committed to the manifest atomically and old inputs are marked for deletion
- An empty `ReadOnlySpan<byte>` (or `default`) is treated as `NULL` and means “unbounded on that side”

Return values

- Returns normally on success

- Throws `TidesDBException` with `ErrorCode.InvalidArgs` if both endpoints are empty (use `Compact` for full CF compaction)
- Throws `TidesDBException` with `ErrorCode.Locked` if another compaction is running for the column family
- Throws `TidesDBException` with the underlying I/O or memory error code if the merge cannot complete

Manual Memtable Flush

```
var cf = db.GetColumnFamily("my_cf");

cf.FlushMemtable();
```

Purge Column Family

Force a synchronous flush and aggressive compaction for a single column family. Unlike `FlushMemtable` and `Compact` (which are non-blocking), `Purge` blocks until all flush and compaction I/O is complete.

```
var cf = db.GetColumnFamily("my_cf");

cf.Purge();
// All data is now flushed to SSTables and compacted
```

When to use

- Before backup or checkpoint · Ensure all data is on disk and compacted
- After bulk deletes · Reclaim space immediately by compacting away tombstones
- Manual maintenance · Force a clean state during a maintenance window
- Pre-shutdown · Ensure all pending work is complete before closing

Purge Database

Force a synchronous flush and aggressive compaction for **all** column families, then drain both the global flush and compaction queues.

```
db.Purge();
// All CFs flushed and compacted, all queues drained
```

Purge vs Manual Flush + Compact

`FlushMemtable` and `Compact` are non-blocking - they enqueue work and return immediately. `Purge` (on both `ColumnFamily` and `TidesDb`) is synchronous - it blocks until all work is complete. Use purge when you need a guarantee that all data is on disk and compacted before proceeding.

Sync Modes

Control the durability vs performance tradeoff.

```
using TidesDB;

db.CreateColumnFamily("fast_cf", new ColumnFamilyConfig
{
    SyncMode = SyncMode.None,
});

db.CreateColumnFamily("balanced_cf", new ColumnFamilyConfig
{
    SyncMode = SyncMode.Interval,
    SyncIntervalUs = 128000, // Sync every 128ms
});

db.CreateColumnFamily("durable_cf", new ColumnFamilyConfig
{
    SyncMode = SyncMode.Full,
});
```

Manual WAL Sync

`SyncWal` forces an immediate fsync of the active write-ahead log for a column family. Useful for explicit durability control when using `SyncMode.None` or `SyncMode.Interval`.

```
var cf = db.GetColumnFamily("my_cf");

// Force WAL durability after a batch of writes
cf.SyncWal();
```

When to use

- Application-controlled durability · Sync the WAL at specific points (e.g., after a batch of related writes) when using `SyncMode.None` or `SyncMode.Interval`
- Pre-checkpoint · Ensure all buffered WAL data is on disk before taking a checkpoint
- Graceful shutdown · Flush WAL buffers before closing the database
- Critical writes · Force durability for specific high-value writes without using `SyncMode.Full` for all writes

Behavior

- Acquires a reference to the active memtable to safely access its WAL
- Calls `fdatsync` on the WAL file descriptor
- Thread-safe - can be called concurrently from multiple threads

Compression Algorithms

TidesDB supports multiple compression algorithms:


```

using TidesDB;

db.CreateColumnFamily("no_compress", new ColumnFamilyConfig
{
    CompressionAlgorithm = CompressionAlgorithm.None,
});

db.CreateColumnFamily("lz4_cf", new ColumnFamilyConfig
{
    CompressionAlgorithm = CompressionAlgorithm.Lz4,
});

db.CreateColumnFamily("lz4_fast_cf", new ColumnFamilyConfig
{
    CompressionAlgorithm = CompressionAlgorithm.Lz4Fast,
});

db.CreateColumnFamily("zstd_cf", new ColumnFamilyConfig
{
    CompressionAlgorithm = CompressionAlgorithm.Zstd,
});

db.CreateColumnFamily("snappy_cf", new ColumnFamilyConfig
{
    CompressionAlgorithm = CompressionAlgorithm.Snappy, // Not available on
SunOS/Illumos/OmniOS
});

```

Whether a compression backend is actually compiled into the loaded native library depends on the build-time `-DTIDESDB_WITH_*` options. Query it up front instead of failing at compress/flush time:

```

if (!Library.IsCompressionAvailable(CompressionAlgorithm.Zstd))
{
    Console.WriteLine("Zstd not compiled into this libtidesdb build;
falling back to Lz4");
}

```

`CompressionAlgorithm.None` is always available.

B+tree KLog Format (Optional)

Column families can optionally use a B+tree structure for the key log instead of the default block-based format. The B+tree klog format offers faster point lookups through $O(\log N)$ tree traversal.

```
using TidesDB;

db.CreateColumnFamily("btree_cf", new ColumnFamilyConfig
{
    UseBtree = true,
    CompressionAlgorithm = CompressionAlgorithm.Lz4,
    EnableBloomFilter = true,
});
```

When to use B+tree klog format

- Read-heavy workloads with frequent point lookups
- Workloads where read latency is more important than write throughput
- Large SSTables where block scanning becomes expensive

Tradeoffs

- Slightly higher write amplification during flush
- Larger metadata overhead per node
- Block-based format may be faster for sequential scans



Important

`UseBtree` cannot be changed after column family creation.

Library Lifecycle and Open-File Limit

The static `Library` class exposes process-wide lifecycle and capability helpers. Initialization is optional — `TidesDb.Open` lazily initializes the library on first use — but you can drive it explicitly when you need work to happen before the first open (such as raising the descriptor ceiling).

```

using TidesDB;

// Optional explicit init (system allocator). Returns false if already
// initialized.
Library.Initialize();

// Raise the process open-file ceiling BEFORE opening a database, so the
// engine can size
// MaxOpenSstables to fit. Must be called before TidesDb.Open. Pass <= 0 to
// just report the
// current ceiling. A failed or partial raise is non-fatal.
long ceiling = Library.RaiseOpenFileLimit(65536);
Console.WriteLine($"open-file ceiling now {ceiling}");

using (var db = TidesDb.Open(Config.Default("./mydb")))
{
    // ... use the database ...
}

// Optional: finalize the library and reset the allocator after all
// databases are closed.
// After this, Library.Initialize() can be called again.
Library.Shutdown();

```

Note

`RaiseOpenFileLimit` is an explicit, opt-in operator action — TidesDB never raises the limit on its own. On POSIX it raises the `RLIMIT_NOFILE` soft limit toward the hard limit; on Windows it raises the CRT stdio cap (max 8192).

Cancelling Background Work

`CancelBackgroundWork` cancels background compaction db-wide: in-flight merges bail out safely and queued compaction is skipped. Flushes are unaffected, so durability is preserved. It blocks (bounded) until compaction is idle, and the effect is sticky for the session (reset on the next open). Call it right before disposing the database for a fast shutdown.

```
db.CancelBackgroundWork();
db.Dispose();
```

Error Handling

```
using TidesDB;

var cf = db.GetColumnFamily("my_cf")!;

using var txn = db.BeginTransaction();

try
{
    txn.Put(cf, Encoding.UTF8.GetBytes("key"),
Encoding.UTF8.GetBytes("value"), -1);
    txn.Commit();
}
catch (TidesDBException ex)
{
    Console.WriteLine($"Error code: {ex.ErrorCode}");
    Console.WriteLine($"Error message: {ex.Message}");

    txn.Rollback();
}
```

Error Codes

- `TDB_SUCCESS` (0) · Operation successful
- `TDB_ERR_MEMORY` (-1) · Memory allocation failed
- `TDB_ERR_INVALID_ARGS` (-2) · Invalid arguments
- `TDB_ERR_NOT_FOUND` (-3) · Key not found
- `TDB_ERR_IO` (-4) · I/O error
- `TDB_ERR_CORRUPTION` (-5) · Data corruption
- `TDB_ERR_EXISTS` (-6) · Resource already exists
- `TDB_ERR_CONFLICT` (-7) · Transaction conflict
- `TDB_ERR_TOO_LARGE` (-8) · Key or value too large
- `TDB_ERR_MEMORY_LIMIT` (-9) · Memory limit exceeded

- `TDB_ERR_INVALID_DB` (-10) · Invalid database handle
- `TDB_ERR_UNKNOWN` (-11) · Unknown error
- `TDB_ERR_LOCKED` (-12) · Database is locked
- `TDB_ERR_READONLY` (-13) · Database is read-only
- `TDB_ERR_BUSY` (-14) · Resource busy
- `TDB_ERR_PRECONDITION` (-15) · A conditional object-store write failed its precondition (HTTP 412). Raised by single-writer fencing — e.g. `PromoteToPrimary` when another node already holds the primary lease

These map to the `ErrorCode` enum (`ErrorCode.Locked`, `ErrorCode.Readonly`, `ErrorCode.Busy`, `ErrorCode.Precondition`, ...) exposed on `TidesDBException.ErrorCode`.

Complete Example

```
using System.Text;
using TidesDB;

var config = new Config
{
    DbPath = "./example_db",
    NumFlushThreads = 1,
    NumCompactionThreads = 1,
    LogLevel = LogLevel.Info,
    BlockCacheSize = 64 * 1024 * 1024,
    MaxOpenSstables = 256,
    MaxMemoryUsage = 0,
    LogToFile = false,
    LogTruncationAt = 0
};

using var db = TidesDb.Open(config);

try
{
    db.CreateColumnFamily("users", new ColumnFamilyConfig
    {
        WriteBufferSize = 64 * 1024 * 1024,
        CompressionAlgorithm = CompressionAlgorithm.Lz4,
        EnableBloomFilter = true,
        BloomFpr = 0.01,
        SyncMode = SyncMode.Interval,
        SyncIntervalUs = 128000,
    });

    var cf = db.GetColumnFamily("users")!;

    using (var txn = db.BeginTransaction())
    {
        txn.Put(cf, Encoding.UTF8.GetBytes("user:1"),
            Encoding.UTF8.GetBytes("Alice"), -1);
        txn.Put(cf, Encoding.UTF8.GetBytes("user:2"),
            Encoding.UTF8.GetBytes("Bob"), -1);

        // Temporary session with 30 second TTL
        var ttl = DateTimeOffset.UtcNow.ToUnixTimeSeconds() + 30;
```

```

        txn.Put(cf, Encoding.UTF8.GetBytes("session:abc"),
Encoding.UTF8.GetBytes("temp_data"), ttl);

        txn.Commit();
    }

    using (var txn = db.BeginTransaction())
    {
        var value = txn.Get(cf, Encoding.UTF8.GetBytes("user:1"));
        Console.WriteLine($"user:1 = {Encoding.UTF8.GetString(value!)}");

        // Iterate over all entries
        using var iter = txn.NewIterator(cf);

        Console.WriteLine("\nAll entries:");
        iter.SeekToFirst();
        while (iter.Valid())
        {
            var key = Encoding.UTF8.GetString(iter.Key());
            var val = Encoding.UTF8.GetString(iter.Value());
            Console.WriteLine($" {key} = {val}");
            iter.Next();
        }
    }

    var stats = cf.GetStats();

    Console.WriteLine("\nColumn Family Statistics:");
    Console.WriteLine($"  Number of Levels: {stats.NumLevels}");
    Console.WriteLine($"  Memtable Size: {stats.MemtableSize} bytes");

    db.DropColumnFamily("users");
}
catch (TidesDBException ex)
{
    Console.WriteLine($"TidesDB error: {ex.Message} (code:
{ex.ErrorCode})");
}

```

Isolation Levels

TidesDB supports five MVCC isolation levels:

```
using TidesDB;

using var txn = db.BeginTransaction(IsolationLevel.ReadCommitted);

txn.Commit();
```

Available Isolation Levels

- `IsolationLevel.ReadUncommitted` · Sees all data including uncommitted changes
- `IsolationLevel.ReadCommitted` · Sees only committed data (default)
- `IsolationLevel.RepeatableRead` · Consistent snapshot, phantom reads possible
- `IsolationLevel.Snapshot` · Write-write conflict detection
- `IsolationLevel.Serializable` · Full read-write conflict detection (SSI)

Savepoints

Savepoints allow partial rollback within a transaction. You can create named savepoints, rollback to them, or release them without rolling back.

```
var cf = db.GetColumnFamily("my_cf");

using var txn = db.BeginTransaction();

txn.Put(cf, Encoding.UTF8.GetBytes("key1"),
Encoding.UTF8.GetBytes("value1"), -1);

txn.Savepoint("sp1");
txn.Put(cf, Encoding.UTF8.GetBytes("key2"),
Encoding.UTF8.GetBytes("value2"), -1);

txn.RollbackToSavepoint("sp1");

txn.Commit();
```

Releasing a savepoint (frees resources without rolling back)


```
var cf = db.GetColumnFamily("my_cf");

using var txn = db.BeginTransaction();

txn.Put(cf, Encoding.UTF8.GetBytes("key1"),
Encoding.UTF8.GetBytes("value1"), -1);
txn.Savepoint("sp1");
txn.Put(cf, Encoding.UTF8.GetBytes("key2"),
Encoding.UTF8.GetBytes("value2"), -1);
txn.ReleaseSavepoint("sp1");
txn.Commit();
```

Savepoint behavior

- `Savepoint(name)` · Create a savepoint
- `RollbackToSavepoint(name)` · Rollback to savepoint, discarding all operations after it
- `ReleaseSavepoint(name)` · Release savepoint without rolling back
- Multiple savepoints can be created with different names
- Creating a savepoint with an existing name updates that savepoint
- Savepoints are automatically freed when the transaction commits or rolls back

Transaction Reset

Reset a committed or aborted transaction for reuse with a new isolation level. This avoids the overhead of freeing and reallocating transaction resources in hot loops.

```
var cf = db.GetColumnFamily("my_cf")!;

using var txn = db.BeginTransaction();

txn.Put(cf, Encoding.UTF8.GetBytes("key1"),
Encoding.UTF8.GetBytes("value1"), -1);
txn.Commit();

txn.Reset(IsolationLevel.ReadCommitted);

txn.Put(cf, Encoding.UTF8.GetBytes("key2"),
Encoding.UTF8.GetBytes("value2"), -1);
txn.Commit();
```

When to use

- Batch processing · Reuse a single transaction across many commit cycles in a loop
- Connection pooling · Reset a transaction for a new request without reallocation
- High-throughput ingestion · Reduce allocation overhead in tight write loops

Backup

Create an on-disk snapshot of an open database without blocking normal reads/writes.

```
using TidesDB;

using var db = TidesDb.Open(config);

db.Backup("./mydb_backup");
```

Behavior

- Requires the directory to be non-existent or empty
- Does not copy the LOCK file, so the backup can be opened normally
- Database stays open and usable during backup

Checkpoint

Create a lightweight, near-instant snapshot of an open database using hard links instead of copying SSTable data.

```
using TidesDB;  
  
using var db = TidesDb.Open(config);  
  
db.Checkpoint("./mydb_checkpoint");
```

Behavior

- Requires the directory to be non-existent or empty
- Uses hard links for SSTable files (near-instant, O(1) per file)
- Falls back to file copy if hard linking fails (e.g., cross-filesystem)
- Flushes memtables and halts compactions to ensure a consistent snapshot
- Database stays open and usable during checkpoint

Checkpoint vs Backup

	Backup	Checkpoint
Speed	Copies every SSTable byte-by-byte	Near-instant (hard links, O(1) per file)
Disk usage	Full independent copy	No extra disk until compaction removes old SSTables
Portability	Can be moved to another filesystem or machine	Same filesystem only (hard link requirement)
Use case	Archival, disaster recovery, remote shipping	Fast local snapshots, point-in-time reads, streaming backups

Notes

- The checkpoint can be opened as a normal TidesDB database with `TidesDb.Open`
- Hard-linked files share storage with the live database; deleting the original does not affect the checkpoint

Checking Flush/Compaction Status

Check if a column family currently has flush or compaction operations in progress.

```
var cf = db.GetColumnFamily("my_cf");

if (cf.IsFlushing())
{
    Console.WriteLine("Flush in progress");
}

if (cf.IsCompacting())
{
    Console.WriteLine("Compaction in progress");
}
```

Database-Level Statistics

Get aggregate statistics across the entire database instance.

```

var dbStats = db.GetDbStats();

Console.WriteLine($"Column families: {dbStats.NumColumnFamilies}");
Console.WriteLine($"Total memory: {dbStats.TotalMemory} bytes");
Console.WriteLine($"Resolved memory limit: {dbStats.ResolvedMemoryLimit}
bytes");
Console.WriteLine($"Memory pressure level: {dbStats.MemoryPressureLevel}");
Console.WriteLine($"Global sequence: {dbStats.GlobalSeq}");
Console.WriteLine($"Flush queue: {dbStats.FlushQueueSize} pending");
Console.WriteLine($"Compaction queue: {dbStats.CompactionQueueSize}
pending");
Console.WriteLine($"Total SSTables: {dbStats.TotalSstableCount}");
Console.WriteLine($"Total data size: {dbStats.TotalDataSizeBytes} bytes");
Console.WriteLine($"Open SSTable handles: {dbStats.NumOpenSstables}");
Console.WriteLine($"In-flight txn memory: {dbStats.TxnMemoryBytes} bytes");
Console.WriteLine($"Immutable memtables: {dbStats.TotalImmutableCount}");
Console.WriteLine($"Memtable bytes: {dbStats.TotalMemtableBytes}");

// Unified memtable stats
Console.WriteLine($"Unified memtable enabled:
{dbStats.UnifiedMemtableEnabled}");
Console.WriteLine($"Unified memtable bytes:
{dbStats.UnifiedMemtableBytes}");
Console.WriteLine($"Unified immutable count:
{dbStats.UnifiedImmutableCount}");
Console.WriteLine($"Unified is flushing: {dbStats.UnifiedIsFlushing}");

// Object store stats (when enabled)
Console.WriteLine($"Object store enabled: {dbStats.ObjectStoreEnabled}");
Console.WriteLine($"Object store connector:
{dbStats.ObjectStoreConnector}");
Console.WriteLine($"Replica mode: {dbStats.ReplicaMode}");

// Single-writer fencing (object store mode)
Console.WriteLine($"Primary epoch (held): {dbStats.PrimaryEpoch}");
Console.WriteLine($"Seen epoch (highest observed): {dbStats.SeenEpoch}");

```

Stack Allocated

Unlike column family `GetStats` (which heap-allocates), `GetDbStats` fills a caller-provided struct. No free is needed.

Cache Statistics

```
var cacheStats = db.GetCacheStats();

Console.WriteLine($"Cache enabled: {cacheStats.Enabled}");
Console.WriteLine($"Total entries: {cacheStats.TotalEntries}");
Console.WriteLine($"Total bytes: {cacheStats.TotalBytes / (1024.0 *
1024.0):F2} MB");
Console.WriteLine($"Hits: {cacheStats.Hits}");
Console.WriteLine($"Misses: {cacheStats.Misses}");
Console.WriteLine($"Hit rate: {cacheStats.HitRate * 100:F1}%");
Console.WriteLine($"Partitions: {cacheStats.NumPartitions}");
```

Range Cost Estimation

`RangeCost` estimates the computational cost of iterating between two keys in a column family. The returned value is an opaque double - meaningful only for comparison with other values from the same method. It uses only in-memory metadata and performs no disk I/O.

```
var cf = db.GetColumnFamily("my_cf")!;

var costA = cf.RangeCost(
    Encoding.UTF8.GetBytes("user:0000"),
    Encoding.UTF8.GetBytes("user:0999"));

var costB = cf.RangeCost(
    Encoding.UTF8.GetBytes("user:1000"),
    Encoding.UTF8.GetBytes("user:1099"));

if (costA < costB)
{
    Console.WriteLine("Range A is cheaper to iterate");
}
```

How it works

The function walks all SSTable levels and uses in-memory metadata to estimate how many blocks and entries fall within the given key range:

- With block indexes enabled · Uses $O(\log B)$ binary search per overlapping SSTable to find the block slots containing each key bound
- Without block indexes · Falls back to byte-level key interpolation against the SSTable's min/max key range
- B+tree SSTables (`UseBtree=true`) · Uses key interpolation against tree node counts, plus tree height as a seek cost
- Compression · Compressed SSTables receive a $1.5\times$ weight multiplier to account for decompression overhead
- Merge overhead · Each overlapping SSTable adds a small fixed cost for merge-heap operations
- Memtable · The active memtable's entry count contributes a small in-memory cost

Key order does not matter - the method normalizes the range so `keyA > keyB` produces the same result as `keyB > keyA`.

Use cases

- Query planning · Compare candidate key ranges to find the cheapest one to scan
- Load balancing · Distribute range scan work across threads by estimating per-range cost
- Adaptive prefetching · Decide how aggressively to prefetch based on range size
- Monitoring · Track how data distribution changes across key ranges over time

Cost Values

The returned cost is not an absolute measure (it does not represent milliseconds, bytes, or entry counts). It is a relative scalar - only meaningful when compared with other `RangeCost` results. A cost of 0.0 means no overlapping SSTables or memtable entries were found for the range.

Custom Comparators

TidesDB provides six built-in comparators that are automatically registered on database open:

- `"memcmp"` (default) · Binary byte-by-byte comparison
- `"lexicographic"` · Null-terminated string comparison
- `"uint64"` · Unsigned 64-bit integer comparison
- `"int64"` · Signed 64-bit integer comparison
- `"reverse"` · Reverse binary comparison
- `"case_insensitive"` · Case-insensitive ASCII comparison

Registering a Built-in Comparator Under a Custom Name

Register one of the built-in native comparators under an additional name. This runs at native speed with no managed transition on the comparison hot path:

```
db.RegisterBuiltInComparator("my_uint64", BuiltInComparator.Uint64);

db.CreateColumnFamily("metrics", new ColumnFamilyConfig
{
    ComparatorName = "my_uint64"
});
```

`BuiltInComparator` values: `Memcmp`, `Lexicographic`, `Uint64`, `Int64`, `ReverseMemcmp`, `CaseInsensitive`.

Registering a Custom Managed Comparator

Provide your own ordering as a C# delegate. The callback returns a negative value when the first key sorts before the second, zero when equal, and positive when after:

```
// Order by key length first, then by bytes (a total order).
db.RegisterComparator("by_length", (key1, key2) =>
{
    if (key1.Length != key2.Length) return key1.Length - key2.Length;
    return key1.SequenceCompareTo(key2);
});

db.CreateColumnFamily("len_cf", new ColumnFamilyConfig
{
    ComparatorName = "by_length"
});
```


Performance & threading

A managed comparator is invoked from native code on the hot path of **every** memtable and compaction comparison, potentially concurrently on background threads. It must be deterministic, total, thread-safe, and must not throw (an exception is caught and treated as “equal”). The key spans are valid only for the duration of the call — do not retain them. For standard key shapes prefer `RegisterBuiltInComparator`, which avoids the managed transition entirely. The delegate is kept alive for the lifetime of the database instance.

Getting a Registered Comparator

Check if a comparator is registered by name:

```
if (db.GetComparator("memcmp"))
{
    Console.WriteLine("memcmp comparator is registered");
}

if (!db.GetComparator("nonexistent"))
{
    Console.WriteLine("comparator not registered");
}
```

Use cases

- Validation · Check if a comparator is registered before creating a column family
- Debugging · Verify comparator registration during development
- Dynamic configuration · Query available comparators at runtime

Important

Comparators must be registered before creating column families that use them. Once set, a comparator cannot be changed for a column family.

Commit Hook (Change Data Capture)

`SetCommitHook` registers a callback that fires synchronously after every transaction commit on a column family. The hook receives the full batch of committed operations atomically, enabling real-time change data capture without WAL parsing or external log consumers.

```

var cf = db.GetColumnFamily("my_cf")!;

cf.SetCommitHook((ops, commitSeq) =>
{
    foreach (var op in ops)
    {
        if (op.IsDelete)
        {
            Console.WriteLine($"[{commitSeq}] DELETE
{Encoding.UTF8.GetString(op.Key)}");
        }
        else
        {
            Console.WriteLine($"[{commitSeq}] PUT
{Encoding.UTF8.GetString(op.Key)} = {Encoding.UTF8.GetString(op.Value!)}");
        }
    }
});

// Normal writes now trigger the hook automatically
using (var txn = db.BeginTransaction())
{
    txn.Put(cf, Encoding.UTF8.GetBytes("key1"),
Encoding.UTF8.GetBytes("value1"), -1);
    txn.Commit(); // hook fires here
}

// Detach hook
cf.ClearCommitHook();

```

Operation fields

Property	Type	Description
<code>Key</code>	byte[]	The key data
<code>Value</code>	byte[]?	The value data (null for deletes)
<code>Ttl</code>	long	Time-to-live (0 = no expiry)
<code>IsDelete</code>	bool	True if this is a delete operation

Behavior

- The hook fires after WAL write, memtable apply, and commit status marking are complete - the data is fully durable before the callback runs
- Hook exceptions are caught internally and do not affect the commit result
- Each column family has its own independent hook; a multi-CF transaction fires the hook once per CF with only that CF's operations
- `commitSeq` is monotonically increasing across commits and can be used as a replication cursor
- Data in `CommitOp` is copied from native memory - safe to retain after the callback returns
- The hook executes synchronously on the committing thread; keep the callback fast to avoid stalling writers
- Calling `clearCommitHook()` disables the hook immediately with no restart required

Use cases

- Replication · Ship committed batches to replicas in commit order
- Event streaming · Publish mutations to Kafka, NATS, or any message broker
- Secondary indexing · Maintain a reverse index or materialized view
- Audit logging · Record every mutation with key, value, TTL, and sequence number
- Debugging · Attach a temporary hook in production to inspect live writes

Runtime-Only

Commit hooks are not persisted across database restarts. After reopening a database, hooks must be re-registered by the application. This is by design - delegates cannot be serialized.

Testing



```
cd tidesdb-cs
dotnet build
dotnet test
```

C# Types

The package exports all necessary types for full C# support:

```

using TidesDB;

// Main classes
// -- TidesDb
// -- ColumnFamily
// -- Transaction
// -- Iterator
// -- TidesDBException

// Configuration classes
// -- Config
// -- ColumnFamilyConfig
// -- ObjectStoreConfig
// -- Stats
// -- DbStats
// -- CacheStats

// Change data capture
// -- CommitOp
// -- CommitHookHandler

// Comparators
// -- ComparatorFunction (delegate)

// Library lifecycle / capability helpers (static)
// -- Library.Initialize()
// -- Library.Shutdown()
// -- Library.RaiseOpenFileLimit(long desired)
// -- Library.IsCompressionAvailable(CompressionAlgorithm algorithm)

// Methods
// -- TidesDb.GetComparator(string name)
// -- TidesDb.RegisterComparator(string name, ComparatorFunction
comparator, string? ctxStr)
// -- TidesDb.RegisterBuiltInComparator(string name, BuiltInComparator
comparator, string? ctxStr)
// -- TidesDb.DeleteColumnFamily(ColumnFamily cf)
// -- TidesDb.Purge()
// -- TidesDb.CancelBackgroundWork()
// -- TidesDb.GetDbStats()
// -- TidesDb.PromoteToPrimary()
// -- ColumnFamily.UpdateRuntimeConfig(ColumnFamilyConfig config, bool
persistToDisk)
// -- ColumnFamily.Purge()

```

```
// -- ColumnFamily.SyncWal()
// -- ColumnFamilyConfig.SaveToIni(string iniFile, string sectionName)
// -- ColumnFamilyConfig.LoadFromIni(string iniFile, string sectionName)
// static
// -- Transaction.SingleDelete(ColumnFamily cf, ReadOnlySpan<byte> key)
// -- Iterator.KeyValue()

// Enums
// -- CompressionAlgorithm
// -- SyncMode
// -- LogLevel
// -- IsolationLevel
// -- ObjectStoreConnectorType
// -- BuiltInComparator
// -- ErrorCode
```

Configuration Reference

Database Configuration (Config)

Property	Type	Default	Description
DbPath	string	required	Path to the database directory
NumFlushThreads	int	2	Number of flush threads
NumCompactionThreads	int	2	Number of compaction threads
LogLevel	LogLevel	Info	Logging level
BlockCacheSize	ulong	64MB	Block cache size in bytes
MaxOpenSstables	ulong	256	Maximum number of open SSTables
MaxMemoryUsage	ulong	0	Global memory limit in bytes (0 = auto, 50% of system RAM; minimum: 5% of system RAM)
LogToFile	bool	false	Write debug logging to a file
LogTruncationAt	ulong	0	Log file truncation threshold (0 = no truncation)
UnifiedMemtable	bool	false	Enable unified memtable mode
UnifiedMemtableWriteBufferSize	ulong	0	Unified memtable write buffer size (0 = auto)
UnifiedMemtableSkipListMaxLevel	int	0	Skip list max level for unified memtable (0 = 12)
UnifiedMemtableSkipListProbability	float	0	Skip list probability for unified memtable (0 = 0.25)
UnifiedMemtableSyncMode	SyncMode	None	Sync mode for unified WAL
UnifiedMemtableSyncInterval	ulong	0	Sync interval for unified WAL in microseconds

Property	Type	Default	Description
<code>lus</code>			
<code>ObjectStoreConfig</code>	ObjectStoreConfig?	null	Object store behavior configuration (null = disabled)
<code>MaxConcurrentFlushes</code>	int	library default	Global semaphore on in-flight memtable flushes across all CFs (0 = library default). Default sourced from <code>tidesdb_default_config()</code> .
<code>FinishCompactionsOnClose</code>	bool	false	Close behavior. False cancels in-flight compactions at their next checkpoint for a fast shutdown (no data loss). True lets them run to completion before close returns.

Column Family Configuration (ColumnFamilyConfig)

Property	Type	Default	Description
WriteBufferSize	ulong	64MB	Memtable flush threshold
LevelSizeRatio	ulong	10	Level size multiplier
MinLevels	int	5	Minimum LSM levels
DividingLevelOffset	int	2	Compaction dividing level offset
KlogValueThreshold	ulong	512	Values larger than this go to vlog
CompressionAlgorithm	CompressionAlgorithm	Lz4	Compression algorithm
EnableBloomFilter	bool	true	Enable bloom filters
BloomFpr	double	0.01	Bloom filter false positive rate
EnableBlockIndexes	bool	true	Enable block indexes
IndexSampleRatio	int	1	Index sample ratio
BlockIndexPrefixLen	int	16	Block index prefix length
SyncMode	SyncMode	Full	Sync mode for durability
SyncIntervalUs	ulong	1000000	Sync interval in microseconds
ComparatorName	string	""	Comparator name (empty for default)
SkipListMaxLevel	int	12	Skip list max level
SkipListProbability	float	0.25	Skip list probability
DefaultIsolationLevel	IsolationLevel	ReadCommitted	Default transaction isolation

Property	Type	Default	Description
<code>L</code>			
<code>MinDiskSpace</code>	ulong	100MB	Minimum disk space required
<code>L1FileCountTrigger</code>	int	4	L1 file count trigger for compaction
<code>L0QueueStallThreshold</code>	int	20	L0 queue stall threshold
<code>TombstoneDensityTrigger</code>	double	0.0	Per-SSTable tombstone density ($\frac{\text{tombstone_count}}{\text{num_entries}}$) above which compaction priority escalates. Range [0.0, 1.0]. 0.0 disables the check.
<code>TombstoneDensityMinEntries</code>	ulong	1024	SSTables with fewer entries than this are ignored by the density trigger (prevents tiny-SSTable noise).
<code>UseBtree</code>	bool	false	Use B+tree format for klog
<code>ObjectLazyCompaction</code>	bool	false	Compact less aggressively in object store mode
<code>ObjectPrefetchCompaction</code>	bool	true	Download all inputs before merge in object store mode

Column Family Statistics (Stats)

Property	Type	Description
<code>NumLevels</code>	int	Number of LSM levels
<code>MemtableSize</code>	ulong	Current memtable size in bytes
<code>LevelSizes</code>	ulong[]	Array of per-level total sizes
<code>LevelNumSstables</code>	int[]	Array of per-level SSTable counts
<code>LevelKeyCounts</code>	ulong[]	Array of per-level key counts
<code>Config</code>	ColumnFamilyConfig?	Full column family configuration
<code>TotalKeys</code>	ulong	Total keys across memtable and all SSTables
<code>TotalDataSize</code>	ulong	Total data size (klog + vlog) in bytes
<code>AvgKeySize</code>	double	Estimated average key size in bytes
<code>AvgValueSize</code>	double	Estimated average value size in bytes
<code>ReadAmp</code>	double	Read amplification factor (point lookup cost)
<code>HitRate</code>	double	Block cache hit rate (0.0 to 1.0)
<code>UseBtree</code>	bool	Whether column family uses B+tree klog format
<code>BtreeTotalNodes</code>	ulong	Total B+tree nodes (only if UseBtree=true)
<code>BtreeMaxHeight</code>	uint	Maximum tree height (only if UseBtree=true)
<code>BtreeAvgHeight</code>	double	Average tree height (only if UseBtree=true)
<code>TotalTombstones</code>	ulong	Sum of tombstone counts across every SSTable in the column family
<code>TombstoneRatio</code>	double	$\frac{\text{TotalTombstones}}{\text{TotalKeys}}$ (0.0 if no keys). Range [0.0, 1.0].
<code>LevelTombstoneCounts</code>	ulong[]	Per-level tombstone counts (parallels <code>LevelKeyCounts</code>)
<code>MaxSstDensity</code>	double	Worst per-SSTable tombstone density observed (0.0 to 1.0)
<code>MaxSstDensityLevel</code>	int	1-based level index where <code>MaxSstDensity</code> was observed (0 if none)

Property	Type	Description
<code>WalBytesWritten</code>	ulong	Framed bytes appended to this CF's WAL (0 in unified mode). Lifetime since open
<code>FlushBytesWritten</code>	ulong	On-disk bytes this CF's flushes wrote to L0 SSTables
<code>CompactionBytesWritten</code>	ulong	On-disk bytes this CF's compactions wrote
<code>CompactionBytesRead</code>	ulong	On-disk bytes this CF's compactions read as input
<code>UserBytesWritten</code>	ulong	Logical key+value bytes committed to this CF (write-amplification denominator)
<code>FlushCount</code>	ulong	Flushed SSTables produced by this CF
<code>CompactionCount</code>	ulong	Compaction output SSTables produced by this CF

Per-CF write amplification = $(\text{WalBytesWritten} + \text{FlushBytesWritten} + \text{CompactionBytesWritten}) / \text{UserBytesWritten}$.

Database Statistics (DbStats)

Property	Type	Description
NumColumnFamilies	int	Number of column families
TotalMemory	ulong	System total memory
AvailableMemory	ulong	System available memory at open time
ResolvedMemoryLimit	ulong	Resolved memory limit (auto or configured)
MemoryPressureLevel	int	Current memory pressure (0=normal, 1=elevated, 2=high, 3=critical)
FlushPendingCount	int	Number of pending flush operations (queued + in-flight)
TotalMemtableBytes	long	Total bytes in active memtables across all CFs
TotalImmutableCount	int	Total immutable memtables across all CFs
TotalSstableCount	int	Total SSTables across all CFs and levels
TotalDataSizeBytes	ulong	Total data size (klog + vlog) across all CFs
NumOpenSstableS	int	Number of currently open SSTable file handles
GlobalSeq	ulong	Current global sequence number
TxnMemoryBytes	long	Bytes held by in-flight transactions
CompactionQueueSize	ulong	Number of pending compaction tasks
FlushQueueSize	ulong	Number of pending flush tasks in queue
UnifiedMemtableEnabled	bool	Whether unified memtable mode is active
UnifiedMemtableBytes	long	Bytes in unified active memtable

Property	Type	Description
<code>UnifiedImmutableCount</code>	int	Number of unified immutable memtables
<code>UnifiedIsFlushing</code>	bool	Whether unified memtable is currently flushing/rotating
<code>UnifiedNextCfIndex</code>	uint	Next CF index to be assigned in unified mode
<code>UnifiedWalGeneration</code>	ulong	Current unified WAL generation counter
<code>ObjectStoreEnabled</code>	bool	Whether object store mode is active
<code>ObjectStoreConnector</code>	string?	Connector name (“s3”, “gcs”, “fs”, etc.)
<code>LocalCacheBytesUsed</code>	ulong	Current local file cache usage in bytes
<code>LocalCacheBytesMax</code>	ulong	Configured maximum local cache size in bytes
<code>LocalCacheNumFiles</code>	int	Number of files tracked in local cache
<code>LastUploadedGeneration</code>	ulong	Highest WAL generation confirmed uploaded
<code>UploadQueueDepth</code>	ulong	Number of pending upload jobs in the queue
<code>TotalUploads</code>	ulong	Lifetime count of objects uploaded to object store
<code>TotalUploadFailures</code>	ulong	Lifetime count of permanently failed uploads
<code>ReplicaMode</code>	bool	Whether running in read-only replica mode
<code>PrimaryEpoch</code>	ulong	Single-writer fencing (object store mode): the lease epoch this primary currently holds (0 when not a primary / no lease)
<code>SeenEpoch</code>	ulong	Single-writer fencing (object store mode): the highest lease epoch this node has observed in the store
<code>UwalBytesWritten</code>	ulong	Framed bytes appended to the shared unified WAL (0 if unified mode off)

Property	Type	Description
<code>WalBytesWritten</code>	ulong	Per-CF WAL bytes summed across all column families
<code>FlushBytesWritten</code>	ulong	Flush output bytes summed across all column families
<code>CompactionBytesWritten</code>	ulong	Compaction output bytes summed across all column families
<code>CompactionBytesRead</code>	ulong	Compaction input bytes summed across all column families
<code>UserBytesWritten</code>	ulong	Logical committed bytes summed across all CFs (write-amplification denominator)
<code>FlushCount</code>	ulong	Flushed SSTables summed across all column families
<code>CompactionCount</code>	ulong	Compaction output SSTables summed across all column families

Db-wide write amplification = $(\text{UwalBytesWritten} + \text{walBytesWritten} + \text{FlushBytesWritten} + \text{CompactionBytesWritten}) / \text{UserBytesWritten}$.

TidesDB GO API Reference

Getting Started

Prerequisites

You **must** have the TidesDB shared C library installed on your system. You can find the installation instructions [here](#).

Installation



```
go get github.com/tidesdb/tidesdb-go
```

Custom Installation Paths

If you installed TidesDB to a non-standard location, you can specify custom paths using CGO environment variables:



```
# Set custom include and library paths
export CGO_CFLAGS="-I/custom/path/include"
export CGO_LDFLAGS="-L/custom/path/lib -ltidesdb"

# Then install/build
go get github.com/tidesdb/tidesdb-go
```

Custom prefix installation



```
# Install TidesDB to custom location
cd tidesdb
cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/opt/tidesdb
cmake --build build
sudo cmake --install build

# Configure Go to use custom location
export CGO_CFLAGS="-I/opt/tidesdb/include"
export CGO_LDFLAGS="-L/opt/tidesdb/lib -ltidesdb"
export LD_LIBRARY_PATH="/opt/tidesdb/lib:$LD_LIBRARY_PATH" # Linux
# or
export DYLD_LIBRARY_PATH="/opt/tidesdb/lib:$DYLD_LIBRARY_PATH" # macOS

go get github.com/tidesdb/tidesdb-go
```

Initialization

TidesDB supports **optional** explicit initialization. If not called, TidesDB auto-initializes with the system allocator on the first `Open`.

```
// Initialize TidesDB (optional - auto-initializes if not called)
err := tidesdb.Init()
if err != nil {
    log.Fatal(err)
}

// ... use TidesDB ...

// Finalize after all operations are complete (optional)
tidesdb.Finalize()
```

Auto-initialization

If `Init()` is not called, TidesDB will auto-initialize with the system allocator on the first call to `Open()`.

Usage

Opening and Closing a Database

```
package main

import (
    "fmt"
    "log"

    tidesdb "github.com/tidesdb/tidesdb-go"
)

func main() {
    config := tidesdb.Config{
        DBPath:          "./mydb",
        NumFlushThreads: 2,
        NumCompactionThreads: 2,
        LogLevel:         tidesdb.LogInfo,
        BlockCacheSize:   64 * 1024 * 1024,
        MaxOpenSSTables: 256,
        MaxMemoryUsage:   0, // Global memory limit
                        // in bytes (0 = auto, 80% of system RAM)
        LogToFile:        false, // Write logs to file
                        // instead of stderr
        LogTruncationAt: 24 * 1024 * 1024, // Log file truncation
                        // size (24MB default)
        MaxConcurrentFlushes: 0, // Cap on in-flight
                        // memtable flushes across all CFs (0 = library default)
        FinishCompactionsOnClose: false, // false = cancel in-
                        // flight compactions at next checkpoint for a fast close; true = let them
                        // finish
        ObjectStore:      nil, // Object store
                        // connector (nil = local only)
        ObjectStoreConfig: nil, // Object store
                        // behavior config (nil = defaults)
    }

    db, err := tidesdb.Open(config)
    if err != nil {
        log.Fatal(err)
    }
    defer db.Close()
}
```

```
    fmt.Println("Database opened successfully")
}
```

Default Configuration

Use `DefaultConfig()` to get a configuration with sensible defaults, then override specific fields as needed. The defaults are pulled from the underlying C library, so values such as `MaxConcurrentFlushes` and the unified-memtable settings track the engine's defaults automatically.

```
config := tidesdb.DefaultConfig()
config.DBPath = "./mydb"
config.LogLevel = tidesdb.LogWarn // Override: only warnings and errors

db, err := tidesdb.Open(config)
if err != nil {
    log.Fatal(err)
}
defer db.Close()
```

MaxConcurrentFlushes

`MaxConcurrentFlushes` is a global semaphore on the number of in-flight memtable flushes across all column families. It bounds total transient memory and work-queue depth when many column families flush at once. Leave it at `0` to inherit the library default (recommended), or lower it on memory-constrained hosts to throttle peak flush concurrency.

FinishCompactionsOnClose

`FinishCompactionsOnClose` controls close behavior. The default (`false`) cancels in-flight compactions at their next checkpoint for a fast shutdown — the merge discards its uncommitted output and leaves inputs intact, so no data is lost (recovery handles a mid-merge state the same way). Set it to `true` to let in-flight compactions run to completion before `Close` returns.

Raising the Open-File Limit

`RaiseOpenFileLimit` raises this process's open-file ceiling toward the requested number of descriptors so a database can keep more SSTables open. Because TidesDB sizes `MaxOpenSSTables` to fit this at open time, **call it before `Open`**. This is an explicit, opt-in operator action; TidesDB never raises the limit itself.

```
// Raise toward 65536 descriptors before opening.
ceiling := tidesdb.RaiseOpenFileLimit(65536)
fmt.Printf("Open-file ceiling in effect: %d\n", ceiling)

db, err := tidesdb.Open(config)
if err != nil {
    log.Fatal(err)
}
defer db.Close()
```

Behavior

- A `desired` value `<= 0` just reports the current ceiling without changing it
- On POSIX systems it raises the `RLIMIT_NOFILE` soft limit toward the hard limit; on Windows it raises the CRT stdio cap (max 8192)
- A failed or partial raise is non-fatal; the returned value is the ceiling in effect after the attempt

Backup

Create an on-disk snapshot of an open database without blocking normal reads/writes.

```
err := db.Backup("./mydb_backup")
if err != nil {
    log.Fatal(err)
}
```

Behavior

- Requires the backup directory to be non-existent or empty
- Does not copy the LOCK file, so the backup can be opened normally
- Database stays open and usable during backup
- The backup represents the database state after the final flush/compaction drain

Checkpoint

Create a lightweight, near-instant snapshot of an open database using hard links instead of copying SSTable data.

```
err := db.Checkpoint("./mydb_checkpoint")
if err != nil {
    log.Fatal(err)
}
```

Behavior

- Requires the checkpoint directory to be non-existent or empty
- For each column family:
 - Flushes the active memtable so all data is in SSTables
 - Halts compactions to ensure a consistent view of live SSTable files
 - Hard links all SSTable files (`.klog` and `.vlog`) into the checkpoint directory
 - Copies small metadata files (manifest, config) into the checkpoint directory
 - Resumes compactions
- Falls back to file copy if hard linking fails (e.g., cross-filesystem)
- Database stays open and usable during checkpoint
- The checkpoint can be opened as a normal TidesDB database with `Open`

Checkpoint vs Backup

	<code>Backup</code>	<code>Checkpoint</code>
Speed	Copies every SSTable byte-by-byte	Near-instant (hard links, O(1) per file)
Disk usage	Full independent copy	No extra disk until compaction removes old SSTables
Portability	Can be moved to another filesystem or machine	Same filesystem only (hard link requirement)
Use case	Archival, disaster recovery, remote shipping	Fast local snapshots, point-in-time reads, streaming backups

Notes

- The checkpoint represents the database state at the point all memtables are flushed and compactions are halted
- Hard-linked files share storage with the live database. Deleting the original database does not affect the checkpoint (hard link semantics)

Creating and Dropping Column Families

Column families are isolated key-value stores with independent configuration.

```

cfConfig := tidesdb.DefaultColumnFamilyConfig()
err := db.CreateColumnFamily("my_cf", cfConfig)
if err != nil {
    log.Fatal(err)
}

// Create with custom configuration based on defaults
cfConfig := tidesdb.DefaultColumnFamilyConfig()

// You can modify the configuration as needed
cfConfig.WriteBufferSize = 128 * 1024 * 1024
cfConfig.LevelSizeRatio = 10
cfConfig.MinLevels = 5
cfConfig.CompressionAlgorithm = tidesdb.LZ4Compression
cfConfig.EnableBloomFilter = true
cfConfig.BloomFPR = 0.01
cfConfig.EnableBlockIndexes = true
cfConfig.SyncMode = tidesdb.SyncInterval
cfConfig.SyncIntervalUs = 128000
cfConfig.DefaultIsolationLevel = tidesdb.IsolationReadCommitted
cfConfig.DividingLevelOffset = 2 // Compaction dividing
level offset
cfConfig.KlogValueThreshold = 512 // Values > 512 bytes go
to vlog
cfConfig.BlockIndexPrefixLen = 16 // Block index prefix
length
cfConfig.MinDiskSpace = 100 * 1024 * 1024 // Minimum disk space
required (100MB)
cfConfig.L1FileCountTrigger = 4 // L1 file count trigger
for compaction
cfConfig.L0QueueStallThreshold = 20 // L0 queue stall
threshold
cfConfig.TombstoneDensityTrigger = 0.0 // Per-SSTable tombstone
density above which compaction escalates (0.0 = disabled, range 0.0 to 1.0)
cfConfig.TombstoneDensityMinEntries = 1024 // Minimum entry count
for an SSTable to be considered by the density trigger
cfConfig.UseBtree = 0 // Use B+tree format for
klog (0 = block-based)
cfConfig.ComparatorName = tidesdb.ComparatorMemcmp // Key ordering
comparator (see Built-in comparators)
cfConfig.ComparatorCtxStr = "" // Optional context
string for parameterized comparators
cfConfig.ObjectLazyCompaction = 0 // Less aggressive
compaction in object store mode

```

```
cfConfig.ObjectPrefetchCompaction = 1           // Download all inputs
before merge

err = db.CreateColumnFamily("my_cf", cfConfig)
if err != nil {
    log.Fatal(err)
}

err = db.DropColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}
```

Dropping by pointer (faster when you already hold the `ColumnFamily` pointer, skips internal name lookup):

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

err = db.DeleteColumnFamily(cf)
if err != nil {
    log.Fatal(err)
}
```

Renaming a Column Family

Atomically rename a column family and its underlying directory. The operation waits for any in-progress flush or compaction to complete before renaming.

```
err := db.RenameColumnFamily("old_name", "new_name")
if err != nil {
    log.Fatal(err)
}
```

Cloning a Column Family

Create a complete copy of an existing column family with a new name. The clone contains all the data from the source at the time of cloning.

```
err := db.CloneColumnFamily("source_cf", "cloned_cf")
if err != nil {
    log.Fatal(err)
}

// Both column families now exist independently
sourceCF, _ := db.GetColumnFamily("source_cf")
clonedCF, _ := db.GetColumnFamily("cloned_cf")
```

Behavior

- Flushes the source column family's memtable to ensure all data is on disk
- Waits for any in-progress flush or compaction to complete
- Copies all SSTable files to the new directory
- The clone is completely independent; modifications to one do not affect the other

Use cases

- Testing · Create a copy of production data for testing without affecting the original
- Branching · Create a snapshot of data before making experimental changes
- Migration · Clone data before schema or configuration changes
- Backup verification · Clone and verify data integrity without modifying the source

Return values

- `nil` · Clone completed successfully
- Error with `ErrNotFound` · Source column family doesn't exist
- Error with `ErrExists` · Destination column family already exists
- Error with `ErrInvalidArgs` · Invalid arguments (nil pointers or same source/destination name)
- Error with `ErrIO` · Failed to copy files or create directory

CRUD Operations

All operations in TidesDB are performed through transactions for ACID guarantees.

Writing Data

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

txn, err := db.BeginTxn()
if err != nil {
    log.Fatal(err)
}
defer txn.Free()

// Put a key-value pair (TTL -1 means no expiration)
err = txn.Put(cf, []byte("key"), []byte("value"), -1)
if err != nil {
    log.Fatal(err)
}

err = txn.Commit()
if err != nil {
    log.Fatal(err)
}
```

Writing with TTL

```
import "time"

cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

txn, err := db.BeginTxn()
if err != nil {
    log.Fatal(err)
}
defer txn.Free()

// Set expiration time (Unix timestamp)
ttl := time.Now().Add(10 * time.Second).Unix()

err = txn.Put(cf, []byte("temp_key"), []byte("temp_value"), ttl)
if err != nil {
    log.Fatal(err)
}

err = txn.Commit()
if err != nil {
    log.Fatal(err)
}
```

TTL Examples

```
ttl := int64(-1)

// Expire in 5 minutes
ttl := time.Now().Add(5 * time.Minute).Unix()

// Expire in 1 hour
ttl := time.Now().Add(1 * time.Hour).Unix()

ttl := time.Date(2026, 12, 31, 23, 59, 59, 0, time.UTC).Unix()
```

Reading Data

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

txn, err := db.BeginTxn()
if err != nil {
    log.Fatal(err)
}
defer txn.Free()

value, err := txn.Get(cf, []byte("key"))
if err != nil {
    log.Fatal(err)
}

fmt.Printf("Value: %s\n", value)
```

Deleting Data

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

txn, err := db.BeginTxn()
if err != nil {
    log.Fatal(err)
}
defer txn.Free()

err = txn.Delete(cf, []byte("key"))
if err != nil {
    log.Fatal(err)
}

err = txn.Commit()
if err != nil {
    log.Fatal(err)
}
```

Single-Delete

`SingleDelete` writes a tombstone with the same read semantics as `Delete`, but carries a caller-provided promise that lets compaction drop the put and the tombstone together as soon as both appear in the same merge input, rather than carrying the tombstone forward until it reaches the largest active level.

Between any two single-deletes on the same key, and between the start of the key's history and its first single-delete, the key has been put **at most once**. The engine does not and cannot verify this at runtime; violating the contract can leave older puts visible after the single-delete and is a bug in the caller.

This is the right choice for workloads that insert each key exactly once and then delete it exactly once (classic insert-benchmark patterns, secondary-index entries on columns that are never updated, log-style tables with scheduled purges). It is **not** safe for tables that issue repeated updates to the same key.

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

txn, err := db.BeginTxn()
if err != nil {
    log.Fatal(err)
}
defer txn.Free()

err = txn.SingleDelete(cf, []byte("key"))
if err != nil {
    log.Fatal(err)
}

err = txn.Commit()
if err != nil {
    log.Fatal(err)
}
```

Returns `nil` on success or a non-nil error on failure. When in doubt, prefer `Delete`.

Multi-Operation Transactions

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

txn, err := db.BeginTxn()
if err != nil {
    log.Fatal(err)
}
defer txn.Free()

// Multiple operations in one transaction, across column families as well
err = txn.Put(cf, []byte("key1"), []byte("value1"), -1)
if err != nil {
    txn.Rollback()
    log.Fatal(err)
}

err = txn.Put(cf, []byte("key2"), []byte("value2"), -1)
if err != nil {
    txn.Rollback()
    log.Fatal(err)
}

err = txn.Delete(cf, []byte("old_key"))
if err != nil {
    txn.Rollback()
    log.Fatal(err)
}

// Commit atomically -- all or nothing
err = txn.Commit()
if err != nil {
    log.Fatal(err)
}
```

Iterating Over Data

Iterators provide efficient bidirectional traversal over key-value pairs.

Forward Iteration

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

txn, err := db.BeginTxn()
if err != nil {
    log.Fatal(err)
}
defer txn.Free()

iter, err := txn.NewIterator(cf)
if err != nil {
    log.Fatal(err)
}
defer iter.Free()

iter.SeekToFirst()

for iter.Valid() {
    key, err := iter.Key()
    if err != nil {
        log.Fatal(err)
    }

    value, err := iter.Value()
    if err != nil {
        log.Fatal(err)
    }

    fmt.Printf("Key: %s, Value: %s\n", key, value)

    iter.Next()
}
```

Backward Iteration

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

txn, err := db.BeginTxn()
if err != nil {
    log.Fatal(err)
}
defer txn.Free()

iter, err := txn.NewIterator(cf)
if err != nil {
    log.Fatal(err)
}
defer iter.Free()

iter.SeekToLast()

for iter.Valid() {
    key, err := iter.Key()
    if err != nil {
        log.Fatal(err)
    }

    value, err := iter.Value()
    if err != nil {
        log.Fatal(err)
    }

    fmt.Printf("Key: %s, Value: %s\n", key, value)

    iter.Prev()
}
```

Seek Operations

Seek to a specific key or key range without scanning from the beginning.

```
iter, err := txn.NewIterator(cf)
if err != nil {
    log.Fatal(err)
}
defer iter.Free()

// Seek to first key >= target
err = iter.Seek([]byte("user:1000"))
if err != nil {
    log.Fatal(err)
}

if iter.Valid() {
    key, _ := iter.Key()
    fmt.Printf("Found: %s\n", key)
}

// Seek to last key <= target (for reverse iteration)
err = iter.SeekForPrev([]byte("user:2000"))
if err != nil {
    log.Fatal(err)
}

for iter.Valid() {
    key, _ := iter.Key()
    fmt.Printf("Reverse: %s\n", key)
    iter.Prev()
}
```


Prefix Seeking

```
iter, err := txn.NewIterator(cf)
if err != nil {
    log.Fatal(err)
}
defer iter.Free()

prefix := []byte("user:")
err = iter.Seek(prefix)
if err != nil {
    log.Fatal(err)
}

for iter.Valid() {
    key, _ := iter.Key()

    // Stop when keys no longer match prefix
    if !bytes.HasPrefix(key, prefix) {
        break
    }

    value, _ := iter.Value()
    fmt.Printf("%s = %s\n", key, value)

    iter.Next()
}
```

Combined Key-Value Retrieval

`KeyValue` retrieves both the current key and value from the iterator in a single call. This is more efficient than calling `Key()` and `Value()` separately.

```
iter, err := txn.NewIterator(cf)
if err != nil {
    log.Fatal(err)
}
defer iter.Free()

iter.SeekToFirst()

for iter.Valid() {
    key, value, err := iter.KeyValue()
    if err != nil {
        log.Fatal(err)
    }

    fmt.Printf("Key: %s, Value: %s\n", key, value)

    iter.Next()
}
```

Getting Column Family Statistics

Retrieve detailed statistics about a column family.

```

cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

stats, err := cf.GetStats()
if err != nil {
    log.Fatal(err)
}

fmt.Printf("Number of Levels: %d\n", stats.NumLevels)
fmt.Printf("Memtable Size: %d bytes\n", stats.MemtableSize)
fmt.Printf("Total Keys: %d\n", stats.TotalKeys)
fmt.Printf("Total Data Size: %d bytes\n", stats.TotalDataSize)
fmt.Printf("Avg Key Size: %.2f bytes\n", stats.AvgKeySize)
fmt.Printf("Avg Value Size: %.2f bytes\n", stats.AvgValueSize)
fmt.Printf("Read Amplification: %.2f\n", stats.ReadAmp)
fmt.Printf("Hit Rate: %.2f%%\n", stats.HitRate * 100)

// B+tree statistics (only populated if UseBtree=1)
if stats.UseBtree {
    fmt.Printf("B+tree Total Nodes: %d\n", stats.BtreeTotalNodes)
    fmt.Printf("B+tree Max Height: %d\n", stats.BtreeMaxHeight)
    fmt.Printf("B+tree Avg Height: %.2f\n", stats.BtreeAvgHeight)
}

// Per-level statistics
for i := 0; i < stats.NumLevels; i++ {
    fmt.Printf("Level %d: %d SSTables, %d bytes, %d keys\n",
        i+1, stats.LevelNumSSTables[i], stats.LevelSizes[i],
        stats.LevelKeyCounts[i])
}

// Tombstone density observability
fmt.Printf("Total Tombstones: %d\n", stats.TotalTombstones)
fmt.Printf("Tombstone Ratio: %.2f%%\n", stats.TombstoneRatio*100.0)
fmt.Printf("Worst SSTable Density: %.2f%% at level %d\n",
    stats.MaxSSTDensity*100.0, stats.MaxSSTDensityLevel)
for i := 0; i < stats.NumLevels; i++ {
    fmt.Printf("Level %d tombstones: %d\n", i+1,
        stats.LevelTombstoneCounts[i])
}

if stats.Config != nil {

```

```

    fmt.Printf("Write Buffer Size: %d\n", stats.Config.WriteBufferSize)
    fmt.Printf("Compression: %d\n", stats.Config.CompressionAlgorithm)
    fmt.Printf("Bloom Filter: %v\n", stats.Config.EnableBloomFilter)
    fmt.Printf("Sync Mode: %d\n", stats.Config.SyncMode)
    fmt.Printf("Use B+tree: %d\n", stats.Config.UseBtree)
}

// Write-amplification counters (lifetime since open, on-disk framed
bytes).
// Divide each write total by UserBytesWritten to get this CF's write
amplification.
if stats.UserBytesWritten > 0 {
    totalWrites := stats.WalBytesWritten + stats.FlushBytesWritten +
stats.CompactionBytesWritten
    fmt.Printf("User bytes written: %d\n", stats.UserBytesWritten)
    fmt.Printf("Write amplification: %.2fx\n",
float64(totalWrites)/float64(stats.UserBytesWritten))
    fmt.Printf("Flush output: %d bytes in %d sstables\n",
stats.FlushBytesWritten, stats.FlushCount)
    fmt.Printf("Compaction: read %d / wrote %d bytes in %d sstables\n",
stats.CompactionBytesRead, stats.CompactionBytesWritten,
stats.CompactionCount)
}

```

Stats Fields

Field	Type	Description
NumLevels	int	Number of LSM levels
MemtableSize	uint64	Current memtable size in bytes
LevelSizes	[]uint64	Array of per-level total sizes
LevelNumSSTables	[]int	Array of per-level SSTable counts
LevelKeyCounts	[]uint64	Array of per-level key counts
Config	*ColumnFamilyConfig	Full column family configuration
TotalKeys	uint64	Total keys across memtable and all SSTables
TotalDataSize	uint64	Total data size (klog + vlog) in bytes
AvgKeySize	float64	Estimated average key size in bytes
AvgValueSize	float64	Estimated average value size in bytes
ReadAmp	float64	Read amplification factor
HitRate	float64	Block cache hit rate (0.0 to 1.0)
UseBtree	bool	Whether column family uses B+tree klog format
BtreeTotalNodes	uint64	Total B+tree nodes across all SSTables
BtreeMaxHeight	uint32	Maximum tree height across all SSTables
BtreeAvgHeight	float64	Average tree height across all SSTables
TotalTombstones	uint64	Total tombstones across all SSTables
TombstoneRatio	float64	Database-wide tombstone count divided by entry count (0.0 to 1.0)
LevelTombstoneCounts	[]uint64	Per-level tombstone counts (parallels LevelKeyCounts)
MaxSSTDensity	float64	Worst single-SSTable tombstone density observed (0.0 to 1.0)
MaxSSTDensityLevel	int	1-based level index where the worst SSTable lives (0 if none)
WalBytesWritten	uint64	Framed bytes appended to this CF's WAL (0 in unified mode)

Field	Type	Description
<code>FlushBytesWritten</code>	<code>uint64</code>	On-disk bytes this CF's flushes wrote to L0 SSTables
<code>CompactionBytesWritten</code>	<code>uint64</code>	On-disk bytes this CF's compactions wrote
<code>CompactionBytesRead</code>	<code>uint64</code>	On-disk bytes this CF's compactions read as input
<code>UserBytesWritten</code>	<code>uint64</code>	Logical key+value bytes committed to this CF (write-amplification denominator)
<code>FlushCount</code>	<code>uint64</code>	Flushed SSTables produced by this CF
<code>CompactionCount</code>	<code>uint64</code>	Compaction output SSTables produced by this CF

Write Amplification

The `*BytesWritten` / `*BytesRead` / `*Count` counters are lifetime totals since the database was opened, measured in on-disk framed bytes. Compute this column family's write amplification as
$$\frac{(\text{WalBytesWritten} + \text{FlushBytesWritten} + \text{CompactionBytesWritten})}{\text{UserBytesWritten}}$$
. `walBytesWritten` is zero in unified-memtable mode — the shared WAL volume is reported db-wide via `DbStats.UwalBytesWritten`.

Getting Block Cache Statistics

Get statistics for the global block cache (shared across all column families).

```

cacheStats, err := db.GetCacheStats()
if err != nil {
    log.Fatal(err)
}

if cacheStats.Enabled {
    fmt.Printf("Cache enabled: yes\n")
    fmt.Printf("Total entries: %d\n", cacheStats.TotalEntries)
    fmt.Printf("Total bytes: %.2f MB\n", float64(cacheStats.TotalBytes) /
(1024.0 * 1024.0))
    fmt.Printf("Hits: %d\n", cacheStats.Hits)
    fmt.Printf("Misses: %d\n", cacheStats.Misses)
    fmt.Printf("Hit rate: %.1f%%\n", cacheStats.HitRate * 100.0)
    fmt.Printf("Partitions: %d\n", cacheStats.NumPartitions)
} else {
    fmt.Printf("Cache enabled: no (BlockCacheSize = 0)\n")
}

```

CacheStats Fields

Field	Type	Description
<code>Enabled</code>	<code>bool</code>	Whether block cache is active
<code>TotalEntries</code>	<code>uint64</code>	Number of cached blocks
<code>TotalBytes</code>	<code>uint64</code>	Total memory used by cached blocks
<code>Hits</code>	<code>uint64</code>	Number of cache hits
<code>Misses</code>	<code>uint64</code>	Number of cache misses
<code>HitRate</code>	<code>float64</code>	Hit rate as a decimal (0.0 to 1.0)
<code>NumPartitions</code>	<code>uint64</code>	Number of cache partitions

Database-Level Statistics

Get aggregate statistics across the entire database instance.

```
dbStats, err := db.GetDbStats()
if err != nil {
    log.Fatal(err)
}

fmt.Printf("Column families: %d\n", dbStats.NumColumnFamilies)
fmt.Printf("Total memory: %d bytes\n", dbStats.TotalMemory)
fmt.Printf("Resolved memory limit: %d bytes\n",
dbStats.ResolvedMemoryLimit)
fmt.Printf("Memory pressure level: %d\n", dbStats.MemoryPressureLevel)
fmt.Printf("Global sequence: %d\n", dbStats.GlobalSeq)
fmt.Printf("Flush queue: %d pending\n", dbStats.FlushQueueSize)
fmt.Printf("Compaction queue: %d pending\n", dbStats.CompactionQueueSize)
fmt.Printf("Total SSTables: %d\n", dbStats.TotalSstableCount)
fmt.Printf("Total data size: %d bytes\n", dbStats.TotalDataSizeBytes)
fmt.Printf("Open SSTable handles: %d\n", dbStats.NumOpenSstables)
fmt.Printf("In-flight txn memory: %d bytes\n", dbStats.TxnMemoryBytes)
fmt.Printf("Immutable memtables: %d\n", dbStats.TotalImmutableCount)
fmt.Printf("Memtable bytes: %d\n", dbStats.TotalMemtableBytes)
```

DbStats Fields

Field	Type	Description
NumColumnFamilies	int	Number of column families
TotalMemory	uint64	System total memory
AvailableMemory	uint64	System available memory at open time
ResolvedMemoryLimit	uint64	Resolved memory limit (auto or configured)
MemoryPressureLevel	int	Current memory pressure (0=normal, 1=elevated, 2=high, 3=critical)
FlushPendingCount	int	Number of pending flush operations (queued + in-flight)
TotalMemtableBytes	int64	Total bytes in active memtables across all CFs
TotalImmutableCount	int	Total immutable memtables across all CFs
TotalSstableCount	int	Total SSTables across all CFs and levels
TotalDataSizeBytes	uint64	Total data size (klog + vlog) across all CFs
NumOpenSstableS	int	Number of currently open SSTable file handles
GlobalSeq	uint64	Current global sequence number
TxnMemoryBytes	int64	Bytes held by in-flight transactions
CompactionQueueSize	uint64	Number of pending compaction tasks
FlushQueueSize	uint64	Number of pending flush tasks in queue
UnifiedMemtableEnabled	bool	Whether unified memtable mode is active

Field	Type	Description
UnifiedMemtableBytes	int64	Bytes in unified active memtable
UnifiedImmutableCount	int	Number of unified immutable memtables
UnifiedIsFlushing	bool	Whether unified memtable is currently flushing/rotating
UnifiedNextCFIndex	uint32	Next CF index to be assigned in unified mode
UnifiedWalGeneration	uint64	Current unified WAL generation counter
ObjectStoreEnabled	bool	Whether object store mode is active
ObjectStoreConnector	string	Connector name (“s3”, “gcs”, “fs”, etc.)
LocalCacheBytesUsed	uint64	Current local file cache usage in bytes
LocalCacheBytesMax	uint64	Configured maximum local cache size in bytes
LocalCacheNumFiles	int	Number of files tracked in local cache
LastUploadedGeneration	uint64	Highest WAL generation confirmed uploaded
UploadQueueDepth	uint64	Number of pending upload jobs in the queue
TotalUploads	uint64	Lifetime count of objects uploaded to object store
TotalUploadFailures	uint64	Lifetime count of permanently failed uploads
ReplicaMode	bool	Whether running in read-only replica mode
PrimaryEpoch	uint64	Single-writer fencing (object-store mode): lease epoch this primary holds (0 when not a primary / no lease)

Field	Type	Description
<code>SeenEpoch</code>	<code>uint64</code>	Single-writer fencing (object-store mode): highest lease epoch a replica has observed
<code>UwalBytesWritten</code>	<code>uint64</code>	Framed bytes appended to the shared unified WAL (0 if unified mode off)
<code>WalBytesWritten</code>	<code>uint64</code>	Per-CF WAL bytes summed across all column families
<code>FlushBytesWritten</code>	<code>uint64</code>	Flush output bytes summed across all column families
<code>CompactionBytesWritten</code>	<code>uint64</code>	Compaction output bytes summed across all column families
<code>CompactionBytesRead</code>	<code>uint64</code>	Compaction input bytes summed across all column families
<code>UserBytesWritten</code>	<code>uint64</code>	Logical committed bytes summed across all column families
<code>FlushCount</code>	<code>uint64</code>	Flushed SSTables summed across all column families
<code>CompactionCount</code>	<code>uint64</code>	Compaction output SSTables summed across all column families

Database-Wide Write Amplification

Compute db-wide write amplification as $(UwalBytesWritten + WalBytesWritten + FlushBytesWritten + CompactionBytesWritten) / UserBytesWritten$. The `*Count` fields count output SSTables, not logical runs.

Stack Allocated

Unlike `GetStats` (which heap-allocates), `GetDbStats` fills a caller-provided struct. No free is needed.

Range Cost Estimation

`RangeCost` estimates the computational cost of iterating between two keys in a column family. The returned value is an opaque double - meaningful only for comparison with other values from the same function. It uses only in-memory metadata and performs no disk I/O.

```

cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

costA, err := cf.RangeCost([]byte("user:0000"), []byte("user:0999"))
if err != nil {
    log.Fatal(err)
}

costB, err := cf.RangeCost([]byte("user:1000"), []byte("user:1099"))
if err != nil {
    log.Fatal(err)
}

if costA < costB {
    fmt.Println("Range A is cheaper to iterate")
}

```

Behavior

- Key order does not matter - the function normalizes the range so `keyA > keyB` produces the same result as `keyB > keyA`
- With block indexes enabled, uses $O(\log B)$ binary search per overlapping SSTable
- Without block indexes, falls back to byte-level key interpolation
- B+tree SSTables use key interpolation against tree node counts plus tree height as seek cost
- Compressed SSTables receive a $1.5\times$ weight multiplier for decompression overhead
- A cost of 0.0 means no overlapping SSTables or memtable entries were found for the range

Use cases

- Query planning · Compare candidate key ranges to find the cheapest one to scan
- Load balancing · Distribute range scan work across goroutines by estimating per-range cost
- Adaptive prefetching · Decide how aggressively to prefetch based on range size
- Monitoring · Track how data distribution changes across key ranges over time

Getting a Comparator

Retrieve a registered comparator by name. Returns whether the comparator is registered.

```

found, err := db.GetComparator("memcmp")
if err != nil {
    log.Fatal(err)
}

if found {
    fmt.Println("Comparator 'memcmp' is registered")
}

```

Built-in comparators (automatically registered on database open). Assign one to `cfConfig.ComparatorName` using the exported constants — no `RegisterComparator` call is needed:

Constant	Name	Ordering
<code>tidesdb.ComparatorMemcmp</code>	<code>"memcmp"</code>	Unsigned byte-by-byte comparison (default)
<code>tidesdb.ComparatorLexicographic</code>	<code>"lexicographic"</code>	Null-terminated string comparison
<code>tidesdb.ComparatorUint64</code>	<code>"uint64"</code>	8-byte key read as a <code>uint64</code> in host byte order
<code>tidesdb.ComparatorInt64</code>	<code>"int64"</code>	8-byte key read as an <code>int64</code> in host byte order
<code>tidesdb.ComparatorReverse</code>	<code>"reverse"</code>	Reverse byte-by-byte comparison (descending)
<code>tidesdb.ComparatorCaseInsensitive</code>	<code>"case_insensitive"</code>	Case-insensitive ASCII comparison

⚠ **uint64 / int64 byte order**

The `uint64` and `int64` comparators reinterpret the raw 8 key bytes in **host byte order** (little-endian on x86-64 and ARM64), not big-endian. Encode keys with `encoding/binary.NativeEndian` so the numeric ordering matches. Keys that are not exactly 8 bytes fall back to `memcmp` ordering.

```

import "encoding/binary"

cfConfig := tidesdb.DefaultColumnFamilyConfig()
cfConfig.ComparatorName = tidesdb.ComparatorUint64
db.CreateColumnFamily("by_id", cfConfig)

cf, _ := db.GetColumnFamily("by_id")
key := make([]byte, 8)
binary.NativeEndian.PutUint64(key, 42) // numerically ordered key

txn, _ := db.BeginTxn()
txn.Put(cf, key, []byte("value"), -1)
txn.Commit()
txn.Free()

```

Use cases

- Validation · Check if a comparator is registered before creating a column family
- Debugging · Verify comparator registration during development
- Dynamic configuration · Query available comparators at runtime

Listing Column Families

```

cfList, err := db.ListColumnFamilies()
if err != nil {
    log.Fatal(err)
}

fmt.Println("Available column families:")
for _, name := range cfList {
    fmt.Printf("  - %s\n", name)
}

```

Compaction

Manual Compaction

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

// Manually trigger compaction (queues compaction from L1+)
err = cf.Compact()
if err != nil {
    log.Printf("Compaction note: %v", err)
}
```

Targeted Range Compaction

`CompactRange` runs a synchronous compaction over a specific key range. Only SSTables whose minimum and maximum keys overlap the requested range participate in the merge, so the work and I/O are bounded to the affected portion of the LSM tree rather than the whole column family.

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

start := []byte("tenant_42:")
end := []byte("tenant_42;")

if err := cf.CompactRange(start, end); err != nil {
    log.Fatalf("Targeted compaction failed: %v", err)
}
```

When to use

- Bulk reclaim after a large range delete, where waiting for natural compaction would leave tombstones and obsolete versions on disk
- Tenant eviction or sliding-window expiration that does not fit TTL semantics
- Post-import cleanup of a known key range loaded with `Put` followed by `Delete`
- Operational counterpart to the automatic tombstone density trigger when an operator wants reclaim now rather than at the next natural threshold crossing

Behavior

- Synchronous, blocks the caller until the merge commits or fails
- Does not enqueue work onto the compaction thread pool, the calling goroutine does the work
- Selects only SSTables whose key range overlaps the requested range using the column family's comparator, SSTables outside the range are not touched
- Applies the same emit-loop logic as background compactions (tombstone reclamation rules, single-delete pair cancellation, sequence-based deduplication, value recompression)
- Output SSTables are committed to the manifest atomically and old inputs are marked for deletion

Return values

- `nil` on success
- Error with `ErrInvalidArgs` if `cf` is nil or both endpoints are nil/empty (use `Compact` for full CF compaction)
- Error with `ErrLocked` if another compaction is running for the column family
- Standard I/O and memory error codes if the merge cannot complete

Manual Memtable Flush

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

// Manually trigger memtable flush (queues memtable for sorted run to disk
// (L1))
err = cf.FlushMemtable()
if err != nil {
    log.Printf("Flush note: %v", err)
}
```

Checking Flush/Compaction Status

Check if a column family currently has flush or compaction operations in progress.


```

cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

// Check if flushing is in progress
if cf.IsFlushing() {
    fmt.Println("Flush in progress")
}

// Check if compaction is in progress
if cf.IsCompacting() {
    fmt.Println("Compaction in progress")
}

```

Purge Column Family

`PurgeCF` forces a synchronous flush and aggressive compaction for a single column family. Unlike `FlushMemtable` and `Compact` (which are non-blocking), `PurgeCF` blocks until all flush and compaction I/O is complete.

```

cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

err = cf.PurgeCF()
if err != nil {
    log.Fatal(err)
}
// All data is now flushed to SSTables and compacted

```

Behavior

1. Waits for any in-progress flush to complete
2. Force-flushes the active memtable (even if below threshold)
3. Waits for flush I/O to fully complete
4. Waits for any in-progress compaction to complete
5. Triggers synchronous compaction inline (bypasses the compaction queue)
6. Waits for any queued compaction to drain

When to use

- Before backup or checkpoint · Ensure all data is on disk and compacted
- After bulk deletes · Reclaim space immediately by compacting away tombstones
- Manual maintenance · Force a clean state during a maintenance window
- Pre-shutdown · Ensure all pending work is complete before closing

Purge Database

`Purge` forces a synchronous flush and aggressive compaction for **all** column families, then drains both the global flush and compaction queues.

```
err := db.Purge()
if err != nil {
    log.Fatal(err)
}
// All CFs flushed and compacted, all queues drained
```

Behavior

1. Calls `PurgeCF` on each column family
2. Drains the global flush queue (waits for queue size and pending count to reach 0)
3. Drains the global compaction queue (waits for queue size to reach 0)

Return values

- `nil` if all column families purged successfully
- First non-nil error if any CF fails (continues processing remaining CFs)



Purge vs Manual Flush + Compact

`FlushMemtable` and `Compact` are non-blocking - they enqueue work and return immediately. `PurgeCF` and `Purge` are synchronous - they block until all work is complete. Use purge when you need a guarantee that all data is on disk and compacted before proceeding.

Cancelling Background Work

`CancelBackgroundWork` cancels background compaction across the whole database. In-flight merges bail out safely and queued compaction is skipped; flushes are unaffected, so durability is preserved. The call blocks (bounded) until compaction is idle.

```
// Fast shutdown: stop compaction, then close.
if err := db.CancelBackgroundWork(); err != nil {
    log.Fatal(err)
}
db.Close()
```

Behavior

- The cancellation is sticky for the session and resets on the next `Open`
- Intended to be called right before `Close` for a fast shutdown
- Flushes still run, so committed data remains durable
- Complements the `FinishCompactionsOnClose` config field — leave that `false` (the default) for the fastest shutdown path

Updating Runtime Configuration

Update runtime-safe configuration settings for a column family. Changes apply to new operations only.

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

newConfig := tidesdb.DefaultColumnFamilyConfig()
newConfig.WriteBufferSize = 256 * 1024 * 1024
newConfig.SkiplistMaxLevel = 16
newConfig.BloomFPR = 0.001 // 0.1% false positive rate

err = cf.UpdateRuntimeConfig(newConfig, true)
if err != nil {
    log.Fatal(err)
}
```

Updatable settings (safe to change at runtime):

- `WriteBufferSize` · Memtable flush threshold
- `SkiplistMaxLevel` · Skip list level for new memtables
- `SkiplistProbability` · Skip list probability for new memtables
- `BloomFPR` · False positive rate for new SSTables

- `IndexSampleRatio` · Index sampling ratio for new SSTables
- `SyncMode` · Durability mode
- `SyncIntervalUs` · Sync interval in microseconds

Commit Hook (Change Data Capture)

`SetCommitHook` registers a callback that fires synchronously after every transaction commit on a column family. The hook receives the full batch of committed operations atomically, enabling real-time change data capture without WAL parsing or external log consumers.

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

err = cf.SetCommitHook(func(ops []tidesdb.CommitOp, commitSeq uint64) int {
    for _, op := range ops {
        if op.IsDelete {
            fmt.Printf("[seq=%d] DELETE key=%s\n", commitSeq,
string(op.Key))
        } else {
            fmt.Printf("[seq=%d] PUT key=%s value=%s ttl=%d\n",
commitSeq, string(op.Key), string(op.Value), op.TTL)
        }
    }
    return 0 // 0 = success; non-zero is logged as warning
})
if err != nil {
    log.Fatal(err)
}

// Normal writes now trigger the hook automatically
txn, _ := db.BeginTxn()
txn.Put(cf, []byte("user:1000"), []byte("John Doe"), -1)
txn.Commit() // hook fires here
txn.Free()

// Detach the hook
err = cf.ClearCommitHook()
if err != nil {
    log.Fatal(err)
}
```

CommitOp fields

Field	Type	Description
Key	[]byte	Key data (copied; safe to retain after callback returns)
Value	[]byte	Value data (nil for deletes; copied; safe to retain)
TTL	int64	Time-to-live Unix timestamp (0 = no expiry)
IsDelete	bool	true if this is a delete operation, false for put

Behavior

- The hook fires after WAL write, memtable apply, and commit status marking are complete - the data is fully durable before the callback runs
- Hook failure (non-zero return) is logged but does not affect the commit result
- Each column family has its own independent hook; a multi-CF transaction fires the hook once per CF with only that CF's operations
- commitSeq is monotonically increasing across commits and can be used as a replication cursor
- Data in CommitOp is copied from C memory - safe to retain after the callback returns
- The hook executes synchronously on the committing goroutine; keep the callback fast to avoid stalling writers
- Setting the hook to nil or calling ClearCommitHook disables it immediately with no restart required
- Setting a new hook replaces any previously set hook for the same column family

Use cases

- Replication · Ship committed batches to replicas in commit order
- Event streaming · Publish mutations to Kafka, NATS, or any message broker
- Secondary indexing · Maintain a reverse index or materialized view
- Audit logging · Record every mutation with key, value, TTL, and sequence number
- Debugging · Attach a temporary hook in production to inspect live writes

Runtime-Only

Commit hooks are not persisted across database restarts. After reopening the database, hooks must be re-registered by the application. This is by design - function pointers cannot be serialized.

Sync Modes

Control the durability vs performance tradeoff.

```
cfConfig := tidesdb.DefaultColumnFamilyConfig()

// SyncNone -- Fastest, least durable (OS handles flushing on sorted runs
and compaction to sync after completion)
cfConfig.SyncMode = tidesdb.SyncNone

// SyncInterval -- Balanced (periodic background syncing)
cfConfig.SyncMode = tidesdb.SyncInterval
cfConfig.SyncIntervalUs = 128000 // Sync every 128ms

// SyncFull -- Most durable (fsync on every write)
cfConfig.SyncMode = tidesdb.SyncFull

err := db.CreateColumnFamily("my_cf", cfConfig)
if err != nil {
    log.Fatal(err)
}
```

Manual WAL Sync

`SyncWal` forces an immediate fsync of the active write-ahead log for a column family. This is useful for explicit durability control when using `SyncNone` or `SyncInterval` modes.

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

// Force WAL durability after a batch of writes
err = cf.SyncWal()
if err != nil {
    log.Fatal(err)
}
```

When to use

- Application-controlled durability · Sync the WAL at specific points (e.g., after a batch of related writes) when using `SyncNone` or `SyncInterval`
- Pre-checkpoint · Ensure all buffered WAL data is on disk before taking a checkpoint
- Graceful shutdown · Flush WAL buffers before closing the database
- Critical writes · Force durability for specific high-value writes without using `SyncFull` for all writes

Behavior

- Acquires a reference to the active memtable to safely access its WAL
- Calls `fdatsync` on the WAL file descriptor
- Thread-safe - can be called concurrently from multiple goroutines
- If the memtable rotates during the call, retries with the new active memtable

Structural Operations

Regardless of sync mode, TidesDB **always** enforces durability for structural operations, memtable flush to SSTable, SSTable compaction and merging, WAL rotation, and column family metadata updates.

Compression Algorithms

TidesDB supports multiple compression algorithms:

```
cfConfig := tidesdb.DefaultColumnFamilyConfig()

cfConfig.CompressionAlgorithm = tidesdb.NoCompression
cfConfig.CompressionAlgorithm = tidesdb.SnappyCompression
cfConfig.CompressionAlgorithm = tidesdb.LZ4Compression
cfConfig.CompressionAlgorithm = tidesdb.LZ4FastCompression
cfConfig.CompressionAlgorithm = tidesdb.ZstdCompression

err := db.CreateColumnFamily("my_cf", cfConfig)
if err != nil {
    log.Fatal(err)
}
```

Object Store

TidesDB supports pluggable object store backends for cloud-native storage. The Go binding exposes the filesystem-backed connector for testing and local replication, and an S3-compatible connector (behind a build tag) for production cloud storage.

Object Store Backends

```
tidesdb.BackendFS      // Filesystem-backed (for testing/local
replication)
tidesdb.BackendS3      // S3-compatible object store
tidesdb.BackendUnknown // Unknown backend
```

Creating a Filesystem Object Store

```
store, err := tidesdb.ObjStoreFsCreate("/path/to/object/root")
if err != nil {
    log.Fatal(err)
}

config := tidesdb.Config{
    DBPath:      "./mydb",
    NumFlushThreads: 2,
    NumCompactionThreads: 2,
    LogLevel:    tidesdb.LogInfo,
    BlockCacheSize: 64 * 1024 * 1024,
    MaxOpenSSTables: 256,
    ObjectStore:  store,
}

db, err := tidesdb.Open(config)
if err != nil {
    log.Fatal(err)
}
defer db.Close()
```


Creating an S3 Object Store

The S3-compatible connector (AWS S3, MinIO, etc.) is available only when the TidesDB C library was built with `TIDESDB_WITH_S3=ON`. Because the `tidesdb_objstore_s3_*` symbols are otherwise unresolved at link time, the Go bindings are gated behind the `tidesdb_s3` **build tag**. Build and test with `-tags tidesdb_s3`:



```
go build -tags tidesdb_s3 ./...
go test -tags tidesdb_s3 ./...
```

The positional `ObjStoreS3Create` covers the common case with secure defaults:

```
store, err := tidesdb.ObjStoreS3Create(
    "s3.amazonaws.com", // endpoint (or "minio.local:9000")
    "my-bucket",         // bucket
    "production/db1/",   // prefix (may be empty)
    "AKIA...",           // access key
    "secret...",         // secret key
    "us-east-1",         // region (empty for MinIO)
    true,                // useSSL (HTTPS)
    false,               // usePathStyle (true for MinIO)
)
if err != nil {
    log.Fatal(err)
}

config := tidesdb.DefaultConfig()
config.DBPath = "./mydb"
config.ObjectStore = store
db, err := tidesdb.Open(config)
```

For TLS and multipart tuning that the positional form cannot express, use `ObjStoreS3CreateConfig` with an `S3Config`. The zero value is secure (TLS verification on, no custom CA) and uses the library's built-in multipart sizes:

```

store, err := tidesdb.ObjStoreS3CreateConfig(tidesdb.S3Config{
    Endpoint:      "minio.local:9000",
    Bucket:        "my-bucket",
    Prefix:        "production/db1/",
    AccessKey:     "minioadmin",
    SecretKey:     "minioadmin",
    Region:        "",          // empty for MinIO
    UseSSL:        true,
    UsePathStyle:  true,        // MinIO requires path-style URLs
    TLSCAPath:     "/etc/ssl/custom-ca.pem", // empty for the system
bundle
    MultipartThreshold: 64 * 1024 * 1024,          // 0 = library default
    MultipartPartSize:  8 * 1024 * 1024,          // 0 = library default
})
if err != nil {
    log.Fatal(err)
}

```

S3Config Fields

Field	Type	Description
Endpoint	string	S3 endpoint (required), e.g. "s3.amazonaws.com" or "minio.local:9000"
Bucket	string	Bucket name (required)
Prefix	string	Key prefix (e.g. "production/db1/"), may be empty
AccessKey	string	AWS access key ID (required)
SecretKey	string	AWS secret access key (required)
Region	string	AWS region (e.g. "us-east-1"), empty for the default / MinIO
UseSSL	bool	true for HTTPS, false for HTTP
UsePathStyle	bool	true for path-style URLs (MinIO), false for virtual-hosted (AWS)
TLSCAPath	string	Custom CA bundle file path, empty for the system bundle
TLSInsecureSkipVerify	bool	true disables TLS peer+host verification (test only, insecure)
MultipartThreshold	uint64	Object size at/above which multipart upload is used; 0 = default
MultipartPartSize	uint64	Multipart chunk size in bytes; 0 = default

Object Store Configuration

Use `ObjStoreDefaultConfig()` to get a configuration with sensible defaults, then override specific fields:

```
objConfig := tidesdb.ObjStoreDefaultConfig()
objConfig.LocalCachePath = "/tmp/tidesdb-cache"
objConfig.LocalCacheMaxBytes = 1024 * 1024 * 1024 // 1GB local cache
objConfig.CacheOnRead = true
objConfig.CacheOnWrite = true
objConfig.MaxConcurrentUploads = 8
objConfig.MaxConcurrentDownloads = 16

config := tidesdb.Config{
    DBPath:          "./mydb",
    ObjectStore:      store,
    ObjectStoreConfig: &objConfig,
    // ... other fields ...
}
```

ObjStoreConfig Fields

Field	Type	Description
LocalCachePath	string	Local directory for cached SSTable files (empty = use db_path)
LocalCacheMaxBytes	uint64	Max local cache size in bytes (0 = unlimited)
CacheOnRead	bool	Cache downloaded files locally (default true)
CacheOnWrite	bool	Keep local copy after upload (default true)
MaxConcurrentUploads	int	Parallel upload threads (default 4)
MaxConcurrentDownloads	int	Parallel download threads (default 8)
MultipartThreshold	uint64	Use multipart upload above this size (default 64MB)
MultipartPartSize	uint64	Multipart chunk size (default 8MB)
SyncManifestToObject	bool	Upload MANIFEST after each compaction (default true)
ReplicateWal	bool	Upload closed WAL segments for recovery (default true)
WalUploadSync	bool	Block flush until WAL uploaded (default false)
WalSyncThresholdBytes	uint64	Sync active WAL when it grows by this many bytes (default 1MB, 0 = off)
WalSyncOnCommit	bool	Upload WAL after every txn commit for RPO=0 (default false)
ReplicaMode	bool	Enable read-only replica mode (default false)
ReplicaSyncIntervalUs	uint64	MANIFEST poll interval in microseconds (default 5000000)
ReplicaReplayWal	bool	Replay WAL for near-real-time reads on replicas (default true)

Per-CF Object Store Tuning

Column family configurations include three object store tuning fields.

Field	Type	Default	Description
<code>ObjectLazyCompaction</code>	<code>int</code>	<code>0</code>	1 to compact less aggressively for remote storage
<code>ObjectPrefetchCompaction</code>	<code>int</code>	<code>1</code>	1 to download all inputs before compaction merge

Object Store Statistics

`GetDbStats` includes object store fields when a connector is active.

```

dbStats, err := db.GetDbStats()
if err != nil {
    log.Fatal(err)
}

if dbStats.ObjectStoreEnabled {
    fmt.Printf("Connector: %s\n", dbStats.ObjectStoreConnector)
    fmt.Printf("Total uploads: %d\n", dbStats.TotalUploads)
    fmt.Printf("Upload failures: %d\n", dbStats.TotalUploadFailures)
    fmt.Printf("Upload queue depth: %d\n", dbStats.UploadQueueDepth)
    fmt.Printf("Local cache: %d / %d bytes\n",
        dbStats.LocalCacheBytesUsed, dbStats.LocalCacheBytesMax)
}

```

Cold Start Recovery

When the local database directory is empty but a connector is configured, TidesDB automatically discovers column families from the object store during recovery. It downloads MANIFEST and config files in parallel, reconstructs the SSTable inventory, and fetches SSTable data on demand as queries arrive.

```

// delete all local state
os.RemoveAll("./mydb")

// reopen with the same connector -- cold start recovery
config := tidesdb.Config{
    DBPath:      "./mydb",
    ObjectStore:  store,
    ObjectStoreConfig: &objConfig,
}

db, err := tidesdb.Open(config)
if err != nil {
    log.Fatal(err)
}
defer db.Close()

// all data is available -- SSTables are fetched from the object store on
demand
cf, err := db.GetColumnFamily("my_cf")

```

How It Works

- Object store mode requires unified memtable mode. Setting `ObjectStore` on the config automatically enables `UnifiedMemtable`
- After each flush, SSTables are uploaded to the object store via an asynchronous upload pipeline with retry (3 attempts with exponential backoff) and post-upload verification (size check)
- Point lookups on frozen SSTables (not present locally) fetch just the single needed klog block (~64KB) via one HTTP range request using `range_get`, bypassing the full file download. The block is cached in the clock cache so subsequent reads are pure memory hits. If the value is in the vlog, a second range request fetches just that vlog block
- Iterators prefetch all needed SStable files in parallel at creation time using bounded threads (`MaxConcurrentDownloads`, default 8), so sequential reads proceed at local disk speed
- A hash-indexed LRU local file cache manages disk usage, evicting least-recently-used SStable file pairs (klog + vlog together) when `LocalCacheMaxBytes` is set
- The MANIFEST is uploaded asynchronously after each flush and compaction so cold start recovery can reconstruct the SStable inventory without blocking the flush worker
- Compaction runs on local files. Input SSTables are downloaded if evicted, output SSTables are uploaded after the merge
- Upload failures are tracked in `TotalUploadFailures` on `DbStats` for operator monitoring

Replica Mode

Replica mode enables read-only nodes that follow a primary through the object store. The primary handles all writes while replicas poll for MANIFEST updates and replay WAL segments for near-real-time reads.

Enabling Replica Mode

```
store, err := tidesdb.ObjStoreFsCreate("/path/to/shared/objects")
if err != nil {
    log.Fatal(err)
}

objConfig := tidesdb.ObjStoreDefaultConfig()
objConfig.ReplicaMode = true
objConfig.ReplicaSyncIntervalUs = 1000000 // 1 second sync interval
objConfig.ReplicaReplayWal = true         // replay WAL for fresh reads

config := tidesdb.Config{
    DBPath:      "./mydb_replica",
    ObjectStore:  store, // same object store as the primary
    ObjectStoreConfig: &objConfig,
}

db, err := tidesdb.Open(config)
if err != nil {
    log.Fatal(err)
}
defer db.Close()

// reads work normally
txn, _ := db.BeginTxn()
value, _ := txn.Get(cf, []byte("key"))
txn.Free()

// writes are rejected with ErrReadonly
```

Sync-on-Commit WAL (Primary Side)

For tighter replication lag, enable sync-on-commit on the primary so every committed write is uploaded to the object store immediately.


```
objConfig := tidesdb.ObjStoreDefaultConfig()
objConfig.WalSyncOnCommit = true // RPO = 0, every commit is durable in
object store

// replica sees committed data within one ReplicaSyncIntervalUs
```

Promoting a Replica to Primary

When the primary fails, promote a replica to accept writes.

```
err := db.PromoteToPrimary()
if err != nil {
    log.Fatal(err)
}

// now writes are accepted
txn, _ := db.BeginTxn()
txn.Put(cf, []byte("key"), []byte("value"), -1)
txn.Commit()
txn.Free()
```

`PromoteToPrimary` performs a final MANIFEST sync and WAL replay, creates a local WAL for crash recovery, and atomically switches to primary mode. The function returns an error if the node is already a primary.

How Replica Sync Works

- The reaper thread polls the remote MANIFEST for each CF every `ReplicaSyncIntervalUs`
- New SSTables from the primary's flushes and compactions are added to the replica's levels
- SSTables compacted away on the primary are removed from the replica's levels
- When `ReplicaReplayWal` is enabled, the latest WAL is downloaded and replayed into the memtable for near-real-time reads
- WAL replay is idempotent using sequence numbers so entries already present are skipped
- SSTable data is not downloaded during sync. It is fetched on demand via `range_get` for point lookups or prefetch for iterators

Configuration Persistence (INI)

Column family configurations can be saved to and loaded from INI files.

Save Configuration to INI

```
cfConfig := tidesdb.DefaultColumnFamilyConfig()
cfConfig.WriteBufferSize = 128 * 1024 * 1024
cfConfig.CompressionAlgorithm = tidesdb.ZstdCompression

err := tidesdb.CfConfigSaveToIni("config.ini", "my_cf", cfConfig)
if err != nil {
    log.Fatal(err)
}
```

Load Configuration from INI

```
loaded, err := tidesdb.CfConfigLoadFromIni("config.ini", "my_cf")
if err != nil {
    log.Fatal(err)
}

err = db.CreateColumnFamily("my_cf", *loaded)
if err != nil {
    log.Fatal(err)
}
```

Error Handling

```
cf, err := db.GetColumnFamily("my_cf")
if err != nil {
    log.Fatal(err)
}

txn, err := db.BeginTxn()
if err != nil {
    log.Fatal(err)
}
defer txn.Free()

err = txn.Put(cf, []byte("key"), []byte("value"), -1)
if err != nil {
    // Errors include context and error codes
    fmt.Printf("Error: %v\n", err)

    // Example error message:
    // "failed to put key-value pair: memory allocation failed (code: -1)"

    txn.Rollback()
    return
}

err = txn.Commit()
if err != nil {
    log.Fatal(err)
}
```

Error Codes

- `ErrSuccess` (0) · Operation successful
- `ErrMemory` (-1) · Memory allocation failed
- `ErrInvalidArgs` (-2) · Invalid arguments
- `ErrNotFound` (-3) · Key not found
- `ErrIO` (-4) · I/O error
- `ErrCorruption` (-5) · Data corruption
- `ErrExists` (-6) · Resource already exists
- `ErrConflict` (-7) · Transaction conflict

- `ErrTooLarge` (-8) · Key or value too large
- `ErrMemoryLimit` (-9) · Memory limit exceeded
- `ErrInvalidDB` (-10) · Invalid database handle
- `ErrUnknown` (-11) · Unknown error
- `ErrLocked` (-12) · Database is locked
- `ErrReadonly` (-13) · Database is read-only (replica mode)
- `ErrBusy` (-14) · Resource busy
- `ErrPrecondition` (-15) · Precondition failed

Complete Example

```
package main

import (
    "fmt"
    "log"
    "time"

    tidesdb "github.com/tidesdb/tidesdb-go"
)

func main() {
    config := tidesdb.Config{
        DBPath:          "./example_db",
        NumFlushThreads: 1,
        NumCompactionThreads: 1,
        LogLevel:         tidesdb.LogInfo,
        BlockCacheSize:   64 * 1024 * 1024,
        MaxOpenSSTables: 256,
    }

    db, err := tidesdb.Open(config)
    if err != nil {
        log.Fatal(err)
    }
    defer db.Close()

    cfConfig := tidesdb.DefaultColumnFamilyConfig()
    cfConfig.WriteBufferSize = 64 * 1024 * 1024
    cfConfig.CompressionAlgorithm = tidesdb.LZ4Compression
    cfConfig.EnableBloomFilter = true
    cfConfig.BloomFPR = 0.01
    cfConfig.SyncMode = tidesdb.SyncInterval
    cfConfig.SyncIntervalUs = 128000

    err = db.CreateColumnFamily("users", cfConfig)
    if err != nil {
        log.Fatal(err)
    }
    defer db.DropColumnFamily("users")

    cf, err := db.GetColumnFamily("users")
    if err != nil {
```

```

        log.Fatal(err)
    }

    txn, err := db.BeginTxn()
    if err != nil {
        log.Fatal(err)
    }

    err = txn.Put(cf, []byte("user:1"), []byte("Alice"), -1)
    if err != nil {
        txn.Rollback()
        log.Fatal(err)
    }

    err = txn.Put(cf, []byte("user:2"), []byte("Bob"), -1)
    if err != nil {
        txn.Rollback()
        log.Fatal(err)
    }

    ttl := time.Now().Add(30 * time.Second).Unix()
    err = txn.Put(cf, []byte("session:abc"), []byte("temp_data"), ttl)
    if err != nil {
        txn.Rollback()
        log.Fatal(err)
    }

    err = txn.Commit()
    if err != nil {
        log.Fatal(err)
    }
    txn.Free()

    readTxn, err := db.BeginTxn()
    if err != nil {
        log.Fatal(err)
    }
    defer readTxn.Free()

    value, err := readTxn.Get(cf, []byte("user:1"))
    if err != nil {
        log.Fatal(err)
    }
    fmt.Printf("user:1 = %s\n", value)

    iter, err := readTxn.NewIterator(cf)

```

```

if err != nil {
    log.Fatal(err)
}
defer iter.Free()

fmt.Println("\nAll entries:")
iter.SeekToFirst()
for iter.Valid() {
    key, _ := iter.Key()
    value, _ := iter.Value()
    fmt.Printf("  %s = %s\n", key, value)
    iter.Next()
}

stats, err := cf.GetStats()
if err != nil {
    log.Fatal(err)
}

fmt.Printf("\nColumn Family Statistics:\n")
fmt.Printf("  Number of Levels: %d\n", stats.NumLevels)
fmt.Printf("  Memtable Size: %d bytes\n", stats.MemtableSize)
}

```

Isolation Levels

TidesDB supports five MVCC isolation levels:

```

txn, err := db.BeginTxnWithIsolation(tidesdb.IsolationReadCommitted)
if err != nil {
    log.Fatal(err)
}
defer txn.Free()

```

Available Isolation Levels

- `IsolationReadUncommitted` · Sees all data including uncommitted changes
- `IsolationReadCommitted` · Sees only committed data (default)
- `IsolationRepeatableRead` · Consistent snapshot, phantom reads possible
- `IsolationSnapshot` · Write-write conflict detection

- `IsolationSerializable` · Full read-write conflict detection (SSI)

Savepoints

Savepoints allow partial rollback within a transaction:

```
txn, err := db.BeginTxn()
if err != nil {
    log.Fatal(err)
}
defer txn.Free()

err = txn.Put(cf, []byte("key1"), []byte("value1"), -1)

err = txn.Savepoint("sp1")
err = txn.Put(cf, []byte("key2"), []byte("value2"), -1)

// Rollback to savepoint -- key2 is discarded, key1 remains
err = txn.RollbackToSavepoint("sp1")

// Or release savepoint without rolling back
err = txn.ReleaseSavepoint("sp1")

// Commit -- only key1 is written
err = txn.Commit()
```

Savepoint API

- `Savepoint(name string)` · Create a savepoint
- `RollbackToSavepoint(name string)` · Rollback to savepoint
- `ReleaseSavepoint(name string)` · Release savepoint without rolling back

Transaction Reset

`Reset` resets a committed or aborted transaction for reuse with a new isolation level. This avoids the overhead of freeing and reallocating transaction resources in hot loops.


```

txn, err := db.BeginTxn()
if err != nil {
    log.Fatal(err)
}

// First batch of work
err = txn.Put(cf, []byte("key1"), []byte("value1"), -1)
if err != nil {
    log.Fatal(err)
}
err = txn.Commit()
if err != nil {
    log.Fatal(err)
}

// Reset instead of Free + BeginTxn
err = txn.Reset(tidesdb.IsolationReadCommitted)
if err != nil {
    log.Fatal(err)
}

// Second batch of work using the same transaction
err = txn.Put(cf, []byte("key2"), []byte("value2"), -1)
if err != nil {
    log.Fatal(err)
}
err = txn.Commit()
if err != nil {
    log.Fatal(err)
}

// Free once when done
txn.Free()

```

Behavior

- The transaction must be committed or aborted before reset; resetting an active transaction returns an error
- Internal buffers are retained to avoid reallocation
- A fresh transaction ID and snapshot sequence are assigned based on the new isolation level
- The isolation level can be changed on each reset (e.g., `IsolationReadCommitted` -> `IsolationRepeatableRead`)

When to use

- Batch processing · Reuse a single transaction across many commit cycles in a loop
- Connection pooling · Reset a transaction for a new request without reallocation
- High-throughput ingestion · Reduce malloc/free overhead in tight write loops

Reset vs Free + BeginTxn

For a single transaction, `Reset` is functionally equivalent to calling `Free` followed by `BeginTxnWithIsolation`. The difference is performance, reset retains allocated buffers and avoids repeated allocation overhead. This matters most in loops that commit and restart thousands of transactions.

B+tree KLog Format

Column families can optionally use a B+tree structure for the key log instead of the default block-based format.

```
cfConfig := tidesdb.DefaultColumnFamilyConfig()
cfConfig.UseBtree = 1 // Enable B+tree klog format

err := db.CreateColumnFamily("btree_cf", cfConfig)
if err != nil {
    log.Fatal(err)
}
```

Characteristics

- Point lookups · $O(\log N)$ tree traversal with binary search at each node
- Range scans · Doubly-linked leaf nodes enable efficient bidirectional iteration
- Immutable · Tree is bulk-loaded from sorted memtable data during flush

When to use B+tree klog format

- Read-heavy workloads with frequent point lookups
- Workloads where read latency is more important than write throughput
- Large SSTables where block scanning becomes expensive

Important

`UseBtree` cannot be changed after column family creation.

Log Levels

TidesDB provides structured logging with multiple severity levels.

```
config := tidesdb.Config{
    DBPath:    "./mydb",
    LogLevel:  tidesdb.LogDebug,
}
```

Available Log Levels

- `LogDebug` · Detailed diagnostic information
- `LogInfo` · General informational messages (default)
- `LogWarn` · Warning messages for potential issues
- `LogError` · Error messages for failures
- `LogFatal` · Critical errors that may cause shutdown
- `LogNone` · Disable all logging

Log to file

```
config := tidesdb.Config{
    DBPath:    "./mydb",
    LogLevel:  tidesdb.LogDebug,
    LogToFile: true,                               // Write to ./mydb/LOG
    instead of stderr
    LogTruncationAt: 24 * 1024 * 1024,           // Truncate log file at 24MB
}
```

Testing



```
# Run all tests
go test -v

# Run specific test
go test -v -run TestOpenClose

# Run with race detector
go test -race -v

# Run B+tree test
go test -v -run TestBtreeColumnFamily

# Run clone column family test
go test -v -run TestCloneColumnFamily

# Run transaction reset test
go test -v -run TestTransactionReset

# Run checkpoint test
go test -v -run TestCheckpoint

# Run range cost estimation test
go test -v -run TestRangeCost

# Run delete column family by pointer test
go test -v -run TestDeleteColumnFamily

# Run commit hook (CDC) tests
go test -v -run TestCommitHook
go test -v -run TestCommitHookReplace
go test -v -run TestCommitHookClear

# Run backup test
go test -v -run TestBackup

# Run rename column family test
go test -v -run TestRenameColumnFamily

# Run update runtime config test
go test -v -run TestUpdateRuntimeConfig
```

```
# Run flush/compaction status test
go test -v -run TestIsFlushingIsCompacting

# Run get comparator test
go test -v -run TestGetComparator

# Run WAL sync test
go test -v -run TestSyncWal

# Run purge column family test
go test -v -run TestPurgeCF

# Run purge database test
go test -v -run TestPurge

# Run database-level stats test
go test -v -run TestGetDbStats

# Run iterator key-value test
go test -v -run TestIterKeyValue

# Run init/finalize test
go test -v -run TestInitFinalize

# Run object store config tests
go test -v -run TestObjStoreDefaultConfig
go test -v -run TestObjStoreFsCreate
go test -v -run TestObjStoreBackendConstants

# Run column family config object store fields test
go test -v -run TestColumnFamilyConfigObjectStoreFields

# Run db stats object store fields test
go test -v -run TestDbStatsObjectStoreFields

# Run INI config test
go test -v -run TestCfConfigIni

# Run error code readonly test
go test -v -run TestErrorCodeReadonly

# Run single-delete transaction test
go test -v -run TestTransactionSingleDelete

# Run raise open-file limit test
```

```
go test -v -run TestRaiseOpenFileLimit

# Run cancel background work test
go test -v -run TestCancelBackgroundWork

# Run finish-compactions-on-close config test
go test -v -run TestFinishCompactionsOnCloseConfig

# Run busy error code test
go test -v -run TestErrorCodeBusy

# Run built-in comparator name + uint64 ordering tests
go test -v -run TestBuiltinComparatorNames
go test -v -run TestUint64ComparatorOrdering

# Run comparator context string INI round-trip test
go test -v -run TestComparatorCtxStrIni

# Run write-amplification stats tests
go test -v -run TestStatsWriteAmplification
go test -v -run TestDbStatsWriteAmplification

# Run S3 object store connector tests (requires a library built with
TIDESDB_WITH_S3=ON,
# plus a reachable endpoint via TIDESDB_S3_ENDPOINT / TIDESDB_S3_BUCKET /
etc.)
go test -tags tidesdb_s3 -v -run TestObjStoreS3
```

TidesDB Python API Reference

Getting Started

Prerequisites

You **must** have the TidesDB shared C library installed on your system. You can find the installation instructions [here](#).

Installation

Or install from source:



```
git clone https://github.com/tidesdb/tidesdb-python.git
cd tidesdb-python
pip install -e .
```

Quick Start

```
import tidesdb

db = tidesdb.TidesDB.open("./mydb")

config = tidesdb.default_column_family_config()
config.compression_algorithm = tidesdb.CompressionAlgorithm.LZ4_COMPRESSION
db.create_column_family("users", config)

cf = db.get_column_family("users")

with db.begin_txn() as txn:
    txn.put(cf, b"user:1", b"Alice")
    txn.put(cf, b"user:2", b"Bob")
    txn.commit()

with db.begin_txn() as txn:
    value = txn.get(cf, b"user:1")
    print(f"user:1 = {value.decode()}")

with db.begin_txn() as txn:
    with txn.new_iterator(cf) as it:
        it.seek_to_first()
        for key, value in it:
            print(f"{key.decode()} = {value.decode()}")

db.drop_column_family("users")
db.close()
```


Usage

Opening a Database

```
import tidesdb

db = tidesdb.TidesDB.open("./mydb")

# With configuration
config = tidesdb.Config(
    db_path="./mydb",
    num_flush_threads=2,
    num_compaction_threads=2,
    log_level=tidesdb.LogLevel.LOG_INFO,
    block_cache_size=64 * 1024 * 1024,           # 64MB
    max_open_sstables=256,
    max_memory_usage=0,                          # Global memory
    limit in bytes (0 = auto, 50% of system RAM)
    log_to_file=False,                          # Write logs to
    file instead of stderr
    log_truncation_at=24 * 1024 * 1024,         # Log file
    truncation size (24MB)
    unified_memtable=False,                     # Enable
    unified memtable mode
    unified_memtable_write_buffer_size=0,       # Unified
    memtable buffer size (0 = default)
    unified_memtable_skip_list_max_level=0,     # Unified
    memtable skip list max level
    unified_memtable_skip_list_probability=0.0, # Unified
    memtable skip list probability
    unified_memtable_sync_mode=tidesdb.SyncMode.SYNC_NONE, # Unified
    memtable WAL sync mode
    unified_memtable_sync_interval_us=0,       # Unified
    memtable sync interval in microseconds
    object_store=None,                         # Object store
    connector (from objstore_fs_create())
    object_store_config=None,                  # Object store
    behavior config (ObjStoreConfig)
    max_concurrent_flushes=0,                  # Cap on in-
    flight memtable flushes across all CFs (0 = library default)
    finish_compactions_on_close=False,          # False =
    cancel in-flight compactions at close for a fast shutdown; True = run them
    to completion
)
```

```
db = tidesdb.TidesDB(config)

# Using context manager
with tidesdb.TidesDB.open("./mydb") as db:
    # ... use database
    pass # automatically closed
```

`tidesdb.default_config()` returns a `Config` whose values are pulled directly from the underlying C library, so settings such as `max_concurrent_flushes` and the unified-memtable defaults track the engine's defaults automatically. Anyone overriding the previously hardcoded Python defaults should re-test after upgrading.

max_concurrent_flushes

`max_concurrent_flushes` is a global semaphore on the number of in-flight memtable flushes across all column families. It bounds total transient memory and work-queue depth when many column families flush at once. Leave it at `0` to inherit the library default (recommended), or lower it on memory-constrained hosts to throttle peak flush concurrency.

finish_compactions_on_close

`finish_compactions_on_close` controls shutdown behavior. The default `False` cancels in-flight compactions at their next checkpoint so `close()` returns quickly — the merge discards its uncommitted output and leaves the inputs intact, so **no data is lost** (recovery handles a mid-merge state the same way). Set it to `True` to let in-flight compactions run to completion before `close()` returns, which can leave the LSM tree in a more compacted state at the cost of a slower shutdown.

Initialization and Process Limits

TidesDB lazily initializes itself on first use, so calling `init()` is optional. It exists so an embedder can initialize the library up front, and a matching `finalize()` resets the allocator once every database handle is closed. Both are process-wide and safe to call more than once.

```
import tidesdb

# Optional: initialize up front (otherwise done lazily on first use)
tidesdb.init()

# ... open databases, do work, close every handle ...

# Optional: finalize after all databases are closed
tidesdb.finalize()
```

`raise_open_file_limit()` raises this process's open-file ceiling so a database can keep more SSTables open. The engine sizes `max_open_sstables` to fit the ceiling at open time, so call this **before** opening a database. It is an explicit, opt-in operator action — TidesDB never raises the limit itself — and a failed or partial raise is non-fatal.

```
# Raise toward 65536 descriptors; returns the ceiling actually in
effect
ceiling = tidesdb.raise_open_file_limit(65536)
print(f"Open-file ceiling now: {ceiling}")

# A non-positive value just reports the current ceiling without changing it
current = tidesdb.raise_open_file_limit(-1)
```

❗ Custom allocators

The C API lets `init()` accept custom `malloc/calloc/realloc/free` functions. The Python binding intentionally does **not** expose these — the lifetime of Python-side callbacks cannot be safely guaranteed for the entire life of the process — so the default system allocator is always used.

Column Families

```
db.create_column_family("my_cf")

# Create with custom config
config = tidesdb.default_column_family_config()
config.write_buffer_size = 128 * 1024 * 1024 # 128MB
config.compression_algorithm =
tidesdb.CompressionAlgorithm.ZSTD_COMPRESSION
config.enable_bloom_filter = True
config.bloom_fpr = 0.01
config.sync_mode = tidesdb.SyncMode.SYNC_INTERVAL
config.sync_interval_us = 128000 # 128ms
config.klog_value_threshold = 512 # Values > 512 bytes go to
vlog
config.min_disk_space = 100 * 1024 * 1024 # 100MB minimum disk space
config.default_isolation_level = tidesdb.IsolationLevel.READ_COMMITTED
config.l1_file_count_trigger = 4 # L1 compaction trigger
config.l0_queue_stall_threshold = 20 # L0 backpressure threshold
db.create_column_family("my_cf", config)

cf = db.get_column_family("my_cf")

names = db.list_column_families()
print(names)

stats = cf.get_stats()
print(f"Levels: {stats.num_levels}, Memtable: {stats.memtable_size} bytes")
print(f"Total keys: {stats.total_keys}, Total data size:
{stats.total_data_size} bytes")
print(f"Avg key size: {stats.avg_key_size:.2f}, Avg value size:
{stats.avg_value_size:.2f}")
print(f"Read amplification: {stats.read_amp:.2f}, Hit rate:
{stats.hit_rate:.2%}")
print(f"Keys per level: {stats.level_key_counts}")

# Tombstone observability
print(f"Total tombstones: {stats.total_tombstones}")
print(f"Tombstone ratio: {stats.tombstone_ratio:.2%}")
print(f"Tombstones per level: {stats.level_tombstone_counts}")
print(
    f"Worst SSTable density: {stats.max_sst_density:.2%} "
    f"(level {stats.max_sst_density_level})"
)
```

```

# B+tree klog stats (only populated if use_btree=True)
if stats.use_btree:
    print(f"B+tree total nodes: {stats.btree_total_nodes}")
    print(f"B+tree max height: {stats.btree_max_height}")
    print(f"B+tree avg height: {stats.btree_avg_height:.2f}")

# Per-CF write-amplification counters (lifetime since open, on-disk framed
bytes).
# wal_bytes_written is 0 in unified-memtable mode (reported db-wide
instead).
print(f"WAL bytes written: {stats.wal_bytes_written}")
print(f"Flush bytes written: {stats.flush_bytes_written}")
print(f"Compaction bytes written: {stats.compaction_bytes_written}")
print(f"Compaction bytes read: {stats.compaction_bytes_read}")
print(f"User bytes written: {stats.user_bytes_written}")
print(f"Flush count: {stats.flush_count}, Compaction count:
{stats.compaction_count}")
if stats.user_bytes_written > 0:
    cf_wa = (
        stats.wal_bytes_written
        + stats.flush_bytes_written
        + stats.compaction_bytes_written
    ) / stats.user_bytes_written
    print(f"CF write amplification: {cf_wa:.2f}x")

db.rename_column_family("my_cf", "new_cf")

db.drop_column_family("new_cf")

# Delete by pointer (skips name lookup, faster when you already have the
handle)
cf = db.get_column_family("my_cf")
db.delete_column_family(cf)

```

Cloning a Column Family

Create a complete copy of an existing column family with a new name. The clone contains all the data from the source at the time of cloning and is completely independent.

```
# Clone an existing column family
db.clone_column_family("source_cf", "cloned_cf")

# Both column families now exist independently
source = db.get_column_family("source_cf")
clone = db.get_column_family("cloned_cf")

# Modifications to one do not affect the other
with db.begin_txn() as txn:
    txn.put(source, b"key", b"new_value")
    txn.commit()

with db.begin_txn() as txn:
    # clone still has the original value
    value = txn.get(clone, b"key")
```

Use cases

- Testing · Create a copy of production data for testing without affecting the original
- Branching · Create a snapshot of data before making experimental changes
- Migration · Clone data before schema or configuration changes
- Backup verification · Clone and verify data integrity without modifying the source

Clone vs Backup

`clone_column_family` creates a new column family within the same database instance. For creating an external backup of the entire database, use `backup()` instead.

Transactions

```
txn = db.begin_txn()

try:
    # Put key-value pairs (TTL -1 means no expiration)
    txn.put(cf, b"key1", b"value1", ttl=-1)
    txn.put(cf, b"key2", b"value2", ttl=-1)

    value = txn.get(cf, b"key1")

    txn.delete(cf, b"key2")

    txn.commit()
except tidesdb.TidesDBError as e:
    txn.rollback()
    raise
finally:
    txn.close()

# Using context manager (auto-rollback on exception, auto-close)
with db.begin_txn() as txn:
    txn.put(cf, b"key", b"value")
    txn.commit()
```

Single-Delete

`single_delete` writes a tombstone with the same read semantics as `delete`, but carries a caller-provided promise that lets compaction drop the put and the tombstone together as soon as both appear in the same merge input, rather than carrying the tombstone forward until it reaches the largest active level.

Between any two single-deletes on the same key, and between the start of the key's history and its first single-delete, the key has been put **at most once**. The engine does not and cannot verify this at runtime; violating the contract can leave older puts visible after the single-delete and is a bug in the caller.

This is the right choice for workloads that insert each key exactly once and then delete it exactly once (classic insert-benchmark patterns, secondary-index entries on columns that are never updated, log-style tables with scheduled purges). It is **not** safe for tables that issue repeated updates to the same key.

```
cf = db.get_column_family("my_cf")

with db.begin_txn() as txn:
    txn.single_delete(cf, b"mykey")
    txn.commit()
```

When in doubt, prefer `delete`.

Transaction Reset

`reset()` resets a committed or aborted transaction for reuse with a new isolation level. This avoids the overhead of freeing and reallocating transaction resources in hot loops.

```
txn = db.begin_txn()

# First batch of work
txn.put(cf, b"key1", b"value1")
txn.commit()

# Reset instead of close + begin_txn
txn.reset(tidesdb.IsolationLevel.READ_COMMITTED)

# Second batch of work using the same transaction
txn.put(cf, b"key2", b"value2")
txn.commit()

# Free once when done
txn.close()
```

Batch processing example


```
txn = db.begin_txn()

for batch in batches:
    for key, value in batch:
        txn.put(cf, key, value)
    txn.commit()
    txn.reset(tidesdb.IsolationLevel.READ_COMMITTED)

txn.close()
```

Behavior

- The transaction must be committed or rolled back before reset; resetting an active transaction raises `TidesDBError`
- Internal buffers are retained to avoid reallocation
- A fresh transaction ID and snapshot sequence are assigned
- The isolation level can be changed on each reset

When to use

- Batch processing · Reuse a single transaction across many commit cycles in a loop
- Connection pooling · Reset a transaction for a new request without reallocation
- High-throughput ingestion · Reduce allocation overhead in tight write loops

Reset vs Close + Begin

`txn.reset()` is functionally equivalent to `txn.close()` followed by `db.begin_txn()`. The difference is performance, reset retains allocated buffers and avoids repeated allocation overhead.

TTL (Time-To-Live)

```
import time

# Set TTL as Unix timestamp (seconds since epoch)
ttl = int(time.time()) + 60 # Expires in 60 seconds

with db.begin_txn() as txn:
    txn.put(cf, b"temp_key", b"temp_value", ttl=ttl)
    txn.commit()
```

Isolation Levels

```
txn = db.begin_txn_with_isolation(tidesdb.IsolationLevel.SERIALIZABLE)

# Available levels
# -- READ_UNCOMMITTED      -- Sees all data including uncommitted changes
# -- READ_COMMITTED        -- Sees only committed data (default)
# -- REPEATABLE_READ       -- Consistent snapshot, phantom reads possible
# -- SNAPSHOT              -- Write-write conflict detection
# -- SERIALIZABLE          -- Full read-write conflict detection (SSI)
```

Savepoints

```
with db.begin_txn() as txn:
    txn.put(cf, b"key1", b"value1")

    txn.savepoint("sp1")

    txn.put(cf, b"key2", b"value2")

    # Rollback to savepoint (key2 discarded, key1 remains)
    txn.rollback_to_savepoint("sp1")

    # Or release savepoint without rollback
    # txn.release_savepoint("sp1")

    txn.commit() # Only key1 is written
```

Iterators

```
with db.begin_txn() as txn:
    with txn.new_iterator(cf) as it:
        # Forward iteration
        it.seek_to_first()
        while it.valid():
            key = it.key()
            value = it.value()
            print(f"{key} = {value}")
            it.next()

        # Backward iteration
        it.seek_to_last()
        while it.valid():
            print(f"{it.key()} = {it.value()}")
            it.prev()

        # Seek to specific key
        it.seek(b"user:") # First key >= "user:"
        it.seek_for_prev(b"user:z") # Last key <= "user:z"

        # Combined key+value retrieval (more efficient than separate
        key()/value())
        it.seek_to_first()
        while it.valid():
            key, value = it.key_value()
            print(f"{key} = {value}")
            it.next()

        # Python iteration protocol (uses key_value() internally)
        it.seek_to_first()
        for key, value in it:
            print(f"{key} = {value}")
```

Maintenance Operations

```
# Manual compaction (queues compaction)
cf.compact()

# Manual memtable flush (sorted run to L1)
cf.flush_memtable()

# Check if flush/compaction is in progress
if cf.is_flushing():
    print("Flush in progress")
if cf.is_compacting():
    print("Compaction in progress")

# Get cache statistics
cache_stats = db.get_cache_stats()
print(f"Cache hits: {cache_stats.hits}, misses: {cache_stats.misses}")
print(f"Hit rate: {cache_stats.hit_rate:.2%}")
```

Targeted Range Compaction

`compact_range` runs a synchronous compaction over a specific key range. Only SSTables whose minimum and maximum keys overlap the requested range participate in the merge, so the work and I/O are bounded to the affected portion of the LSM tree rather than the whole column family.

```
cf = db.get_column_family("my_cf")

start = b"tenant_42:"
end = b"tenant_42;"

cf.compact_range(start, end)

# Pass None for an unbounded endpoint
cf.compact_range(None, b"tenant_42;") # everything below "tenant_42;"
cf.compact_range(b"tenant_42:", None) # everything from "tenant_42:"
onward
```

When to use

- Bulk reclaim after a large range delete, where waiting for natural compaction would leave tombstones and obsolete versions on disk

- Tenant eviction or sliding-window expiration that does not fit TTL semantics
- Post-import cleanup of a known key range loaded with `put` followed by `delete`
- Operational counterpart to the automatic tombstone density trigger when an operator wants reclaim now rather than at the next natural threshold crossing

Behavior

- Synchronous, blocks the caller until the merge commits or fails
- Does not enqueue work onto the compaction thread pool; the calling thread does the work
- Selects only SSTables whose key range overlaps the requested range using the column family's comparator; SSTables outside the range are not touched
- Applies the same emit-loop logic as background compactions (tombstone reclamation rules, single-delete pair cancellation, sequence-based deduplication, value recompression)
- Output SSTables are committed to the manifest atomically and old inputs are marked for deletion

Return values

- Returns `None` on success
- Raises `TidesDBError` with code `TDB_ERR_INVALID_ARGS` if both endpoints are `None`/empty (use `compact()` for full CF compaction)
- Raises `TidesDBError` with code `TDB_ERR_LOCKED` if another compaction is running for the column family
- Standard I/O and memory error codes if the merge cannot complete

Manual WAL Sync

`sync_wal()` forces an immediate `fsync` of the active write-ahead log for a column family. This is useful for explicit durability control when using `SYNC_NONE` or `SYNC_INTERVAL` modes.

```
cf = db.get_column_family("my_cf")

# Force WAL durability after a batch of writes
cf.sync_wal()
```

When to use

- Application-controlled durability · Sync the WAL at specific points (e.g., after a batch of related writes) when using `SYNC_NONE` or `SYNC_INTERVAL`
- Pre-checkpoint · Ensure all buffered WAL data is on disk before taking a checkpoint

- Graceful shutdown · Flush WAL buffers before closing the database
- Critical writes · Force durability for specific high-value writes without using `SYNC_FULL` for all writes

Structural Operations

Regardless of sync mode, TidesDB **always** enforces durability for structural operations, memtable flush, SSTable compaction, WAL rotation, and column family metadata updates.

Purge Column Family

`purge()` forces a synchronous flush and aggressive compaction for a single column family. Unlike `flush_memtable()` and `compact()` (which are non-blocking), `purge` blocks until all flush and compaction I/O is complete.

```
cf = db.get_column_family("my_cf")

cf.purge()
# All data is now flushed to SSTables and compacted
```

Behavior

1. Waits for any in-progress flush to complete
2. Force-flushes the active memtable (even if below threshold)
3. Waits for flush I/O to fully complete
4. Waits for any in-progress compaction to complete
5. Triggers synchronous compaction inline (bypasses the compaction queue)
6. Waits for any queued compaction to drain

When to use

- Before backup or checkpoint · Ensure all data is on disk and compacted
- After bulk deletes · Reclaim space immediately by compacting away tombstones
- Manual maintenance · Force a clean state during a maintenance window
- Pre-shutdown · Ensure all pending work is complete before closing

Purge Database

`purge()` on the database forces a synchronous flush and aggressive compaction for **all** column families, then drains both the global flush and compaction queues.

```
db.purge()  
# All CFs flushed and compacted, all queues drained
```

Behavior

1. Calls purge on each column family
2. Drains the global flush queue (waits for queue size and pending count to reach 0)
3. Drains the global compaction queue (waits for queue size to reach 0)

Purge vs Manual Flush + Compact

`flush_memtable()` and `compact()` are non-blocking - they enqueue work and return immediately. `cf.purge()` and `db.purge()` are synchronous - they block until all work is complete. Use purge when you need a guarantee that all data is on disk and compacted before proceeding.

Fast Shutdown

`cancel_background_work()` cancels background compaction database-wide so a subsequent `close()` returns quickly. It is intended to be called right before `close()` on databases carrying a heavy compaction backlog.

```
db.cancel_background_work()  
db.close()
```

Behavior

- In-flight merges bail out safely — their uncommitted output is discarded and the inputs are left intact, so **no data is lost**
- Queued compaction tasks are skipped
- Flushes are **unaffected**, so durability is preserved
- The call blocks (bounded) until compaction is idle
- The cancellation is sticky for the session and is reset on the next `open()`

`finish_compactions_on_close` vs `cancel_background_work`

These are two ends of the same dial. `finish_compactions_on_close=True` makes `close()` wait for in-flight compactions to *finish*; `cancel_background_work()` does the opposite, actively *cancelling* them for the fastest possible shutdown. Use the latter when shutdown latency matters more than leaving the tree maximally compacted.

Database-Level Statistics

Get aggregate statistics across the entire database instance.

```
db_stats = db.get_db_stats()
print(f"Column families: {db_stats.num_column_families}")
print(f"Total memory: {db_stats.total_memory} bytes")
print(f"Resolved memory limit: {db_stats.resolved_memory_limit} bytes")
print(f"Memory pressure level: {db_stats.memory_pressure_level}")
print(f"Global sequence: {db_stats.global_seq}")
print(f"Flush queue: {db_stats.flush_queue_size} pending")
print(f"Compaction queue: {db_stats.compaction_queue_size} pending")
print(f"Total SSTables: {db_stats.total_sstable_count}")
print(f"Total data size: {db_stats.total_data_size_bytes} bytes")
print(f"Open SSTable handles: {db_stats.num_open_sstables}")
print(f"In-flight txn memory: {db_stats.txn_memory_bytes} bytes")
print(f"Immutable memtables: {db_stats.total_immutable_count}")
print(f"Memtable bytes: {db_stats.total_memtable_bytes}")

# Unified memtable stats (when enabled)
if db_stats.unified_memtable_enabled:
    print(f"Unified memtable bytes: {db_stats.unified_memtable_bytes}")
    print(f"Unified immutable count: {db_stats.unified_immutable_count}")
    print(f"Unified is flushing: {db_stats.unified_is_flushing}")
    print(f"Unified WAL generation: {db_stats.unified_wal_generation}")

# Object store stats (when enabled)
if db_stats.object_store_enabled:
    print(f"Object store connector: {db_stats.object_store_connector}")
    print(f"Local cache:
{db_stats.local_cache_bytes_used}/{db_stats.local_cache_bytes_max} bytes")
    print(f"Total uploads: {db_stats.total_uploads}")
    print(f"Replica mode: {db_stats.replica_mode}")
```

Database statistics include

Field	Type	Description
num_column_families	int	Number of column families
total_memory	int	System total memory
available_memory	int	System available memory at open time
resolved_memory_limit	int	Resolved memory limit (auto or configured)
memory_pressure_level	int	Current memory pressure (0=normal, 1=elevated, 2=high, 3=critical)
flush_pending_count	int	Number of pending flush operations (queued + in-flight)
total_memtable_bytes	int	Total bytes in active memtables across all CFs
total_immutable_count	int	Total immutable memtables across all CFs
total_sstable_count	int	Total SSTables across all CFs and levels
total_data_size_bytes	int	Total data size (klog + vlog) across all CFs
num_open_sstables	int	Number of currently open SSTable file handles
global_seq	int	Current global sequence number
txn_memory_bytes	int	Bytes held by in-flight transactions
compaction_queue_size	int	Number of pending compaction tasks
flush_queue_size	int	Number of pending flush tasks in queue
unified_memtable_enabled	bool	Whether unified memtable mode is active
unified_memtable_bytes	int	Bytes in the unified memtable
unified_immutable_count	int	Number of unified immutable memtables

Field	Type	Description
<code>unified_is_flushing</code>	<code>bool</code>	Whether the unified memtable is flushing
<code>unified_next_cf_index</code>	<code>int</code>	Next column family index for unified memtable
<code>unified_wal_generation</code>	<code>int</code>	Current unified WAL generation number
<code>object_store_enabled</code>	<code>bool</code>	Whether object store mode is active
<code>object_store_connector</code>	<code>str</code>	Object store connector identifier
<code>local_cache_bytes_used</code>	<code>int</code>	Bytes used in local cache
<code>local_cache_bytes_max</code>	<code>int</code>	Maximum local cache size in bytes
<code>local_cache_num_files</code>	<code>int</code>	Number of files in local cache
<code>last_uploaded_generation</code>	<code>int</code>	Last uploaded WAL generation
<code>upload_queue_depth</code>	<code>int</code>	Pending uploads in queue
<code>total_uploads</code>	<code>int</code>	Total successful uploads
<code>total_upload_failures</code>	<code>int</code>	Total failed uploads
<code>replica_mode</code>	<code>bool</code>	Whether the database is in replica mode
<code>primary_epoch</code>	<code>int</code>	Single-writer fencing lease epoch this primary currently holds (0 when not a primary / no lease)
<code>seen_epoch</code>	<code>int</code>	Highest fencing lease epoch a replica has observed (0 in local mode)
<code>uwal_bytes_written</code>	<code>int</code>	Framed bytes appended to the shared unified WAL (0 if unified mode off)
<code>wal_bytes_written</code>	<code>int</code>	Per-CF WAL bytes summed across all column families

Field	Type	Description
<code>flush_bytes_written</code>	<code>int</code>	Flush output bytes summed across all column families
<code>compaction_bytes_written</code>	<code>int</code>	Compaction output bytes summed across all column families
<code>compaction_bytes_read</code>	<code>int</code>	Compaction input bytes summed across all column families
<code>user_bytes_written</code>	<code>int</code>	Logical committed key+value bytes summed across all column families
<code>flush_count</code>	<code>int</code>	Flushed SSTables summed across all column families
<code>compaction_count</code>	<code>int</code>	Compaction output SSTables summed across all column families

The trailing fields are **write-amplification counters** (lifetime since open, on-disk framed bytes). Database-wide write amplification is `(uwal_bytes_written + wal_bytes_written + flush_bytes_written + compaction_bytes_written) / user_bytes_written`.

```
db_stats = db.get_db_stats()
if db_stats.user_bytes_written > 0:
    wa = (
        db_stats.uwal_bytes_written
        + db_stats.wal_bytes_written
        + db_stats.flush_bytes_written
        + db_stats.compaction_bytes_written
    ) / db_stats.user_bytes_written
    print(f"Write amplification: {wa:.2f}x")
```

Stack Allocated

Unlike `get_stats()` (which returns a heap-allocated struct), `get_db_stats()` fills a caller-provided struct. No manual free is needed - the Python binding handles this automatically.

Range Cost Estimation

`range_cost` estimates the computational cost of iterating between two keys in a column family. The returned value is an opaque double - meaningful only for comparison with other values from the same method. It uses only in-memory metadata and performs no disk I/O.

```
cf = db.get_column_family("my_cf")

cost_a = cf.range_cost(b"user:0000", b"user:0999")
cost_b = cf.range_cost(b"user:1000", b"user:1099")

if cost_a < cost_b:
    print("Range A is cheaper to iterate")
```

How it works

The method walks all SSTable levels and uses in-memory metadata to estimate how many blocks and entries fall within the given key range:

- With block indexes enabled · Uses $O(\log B)$ binary search per overlapping SSTable to find the block slots containing each key bound
- Without block indexes · Falls back to byte-level key interpolation within the SSTable's min/max key range
- B+tree SSTables (`use_btree=True`) · Uses key interpolation against tree node counts, plus tree height as a seek cost
- Compression · Compressed SSTables receive a $1.5\times$ weight multiplier to account for decompression overhead
- Merge overhead · Each overlapping SSTable adds a small fixed cost for merge-heap operations
- Memtable · The active memtable's entry count contributes a small in-memory cost

Key order does not matter - `range_cost(a, b)` produces the same result as `range_cost(b, a)`.

Use cases

- Query planning · Compare candidate key ranges to find the cheapest one to scan
- Load balancing · Distribute range scan work across threads by estimating per-range cost
- Adaptive prefetching · Decide how aggressively to prefetch based on range size
- Monitoring · Track how data distribution changes across key ranges over time

Cost Values

The returned cost is not an absolute measure (it does not represent milliseconds, bytes, or entry counts). It is a relative scalar - only meaningful when compared with other `range_cost` results. A cost of 0.0 means no overlapping SSTables or memtable entries were found for the range.

Commit Hook (Change Data Capture)

`set_commit_hook` registers a callback that fires synchronously after every transaction commit on a column family. The hook receives the full batch of committed operations atomically, enabling real-time change data capture without WAL parsing or external log consumers.

```
cf = db.get_column_family("my_cf")

def on_commit(ops: list[tidesdb.CommitOp], commit_seq: int) -> int:
    for op in ops:
        if op.is_delete:
            print(f"[seq={commit_seq}] DELETE key={op.key}")
        else:
            print(f"[seq={commit_seq}] PUT key={op.key} value={op.value}
ttl={op.ttl}")
    return 0 # 0 = success

cf.set_commit_hook(on_commit)

# Normal writes now trigger the hook automatically
with db.begin_txn() as txn:
    txn.put(cf, b"user:1000", b"Alice")
    txn.put(cf, b"user:1001", b"Bob")
    txn.commit() # on_commit fires here with both ops

# Disable the hook
cf.clear_commit_hook()
```

CommitOp fields

Field	Type	Description
<code>key</code>	<code>bytes</code>	Key data
<code>value</code>	<code>bytes None</code>	Value data (<code>None</code> for deletes)
<code>ttl</code>	<code>int</code>	Time-to-live as Unix timestamp (-1 = no expiry)
<code>is_delete</code>	<code>bool</code>	<code>True</code> if this is a delete operation

Callback signature

```
def callback(ops: list[tidesdb.CommitOp], commit_seq: int) -> int:
    ...
```

Return `0` on success. A non-zero return is logged as a warning but does **not** roll back the commit - the data is already durable before the callback runs.

Behavior

- The hook fires after WAL write, memtable apply, and commit status marking are complete
- Each column family has its own independent hook; a multi-CF transaction fires the hook once per CF with only that CF's operations
- `commit_seq` is monotonically increasing across commits and can be used as a replication cursor
- The hook executes synchronously on the committing thread - keep the callback fast to avoid stalling writers
- Python exceptions in the callback are caught internally and treated as non-zero return (logged, commit unaffected)
- Calling `clear_commit_hook()` disables it immediately with no restart required

Use cases

- Replication · Ship committed batches to replicas in commit order
- Event streaming · Publish mutations to Kafka, NATS, or any message broker
- Secondary indexing · Maintain a reverse index or materialized view
- Audit logging · Record every mutation with key, value, TTL, and sequence number
- Debugging · Attach a temporary hook in production to inspect live writes

Runtime-Only

Commit hooks are not persisted to `config.ini`. After a database restart, hooks must be re-registered by the application. This is by design - function pointers cannot be serialized.

Backup

```
# Create an on-disk snapshot without blocking reads/writes
db.backup("./mydb_backup")

# The backup directory must be non-existent or empty
# Backup can be opened as a normal database
with tidesdb.TidesDB.open("./mydb_backup") as backup_db:
    # ... read from backup
    pass
```

Checkpoint

Create a lightweight, near-instant snapshot of the database using hard links instead of copying SSTable data.

```
# Create a checkpoint (near-instant, uses hard links)
db.checkpoint("./mydb_checkpoint")

# The checkpoint directory must be non-existent or empty
# Checkpoint can be opened as a normal database
with tidesdb.TidesDB.open("./mydb_checkpoint") as checkpoint_db:
    # ... read from checkpoint
    pass
```

Checkpoint vs Backup

	<code>backup()</code>	<code>checkpoint()</code>
Speed	Copies every SSTable byte-by-byte	Near-instant (hard links, O(1) per file)
Disk usage	Full independent copy	No extra disk until compaction removes old SSTables
Portability	Can be moved to another filesystem or machine	Same filesystem only (hard link requirement)
Use case	Archival, disaster recovery, remote shipping	Fast local snapshots, point-in-time reads, streaming backups

Behavior

- Requires `checkpoint_dir` to be a non-existent directory or an empty directory

- For each column family, flushes the active memtable, halts compactions, hard links all SSTable files, copies small metadata files, then resumes compactions
- Falls back to file copy if hard linking fails (e.g., cross-filesystem)
- Database stays open and usable during checkpoint

Note

Hard-linked files share storage with the live database. Deleting the original database does not affect the checkpoint (hard link semantics). The checkpoint can be opened as a normal TidesDB database.

Updating Runtime Configuration

```
cf = db.get_column_family("my_cf")

# Update runtime-safe configuration settings
new_config = tidesdb.default_column_family_config()
new_config.write_buffer_size = 256 * 1024 * 1024 # 256MB
new_config.bloom_fpr = 0.001 # 0.1% false positive rate
new_config.sync_mode = tidesdb.SyncMode.SYNC_FULL

# persist_to_disk=True saves to config.ini (default)
cf.update_runtime_config(new_config, persist_to_disk=True)

# Updatable settings (safe to change at runtime):
# -- write_buffer_size      -- Memtable flush threshold
# -- skip_list_max_level   -- Skip list level for new memtables
# -- skip_list_probability -- Skip list probability for new memtables
# -- bloom_fpr             -- False positive rate for new SSTables
# -- index_sample_ratio    -- Index sampling ratio for new SSTables
# -- sync_mode             -- Durability mode
# -- sync_interval_us      -- Sync interval in microseconds

# Save config to custom INI file
tidesdb.save_config_to_ini("custom_config.ini", "my_cf", new_config)

# Load config from INI file
loaded_config = tidesdb.load_config_from_ini("custom_config.ini", "my_cf")
db.create_column_family("restored_cf", loaded_config)
```


B+tree KLog Format (Optional)

Column families can optionally use a B+tree structure for the key log instead of the default block-based format. The B+tree klog format offers faster point lookups through $O(\log N)$ tree traversal.

```
config = tidesdb.default_column_family_config()
config.use_btree = True # Enable B+tree klog format

db.create_column_family("btree_cf", config)
```

Characteristics

- Point lookups · $O(\log N)$ tree traversal with binary search at each node
- Range scans · Doubly-linked leaf nodes enable efficient bidirectional iteration
- Immutable · Tree is bulk-loaded from sorted memtable data during flush
- Compression · Nodes compress independently using the same algorithms

When to use B+tree klog format

- Read-heavy workloads with frequent point lookups
- Workloads where read latency is more important than write throughput
- Large SSTables where block scanning becomes expensive

Tradeoffs

- Slightly higher write amplification during flush
- Larger metadata overhead per node
- Block-based format may be faster for sequential scans

Note

`use_btree` **cannot be changed** after column family creation. Different column families can use different formats.

Compression Algorithms

```
config = tidesdb.default_column_family_config()

# Available algorithms:
config.compression_algorithm = tidesdb.CompressionAlgorithm.NO_COMPRESSION
config.compression_algorithm =
tidesdb.CompressionAlgorithm.SNAPPY_COMPRESSION
config.compression_algorithm = tidesdb.CompressionAlgorithm.LZ4_COMPRESSION
config.compression_algorithm =
tidesdb.CompressionAlgorithm.LZ4_FAST_COMPRESSION
config.compression_algorithm =
tidesdb.CompressionAlgorithm.ZSTD_COMPRESSION
```

Each backend except `NO_COMPRESSION` is an optional build-time feature. Use `compression_available()` to check whether a given algorithm is compiled into the linked library **before** assigning it, rather than discovering an unsupported codec at flush time:

```
import tidesdb

# NO_COMPRESSION is always available; the rest depend on how the library
was built
algo = tidesdb.CompressionAlgorithm.ZSTD_COMPRESSION
if tidesdb.compression_available(algo):
    config.compression_algorithm = algo
else:
    config.compression_algorithm =
tidesdb.CompressionAlgorithm.LZ4_COMPRESSION

# Report what this build supports
for a in tidesdb.CompressionAlgorithm:
    print(a.name, tidesdb.compression_available(a))
```

Sync Modes

```
config = tidesdb.default_column_family_config()

# SYNC_NONE: Fastest, least durable (OS handles flushing)
config.sync_mode = tidesdb.SyncMode.SYNC_NONE

# SYNC_INTERVAL: Balanced (periodic background syncing)
config.sync_mode = tidesdb.SyncMode.SYNC_INTERVAL
config.sync_interval_us = 128000 # 128ms

# SYNC_FULL: Most durable (fsync on every write)
config.sync_mode = tidesdb.SyncMode.SYNC_FULL
```

Log Levels

```
import tidesdb

# Available log levels
config = tidesdb.Config(
    db_path="./mydb",
    log_level=tidesdb.LogLevel.LOG_DEBUG, # Detailed diagnostic info
    # log_level=tidesdb.LogLevel.LOG_INFO,   General info (default)
    # log_level=tidesdb.LogLevel.LOG_WARN,   Warnings only
    # log_level=tidesdb.LogLevel.LOG_ERROR,  Errors only
    # log_level=tidesdb.LogLevel.LOG_FATAL,  Critical errors only
    # log_level=tidesdb.LogLevel.LOG_NONE,   Disable logging
    log_to_file=True, # Write to ./mydb/LOG instead
    of stderr
    log_truncation_at=24 * 1024 * 1024, # Truncate log file at 24MB (0
    = no truncation)
)
```

Column Family Configuration Reference

All available configuration options for column families:

```

config = tidesdb.default_column_family_config()

# Memory and LSM structure
config.write_buffer_size = 64 * 1024 * 1024    # Memtable flush threshold
(default: 64MB)
config.level_size_ratio = 10                    # Level size multiplier
(default: 10)
config.min_levels = 5                           # Minimum LSM levels
(default: 5)
config.dividing_level_offset = 2                 # Compaction dividing level
offset (default: 2)

# Skip list settings
config.skip_list_max_level = 12                  # Skip list max level
(default: 12)
config.skip_list_probability = 0.25              # Skip list probability
(default: 0.25)

# Compression
config.compression_algorithm = tidesdb.CompressionAlgorithm.LZ4_COMPRESSION

# Bloom filter
config.enable_bloom_filter = True                # Enable bloom filters
(default: True)
config.bloom_fpr = 0.01                         # 1% false positive rate
(default: 0.01)

# Block indexes
config.enable_block_indexes = True               # Enable block indexes
(default: True)
config.index_sample_ratio = 1                    # Sample every block
(default: 1)
config.block_index_prefix_len = 16               # Block index prefix length
(default: 16)

# Durability
config.sync_mode = tidesdb.SyncMode.SYNC_INTERVAL
config.sync_interval_us = 128000                 # Sync interval in
microseconds (default: 128ms)

# Key ordering
config.comparator_name = "memcmp"                # Comparator name (default:
"memcmp")

```

```

# Value separation
config.klog_value_threshold = 512          # Values > threshold go to
vlog (default: 512)

# Resource limits
config.min_disk_space = 100 * 1024 * 1024  # Minimum disk space required
(default: 100MB)

# Transaction isolation
config.default_isolation_level = tidesdb.IsolationLevel.READ_COMMITTED

# Compaction triggers
config.l1_file_count_trigger = 4           # L1 file count trigger
(default: 4)
config.l0_queue_stall_threshold = 20       # L0 queue stall threshold
(default: 20)

# Tombstone-driven compaction escalation
config.tombstone_density_trigger = 0.0     # Per-SSTable tombstone
density that escalates compaction priority (default: 0.0 = disabled, range
[0.0, 1.0])
config.tombstone_density_min_entries = 1024 # SSTables with fewer entries
are ignored by the density trigger (default: 1024)

# B+tree klog format (optional)
config.use_btree = False                   # Use B+tree klog format
(default: False)

# Object store settings (for object store mode)
config.object_lazy_compaction = False      # Enable lazy compaction for
object store
config.object_prefetch_compaction = True   # Prefetch data during
compaction (default: True)

```

Custom Comparators

TidesDB uses comparators to determine the sort order of keys. Built-in comparators are automatically registered:

- `"memcmp"` (default) · Binary byte-by-byte comparison
- `"lexicographic"` · Null-terminated string comparison
- `"uint64"` · Unsigned 64-bit integer comparison
- `"int64"` · Signed 64-bit integer comparison

- `"reverse"` · Reverse binary comparison
- `"case_insensitive"` · Case-insensitive ASCII comparison

```
# Check if a comparator is registered
if db.get_comparator("memcmp"):
    print("memcmp comparator is registered")

# Use a built-in comparator
config = tidesdb.default_column_family_config()
config.comparator_name = "reverse" # Descending order
db.create_column_family("reverse_cf", config)

# Register a custom comparator
def timestamp_desc_compare(key1: bytes, key2: bytes) -> int:
    """Compare 8-byte timestamps in descending order."""
    import struct
    if len(key1) != 8 or len(key2) != 8:
        # Fallback to memcmp for invalid sizes
        if key1 < key2:
            return -1
        elif key1 > key2:
            return 1
        return 0

    ts1 = struct.unpack("<Q", key1)[0]
    ts2 = struct.unpack("<Q", key2)[0]

    # Reverse order for newest-first
    if ts1 > ts2:
        return -1
    elif ts1 < ts2:
        return 1
    return 0

# Register before creating column families that use it
db.register_comparator("timestamp_desc", timestamp_desc_compare)

# Use the custom comparator
config = tidesdb.default_column_family_config()
config.comparator_name = "timestamp_desc"
db.create_column_family("events", config)
```

Note

Comparators must be registered **before** creating column families that use them. Once set, a comparator **cannot be changed** for a column family.

Unified Memtable Mode

TidesDB supports a unified memtable mode where all column families share a single memtable and WAL. This can reduce write amplification and improve throughput for workloads that write to many column families simultaneously.

```
# Open with unified memtable enabled
db = tidesdb.TidesDB.open(
    "./mydb",
    unified_memtable=True,
    unified_memtable_write_buffer_size=32 * 1024 * 1024, # 32MB
    unified_memtable_skip_list_max_level=12,
    unified_memtable_skip_list_probability=0.25,
    unified_memtable_sync_mode=tidesdb.SyncMode.SYNC_INTERVAL,
    unified_memtable_sync_interval_us=128000, # 128ms
)

# Or via Config object
config = tidesdb.Config(
    db_path="./mydb",
    unified_memtable=True,
    unified_memtable_write_buffer_size=32 * 1024 * 1024,
)
db = tidesdb.TidesDB(config)

# Check if unified memtable is active
stats = db.get_db_stats()
if stats.unified_memtable_enabled:
    print(f"Unified memtable: {stats.unified_memtable_bytes} bytes")
```

Configuration options

Field	Type	Default	Description
<code>unified_memtable</code>	<code>bool</code>	<code>False</code>	Enable unified memtable mode
<code>unified_memtable_write_buffer_size</code>	<code>int</code>	<code>0</code>	Write buffer size (0 = default)
<code>unified_memtable_skip_list_max_level</code>	<code>int</code>	<code>0</code>	Skip list max level (0 = default)
<code>unified_memtable_skip_list_probability</code>	<code>float</code>	<code>0.0</code>	Skip list probability (0.0 = default)
<code>unified_memtable_sync_mode</code>	<code>SyncMode</code>	<code>SYNC_NON</code> <code>E</code>	WAL sync mode
<code>unified_memtable_sync_interval_us</code>	<code>int</code>	<code>0</code>	Sync interval in microseconds

Object Store Mode

Object store mode allows TidesDB to store SSTables in a remote object store (S3, MinIO, GCS, or any S3-compatible service) while using local disk as a cache. This separates compute from storage and enables cold start recovery from the remote store. Object store mode requires unified memtable mode and is automatically enforced when a connector is set.

Enabling Object Store Mode (Filesystem Connector)

```
import tidesdb

# Create a filesystem connector (for testing and local replication)
store = tidesdb.objstore_fs_create("/mnt/nfs/tidesdb-objects")

# Get default object store config, then customize
os_cfg = tidesdb.objstore_default_config()
os_cfg.local_cache_max_bytes = 512 * 1024 * 1024 # 512MB local cache
os_cfg.max_concurrent_uploads = 8

# Open database with object store
db = tidesdb.TidesDB.open(
    "./mydb",
    object_store=store,
    object_store_config=os_cfg,
)

# Use the database normally -- SSTables are uploaded after flush
db.close()
```

Using Config Object

```
import tidesdb

store = tidesdb.objstore_fs_create("/mnt/nfs/tidesdb-objects")

os_cfg = tidesdb.ObjStoreConfig(
    local_cache_max_bytes=512 * 1024 * 1024,
    max_concurrent_uploads=8,
    max_concurrent_downloads=16,
)

config = tidesdb.Config(
    db_path="./mydb",
    object_store=store,
    object_store_config=os_cfg,
)

db = tidesdb.TidesDB(config)
```

S3-Compatible Connector

For production use, connect to AWS S3, MinIO, or any S3-compatible service. `objstore_s3_create()` covers the common case; `objstore_s3_create_config()` additionally exposes TLS and multipart tuning.

```

import tidesdb

# Common case (AWS S3)
store = tidesdb.objstore_s3_create(
    endpoint="s3.amazonaws.com",
    bucket="my-tidesdb-bucket",
    access_key="AKIA...",
    secret_key="...",
    prefix="production/db1/", # optional key prefix
    region="us-east-1",
    use_ssl=True,
    use_path_style=False,      # virtual-hosted style for AWS
)

# MinIO (path-style URLs, no region)
store = tidesdb.objstore_s3_create(
    endpoint="minio.local:9000",
    bucket="testbucket",
    access_key="minioadmin",
    secret_key="minioadmin",
    use_path_style=True,
)

# Full configuration with TLS + multipart tuning
store = tidesdb.objstore_s3_create_config(
    endpoint="s3.amazonaws.com",
    bucket="my-tidesdb-bucket",
    access_key="AKIA...",
    secret_key="...",
    region="us-east-1",
    tls_ca_path="/etc/ssl/certs/custom-ca.pem", # None = system bundle
    tls_insecure_skip_verify=False,             # True disables TLS verify
    (test only, insecure)
    multipart_threshold=64 * 1024 * 1024,      # 0 = library default
    multipart_part_size=8 * 1024 * 1024,        # 0 = library default
)

db = tidesdb.TidesDB.open("./mydb", object_store=store)

```

S3 is a build-time feature

The S3 connector is only available when the TidesDB library is compiled with `-DTIDESDB_WITH_S3=ON`. On a build without it, `objstore_s3_create()` / `objstore_s3_create_config()` raise a `TidesDBError` explaining that S3 support is missing — they never crash the import. The filesystem connector (`objstore_fs_create()`) is always available.

Object Store Configuration

Use `objstore_default_config()` for sensible defaults, then override fields as needed.

Field	Type	Default	Description
<code>local_cache_path</code>	<code>str None</code>	<code>None</code> (uses <code>db_path</code>)	Local directory for cached SSTable files
<code>local_cache_max_bytes</code>	<code>int</code>	<code>0</code> (unlimited)	Maximum local cache size in bytes
<code>cache_on_read</code>	<code>bool</code>	<code>True</code>	Cache downloaded files locally
<code>cache_on_write</code>	<code>bool</code>	<code>True</code>	Keep local copy after upload
<code>max_concurrent_uploads</code>	<code>int</code>	<code>4</code>	Number of parallel upload threads
<code>max_concurrent_downloads</code>	<code>int</code>	<code>8</code>	Number of parallel download threads
<code>multipart_threshold</code>	<code>int</code>	<code>67108864</code> (64MB)	Use multipart upload above this size
<code>multipart_part_size</code>	<code>int</code>	<code>8388608</code> (8MB)	Chunk size for multipart uploads
<code>sync_manifest_to_object</code>	<code>bool</code>	<code>True</code>	Upload MANIFEST after each compaction
<code>replicate_wal</code>	<code>bool</code>	<code>True</code>	Upload closed WAL segments for replication
<code>wal_upload_sync</code>	<code>bool</code>	<code>False</code>	<code>False</code> for background WAL upload, <code>True</code> to block flush
<code>wal_sync_threshold_bytes</code>	<code>int</code>	<code>1048576</code> (1MB)	Sync active WAL to object store when it grows by this many bytes (0 to disable)
<code>wal_sync_on_commit</code>	<code>bool</code>	<code>False</code>	Upload WAL after every txn commit for RPO=0 replication
<code>replica_mode</code>	<code>bool</code>	<code>False</code>	Enable read-only replica mode (writes raise <code>TidesDBError</code>)
<code>replica_sync_interval_us</code>	<code>int</code>	<code>5000000</code> (5s)	MANIFEST poll interval for replica sync in microseconds
<code>replica_replay_wal</code>	<code>bool</code>	<code>True</code>	Replay WAL from object store for near-real-time reads on replicas

Per-CF Object Store Tuning

Column family configurations include three object store tuning fields.

Field	Type	Default	Description
<code>object_lazy_compaction</code>	<code>bool</code>	<code>False</code>	Compact less aggressively for remote storage
<code>object_prefetch_compaction</code>	<code>bool</code>	<code>True</code>	Download all inputs before compaction merge

Replica Mode

Replica mode enables read-only nodes that follow a primary through the object store.

```
import tidesdb

store = tidesdb.objstore_fs_create("/mnt/shared/tidesdb-objects")

os_cfg = tidesdb.ObjStoreConfig(
    replica_mode=True,
    replica_sync_interval_us=1000000, # 1 second sync interval
    replica_replay_wal=True,
)

db = tidesdb.TidesDB.open(
    "./mydb_replica",
    object_store=store,
    object_store_config=os_cfg,
)

# Reads work normally
cf = db.get_column_family("my_cf")
with db.begin_txn() as txn:
    value = txn.get(cf, b"key")
    txn.commit()

# Writes raise TidesDBError with code TDB_ERR_READONLY
```

Sync-on-Commit WAL (Primary Side)

For tighter replication lag, enable sync-on-commit on the primary so every committed write is uploaded to the object store immediately.

```
os_cfg = tidesdb.ObjStoreConfig(
    wal_sync_on_commit=True, # RPO = 0, every commit is durable in object
    store
)
```

Cold Start Recovery

When the local database directory is empty but a connector is configured, TidesDB automatically discovers column families from the object store during recovery. It downloads MANIFEST and config files in parallel, reconstructs the SSTable inventory, and fetches SSTable data on demand as queries arrive.

```
import shutil
shutil.rmtree("./mydb", ignore_errors=True)

# Reopen with the same connector -- cold start recovery
db = tidesdb.TidesDB.open(
    "./mydb",
    object_store=store,
    object_store_config=os_cfg,
)

# All data is available -- SSTables are fetched from the object store on
demand
cf = db.get_column_family("my_cf")
```

Promote to Primary

When a database is opened with an object store in replica mode, it is read-only. Use `promote_to_primary()` to switch to primary mode and enable writes.

```
# Promote a replica to primary
db.promote_to_primary()

# Now writes are allowed
with db.begin_txn() as txn:
    txn.put(cf, b"key", b"value")
    txn.commit()
```

Error Handling

```
try:
    value = txn.get(cf, b"nonexistent_key")
except tidesdb.TidesDBError as e:
    print(f"Error: {e}")
    print(f"Error code: {e.code}")
```

Every error code is exported as a module-level constant, so failures can be handled by comparing `e.code`:

```
try:
    txn.commit()
except tidesdb.TidesDBError as e:
    if e.code == tidesdb.TDB_ERR_CONFLICT:
        # retry the transaction
        ...
    elif e.code == tidesdb.TDB_ERR_BUSY:
        # resource temporarily busy, back off and retry
        ...
    else:
        raise
```


Constant	Value	Meaning
TDB_SUCCESS	0	Operation succeeded
TDB_ERR_MEMORY	-1	Memory allocation failed
TDB_ERR_INVALID_ARGS	-2	Invalid arguments
TDB_ERR_NOT_FOUND	-3	Key or resource not found
TDB_ERR_IO	-4	I/O error
TDB_ERR_CORRUPTION	-5	Data corruption detected
TDB_ERR_EXISTS	-6	Resource already exists
TDB_ERR_CONFLICT	-7	Transaction conflict
TDB_ERR_TOO_LARGE	-8	Key or value too large
TDB_ERR_MEMORY_LIMIT	-9	Memory limit exceeded
TDB_ERR_INVALID_DB	-10	Invalid database handle
TDB_ERR_UNKNOWN	-11	Unknown error
TDB_ERR_LOCKED	-12	Database or resource is locked
TDB_ERR_READONLY	-13	Database is read-only (e.g. replica mode)
TDB_ERR_BUSY	-14	Resource is busy
TDB_ERR_PRECONDITION	-15	A precondition was not met

TidesDB Java API Reference

Getting Started

Prerequisites

You **must** have the TidesDB shared C library installed on your system. You can find the installation instructions [here](#).

Requirements

- Java 11 or higher
- Maven 3.6+
- TidesDB native library installed on the system

Building the JNI Library



```
cd src/main/c
cmake -S . -B build
cmake --build build
sudo cmake --install build
```

Adding to Your Project

Maven

```
<dependency>
  <groupId>com.tidesdb</groupId>
  <artifactId>tidesdb-java</artifactId>
  <version>0.8.2</version>
</dependency>
```

Usage

Opening and Closing a Database

```
import com.tidesdb.*;

public class Example {
    public static void main(String[] args) throws TidesDBException {
        Config config = Config.builder("./mydb")
            .numFlushThreads(2)
            .numCompactionThreads(2)
            .logLevel(LogLevel.INFO)
            .blockCacheSize(64 * 1024 * 1024)
            .maxOpenSSTables(256)
            .maxMemoryUsage(0)
            .maxConcurrentFlushes(0)           // Cap on in-flight
memtable flushes across all CFs (0 = library default)
            .unifiedMemtable(false)
            .build();

        try (TidesDB db = TidesDB.open(config)) {
            System.out.println("Database opened successfully");
        }
    }
}
```

`Config.defaultConfig()` returns a configuration with sensible defaults. The `maxConcurrentFlushes` default is sourced from the underlying C library via `tidesdb_default_config()`, so the binding tracks the engine's defaults automatically. If you previously hardcoded values such as `maxConcurrentFlushes`, re-test after upgrading.

maxConcurrentFlushes

`maxConcurrentFlushes` is a global semaphore on the number of in-flight memtable flushes across all column families. It bounds total transient memory and work-queue depth when many column families flush at once. Leave it at `0` to inherit the library default (recommended), or lower it on memory-constrained hosts to throttle peak flush concurrency.

Raising the Open-File Limit

`TidesDB.raiseOpenFileLimit(long)` raises this process's open-file ceiling so the database can keep more SSTables open. The engine sizes `maxOpenSSTables` to fit this ceiling at open time, so call it **before** `TidesDB.open()`. This is an explicit, opt-in operator action - TidesDB never raises the limit itself.

```
// Raise the soft limit toward the hard limit before opening
long ceiling = TidesDB.raiseOpenFileLimit(65536);
System.out.println("Open-file ceiling now: " + ceiling);

try (TidesDB db = TidesDB.open(config)) {
    // engine can now keep more SSTables open
}
```

Behavior

- On POSIX systems (Linux, macOS, the BSDs, illumos) it raises the `RLIMIT_NOFILE` soft limit toward the hard limit; on Windows it raises the CRT stdio cap (max 8192)
- A failed or partial raise is non-fatal
- Passing a value `<= 0` just reports the current ceiling without changing it
- Returns the open-file ceiling in effect after the attempt

Unified Memtable Mode

Enable unified memtable mode to share a single memtable and WAL across all column families. This reduces write amplification for workloads with many small column families.

```

Config config = Config.builder("./mydb")
    .numFlushThreads(2)
    .numCompactionThreads(2)
    .logLevel(LogLevel.INFO)
    .blockCacheSize(64 * 1024 * 1024)
    .maxOpenSSTables(256)
    .unifiedMemtable(true)
    .unifiedMemtableWriteBufferSize(0)           // 0 = auto (64MB default)
    .unifiedMemtableSkipListMaxLevel(0)         // 0 = default (12)
    .unifiedMemtableSkipListProbability(0)       // 0 = default (0.25)
    .unifiedMemtableSyncMode(0)                  // 0 = SYNC_NONE
    .unifiedMemtableSyncIntervalUs(0)            // 0 = default
    .build();

try (TidesDB db = TidesDB.open(config)) {
    // All column families share a single memtable
}

```

Creating and Dropping Column Families

Column families are isolated key-value stores with independent configuration.

```

// Create with default configuration
ColumnFamilyConfig cfConfig = ColumnFamilyConfig.defaultConfig();
db.createColumnFamily("my_cf", cfConfig);

// Create with custom configuration
ColumnFamilyConfig customConfig = ColumnFamilyConfig.builder()
    .writeBufferSize(128 * 1024 * 1024)
    .levelSizeRatio(10)
    .minLevels(5)
    .compressionAlgorithm(CompressionAlgorithm.LZ4_COMPRESSION)
    .enableBloomFilter(true)
    .bloomFPR(0.01)
    .enableBlockIndexes(true)
    .syncMode(SyncMode.SYNC_INTERVAL)
    .syncIntervalUs(128000)
    .defaultIsolationLevel(IsolationLevel.READ_COMMITTED)
    .tombstoneDensityTrigger(0.0) // Per-SSTable tombstone
density above which compaction escalates (0.0 = disabled, range 0.0 to 1.0)
    .tombstoneDensityMinEntries(1024) // Minimum entry count for
an SSTable to be considered by the density trigger
    .build();

db.createColumnFamily("custom_cf", customConfig);

db.dropColumnFamily("my_cf");

// Delete via column family handle (alternative to dropColumnFamily)
ColumnFamily cfToDelete = db.getColumnFamily("old_cf");
db.deleteColumnFamily(cfToDelete);

String[] cfNames = db.listColumnFamilies();
for (String name : cfNames) {
    System.out.println("Column family: " + name);
}

```

Working with Transactions

Writing Data

```
ColumnFamily cf = db.getColumnFamily("my_cf");

try (Transaction txn = db.beginTransaction()) {
    txn.put(cf, "key".getBytes(), "value".getBytes());
    txn.commit();
}
```

Writing with TTL

```
import java.time.Instant;

ColumnFamily cf = db.getColumnFamily("my_cf");

try (Transaction txn = db.beginTransaction()) {
    long ttl = Instant.now().getEpochSecond() + 10;

    txn.put(cf, "temp_key".getBytes(), "temp_value".getBytes(), ttl);
    txn.commit();
}
```

TTL Examples

```
long ttl = -1;

long ttl = Instant.now().getEpochSecond() + 5 * 60;

long ttl = Instant.now().getEpochSecond() + 60 * 60;

long ttl = LocalDateTime.of(2026, 12, 31, 23, 59, 59)
    .toEpochSecond(ZoneOffset.UTC);
```

Reading Data

```
ColumnFamily cf = db.getColumnFamily("my_cf");

try (Transaction txn = db.beginTransaction()) {
    byte[] value = txn.get(cf, "key".getBytes());
    System.out.println("Value: " + new String(value));
}
```

Deleting Data

```
ColumnFamily cf = db.getColumnFamily("my_cf");

try (Transaction txn = db.beginTransaction()) {
    txn.delete(cf, "key".getBytes());
    txn.commit();
}
```

Single-Delete

`txn.singleDelete` writes a tombstone with the same read semantics as `txn.delete`, but carries a caller-provided promise that lets compaction drop the put and the tombstone together as soon as both appear in the same merge input, rather than carrying the tombstone forward until it reaches the largest active level.

Between any two single-deletes on the same key, and between the start of the key's history and its first single-delete, the key has been put **at most once**. The engine does not and cannot verify this at runtime; violating the contract can leave older puts visible after the single-delete and is a bug in the caller.

This is the right choice for workloads that insert each key exactly once and then delete it exactly once (classic insert-benchmark patterns, secondary-index entries on columns that are never updated, log-style tables with scheduled purges). It is **not** safe for tables that issue repeated updates to the same key.


```
ColumnFamily cf = db.getColumnFamily("my_cf");

try (Transaction txn = db.beginTransaction()) {
    txn.singleDelete(cf, "key".getBytes());
    txn.commit();
}
```

When in doubt, prefer `txn.delete`.

Transaction Rollback

```
ColumnFamily cf = db.getColumnFamily("my_cf");

try (Transaction txn = db.beginTransaction()) {
    txn.put(cf, "key".getBytes(), "value".getBytes());

    txn.rollback();
}
```

Multi-Operation Transactions

```
ColumnFamily cf = db.getColumnFamily("my_cf");

try (Transaction txn = db.beginTransaction()) {
    txn.put(cf, "key1".getBytes(), "value1".getBytes());
    txn.put(cf, "key2".getBytes(), "value2".getBytes());
    txn.delete(cf, "old_key".getBytes());

    txn.commit();
} catch (TidesDBException e) {
    throw e;
}
```

Iterating Over Data

Iterators provide efficient bidirectional traversal over key-value pairs.

Forward Iteration

```
ColumnFamily cf = db.getColumnFamily("my_cf");

try (Transaction txn = db.beginTransaction()) {
    try (TidesDBIterator iter = txn.newIterator(cf)) {
        iter.seekToFirst();

        while (iter.isValid()) {
            byte[] key = iter.key();
            byte[] value = iter.value();

            System.out.printf("Key: %s, Value: %s\n",
                new String(key), new String(value));

            iter.next();
        }
    }
}
```

Backward Iteration

```
ColumnFamily cf = db.getColumnFamily("my_cf");

try (Transaction txn = db.beginTransaction()) {
    try (TidesDBIterator iter = txn.newIterator(cf)) {
        iter.seekToLast();

        while (iter.isValid()) {
            byte[] key = iter.key();
            byte[] value = iter.value();

            System.out.printf("Key: %s, Value: %s\n",
                new String(key), new String(value));

            iter.prev();
        }
    }
}
```

Combined Key-Value Retrieval

For better performance when you need both the key and value, use `keyValue()` which retrieves both in a single JNI call:

```
try (TidesDBIterator iter = txn.newIterator(cf)) {
    iter.seekToFirst();

    while (iter.isValid()) {
        KeyValue kv = iter.keyValue();
        System.out.printf("Key: %s, Value: %s%n",
            new String(kv.getKey()), new String(kv.getValue()));

        iter.next();
    }
}
```

Seeking to a Specific Key

```
try (TidesDBIterator iter = txn.newIterator(cf)) {
    iter.seek("prefix".getBytes());

    iter.seekForPrev("prefix".getBytes());
}
```

Prefix Seeking

Since `seek` positions the iterator at the first key $\geq$ target, you can use a prefix as the seek target to efficiently scan all keys sharing that prefix:

```

ColumnFamily cf = db.getColumnFamily("my_cf");

try (Transaction txn = db.beginTransaction()) {
    try (TidesDBIterator iter = txn.newIterator(cf)) {
        byte[] prefix = "user:".getBytes();
        iter.seek(prefix);

        while (iter.isValid()) {
            byte[] key = iter.key();
            String keyStr = new String(key);

            if (!keyStr.startsWith("user:")) break;

            byte[] value = iter.value();
            System.out.printf("Key: %s, Value: %s\n", keyStr, new
String(value));

            iter.next();
        }
    }
}

```

This pattern works across both memtables and SSTables. When block indexes are enabled, the seek operation uses binary search to jump directly to the relevant block, making prefix scans efficient even on large datasets.

Getting Column Family Statistics

```
ColumnFamily cf = db.getColumnFamily("my_cf");
Stats stats = cf.getStats();

System.out.println("Number of levels: " + stats.getNumLevels());
System.out.println("Memtable size: " + stats.getMemtableSize());
System.out.println("Total keys: " + stats.getTotalKeys());
System.out.println("Total data size: " + stats.getTotalDataSize());
System.out.println("Average key size: " + stats.getAvgKeySize());
System.out.println("Average value size: " + stats.getAvgValueSize());
System.out.println("Read amplification: " + stats.getReadAmp());
System.out.println("Hit rate: " + stats.getHitRate());

if (stats.isUseBtree()) {
    System.out.println("B+tree total nodes: " +
stats.getBtreeTotalNodes());
    System.out.println("B+tree max height: " + stats.getBtreeMaxHeight());
    System.out.println("B+tree avg height: " + stats.getBtreeAvgHeight());
}

long[] levelSizes = stats.getLevelSizes();
int[] levelSSTables = stats.getLevelNumSSTables();
long[] levelKeyCounts = stats.getLevelKeyCounts();

// Tombstone density observability
System.out.println("Total tombstones: " + stats.getTotalTombstones());
System.out.printf("Tombstone ratio: %.2f%%\n", stats.getTombstoneRatio() *
100.0);
System.out.printf("Worst SSTable density: %.2f%% at level %d\n",
    stats.getMaxSstDensity() * 100.0, stats.getMaxSstDensityLevel());
long[] levelTombstoneCounts = stats.getLevelTombstoneCounts();
for (int i = 0; i < stats.getNumLevels(); i++) {
    System.out.printf("Level %d tombstones: %d\n", i + 1,
levelTombstoneCounts[i]);
}
```

Stats Fields

Field	Type	Description
<code>numLevels</code>	int	Number of LSM levels
<code>memtableSize</code>	long	Current memtable size in bytes
<code>levelSizes</code>	long[]	Size of each level in bytes
<code>levelNumSSTables</code>	int[]	Number of SSTables at each level
<code>levelKeyCounts</code>	long[]	Number of keys per level
<code>totalKeys</code>	long	Total keys across memtable and all SSTables
<code>totalDataSize</code>	long	Total data size (klog + vlog) in bytes
<code>avgKeySize</code>	double	Average key size in bytes
<code>avgValueSize</code>	double	Average value size in bytes
<code>readAmp</code>	double	Read amplification (point lookup cost multiplier)
<code>hitRate</code>	double	Cache hit rate (0.0 if cache disabled)
<code>useBtree</code>	boolean	Whether column family uses B+tree klog format
<code>btreeTotalNodes</code>	long	Total B+tree nodes (only if useBtree=true)
<code>btreeMaxHeight</code>	int	Maximum B+tree height (only if useBtree=true)
<code>btreeAvgHeight</code>	double	Average B+tree height (only if useBtree=true)
<code>totalTombstones</code>	long	Total tombstones across every SSTable in the column family
<code>tombstoneRatio</code>	double	$\text{totalTombstones} / \text{totalKeys}$ (0.0 if <code>totalKeys</code> is 0; range 0.0 to 1.0)
<code>levelTombstoneCounts</code>	long[]	Per-level tombstone counts (parallels <code>levelKeyCounts</code>)
<code>maxSstDensity</code>	double	Worst single-SSTable tombstone density observed (0.0 to 1.0)
<code>maxSstDensityLevel</code>	int	1-based level index where the worst SSTable lives (0 if none)

Field	Type	Description
<code>walBytesWritten</code>	long	Framed bytes appended to this CF's WAL (0 in unified mode; see <code>DbStats.uwalBytesWritten</code>)
<code>flushBytesWritten</code>	long	On-disk bytes this CF's flushes wrote to L0 SSTables
<code>compactionBytesWritten</code>	long	On-disk bytes this CF's compactions wrote
<code>compactionBytesRead</code>	long	On-disk bytes this CF's compactions read as input
<code>userBytesWritten</code>	long	Logical key+value bytes committed to this CF (write-amplification denominator)
<code>flushCount</code>	long	Flushed SSTables produced by this CF
<code>compactionCount</code>	long	Compaction output SSTables produced by this CF
<code>config</code>	ColumnFamilyConfig	Column family configuration

Write amplification

The `*BytesWritten`/`*BytesRead` and `*Count` fields are lifetime counters since the database was opened (on-disk framed bytes). A column family's write amplification is $(\text{walBytesWritten} + \text{flushBytesWritten} + \text{compactionBytesWritten}) / \text{userBytesWritten}$. In unified memtable mode `walBytesWritten` is 0 — the shared WAL volume is reported db-wide via `DbStats.uwalBytesWritten`.

Getting Cache Statistics

```
CacheStats cacheStats = db.getCacheStats();

System.out.println("Cache enabled: " + cacheStats.isEnabled());
System.out.println("Total entries: " + cacheStats.getTotalEntries());
System.out.println("Hit rate: " + cacheStats.getHitRate());
```

Getting Database-Level Statistics

Get aggregate statistics across the entire database instance.

```

DbStats dbStats = db.getDbStats();

System.out.println("Column families: " + dbStats.getNumColumnFamilies());
System.out.println("Total memory: " + dbStats.getTotalMemory() + " bytes");
System.out.println("Resolved memory limit: " +
dbStats.getResolvedMemoryLimit() + " bytes");
System.out.println("Memory pressure level: " +
dbStats.getMemoryPressureLevel());
System.out.println("Global sequence: " + dbStats.getGlobalSeq());
System.out.println("Flush queue: " + dbStats.getFlushQueueSize() + "
pending");
System.out.println("Compaction queue: " + dbStats.getCompactionQueueSize()
+ " pending");
System.out.println("Total SSTables: " + dbStats.getTotalSstableCount());
System.out.println("Total data size: " + dbStats.getTotalDataSizeBytes() +
" bytes");
System.out.println("Open SSTable handles: " +
dbStats.getNumOpenSstables());
System.out.println("In-flight txn memory: " + dbStats.getTxnMemoryBytes() +
" bytes");
System.out.println("Immutable memtables: " +
dbStats.getTotalImmutableCount());
System.out.println("Memtable bytes: " + dbStats.getTotalMemtableBytes());

// Unified memtable stats
System.out.println("Unified memtable enabled: " +
dbStats.isUnifiedMemtableEnabled());
System.out.println("Unified memtable bytes: " +
dbStats.getUnifiedMemtableBytes());
System.out.println("Unified immutable count: " +
dbStats.getUnifiedImmutableCount());
System.out.println("Unified is flushing: " +
dbStats.isUnifiedIsFlushing());
System.out.println("Unified WAL generation: " +
dbStats.getUnifiedWalGeneration());

// Object store stats
System.out.println("Object store enabled: " +
dbStats.isObjectStoreEnabled());
System.out.println("Object store connector: " +
dbStats.getObjectStoreConnector());
System.out.println("Local cache used: " + dbStats.getLocalCacheBytesUsed()
+ " bytes");
System.out.println("Total uploads: " + dbStats.getTotalUploads());

```



```
System.out.println("Replica mode: " + dbStats.isReplicaMode());
```

DbStats Fields

Field	Type	Description
<code>numColumnFamilies</code>	int	Number of column families
<code>totalMemory</code>	long	System total memory
<code>availableMemory</code>	long	System available memory at open time
<code>resolvedMemoryLimit</code>	long	Resolved memory limit (auto or configured)
<code>memoryPressureLevel</code>	int	Current memory pressure (0=normal, 1=elevated, 2=high, 3=critical)
<code>flushPendingCount</code>	int	Number of pending flush operations (queued + in-flight)
<code>totalMemtableBytes</code>	long	Total bytes in active memtables across all CFs
<code>totalImmutableCount</code>	int	Total immutable memtables across all CFs
<code>totalSstableCount</code>	int	Total SSTables across all CFs and levels
<code>totalDataSizeBytes</code>	long	Total data size (klog + vlog) across all CFs
<code>numOpenSstableS</code>	int	Number of currently open SSTable file handles
<code>globalSeq</code>	long	Current global sequence number
<code>txnMemoryBytes</code>	long	Bytes held by in-flight transactions
<code>compactionQueueSize</code>	long	Number of pending compaction tasks
<code>flushQueueSize</code>	long	Number of pending flush tasks in queue
<code>unifiedMemtableEnabled</code>	boolean	Whether unified memtable mode is active
<code>unifiedMemtableBytes</code>	long	Bytes in unified active memtable
<code>unifiedImmutableCount</code>	int	Number of unified immutable memtables

Field	Type	Description
<code>unifiedIsFlushing</code>	boolean	Whether unified memtable is currently flushing
<code>unifiedNextCfIndex</code>	int	Next CF index to be assigned in unified mode
<code>unifiedWalGeneration</code>	long	Current unified WAL generation counter
<code>objectStoreEnabled</code>	boolean	Whether object store mode is active
<code>objectStoreConnector</code>	String	Connector name (“s3”, “gcs”, “fs”, etc.)
<code>localCacheBytesUsed</code>	long	Current local file cache usage in bytes
<code>localCacheBytesMax</code>	long	Configured maximum local cache size in bytes
<code>localCacheNumFiles</code>	int	Number of files tracked in local cache
<code>lastUploadedGeneration</code>	long	Highest WAL generation confirmed uploaded
<code>uploadQueueDepth</code>	long	Number of pending upload jobs in the queue
<code>totalUploads</code>	long	Lifetime count of objects uploaded to object store
<code>totalUploadFailures</code>	long	Lifetime count of permanently failed uploads
<code>replicaMode</code>	boolean	Whether running in read-only replica mode
<code>primaryEpoch</code>	long	Object-store single-writer fencing: lease epoch this primary currently holds (0 when not a primary / no lease)
<code>seenEpoch</code>	long	Object-store single-writer fencing: highest lease epoch this node has observed
<code>uwalBytesWritten</code>	long	Framed bytes appended to the shared unified WAL (0 if unified mode off)
<code>walBytesWritten</code>	long	Per-CF WAL bytes summed across all column families

Field	Type	Description
<code>flushBytesWritten</code>	long	Flush output bytes summed across all column families
<code>compactionBytesWritten</code>	long	Compaction output bytes summed across all column families
<code>compactionBytesRead</code>	long	Compaction input bytes summed across all column families
<code>userBytesWritten</code>	long	Logical committed bytes summed across all column families
<code>flushCount</code>	long	Flushed SSTables summed across all column families
<code>compactionCount</code>	long	Compaction output SSTables summed across all column families

The database-wide write amplification is $(\text{uwalBytesWritten} + \text{walBytesWritten} + \text{flushBytesWritten} + \text{compactionBytesWritten}) / \text{userBytesWritten}$. All fields are lifetime counters (on-disk framed bytes) since the database was opened.

Manual Compaction and Flush

```
ColumnFamily cf = db.getColumnFamily("my_cf");

cf.compact();

cf.flushMemtable();

boolean flushing = cf.isFlushing();
boolean compacting = cf.isCompacting();
```

Targeted Range Compaction

`compactRange` runs a synchronous compaction over a specific key range. Only SSTables whose minimum and maximum keys overlap the requested range participate in the merge, so the work and I/O are bounded to the affected portion of the LSM tree rather than the whole column family.

```
ColumnFamily cf = db.getColumnFamily("my_cf");

byte[] start = "tenant_42:".getBytes(StandardCharsets.UTF_8);
byte[] end   = "tenant_42;".getBytes(StandardCharsets.UTF_8);

cf.compactRange(start, end);
```

When to use

- Bulk reclaim after a large range delete, where waiting for natural compaction would leave tombstones and obsolete versions on disk
- Tenant eviction or sliding-window expiration that does not fit TTL semantics
- Post-import cleanup of a known key range loaded with `put` followed by `delete`
- Operational counterpart to the automatic tombstone density trigger when an operator wants reclaim now rather than at the next natural threshold crossing

Behavior

- Synchronous, blocks the caller until the merge commits or fails
- Does not enqueue work onto the compaction thread pool, the calling thread does the work
- A `null` or empty endpoint means unbounded on that side; both `null`/empty is rejected with `ERR_INVALID_ARGS` (use `compact()` for full CF compaction)
- Selects only SSTables whose key range overlaps the requested range using the column family's comparator, SSTables outside the range are not touched
- Applies the same emit-loop logic as background compactions (tombstone reclamation rules, single-delete pair cancellation, sequence-based deduplication, value recompression)
- Output SSTables are committed to the manifest atomically and old inputs are marked for deletion

Return values

- Returns normally on success
- Throws `TidesDBException` with `ERR_INVALID_ARGS` if both endpoints are null/empty
- Throws `TidesDBException` with `ERR_LOCKED` if another compaction is running for the column family
- Throws `TidesDBException` with standard I/O or memory error codes if the merge cannot complete

Purge Column Family

Forces a synchronous flush and aggressive compaction for a single column family. Unlike `compact()` and `flushMemtable()` (which are non-blocking), `purge` blocks until all flush and compaction I/O is complete.

```
ColumnFamily cf = db.getColumnFamily("my_cf");
cf.purge();
// All data is now flushed to SSTables and compacted
```

When to use

- Before backup or checkpoint · Ensure all data is on disk and compacted
- After bulk deletes · Reclaim space immediately by compacting away tombstones
- Manual maintenance · Force a clean state during a maintenance window
- Pre-shutdown · Ensure all pending work is complete before closing

Purge Database

Forces a synchronous flush and aggressive compaction for **all** column families, then drains both the global flush and compaction queues.

```
db.purge();
// All CFs flushed and compacted, all queues drained
```

Purge vs Manual Flush + Compact

`flushMemtable()` and `compact()` are non-blocking - they enqueue work and return immediately. `purge()` is synchronous - it blocks until all work is complete. Use `purge` when you need a guarantee that all data is on disk and compacted before proceeding.

Fast Shutdown (Cancel Background Work)

`cancelBackgroundWork()` cancels background compaction database-wide so a subsequent `close()` returns quickly. In-flight merges bail safely at their next checkpoint - their uncommitted output is discarded and the inputs are left intact, so no data is lost (recovery handles a mid-merge state the same way). Queued compaction is skipped. Flushes are unaffected, so durability is preserved. The call blocks (bounded) until compaction is idle.

```
db.cancelBackgroundWork();  
db.close(); // returns quickly without waiting for compaction to drain
```

Behavior

- The cancellation is sticky for the session and is reset on the next `open()`
- Intended to be called immediately before `close()` for a fast shutdown
- This is the opposite end of the spectrum from `Config.finishCompactionsOnClose(true)`, which instead lets in-flight compactions run to completion before `close()` returns

Manual WAL Sync

Forces an immediate fsync of the active write-ahead log for a column family. Useful for explicit durability control when using `SYNC_NONE` or `SYNC_INTERVAL` modes.

```
ColumnFamily cf = db.getColumnFamily("my_cf");  
cf.syncWal();
```

When to use

- Application-controlled durability · Sync the WAL at specific points after a batch of writes
- Pre-checkpoint · Ensure all buffered WAL data is on disk before taking a checkpoint
- Graceful shutdown · Flush WAL buffers before closing the database
- Critical writes · Force durability for specific high-value writes without using `SYNC_FULL` for all writes

Range Cost Estimation

Estimate the computational cost of iterating between two keys in a column family. The returned value is an opaque double - meaningful only for comparison with other values from the same method. Uses only in-memory metadata and performs no disk I/O.

```

ColumnFamily cf = db.getColumnFamily("my_cf");

double costA = cf.rangeCost("user:0000".getBytes(),
    "user:0999".getBytes());
double costB = cf.rangeCost("user:1000".getBytes(),
    "user:1099".getBytes());

if (costA < costB) {
    System.out.println("Range A is cheaper to iterate");
}

```

Key order does not matter - the method normalizes the range so `keyA > keyB` produces the same result as `keyB > keyA`. A cost of 0.0 means no overlapping SSTables or memtable entries were found for the range.

Use cases

- Query planning · Compare candidate key ranges to find the cheapest one to scan
- Load balancing · Distribute range scan work across threads by estimating per-range cost
- Adaptive prefetching · Decide how aggressively to prefetch based on range size
- Monitoring · Track how data distribution changes across key ranges over time

Updating Runtime Configuration

Update runtime-safe configuration settings for a column family:

```

ColumnFamily cf = db.getColumnFamily("my_cf");

ColumnFamilyConfig newConfig = ColumnFamilyConfig.builder()
    .writeBufferSize(256 * 1024 * 1024)
    .skipListMaxLevel(16)
    .bloomFPR(0.001)
    .syncMode(SyncMode.SYNC_INTERVAL)
    .syncIntervalUs(1000000)
    .build();

cf.updateRuntimeConfig(newConfig, true);

```

Updatable settings (safe to change at runtime):

- `writeBufferSize` · Memtable flush threshold
- `skipListMaxLevel` · Skip list level for new memtables
- `skipListProbability` · Skip list probability for new memtables
- `bloomFPR` · False positive rate for new SSTables
- `indexSampleRatio` · Index sampling ratio for new SSTables
- `syncMode` · Durability mode
- `syncIntervalUs` · Sync interval in microseconds

Saving and Loading Column Family Config (INI)

A `ColumnFamilyConfig` can be serialized to an INI file and read back. This is useful for templating column family settings, version-controlling configuration, or inspecting the on-disk `config.ini` files the engine writes for each column family.

```
ColumnFamilyConfig cfConfig = ColumnFamilyConfig.builder()
    .writeBufferSize(96 * 1024 * 1024)
    .compressionAlgorithm(CompressionAlgorithm.ZSTD_COMPRESSION)
    .syncMode(SyncMode.SYNC_INTERVAL)
    .syncIntervalUs(250000)
    .tombstoneDensityTrigger(0.4)
    .useBtree(true)
    .build();

// Write the configuration under a named section
cfConfig.saveToIni("./cf_templates.ini", "analytics_cf");

// Read it back (fields absent from the section fall back to engine
defaults)
ColumnFamilyConfig loaded =
ColumnFamilyConfig.loadFromIni("./cf_templates.ini", "analytics_cf");

db.createColumnFamily("analytics_cf", loaded);
```

Behavior

- `saveToIni(iniFile, section)` is an instance method that writes this configuration under the given `[section]`. If the file exists it is overwritten, then fsynced.
- `loadFromIni(iniFile, section)` is a static factory that returns a new `ColumnFamilyConfig`. Fields missing from the section keep the engine defaults.
- Persisted fields: write buffer size, level ratios, value threshold, compression, bloom/index settings, sync mode and interval, skip-list parameters, default isolation level, compaction

triggers, tombstone density settings, min disk space, B+tree and object-store flags, and the comparator name. Runtime-only fields (commit hooks) are not persisted.

- Both methods throw `TidesDBException` on I/O errors (for example, a missing file on load) and `IllegalArgumentException` for null/empty paths or section names.

Commit Hook (Change Data Capture)

Register a callback that fires synchronously after every transaction commit on a column family. The hook receives the full batch of committed operations atomically, enabling real-time change data capture without WAL parsing.

```
ColumnFamily cf = db.getColumnFamily("my_cf");

cf.setCommitHook((ops, commitSeq) -> {
    for (CommitOp op : ops) {
        if (op.isDelete()) {
            System.out.println("DELETE key=" + new String(op.getKey()));
        } else {
            System.out.println("PUT key=" + new String(op.getKey())
                + " value=" + new String(op.getValue()));
        }
    }
    System.out.println("Commit seq: " + commitSeq);
    return 0;
});

// Normal writes now trigger the hook automatically
try (Transaction txn = db.beginTransaction()) {
    txn.put(cf, "key1".getBytes(), "value1".getBytes());
    txn.commit(); // hook fires here
}

// Detach hook
cf.clearCommitHook();
```

The `CommitHook` functional interface receives a `CommitOp[]` array and a monotonic `commitSeq` number. Each `CommitOp` contains:

- `getKey()` · Key bytes
- `getValue()` · Value bytes (null for deletes)
- `getTtl()` · Time-to-live (-1 for no expiry)
- `isDelete()` · True if this is a delete operation

Behavior

- The hook fires after WAL write, memtable apply, and commit status marking - data is fully durable before the callback runs
- Hook failure (non-zero return) is logged but does not roll back the commit
- Each column family has its own independent hook; a multi-CF transaction fires the hook once per CF with only that CF's operations
- `commitSeq` is monotonically increasing and can be used as a replication cursor
- The hook executes synchronously on the committing thread - keep the callback fast to avoid stalling writers
- Hooks are runtime-only and not persisted. After a database restart, hooks must be re-registered by the application

Use cases

- Replication · Ship committed batches to replicas in commit order
- Event streaming · Publish mutations to Kafka, NATS, or any message broker
- Secondary indexing · Maintain a reverse index or materialized view
- Audit logging · Record every mutation with key, value, TTL, and sequence number
- Debugging · Attach a temporary hook in production to inspect live writes

Multi-Column-Family Transactions

TidesDB supports atomic transactions across multiple column families with true all-or-nothing semantics.

```
ColumnFamily usersCf = db.getColumnFamily("users");
ColumnFamily ordersCf = db.getColumnFamily("orders");

try (Transaction txn = db.beginTransaction()) {
    txn.put(usersCf, "user:1000".getBytes(), "John Doe".getBytes());
    txn.put(ordersCf, "order:5000".getBytes(),
"user:1000|product:A".getBytes());

    txn.commit();
}
```

Multi-CF guarantees

- Either all CFs commit or none do (atomic)

- Automatically detected when operations span multiple CFs
- Uses global sequence numbers for atomic ordering
- Each CF's WAL receives operations with the same commit sequence number
- No two-phase commit or coordinator overhead

Custom Comparators

TidesDB uses comparators to determine the sort order of keys. Once a comparator is set for a column family, it cannot be changed without corrupting data.

Built-in Comparators

- `"memcmp"` (default) · Binary byte-by-byte comparison
- `"lexicographic"` · Null-terminated string comparison
- `"uint64"` · Unsigned 64-bit integer comparison
- `"int64"` · Signed 64-bit integer comparison
- `"reverse"` · Reverse binary comparison (descending order)
- `"case_insensitive"` · Case-insensitive ASCII comparison

Registering a Comparator

```
db.registerComparator("reverse", null);

ColumnFamilyConfig cfConfig = ColumnFamilyConfig.builder()
    .comparatorName("reverse")
    .build();

db.createColumnFamily("sorted_cf", cfConfig);
```

Important

Comparators must be registered before creating column families that use them. Once set, a comparator cannot be changed for a column family.

Database Backup

Create an on-disk snapshot without blocking normal reads/writes:

```
db.backup("./mydb_backup");
```

Database Checkpoint

Create a lightweight, near-instant snapshot of an open database using hard links instead of copying SSTable data:

```
db.checkpoint("./mydb_checkpoint");
```

Checkpoint vs Backup

	<code>backup()</code>	<code>checkpoint()</code>
Speed	Copies every SSTable byte-by-byte	Near-instant (hard links, O(1) per file)
Disk usage	Full independent copy	No extra disk until compaction removes old SSTables
Portability	Can be moved to another filesystem or machine	Same filesystem only (hard link requirement)
Use case	Archival, disaster recovery, remote shipping	Fast local snapshots, point-in-time reads, streaming backups

Behavior

- Requires the directory to be non-existent or empty
- For each column family, flushes the active memtable, halts compactions, hard links all SSTable files, copies small metadata files, then resumes compactions
- Falls back to file copy if hard linking fails (e.g., cross-filesystem)
- Database stays open and usable during checkpoint
- The checkpoint can be opened as a normal TidesDB database with `TidesDB.open()`

Object Store Mode

Object store mode allows TidesDB to store SSTables in a remote object store (S3, MinIO, GCS, or any S3-compatible service) while using local disk as a cache. This separates compute from storage and enables cold start recovery from the remote store. Object store mode requires unified memtable mode and is automatically enforced when a connector is set.

Connector availability

The Java binding exposes two connectors: the **filesystem connector** (`objectStoreFsPath`), which mirrors objects as files under a directory (ideal for testing and NFS/shared-volume replication), and the **S3 connector** (`objectStoreS3Config`) for AWS S3, MinIO, GCS, and other S3-compatible services. The S3 connector requires a native library built with `-DTIDESDB_WITH_S3=ON`; call `TidesDB.isS3Available()` to probe support at runtime. When S3 is not compiled in, opening an S3-configured database throws a `TidesDBException` explaining that S3 support is missing.

Enabling Object Store Mode (Filesystem Connector)

```
ObjectStoreConfig osConfig = ObjectStoreConfig.defaultConfig();

Config config = Config.builder("./mydb")
    .numFlushThreads(2)
    .numCompactionThreads(2)
    .logLevel(LogLevel.INFO)
    .blockCacheSize(64 * 1024 * 1024)
    .maxOpenSSTables(256)
    .objectStoreFsPath("/mnt/nfs/tidesdb-objects")
    .objectStoreConfig(osConfig)
    .build();

try (TidesDB db = TidesDB.open(config)) {
    // SSTables are uploaded after flush
}
```

Enabling Object Store Mode (S3 Connector)

Back the database with an S3-compatible object store. `endpoint`, `bucket`, `accessKey`, and `secretKey` are required; the remaining fields default to secure, AWS-friendly values (HTTPS on, virtual-hosted URLs, TLS verification enabled). The S3 connector takes precedence over `objectStoreFsPath` when both are set, and composes with `ObjectStoreConfig` for cache, multipart, and WAL-replication tuning.

```

// Probe support first (requires a core build with -
DTIDESDB_WITH_S3=ON)
if (!TidesDB.isS3Available()) {
    throw new IllegalStateException("TidesDB native library was built
without S3 support");
}

S3Config s3 = S3Config.builder()
    .endpoint("s3.amazonaws.com")
    .bucket("my-tidesdb-bucket")
    .prefix("production/db1/") // optional
    .accessKey(System.getenv("AWS_ACCESS_KEY_ID"))
    .secretKey(System.getenv("AWS_SECRET_ACCESS_KEY"))
    .region("us-east-1")
    .build();

Config config = Config.builder("./mydb")
    .objectStoreS3Config(s3)
    .objectStoreConfig(ObjectStoreConfig.builder()
        .localCacheMaxBytes(1024L * 1024 * 1024) // 1GB local cache
        .maxConcurrentUploads(8)
        .build())
    .build();

try (TidesDB db = TidesDB.open(config)) {
    // SSTables are uploaded to S3 after flush; local disk is used as a
    cache
}

```

For MinIO or another self-hosted S3 service, use path-style URLs and (for plain HTTP) disable SSL:

```

S3Config minio = S3Config.builder()
    .endpoint("minio.local:9000")
    .bucket("tidesdb")
    .accessKey("minioadmin")
    .secretKey("minioadmin")
    .usePathStyle(true) // MinIO requires path-style addressing
    .useSsl(false) // plain HTTP; omit for HTTPS
    .build();

```

Custom Object Store Configuration

```
ObjectStoreConfig osConfig = ObjectStoreConfig.builder()
    .localCacheMaxBytes(512 * 1024 * 1024) // 512MB local cache
    .maxConcurrentUploads(8)
    .maxConcurrentDownloads(16)
    .cacheOnRead(true)
    .cacheOnWrite(true)
    .syncManifestToObject(true)
    .replicateWal(true)
    .walSyncThresholdBytes(1048576) // 1MB
    .build();

Config config = Config.builder("./mydb")
    .numFlushThreads(2)
    .numCompactionThreads(2)
    .logLevel(LogLevel.INFO)
    .objectStoreFsPath("/mnt/nfs/tidesdb-objects")
    .objectStoreConfig(osConfig)
    .build();

try (TidesDB db = TidesDB.open(config)) {
    // use the database normally
}
```

Per-CF Object Store Tuning

Column family configurations include two object store tuning fields:

```
ColumnFamilyConfig cfConfig = ColumnFamilyConfig.builder()
    .writeBufferSize(128 * 1024 * 1024)
    .compressionAlgorithm(CompressionAlgorithm.LZ4_COMPRESSION)
    .objectLazyCompaction(true) // compact less aggressively
    .objectPrefetchCompaction(true) // download all inputs
    before merge
    .build();

db.createColumnFamily("remote_cf", cfConfig);
```


Object Store Statistics

`getDbStats()` includes object store fields when a connector is active:

```
DbStats dbStats = db.getDbStats();

if (dbStats.isObjectStoreEnabled()) {
    System.out.println("Connector: " + dbStats.getObjectStoreConnector());
    System.out.println("Total uploads: " + dbStats.getTotalUploads());
    System.out.println("Upload failures: " +
dbStats.getTotalUploadFailures());
    System.out.println("Upload queue depth: " +
dbStats.getUploadQueueDepth());
    System.out.println("Local cache: " + dbStats.getLocalCacheBytesUsed()
        + " / " + dbStats.getLocalCacheBytesMax() + " bytes");
}
```

Cold Start Recovery

When the local database directory is empty but a connector is configured, TidesDB automatically discovers column families from the object store during recovery.

```
ObjectStoreConfig osConfig = ObjectStoreConfig.defaultConfig();

Config config = Config.builder("./mydb")
    .objectStoreFsPath("/mnt/nfs/tidesdb-objects")
    .objectStoreConfig(osConfig)
    .build();

try (TidesDB db = TidesDB.open(config)) {
    // all data is available -- SSTables are fetched on demand
    ColumnFamily cf = db.getColumnFamily("my_cf");
}
```

How It Works

- Object store mode requires unified memtable mode. Setting `objectStoreFsPath` automatically enables `unifiedMemtable`
- After each flush, SSTables are uploaded via an asynchronous upload pipeline with retry and post-upload verification
- Point lookups on frozen SSTables fetch just the needed klog block via one HTTP range request

- Iterators prefetch all needed SSTable files in parallel at creation time
- A hash-indexed LRU local file cache manages disk usage when `localCacheMaxBytes` is set
- The MANIFEST is uploaded after each flush and compaction for cold start recovery
- Upload failures are tracked in `totalUploadFailures` on `DbStats`

Replica Mode

Replica mode enables read-only nodes that follow a primary through the object store.

```
ObjectStoreConfig osConfig = ObjectStoreConfig.builder()
    .replicaMode(true)
    .replicaSyncIntervalUs(1000000) // 1 second sync interval
    .replicaReplayWal(true)         // replay WAL for fresh reads
    .build();

Config config = Config.builder("./mydb_replica")
    .objectStoreFsPath("/mnt/nfs/tidesdb-objects") // same path as primary
    .objectStoreConfig(osConfig)
    .build();

try (TidesDB db = TidesDB.open(config)) {
    // reads work normally
    ColumnFamily cf = db.getColumnFamily("my_cf");
    try (Transaction txn = db.beginTransaction()) {
        byte[] value = txn.get(cf, "key".getBytes());
        // writes throw TidesDBException with ERR_READONLY
    }
}
```

Sync-on-Commit WAL (Primary Side)

For tighter replication lag, enable sync-on-commit on the primary:

```
ObjectStoreConfig osConfig = ObjectStoreConfig.builder()
    .walSyncOnCommit(true) // RPO = 0, every commit is durable in object
store
    .build();
// replica sees committed data within one replicaSyncIntervalUs
```

Promote Replica to Primary

When the primary fails, promote a replica to accept writes:

```
db.promoteToPrimary();  
// now writes are accepted
```

Renaming Column Families

Atomically rename a column family:

```
db.renameColumnFamily("old_name", "new_name");
```

Cloning Column Families

Create a complete copy of an existing column family with a new name. The clone is completely independent; modifications to one do not affect the other.

```
db.cloneColumnFamily("source_cf", "cloned_cf");  
  
ColumnFamily original = db.getColumnFamily("source_cf");  
ColumnFamily clone = db.getColumnFamily("cloned_cf");
```

Use cases

- Testing · Create a copy of production data for testing without affecting the original
- Branching · Create a snapshot of data before making experimental changes
- Migration · Clone data before schema or configuration changes
- Backup verification · Clone and verify data integrity without modifying the source

B+tree KLog Format (Optional)

Column families can optionally use a B+tree structure for the key log instead of the default block-based format. The B+tree klog format offers faster point lookups through $O(\log N)$ tree traversal.

```

ColumnFamilyConfig btreeConfig = ColumnFamilyConfig.builder()
    .writeBufferSize(128 * 1024 * 1024)
    .compressionAlgorithm(CompressionAlgorithm.LZ4_COMPRESSION)
    .enableBloomFilter(true)
    .useBtree(true)
    .build();

db.createColumnFamily("btree_cf", btreeConfig);

ColumnFamily cf = db.getColumnFamily("btree_cf");

try (Transaction txn = db.beginTransaction()) {
    txn.put(cf, "key".getBytes(), "value".getBytes());
    txn.commit();
}

Stats stats = cf.getStats();
if (stats.isUseBtree()) {
    System.out.println("B+tree nodes: " + stats.getBtreeTotalNodes());
    System.out.println("B+tree max height: " + stats.getBtreeMaxHeight());
    System.out.println("B+tree avg height: " + stats.getBtreeAvgHeight());
}

```

When to use B+tree klog format

- Read-heavy workloads with frequent point lookups
- Workloads where read latency is more important than write throughput
- Large SSTables where block scanning becomes expensive

Tradeoffs

- Slightly higher write amplification during flush
- Larger metadata overhead per node
- Block-based format may be faster for sequential scans

Important

`useBtree` cannot be changed after column family creation.

Transaction Isolation Levels

```
try (Transaction txn =
    db.beginTransaction(IsolationLevel.SERIALIZABLE)) {
    txn.commit();
}
```

Available Isolation Levels

- `READ_UNCOMMITTED` · Sees all data including uncommitted changes
- `READ_COMMITTED` · Sees only committed data (default)
- `REPEATABLE_READ` · Consistent snapshot, phantom reads possible
- `SNAPSHOT` · Write-write conflict detection
- `SERIALIZABLE` · Full read-write conflict detection (SSI)

Savepoints

Savepoints allow partial rollback within a transaction:

```
try (Transaction txn = db.beginTransaction()) {
    txn.put(cf, "key1".getBytes(), "value1".getBytes());

    txn.savepoint("sp1");
    txn.put(cf, "key2".getBytes(), "value2".getBytes());

    txn.rollbackToSavepoint("sp1");

    txn.commit();
}
```

Savepoint API

- `savepoint(name)` · Create a savepoint
- `rollbackToSavepoint(name)` · Rollback to savepoint
- `releaseSavepoint(name)` · Release savepoint without rolling back

Transaction Reset

`reset` resets a committed or aborted transaction for reuse with a new isolation level. This avoids the overhead of freeing and reallocating transaction resources in hot loops.

```
ColumnFamily cf = db.getColumnFamily("my_cf");

Transaction txn = db.beginTransaction();
txn.put(cf, "key1".getBytes(), "value1".getBytes());
txn.commit();

txn.reset(IsolationLevel.READ_COMMITTED);

txn.put(cf, "key2".getBytes(), "value2".getBytes());
txn.commit();

txn.free();
```

Behavior

- The transaction must be committed or aborted before reset; resetting an active transaction throws `TidesDBException`
- Internal buffers are retained to avoid reallocation
- A fresh transaction ID and snapshot sequence are assigned based on the new isolation level
- The isolation level can be changed on each reset (e.g., `READ_COMMITTED` to `REPEATABLE_READ`)

When to use

- Batch processing · Reuse a single transaction across many commit cycles in a loop
- Connection pooling · Reset a transaction for a new request without reallocation
- High-throughput ingestion · Reduce allocation overhead in tight write loops

Reset vs Free + Begin

For a single transaction, `reset` is functionally equivalent to calling `free` followed by `beginTransaction`. The difference is performance, reset retains allocated buffers and avoids repeated allocation overhead. This matters most in loops that commit and restart thousands of transactions.

Configuration Options

Database Configuration

Option	Type	Default	Description
<code>dbPath</code>	String	-	Path to the database directory
<code>numFlushThreads</code>	int	2	Number of flush threads
<code>numCompactionThreads</code>	int	2	Number of compaction threads
<code>logLevel</code>	LogLevel	INFO	Logging level
<code>blockCacheSize</code>	long	64MB	Block cache size in bytes
<code>maxOpenSSTables</code>	long	256	Maximum open SSTable files
<code>logToFile</code>	boolean	false	Write logs to file instead of stderr
<code>logTruncationAt</code>	long	24MB	Log file truncation size (0 = no truncation)
<code>maxMemoryUsage</code>	long	0	Global memory limit in bytes (0 = auto, 50% of system RAM)
<code>unifiedMemtable</code>	boolean	false	Enable unified memtable mode (single memtable + WAL for all CFs)
<code>unifiedMemtableWriteBufferSize</code>	long	0	Unified memtable write buffer size (0 = auto, 64MB)
<code>unifiedMemtableSkipListMaxLevel</code>	int	0	Skip list max level for unified memtable (0 = default 12)
<code>unifiedMemtableSkipListProbability</code>	float	0	Skip list probability for unified memtable (0 = default 0.25)
<code>unifiedMemtableSyncMode</code>	int	0	Sync mode for unified WAL (0 = SYNC_NONE)

Option	Type	Default	Description
<code>ode</code>			
<code>unifiedMemtableSyncIntervalUs</code>	long	0	Sync interval for unified WAL in microseconds
<code>objectStoreFsPath</code>	String	null	Filesystem connector root directory (null = no object store)
<code>objectStoreConfig</code>	ObjectStoreConfig	null	Object store behavior configuration (null = use defaults)
<code>maxConcurrentFlushes</code>	int	0 (library default)	Global semaphore on in-flight memtable flushes across all CFs (0 = library default)
<code>finishCompactionsOnClose</code>	boolean	false	Close behavior. <code>false</code> cancels in-flight compactions at their next checkpoint for a fast shutdown (no data is lost; recovery handles a mid-merge state). <code>true</code> lets in-flight compactions run to completion before <code>close()</code> returns

Object Store Configuration

Option	Type	Default	Description
<code>localCachePath</code>	String	null	Local directory for cached SSTable files (null = use db_path)
<code>localCacheMaxBytes</code>	long	0	Maximum local cache size in bytes (0 = unlimited)
<code>cacheOnRead</code>	boolean	true	Cache downloaded files locally
<code>cacheOnWrite</code>	boolean	true	Keep local copy after upload
<code>maxConcurrentUploads</code>	int	4	Number of parallel upload threads
<code>maxConcurrentDownloads</code>	int	8	Number of parallel download threads
<code>multipartThreshold</code>	long	64MB	Use multipart upload above this size
<code>multipartPartSize</code>	long	8MB	Chunk size for multipart uploads
<code>syncManifestToObject</code>	boolean	true	Upload MANIFEST after each compaction
<code>replicateWal</code>	boolean	true	Upload closed WAL segments for replication
<code>walUploadSync</code>	boolean	false	false = background WAL upload, true = block flush until uploaded
<code>walSyncThresholdBytes</code>	long	1MB	Sync active WAL when it grows by this many bytes (0 = disable)
<code>walSyncOnCommit</code>	boolean	false	Upload WAL after every txn commit for RPO=0 replication
<code>replicaMode</code>	boolean	false	Enable read-only replica mode
<code>replicaSyncIntervalus</code>	long	5000000	MANIFEST poll interval for replica sync in microseconds
<code>replicaReplayWal</code>	boolean	true	Replay WAL from object store for near-real-time reads on replicas

S3 Connector Configuration

`S3Config` selects an S3-compatible backend (set via `Config.Builder.objectStoreS3Config`). Requires a native build with `-DTIDESDB_WITH_S3=ON`; probe with `TidesDB.isS3Available()`.

Option	Type	Default	Description
<code>endpoint</code>	String	— (required)	S3 endpoint, e.g. <code>s3.amazonaws.com</code> or <code>minio.local:9000</code>
<code>bucket</code>	String	— (required)	Bucket name
<code>prefix</code>	String	null	Key prefix, e.g. <code>production/db1/</code>
<code>accessKey</code>	String	— (required)	AWS access key ID
<code>secretKey</code>	String	— (required)	AWS secret access key
<code>region</code>	String	null	AWS region, e.g. <code>us-east-1</code> ; null for MinIO/default
<code>useSsl</code>	boolean	true	HTTPS when true, HTTP when false
<code>usePathStyle</code>	boolean	false	Path-style URLs (MinIO) when true, virtual-hosted (AWS) when false
<code>tlsCaPath</code>	String	null	Custom CA bundle file path, or null for the system bundle
<code>tlsInsecureSkipVerify</code>	boolean	false	Disable TLS peer/host verification (test only, insecure)
<code>multipartThreshold</code>	long	0 (library default)	Object size at/above which multipart upload is used
<code>multipartPartSize</code>	long	0 (library default)	Multipart chunk size in bytes

Column Family Configuration

Option	Type	Default	Description
<code>writeBufferSize</code>	long	128MB	Memtable flush threshold
<code>levelSizeRatio</code>	long	10	Level size multiplier
<code>minLevels</code>	int	5	Minimum LSM levels
<code>dividingLevelOffset</code>	int	2	Compaction dividing level offset
<code>klogValueThreshold</code>	long	512	Values > threshold go to vlog
<code>compressionAlgorithm</code>	CompressionAlgorithm	LZ4_COMPRESSION	Compression algorithm
<code>enableBloomFilter</code>	boolean	true	Enable bloom filters
<code>bloomFPR</code>	double	0.01	Bloom filter false positive rate (1%)
<code>enableBlockIndexes</code>	boolean	true	Enable compact block indexes
<code>indexSampleRatio</code>	int	1	Sample every block for index
<code>blockIndexPrefixLength</code>	int	16	Block index prefix length
<code>syncMode</code>	SyncMode	SYNC_FULL	Sync mode for durability
<code>syncInterval</code>	long	1000000	Sync interval (1 second, for SYNC_INTERVAL)
<code>comparatorName</code>	String	""	Custom comparator name (empty = memcmp)
<code>skipListMaxLevel</code>	int	12	Skip list max level

Option	Type	Default	Description
<code>skipListProbability</code>	float	0.25	Skip list probability
<code>defaultIsolationLevel</code>	IsolationLevel	READ_COMMITTED	Default transaction isolation
<code>minDiskSpace</code>	long	100MB	Minimum disk space required
<code>l1FileCountTrigger</code>	int	4	L1 file count trigger for compaction
<code>l0QueueStallThreshold</code>	int	20	L0 queue stall threshold
<code>tombstoneDensityTrigger</code>	double	0.0	Per-SSTable tombstone density above which compaction priority escalates (0.0 = disabled, range 0.0 to 1.0)
<code>tombstoneDensityMinEntries</code>	long	1024	Minimum entry count for an SSTable to be considered by the tombstone density trigger
<code>useBtree</code>	boolean	false	Use B+tree format for klog (faster point lookups)
<code>objectLazyCompaction</code>	boolean	false	Compact less aggressively for remote storage
<code>objectPrefetchCompaction</code>	boolean	true	Download all inputs before compaction merge

Compression Algorithms

Algorithm	Value	Description
<code>NO_COMPRESSION</code>	0	No compression
<code>SNAPPY_COMPRESSION</code>	1	Snappy compression
<code>LZ4_COMPRESSION</code>	2	LZ4 standard compression (default)
<code>ZSTD_COMPRESSION</code>	3	Zstandard compression (best ratio)
<code>LZ4_FAST_COMPRESSION</code>	4	LZ4 fast mode (higher throughput)

Sync Modes

Mode	Description
<code>SYNC_NONE</code>	No explicit sync, relies on OS page cache (fastest)
<code>SYNC_FULL</code>	Fsync on every write (most durable)
<code>SYNC_INTERVAL</code>	Periodic background syncing at configurable intervals

Error Codes

Code	Value	Description
ERR_SUCCESS	0	Operation completed successfully
ERR_MEMORY	-1	Memory allocation failed
ERR_INVALID_ARGS	-2	Invalid arguments passed
ERR_NOT_FOUND	-3	Key not found
ERR_IO	-4	I/O operation failed
ERR_CORRUPTION	-5	Data corruption detected
ERR_EXISTS	-6	Resource already exists
ERR_CONFLICT	-7	Transaction conflict detected
ERR_TOO_LARGE	-8	Key or value size exceeds maximum
ERR_MEMORY_LIMIT	-9	Memory limit exceeded
ERR_INVALID_DB	-10	Database handle is invalid
ERR_UNKNOWN	-11	Unknown error
ERR_LOCKED	-12	Database is locked
ERR_READONLY	-13	Database is read-only (replica mode)
ERR_BUSY	-14	Resource is busy

Testing



```
# Run all tests
mvn test

# Run specific test
mvn test -Dtest=TidesDBTest#testOpenClose

# Run with verbose output
mvn test -X
```

Building from Source



```
# Clone the repository
git clone https://github.com/tidesdb/tidesdb-java.git
cd tidesdb-java

# Build the JNI library
cd src/main/c
cmake -S . -B build
cmake --build build
sudo cmake --install build
cd ../../..

# Build the Java package
mvn clean package

# Install to local Maven repository
mvn install
```

TidesDB Rust API Reference

Getting Started

Prerequisites

You need a C compiler and CMake installed, along with compression libraries. The crate will automatically download and build TidesDB from source if the C library is not already installed.

Debian/Ubuntu



```
sudo apt install build-essential cmake libzstd-dev liblz4-dev  
libsnappy-dev
```

macOS



```
brew install cmake zstd lz4 snappy
```

Windows



```
vcpkg install zstd:x64-windows lz4:x64-windows snappy:x64-windows
```

For S3 object store support, you also need `libcurl` and `openssl` (see [Object Store Support](#) below).

Installing from crates.io

The easiest way to add TidesDB to your project is via crates.io:


```
[dependencies]
tidesdb = "0.11"
```

Or using `cargo add`:



```
cargo add tidesdb
```

How the Build System Works

The build script (`build.rs`) handles linking automatically:

1. `pkg-config` check · It first tries to find TidesDB already installed on your system via `pkg-config` with an exact version match. If found, it links against the system library and no further steps are needed.
2. Auto-build from source · If the system library is not found, the build script automatically downloads the matching TidesDB C library source from GitHub, builds it with CMake as a static library, and links it into your project.
3. Compression libraries · TidesDB depends on `zstd`, `lz4`, and `snappy`. The build script tries to find these via `pkg-config` and falls back to linking by name if not found.

Version Selection

Each crate release pins the TidesDB C library version it builds against via a Cargo feature. `tidesdb` 0.11 ships and defaults to `v9_3_6` (TidesDB v9.3.6):

```
[dependencies]
# Uses the default version (v9.3.6)
tidesdb = "0.11"
```

A version feature maps directly to the TidesDB C library release tag (e.g., `v9_3_6` → `v9.3.6`). Additional versions are published over time; when more than one is available you can pin a specific one with `default-features = false`:

```
[dependencies]
# Pin to a specific TidesDB version (once that feature is offered)
tidesdb = { version = "0.11", default-features = false, features =
["v9_3_6"] }
```

Only one version feature should be enabled at a time; when several are enabled (e.g. via dependency unification), the highest version is selected.

Object Store Support

To enable S3 object store support, enable the `objectstore` feature:

```
[dependencies]
tidesdb = { version = "0.11", features = ["objectstore"] }
```

This requires additional dependencies:

- Debian/Ubuntu · `sudo apt install libcurl4-openssl-dev libssl-dev`
- macOS · `brew install curl openssl`
- Windows · `vcpkg install curl:x64-windows openssl:x64-windows` (requires tidesdb >= v9.0.6)

Building from GitHub

To build the Rust bindings directly from the GitHub repository:



```
git clone https://github.com/tidesdb/tidesdb-rs.git
cd tidesdb-rs
cargo build --release
cargo test -- --test-threads=1
```

Using directly from GitHub in your Cargo.toml

```
[dependencies]
tidesdb = { git = "https://github.com/tidesdb/tidesdb-rs.git" }
```

Using pkg-config



```
# If TidesDB was installed with pkg-config support
export PKG_CONFIG_PATH="/custom/path/lib/pkgconfig:$PKG_CONFIG_PATH"
cargo build
```

Custom prefix installation



```
# Install TidesDB to custom location
cd tidesdb
cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/opt/tidesdb
cmake --build build
sudo cmake --install build

# Configure environment
export LIBRARY_PATH="/opt/tidesdb/lib:$LIBRARY_PATH"
export LD_LIBRARY_PATH="/opt/tidesdb/lib:$LD_LIBRARY_PATH" # Linux
# or
export DYLD_LIBRARY_PATH="/opt/tidesdb/lib:$DYLD_LIBRARY_PATH" # macOS

cargo build
```

Initialization

TidesDB supports optional custom memory allocators for integration with custom memory managers (e.g., jemalloc, mimalloc).

`init`

Initializes TidesDB with the system allocator. Must be called exactly once before any other TidesDB function when using the explicit initialization path.

```
use tidesdb;

fn main() -> tidesdb::Result<()> {
    tidesdb::init()?;

    // ... use TidesDB ...

    tidesdb::finalize();
    Ok(())
}
```

`init_with_allocator`

Initializes TidesDB with custom C-level memory allocator functions. This is an `unsafe` function for advanced use cases.

```
use tidesdb;

// Example with custom allocator function pointers
unsafe {
    tidesdb::init_with_allocator(
        Some(my_malloc),
        Some(my_calloc),
        Some(my_realloc),
        Some(my_free),
    )?;
}
```

`finalize`

Finalizes TidesDB and resets the allocator. Should be called after all TidesDB operations are complete (all databases closed). After calling this, `init()` or `init_with_allocator()` can be called again.

```
tidesdb::finalize();
```

Auto-initialization

If `init()` is not called, TidesDB will auto-initialize with the system allocator on the first call to `TidesDB::open()`.

Usage

Opening and Closing a Database

```
use tidesdb::{TidesDB, Config, LogLevel, SyncMode};

fn main() -> tidesdb::Result<()> {
    let config = Config::new("./mydb")
        .num_flush_threads(2)
        .num_compaction_threads(2)
        .log_level(LogLevel::Info)
        .block_cache_size(64 * 1024 * 1024)
        .max_open_sstables(256)
        .max_memory_usage(0) // 0 = auto (50% of system
RAM)
        .log_to_file(false) // Write logs to file
instead of stderr
        .log_truncation_at(24 * 1024 * 1024) // Log file truncation
threshold (24MB)
        .unified_memtable(false) // Enable unified memtable
mode (default: false)
        .unified_memtable_write_buffer_size(0) // 0 = auto
        .unified_memtable_sync_mode(SyncMode::None)
        .max_concurrent_flushes(0); // Cap on in-flight memtable
flushes across all CFs (0 = library default)

    let db = TidesDB::open(config)?;

    println!("Database opened successfully");

    Ok(())
}
```

`Config::default()` (and therefore `Config::new()`) now mirrors the library's defaults rather than carrying duplicate constants in Rust, so values such as `max_concurrent_flushes` and the unified-memtable settings track the engine's defaults automatically. If you previously relied on the binding's

hardcoded defaults (e.g. `block_cache_size = 64 MiB`), re-test with the library defaults or set the values explicitly.

max_concurrent_flushes

`max_concurrent_flushes` is a global semaphore on the number of in-flight memtable flushes across all column families. It bounds total transient memory and work-queue depth when many column families flush at once. Leave it at `0` to inherit the library default (recommended), or lower it on memory-constrained hosts to throttle peak flush concurrency.

Creating and Dropping Column Families

Column families are isolated key-value stores with independent configuration.

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig,
CompressionAlgorithm, SyncMode, IsolationLevel};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;

    let cf_config = ColumnFamilyConfig::default();
    db.create_column_family("my_cf", cf_config)?;

    let cf_config = ColumnFamilyConfig::new()
        .write_buffer_size(128 * 1024 * 1024)
        .level_size_ratio(10)
        .min_levels(5)
        .compression_algorithm(CompressionAlgorithm::Lz4)
        .enable_bloom_filter(true)
        .bloom_fpr(0.01)
        .enable_block_indexes(true)
        .sync_mode(SyncMode::Interval)
        .sync_interval_us(128000)
        .default_isolation_level(IsolationLevel::ReadCommitted)
        .tombstone_density_trigger(0.5)           // Per-SSTable tombstone
ratio that escalates compaction priority (0.0 = disabled)
        .tombstone_density_min_entries(1024)     // Minimum entry count for
an SSTable to be considered by the density trigger
        .use_btree(false);                      // Use block-based format
(default)

    db.create_column_family("custom_cf", cf_config)?;

    db.drop_column_family("my_cf")?;

    Ok(())
}

```

Dropping by Pointer

When you already hold a `ColumnFamily`, you can skip the name lookup:

```

let cf = db.get_column_family("my_cf")?;
db.delete_column_family(cf)?; // cf is consumed and cannot be used after
this

```



Which to use

- `drop_column_family(name)` · Convenient when you only have the name
- `delete_column_family(cf)` · Faster when you already hold a `ColumnFamily`, avoids a redundant linear scan

CRUD Operations

All operations in TidesDB are performed through transactions for ACID guarantees.

Writing Data

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let mut txn = db.begin_transaction()?;

    txn.put(&cf, b"key", b"value", -1)?;

    txn.commit()?;

    Ok(())
}
```


Writing with TTL

```
use std::time::{SystemTime, UNIX_EPOCH};
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let mut txn = db.begin_transaction()?;

    let ttl = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .unwrap()
        .as_secs() as i64 + 10; // Expires in 10 seconds

    txn.put(&cf, b"temp_key", b"temp_value", ttl)?;

    txn.commit()?;

    Ok(())
}
```

TTL Examples

```

use std::time::{SystemTime, UNIX_EPOCH, Duration};

let ttl: i64 = -1;

let ttl = SystemTime::now()
    .duration_since(UNIX_EPOCH)
    .unwrap()
    .as_secs() as i64 + 5 * 60;

let ttl = SystemTime::now()
    .duration_since(UNIX_EPOCH)
    .unwrap()
    .as_secs() as i64 + 60 * 60;

let ttl: i64 = 1798761599;

```

Reading Data

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let txn = db.begin_transaction()?;

    let value = txn.get(&cf, b"key")?;
    println!("Value: {:?}", String::from_utf8_lossy(&value));

    Ok(())
}

```

Deleting Data

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let mut txn = db.begin_transaction()?;
    txn.delete(&cf, b"key")?;
    txn.commit()?;

    Ok(())
}
```

Single-Delete

`Transaction::single_delete` writes a tombstone with the same read semantics as `Transaction::delete`, but carries a caller-provided promise that lets compaction drop the put and the tombstone together as soon as both appear in the same merge input, rather than carrying the tombstone forward until it reaches the largest active level.

Between any two single-deletes on the same key, and between the start of the key's history and its first single-delete, the key has been put **at most once**. The engine does not and cannot verify this at runtime; violating the contract can leave older puts visible after the single-delete and is a bug in the caller.

This is the right choice for workloads that insert each key exactly once and then delete it exactly once (classic insert-benchmark patterns, secondary-index entries on columns that are never updated, log-style tables with scheduled purges). It is **not** safe for tables that issue repeated updates to the same key. When in doubt, prefer `Transaction::delete`.

Requires tidesdb >= 9.1.0; the default `v9_3_6` build satisfies this.

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let mut txn = db.begin_transaction()?;
    txn.single_delete(&cf, b"key")?;
    txn.commit()?;

    Ok(())
}
```

Signature:

```
pub fn single_delete(&self, cf: &ColumnFamily, key: &[u8]) ->
Result<()>;
```

Returns `Ok(())` on success or an `Error` on failure.

Multi-Operation Transactions

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let mut txn = db.begin_transaction()?;

    txn.put(&cf, b"key1", b"value1", -1)?;
    txn.put(&cf, b"key2", b"value2", -1)?;
    txn.delete(&cf, b"old_key")?;

    txn.commit()?;

    Ok(())
}
```

Transaction Rollback

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let mut txn = db.begin_transaction()?;
    txn.put(&cf, b"key", b"value", -1)?;

    txn.rollback()?;

    Ok(())
}
```

Multi-Column-Family Transactions

TidesDB supports atomic transactions across multiple column families with true all-or-nothing semantics.

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("users", ColumnFamilyConfig::default())?;
    db.create_column_family("orders", ColumnFamilyConfig::default())?;

    let users_cf = db.get_column_family("users")?;
    let orders_cf = db.get_column_family("orders")?;

    let mut txn = db.begin_transaction()?;

    txn.put(&users_cf, b"user:1000", b"John Doe", -1)?;
    txn.put(&orders_cf, b"order:5000", b"user:1000|product:A", -1)?;

    // Commit atomically -- all or nothing
    txn.commit()?;

    Ok(())
}
```

Multi-CF guarantees

- Either all CFs commit or none do (atomic)
- Automatically detected when operations span multiple CFs
- Uses global sequence numbers for atomic ordering
- Each CF's WAL receives operations with the same commit sequence number

Transaction Reset

`Transaction::reset` resets a committed or aborted transaction for reuse with a new isolation level. This avoids the overhead of freeing and reallocating transaction resources in hot loops.

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig, IsolationLevel};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let mut txn = db.begin_transaction()?;

    txn.put(&cf, b"key1", b"value1", -1)?;
    txn.commit()?;

    txn.reset(IsolationLevel::ReadCommitted)?;

    txn.put(&cf, b"key2", b"value2", -1)?;
    txn.commit()?;

    Ok(())
}

```

Behavior

- The transaction must be committed or aborted before reset; resetting an active transaction returns an error
- Internal buffers are retained to avoid reallocation
- A fresh transaction ID and snapshot sequence are assigned based on the new isolation level
- The isolation level can be changed on each reset (e.g., `ReadCommitted` -> `RepeatableRead`)

When to use

- Batch processing · Reuse a single transaction across many commit cycles in a loop
- Connection pooling · Reset a transaction for a new request without reallocation
- High-throughput ingestion · Reduce allocation overhead in tight write loops



Reset vs Drop + Begin

For a single transaction, `reset` is functionally equivalent to dropping the transaction and calling `begin_transaction_with_isolation`. The difference is performance, reset retains allocated buffers and avoids repeated allocation overhead. This matters most in loops that commit and restart thousands of transactions.

Iterating Over Data

Iterators provide efficient bidirectional traversal over key-value pairs.

Forward Iteration

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let txn = db.begin_transaction()?;
    let mut iter = txn.new_iterator(&cf)?;

    iter.seek_to_first()?;

    while iter.is_valid() {
        let key = iter.key()?;
        let value = iter.value()?;

        println!("Key: {:?}, Value: {:?}",
            String::from_utf8_lossy(&key),
            String::from_utf8_lossy(&value));

        iter.next()?;
    }

    Ok(())
}
```


Backward Iteration

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let txn = db.begin_transaction()?;
    let mut iter = txn.new_iterator(&cf)?;

    iter.seek_to_last()?;

    while iter.is_valid() {
        let key = iter.key()?;
        let value = iter.value()?;

        println!("Key: {:?}, Value: {:?}",
            String::from_utf8_lossy(&key),
            String::from_utf8_lossy(&value));

        iter.prev()?;
    }

    Ok(())
}
```

Seek Operations

`seek(key)` positions the iterator at the first key $\geq$ target key:

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let txn = db.begin_transaction()?;
    let mut iter = txn.new_iterator(&cf)?;

    // Seek to prefix and iterate all matching keys
    iter.seek(b"user:")?;

    while iter.is_valid() {
        let key = iter.key()?;

        if !key.starts_with(b"user:") {
            break;
        }

        let value = iter.value()?;
        println!("Key: {:?}, Value: {:?}",
            String::from_utf8_lossy(&key),
            String::from_utf8_lossy(&value));

        iter.next()?;
    }

    Ok(())
}

```

seek_for_prev(key) positions the iterator at the last key $\leq$ target key:

```

let txn = db.begin_transaction()?;
let mut iter = txn.new_iterator(&cf)?;

// Seek for reverse iteration from a specific key
iter.seek_for_prev(b"user:2000")?;

while iter.is_valid() {
    let key = iter.key()?;
    let value = iter.value()?;
    println!("Key: {:?}, Value: {:?}",
        String::from_utf8_lossy(&key),
        String::from_utf8_lossy(&value));
    iter.prev()?;
}

```

Combined Key-Value Retrieval

`key_value()` retrieves both key and value in a single FFI call, which is more efficient than calling `key()` and `value()` separately:

```

let txn = db.begin_transaction()?;
let mut iter = txn.new_iterator(&cf)?;
iter.seek_to_first()?;

while iter.is_valid() {
    let (key, value) = iter.key_value()?;
    println!("Key: {:?}, Value: {:?}",
        String::from_utf8_lossy(&key),
        String::from_utf8_lossy(&value));
    iter.next()?;
}

```

Getting Column Family Statistics

Retrieve detailed statistics about a column family.

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let stats = cf.get_stats()?;

    println!("Number of Levels: {}", stats.num_levels);
    println!("Memtable Size: {} bytes", stats.memtable_size);
    println!("Total Keys: {}", stats.total_keys);
    println!("Total Data Size: {} bytes", stats.total_data_size);
    println!("Average Key Size: {:.2} bytes", stats.avg_key_size);
    println!("Average Value Size: {:.2} bytes", stats.avg_value_size);
    println!("Read Amplification: {:.2}", stats.read_amp);
    println!("Hit Rate: {:.1}%", stats.hit_rate * 100.0);

    for (i, (size, count)) in stats.level_sizes.iter()
        .zip(stats.level_num_sstables.iter())
        .enumerate()
    {
        println!("Level {}: {} SSTables, {} bytes", i + 1, count, size);
    }

    // B+tree stats (only populated if use_btree=true)
    if stats.use_btree {
        println!("B+tree Total Nodes: {}", stats.btree_total_nodes);
        println!("B+tree Max Height: {}", stats.btree_max_height);
        println!("B+tree Avg Height: {:.2}", stats.btree_avg_height);
    }

    // Tombstone stats
    println!("Total Tombstones: {}", stats.total_tombstones);
    println!("Tombstone Ratio: {:.2}%", stats.tombstone_ratio * 100.0);
    println!("Worst SSTable Density: {:.2}% (level {})",
        stats.max_sst_density * 100.0, stats.max_sst_density_level);
    for (i, count) in stats.level_tombstone_counts.iter().enumerate() {
        println!("Level {} tombstones: {}", i + 1, count);
    }

    // Write-amplification counters (tidesdb >= 9.3.4)
    let write_bytes = stats.wal_bytes_written

```

```
        + stats.flush_bytes_written
        + stats.compaction_bytes_written;
if stats.user_bytes_written > 0 {
    println!("Write amplification: {:.2}x",
        write_bytes as f64 / stats.user_bytes_written as f64);
}
println!("Flush count: {}, Compaction count: {}",
    stats.flush_count, stats.compaction_count);

Ok(())
}
```

Statistics Fields

Field	Type	Description
<code>num_levels</code>	<code>i32</code>	Number of LSM levels
<code>memtable_size</code>	<code>usize</code>	Current memtable size in bytes
<code>level_sizes</code>	<code>Vec<usize></code>	Array of per-level total sizes
<code>level_num_sstables</code>	<code>Vec<i32></code>	Array of per-level SSTable counts
<code>level_key_counts</code>	<code>Vec<u64></code>	Array of per-level key counts
<code>config</code>	<code>Option<ColumnFamilyConfig></code>	Column family configuration
<code>total_keys</code>	<code>u64</code>	Total keys across memtable and all SSTables
<code>total_data_size</code>	<code>u64</code>	Total data size (klog + vlog) in bytes
<code>avg_key_size</code>	<code>f64</code>	Estimated average key size in bytes
<code>avg_value_size</code>	<code>f64</code>	Estimated average value size in bytes
<code>read_amp</code>	<code>f64</code>	Read amplification factor (point lookup cost)
<code>hit_rate</code>	<code>f64</code>	Block cache hit rate (0.0 to 1.0)
<code>use_btree</code>	<code>bool</code>	Whether column family uses B+tree KLog format
<code>btree_total_nodes</code>	<code>u64</code>	Total B+tree nodes across all SSTables
<code>btree_max_height</code>	<code>u32</code>	Maximum tree height across all SSTables
<code>btree_avg_height</code>	<code>f64</code>	Average tree height across all SSTables
<code>total_tombstones</code>	<code>u64</code>	Sum of <code>tombstone_count</code> across every SSTable in the CF
<code>tombstone_ratio</code>	<code>f64</code>	<code>total_tombstones / total_keys</code> (0.0 if <code>total_keys == 0</code>)
<code>level_tombstone_counts</code>	<code>Vec<u64></code>	Per-level tombstone counts (parallels <code>level_key_counts</code>)
<code>max_sst_density</code>	<code>f64</code>	Worst per-SSTable tombstone density observed in the CF

Field	Type	Description
<code>max_sst_density_level</code>	i32	1-based level where <code>max_sst_density</code> was observed (0 if none)
<code>wal_bytes_written</code>	u64	Framed bytes appended to this CF's WAL (0 in unified mode)
<code>flush_bytes_written</code>	u64	On-disk bytes this CF's flushes wrote to L0 SSTables
<code>compaction_bytes_written</code>	u64	On-disk bytes this CF's compactions wrote
<code>compaction_bytes_read</code>	u64	On-disk bytes this CF's compactions read as input
<code>user_bytes_written</code>	u64	Logical key+value bytes committed to this CF (write-amplification denominator)
<code>flush_count</code>	u64	Flushed SSTables produced by this CF
<code>compaction_count</code>	u64	Compaction output SSTables produced by this CF

Write-amplification counters

The `wal_bytes_written`, `flush_bytes_written`, `compaction_bytes_written`, `compaction_bytes_read`, `user_bytes_written`, `flush_count`, and `compaction_count` fields require tidesdb >= 9.3.4; the default `v9_3_6` build includes them. On older libraries (e.g. an older system library found via pkg-config) they are reported as 0. This CF's write amplification is
$$\frac{(\text{wal_bytes_written} + \text{flush_bytes_written} + \text{compaction_bytes_written})}{\text{user_bytes_written}}$$
. In unified-memtable mode `wal_bytes_written` is 0 here — the shared WAL volume is reported db-wide as `DbStats::uwal_bytes_written`.

Getting Cache Statistics

Retrieve statistics about the global block cache.

```

use tidesdb::{TidesDB, Config};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;

    let stats = db.get_cache_stats()?;

    if stats.enabled {
        println!("Cache enabled: yes");
        println!("Total entries: {}", stats.total_entries);
        println!("Total bytes: {:.2} MB", stats.total_bytes as f64 /
(1024.0 * 1024.0));
        println!("Hits: {}", stats.hits);
        println!("Misses: {}", stats.misses);
        println!("Hit rate: {:.1}%", stats.hit_rate * 100.0);
        println!("Partitions: {}", stats.num_partitions);
    } else {
        println!("Cache enabled: no");
    }

    Ok(())
}

```

Getting Database-Level Statistics

Get aggregate statistics across the entire database instance.


```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let stats = db.get_db_stats()?;

    println!("Column families: {}", stats.num_column_families);
    println!("Total memory: {} bytes", stats.total_memory);
    println!("Resolved memory limit: {} bytes",
stats.resolved_memory_limit);
    println!("Memory pressure level: {}", stats.memory_pressure_level);
    println!("Global sequence: {}", stats.global_seq);
    println!("Flush queue: {} pending", stats.flush_queue_size);
    println!("Compaction queue: {} pending", stats.compaction_queue_size);
    println!("Total SSTables: {}", stats.total_sstable_count);
    println!("Total data size: {} bytes", stats.total_data_size_bytes);
    println!("Open SSTable handles: {}", stats.num_open_sstables);
    println!("In-flight txn memory: {} bytes", stats.txn_memory_bytes);
    println!("Immutable memtables: {}", stats.total_immutable_count);
    println!("Memtable bytes: {}", stats.total_memtable_bytes);

    // Db-wide write-amplification counters (tidesdb >= 9.3.4)
    let write_bytes = stats.uwal_bytes_written
        + stats.wal_bytes_written
        + stats.flush_bytes_written
        + stats.compaction_bytes_written;
    if stats.user_bytes_written > 0 {
        println!("Write amplification (db-wide): {:.2}x",
            write_bytes as f64 / stats.user_bytes_written as f64);
    }

    Ok(())
}

```

Database statistics include

Field	Type	Description
num_column_families	i32	Number of column families
total_memory	u64	System total memory
available_memory	u64	System available memory at open time
resolved_memory_limit	usize	Resolved memory limit (auto or configured)
memory_pressure_level	i32	Current memory pressure (0=normal, 1=elevated, 2=high, 3=critical)
flush_pending_count	i32	Number of pending flush operations (queued + in-flight)
total_memtable_bytes	i64	Total bytes in active memtables across all CFs
total_immutable_count	i32	Total immutable memtables across all CFs
total_sstable_count	i32	Total SSTables across all CFs and levels
total_data_size_bytes	u64	Total data size (klog + vlog) across all CFs
num_open_sstables	i32	Number of currently open SSTable file handles
global_seq	u64	Current global sequence number
txn_memory_bytes	i64	Bytes held by in-flight transactions
compaction_queue_size	usize	Number of pending compaction tasks
flush_queue_size	usize	Number of pending flush tasks in queue
unified_memtable_enabled	bool	Whether unified memtable mode is active
unified_memtable_bytes	i64	Bytes in unified active memtable
unified_immutable_count	i32	Number of unified immutable memtables

Field	Type	Description
<code>unified_is_flushing</code>	<code>bool</code>	Whether unified memtable is currently flushing/rotating
<code>unified_next_cf_index</code>	<code>u32</code>	Next CF index to be assigned in unified mode
<code>unified_wal_generation</code>	<code>u64</code>	Current unified WAL generation counter
<code>object_store_enabled</code>	<code>bool</code>	Whether object store mode is active
<code>object_store_connector</code>	<code>String</code>	Connector name (“s3”, “gcs”, “fs”, etc.)
<code>local_cache_bytes_used</code>	<code>usize</code>	Current local file cache usage in bytes
<code>local_cache_bytes_max</code>	<code>usize</code>	Configured maximum local cache size in bytes
<code>local_cache_num_files</code>	<code>i32</code>	Number of files tracked in local cache
<code>last_uploaded_generation</code>	<code>u64</code>	Highest WAL generation confirmed uploaded
<code>upload_queue_depth</code>	<code>usize</code>	Number of pending upload jobs in the queue
<code>total_uploads</code>	<code>u64</code>	Lifetime count of objects uploaded to object store
<code>total_upload_failures</code>	<code>u64</code>	Lifetime count of permanently failed uploads
<code>replica_mode</code>	<code>bool</code>	Whether running in read-only replica mode
<code>primary_epoch</code>	<code>u64</code>	Single-writer fencing: lease epoch this primary currently holds (0 when not a primary / no lease)
<code>seen_epoch</code>	<code>u64</code>	Single-writer fencing: highest lease epoch a replica has observed
<code>uwal_bytes_written</code>	<code>u64</code>	Framed bytes appended to the shared unified WAL (0 if unified mode off)
<code>wal_bytes_written</code>	<code>u64</code>	Per-CF WAL bytes summed across all column families
<code>flush_bytes_written</code>	<code>u64</code>	Flush output bytes summed across all column families

Field	Type	Description
<code>compaction_bytes_written</code>	u64	Compaction output bytes summed across all column families
<code>compaction_bytes_read</code>	u64	Compaction input bytes summed across all column families
<code>user_bytes_written</code>	u64	Logical committed bytes summed across all column families
<code>flush_count</code>	u64	Flushed SSTables summed across all column families
<code>compaction_count</code>	u64	Compaction output SSTables summed across all column families

Write-amplification counters

The eight write-amplification fields (`uwal_bytes_written` through `compaction_count`) require `tidesdb >= 9.3.4` (enabled by default) and are reported as `0` on older libraries. Db-wide write amplification is $(\text{uwal_bytes_written} + \text{wal_bytes_written} + \text{flush_bytes_written} + \text{compaction_bytes_written}) / \text{user_bytes_written}$. The `*_count` fields count output SSTables, not logical runs.

Single-writer fencing

The `primary_epoch` and `seen_epoch` fields require `tidesdb >= 9.3.8` and are reported as `0` on older libraries. They expose object-store single-writer fencing: a promotion that took bumps `primary_epoch`, while a fenced primary sees `replica_mode` flip back to `true`. Both are `0` on a local (non-object-store) database.

Stack Allocated

Unlike `ColumnFamily::get_stats` (which heap-allocates), `get_db_stats` fills a struct on the stack. No manual free is needed.

Range Cost Estimation

Estimate the computational cost of iterating between two keys in a column family. The returned value is an opaque double - meaningful only for comparison with other `range_cost` results. It uses only in-memory metadata and performs no disk I/O.

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let cost_a = cf.range_cost(b"user:0000", b"user:0999")?;
    let cost_b = cf.range_cost(b"user:1000", b"user:1099")?;

    if cost_a < cost_b {
        println!("Range A is cheaper to iterate");
    }

    Ok(())
}

```

Behavior

- Key order does not matter - the function normalizes the range so `key_a > key_b` produces the same result as `key_b > key_a`
- A cost of 0.0 means no overlapping SSTables or memtable entries were found for the range
- With block indexes enabled, uses $O(\log B)$ binary search per overlapping SSTable
- Without block indexes, falls back to byte-level key interpolation
- B+tree SSTables use key interpolation against tree node counts plus tree height as seek cost
- Compressed SSTables receive a $1.5\times$ weight multiplier for decompression overhead

Use cases

- Query planning · Compare candidate key ranges to find the cheapest one to scan
- Load balancing · Distribute range scan work across threads by estimating per-range cost
- Adaptive prefetching · Decide how aggressively to prefetch based on range size
- Monitoring · Track how data distribution changes across key ranges over time

Cost Values

The returned cost is not an absolute measure (it does not represent milliseconds, bytes, or entry counts). It is a relative scalar - only meaningful when compared with other `range_cost` results.

Listing Column Families

```
use tidesdb::{TidesDB, Config};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;

    let cf_list = db.list_column_families()?;

    println!("Available column families:");
    for name in cf_list {
        println!("  - {}", name);
    }

    Ok(())
}
```

Renaming Column Families

Atomically rename a column family and its underlying directory:

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("old_name", ColumnFamilyConfig::default())?;

    // Rename column family (waits for flush/compaction to complete)
    db.rename_column_family("old_name", "new_name")?;

    // Access with new name
    let cf = db.get_column_family("new_name")?;

    Ok(())
}
```

Cloning Column Families

Create a complete copy of an existing column family with a new name. The clone contains all the data from the source at the time of cloning.

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("source_cf", ColumnFamilyConfig::default())?;

    // Insert some data into source
    let cf = db.get_column_family("source_cf")?;
    let mut txn = db.begin_transaction()?;
    txn.put(&cf, b"key1", b"value1", -1)?;
    txn.put(&cf, b"key2", b"value2", -1)?;
    txn.commit()?;

    // Clone the column family
    db.clone_column_family("source_cf", "cloned_cf")?;

    // Both column families now exist independently
    let cloned_cf = db.get_column_family("cloned_cf")?;
    let txn = db.begin_transaction()?;
    let value = txn.get(&cloned_cf, b"key1")?;
    println!("Cloned value: {:?}", String::from_utf8_lossy(&value));

    Ok(())
}

```

Behavior

- Flushes the source column family's memtable to ensure all data is on disk
- Waits for any in-progress flush or compaction to complete
- Copies all SSTable files to the new directory
- The clone is completely independent — modifications to one do not affect the other

Use cases

- Testing · Create a copy of production data for testing without affecting the original
- Branching · Create a snapshot of data before making experimental changes
- Migration · Clone data before schema or configuration changes

Return values

- `Ok(())` · Clone completed successfully
- `ErrorCode::NotFound` · Source column family doesn't exist

- `ErrorCode::Exists` · Destination column family already exists
- `ErrorCode::InvalidArgs` · Invalid arguments (same source/destination name)
- `ErrorCode::Io` · Failed to copy files or create directory

Compaction

Manual Compaction

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    // Manually trigger compaction (queues compaction from L1+)
    cf.compact()?;

    Ok(())
}
```

Targeted Range Compaction

`ColumnFamily::compact_range` runs a synchronous compaction over a specific key range. Only SSTables whose minimum and maximum keys overlap the requested range participate in the merge, so the work and I/O are bounded to the affected portion of the LSM tree rather than the whole column family.


```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;
    let cf = db.get_column_family("my_cf")?;

    // Compact only SSTables whose key range overlaps [b"user:000",
    b"user:100").
    cf.compact_range(Some(b"user:000"), Some(b"user:100"))?;

    // Pass `None` for an unbounded endpoint.
    cf.compact_range(None, Some(b"user:050"))?;

    Ok(())
}

```

When to use

- After a bulk delete or backfill scoped to a known key range, to reclaim space without scheduling a full-CF compaction.
- To pre-warm or reshape a hot range before an expected read burst.
- To bound the worst-case I/O of an opportunistic maintenance pass; the merge is restricted to overlapping SSTables.

Behavior

- Synchronous: blocks the calling thread until the merge commits or fails. It does *not* enqueue work onto the compaction thread pool, so it is safe to call from a maintenance worker without contending with background compactions.
- Endpoints are interpreted as `[start_key, end_key)`. `None` means unbounded on that side. Empty slices (`Some(&[])`) are treated as `None`.
- Both endpoints `None` (or both empty) is rejected with `ErrorCode::InvalidArgs` — callers wanting to compact the whole column family must use `ColumnFamily::compact` instead.
- Concurrent calls return `ErrorCode::Locked` if another compaction is already running on the column family.

Return values

- `Ok(())` — the targeted range was compacted.
- `Err(ErrorCode::InvalidArgs)` — both endpoints were `None`/empty.

- `Err(ErrorCode::Locked)` — another compaction is already running.
- Standard I/O / memory error codes for failures during the merge.

Manual Memtable Flush

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    // Manually trigger memtable flush (Queues sorted run for L1)
    cf.flush_memtable()?;

    Ok(())
}
```

Checking Flush/Compaction Status

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    // Check if operations are in progress
    if cf.is_flushing() {
        println!("Flush operation in progress");
    }

    if cf.is_compacting() {
        println!("Compaction operation in progress");
    }

    Ok(())
}
```

Purge Column Family

`purge` forces a synchronous flush and aggressive compaction for a single column family. Unlike `flush_memtable` and `compact` (which are non-blocking), `purge` blocks until all flush and compaction I/O is complete.

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    // Force synchronous flush + compaction
    cf.purge()?;
    // All data is now flushed to SSTables and compacted

    Ok(())
}
```

Behavior

1. Waits for any in-progress flush to complete
2. Force-flushes the active memtable (even if below threshold)
3. Waits for flush I/O to fully complete
4. Waits for any in-progress compaction to complete
5. Triggers synchronous compaction inline (bypasses the compaction queue)
6. Waits for any queued compaction to drain

When to use

- Before backup or checkpoint — ensure all data is on disk and compacted
- After bulk deletes — reclaim space immediately by compacting away tombstones
- Manual maintenance — force a clean state during a maintenance window
- Pre-shutdown — ensure all pending work is complete before closing

Purge Database

`purge` on the database forces a synchronous flush and aggressive compaction for **all** column families, then drains both the global flush and compaction queues.

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("cf_a", ColumnFamilyConfig::default())?;
    db.create_column_family("cf_b", ColumnFamilyConfig::default())?;

    // Purge all column families
    db.purge()?;
    // All CFs flushed and compacted, all queues drained

    Ok(())
}

```

Purge vs Manual Flush + Compact

`flush_memtable` and `compact` are non-blocking — they enqueue work and return immediately. `ColumnFamily::purge` and `TidesDB::purge` are synchronous — they block until all work is complete. Use purge when you need a guarantee that all data is on disk and compacted before proceeding.

Manual WAL Sync

`sync_wal` forces an immediate fsync of the active write-ahead log for a column family. This is useful for explicit durability control when using `SyncMode::None` or `SyncMode::Interval` modes.

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig, SyncMode};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;

    let cf_config = ColumnFamilyConfig::new()
        .sync_mode(SyncMode::None);
    db.create_column_family("my_cf", cf_config)?;

    let cf = db.get_column_family("my_cf")?;

    // Write some data
    let mut txn = db.begin_transaction()?;
    txn.put(&cf, b"key1", b"value1", -1)?;
    txn.put(&cf, b"key2", b"value2", -1)?;
    txn.commit()?;

    // Force WAL durability after the batch
    cf.sync_wal()?;

    Ok(())
}

```

When to use

- Application-controlled durability — sync the WAL at specific points (e.g., after a batch of related writes) when using `SyncMode::None` or `SyncMode::Interval`
- Pre-checkpoint — ensure all buffered WAL data is on disk before taking a checkpoint
- Graceful shutdown — flush WAL buffers before closing the database
- Critical writes — force durability for specific high-value writes without using `SyncMode::Full` for all writes

Behavior

- Acquires a reference to the active memtable to safely access its WAL
- Calls `fdatasync` on the WAL file descriptor
- Thread-safe — can be called concurrently from multiple threads
- If the memtable rotates during the call, retries with the new active memtable

Structural Operations

Regardless of sync mode, TidesDB **always** enforces durability for structural operations:

- Memtable flush to SSTable
- SSTable compaction and merging
- WAL rotation
- Column family metadata updates

This ensures the database structure remains consistent even if user data syncing is delayed.

Database Backup

```
use tidesdb::{TidesDB, Config};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;

    // Create a backup to the specified directory
    db.backup("./mydb_backup")?;

    Ok(())
}
```

Database Checkpoint

`checkpoint` creates a lightweight, near-instant snapshot of an open database using hard links instead of copying SSTable data.

```
use tidesdb::{TidesDB, Config};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;

    // Create a checkpoint to the specified directory
    db.checkpoint("./mydb_checkpoint")?;

    Ok(())
}
```

Behavior

- Requires `checkpoint_dir` to be a non-existent or empty directory
- For each column family:
 - Flushes the active memtable so all data is in SSTables
 - Halts compactions to ensure a consistent view of live SSTable files
 - Hard links all SSTable files (`.klog` and `.vlog`) into the checkpoint directory
 - Copies small metadata files (manifest, config) into the checkpoint directory
 - Resumes compactions
- Falls back to file copy if hard linking fails (e.g., cross-filesystem)
- Database stays open and usable during checkpoint

Checkpoint vs Backup

	<code>backup</code>	<code>checkpoint</code>
Speed	Copies every SSTable byte-by-byte	Near-instant (hard links, O(1) per file)
Disk usage	Full independent copy	No extra disk until compaction removes old SSTables
Portability	Can be moved to another filesystem or machine	Same filesystem only (hard link requirement)
Use case	Archival, disaster recovery, remote shipping	Fast local snapshots, point-in-time reads, streaming backups

Notes

- The checkpoint represents the database state at the point all memtables are flushed and compactions are halted
- Hard-linked files share storage with the live database. Deleting the original database does not affect the checkpoint (hard link semantics)
- The checkpoint can be opened as a normal TidesDB database with `TidesDB::open`

Promote Replica to Primary

`promote_to_primary` switches a read-only replica to primary mode. This is only valid when the database was opened in replica mode (via object store configuration with `replica_mode` enabled).

```

use tidesdb::{TidesDB, Config};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;

    // Promote from replica to primary mode
    db.promote_to_primary()?;

    Ok(())
}

```

Return values

- `Ok(())` — Promotion completed successfully
- `ErrorCode::InvalidArgs` — Database is not in replica mode

Runtime Configuration Updates

Update column family configuration at runtime:

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig,
CompressionAlgorithm};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    // Create new configuration
    let new_config = ColumnFamilyConfig::new()
        .write_buffer_size(256 * 1024 * 1024)
        .compression_algorithm(CompressionAlgorithm::Zstd);

    // Update runtime config (persist_to_disk = true to save changes)
    cf.update_runtime_config(&new_config, true)?;

    Ok(())
}

```


Commit Hook (Change Data Capture)

`ColumnFamily::set_commit_hook` registers a callback that fires synchronously after every transaction commit on a column family. The hook receives the full batch of committed operations atomically, enabling real-time change data capture without WAL parsing or external log consumers.

```
use std::sync::{Arc, Mutex};
use tidesdb::{TidesDB, Config, ColumnFamilyConfig, CommitOp};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let mut cf = db.get_column_family("my_cf")?;

    // Track all committed operations
    let log: Arc<Mutex<Vec<CommitOp>>> = Arc::new(Mutex::new(Vec::new()));
    let log_clone = log.clone();

    cf.set_commit_hook(move |ops, commit_seq| {
        println!("Commit seq {}: {} ops", commit_seq, ops.len());
        let mut l = log_clone.lock().unwrap();
        for op in ops {
            l.push(op.clone());
        }
        0 // return 0 on success
    })?;

    // Normal writes now trigger the hook automatically
    let mut txn = db.begin_transaction()?;
    txn.put(&cf, b"key1", b"value1", -1)?;
    txn.put(&cf, b"key2", b"value2", -1)?;
    txn.commit()?; // hook fires here

    // Detach hook
    cf.clear_commit_hook()?;

    Ok(())
}
```

`CommitOp` fields

Field	Type	Description
<code>key</code>	<code>Vec<u8></code>	The key
<code>value</code>	<code>Option<Vec<u8>></code>	The value (<code>None</code> for deletes)
<code>ttl</code>	<code>i64</code>	TTL as Unix timestamp (0 = no expiry)
<code>is_delete</code>	<code>bool</code>	Whether this is a delete operation

Behavior

- The hook fires after WAL write, memtable apply, and commit status marking are complete - the data is fully durable before the callback runs
- Hook failure (non-zero return) is logged but does not affect the commit result
- Each column family has its own independent hook; a multi-CF transaction fires the hook once per CF with only that CF's operations
- `commit_seq` is monotonically increasing across commits and can be used as a replication cursor
- The hook executes synchronously on the committing thread; keep the callback fast to avoid stalling writers
- Calling `set_commit_hook` again replaces the previous hook (the old callback is freed automatically)
- Calling `clear_commit_hook` or dropping the `ColumnFamily` disables the hook immediately

Use cases

- Replication · Ship committed batches to replicas in commit order
- Event streaming · Publish mutations to Kafka, NATS, or any message broker
- Secondary indexing · Maintain a reverse index or materialized view
- Audit logging · Record every mutation with key, value, TTL, and sequence number
- Debugging · Attach a temporary hook in production to inspect live writes

Runtime-Only

Commit hooks are not persisted to `config.ini`. After a database restart, hooks must be re-registered by the application. This is by design - closures cannot be serialized.

INI Configuration Files

Load and save column family configurations from/to INI files:

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    // Load configuration from INI file
    let cf_config = ColumnFamilyConfig::load_from_ini("config.ini",
"my_column_family"?;

    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", cf_config.clone())?;

    // Save configuration to INI file
    cf_config.save_to_ini("config_backup.ini", "my_column_family"?;

    Ok(())
}
```

Sync Modes

Control the durability vs performance tradeoff.

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig, SyncMode};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;

    // SyncMode::None -- Fastest, least durable (OS handles flushing)
    let cf_config = ColumnFamilyConfig::new()
        .sync_mode(SyncMode::None);
    db.create_column_family("fast_cf", cf_config)?;

    // SyncMode::Interval -- Balanced (periodic background syncing)
    let cf_config = ColumnFamilyConfig::new()
        .sync_mode(SyncMode::Interval)
        .sync_interval_us(128000); // Sync every 128ms
    db.create_column_family("balanced_cf", cf_config)?;

    // SyncMode::Full -- Most durable (fsync on every write)
    let cf_config = ColumnFamilyConfig::new()
        .sync_mode(SyncMode::Full);
    db.create_column_family("durable_cf", cf_config)?;

    Ok(())
}

```

Compression Algorithms

TidesDB supports multiple compression algorithms:

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig,
CompressionAlgorithm};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;

    // No compression
    let cf_config = ColumnFamilyConfig::new()
        .compression_algorithm(CompressionAlgorithm::None);

    // LZ4 compression (default, balanced)
    let cf_config = ColumnFamilyConfig::new()
        .compression_algorithm(CompressionAlgorithm::Lz4);

    // LZ4 fast compression (faster, slightly lower ratio)
    let cf_config = ColumnFamilyConfig::new()
        .compression_algorithm(CompressionAlgorithm::Lz4Fast);

    // Zstandard compression (best ratio)
    let cf_config = ColumnFamilyConfig::new()
        .compression_algorithm(CompressionAlgorithm::Zstd);

    // Snappy compression
    let cf_config = ColumnFamilyConfig::new()
        .compression_algorithm(CompressionAlgorithm::Snappy);

    db.create_column_family("my_cf", cf_config)?;

    Ok(())
}

```

B+tree KLog Format

Column families can optionally use a B+tree structure for the key log instead of the default block-based format. The B+tree klog format offers faster point lookups through $O(\log N)$ tree traversal.

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig,
CompressionAlgorithm};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;

    // Create column family with B+tree klog format
    let cf_config = ColumnFamilyConfig::new()
        .use_btree(true)
        .compression_algorithm(CompressionAlgorithm::Lz4);

    db.create_column_family("btree_cf", cf_config)?;

    // Create column family with block-based format (default)
    let cf_config = ColumnFamilyConfig::new()
        .use_btree(false);

    db.create_column_family("block_cf", cf_config)?;

    Ok(())
}

```

B+tree Characteristics

- Point lookups · $O(\log N)$ tree traversal with binary search at each node
- Range scans · Doubly-linked leaf nodes enable efficient bidirectional iteration
- Immutable · Tree is bulk-loaded from sorted memtable data during flush
- Compression · Nodes compress independently using the same algorithms

When to use B+tree klog format

- Read-heavy workloads with frequent point lookups
- Workloads where read latency is more important than write throughput
- Large SSTables where block scanning becomes expensive

Tradeoffs

- Slightly higher write amplification during flush (building tree structure)
- Larger metadata overhead per node compared to block-based format
- Block-based format may be faster for sequential scans of entire SSTables

 Note

`use_btree` **cannot be changed** after column family creation. Different column families can use different formats.

Unified Memtable Mode

When enabled, all column families share a single memtable and WAL instead of having independent per-CF memtables. This can reduce memory overhead and write amplification when using many column families with small write volumes.

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig, SyncMode,
LogLevel};

fn main() -> tidesdb::Result<()> {
    let config = Config::new("./mydb")
        .num_flush_threads(2)
        .num_compaction_threads(2)
        .log_level(LogLevel::Info)
        .unified_memtable(true)
        .unified_memtable_write_buffer_size(128 * 1024 * 1024)
        .unified_memtable_skip_list_max_level(16)
        .unified_memtable_skip_list_probability(0.25)
        .unified_memtable_sync_mode(SyncMode::None)
        .unified_memtable_sync_interval_us(0);

    let db = TidesDB::open(config)?;

    // Column families work the same way
    db.create_column_family("cf_a", ColumnFamilyConfig::default()?);
    db.create_column_family("cf_b", ColumnFamilyConfig::default()?);

    let cf_a = db.get_column_family("cf_a")?;
    let cf_b = db.get_column_family("cf_b")?;

    let mut txn = db.begin_transaction()?;
    txn.put(&cf_a, b"key1", b"value1", -1)?;
    txn.put(&cf_b, b"key2", b"value2", -1)?;
    txn.commit()?;

    // Check unified memtable status via db stats
    let stats = db.get_db_stats()?;
    println!("Unified memtable enabled: {}",
stats.unified_memtable_enabled);
    println!("Unified memtable bytes: {}", stats.unified_memtable_bytes);

    Ok(())
}

```

When to use

- Many column families with small individual write volumes
- Reducing total memory overhead from per-CF memtable allocation
- Simplifying WAL management across multiple column families

Configuration

- `unified_memtable_write_buffer_size` · Total write buffer threshold (0 = auto)
- `unified_memtable_skip_list_max_level` · Skip list max level (0 = default 12)
- `unified_memtable_skip_list_probability` · Skip list probability (0 = default 0.25)
- `unified_memtable_sync_mode` · WAL sync mode (default: `SyncMode::None`)
- `unified_memtable_sync_interval_us` · WAL sync interval in microseconds

Object Store Mode

Object store mode allows TidesDB to store SSTables in a remote object store (S3, MinIO, GCS, or any S3-compatible service) while using local disk as a cache. This separates compute from storage and enables cold start recovery from the remote store. Object store mode requires unified memtable mode and is automatically enforced when a connector is set.

Enabling Object Store Mode (Filesystem Connector)

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig, ObjectStoreConfig};

fn main() -> tidesdb::Result<()> {
    let config = Config::new("./mydb")
        .object_store_fs("/mnt/nfs/tidesdb-objects");

    let db = TidesDB::open(config)?;

    // use the database normally -- SSTables are uploaded after flush

    Ok(())
}
```

Enabling Object Store Mode (S3 Connector)

To back a database with an S3-compatible bucket (AWS S3, MinIO, etc.), enable the `objectstore` Cargo feature and use `Config::object_store_s3` with an `S3Config`. This requires a TidesDB C library built with `TIDESDB_WITH_S3=ON` (the `objectstore` feature builds it that way automatically) plus `libcurl` and `openssl`.

```
[dependencies]
tidesdb = { version = "0.11", features = ["objectstore"] }
```

```

use tidesdb::{TidesDB, Config, S3Config};

fn main() -> tidesdb::Result<()> {
    // AWS S3
    let s3 = S3Config::new(
        "s3.amazonaws.com", // endpoint
        "my-bucket",         // bucket
        "AKIA...",           // access key
        "secret...",         // secret key
    )
    .prefix("production/db1/")
    .region("us-east-1")
    .use_ssl(true);

    let config = Config::new("./mydb").object_store_s3(s3);
    let db = TidesDB::open(config)?;

    Ok(())
}

```

For MinIO (or any path-style, self-hosted endpoint), enable path-style URLs and, for development against a self-signed certificate, optionally skip TLS verification:

```

use tidesdb::{TidesDB, Config, S3Config};

fn main() -> tidesdb::Result<()> {
    let s3 = S3Config::new("minio.local:9000", "my-bucket", "minioadmin",
        "minioadmin")
        .use_ssl(false)
        .use_path_style(true);           // required for MinIO

    let config = Config::new("./mydb").object_store_s3(s3);
    let db = TidesDB::open(config)?;

    Ok(())
}

```

`object_store_s3` takes precedence over `object_store_fs` when both are set. The behavior knobs in `ObjectStoreConfig` (local cache, upload/download parallelism, WAL replication, replica mode) apply to S3 connectors exactly as they do to the filesystem connector.

S3Config builder reference

Method	Type	Default	Description
<code>S3Config::new(endpoint, bucket, access_key, secret_key)</code>	<code>&str</code>	—	Required connection fields
<code>prefix(p)</code>	<code>&str</code>	None	Key prefix (e.g. <code>"production/db1/"</code>)
<code>region(r)</code>	<code>&str</code>	None	AWS region; omit for MinIO
<code>use_ssl(enable)</code>	<code>bool</code>	false	<code>true</code> for HTTPS, <code>false</code> for HTTP
<code>use_path_style(enable)</code>	<code>bool</code>	false	<code>true</code> for path-style URLs (MinIO)
<code>tls_ca_path(path)</code>	<code>&str</code>	None	Custom CA bundle file path
<code>tls_insecure_skip_verify(enable)</code>	<code>bool</code>	false	Disable TLS verification (test only, insecure)
<code>multipart_threshold(size)</code>	<code>usize</code>	0 (library default)	Object size at/above which multipart upload is used
<code>multipart_part_size(size)</code>	<code>usize</code>	0 (library default)	Multipart chunk size in bytes

S3 build requirements

The S3 connector symbols are only present when the C library is built with `TIDESDB_WITH_S3=ON`. Without the `objectstore` feature, `S3Config` and `object_store_s3` are not compiled in. On Windows, S3 support requires `tidesdb` `>= v9.0.6`.

Custom Object Store Configuration

Use `ObjectStoreConfig::default()` for sensible defaults, then override fields as needed:

```
use tidesdb::{TidesDB, Config, ObjectStoreConfig};

fn main() -> tidesdb::Result<()> {
    let os_config = ObjectStoreConfig::new()
        .local_cache_max_bytes(512 * 1024 * 1024) // 512MB local cache
        .max_concurrent_uploads(8)
        .max_concurrent_downloads(16);

    let config = Config::new("./mydb")
        .object_store_fs("/mnt/nfs/tidesdb-objects")
        .object_store_config(os_config);

    let db = TidesDB::open(config)?;

    Ok(())
}
```

Object Store Configuration Reference

Method	Type	Default	Description
<code>local_cache_path(path)</code>	<code>&str</code>	None (uses <code>db_path</code>)	Local directory for cached SSTable files
<code>local_cache_max_bytes(size)</code>	<code>usize</code>	0 (unlimited)	Maximum local cache size in bytes
<code>cache_on_read(enable)</code>	<code>bool</code>	true	Cache downloaded files locally
<code>cache_on_write(enable)</code>	<code>bool</code>	true	Keep local copy after upload
<code>max_concurrent_uploads(n)</code>	<code>i32</code>	4	Number of parallel upload threads
<code>max_concurrent_downloads(n)</code>	<code>i32</code>	8	Number of parallel download threads
<code>multipart_threshold(size)</code>	<code>usize</code>	64MB	Use multipart upload above this size
<code>multipart_part_size(size)</code>	<code>usize</code>	8MB	Chunk size for multipart uploads
<code>sync_manifest_to_object(enable)</code>	<code>bool</code>	true	Upload MANIFEST after each compaction
<code>replicate_wal(enable)</code>	<code>bool</code>	true	Upload closed WAL segments for replication
<code>wal_upload_sync(enable)</code>	<code>bool</code>	false	false for background WAL upload, true to block flush
<code>wal_sync_threshold_bytes(size)</code>	<code>usize</code>	1MB	Sync active WAL when it grows by this many bytes (0 = off)
<code>wal_sync_on_commit(enable)</code>	<code>bool</code>	false	Upload WAL after every txn commit for RPO=0 replication
<code>replica_mode(enable)</code>	<code>bool</code>	false	Enable read-only replica mode (writes return <code>ErrorCode::ReadOnly</code>)
<code>replica_sync_interval_us(interval)</code>	<code>u64</code>	5000000 (5s)	MANIFEST poll interval for replica sync in microseconds

Method	Type	Default	Description
<code>replica_replay_wal(enable)</code>	<code>bool</code>	true	Replay WAL from object store for near-real-time reads on replicas

Per-CF Object Store Tuning

Column family configurations include three object store tuning fields (see Column Family Configuration Reference):

- `object_lazy_compaction` · Compact less aggressively for remote storage
- `object_prefetch_compaction` · Download all inputs before compaction merge

Object Store Statistics

`get_db_stats` includes object store fields when a connector is active:

```
use tidesdb::{TidesDB, Config, ObjectStoreConfig};

fn main() -> tidesdb::Result<()> {
    let config = Config::new("./mydb")
        .object_store_fs("/mnt/nfs/tidesdb-objects");

    let db = TidesDB::open(config)?;

    let stats = db.get_db_stats()?;
    if stats.object_store_enabled {
        println!("connector: {}", stats.object_store_connector);
        println!("total uploads: {}", stats.total_uploads);
        println!("upload failures: {}", stats.total_upload_failures);
        println!("upload queue depth: {}", stats.upload_queue_depth);
        println!("local cache: {} / {} bytes",
            stats.local_cache_bytes_used, stats.local_cache_bytes_max);
    }

    Ok(())
}
```

Replica Mode

Replica mode enables read-only nodes that follow a primary through the object store.

```

use tidesdb::{TidesDB, Config, ObjectStoreConfig};

fn main() -> tidesdb::Result<()> {
    let os_config = ObjectStoreConfig::new()
        .replica_mode(true)
        .replica_sync_interval_us(1_000_000)           // 1 second sync
interval
        .replica_replay_wal(true);                     // replay WAL for
fresh reads

    let config = Config::new("./mydb_replica")
        .object_store_fs("/mnt/nfs/tidesdb-objects") // same store as the
primary
        .object_store_config(os_config);

    let db = TidesDB::open(config)?;

    // reads work normally, writes return ErrorCode::ReadOnly

    // promote to primary when the original primary fails
    db.promote_to_primary()?;

    Ok(())
}

```

Sync-on-Commit WAL (Primary Side)

For tighter replication lag, enable sync-on-commit on the primary so every committed write is uploaded to the object store immediately:

```

use tidesdb::{TidesDB, Config, ObjectStoreConfig};

fn main() -> tidesdb::Result<()> {
    let os_config = ObjectStoreConfig::new()
        .wal_sync_on_commit(true); // RPO = 0, every commit is durable in
the store

    let config = Config::new("./mydb")
        .object_store_fs("/mnt/nfs/tidesdb-objects")
        .object_store_config(os_config);

    let db = TidesDB::open(config)?;

    // replica sees committed data within one replica_sync_interval_us

    Ok(())
}

```

Object Store Requirements

- Object store mode automatically enables unified memtable mode
- After each flush, SSTables are uploaded via an asynchronous upload pipeline with retry (3 attempts with exponential backoff)
- Point lookups on remote SSTables fetch just the needed block (~64KB) via range request
- Iterators prefetch all needed SSTable files in parallel at creation time
- The MANIFEST is uploaded after each flush/compaction for cold start recovery

Custom Comparators

A comparator defines the sort order of keys throughout the entire system, memtables, SSTables, block indexes, and iterators. TidesDB ships with six built-in comparators and supports registering custom ones.

Built-in comparators `memcmp` (default), `lexicographic`, `uint64`, `int64`, `reverse`, `case_insensitive`

Registering a Custom Comparator

Register a custom comparator **after** opening the database but **before** creating any column family that uses it. Once a comparator is set for a column family, it **cannot be changed** without corrupting data.


```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;

    // Register a reverse byte comparator
    db.register_comparator("my_reverse", |key1, key2| {
        let min_len = key1.len().min(key2.len());
        for i in 0..min_len {
            if key1[i] != key2[i] {
                return key2[i] as i32 - key1[i] as i32;
            }
        }
        key2.len() as i32 - key1.len() as i32
    })?;

    // Use the custom comparator in a column family
    let cf_config = ColumnFamilyConfig::new()
        .comparator_name("my_reverse");
    db.create_column_family("reverse_cf", cf_config)?;

    Ok(())
}

```

The comparator function receives two key byte slices and must return:

- `< 0` if `key1 < key2`
- `0` if `key1 == key2`
- `> 0` if `key1 > key2`

Checking if a Comparator Exists

```

if db.has_comparator("my_reverse") {
    println!("Comparator is registered");
}

// Built-in comparators are always available
assert!(db.has_comparator("memcmp"));
assert!(db.has_comparator("reverse"));

```

Comparator Permanence

Once a column family is created with a specific comparator, it **must always** be opened with the same comparator. Changing or removing a comparator after column family creation will corrupt the data ordering.

Utility Functions

`tidesdb::free`

Frees memory allocated by TidesDB. This is primarily useful for advanced FFI scenarios. For normal Rust usage, the safe wrappers handle memory management automatically.

```
// Safety: ptr must have been allocated by TidesDB
unsafe { tidesdb::free(ptr); }
```

Database Configuration Reference

All available `Config` builder methods:

Method	Type	Default	Description
<code>num_flush_threads(n)</code>	<code>i32</code>	2	Number of background flush threads
<code>num_compaction_threads(n)</code>	<code>i32</code>	2	Number of background compaction threads
<code>log_level(level)</code>	<code>LogLevel</code>	Info	Minimum log level (<code>Debug</code> , <code>Info</code> , <code>Warn</code> , <code>Error</code> , <code>Fatal</code> , <code>None</code>)
<code>block_cache_size(size)</code>	<code>usize</code>	64MB	Global block cache size in bytes
<code>max_open_sstables(n)</code>	<code>usize</code>	256	Maximum number of open SSTable file descriptors
<code>max_memory_usage(size)</code>	<code>usize</code>	0	Global memory limit in bytes (0 = auto, 50% of system RAM)
<code>log_to_file(enable)</code>	<code>bool</code>	false	Write logs to file instead of stderr
<code>log_truncation_at(size)</code>	<code>usize</code>	24MB	Log file truncation threshold in bytes (0 = no truncation)
<code>unified_memtable(enable)</code>	<code>bool</code>	false	Enable unified memtable mode (all CFs share one memtable/WAL)
<code>unified_memtable_write_buffer_size(size)</code>	<code>usize</code>	0	Unified memtable write buffer size (0 = auto)
<code>unified_memtable_skip_list_max_level(level)</code>	<code>i32</code>	0	Skip list max level for unified memtable (0 = default 12)
<code>unified_memtable_skip_list_probability(prob)</code>	<code>f32</code>	0.0	Skip list probability for unified memtable (0 = default 0.25)
<code>unified_memtable_sync_mode(mode)</code>	<code>SyncMode</code>	None	Sync mode for unified WAL
<code>unified_memtable_sync_interval_us(interval)</code>	<code>u64</code>	0	Sync interval for unified WAL in microseconds
<code>object_store_fs(root_dir)</code>	<code>&str</code>	None	Enable object store with filesystem connector at <code>root_dir</code>
<code>object_store_config(config)</code>	<code>ObjectStoreConfig</code>	None	Object store behavior configuration (see Object Store Configuration Reference)

Method	Type	Default	Description
<code>max_concurrent_flushes(n)</code>	<code>i32</code>	library default	Global cap on in-flight memtable flushes across all CFs (0 = library default)

Column Family Configuration Reference

All available `ColumnFamilyConfig` builder methods:

Method	Type	Default	Description
<code>write_buffer_size(size)</code>	<code>usize</code>	64MB	Memtable flush threshold in bytes
<code>level_size_ratio(ratio)</code>	<code>usize</code>	10	Level size multiplier
<code>min_levels(lev</code> <code>els)</code>	<code>i32</code>	5	Minimum LSM levels
<code>dividing_level_offset(offset)</code>	<code>i32</code>	2	Compaction dividing level offset
<code>klog_value_threshold(threshold)</code>	<code>usize</code>	512	Values > threshold go to vlog
<code>compression_algorithm(algo)</code>	<code>CompressionAlgorithm</code>	Lz4	Compression algorithm
<code>enable_bloom_filter(enable)</code>	<code>bool</code>	true	Enable bloom filters
<code>bloom_fpr(fpr)</code>	<code>f64</code>	0.01	Bloom filter false positive rate
<code>enable_block_indexes(enable)</code>	<code>bool</code>	true	Enable compact block indexes
<code>index_sample_ratio(ratio)</code>	<code>i32</code>	1	Sample every N blocks for index
<code>block_index_prefix_len(len)</code>	<code>i32</code>	16	Block index prefix length
<code>sync_mode(mode)</code>	<code>SyncMode</code>	Full	Durability mode
<code>sync_interval_us(interval)</code>	<code>u64</code>	128000	Sync interval (for Interval mode)
<code>comparator_name(name)</code>	<code>&str</code>	"memcmp"	Key comparator name
<code>skip_list_max_level(level)</code>	<code>i32</code>	12	Skip list max level
<code>skip_list_probability(prob)</code>	<code>f32</code>	0.25	Skip list probability

Method	Type	Default	Description
<code>default_isolation_level(level)</code>	<code>IsolationLevel</code>	ReadCommitted	Default transaction isolation
<code>min_disk_space(space)</code>	<code>u64</code>	100MB	Minimum disk space required
<code>l1_file_count_trigger(trigger)</code>	<code>i32</code>	4	L1 file count trigger for compaction
<code>l0_queue_stall_threshold(threshold)</code>	<code>i32</code>	20	L0 queue stall threshold
<code>tombstone_density_trigger(ratio)</code>	<code>f64</code>	0.0	Per-SSTable tombstone density above which compaction priority escalates (range <code>[0.0, 1.0]</code> ; <code>0.0</code> = disabled)
<code>tombstone_density_min_entries(n)</code>	<code>u64</code>	1024	Minimum entry count for an SSTable to be considered by the density trigger (prevents tiny-sstable noise)
<code>use_btree(enable)</code>	<code>bool</code>	false	Use B+tree format for klog
<code>object_lazy_compaction(enable)</code>	<code>bool</code>	false	Compact less aggressively in object store mode
<code>object_prefetch_compaction(enable)</code>	<code>bool</code>	true	Download all inputs before merge in object store mode

Updatable at runtime (via `update_runtime_config`):

- `write_buffer_size`, `skip_list_max_level`, `skip_list_probability`
- `bloom_fpr`, `enable_bloom_filter`, `enable_block_indexes`, `block_index_prefix_len`
- `index_sample_ratio`, `compression_algorithm`, `klog_value_threshold`
- `sync_mode`, `sync_interval_us`, `level_size_ratio`, `min_levels`
- `dividing_level_offset`, `l1_file_count_trigger`, `l0_queue_stall_threshold`
- `default_isolation_level`, `min_disk_space`

Non-updatable settings (cannot be changed after column family creation):

- `comparator_name` · Cannot change sort order after creation (would corrupt key ordering in existing SSTables)

- `use_btree` · Cannot change klog format after creation (existing SSTables use the original format)

Error Handling

TidesDB uses a custom `Result` type with detailed error information:

```
use tidesdb::{TidesDB, Config, ColumnFamilyConfig, Error, ErrorCode};

fn main() {
    let db = match TidesDB::open(Config::new("./mydb")) {
        Ok(db) => db,
        Err(e) => {
            eprintln!("Failed to open database: {}", e);
            return;
        }
    };

    db.create_column_family("my_cf",
ColumnFamilyConfig::default()).unwrap();
    let cf = db.get_column_family("my_cf").unwrap();

    let txn = db.begin_transaction().unwrap();

    match txn.get(&cf, b"nonexistent_key") {
        Ok(value) => println!("Value: {:?}", value),
        Err(Error::TidesDB { code, context }) => {
            match code {
                ErrorCode::NotFound => println!("Key not found"),
                ErrorCode::Memory => println!("Memory allocation failed"),
                ErrorCode::Io => println!("I/O error"),
                _ => println!("Error ({}): {}", code as i32, context),
            }
        }
        Err(e) => println!("Other error: {}", e),
    }
}
```

Error Codes

- `ErrorCode::Success` (0) · Operation successful
- `ErrorCode::Memory` (-1) · Memory allocation failed

- `ErrorCode::InvalidArgs` (-2) · Invalid arguments
- `ErrorCode::NotFound` (-3) · Key not found
- `ErrorCode::Io` (-4) · I/O error
- `ErrorCode::Corruption` (-5) · Data corruption
- `ErrorCode::Exists` (-6) · Resource already exists
- `ErrorCode::Conflict` (-7) · Transaction conflict
- `ErrorCode::TooLarge` (-8) · Key or value too large
- `ErrorCode::MemoryLimit` (-9) · Memory limit exceeded
- `ErrorCode::InvalidDb` (-10) · Invalid database handle
- `ErrorCode::Unknown` (-11) · Unknown error
- `ErrorCode::Locked` (-12) · Database is locked
- `ErrorCode::ReadOnly` (-13) · Database is in read-only mode
- `ErrorCode::Busy` (-14) · Resource is busy (another operation is in progress)
- `ErrorCode::Precondition` (-15) · A required precondition was not met

Complete Example

```
use std::time::{SystemTime, UNIX_EPOCH};
use tidesdb::{
    TidesDB, Config, ColumnFamilyConfig, CompressionAlgorithm,
    SyncMode, IsolationLevel, LogLevel,
};

fn main() -> tidesdb::Result<()> {
    let config = Config::new("./example_db")
        .num_flush_threads(1)
        .num_compaction_threads(1)
        .log_level(LogLevel::Info)
        .block_cache_size(64 * 1024 * 1024)
        .max_open_sstables(256)
        .max_memory_usage(0); // 0 = auto (50% of system RAM)

    let db = TidesDB::open(config)?;

    // Create column family with custom configuration
    let cf_config = ColumnFamilyConfig::new()
        .write_buffer_size(64 * 1024 * 1024)
        .compression_algorithm(CompressionAlgorithm::Lz4)
        .enable_bloom_filter(true)
        .bloom_fpr(0.01)
        .sync_mode(SyncMode::Interval)
        .sync_interval_us(128000);

    db.create_column_family("users", cf_config)?;

    let cf = db.get_column_family("users")?;

    {
        let mut txn = db.begin_transaction()?;

        txn.put(&cf, b"user:1", b"Alice", -1)?;
        txn.put(&cf, b"user:2", b"Bob", -1)?;

        // Write with TTL (expires in 30 seconds)
        let ttl = SystemTime::now()
            .duration_since(UNIX_EPOCH)
            .unwrap()
            .as_secs() as i64 + 30;
        txn.put(&cf, b"session:abc", b"temp_data", ttl)?;
    }
}
```

```

        txn.commit()?;
    }

    {
        let txn = db.begin_transaction()?;

        let value = txn.get(&cf, b"user:1"?;
        println!("user:1 = {}", String::from_utf8_lossy(&value));
    }

    {
        let txn = db.begin_transaction()?;
        let mut iter = txn.new_iterator(&cf)?;

        println!("\nAll entries:");
        iter.seek_to_first()?;
        while iter.is_valid() {
            let key = iter.key()?;
            let value = iter.value()?;
            println!("  {} = {}",
                String::from_utf8_lossy(&key),
                String::from_utf8_lossy(&value));
            iter.next()?;
        }
    }

    let stats = cf.get_stats()?;
    println!("\nColumn Family Statistics:");
    println!("  Number of Levels: {}", stats.num_levels);
    println!("  Memtable Size: {} bytes", stats.memtable_size);

    db.drop_column_family("users"?;

    Ok(())
}

```

Isolation Levels

TidesDB supports five MVCC isolation levels:

```
use tidesdb::{TidesDB, Config, IsolationLevel};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    let txn =
db.begin_transaction_with_isolation(IsolationLevel::ReadCommitted)?;

    // Use transaction...

    Ok(())
}
```

Available Isolation Levels

- `IsolationLevel::ReadUncommitted` · Sees all data including uncommitted changes
- `IsolationLevel::ReadCommitted` · Sees only committed data (default)
- `IsolationLevel::RepeatableRead` · Consistent snapshot, phantom reads possible
- `IsolationLevel::Snapshot` · Write-write conflict detection
- `IsolationLevel::Serializable` · Full read-write conflict detection (SSI)

Savepoints

Savepoints allow partial rollback within a transaction:

```

use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = TidesDB::open(Config::new("./mydb"))?;
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let mut txn = db.begin_transaction()?;

    txn.put(&cf, b"key1", b"value1", -1)?;

    txn.savepoint("sp1")?;

    txn.put(&cf, b"key2", b"value2", -1)?;

    // Rollback to savepoint -- key2 is discarded, key1 remains
    txn.rollback_to_savepoint("sp1")?;

    // Add different data after rollback
    txn.put(&cf, b"key3", b"value3", -1)?;

    // Commit -- only key1 and key3 are written
    txn.commit()?;

    Ok(())
}

```

Thread Safety

TidesDB is thread-safe. The `TidesDB` and `ColumnFamily` types implement `Send` and `Sync`, allowing them to be shared across threads:

```

use std::sync::Arc;
use std::thread;
use tidesdb::{TidesDB, Config, ColumnFamilyConfig};

fn main() -> tidesdb::Result<()> {
    let db = Arc::new(TidesDB::open(Config::new("./mydb"))?);
    db.create_column_family("my_cf", ColumnFamilyConfig::default())?;

    let cf = db.get_column_family("my_cf")?;

    let handles: Vec<_> = (0..4).map(|i| {
        let db = Arc::clone(&db);
        let cf_name = "my_cf".to_string();

        thread::spawn(move || {
            let cf = db.get_column_family(&cf_name).unwrap();
            let mut txn = db.begin_transaction().unwrap();

            let key = format!("key:{}", i);
            let value = format!("value:{}", i);
            txn.put(&cf, key.as_bytes(), value.as_bytes(), -1).unwrap();
            txn.commit().unwrap();
        })
    }).collect();

    for handle in handles {
        handle.join().unwrap();
    }

    Ok(())
}

```

Testing



```
cargo test
```

```
# Run tests with output
```

```
cargo test -- --nocapture
```

```
# Run specific test
```

```
cargo test test_open_close
```

```
# Run with release optimizations
```

```
cargo test --release
```

```
# Run single-threaded (recommended for TidesDB tests)
```

```
cargo test -- --test-threads=1
```

TidesDB TypeScript API Reference

Getting Started

Prerequisites

You **must** have the TidesDB shared C library installed on your system. You can find the installation instructions [here](#).

Installation



```
npm install git+https://github.com/tidesdb/tidesdb-node.git
```



```
npm i tidesdb
```

Or clone and install locally:



```
git clone https://github.com/tidesdb/tidesdb-node.git
cd tidesdb-node
npm install
npm run build
npm link

# In your project directory
npm link tidesdb
```

Custom Installation Paths

If you installed TidesDB to a non-standard location, you can specify custom paths using environment variables:



```
# Linux
export LD_LIBRARY_PATH="/custom/path/lib:$LD_LIBRARY_PATH"

# macOS
export DYLD_LIBRARY_PATH="/custom/path/lib:$DYLD_LIBRARY_PATH"

# Windows (add to PATH)
set PATH=C:\custom\path\bin;%PATH%
```

Custom prefix installation



```
# Install TidesDB to custom location
cd tidesdb
cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/opt/tidesdb
cmake --build build
sudo cmake --install build

# Configure environment to use custom location
export LD_LIBRARY_PATH="/opt/tidesdb/lib:$LD_LIBRARY_PATH"      # Linux
# or
export DYLD_LIBRARY_PATH="/opt/tidesdb/lib:$DYLD_LIBRARY_PATH" # macOS

npm install tidesdb
```


Usage

Opening and Closing a Database

```
import { TidesDB, LogLevel, SyncMode } from 'tidesdb';

const db = TidesDB.open({
  dbPath: './mydb',
  numFlushThreads: 2,
  numCompactionThreads: 2,
  logLevel: LogLevel.Info,
  blockCacheSize: 64 * 1024 * 1024,
  maxOpenSSTables: 256,
  maxMemoryUsage: 0, // Global memory limit in bytes
  (0 = auto, 50% of system RAM)
  logToFile: false, // Write logs to file instead of
stderr
  logTruncationAt: 24 * 1024 * 1024, // Log file truncation size
(24MB), 0 = no truncation
  unifiedMemtable: false, // Enable unified memtable mode
  unifiedMemtableWriteBufferSize: 0, // Write buffer size (0 = auto)
  unifiedMemtableSkipListMaxLevel: 0, // Skip list max level (0 =
default 12)
  unifiedMemtableSkipListProbability: 0, // Skip list probability (0 =
default 0.25)
  unifiedMemtableSyncMode: SyncMode.None, // Sync mode for unified WAL
  unifiedMemtableSyncIntervalUs: 0, // Sync interval for unified WAL
(0 = default)
  maxConcurrentFlushes: 0, // Cap on in-flight memtable
flushes across all CFs (0 = library default)
  finishCompactionsOnClose: false, // false = fast shutdown (cancel
in-flight compactions); true = let them finish
  objectStoreFsPath: '/path/to/store', // Enable object store mode (FS
connector)
  objectStoreS3Config: { ... }, // Alternative connector: S3-
compatible store (see Object Store Mode)
  objectStoreConfig: { ... }, // Object store behavior config
(optional)
});

console.log('Database opened successfully');

db.close();
```

maxConcurrentFlushes

`maxConcurrentFlushes` is a global semaphore on the number of in-flight memtable flushes across all column families. It bounds total transient memory and work-queue depth when many column families flush at once. Leave it at `0` to inherit the library default (recommended), or lower it on memory-constrained hosts to throttle peak flush concurrency.

finishCompactionsOnClose

`finishCompactionsOnClose` controls shutdown behavior. The default `false` cancels in-flight compactions at their next checkpoint for a fast `close()` — the merge discards its uncommitted output and leaves inputs intact, so **no data is lost** (recovery handles a mid-merge state the same way). Set it to `true` to let in-flight compactions run to completion before `close()` returns, at the cost of a slower shutdown. For an even more aggressive fast shutdown, call `db.cancelBackgroundWork()` right before `db.close()`.

Default Configuration

`defaultConfig()` returns values sourced from the underlying C library, so settings such as `maxConcurrentFlushes` and the unified-memtable defaults track the engine automatically. If you previously relied on the binding's hardcoded defaults (`numFlushThreads: 2`, `numCompactionThreads: 2`, `blockCacheSize: 64MB`, `maxOpenSSTables: 256`), re-test after upgrading — the library may have evolved past those constants.

Library Initialization & Utilities

These static helpers on the `TidesDB` class operate at the library/process level rather than on a specific database handle.

```

import { TidesDB, CompressionAlgorithm } from 'tidesdb';

// Optional: explicitly initialize the library before any other call.
// The library auto-initializes on first use, so this is rarely needed.
// Returns true if this call performed the initialization, false if already
// initialized.
const didInit = TidesDB.init();

// Raise this process's open-file ceiling BEFORE opening a database so the
// engine
// can keep more SSTables open. Pass <= 0 to just read the current ceiling.
const ceiling = TidesDB.raiseOpenFileLimit(0);           // report current
ceiling
const raised = TidesDB.raiseOpenFileLimit(65536);        // attempt to raise
toward 65536

// Query whether a compression backend was compiled into the linked
// library.
if (TidesDB.isCompressionAvailable(CompressionAlgorithm.ZstdCompression)) {
  // safe to create a Zstd-compressed column family
}

// Advanced / teardown only: finalize the library and reset the allocator.
// Do NOT call while any database, transaction, iterator, or callback is
// still live.
TidesDB.finalize();

```

TidesDB.init()

- Optional. The library auto-initializes with the system allocator on first use.
- Custom memory allocators are not supported from TypeScript, so this always uses the system allocator.
- Returns `true` if this call performed the initialization, `false` if it was already initialized.

TidesDB.raiseOpenFileLimit(desired)

- Raises this process's open-file ceiling (POSIX `RLIMIT_NOFILE` soft limit toward the hard limit; on Windows, the CRT stdio cap up to 8192).
- Call **before** `TidesDB.open()` — the engine sizes `maxOpenSSTables` to fit the ceiling at open time. TidesDB never raises the limit itself; this is an explicit, opt-in operator action.
- `desired <= 0` just reports the current ceiling without changing it.
- Returns the open-file ceiling in effect after the attempt. A failed or partial raise is non-fatal.

`TidesDB.isCompressionAvailable(algorithm)`

- Every algorithm enum value is always defined, but whether a backend can actually be used is a build-time choice (the `-DTIDESDB_WITH_*` CMake options).
- Returns `true` if the backend is available at runtime. `CompressionAlgorithm.NoCompression` is always available.

`TidesDB.finalize()`

- Finalizes the library and resets the allocator; after this, `TidesDB.init()` can be called again.
- Advanced / teardown use only — calling it while any database, transaction, iterator, or registered callback is still live invalidates them.

Creating and Dropping Column Families

Column families are isolated key-value stores with independent configuration.

```

import {
  TidesDB,
  CompressionAlgorithm,
  SyncMode,
  IsolationLevel,
  defaultColumnFamilyConfig
} from 'tidesdb';

db.createColumnFamily('my_cf');

db.createColumnFamily('my_cf', {
  writeBufferSize: 128 * 1024 * 1024,
  levelSizeRatio: 10,
  minLevels: 5,
  compressionAlgorithm: CompressionAlgorithm.Lz4Compression,
  enableBloomFilter: true,
  bloomFpr: 0.01,
  enableBlockIndexes: true,
  syncMode: SyncMode.Interval,
  syncIntervalUs: 128000,
  defaultIsolationLevel: IsolationLevel.ReadCommitted,
  tombstoneDensityTrigger: 0.0,    // Per-SSTable tombstone density above
  // which compaction escalates (0.0 = disabled, range 0.0 to 1.0)
  tombstoneDensityMinEntries: 1024, // Minimum entry count for an SSTable
  // to be considered by the density trigger
  useBtree: false,                // Use B+tree format for klog (default:
  // false = block-based)
});

db.dropColumnFamily('my_cf');

// Delete by column family handle
const cfToDelete = db.getColumnFamily('my_cf');
db.deleteColumnFamily(cfToDelete);

db.renameColumnFamily('old_name', 'new_name');

db.cloneColumnFamily('source_cf', 'cloned_cf');

```

Column Family Configuration Reference

Every field on `ColumnFamilyConfig` is optional; omitted fields inherit the library default returned by `defaultColumnFamilyConfig()`. The example above sets only a common subset.

Field	Type	Description
<code>writeBufferSize</code>	<code>number</code>	Memtable size in bytes that triggers a flush
<code>levelSizeRatio</code>	<code>number</code>	Size ratio between adjacent LSM levels
<code>minLevels</code>	<code>number</code>	Minimum number of LSM levels
<code>dividingLevelOffset</code>	<code>number</code>	Offset for the dividing level
<code>klogValueThreshold</code>	<code>number</code>	Value size at/above which values are separated into the vlog
<code>compressionAlgorithm</code>	<code>CompressionAlgorithm</code>	Compression backend for this CF's SSTables
<code>enableBloomFilter</code>	<code>boolean</code>	Enable bloom filters on SSTables
<code>bloomFpr</code>	<code>number</code>	Bloom filter target false-positive rate
<code>enableBlockIndexes</code>	<code>boolean</code>	Enable per-block indexes
<code>indexSampleRatio</code>	<code>number</code>	Sample 1 in N keys when building sparse indexes
<code>blockIndexPrefixLength</code>	<code>number</code>	Block index key prefix length
<code>syncMode</code>	<code>SyncMode</code>	WAL durability mode
<code>syncIntervalUs</code>	<code>number</code>	Sync interval in microseconds (used by <code>SyncMode.Interval</code>)
<code>comparatorName</code>	<code>string</code>	Key comparator name (built-in or registered)
<code>skipListMaxLevel</code>	<code>number</code>	Skip list max level for new memtables
<code>skipListProbability</code>	<code>number</code>	Skip list level probability for new memtables
<code>defaultIsolationLevel</code>	<code>IsolationLevel</code>	Default isolation level for transactions on this CF
<code>minDiskSpace</code>	<code>number</code>	Minimum free disk space required, in bytes
<code>l1FileCountTrigger</code>	<code>number</code>	L1 file count that triggers compaction

Field	Type	Description
<code>l0QueueStallThreshold</code>	number	L0 queue depth that applies write backpressure
<code>tombstoneDensityTrigger</code>	number	Per-SSTable tombstone density above which compaction escalates (<code>[0.0, 1.0]</code> , <code>0.0</code> = disabled)
<code>tombstoneDensityMinEntries</code>	number	Minimum entry count for an SSTable to be considered by the density trigger (default <code>1024</code>)
<code>useBtree</code>	boolean	Use B+tree klog format instead of block-based (default <code>false</code>); cannot be changed after creation
<code>objectLazyCompaction</code>	boolean	Compact less aggressively in object store mode (default <code>false</code>)
<code>objectPrefetchCompaction</code>	boolean	Download all inputs before a compaction merge in object store mode (default <code>true</code>)

Immutable fields

Several fields cannot be changed once a column family exists — see [Updating Runtime Configuration](#) for the exact updatable/non-updatable split.

CRUD Operations

All operations in TidesDB are performed through transactions for ACID guarantees.

Writing Data

```
const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();

txn.put(cf, Buffer.from('key'), Buffer.from('value'), -1);

txn.commit();
txn.free();
```

Writing with TTL

```
const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();

const ttl = Math.floor(Date.now() / 1000) + 10;

txn.put(cf, Buffer.from('temp_key'), Buffer.from('temp_value'), ttl);

txn.commit();
txn.free();
```

TTL Examples

```
const ttl = -1;

const ttl = Math.floor(Date.now() / 1000) + 5 * 60;

const ttl = Math.floor(Date.now() / 1000) + 60 * 60;

const ttl = Math.floor(new Date('2026-12-31T23:59:59Z').getTime() / 1000);
```

Reading Data

```
const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();

const value = txn.get(cf, Buffer.from('key'));
console.log(`Value: ${value.toString()}`);

txn.free();
```


Deleting Data

```
const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();

txn.delete(cf, Buffer.from('key'));

txn.commit();
txn.free();
```

Single-Delete

`txn.singleDelete()` writes a tombstone with the same read semantics as `txn.delete()`, but carries a caller-provided promise that lets compaction drop the put and the tombstone together as soon as both appear in the same merge input, rather than carrying the tombstone forward until it reaches the largest active level.

Between any two single-deletes on the same key, and between the start of the key's history and its first single-delete, the key has been put **at most once**. The engine does not and cannot verify this at runtime; violating the contract can leave older puts visible after the single-delete and is a bug in the caller.

This is the right choice for workloads that insert each key exactly once and then delete it exactly once (classic insert-benchmark patterns, secondary-index entries on columns that are never updated, log-style tables with scheduled purges). It is **not** safe for tables that issue repeated updates to the same key.

```
const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();

txn.singleDelete(cf, Buffer.from('key'));

txn.commit();
txn.free();
```

When in doubt, prefer `txn.delete()`.

Multi-Operation Transactions

```
const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();

try {
  txn.put(cf, Buffer.from('key1'), Buffer.from('value1'), -1);
  txn.put(cf, Buffer.from('key2'), Buffer.from('value2'), -1);
  txn.delete(cf, Buffer.from('old_key'));

  txn.commit();
} catch (err) {
  txn.rollback();
  throw err;
} finally {
  txn.free();
}
```

Iterating Over Data

Iterators provide efficient bidirectional traversal over key-value pairs.

Forward Iteration

```
const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();
const iter = txn.newIterator(cf);

iter.seekToFirst();

while (iter.isValid()) {
  const key = iter.key();
  const value = iter.value();

  console.log(`Key: ${key.toString()}, Value: ${value.toString()}`);

  iter.next();
}

iter.free();
txn.free();
```

Backward Iteration

```
const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();
const iter = txn.newIterator(cf);

iter.seekToLast();

while (iter.isValid()) {
  const key = iter.key();
  const value = iter.value();

  console.log(`Key: ${key.toString()}, Value: ${value.toString()}`);

  iter.prev();
}

iter.free();
txn.free();
```

Seek to Specific Key

```
const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();
const iter = txn.newIterator(cf);

iter.seek(Buffer.from('user:1000'));

if (iter.isValid()) {
  console.log(`Found: ${iter.key().toString()}`);
}

iter.free();
txn.free();
```

Seek for Previous

```
const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();
const iter = txn.newIterator(cf);

iter.seekForPrev(Buffer.from('user:2000'));

while (iter.isValid()) {
  console.log(`Key: ${iter.key().toString()}`);
  iter.prev();
}

iter.free();
txn.free();
```

Combined Key-Value Retrieval

Retrieve both the key and value in a single FFI call for better performance when you need both.

```
const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();
const iter = txn.newIterator(cf);

iter.seekToFirst();

while (iter.isValid()) {
  const { key, value } = iter.keyValue();

  console.log(`Key: ${key.toString()}, Value: ${value.toString()}`);

  iter.next();
}

iter.free();
txn.free();
```

Prefix Scanning

```
const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();
const iter = txn.newIterator(cf);

const prefix = 'user: ';
iter.seek(Buffer.from(prefix));

while (iter.isValid()) {
  const key = iter.key().toString();

  if (!key.startsWith(prefix)) break;

  console.log(`Found: ${key}`);
  iter.next();
}

iter.free();
txn.free();
```

Getting Column Family Statistics

Retrieve detailed statistics about a column family.

```

const cf = db.getColumnFamily('my_cf');

const stats = cf.getStats();

console.log(`Number of Levels: ${stats.numLevels}`);
console.log(`Memtable Size: ${stats.memtableSize} bytes`);
console.log(`Total Keys: ${stats.totalKeys}`);
console.log(`Total Data Size: ${stats.totalDataSize} bytes`);
console.log(`Average Key Size: ${stats.avgKeySize} bytes`);
console.log(`Average Value Size: ${stats.avgValueSize} bytes`);
console.log(`Read Amplification: ${stats.readAmp}`);
console.log(`Cache Hit Rate: ${stats.hitRate}`);

for (let i = 0; i < stats.numLevels; i++) {
    console.log(`Level ${i + 1}: ${stats.levelNumSSTables[i]} SSTables,
${stats.levelSizes[i]} bytes, ${stats.levelKeyCounts[i]} keys,
${stats.levelTombstoneCounts[i]} tombstones`);
}

if (stats.useBtree) {
    console.log(`B+tree Total Nodes: ${stats.btreeTotalNodes}`);
    console.log(`B+tree Max Height: ${stats.btreeMaxHeight}`);
    console.log(`B+tree Avg Height: ${stats.btreeAvgHeight}`);
}

console.log(`Total Tombstones: ${stats.totalTombstones}`);
console.log(`Tombstone Ratio: ${((stats.tombstoneRatio *
100).toFixed(2))}%`);
console.log(`Worst Per-SSTable Density: ${((stats.maxSstDensity *
100).toFixed(2))}%`);
if (stats.maxSstDensityLevel > 0) {
    console.log(`Worst-density level: ${stats.maxSstDensityLevel}`); // 1-
based
}

if (stats.config) {
    console.log(`Write Buffer Size: ${stats.config.writeBufferSize}`);
    console.log(`Compression: ${stats.config.compressionAlgorithm}`);
    console.log(`Bloom Filter: ${stats.config.enableBloomFilter}`);
    console.log(`Sync Mode: ${stats.config.syncMode}`);
    console.log(`Tombstone Density Trigger:
${stats.config.tombstoneDensityTrigger}`);
    console.log(`Tombstone Density Min Entries:
${stats.config.tombstoneDensityMinEntries}`);
}

```

```
}
```

Tombstone observability fields

Field	Type	Description
<code>totalTombstones</code>	<code>number</code>	Sum of tombstone counts across every SSTable in the column family
<code>tombstoneRatio</code>	<code>number</code>	<code>totalTombstones / totalKeys</code> (0.0 if <code>totalKeys == 0</code>); range <code>[0.0, 1.0]</code>
<code>levelTombstoneCounts</code>	<code>number[]</code>	Per-level tombstone counts; parallels <code>levelKeyCounts</code> , length equals <code>numLevels</code>
<code>maxSstDensity</code>	<code>number</code>	Worst per-SSTable tombstone density observed in the column family; range <code>[0.0, 1.0]</code>
<code>maxSstDensityLevel</code>	<code>number</code>	1-based level index where <code>maxSstDensity</code> was observed (<code>0</code> if no SSTables)

Write-amplification fields

All counters are lifetime-since-open totals of on-disk framed bytes. Divide the write totals by `userBytesWritten` to compute this column family's write amplification.

```
const stats = cf.getStats();

const writeBytes =
  stats.walBytesWritten + stats.flushBytesWritten +
  stats.compactionBytesWritten;
const writeAmp = stats.userBytesWritten > 0 ? writeBytes /
  stats.userBytesWritten : 0;

console.log(`Write amplification: ${writeAmp.toFixed(2)}x`);
console.log(`Flushes: ${stats.flushCount}, Compactions:
  ${stats.compactionCount}`);
```


Field	Type	Description
<code>walBytesWritten</code>	number	Framed bytes appended to this CF's WAL. Always 0 in unified-memtable mode (see <code>DbStats.uwalBytesWritten</code>)
<code>flushBytesWritten</code>	number	On-disk bytes this CF's flushes wrote to L0 SSTables
<code>compactionBytesWritten</code>	number	On-disk bytes this CF's compactions wrote
<code>compactionBytesRead</code>	number	On-disk bytes this CF's compactions read as input
<code>userBytesWritten</code>	number	Logical key+value bytes committed to this CF (write-amplification denominator)
<code>flushCount</code>	number	Flushed SSTables produced by this CF
<code>compactionCount</code>	number	Compaction output SSTables produced by this CF

Listing Column Families

```
const cfList = db.listColumnFamilies();

console.log('Available column families:');
for (const name of cfList) {
  console.log(` - ${name}`);
}
```

Backup

Create an on-disk backup of the database without blocking reads/writes.

```
db.backup(' ./mydb_backup');
```

Behavior

- Requires the backup directory to be non-existent or empty
- Does not copy the `LOCK` file, so the backup can be opened normally

- Database stays open and usable during backup
- The backup represents the database state after all pending flushes complete

Checkpoint

Create a lightweight, near-instant snapshot of the database using hard links instead of copying SSTable data.

```
db.checkpoint('./mydb_checkpoint');
```

Behavior

- Requires the checkpoint directory to be non-existent or empty
- For each column family:
 - Flushes the active memtable so all data is in SSTables
 - Halts compactions to ensure a consistent view of live SSTable files
 - Hard links all SSTable files (`.klog` and `.vlog`) into the checkpoint directory
 - Copies small metadata files (manifest, config) into the checkpoint directory
 - Resumes compactions
- Falls back to file copy if hard linking fails (e.g., cross-filesystem)
- Database stays open and usable during checkpoint

Checkpoint vs Backup

	<code>backup()</code>	<code>checkpoint()</code>
Speed	Copies every SSTable byte-by-byte	Near-instant (hard links, O(1) per file)
Disk usage	Full independent copy	No extra disk until compaction removes old SSTables
Portability	Can be moved to another filesystem or machine	Same filesystem only (hard link requirement)
Use case	Archival, disaster recovery, remote shipping	Fast local snapshots, point-in-time reads, streaming backups

Notes

- The checkpoint can be opened as a normal TidesDB database with `TidesDB.open()`
- Hard-linked files share storage with the live database. Deleting the original database does not affect the checkpoint

Return values

- Throws `TidesDBError` with `ErrorCode.ErrExists` if checkpoint directory is not empty
- Throws `TidesDBError` with `ErrorCode.ErrInvalidArgs` for invalid arguments

Cloning a Column Family

Create a complete copy of an existing column family with a new name. The clone contains all the data from the source at the time of cloning and is completely independent.

```
db.cloneColumnFamily('source_cf', 'cloned_cf');

const original = db.getColumnFamily('source_cf');
const clone = db.getColumnFamily('cloned_cf');
```

Behavior

- Flushes the source column family's memtable to ensure all data is on disk
- Waits for any in-progress flush or compaction to complete
- Copies all SSTable files to the new directory
- The clone is completely independent — modifications to one do not affect the other

Use cases

- Testing · Create a copy of production data for testing without affecting the original
- Branching · Create a snapshot of data before making experimental changes
- Migration · Clone data before schema or configuration changes
- Backup verification · Clone and verify data integrity without modifying the source

Return values

- Throws `TidesDBError` with `ErrorCode.ErrNotFound` if source column family doesn't exist
- Throws `TidesDBError` with `ErrorCode.ErrExists` if destination column family already exists
- Throws `TidesDBError` with `ErrorCode.ErrInvalidArgs` for invalid arguments

Compaction

Manual Compaction

```
const cf = db.getColumnFamily('my_cf');

cf.compact();
```

Targeted Range Compaction

`compactRange` runs a synchronous compaction over a specific key range. Only SSTables whose minimum and maximum keys overlap the requested range participate in the merge, so the work and I/O are bounded to the affected portion of the LSM tree rather than the whole column family.

```
const cf = db.getColumnFamily('my_cf');

const start = Buffer.from('tenant_42:');
const end = Buffer.from('tenant_42;');

cf.compactRange(start, end);

// Pass null for an unbounded endpoint on that side
cf.compactRange(null, Buffer.from('cutoff_key'));
cf.compactRange(Buffer.from('start_key'), null);
```

When to use

- Bulk reclaim after a large range delete, where waiting for natural compaction would leave tombstones and obsolete versions on disk
- Tenant eviction or sliding-window expiration that does not fit TTL semantics
- Post-import cleanup of a known key range loaded with `put` followed by `delete`
- Operational counterpart to the automatic tombstone density trigger when an operator wants reclaim now rather than at the next natural threshold crossing

Behavior

- Synchronous; blocks the calling thread until the merge commits or fails
- Does not enqueue work onto the compaction thread pool — the calling thread does the work itself

- Selects only SSTables whose key range overlaps the requested range using the column family's comparator; SSTables outside the range are not touched
- Applies the same emit-loop logic as background compactions (tombstone reclamation rules, single-delete pair cancellation, sequence-based deduplication, value recompression)
- Output SSTables are committed to the manifest atomically and old inputs are marked for deletion

Return values

- Returns normally on success
- Throws `TidesDBError` with `ErrorCode.ErrInvalidArgs` if both endpoints are `null`/empty (use `compact()` for full CF compaction)
- Throws `TidesDBError` with `ErrorCode.ErrLocked` if another compaction is running for the column family
- Throws `TidesDBError` with standard I/O or memory error codes if the merge cannot complete

Manual Memtable Flush

```
const cf = db.getColumnFamily('my_cf');

cf.flushMemtable();
```

Checking Flush/Compaction Status

Check if a column family has background operations in progress.

```
const cf = db.getColumnFamily('my_cf');

if (cf.isFlushing()) {
  console.log('Flush in progress');
}

if (cf.isCompacting()) {
  console.log('Compaction in progress');
}
```

Use cases

- Graceful shutdown · Wait for background operations to complete before closing
- Maintenance windows · Check if operations are running before triggering manual compaction

- Monitoring · Track background operation status for observability

Purge (Force Flush + Compaction)

Purge forces a full flush of the active memtable followed by aggressive compaction, blocking until all flush and compaction I/O has drained. Use it to reclaim space and collapse the LSM tree on demand — for example before a backup, or after a large batch of deletes — rather than waiting for background thresholds.

Purge a single column family:

```
const cf = db.getColumnFamily('my_cf');

cf.purgeColumnFamily(); // flush + aggressive compaction for this CF,
                        // waits for I/O to finish
```

Purge every column family in the database:

```
db.purge(); // flush + aggressive compaction across all
            // CFs, waits for queues to drain
```

Behavior

- Synchronous; blocks until all flush and compaction work has completed
- `cf.purgeColumnFamily()` operates on one column family; `db.purge()` applies to all of them
- Throws `TidesDBError` on failure (`db.purge()` reports the first non-zero error code encountered)

Updating Runtime Configuration

Update runtime-safe configuration settings. Changes apply to new operations only.

```
const cf = db.getColumnFamily('my_cf');

// Update configuration (changes apply to new operations)
cf.updateRuntimeConfig({
  writeBufferSize: 256 * 1024 * 1024,
  skipListMaxLevel: 16,
  skipListProbability: 0.25,
  bloomFpr: 0.001, // 0.1% false positive rate
  indexSampleRatio: 8, // sample 1 in 8 keys
  syncMode: SyncMode.Interval,
  syncIntervalUs: 100000, // 100ms
}, true); // persist to disk (config.ini)
```

Updatable settings (safe to change at runtime):

- `writeBufferSize` · Memtable flush threshold
- `skipListMaxLevel` · Skip list level for new memtables
- `skipListProbability` · Skip list probability for new memtables
- `bloomFpr` · False positive rate for new SSTables
- `indexSampleRatio` · Index sampling ratio for new SSTables
- `syncMode` · Durability mode
- `syncIntervalUs` · Sync interval in microseconds

Non-updatable settings (would corrupt existing data):

- `compressionAlgorithm`, `enableBlockIndexes`, `enableBloomFilter`, `comparatorName`,
`levelSizeRatio`, `klogValueThreshold`, `minLevels`, `dividingLevelOffset`,
`blockIndexPrefixLen`, `l1FileCountTrigger`, `l0QueueStallThreshold`, `useBtree`

Persisting Column Family Config to INI

A `ColumnFamilyConfig` can be saved to and loaded from an INI file section. These are standalone static helpers on `TidesDB` — they operate purely on files and do **not** require an open database. This is handy for managing column family templates outside the database directory.

```

import { TidesDB, CompressionAlgorithm, SyncMode } from 'tidesdb';

// Save a config to a named section of an INI file (created if absent)
TidesDB.saveColumnFamilyConfigToIni('./cf-templates.ini', 'analytics', {
  writeBufferSize: 64 * 1024 * 1024,
  compressionAlgorithm: CompressionAlgorithm.ZstdCompression,
  enableBloomFilter: true,
  bloomFpr: 0.01,
  syncMode: SyncMode.Interval,
  syncIntervalUs: 100000,
});

// Load it back -- fields absent from the section keep their library
defaults,
// so the result is a complete config ready for createColumnFamily()
const config = TidesDB.loadColumnFamilyConfigFromIni('./cf-templates.ini',
'analytics');

db.createColumnFamily('analytics', config);

```

The configuration passed to `saveColumnFamilyConfigToIni` is merged over the library defaults before writing, so a partial config still produces a complete, round-trippable section. Loading a missing file or section throws a `TidesDBError`.

Range Cost Estimation

Estimate the computational cost of iterating between two keys in a column family. The returned value is an opaque double - meaningful only for comparison with other values from the same method. It uses only in-memory metadata and performs no disk I/O.

```

const cf = db.getColumnFamily('my_cf');

const costA = cf.rangeCost(Buffer.from('user:0000'),
Buffer.from('user:0999'));
const costB = cf.rangeCost(Buffer.from('user:1000'),
Buffer.from('user:1099'));

if (costA < costB) {
  console.log('Range A is cheaper to iterate');
}

```


Parameters

- `keyA` · `Buffer` · First key (bound of range)
- `keyB` · `Buffer` · Second key (bound of range)

Returns · `number` · Estimated traversal cost (higher = more expensive)

How it works

- With block indexes enabled · Uses $O(\log B)$ binary search per overlapping SSTable to estimate block count
- Without block indexes · Falls back to byte-level key interpolation against SSTable min/max keys
- B+tree SSTables · Uses key interpolation against tree node counts, plus tree height as a seek cost
- Compressed SSTables receive a $1.5\times$ weight multiplier to account for decompression overhead
- Each overlapping SSTable adds a small fixed cost for merge-heap operations
- The active memtable's entry count contributes a small in-memory cost

Key order does not matter - the method normalizes the range so `keyA > keyB` produces the same result as `keyB > keyA`.

Use cases

- Query planning · Compare candidate key ranges to find the cheapest one to scan
- Load balancing · Distribute range scan work across threads by estimating per-range cost
- Adaptive prefetching · Decide how aggressively to prefetch based on range size
- Monitoring · Track how data distribution changes across key ranges over time

Cost Values

The returned cost is not an absolute measure (it does not represent milliseconds, bytes, or entry counts). It is a relative scalar - only meaningful when compared with other `rangeCost` results. A cost of 0 means no overlapping SSTables or memtable entries were found for the range.

Commit Hook (Change Data Capture)

`setCommitHook` registers an optional callback that fires synchronously after every transaction commit on a column family. The hook receives the full batch of committed operations atomically, enabling real-time change data capture without WAL parsing or external log consumers.

```

import { CommitOp } from 'tidesdb';

const cf = db.getColumnFamily('my_cf');

// Attach a commit hook
cf.setCommitHook((ops: CommitOp[], commitSeq: number): number => {
  for (const op of ops) {
    if (op.isDelete) {
      console.log(`[${commitSeq}] DELETE key=${op.key.toString()}`);
    } else {
      console.log(`[${commitSeq}] PUT key=${op.key.toString()}
value=${op.value!.toString()}`);
    }
  }
  return 0; // 0 = success
});

// Normal writes now trigger the hook automatically
const txn = db.beginTransaction();
txn.put(cf, Buffer.from('key1'), Buffer.from('value1'), -1);
txn.commit(); // hook fires here
txn.free();

// Detach the hook
cf.clearCommitHook();

```

Parameters (setCommitHook)

- `callback` · `CommitHookCallback` - Function invoked with `(ops, commitSeq)`. Return `0` on success; non-zero is logged as a warning but does not roll back the commit.

CommitOp fields

- `key` · `Buffer` - Key data
- `value` · `Buffer | null` - Value data (`null` for deletes)
- `ttl` · `number` - Time-to-live (Unix timestamp, `-1` = no expiry)
- `isDelete` · `boolean` - Whether this is a delete operation

Behavior

- The hook fires after WAL write, memtable apply, and commit status marking are complete - the data is fully durable before the callback runs

- Hook failure (non-zero return) is logged but does not affect the commit result
- Each column family has its own independent hook; a multi-CF transaction fires the hook once per CF with only that CF's operations
- `commitSeq` is monotonically increasing across commits and can be used as a replication cursor
- Data in `CommitOp` is copied and safe to retain beyond the callback
- The hook executes synchronously on the committing thread; keep the callback fast to avoid stalling writers
- Calling `clearCommitHook()` disables it immediately with no restart required

Use cases

- Replication · Ship committed batches to replicas in commit order
- Event streaming · Publish mutations to Kafka, NATS, or any message broker
- Secondary indexing · Maintain a reverse index or materialized view
- Audit logging · Record every mutation with key, value, TTL, and sequence number
- Debugging · Attach a temporary hook in production to inspect live writes

Runtime-Only

Commit hooks are not persisted. After a database restart, hooks must be re-registered by the application. This is by design - function pointers cannot be serialized.

Manual WAL Sync

Force an immediate fsync of the active write-ahead log for a column family. This is useful for explicit durability control when using `SyncMode.None` or `SyncMode.Interval` modes.

```
const cf = db.getColumnFamily('my_cf');

cf.syncWal();
```

When to use

- Application-controlled durability · Sync the WAL at specific points (e.g., after a batch of related writes) when using `SyncMode.None` or `SyncMode.Interval`
- Pre-checkpoint · Ensure all buffered WAL data is on disk before taking a checkpoint
- Graceful shutdown · Flush WAL buffers before closing the database
- Critical writes · Force durability for specific high-value writes without using `SyncMode.Full` for all writes

Behavior

- Acquires a reference to the active memtable to safely access its WAL
- Calls `fdatsync` on the WAL file descriptor
- Thread-safe - can be called concurrently from multiple threads
- If the memtable rotates during the call, retries with the new active memtable

Return values

- Throws `TidesDBError` with `ErrorCode.ErrInvalidArgs` if column family is invalid
- Throws `TidesDBError` with `ErrorCode.ErrIO` if the fsync operation fails

Structural Operations

Regardless of sync mode, TidesDB **always** enforces durability for structural operations:

- Memtable flush to SSTable
- SSTable compaction and merging
- WAL rotation
- Column family metadata updates

Database-Level Statistics

Get aggregate statistics across the entire database instance.

```
const dbStats = db.getDbStats();

console.log(`Column families: ${dbStats.numColumnFamilies}`);
console.log(`Total memory: ${dbStats.totalMemory} bytes`);
console.log(`Resolved memory limit: ${dbStats.resolvedMemoryLimit} bytes`);
console.log(`Memory pressure level: ${dbStats.memoryPressureLevel}`);
console.log(`Global sequence: ${dbStats.globalSeq}`);
console.log(`Flush queue: ${dbStats.flushQueueSize} pending`);
console.log(`Compaction queue: ${dbStats.compactionQueueSize} pending`);
console.log(`Total SSTables: ${dbStats.totalSstableCount}`);
console.log(`Total data size: ${dbStats.totalDataSizeBytes} bytes`);
console.log(`Open SSTable handles: ${dbStats.numOpenSstables}`);
console.log(`In-flight txn memory: ${dbStats.txnMemoryBytes} bytes`);
console.log(`Immutable memtables: ${dbStats.totalImmutableCount}`);
console.log(`Memtable bytes: ${dbStats.totalMemtableBytes}`);
```

Database statistics include

Field	Type	Description
numColumnFamilies	number	Number of column families
totalMemory	number	System total memory
availableMemory	number	System available memory at open time
resolvedMemoryLimit	number	Resolved memory limit (auto or configured)
memoryPressureLevel	number	Current memory pressure (0=normal, 1=elevated, 2=high, 3=critical)
flushPendingCount	number	Number of pending flush operations (queued + in-flight)
totalMemtableBytes	number	Total bytes in active memtables across all CFs
totalImmutableCount	number	Total immutable memtables across all CFs
totalSstableCount	number	Total SSTables across all CFs and levels
totalDataSizeBytes	number	Total data size (klog + vlog) across all CFs
numOpenSstables	number	Number of currently open SSTable file handles
globalSeq	number	Current global sequence number
txnMemoryBytes	number	Bytes held by in-flight transactions
compactionQueueSize	number	Number of pending compaction tasks
flushQueueSize	number	Number of pending flush tasks in queue
unifiedMemtableEnabled	boolean	Whether unified memtable mode is active

Field	Type	Description
<code>unifiedMemtableBytes</code>	number	Bytes in unified active memtable
<code>unifiedImmutableCount</code>	number	Number of unified immutable memtables
<code>unifiedIsFlushing</code>	boolean	Whether unified memtable is currently flushing/rotating
<code>unifiedNextCfIndex</code>	number	Next CF index to be assigned in unified mode
<code>unifiedWalGeneration</code>	number	Current unified WAL generation counter
<code>objectStoreEnabled</code>	boolean	Whether object store mode is active
<code>objectStoreConnector</code>	string	Object store connector name (“s3”, “gcs”, “fs”, etc.)
<code>localCacheBytesUsed</code>	number	Current local file cache usage in bytes
<code>localCacheBytesMax</code>	number	Configured maximum local cache size in bytes
<code>localCacheNumFiles</code>	number	Number of files tracked in local cache
<code>lastUploadedGeneration</code>	number	Highest WAL generation confirmed uploaded
<code>uploadQueueDepth</code>	number	Number of pending upload jobs in the queue
<code>totalUploads</code>	number	Lifetime count of objects uploaded to object store
<code>totalUploadFailures</code>	number	Lifetime count of permanently failed uploads
<code>replicaMode</code>	boolean	Whether running in read-only replica mode
<code>primaryEpoch</code>	number	Single-writer fencing (object-store mode): the lease epoch this primary currently holds (0 when not a primary / no lease)

Field	Type	Description
<code>seenEpoch</code>	<code>number</code>	Single-writer fencing (object-store mode): the highest lease epoch a replica has observed
<code>uwalBytesWritten</code>	<code>number</code>	Framed bytes appended to the shared unified WAL (<code>0</code> when unified mode is off)
<code>walBytesWritten</code>	<code>number</code>	Per-CF WAL bytes summed across all column families
<code>flushBytesWritten</code>	<code>number</code>	Flush output bytes summed across all column families
<code>compactionBytesWritten</code>	<code>number</code>	Compaction output bytes summed across all column families
<code>compactionBytesRead</code>	<code>number</code>	Compaction input bytes summed across all column families
<code>userBytesWritten</code>	<code>number</code>	Logical committed bytes summed across all column families
<code>flushCount</code>	<code>number</code>	Flushed SSTables summed across all column families
<code>compactionCount</code>	<code>number</code>	Compaction output SSTables summed across all column families

All write-amplification counters are lifetime-since-open totals of on-disk framed bytes. Db-wide write amplification = $(\text{uwalBytesWritten} + \text{walBytesWritten} + \text{flushBytesWritten} + \text{compactionBytesWritten}) / \text{userBytesWritten}$.

Stack Allocated

Unlike `cf.getStats()` (which heap-allocates), `db.getDbStats()` fills a caller-provided struct. No free is needed.

Sync Modes

Control the durability vs performance tradeoff.


```
import { SyncMode } from 'tidesdb';

// SyncNone -- Fastest, least durable (OS handles flushing)
db.createColumnFamily('fast_cf', {
  syncMode: SyncMode.None,
});

// SyncInterval -- Balanced (periodic background syncing)
db.createColumnFamily('balanced_cf', {
  syncMode: SyncMode.Interval,
  syncIntervalUs: 128000, // Sync every 128ms
});

// SyncFull -- Most durable (fsync on every write)
db.createColumnFamily('durable_cf', {
  syncMode: SyncMode.Full,
});
```

Compression Algorithms

TidesDB supports multiple compression algorithms:

```
import { CompressionAlgorithm } from 'tidesdb';

db.createColumnFamily('no_compress', {
  compressionAlgorithm: CompressionAlgorithm.NoCompression,
});

db.createColumnFamily('lz4_cf', {
  compressionAlgorithm: CompressionAlgorithm.Lz4Compression,
});

db.createColumnFamily('lz4_fast_cf', {
  compressionAlgorithm: CompressionAlgorithm.Lz4FastCompression,
});

db.createColumnFamily('zstd_cf', {
  compressionAlgorithm: CompressionAlgorithm.ZstdCompression,
});

db.createColumnFamily('snappy_cf', {
  compressionAlgorithm: CompressionAlgorithm.SnappyCompression,
});
```

Snappy Availability

Snappy compression is not available on SunOS/Illumos/OmniOS platforms.

B+tree KLog Format (Optional)

Column families can optionally use a B+tree structure for the key log instead of the default block-based format. The B+tree klog format offers faster point lookups through $O(\log N)$ tree traversal.

```
db.createColumnFamily('btree_cf', {
  useBtree: true,
  enableBloomFilter: true,
  bloomFpr: 0.01,
});

// Check if column family uses B+tree klog format
const cf = db.getColumnFamily('btree_cf');
const stats = cf.getStats();

console.log(`Uses B+tree: ${stats.useBtree}`);
if (stats.useBtree) {
  console.log(`B+tree Total Nodes: ${stats.btreeTotalNodes}`);
  console.log(`B+tree Max Height: ${stats.btreeMaxHeight}`);
  console.log(`B+tree Avg Height: ${stats.btreeAvgHeight}`);
}
```

Characteristics

- Point lookups · $O(\log N)$ tree traversal with binary search at each node
- Range scans · Doubly-linked leaf nodes enable efficient bidirectional iteration
- Immutable · Tree is bulk-loaded from sorted memtable data during flush
- Compression · Nodes compress independently using the same algorithms (LZ4, Zstd, etc.)

When to use B+tree klog format

- Read-heavy workloads with frequent point lookups
- Workloads where read latency is more important than write throughput
- Large SSTables where block scanning becomes expensive

Important

`useBtree` **cannot be changed** after column family creation. Different column families can use different formats.

Unified Memtable Mode

In unified memtable mode, all column families share a single memtable and WAL, reducing write amplification and flush overhead for multi-CF workloads.

```
import { TidesDB, SyncMode } from 'tidesdb';

const db = TidesDB.open({
  dbPath: './mydb',
  numFlushThreads: 2,
  numCompactionThreads: 2,
  unifiedMemtable: true,
  unifiedMemtableWriteBufferSize: 128 * 1024 * 1024,
  unifiedMemtableSkipListMaxLevel: 12,
  unifiedMemtableSkipListProbability: 0.25,
  unifiedMemtableSyncMode: SyncMode.Interval,
  unifiedMemtableSyncIntervalUs: 128000,
});

// Check unified memtable status via database stats
const stats = db.getDbStats();
console.log(`Unified memtable enabled: ${stats.unifiedMemtableEnabled}`);
console.log(`Unified memtable bytes: ${stats.unifiedMemtableBytes}`);
console.log(`Unified immutable count: ${stats.unifiedImmutableCount}`);
console.log(`Unified is flushing: ${stats.unifiedIsFlushing}`);
console.log(`Unified WAL generation: ${stats.unifiedWalGeneration}`);

db.close();
```

Configuration

Field	Type	Default	Description
<code>unifiedMemtable</code>	boolean	<code>false</code>	Enable unified memtable mode
<code>unifiedMemtableWriteBufferSize</code>	number	0 (auto)	Write buffer size for unified memtable
<code>unifiedMemtableSkipListMaxLevel</code>	number	0 (default 12)	Skip list max level
<code>unifiedMemtableSkipListProbability</code>	number	0 (default 0.25)	Skip list probability
<code>unifiedMemtableSyncMode</code>	SyncMode	<code>SyncMode.None</code>	Sync mode for unified WAL
<code>unifiedMemtableSyncIntervalUs</code>	number	0 (default)	Sync interval in microseconds

Object Store Mode

Object store mode allows TidesDB to store SSTables in a remote object store (or local filesystem for testing) while using local disk as a cache. This separates compute from storage and enables cold start recovery. Object store mode requires unified memtable mode and is automatically enforced when a connector is set.

Enabling Object Store Mode (Filesystem Connector)

```
import { TidesDB, LogLevel } from 'tidesdb';

const db = TidesDB.open({
  dbPath: './mydb',
  objectStoreFsPath: '/mnt/nfs/tidesdb-objects',
});

// Use the database normally -- SSTables are uploaded after flush

db.close();
```

Object Store with Custom Configuration

```
import { TidesDB } from 'tidesdb';

const db = TidesDB.open({
  dbPath: './mydb',
  objectStoreFsPath: '/mnt/nfs/tidesdb-objects',
  objectStoreConfig: {
    localCacheMaxBytes: 512 * 1024 * 1024, // 512MB local cache
    maxConcurrentUploads: 8,
    maxConcurrentDownloads: 16,
    cacheOnRead: true,
    cacheOnWrite: true,
    syncManifestToObject: true,
    replicateWal: true,
    walSyncThresholdBytes: 1048576, // 1MB
  },
});

db.close();
```

Object Store Configuration Options

Field	Type	Default	Description
<code>localCachePath</code>	<code>string null</code>	<code>null</code> (uses <code>dbPath</code>)	Local directory for cached SSTable files
<code>localCacheMaxBytes</code>	<code>number</code>	<code>0</code> (unlimited)	Maximum local cache size in bytes
<code>cacheOnRead</code>	<code>boolean</code>	<code>true</code>	Cache downloaded files locally
<code>cacheOnWrite</code>	<code>boolean</code>	<code>true</code>	Keep local copy after upload
<code>maxConcurrentUploads</code>	<code>number</code>	<code>4</code>	Number of parallel upload threads
<code>maxConcurrentDownloads</code>	<code>number</code>	<code>8</code>	Number of parallel download threads
<code>multipartThreshold</code>	<code>number</code>	<code>67108864</code> (64MB)	Use multipart upload above this size
<code>multipartPartSize</code>	<code>number</code>	<code>8388608</code> (8MB)	Chunk size for multipart uploads
<code>syncManifestToObject</code>	<code>boolean</code>	<code>true</code>	Upload MANIFEST after each compaction
<code>replicateWal</code>	<code>boolean</code>	<code>true</code>	Upload closed WAL segments for replication
<code>walUploadSync</code>	<code>boolean</code>	<code>false</code>	Block flush until WAL is uploaded
<code>walSyncThresholdBytes</code>	<code>number</code>	<code>1048576</code> (1MB)	Sync active WAL to object store when it grows by this many bytes (0 = off)
<code>walSyncOnCommit</code>	<code>boolean</code>	<code>false</code>	Upload WAL after every txn commit for RPO=0 replication
<code>replicaMode</code>	<code>boolean</code>	<code>false</code>	Enable read-only replica mode (writes return <code>ErrorCode.ErrReadOnly</code>)
<code>replicaSyncIntervalUs</code>	<code>number</code>	<code>5000000</code> (5s)	MANIFEST poll interval for replica sync in microseconds
<code>replicaReplayWal</code>	<code>boolean</code>	<code>true</code>	Replay WAL from object store for near-real-time reads on replicas

Per-CF Object Store Tuning

Column family configurations include two object store tuning fields.

Field	Type	Default	Description
<code>objectLazyCompaction</code>	<code>boolean</code>	<code>false</code>	Compact less aggressively for remote storage
<code>objectPrefetchCompaction</code>	<code>boolean</code>	<code>true</code>	Download all inputs before compaction merge

```
db.createColumnFamily('remote_cf', {
  objectLazyCompaction: true,
  objectPrefetchCompaction: true,
});
```

Cold Start Recovery

When the local database directory is empty but a connector is configured, TidesDB automatically discovers column families from the object store, downloads MANIFEST and config files in parallel, and fetches SSTable data on demand.

```
// Reopen with the same connector -- cold start recovery
const db = TidesDB.open({
  dbPath: './mydb',
  objectStoreFsPath: '/mnt/nfs/tidesdb-objects',
});

// All data is available -- SSTables are fetched on demand
const cf = db.getColumnFamily('my_cf');
```

Replica Mode

Replica mode enables read-only nodes that follow a primary through the object store.

```
// Open as a replica
const replica = TidesDB.open({
  dbPath: './mydb_replica',
  objectStoreFsPath: '/mnt/nfs/tidesdb-objects', // Same store as primary
  objectStoreConfig: {
    replicaMode: true,
    replicaSyncIntervalUs: 1000000, // 1 second sync interval
    replicaReplayWal: true,
  },
});

// Reads work normally
const cf = replica.getColumnFamily('my_cf');
const txn = replica.beginTransaction();
const value = txn.get(cf, Buffer.from('key'));
txn.free();

// Writes are rejected with ErrorCode.ErrReadonly
```

Sync-on-Commit WAL (Primary Side)

For tighter replication lag, enable sync-on-commit on the primary so every committed write is uploaded immediately.

```
const primary = TidesDB.open({
  dbPath: './mydb',
  objectStoreFsPath: '/mnt/nfs/tidesdb-objects',
  objectStoreConfig: {
    walSyncOnCommit: true, // RPO = 0, every commit is durable
  },
});
```

Object Store Statistics

Object store fields are included in database-level statistics when a connector is active.


```

const stats = db.getDbStats();

if (stats.objectStoreEnabled) {
  console.log(`Connector: ${stats.objectStoreConnector}`);
  console.log(`Total uploads: ${stats.totalUploads}`);
  console.log(`Upload failures: ${stats.totalUploadFailures}`);
  console.log(`Upload queue depth: ${stats.uploadQueueDepth}`);
  console.log(`Local cache: ${stats.localCacheBytesUsed} /
${stats.localCacheBytesMax} bytes`);
  console.log(`Replica mode: ${stats.replicaMode}`);
}

```

S3 / MinIO Connector

The S3-compatible connector (AWS S3, MinIO, and any S3 API-compatible store) is exposed through the TypeScript binding via the `objectStoreS3Config` option on `TidesDB.open()`. Provide an `S3Config` instead of `objectStoreFsPath`; the `objectStoreConfig` behavior options (cache sizing, WAL replication, replica mode, etc.) apply identically to both connectors.

```

import { TidesDB } from 'tidesdb';

const db = TidesDB.open({
  dbPath: './mydb',
  objectStoreS3Config: {
    endpoint: 's3.amazonaws.com',
    bucket: 'my-tidesdb-bucket',
    prefix: 'production/db1/',
    accessKey: process.env.AWS_ACCESS_KEY_ID!,
    secretKey: process.env.AWS_SECRET_ACCESS_KEY!,
    region: 'us-east-1',
    useSsl: true,
  },
  objectStoreConfig: {
    localCacheMaxBytes: 512 * 1024 * 1024, // 512MB local cache
    maxConcurrentUploads: 8,
  },
});

db.close();

```

For MinIO (or any self-hosted S3 gateway), use path-style URLs and the gateway endpoint:

```
const db = TidesDB.open({
  dbPath: './mydb',
  objectStoreS3Config: {
    endpoint: 'minio.local:9000',
    bucket: 'tidesdb',
    accessKey: 'minioadmin',
    secretKey: 'minioadmin',
    useSsl: false,
    usePathStyle: true,    // required for MinIO
  },
});
```

S3Config Options

Field	Type	Default	Description
<code>endpoint</code>	<code>string</code>	<i>(required)</i>	S3 endpoint (e.g. <code>"s3.amazonaws.com"</code> or <code>"minio.local:9000"</code>)
<code>bucket</code>	<code>string</code>	<i>(required)</i>	Bucket name
<code>accessKey</code>	<code>string</code>	<i>(required)</i>	AWS access key ID
<code>secretKey</code>	<code>string</code>	<i>(required)</i>	AWS secret access key
<code>prefix</code>	<code>string</code> <code>null</code>	<code>null</code>	Key prefix (e.g. <code>"production/db1/"</code>)
<code>region</code>	<code>string</code> <code>null</code>	<code>null</code>	AWS region (e.g. <code>"us-east-1"</code>); <code>null</code> for MinIO
<code>useSsl</code>	<code>boolean</code>	<code>true</code>	Use HTTPS
<code>usePathStyle</code>	<code>boolean</code>	<code>false</code>	Use path-style URLs (required for MinIO); <code>false</code> = virtual-hosted (AWS)
<code>tlsCaPath</code>	<code>string</code> <code>null</code>	<code>null</code> (system bundle)	Custom CA bundle file path
<code>tlsInsecureSkipVerify</code>	<code>boolean</code>	<code>false</code>	Disable TLS peer + host verification (test only, insecure)
<code>multipartThreshold</code>	<code>number</code>	0 (library default)	Object size at/above which multipart upload is used
<code>multipartPartSize</code>	<code>number</code>	0 (library default)	Multipart chunk size in bytes

S3 build requirement

The filesystem connector is always available. The S3 connector requires a `libtidesdb` built with `-DTIDESDB_WITH_S3=ON`. On a library built without S3 support, passing `objectStoreS3Config` throws a `TidesDBError` explaining that the S3 connector is unavailable; rebuild with `TIDESDB_WITH_S3` to enable it.

The `ObjStoreBackend` enum identifies which connector a database reports:

Member	Value	Backend
<code>ObjStoreBackend.Fs</code>	<code>0</code>	Filesystem connector
<code>ObjStoreBackend.S3</code>	<code>1</code>	S3-compatible connector
<code>ObjStoreBackend.Unknown</code>	<code>99</code>	Unknown / not set

Promote Replica to Primary

Switch a read-only replica database to primary mode.

```
db.promoteToPrimary();
```

Behavior

- Only valid when the database was opened in replica mode
- Throws `TidesDBError` with `ErrorCode.ErrInvalidArgs` if the database is not a replica

Cancelling Background Work

`db.cancelBackgroundWork()` cancels background compaction db-wide for a fast shutdown. It is the most aggressive shutdown option, complementing the `finishCompactionsOnClose` open-time flag.

```
// Fast shutdown: stop compaction, then close
db.cancelBackgroundWork();
db.close();
```

Behavior

- In-flight merges bail safely at their next checkpoint and queued compaction is skipped

- Flushes are **unaffected**, so durability is preserved — no committed data is lost
- Blocks (bounded) until compaction is idle
- Sticky for the session and reset on the next `open()` — intended to be called right before `close()`
- Throws `TidesDBError` with `ErrorCode.ErrInvalidArgs` if the database handle is invalid

Shutdown options compared

Approach	Speed	Behavior
<code>finishCompactionsOnClose: true</code> (config)	Slowest	Let in-flight compactions finish before <code>close()</code> returns
Default <code>close()</code>	Fast	Cancel in-flight compactions at next checkpoint
<code>cancelBackgroundWork()</code> then <code>close()</code>	Fastest	Proactively stop compaction db-wide, then close

Delete Column Family by Handle

Delete a column family using its handle instead of its name.

```
const cf = db.getColumnFamily('my_cf');
db.deleteColumnFamily(cf);
```

Error Handling

```
import { TidesDBError, ErrorCode } from 'tidesdb';

const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();

try {
  txn.put(cf, Buffer.from('key'), Buffer.from('value'), -1);
  txn.commit();
} catch (err) {
  if (err instanceof TidesDBError) {
    console.log(`Error code: ${err.code}`);
    console.log(`Error message: ${err.message}`);
    console.log(`Context: ${err.context}`);

    // Example error message:
    // "failed to put key-value pair: memory allocation failed"
  }

  txn.rollback();
} finally {
  txn.free();
}
```

Error Codes

- `ErrorCode.Success` (0) · Operation successful
- `ErrorCode.ErrMemory` (-1) · Memory allocation failed
- `ErrorCode.ErrInvalidArgs` (-2) · Invalid arguments
- `ErrorCode.ErrNotFound` (-3) · Key not found
- `ErrorCode.ErrIO` (-4) · I/O error
- `ErrorCode.ErrCorruption` (-5) · Data corruption
- `ErrorCode.ErrExists` (-6) · Resource already exists
- `ErrorCode.ErrConflict` (-7) · Transaction conflict
- `ErrorCode.ErrTooLarge` (-8) · Key or value too large
- `ErrorCode.ErrMemoryLimit` (-9) · Memory limit exceeded
- `ErrorCode.ErrInvalidDB` (-10) · Invalid database handle
- `ErrorCode.ErrUnknown` (-11) · Unknown error

- `ErrorCode.ErrLocked` (-12) · Database is locked
- `ErrorCode.ErrReadonly` (-13) · Database is read-only
- `ErrorCode.ErrBusy` (-14) · Resource is busy
- `ErrorCode.ErrPrecondition` (-15) · Precondition failed

Complete Example

```
import {
  TidesDB,
  LogLevel,
  CompressionAlgorithm,
  SyncMode
} from 'tidesdb';

function main() {
  const db = TidesDB.open({
    dbPath: './example_db',
    numFlushThreads: 1,
    numCompactionThreads: 1,
    logLevel: LogLevel.Info,
    blockCacheSize: 64 * 1024 * 1024,
    maxOpenSSTables: 256,
  });

  try {
    db.createColumnFamily('users', {
      writeBufferSize: 64 * 1024 * 1024,
      compressionAlgorithm: CompressionAlgorithm.Lz4Compression,
      enableBloomFilter: true,
      bloomFpr: 0.01,
      syncMode: SyncMode.Interval,
      syncIntervalUs: 128000,
    });

    const cf = db.getColumnFamily('users');

    const txn = db.beginTransaction();

    txn.put(cf, Buffer.from('user:1'), Buffer.from('Alice'), -1);
    txn.put(cf, Buffer.from('user:2'), Buffer.from('Bob'), -1);

    // Temporary session with 30 second TTL
    const ttl = Math.floor(Date.now() / 1000) + 30;
    txn.put(cf, Buffer.from('session:abc'), Buffer.from('temp_data'), ttl);

    txn.commit();
    txn.free();
  }
}
```

```

const readTxn = db.beginTransaction();

const value = readTxn.get(cf, Buffer.from('user:1'));
console.log(`user:1 = ${value.toString()}`);

// Iterate over all entries
const iter = readTxn.newIterator(cf);

console.log(`\nAll entries:`);
iter.seekToFirst();
while (iter.isValid()) {
    const key = iter.key();
    const val = iter.value();
    console.log(`  ${key.toString()} = ${val.toString()}`);
    iter.next();
}

iter.free();
readTxn.free();

const stats = cf.getStats();

console.log(`\nColumn Family Statistics:`);
console.log(`  Number of Levels: ${stats.numLevels}`);
console.log(`  Memtable Size: ${stats.memtableSize} bytes`);
console.log(`  Total Keys: ${stats.totalKeys}`);
console.log(`  Read Amplification: ${stats.readAmp}`);

// Check background operation status
console.log(`\nBackground Operations:`);
console.log(`  Flushing: ${cf.isFlushing()}`);
console.log(`  Compacting: ${cf.isCompacting()}`);

// Create a backup
db.backup('./example_db_backup');
console.log(`\nBackup created successfully`);

// Create a lightweight checkpoint (hard links, near-instant)
db.checkpoint('./example_db_checkpoint');
console.log(`Checkpoint created successfully`);

// Cleanup
db.dropColumnFamily('users');
} finally {
    db.close();
}

```



```
}  
  
main();
```

Isolation Levels

TidesDB supports five MVCC isolation levels:

```
import { IsolationLevel } from 'tidesdb';  
  
const txn = db.beginTransactionWithIsolation(IsolationLevel.ReadCommitted);  
  
// ... perform operations  
  
txn.commit();  
txn.free();
```

Available Isolation Levels

- `IsolationLevel.ReadUncommitted` · Sees all data including uncommitted changes
- `IsolationLevel.ReadCommitted` · Sees only committed data (default)
- `IsolationLevel.RepeatableRead` · Consistent snapshot, phantom reads possible
- `IsolationLevel.Snapshot` · Write-write conflict detection
- `IsolationLevel.Serializable` · Full read-write conflict detection (SSI)

Savepoints

Savepoints allow partial rollback within a transaction. You can create named savepoints and rollback to them without aborting the entire transaction.

```
const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();

txn.put(cf, Buffer.from('key1'), Buffer.from('value1'), -1);

txn.savepoint('sp1');
txn.put(cf, Buffer.from('key2'), Buffer.from('value2'), -1);

// Rollback to savepoint -- key2 is discarded, key1 remains
txn.rollbackToSavepoint('sp1');

// Commit -- only key1 is written
txn.commit();
txn.free();
```

Savepoint API

- `txn.savepoint('name')` · Create a savepoint
- `txn.rollbackToSavepoint('name')` · Rollback to savepoint
- `txn.releaseSavepoint('name')` · Release savepoint without rolling back

Savepoint behavior

- Savepoints capture the transaction state at a specific point
- Rolling back to a savepoint discards all operations after that savepoint
- Releasing a savepoint frees its resources without rolling back
- Multiple savepoints can be created with different names
- Creating a savepoint with an existing name updates that savepoint
- Savepoints are automatically freed when the transaction commits or rolls back

Transaction Reset

`txn.reset()` resets a committed or aborted transaction for reuse with a new isolation level. This avoids the overhead of freeing and reallocating transaction resources in hot loops.

```

const cf = db.getColumnFamily('my_cf');

const txn = db.beginTransaction();

// First batch of work
txn.put(cf, Buffer.from('key1'), Buffer.from('value1'), -1);
txn.commit();

// Reset instead of free + beginTransaction
txn.reset(IsolationLevel.ReadCommitted);

// Second batch of work using the same transaction
txn.put(cf, Buffer.from('key2'), Buffer.from('value2'), -1);
txn.commit();

// Free once when done
txn.free();

```

Behavior

- The transaction must be committed or aborted before reset; resetting an active transaction throws an error
- Internal buffers are retained to avoid reallocation
- A fresh transaction ID and snapshot sequence are assigned based on the new isolation level
- The isolation level can be changed on each reset (e.g., `ReadCommitted` -> `Serializable`)

When to use

- Batch processing · Reuse a single transaction across many commit cycles in a loop
- Connection pooling · Reset a transaction for a new request without reallocation
- High-throughput ingestion · Reduce allocation overhead in tight write loops

Reset vs Free + Begin

For a single transaction, `txn.reset()` is functionally equivalent to calling `txn.free()` followed by `db.beginTransactionWithIsolation()`. The difference is performance - reset retains allocated buffers and avoids repeated allocation overhead. This matters most in loops that commit and restart thousands of transactions.

Cache Statistics

```
const cacheStats = db.getCacheStats();

console.log(`Cache enabled: ${cacheStats.enabled}`);
console.log(`Total entries: ${cacheStats.totalEntries}`);
console.log(`Total bytes: ${((cacheStats.totalBytes / (1024 * 1024)).toFixed(2))} MB`);
console.log(`Hits: ${cacheStats.hits}`);
console.log(`Misses: ${cacheStats.misses}`);
console.log(`Hit rate: ${((cacheStats.hitRate * 100).toFixed(1))}%`);
console.log(`Partitions: ${cacheStats.numPartitions}`);
```

Multi-Column-Family Transactions

TidesDB supports atomic transactions across multiple column families with true all-or-nothing semantics.

```
const usersCf = db.getColumnFamily('users');
const ordersCf = db.getColumnFamily('orders');

const txn = db.beginTransaction();

txn.put(usersCf, Buffer.from('user:1000'), Buffer.from('John Doe'), -1);
txn.put(ordersCf, Buffer.from('order:5000'),
Buffer.from('user:1000|product:A'), -1);

txn.commit();
txn.free();
```

Multi-CF guarantees

- Either all CFs commit or none do (atomic)
- Automatically detected when operations span multiple CFs
- Uses global sequence numbers for atomic ordering
- Each CF's WAL receives operations with the same commit sequence number
- No two-phase commit or coordinator overhead

Custom Comparators

TidesDB uses comparators to determine the sort order of keys throughout the entire system, memtables, SSTables, block indexes, and iterators all use the same comparison logic. Once a comparator is set for a column family, it **cannot be changed** without corrupting data.

Built-in Comparators

TidesDB provides six built-in comparators that are automatically registered on database open:

- `"memcmp"` (**default**) · Binary byte-by-byte comparison. Shorter key sorts first if bytes are equal.
- `"lexicographic"` · Null-terminated string comparison using `strcmp()`. Keys must be null-terminated.
- `"uint64"` · Unsigned 64-bit integer comparison. Interprets 8-byte keys as uint64 values. Falls back to memcmp if key size != 8.
- `"int64"` · Signed 64-bit integer comparison. Interprets 8-byte keys as int64 values. Falls back to memcmp if key size != 8.
- `"reverse"` · Reverse binary comparison. Sorts keys in descending order.
- `"case_insensitive"` · Case-insensitive ASCII comparison. Converts A-Z to a-z during comparison.

Because they are auto-registered on open, built-in comparators can be used directly via `comparatorName` without calling `registerComparator()` first. The `BuiltinComparator` enum provides typo-safe constants for these names:

```
import { TidesDB, BuiltinComparator } from 'tidesdb';

db.createColumnFamily('numeric_cf', {
  comparatorName: BuiltinComparator.Uint64,    // 8-byte keys ordered
  numerically
});

db.createColumnFamily('descending_cf', {
  comparatorName: BuiltinComparator.Reverse,
});
```

Constant	Name
<code>BuiltinComparator.Memcmp</code>	<code>"memcmp"</code>
<code>BuiltinComparator.Lexicographic</code>	<code>"lexicographic"</code>
<code>BuiltinComparator.Uint64</code>	<code>"uint64"</code>
<code>BuiltinComparator.Int64</code>	<code>"int64"</code>
<code>BuiltinComparator.Reverse</code>	<code>"reverse"</code>
<code>BuiltinComparator.CaseInsensitive</code>	<code>"case_insensitive"</code>

Registering a Comparator

Built-in comparators are auto-registered, so `registerComparator()` is for custom comparators (and is also accepted as an explicit, harmless call for built-ins).

```
// Register comparator after opening database but before creating CF
db.registerComparator('reverse');

db.createColumnFamily('sorted_cf', {
    comparatorName: 'reverse',
});
```

`registerComparator(name, ctxStr?)` accepts an optional second argument, a context string passed through to the comparator's context (default `""`). Use it to parameterize a comparator (for example, a collation locale) without a separate registration call:

```
db.registerComparator('locale_aware', 'en_US');
```

Retrieving a Registered Comparator

Use `getComparator` to check whether a comparator is registered:

```
if (db.getComparator('reverse')) {  
  console.log('Comparator "reverse" is registered');  
} else {  
  console.log('Comparator not registered');  
}
```

Use cases

- Validation · Check if a comparator is registered before creating a column family
- Debugging · Verify comparator registration during development
- Dynamic configuration · Query available comparators at runtime

Important

Comparators must be **registered before** creating column families that use them. Once set, a comparator **cannot be changed** for a column family. The same comparator is used across memtables, SSTables, block indexes, and iterators.

Testing



```
npm install  
npm run build  
  
# Run all tests  
npm test  
  
# Run specific test  
npm test -- --testNamePattern="put and get"
```

TypeScript Types

The package exports all necessary types for full TypeScript support:

```
import {
    // Main classes
    TidesDB,
    ColumnFamily,
    Transaction,
    Iterator,
    TidesDBError,

    // Configuration interfaces
    Config,
    ColumnFamilyConfig,
    ObjectStoreConfig,
    S3Config,
    Stats,
    DbStats,
    CacheStats,

    // Enums
    CompressionAlgorithm,
    SyncMode,
    LogLevel,
    IsolationLevel,
    ErrorCode,
    BuiltinComparator,
    ObjStoreBackend,

    // Commit hook (change data capture)
    CommitOp,
    CommitHookCallback,

    // Helper functions
    defaultConfig,
    defaultColumnFamilyConfig,
    defaultObjectStoreConfig,
    checkResult,
} from 'tidesdb';
```

Library-Level Static Methods

In addition to the instance methods on a database handle, the `TidesDB` class exposes process/library-level static helpers:


```
TidesDB.init(); // optional
explicit init -> boolean
TidesDB.finalize(); // teardown
(advanced)
TidesDB.raiseOpenFileLimit(65536); // raise open-file
ceiling -> number
TidesDB.isCompressionAvailable(CompressionAlgorithm.ZstdCompression); // ->
boolean

// Column family config <-> INI file (no open database required)
TidesDB.saveColumnFamilyConfigToIni('./cf.ini', 'my_cf', { /*
ColumnFamilyConfig */ });
TidesDB.loadColumnFamilyConfigFromIni('./cf.ini', 'my_cf'); // ->
ColumnFamilyConfig
```

TidesDB Lua API Reference

Getting Started

Prerequisites

You **must** have the TidesDB shared C library installed on your system. You can find the installation instructions [here](#).

You also need **LuaJIT 2.1+** or **Lua 5.1+** with LuaFFI installed.

Installation

Using LuaRocks



```
luarocks install tidesdb
```

Manual Installation



```
# Clone the repository
git clone https://github.com/tidesdb/tidesdb-lua.git

# Copy to your Lua package path
cp tidesdb-lua/src/tidesdb.lua /usr/local/share/lua/5.1/
```

Custom Installation Paths

If you installed TidesDB to a non-standard location, you can specify custom paths using environment variables:



```
# Set custom library path
export LD_LIBRARY_PATH="/custom/path/lib:$LD_LIBRARY_PATH" # Linux
# or
export DYLD_LIBRARY_PATH="/custom/path/lib:$DYLD_LIBRARY_PATH" # macOS
```

Custom prefix installation



```
# Install TidesDB to custom location
cd tidesdb
cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/opt/tidesdb
cmake --build build
sudo cmake --install build

# Configure library path
export LD_LIBRARY_PATH="/opt/tidesdb/lib:$LD_LIBRARY_PATH" # Linux
# or
export DYLD_LIBRARY_PATH="/opt/tidesdb/lib:$DYLD_LIBRARY_PATH" # macOS
```

Usage

Opening and Closing a Database

```
local tidesdb = require("tidesdb")

local db = tidesdb.TidesDB.open("./mydb", {
    num_flush_threads = 2,
    num_compaction_threads = 2,
    log_level = tidesdb.LogLevel.LOG_INFO,
    block_cache_size = 64 * 1024 * 1024,
    max_open_sstables = 256,
    max_memory_usage = 0, -- Global
memory limit in bytes (0 = auto, 50% of system RAM)
    log_to_file = false, -- Write
logs to file instead of stderr
    log_truncation_at = 24 * 1024 * 1024, -- Log
file truncation size (24MB), 0 = no truncation
    unified_memtable = false, -- Enable
unified memtable (shared across CFs)
    unified_memtable_write_buffer_size = 64 * 1024 * 1024, -- Unified
memtable buffer size
    unified_memtable_skip_list_max_level = 12, -- Skip
list max level for unified memtable
    unified_memtable_skip_list_probability = 0.25, -- Skip
list probability for unified memtable
    unified_memtable_sync_mode = tidesdb.SyncMode.SYNC_INTERVAL, -- Unified
memtable sync mode
    unified_memtable_sync_interval_us = 128000, -- Unified
memtable sync interval (128ms)
    max_concurrent_flushes = 0, -- Cap on
in-flight memtable flushes across all CFs (0 = library default)
    finish_compactions_on_close = false, -- false
(default) cancels in-flight compactions at close for a fast shutdown; true
lets them finish first
})

print("Database opened successfully")

db:close()
```

max_concurrent_flushes

`max_concurrent_flushes` is a global semaphore on the number of in-flight memtable flushes across all column families. It bounds total transient memory and work-queue depth when many column families flush at once. Leave it at `0` to inherit the library default (recommended), or lower it on memory-constrained hosts to throttle peak flush concurrency.

finish_compactions_on_close

Controls close behavior. `false` (default) cancels in-flight compactions at their next checkpoint for a fast shutdown - the merge discards its uncommitted output and leaves inputs intact, so no data is lost (recovery handles a mid-merge state the same way). `true` lets in-flight compactions run to completion before `db:close()` returns. For an explicit fast shutdown at runtime, see `db:cancel_background_work()`.

Raising the Open-File Limit

`tidesdb.raise_open_file_limit(desired)` raises this process's open-file ceiling toward `desired` descriptors so a database can keep more SSTables open. The engine sizes `max_open_sstables` to fit this at open time, so **call it before opening a database**. A `desired` value `<= 0` just reports the current ceiling. It returns the open-file ceiling in effect after the attempt. A failed or partial raise is non-fatal; TidesDB never raises the limit itself (it is an explicit, opt-in operator action).

```
local tidesdb = require("tidesdb")

local ceiling = tidesdb.raise_open_file_limit(65536)
print("open-file ceiling is now", ceiling)

local db = tidesdb.TidesDB.open("./mydb")
```

Custom Allocators (Optional)

TidesDB initializes itself lazily the first time you use it, so most programs never call `tidesdb.init`. Call it only to install custom allocation functions, and call it **before** any other TidesDB operation. Pass `nil` for any function to use the system default. It returns `0` on success or `-1` if the library was already initialized. `tidesdb.finalize()` finalizes the library and resets the allocator; afterwards `tidesdb.init()` may be called again.

```
-- Use system allocators explicitly (equivalent to lazy init)
tidesdb.init(nil, nil, nil, nil)

-- ... use the database ...

tidesdb.finalize()
```

Default Configuration

`tidesdb.default_config()` and `tidesdb.default_column_family_config()` mirror the underlying C library defaults. Values such as `max_concurrent_flushes`, `tombstone_density_min_entries`, and the unified-memtable settings track the engine's defaults automatically — if you previously hardcoded copies of these values, re-test after upgrading.

```
local cfg = tidesdb.default_config()
cfg.db_path = "./mydb"
cfg.log_level = tidesdb.LogLevel.LOG_WARN

local db = tidesdb.TidesDB.new(cfg)
```

Creating and Dropping Column Families

Column families are isolated key-value stores with independent configuration.

```

local cf_config = tidesdb.default_column_family_config()
db:create_column_family("my_cf", cf_config)

local cf_config = tidesdb.default_column_family_config()
cf_config.write_buffer_size = 128 * 1024 * 1024
cf_config.level_size_ratio = 10
cf_config.min_levels = 5
cf_config.dividing_level_offset = 2
cf_config.skip_list_max_level = 12
cf_config.skip_list_probability = 0.25
cf_config.compression_algorithm =
tidesdb.CompressionAlgorithm.LZ4_COMPRESSION
cf_config.enable_bloom_filter = true
cf_config.bloom_fpr = 0.01
cf_config.enable_block_indexes = true
cf_config.index_sample_ratio = 1
cf_config.block_index_prefix_len = 16
cf_config.sync_mode = tidesdb.SyncMode.SYNC_INTERVAL
cf_config.sync_interval_us = 128000
cf_config.klog_value_threshold = 512          -- Values >= 512 bytes go to
vlog
cf_config.min_disk_space = 100 * 1024 * 1024  -- Minimum disk space
required (100MB)
cf_config.default_isolation_level = tidesdb.IsolationLevel.READ_COMMITTED
cf_config.l1_file_count_trigger = 4           -- L1 file count trigger for
compaction
cf_config.l0_queue_stall_threshold = 20       -- L0 queue stall threshold
cf_config.tombstone_density_trigger = 0.0     -- Per-SSTable tombstone
density above which compaction priority escalates (0.0 = disabled, range
[0.0, 1.0])
cf_config.tombstone_density_min_entries = 1024 -- Minimum entry count for
an SSTable to be considered by the density trigger
cf_config.use_btree = false                   -- Use B+tree format for
klog (default: false)
cf_config.object_lazy_compaction = false      -- Enable lazy compaction
for object store
cf_config.object_prefetch_compaction = false  -- Enable prefetch during
object store compaction

db:create_column_family("my_cf", cf_config)

db:drop_column_family("my_cf")

```

Dropping a Column Family by Pointer

When you already hold a column family handle, use `delete_column_family` to skip the name lookup.

```
local cf = db:get_column_family("my_cf")

db:delete_column_family(cf)
```

Which to use

- `db:drop_column_family(name)` · convenient when you only have the name
- `db:delete_column_family(cf)` · faster when you already hold a column family handle, avoids a redundant linear scan

B+tree KLog Format (Optional)

Column families can optionally use a B+tree structure for the key log instead of the default block-based format. The B+tree klog format offers faster point lookups through $O(\log N)$ tree traversal rather than linear block scanning.

```
local cf_config = tidesdb.default_column_family_config()
cf_config.use_btree = true  -- Enable B+tree klog format

db:create_column_family("btree_cf", cf_config)
```

Characteristics

- Point lookups · $O(\log N)$ tree traversal with binary search at each node
- Range scans · Doubly-linked leaf nodes enable efficient bidirectional iteration
- Immutable · Tree is bulk-loaded from sorted memtable data during flush
- Compression · Nodes compress independently using the same algorithms

When to use B+tree klog format

- Read-heavy workloads with frequent point lookups
- Workloads where read latency is more important than write throughput
- Large SSTables where block scanning becomes expensive

Important

`use_btree` **cannot be changed** after column family creation. Different column families can use different formats.

Renaming Column Families

Atomically rename a column family and its underlying directory. The operation waits for any in-progress flush or compaction to complete before renaming.

```
db:rename_column_family("old_name", "new_name")

local cf = db:get_column_family("new_name")
```

Cloning Column Families

Create a complete copy of an existing column family with a new name. The clone contains all the data from the source at the time of cloning.

```
db:clone_column_family("source_cf", "cloned_cf")

local original = db:get_column_family("source_cf")
local clone = db:get_column_family("cloned_cf")
```

Behavior

- Flushes the source column family's memtable to ensure all data is on disk
- Waits for any in-progress flush or compaction to complete
- Copies all SSTable files to the new directory
- The clone is completely independent — modifications to one do not affect the other

Use cases

- Testing · Create a copy of production data for testing without affecting the original
- Branching · Create a snapshot of data before making experimental changes
- Migration · Clone data before schema or configuration changes
- Backup verification · Clone and verify data integrity without modifying the source

CRUD Operations

All operations in TidesDB are performed through transactions for ACID guarantees.

Writing Data

```
local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

txn:put(cf, "key", "value", -1)

txn:commit()
txn:free()
```

Writing with TTL

```
local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

local ttl = os.time() + 10

txn:put(cf, "temp_key", "temp_value", ttl)

txn:commit()
txn:free()
```

TTL Examples

```
local ttl = -1

local ttl = os.time() + 5 * 60

local ttl = os.time() + 60 * 60

local ttl = os.time({year=2026, month=12, day=31, hour=23, min=59, sec=59})
```

Reading Data

```
local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

local value = txn:get(cf, "key")

print("Value: " .. value)

txn:free()
```

Deleting Data

```
local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

txn:delete(cf, "key")

txn:commit()

txn:free()
```

Single-Delete

`txn:single_delete(cf, key)` writes a tombstone with the same read semantics as `txn:delete`, but carries a caller-provided promise that lets compaction drop the put and the tombstone together as soon as both appear in the same merge input, rather than carrying the tombstone forward until it reaches the largest active level.

Between any two single-deletes on the same key, and between the start of the key's history and its first single-delete, the key has been put **at most once**. The engine does not and cannot verify this at runtime; violating the contract can leave older puts visible after the single-delete and is a bug in the caller.

This is the right choice for workloads that insert each key exactly once and then delete it exactly once (classic insert-benchmark patterns, secondary-index entries on columns that are never updated, log-style tables with scheduled purges). It is **not** safe for tables that issue repeated updates to the same key.

```
local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

txn:single_delete(cf, "mykey")

txn:commit()
txn:free()
```

When in doubt, prefer `txn:delete`.

Multi-Operation Transactions

```
local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

local ok, err = pcall(function()
    txn:put(cf, "key1", "value1", -1)
    txn:put(cf, "key2", "value2", -1)
    txn:delete(cf, "old_key")
end)

if not ok then
    txn:rollback()
    error(err)
end

txn:commit()
txn:free()
```

Multi-Column-Family Transactions

TidesDB supports atomic transactions across multiple column families with true all-or-nothing semantics.

```
local users_cf = db:get_column_family("users")
local orders_cf = db:get_column_family("orders")

local txn = db:begin_txn()

txn:put(users_cf, "user:1000", "John Doe", -1)
txn:put(orders_cf, "order:5000", "user:1000|product:A", -1)

txn:commit()
txn:free()
```

Multi-CF guarantees

- Either all CFs commit or none do (atomic)
- Automatically detected when operations span multiple CFs
- Uses global sequence numbers for atomic ordering
- Each CF's WAL receives operations with the same commit sequence number

Iterating Over Data

Iterators provide efficient bidirectional traversal over key-value pairs.

Forward Iteration

```
local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

local iter = txn:new_iterator(cf)

iter:seek_to_first()

while iter:valid() do
    local key = iter:key()
    local value = iter:value()

    print(string.format("Key: %s, Value: %s", key, value))

    iter:next()
end

iter:free()
txn:free()
```

Backward Iteration

```
local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

local iter = txn:new_iterator(cf)

iter:seek_to_last()

while iter:valid() do
    local key = iter:key()
    local value = iter:value()

    print(string.format("Key: %s, Value: %s", key, value))

    iter:prev()
end

iter:free()
txn:free()
```

Seek to Specific Key

`iter:seek(key)` positions the iterator at the first key $\geq$ target key.

```
local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

local iter = txn:new_iterator(cf)

iter:seek("user:1000")

if iter:valid() then
    local key = iter:key()
    print("Found: " .. key)
end

iter:free()
txn:free()
```

`iter:seek_for_prev(key)` positions the iterator at the last key $\leq$ target key.

```
local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

local iter = txn:new_iterator(cf)

iter:seek_for_prev("user:2000")

while iter:valid() do
    local key = iter:key()
    print("Found: " .. key)
    iter:prev()
end

iter:free()
txn:free()
```

Prefix Seeking

Since `iter:seek(key)` positions the iterator at the first key $\geq$ target, you can use a prefix as the seek target to efficiently scan all keys sharing that prefix:


```
local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

local iter = txn:new_iterator(cf)

local prefix = "user:"
iter:seek(prefix)

while iter:valid() do
    local key = iter:key()
    if key:sub(1, #prefix) ~= prefix then break end

    local value = iter:value()
    print(string.format("Key: %s, Value: %s", key, value))

    iter:next()
end

iter:free()
txn:free()
```

Combined Key-Value Access

`iter:key_value()` retrieves both the key and value in a single call, which is more efficient than calling `iter:key()` and `iter:value()` separately.

```
local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

local iter = txn:new_iterator(cf)

iter:seek_to_first()

while iter:valid() do
    local key, value = iter:key_value()

    print(string.format("Key: %s, Value: %s", key, value))

    iter:next()
end

iter:free()
txn:free()
```

Getting Column Family Statistics

Retrieve detailed statistics about a column family.

```

local cf = db:get_column_family("my_cf")

local stats = cf:get_stats()

print(string.format("Number of Levels: %d", stats.num_levels))
print(string.format("Memtable Size: %d bytes", stats.memtable_size))
print(string.format("Total Keys: %d", stats.total_keys))
print(string.format("Total Data Size: %d bytes", stats.total_data_size))
print(string.format("Average Key Size: %.2f bytes", stats.avg_key_size))
print(string.format("Average Value Size: %.2f bytes",
stats.avg_value_size))
print(string.format("Read Amplification: %.2f", stats.read_amp))
print(string.format("Hit Rate: %.2f%%", stats.hit_rate * 100))

for i, size in ipairs(stats.level_sizes) do
    print(string.format("Level %d: %d bytes, %d SSTables, %d keys",
        i, size, stats.level_num_sstables[i], stats.level_key_counts[i]))
end

if stats.use_btree then
    print(string.format("B+tree Total Nodes: %d", stats.btree_total_nodes))
    print(string.format("B+tree Max Height: %d", stats.btree_max_height))
    print(string.format("B+tree Avg Height: %.2f", stats.btree_avg_height))
end

-- Tombstone density observability
print(string.format("Total Tombstones: %d", stats.total_tombstones))
print(string.format("Tombstone Ratio: %.2f%%", stats.tombstone_ratio *
100))
print(string.format("Worst SSTable Density: %.2f%% at level %d",
    stats.max_sst_density * 100, stats.max_sst_density_level))
for i, count in ipairs(stats.level_tombstone_counts) do
    print(string.format("Level %d tombstones: %d", i, count))
end

if stats.config then
    print(string.format("Write Buffer Size: %d",
stats.config.write_buffer_size))
    print(string.format("Compression: %d",
stats.config.compression_algorithm))
    print(string.format("Bloom Filter: %s",
tostring(stats.config.enable_bloom_filter)))
    print(string.format("Sync Mode: %d", stats.config.sync_mode))
end

```

```

        print(string.format("Use B+tree: %s",
tostring(stats.config.use_btree)))
end

```

Statistics Fields

- `num_levels` · Number of LSM levels
- `memtable_size` · Current memtable size in bytes
- `level_sizes` · Array of sizes per level
- `level_num_sstables` · Array of SSTable counts per level
- `level_key_counts` · Array of key counts per level
- `total_keys` · Total keys across memtable and all SSTables
- `total_data_size` · Total data size (klog + vlog) across all SSTables
- `avg_key_size` · Average key size in bytes
- `avg_value_size` · Average value size in bytes
- `read_amp` · Read amplification (point lookup cost multiplier)
- `hit_rate` · Cache hit rate (0.0 if cache disabled)
- `use_btree` · Whether column family uses B+tree klog format
- `btree_total_nodes` · Total B+tree nodes across all SSTables (only if `use_btree=true`)
- `btree_max_height` · Maximum tree height across all SSTables (only if `use_btree=true`)
- `btree_avg_height` · Average tree height across all SSTables (only if `use_btree=true`)
- `total_tombstones` · Sum of `tombstone_count` across every SSTable in the column family
- `tombstone_ratio` · `total_tombstones / total_keys` (range `[0.0, 1.0]`, 0.0 if `total_keys == 0`)
- `level_tombstone_counts` · Per-level tombstone counts (parallels `level_key_counts`)
- `max_sst_density` · Worst per-SSTable tombstone density observed in the column family (range `[0.0, 1.0]`)
- `max_sst_density_level` · 1-based level index where `max_sst_density` was observed (0 if none)
- `config` · Column family configuration

Write-amplification counters (lifetime since open, on-disk framed bytes; divide the write totals by `user_bytes_written` for this column family's write amplification)

- `wal_bytes_written` · Framed bytes appended to this CF's WAL (0 in unified memtable mode - see `uwal_bytes_written` in database stats)
- `flush_bytes_written` · On-disk bytes this CF's flushes wrote to L0 SSTables
- `compaction_bytes_written` · On-disk bytes this CF's compactions wrote

- `compaction_bytes_read` · On-disk bytes this CF's compactions read as input
- `user_bytes_written` · Logical key+value bytes committed to this CF (the write-amplification denominator)
- `flush_count` · Number of flushed SSTables produced by this CF
- `compaction_count` · Number of compaction output SSTables produced by this CF

Getting Database-Level Statistics

Get aggregate statistics across the entire database instance.

```

local db_stats = db:get_db_stats()

print(string.format("Column families: %d", db_stats.num_column_families))
print(string.format("Total memory: %d bytes", db_stats.total_memory))
print(string.format("Resolved memory limit: %d bytes",
db_stats.resolved_memory_limit))
print(string.format("Memory pressure level: %d",
db_stats.memory_pressure_level))
print(string.format("Global sequence: %d", db_stats.global_seq))
print(string.format("Flush queue: %d pending", db_stats.flush_queue_size))
print(string.format("Compaction queue: %d pending",
db_stats.compaction_queue_size))
print(string.format("Total SSTables: %d", db_stats.total_sstable_count))
print(string.format("Total data size: %d bytes",
db_stats.total_data_size_bytes))
print(string.format("Open SSTable handles: %d",
db_stats.num_open_sstables))
print(string.format("In-flight txn memory: %d bytes",
db_stats.txn_memory_bytes))
print(string.format("Immutable memtables: %d",
db_stats.total_immutable_count))
print(string.format("Memtable bytes: %d", db_stats.total_memtable_bytes))

-- Unified memtable stats
if db_stats.unified_memtable_enabled then
    print(string.format("Unified memtable bytes: %d",
db_stats.unified_memtable_bytes))
    print(string.format("Unified immutable count: %d",
db_stats.unified_immutable_count))
    print(string.format("Unified is flushing: %s",
toString(db_stats.unified_is_flushing)))
    print(string.format("Unified next CF index: %d",
db_stats.unified_next_cf_index))
    print(string.format("Unified WAL generation: %d",
db_stats.unified_wal_generation))
end

-- Object store stats
if db_stats.object_store_enabled then
    print(string.format("Object store connector: %s",
db_stats.object_store_connector or "N/A"))
    print(string.format("Local cache used: %d bytes",
db_stats.local_cache_bytes_used))

```

```

    print(string.format("Local cache max: %d bytes",
db_stats.local_cache_bytes_max))
    print(string.format("Total uploads: %d", db_stats.total_uploads))
    print(string.format("Upload failures: %d",
db_stats.total_upload_failures))
end

print(string.format("Replica mode: %s", tostring(db_stats.replica_mode)))

```

Database statistics fields

- `num_column_families` · Number of column families
- `total_memory` · System total memory
- `available_memory` · System available memory at open time
- `resolved_memory_limit` · Resolved memory limit (auto or configured)
- `memory_pressure_level` · Current memory pressure (0=normal, 1=elevated, 2=high, 3=critical)
- `flush_pending_count` · Number of pending flush operations (queued + in-flight)
- `total_memtable_bytes` · Total bytes in active memtables across all CFs
- `total_immutable_count` · Total immutable memtables across all CFs
- `total_sstable_count` · Total SSTables across all CFs and levels
- `total_data_size_bytes` · Total data size (klog + vlog) across all CFs
- `num_open_sstables` · Number of currently open SSTable file handles
- `global_seq` · Current global sequence number
- `txn_memory_bytes` · Bytes held by in-flight transactions
- `compaction_queue_size` · Number of pending compaction tasks
- `flush_queue_size` · Number of pending flush tasks in queue
- `unified_memtable_enabled` · Whether unified memtable is enabled
- `unified_memtable_bytes` · Bytes in the unified memtable
- `unified_immutable_count` · Number of unified immutable memtables
- `unified_is_flushing` · Whether unified memtable is currently flushing
- `unified_next_cf_index` · Next column family index for unified memtable round-robin
- `unified_wal_generation` · Current WAL generation number for unified memtable
- `object_store_enabled` · Whether object store is enabled
- `object_store_connector` · Object store connector name (or nil)
- `local_cache_bytes_used` · Bytes used in the local object cache
- `local_cache_bytes_max` · Maximum bytes for the local object cache

- `local_cache_num_files` · Number of files in the local cache
- `last_uploaded_generation` · Last WAL generation uploaded to object store
- `upload_queue_depth` · Number of pending uploads in the queue
- `total_uploads` · Total number of successful uploads
- `total_upload_failures` · Total number of failed uploads
- `replica_mode` · Whether the database is in replica mode
- `primary_epoch` · Single-writer lease epoch this primary currently holds (0 when not a primary / no lease)
- `seen_epoch` · Highest lease epoch a replica has observed

Write-amplification counters (lifetime since open, on-disk framed bytes; db-wide write amplification)

$$= \frac{(\text{uwal_bytes_written} + \text{wal_bytes_written} + \text{flush_bytes_written} + \text{compaction_bytes_written})}{\text{user_bytes_written}}$$

- `uwal_bytes_written` · Framed bytes appended to the shared unified WAL (0 when unified memtable mode is off)
- `wal_bytes_written` · Per-CF WAL bytes summed across all column families
- `flush_bytes_written` · Flush output bytes summed across all column families
- `compaction_bytes_written` · Compaction output bytes summed across all column families
- `compaction_bytes_read` · Compaction input bytes summed across all column families
- `user_bytes_written` · Logical committed bytes summed across all column families
- `flush_count` · Flushed SSTables summed across all column families
- `compaction_count` · Compaction output SSTables summed across all column families

Stack Allocated

Unlike `cf:get_stats()` (which heap-allocates), `db:get_db_stats()` fills a caller-provided struct on the stack. No free is needed.

Getting Block Cache Statistics

Get statistics for the global block cache (shared across all column families).


```

local cache_stats = db:get_cache_stats()

if cache_stats.enabled then
    print("Cache enabled: yes")
    print(string.format("Total entries: %d", cache_stats.total_entries))
    print(string.format("Total bytes: %.2f MB", cache_stats.total_bytes /
(1024 * 1024)))
    print(string.format("Hits: %d", cache_stats.hits))
    print(string.format("Misses: %d", cache_stats.misses))
    print(string.format("Hit rate: %.1f%%", cache_stats.hit_rate * 100))
    print(string.format("Partitions: %d", cache_stats.num_partitions))
else
    print("Cache enabled: no (block_cache_size = 0)")
end

```

Range Cost Estimation

Estimate the computational cost of iterating between two keys in a column family. The returned value is an opaque double - meaningful only for comparison with other values from the same function. It uses only in-memory metadata and performs no disk I/O.

```

local cf = db:get_column_family("my_cf")

local cost_a = cf:range_cost("user:0000", "user:0999")
local cost_b = cf:range_cost("user:1000", "user:1099")

if cost_a < cost_b then
    print("Range A is cheaper to iterate")
end

```

How it works

- With block indexes enabled · Uses $O(\log B)$ binary search per overlapping SSTable to find the block slots containing each key bound
- Without block indexes · Falls back to byte-level key interpolation using the leading 8 bytes of each key
- B+tree SSTables · Uses key interpolation against tree node counts, plus tree height as a seek cost
- Compression · Compressed SSTables receive a $1.5\times$ weight multiplier to account for decompression overhead

- Key order does not matter - the function normalizes the range internally

Use cases

- Query planning · Compare candidate key ranges to find the cheapest one to scan
- Load balancing · Distribute range scan work across threads by estimating per-range cost
- Adaptive prefetching · Decide how aggressively to prefetch based on range size
- Monitoring · Track how data distribution changes across key ranges over time

Cost Values

The returned cost is not an absolute measure (it does not represent milliseconds, bytes, or entry counts). It is a relative scalar - only meaningful when compared with other `cf:range_cost` results. A cost of 0.0 means no overlapping SSTables or memtable entries were found for the range.

Listing Column Families

```
local cf_list = db:list_column_families()

print("Available column families:")
for _, name in ipairs(cf_list) do
    print("  - " .. name)
end
```

Compaction

Manual Compaction

```
local cf = db:get_column_family("my_cf")

local ok, err = pcall(function()
    cf:compact()
end)
if not ok then
    print("Compaction note: " .. tostring(err))
end
```

Targeted Range Compaction

`cf:compact_range(start_key, end_key)` runs a synchronous compaction over a specific key range. Only SSTables whose minimum and maximum keys overlap the requested range participate in the merge, so the work and I/O are bounded to the affected portion of the LSM tree rather than the whole column family.

```
local cf = db:get_column_family("my_cf")

local start_key = "tenant_42:"
local end_key = "tenant_42;"

cf:compact_range(start_key, end_key)
```

Pass `nil` for either endpoint to make that side unbounded:

```
-- Compact everything from "tenant_42:" upward
cf:compact_range("tenant_42:", nil)

-- Compact everything up to "tenant_42;"
cf:compact_range(nil, "tenant_42;")
```

When to use

- Bulk reclaim after a large range delete, where waiting for natural compaction would leave tombstones and obsolete versions on disk
- Tenant eviction or sliding-window expiration that does not fit TTL semantics
- Post-import cleanup of a known key range loaded with `txn:put` followed by `txn:delete`
- Operational counterpart to the automatic tombstone density trigger when an operator wants reclaim now rather than at the next natural threshold crossing

Behavior

- Synchronous, blocks the caller until the merge commits or fails
- Does not enqueue work onto the compaction thread pool, the calling thread does the work
- Selects only SSTables whose key range overlaps `[start_key, end_key)` using the column family's comparator, SSTables outside the range are not touched
- Applies the same emit-loop logic as background compactions (tombstone reclamation rules, single-delete pair cancellation, sequence-based deduplication, value recompression)

- Output SSTables are committed to the manifest atomically and old inputs are marked for deletion

Return values

- Returns nothing on success
- Raises `TDB_ERR_INVALID_ARGS` if both endpoints are `nil` or empty (use `cf:compact()` for full CF compaction)
- Raises `TDB_ERR_LOCKED` if another compaction is running for the column family
- Raises a standard I/O or memory error if the merge cannot complete

Manual Memtable Flush

```
local cf = db:get_column_family("my_cf")

local ok, err = pcall(function()
    cf:flush_memtable()
end)
if not ok then
    print("Flush note: " .. tostring(err))
end
```

Checking Flush/Compaction Status

Check if a column family currently has flush or compaction operations in progress.

```
local cf = db:get_column_family("my_cf")

if cf:is_flushing() then
    print("Flush in progress")
end

if cf:is_compacting() then
    print("Compaction in progress")
end

while cf:is_flushing() or cf:is_compacting() do
    os.execute("sleep 0.1")
end
print("Background operations completed")
```

Use cases

- Graceful shutdown · Wait for background operations to complete before closing
- Maintenance windows · Check if operations are running before triggering manual compaction
- Monitoring · Track background operation status for observability

Purge Column Family

`cf:purge()` forces a synchronous flush and aggressive compaction for a single column family. Unlike `cf:flush_memtable()` and `cf:compact()` (which are non-blocking), `purge` blocks until all flush and compaction I/O is complete.

```
local cf = db:get_column_family("my_cf")

cf:purge()
-- All data is now flushed to SSTables and compacted
```

Behavior

1. Waits for any in-progress flush to complete
2. Force-flushes the active memtable (even if below threshold)
3. Waits for flush I/O to fully complete
4. Waits for any in-progress compaction to complete
5. Triggers synchronous compaction inline (bypasses the compaction queue)
6. Waits for any queued compaction to drain

When to use

- Before backup or checkpoint · Ensure all data is on disk and compacted
- After bulk deletes · Reclaim space immediately by compacting away tombstones
- Manual maintenance · Force a clean state during a maintenance window
- Pre-shutdown · Ensure all pending work is complete before closing

Purge Database

`db:purge()` forces a synchronous flush and aggressive compaction for **all** column families, then drains both the global flush and compaction queues.

```
db:purge()  
-- All CFs flushed and compacted, all queues drained
```

Behavior

1. Calls `cf:purge()` on each column family
2. Drains the global flush queue (waits for queue size and pending count to reach 0)
3. Drains the global compaction queue (waits for queue size to reach 0)

Purge vs Manual Flush + Compact

`cf:flush_memtable()` and `cf:compact()` are non-blocking - they enqueue work and return immediately. `cf:purge()` and `db:purge()` are synchronous - they block until all work is complete. Use purge when you need a guarantee that all data is on disk and compacted before proceeding.

Cancel Background Work (Fast Shutdown)

`db:cancel_background_work()` cancels background compaction db-wide. In-flight merges bail safely and queued compaction is skipped; flushes are unaffected so durability is preserved. It blocks (bounded) until compaction is idle. The cancellation is sticky for the session and reset on the next open - it is intended to be called right before `db:close()` for a fast shutdown.

```
db:cancel_background_work() -- stop compaction so close returns  
quickly  
db:close()
```

This is the runtime counterpart to the `finish_compactions_on_close` config flag: set `finish_compactions_on_close = true` to let in-flight compactions run to completion before `db:close()` returns, or call `db:cancel_background_work()` for the opposite (fastest) behavior.

Promote Replica to Primary

`db:promote_to_primary()` switches a replica database to primary mode, allowing it to accept writes.

```
db:promote_to_primary()
```

Behavior

- Transitions a read-only replica database to a writable primary
- Should only be called on databases opened in replica mode
- Calling on a non-replica database may return an error

Use cases

- Failover · Promote a replica to primary when the original primary becomes unavailable
- Maintenance · Temporarily promote a replica for maintenance operations

Object Store Mode

Object store mode allows TidesDB to store SSTables in a remote object store (S3, MinIO, GCS, or any S3-compatible service) while using local disk as a cache. This separates compute from storage and enables cold start recovery from the remote store.

Object Store Configuration

Use `tidesdb.default_objstore_config()` for sensible defaults, then override fields as needed.

```
local os_config = tidesdb.default_objstore_config()
```

Configuration fields

- `local_cache_path` · Local directory for cached SSTable files (nil = use db_path)
- `local_cache_max_bytes` · Maximum local cache size in bytes (0 = unlimited)
- `cache_on_read` · Cache downloaded files locally (default: true)
- `cache_on_write` · Keep local copy after upload (default: true)
- `max_concurrent_uploads` · Number of parallel upload threads (default: 4)
- `max_concurrent_downloads` · Number of parallel download threads (default: 8)
- `multipart_threshold` · Use multipart upload above this size in bytes (default: 64MB)
- `multipart_part_size` · Chunk size for multipart uploads in bytes (default: 8MB)
- `sync_manifest_to_object` · Upload MANIFEST after each compaction (default: true)
- `replicate_wal` · Upload closed WAL segments for replication (default: true)
- `wal_upload_sync` · Block flush until WAL is uploaded (default: false)
- `wal_sync_threshold_bytes` · Sync active WAL when it grows by this many bytes (default: 1MB, 0 = off)

- `wal_sync_on_commit` · Upload WAL after every txn commit for RPO=0 replication (default: false)
- `replica_mode` · Enable read-only replica mode (default: false)
- `replica_sync_interval_us` · MANIFEST poll interval in microseconds (default: 5000000 / 5s)
- `replica_replay_wal` · Replay WAL from object store for near-real-time reads on replicas (default: true)

Enabling Object Store Mode (Filesystem Connector)

```
local tidesdb = require("tidesdb")

-- Create a filesystem connector (for testing and local replication)
local store = tidesdb.objstore_fs_create("/mnt/nfs/tidesdb-objects")

-- Configure object store behavior
local os_config = tidesdb.default_objstore_config()
os_config.local_cache_max_bytes = 512 * 1024 * 1024 -- 512MB local cache
os_config.max_concurrent_uploads = 8

local db = tidesdb.TidesDB.open("./mydb", {
    object_store = store,
    object_store_config = os_config,
})

-- Use the database normally -- SSTables are uploaded after flush

db:close()
```

Unified Memtable

Object store mode requires unified memtable mode. Setting `object_store` on the config automatically enables `unified_memtable`.

Enabling Object Store Mode (S3 Connector)

For AWS S3, MinIO, or any S3-compatible service, use `tidesdb.objstore_s3_create` (positional) or `tidesdb.objstore_s3_create_config` (full configuration with TLS and multipart tuning).


```

local tidesdb = require("tidesdb")

-- Positional form
local store = tidesdb.objstore_s3_create({
    endpoint = "s3.amazonaws.com",    -- or "minio.local:9000"
    bucket = "my-bucket",
    prefix = "production/db1/",        -- optional, may be nil
    access_key = "AKIA...",
    secret_key = "...",
    region = "us-east-1",              -- nil for MinIO
    use_ssl = true,                    -- HTTPS
    use_path_style = false,            -- true for MinIO, false for AWS
    virtual-hosted
})

local db = tidesdb.TidesDB.open("./mydb", {
    object_store = store,
    object_store_config = tidesdb.default_objstore_config(),
})

```

The configuration form additionally exposes TLS and multipart controls:

```

local store = tidesdb.objstore_s3_create_config({
    endpoint = "minio.local:9000",
    bucket = "my-bucket",
    access_key = "minioadmin",
    secret_key = "minioadmin",
    use_ssl = true,
    use_path_style = true,
    tls_ca_path = "/etc/ssl/certs/ca.pem",    -- custom CA bundle, or nil
    for the system bundle
    tls_insecure_skip_verify = false,          -- true disables TLS
    verification (test only, insecure)
    multipart_threshold = 64 * 1024 * 1024,   -- 0 = built-in default
    multipart_part_size = 8 * 1024 * 1024,    -- 0 = built-in default
})

```

S3 build flag

The S3 connector is only available when the TidesDB C library was built with `TIDESDB_WITH_S3=ON`. If the library lacks S3 support, both factory functions raise a `TidesDBError` explaining that S3 support is unavailable.

Per-CF Object Store Tuning

Column family configurations include three object store tuning fields:

```
local cf_config = tidesdb.default_column_family_config()
cf_config.object_lazy_compaction = true           -- Compact less
                                                    aggressively for remote storage
cf_config.object_prefetch_compaction = true       -- Download all
inputs before compaction merge

db:create_column_family("my_cf", cf_config)
```

Replica Mode

Replica mode enables read-only nodes that follow a primary through the object store. The primary handles all writes while replicas poll for MANIFEST updates and replay WAL segments for near-real-time reads.

```

-- Configure as a replica
local os_config = tidesdb.default_objstore_config()
os_config.replica_mode = true
os_config.replica_sync_interval_us = 1000000    -- 1 second sync interval
os_config.replica_replay_wal = true             -- Replay WAL for fresh
reads

local db = tidesdb.TidesDB.open("./mydb_replica", {
    object_store = store, -- Same store as the primary
    object_store_config = os_config,
})

-- Reads work normally
local cf = db:get_column_family("my_cf")
local txn = db:begin_txn()
local value = txn:get(cf, "key1")
txn:free()

-- Writes are rejected with TDB_ERR_READONLY

```

Sync-on-Commit WAL (Primary Side)

For tighter replication lag, enable sync-on-commit on the primary so every committed write is uploaded to the object store immediately.

```

local os_config = tidesdb.default_objstore_config()
os_config.wal_sync_on_commit = true -- RPO = 0, every commit is durable in
the object store

-- Replica sees committed data within one replica_sync_interval_us

```

Cold Start Recovery

When the local database directory is empty but a connector is configured, TidesDB automatically discovers column families from the object store during recovery.

```
-- Delete all local state
os.execute("rm -rf ./mydb")

-- Reopen with the same connector -- cold start recovery
local db = tidesdb.TidesDB.open("./mydb", {
    object_store = store,
    object_store_config = os_config,
})

-- All data is available -- SSTables are fetched from the object store on
demand
local cf = db:get_column_family("my_cf")
```

Manual WAL Sync

`cf:sync_wal()` forces an immediate fsync of the active write-ahead log for a column family. This is useful for explicit durability control when using `SYNC_NONE` or `SYNC_INTERVAL` modes.

```
local cf = db:get_column_family("my_cf")

-- Force WAL durability after a batch of writes
cf:sync_wal()
```

When to use

- Application-controlled durability · Sync the WAL at specific points (e.g., after a batch of related writes) when using `SYNC_NONE` or `SYNC_INTERVAL`
- Pre-checkpoint · Ensure all buffered WAL data is on disk before taking a checkpoint
- Graceful shutdown · Flush WAL buffers before closing the database
- Critical writes · Force durability for specific high-value writes without using `SYNC_FULL` for all writes

Behavior

- Acquires a reference to the active memtable to safely access its WAL
- Calls `fdatasync` on the WAL file descriptor
- Thread-safe - can be called concurrently from multiple threads
- If the memtable rotates during the call, retries with the new active memtable

Sync Modes

Control the durability vs performance tradeoff.

```
local cf_config = tidesdb.default_column_family_config()

cf_config.sync_mode = tidesdb.SyncMode.SYNC_NONE

cf_config.sync_mode = tidesdb.SyncMode.SYNC_INTERVAL
cf_config.sync_interval_us = 128000 -- Sync every 128ms

cf_config.sync_mode = tidesdb.SyncMode.SYNC_FULL

db:create_column_family("my_cf", cf_config)
```

Compression Algorithms

TidesDB supports multiple compression algorithms:

```
local cf_config = tidesdb.default_column_family_config()

cf_config.compression_algorithm =
tidesdb.CompressionAlgorithm.NO_COMPRESSION
cf_config.compression_algorithm =
tidesdb.CompressionAlgorithm.SNAPPY_COMPRESSION -- Not available on
SunOS/Illumos
cf_config.compression_algorithm =
tidesdb.CompressionAlgorithm.LZ4_COMPRESSION -- Default, balanced
cf_config.compression_algorithm =
tidesdb.CompressionAlgorithm.LZ4_FAST_COMPRESSION -- Faster, slightly lower
ratio
cf_config.compression_algorithm =
tidesdb.CompressionAlgorithm.ZSTD_COMPRESSION -- Best ratio, moderate
speed

db:create_column_family("my_cf", cf_config)
```

Choosing a Compression Algorithm

Workload	Recommended	Rationale
General purpose	<code>LZ4_COMPRESSION</code>	Best balance of speed and compression
Write-heavy	<code>LZ4_FAST_COMPRESSION</code>	Minimize CPU overhead on writes
Storage-constrained	<code>ZSTD_COMPRESSION</code>	Maximum compression ratio
Pre-compressed data	<code>NO_COMPRESSION</code>	Avoid double compression overhead

Database Backup

Create an on-disk snapshot of an open database without blocking normal reads/writes.

```
db:backup("./mydb_backup")

local backup_db = tidesdb.TidesDB.open("./mydb_backup")
```

Behavior

- Requires the backup directory to be non-existent or empty
- Does not copy the `LOCK` file, so the backup can be opened normally
- Database stays open and usable during backup
- The backup represents the database state after the final flush/compaction drain

Database Checkpoint

Create a lightweight, near-instant snapshot of an open database using hard links instead of copying SSTable data.

```
db:checkpoint("./mydb_checkpoint")

local checkpoint_db = tidesdb.TidesDB.open("./mydb_checkpoint")
```

Behavior

- Requires the checkpoint directory to be non-existent or empty
- For each column family:
 - Flushes the active memtable so all data is in SSTables
 - Halts compactions to ensure a consistent view of live SSTable files
 - Hard links all SSTable files (`.klog` and `.vlog`) into the checkpoint directory

- Copies small metadata files (manifest, config) into the checkpoint directory
- Resumes compactions
- Falls back to file copy if hard linking fails (e.g., cross-filesystem)
- Database stays open and usable during checkpoint

Checkpoint vs Backup

	<code>db:backup(dir)</code>	<code>db:checkpoint(dir)</code>
Speed	Copies every SSTable byte-by-byte	Near-instant (hard links, O(1) per file)
Disk usage	Full independent copy	No extra disk until compaction removes old SSTables
Portability	Can be moved to another filesystem or machine	Same filesystem only (hard link requirement)
Use case	Archival, disaster recovery, remote shipping	Fast local snapshots, point-in-time reads, streaming backups

Updating Runtime Configuration

Update runtime-safe configuration settings for a column family. Changes apply to new operations only.

```
local cf = db:get_column_family("my_cf")

local new_config = tidesdb.default_column_family_config()
new_config.write_buffer_size = 256 * 1024 * 1024 -- 256MB
new_config.bloom_fpr = 0.001 -- 0.1% false positive rate

cf:update_runtime_config(new_config, true)
```

Updatable settings (safe to change at runtime):

- `write_buffer_size` · Memtable flush threshold
- `skip_list_max_level` · Skip list level for new memtables
- `skip_list_probability` · Skip list probability for new memtables
- `bloom_fpr` · False positive rate for new SSTables
- `index_sample_ratio` · Index sampling ratio for new SSTables
- `sync_mode` · Durability mode
- `sync_interval_us` · Sync interval in microseconds

Non-updatable settings (would corrupt existing data):

- `compression_algorithm`, `enable_block_indexes`, `enable_bloom_filter`, `comparator_name`, `level_size_ratio`, `klog_value_threshold`, `min_levels`, `dividing_level_offset`, `block_index_prefix_len`, `l1_file_count_trigger`, `l0_queue_stall_threshold`, `use_btree`

Commit Hook (Change Data Capture)

`cf:set_commit_hook` registers a callback that fires synchronously after every transaction commit on a column family. The hook receives the full batch of committed operations atomically, enabling real-time change data capture without WAL parsing.


```

local ffi = require("ffi")

local cf = db:get_column_family("my_cf")

local my_hook = ffi.cast("tidesdb_commit_hook_fn", function(ops, num_ops,
commit_seq, ctx)
    for i = 0, num_ops - 1 do
        local key = ffi.string(ops[i].key, ops[i].key_size)
        if ops[i].is_delete ~= 0 then
            print(string.format("[seq=%d] DELETE %s", tonumber(commit_seq),
key))
        else
            local value = ffi.string(ops[i].value, ops[i].value_size)
            print(string.format("[seq=%d] PUT %s = %s",
tonumber(commit_seq), key, value))
        end
    end
    return 0
end)

-- Attach hook
cf:set_commit_hook(my_hook, nil)

-- Normal writes now trigger the hook automatically
local txn = db:begin_txn()
txn:put(cf, "key1", "value1", -1)
txn:commit() -- my_hook fires here
txn:free()

-- Detach hook
cf:clear_commit_hook()

-- Free the callback when no longer needed
my_hook:free()

```

Operation fields (available inside the callback)

- `ops[i].key` / `ops[i].key_size` · Key data and size
- `ops[i].value` / `ops[i].value_size` · Value data and size (`NULL`/0 for deletes)
- `ops[i].ttl` · Time-to-live for the entry
- `ops[i].is_delete` · 1 for delete operations, 0 for puts

Behavior

- The hook fires after WAL write, memtable apply, and commit status marking are complete - data is fully durable before the callback runs
- Hook failure (non-zero return) is logged but does not affect the commit result
- Each column family has its own independent hook; a multi-CF transaction fires the hook once per CF with only that CF's operations
- `commit_seq` is monotonically increasing across commits and can be used as a replication cursor
- Pointers in the operation struct are valid only during the callback invocation - copy any data you need to retain
- The hook executes synchronously on the committing thread; keep the callback fast to avoid stalling writers
- Setting the hook to `nil` via `cf:clear_commit_hook()` disables it immediately

Use cases

- Replication · Ship committed batches to replicas in commit order
- Event streaming · Publish mutations to Kafka, NATS, or any message broker
- Secondary indexing · Maintain a reverse index or materialized view
- Audit logging · Record every mutation with key, value, TTL, and sequence number
- Debugging · Attach a temporary hook in production to inspect live writes

Runtime-Only

Commit hooks are not persisted. After a database restart, hooks must be re-registered by the application. This is by design - function pointers cannot be serialized.

Configuration File Operations

Load and save column family configurations from/to INI files.

```
local config = tidesdb.load_config_from_ini("config.ini", "my_cf")

tidesdb.save_config_to_ini("config.ini", "my_cf", config)
```

Error Handling

```
local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

local ok, err = pcall(function()
    txn:put(cf, "key", "value", -1)
end)

if not ok then
    print("Error: " .. tostring(err))
    txn:rollback()
    return
end

txn:commit()
txn:free()
```

Error Codes

- `TDB_SUCCESS` (0) · Operation successful
- `TDB_ERR_MEMORY` (-1) · Memory allocation failed
- `TDB_ERR_INVALID_ARGS` (-2) · Invalid arguments
- `TDB_ERR_NOT_FOUND` (-3) · Key not found
- `TDB_ERR_IO` (-4) · I/O error
- `TDB_ERR_CORRUPTION` (-5) · Data corruption
- `TDB_ERR_EXISTS` (-6) · Resource already exists
- `TDB_ERR_CONFLICT` (-7) · Transaction conflict
- `TDB_ERR_TOO_LARGE` (-8) · Key or value too large
- `TDB_ERR_MEMORY_LIMIT` (-9) · Memory limit exceeded
- `TDB_ERR_INVALID_DB` (-10) · Invalid database handle
- `TDB_ERR_UNKNOWN` (-11) · Unknown error
- `TDB_ERR_LOCKED` (-12) · Database is locked
- `TDB_ERR_READONLY` (-13) · Database is read-only
- `TDB_ERR_BUSY` (-14) · Resource busy
- `TDB_ERR_PRECONDITION` (-15) · Precondition failed

Complete Example

```
local tidesdb = require("tidesdb")

local db = tidesdb.TidesDB.open("./example_db", {
    num_flush_threads = 1,
    num_compaction_threads = 1,
    log_level = tidesdb.LogLevel.LOG_INFO,
    block_cache_size = 64 * 1024 * 1024,
    max_open_sstables = 256,
})

local cf_config = tidesdb.default_column_family_config()
cf_config.write_buffer_size = 64 * 1024 * 1024
cf_config.compression_algorithm =
tidesdb.CompressionAlgorithm.LZ4_COMPRESSION
cf_config.enable_bloom_filter = true
cf_config.bloom_fpr = 0.01
cf_config.sync_mode = tidesdb.SyncMode.SYNC_INTERVAL
cf_config.sync_interval_us = 128000

db:create_column_family("users", cf_config)

local cf = db:get_column_family("users")

local txn = db:begin_txn()

txn:put(cf, "user:1", "Alice", -1)
txn:put(cf, "user:2", "Bob", -1)

local ttl = os.time() + 30 -- Expire in 30 seconds
txn:put(cf, "session:abc", "temp_data", ttl)

txn:commit()
txn:free()

local read_txn = db:begin_txn()

local value = read_txn:get(cf, "user:1")
print("user:1 = " .. value)

local iter = read_txn:new_iterator(cf)

print("\nAll entries:")
```

```

iter:seek_to_first()
while iter:valid() do
    local key = iter:key()
    local val = iter:value()
    print(string.format(" %s = %s", key, val))
    iter:next()
end
iter:free()

read_txn:free()

local stats = cf:get_stats()

print("\nColumn Family Statistics:")
print(string.format("  Number of Levels: %d", stats.num_levels))
print(string.format("  Memtable Size: %d bytes", stats.memtable_size))

db:drop_column_family("users")
db:close()

```

Transaction Reset

Reset a committed or aborted transaction for reuse with a new isolation level. This avoids the overhead of freeing and reallocating transaction resources in hot loops.

```

local cf = db:get_column_family("my_cf")

local txn = db:begin_txn()

txn:put(cf, "key1", "value1", -1)
txn:commit()

txn:reset(tidesdb.IsolationLevel.READ_COMMITTED)

txn:put(cf, "key2", "value2", -1)
txn:commit()

txn:free()

```

Behavior

- The transaction must be committed or aborted before reset; resetting an active transaction raises an error
- Internal buffers are retained to avoid reallocation
- A fresh transaction ID and snapshot sequence are assigned based on the new isolation level
- The isolation level can be changed on each reset

When to use

- Batch processing · Reuse a single transaction across many commit cycles in a loop
- Connection pooling · Reset a transaction for a new request without reallocation
- High-throughput ingestion · Reduce allocation overhead in tight write loops

Isolation Levels

TidesDB supports five MVCC isolation levels:

```
local txn =  
db:begin_txn_with_isolation(tidesdb.IsolationLevel.READ_COMMITTED)  
  
txn:free()
```

Available Isolation Levels

- `READ_UNCOMMITTED` · Sees all data including uncommitted changes
- `READ_COMMITTED` · Sees only committed data (default)
- `REPEATABLE_READ` · Consistent snapshot, phantom reads possible
- `SNAPSHOT` · Write-write conflict detection
- `SERIALIZABLE` · Full read-write conflict detection (SSI)

Savepoints

Savepoints allow partial rollback within a transaction:

```
local txn = db:begin_txn()

txn:put(cf, "key1", "value1", -1)

txn:savepoint("sp1")
txn:put(cf, "key2", "value2", -1)

txn:rollback_to_savepoint("sp1")

txn:commit()
txn:free()
```

Savepoint API

- `txn:savepoint(name)` · Create a savepoint
- `txn:rollback_to_savepoint(name)` · Rollback to savepoint
- `txn:release_savepoint(name)` · Release savepoint without rolling back

Custom Comparators

TidesDB uses comparators to determine the sort order of keys. Once a comparator is set for a column family, it **cannot be changed** without corrupting data.

Built-in Comparators

TidesDB provides six built-in comparators that are automatically registered:

- `"memcmp"` (default) · Binary byte-by-byte comparison
- `"lexicographic"` · Null-terminated string comparison (uses `strcmp`)
- `"uint64"` · Unsigned 64-bit integer comparison (8-byte keys)
- `"int64"` · Signed 64-bit integer comparison (8-byte keys)
- `"reverse"` · Reverse binary comparison (descending order)
- `"case_insensitive"` · Case-insensitive ASCII comparison

Select one by setting `comparator_name` on the column family config. The names are also available as the `tidesdb.Comparator` constant table, so you can avoid hard-coding the string literals:

```

local cf_config = tidesdb.default_column_family_config()
cf_config.comparator_name = tidesdb.Comparator.REVERSE -- same as
"reverse"

db:create_column_family("sorted_cf", cf_config)

```

Constant	Name string
<code>tidesdb.Comparator.MEMCMP</code>	<code>"memcmp"</code>
<code>tidesdb.Comparator.LEXICOGRAPHIC</code>	<code>"lexicographic"</code>
<code>tidesdb.Comparator.UINT64</code>	<code>"uint64"</code>
<code>tidesdb.Comparator.INT64</code>	<code>"int64"</code>
<code>tidesdb.Comparator.REVERSE</code>	<code>"reverse"</code>
<code>tidesdb.Comparator.CASE_INSENSITIVE</code>	<code>"case_insensitive"</code>

The underlying C comparator function pointers are also exposed in `tidesdb.builtin_comparators` (keys: `memcmp`, `lexicographic`, `uint64`, `int64`, `reverse_memcmp`, `case_insensitive`) for callers that want to register a built-in implementation under a custom name:

```

-- Alias the built-in uint64 comparator under a project-specific name.
db:register_comparator("event_id", tidesdb.builtin_comparators.uint64, nil,
nil)

```

The `uint64` and `int64` comparators require 8-byte keys and interpret them in **native byte order** (little-endian on x86-64); encode integer keys accordingly so they sort numerically.

Registering Custom Comparators

You can register custom comparators using FFI callbacks:


```
local ffi = require("ffi")

local my_compare = ffi.cast("tidesdb_comparator_fn", function(key1,
key1_size, key2, key2_size, ctx)
    local s1 = ffi.string(key1, key1_size)
    local s2 = ffi.string(key2, key2_size)
    if s1 < s2 then return -1
    elseif s1 > s2 then return 1
    else return 0 end
end)

db:register_comparator("my_comparator", my_compare, nil, nil)

local cf_config = tidesdb.default_column_family_config()
cf_config.comparator_name = "my_comparator"
db:create_column_family("custom_cf", cf_config)
```

Retrieving Comparators

```
local fn, ctx = db:get_comparator("memcmp")
```

Testing



```
# Run all tests with LuaJIT
cd tests
luajit test_tidesdb.lua

# Run with standard Lua (requires LuaFFI)
lua test_tidesdb.lua
```

API Reference

Module Functions

Function	Description
<code>tidesdb.TidesDB.open(path, options)</code>	Open a database
<code>tidesdb.default_config()</code>	Get default database configuration
<code>tidesdb.default_column_family_config()</code>	Get default column family configuration
<code>tidesdb.default_objstore_config()</code>	Get default object store configuration
<code>tidesdb.load_config_from_ini(file, section)</code>	Load config from INI file
<code>tidesdb.save_config_to_ini(file, section, config)</code>	Save config to INI file
<code>tidesdb.objstore_fs_create(root_dir)</code>	Create a filesystem-backed object store connector
<code>tidesdb.objstore_s3_create(opts)</code>	Create an S3-compatible object store connector (positional form)
<code>tidesdb.objstore_s3_create_config(config)</code>	Create an S3 connector with full TLS / multipart tuning
<code>tidesdb.init(malloc_fn, calloc_fn, realloc_fn, free_fn)</code>	Install custom allocators (optional; lazy init otherwise)
<code>tidesdb.finalize()</code>	Finalize the library and reset the allocator
<code>tidesdb.raise_open_file_limit(desired)</code>	Raise the process open-file ceiling before opening a database

TidesDB Class

Method	Description
<code>db:close()</code>	Close the database
<code>db:create_column_family(name, config)</code>	Create a column family
<code>db:drop_column_family(name)</code>	Drop a column family
<code>db:delete_column_family(cf)</code>	Drop a column family by handle (skips name lookup)
<code>db:rename_column_family(old_name, new_name)</code>	Rename a column family
<code>db:clone_column_family(source_name, dest_name)</code>	Clone a column family
<code>db:get_column_family(name)</code>	Get a column family handle
<code>db:list_column_families()</code>	List all column family names
<code>db:begin_txn()</code>	Begin a transaction
<code>db:begin_txn_with_isolation(level)</code>	Begin transaction with isolation level
<code>db:get_cache_stats()</code>	Get block cache statistics
<code>db:get_db_stats()</code>	Get database-level aggregate statistics
<code>db:purge()</code>	Synchronous flush and compaction for all column families
<code>db:cancel_background_work()</code>	Cancel background compaction db-wide for a fast shutdown
<code>db:backup(dir)</code>	Create database backup
<code>db:checkpoint(dir)</code>	Create lightweight database checkpoint using hard links
<code>db:register_comparator(name, fn, ctx_str, ctx)</code>	Register custom comparator
<code>db:get_comparator(name)</code>	Get registered comparator
<code>db:promote_to_primary()</code>	Promote a replica database to primary mode

ColumnFamily Class

Method	Description
<code>cf:compact()</code>	Trigger manual compaction
<code>cf:compact_range(start_key, end_key)</code>	Synchronous compaction over a key range (<code>nil</code> endpoint = unbounded)
<code>cf:flush_memtable()</code>	Trigger manual memtable flush
<code>cf:is_flushing()</code>	Check if flush is in progress
<code>cf:is_compacting()</code>	Check if compaction is in progress
<code>cf:get_stats()</code>	Get column family statistics
<code>cf:range_cost(key_a, key_b)</code>	Estimate range iteration cost between two keys
<code>cf:sync_wal()</code>	Force immediate fsync of the active WAL
<code>cf:purge()</code>	Synchronous flush and aggressive compaction
<code>cf:set_commit_hook(fn, ctx)</code>	Set commit hook callback for change data capture
<code>cf:clear_commit_hook()</code>	Clear (disable) the commit hook
<code>cf:update_runtime_config(config, persist)</code>	Update runtime configuration

Transaction Class

Method	Description
<code>txn:put(cf, key, value, ttl)</code>	Put a key-value pair
<code>txn:get(cf, key)</code>	Get a value by key
<code>txn:delete(cf, key)</code>	Delete a key
<code>txn:single_delete(cf, key)</code>	Delete a key with at-most-one-put promise (see Single-Delete)
<code>txn:commit()</code>	Commit the transaction
<code>txn:rollback()</code>	Rollback the transaction
<code>txn:reset(isolation)</code>	Reset transaction for reuse with new isolation level
<code>txn:savepoint(name)</code>	Create a savepoint
<code>txn:rollback_to_savepoint(name)</code> <code>)</code>	Rollback to savepoint
<code>txn:release_savepoint(name)</code>	Release a savepoint
<code>txn:new_iterator(cf)</code>	Create an iterator
<code>txn:free()</code>	Free transaction resources

Iterator Class

Method	Description
<code>iter:seek_to_first()</code>	Seek to first entry
<code>iter:seek_to_last()</code>	Seek to last entry
<code>iter:seek(key)</code>	Seek to key (or next key $\geq$ target)
<code>iter:seek_for_prev(key)</code>	Seek to key (or prev key $\leq$ target)
<code>iter:valid()</code>	Check if iterator is valid
<code>iter:next()</code>	Move to next entry
<code>iter:prev()</code>	Move to previous entry
<code>iter:key()</code>	Get current key
<code>iter:value()</code>	Get current value
<code>iter:key_value()</code>	Get current key and value in a single call
<code>iter:free()</code>	Free iterator resources

Constants

Compression Algorithms (`tidesdb.CompressionAlgorithm`)

- `NO_COMPRESSION` (0)
- `SNAPPY_COMPRESSION` (1)
- `LZ4_COMPRESSION` (2) — default
- `ZSTD_COMPRESSION` (3)
- `LZ4_FAST_COMPRESSION` (4)

Sync Modes (`tidesdb.SyncMode`)

- `SYNC_NONE` (0)
- `SYNC_FULL` (1)
- `SYNC_INTERVAL` (2)

Log Levels (`tidesdb.LogLevel`)

- `LOG_DEBUG` (0)
- `LOG_INFO` (1)
- `LOG_WARN` (2)

- `LOG_ERROR` (3)
- `LOG_FATAL` (4)
- `LOG_NONE` (99)

Isolation Levels (`tidesdb.IsolationLevel`)

- `READ_UNCOMMITTED` (0)
- `READ_COMMITTED` (1) — default
- `REPEATABLE_READ` (2)
- `SNAPSHOT` (3)
- `SERIALIZABLE` (4)

Built-in Comparators (`tidesdb.Comparator`)

- `MEMCMP` ("memcmp") — default
- `LEXICOGRAPHIC` ("lexicographic")
- `UINT64` ("uint64")
- `INT64` ("int64")
- `REVERSE` ("reverse")
- `CASE_INSENSITIVE` ("case_insensitive")

Error Codes

- `TDB_SUCCESS` (0)
- `TDB_ERR_MEMORY` (-1)
- `TDB_ERR_INVALID_ARGS` (-2)
- `TDB_ERR_NOT_FOUND` (-3)
- `TDB_ERR_IO` (-4)
- `TDB_ERR_CORRUPTION` (-5)
- `TDB_ERR_EXISTS` (-6)
- `TDB_ERR_CONFLICT` (-7)
- `TDB_ERR_TOO_LARGE` (-8)
- `TDB_ERR_MEMORY_LIMIT` (-9)
- `TDB_ERR_INVALID_DB` (-10)
- `TDB_ERR_UNKNOWN` (-11)
- `TDB_ERR_LOCKED` (-12)
- `TDB_ERR_READONLY` (-13)
- `TDB_ERR_BUSY` (-14)

- `TDB_ERR_PRECONDITION` (-15)

TidesDB Engine for MariaDB Reference

TideSQL is a pluggable storage engine for [MariaDB](#), built on top of TidesDB.

The engine supports ACID transactions through multi-version concurrency control (MVCC), letting readers proceed without blocking writers. It handles primary keys, secondary indexes, auto-increment columns, virtual and stored generated columns, savepoints, TTL-based expiration, data-at-rest encryption, online DDL, partitioning, and online backups. All of these features are accessible through standard SQL, so switching from InnoDB to TidesDB for a particular table requires nothing more than changing the `ENGINE` clause.

The TidesDB data files live in a sibling directory next to the MariaDB data directory, named `tidesdb_data`. The engine manages its own file layout entirely. In object store mode, the engine uses MariaDB's schema discovery mechanism to replicate table definitions across nodes (see “Schema Discovery” under Object Store Mode).

Getting Started

The engine is loaded as a shared plugin. Once the server is built with the TidesDB plugin compiled, it can be loaded at startup:

```
[mysqld]
plugin-load-add=ha_tidesdb.so
```

Or loaded dynamically:

```
INSTALL SONAME 'ha_tidesdb';
```

Once loaded, the engine appears in `SHOW ENGINES`:

```

MariaDB [(none)]> SHOW ENGINES
...
***** 1. row *****
      Engine: TIDESDB
      Support: YES
      Comment: LSM-tree engine with ACID transactions, MVCC concurrency,
secondary/spatial/full-text/vector indexes, and encryption
      Transactions: YES
              XA: NO
      Savepoints: YES
...

```

After that, creating a table with TidesDB is straightforward:

```

CREATE TABLE events (
  id    INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  ts    DATETIME NOT NULL,
  kind  VARCHAR(50),
  data  TEXT
) ENGINE=TIDESDB;

```

This creates a column family inside TidesDB for the table's data. If you later drop the table, the column family and all of its SSTables are removed as well.

Quick Install

The repository includes `install.sh`, a cross-platform script that clones MariaDB, builds it with the TidesDB plugin, and sets up a ready-to-run server. It handles dependencies, submodules, and configuration automatically.



```

git clone https://github.com/tidesdb/tidesql.git
cd tidesql
./install.sh --mariadb-prefix ~/mariadb-tidesdb

```

The script accepts several options:

Option	Description
<code>--mariadb-prefix</code> <code><path></code>	MariaDB installation directory (default: <code>/usr/local/mariadb</code>)
<code>--tidesdb-prefix</code> <code><path></code>	TidesDB library installation directory (default: <code>/usr/local</code>)
<code>--mariadb-version</code> <code><tag></code>	MariaDB version/branch to build (i.e: <code>mariadb-11.4.5</code>)
<code>--tidesdb-version</code> <code><tag></code>	TidesDB library version/tag (i.e: <code>v8.9.2</code>)
<code>--build-dir</code> <code><path></code>	Build directory (default: <code>/tmp/tidesql-build</code>)
<code>--jobs</code> <code><n></code>	Parallel build jobs (default: number of CPU cores)
<code>--skip-deps</code>	Skip system dependency installation
<code>--skip-tidesdb</code>	Skip building the TidesDB library (use if already installed)
<code>--rebuild-plugin</code>	Rebuild and reinstall only <code>ha_tidesdb.so</code> against an existing MariaDB install (does not rebuild MariaDB or <code>libtidesdb</code>)
<code>--skip-engines</code> <code><list></code>	Comma-separated storage engines to exclude from the build
<code>--list-engines</code>	List available storage engines and exit
<code>--pgo</code>	Enable profile-guided optimization (longer build, faster binaries)
<code>--s3</code>	Build with S3 object store connector (requires libcurl)
<code>--allocator</code> <code><name></code>	Link libtidesdb against <code>system</code> (default), <code>jemalloc</code> , <code>mimalloc</code> , or <code>tcmalloc</code>

For object store mode with S3:



```
./install.sh --mariadb-prefix ~/mariadb-tidesdb --s3
```

For a non-default allocator:



```
./install.sh --allocator jemalloc
```

The `--allocator` flag only affects `libtidesdb.so`'s internal allocations (memtable, klog/vlog buffers, compaction scratch, txn ops). `mariadb`'s allocator is unchanged. For a process-wide swap, also `LD_PRELOAD` the allocator at server startup:



```
LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libjemalloc.so.2 \  
~/mariadb-tidesdb/bin/mariadb-safe --defaults-file=~/mariadb-  
tidesdb/my.cnf &
```

Note that `--rebuild-plugin` does not rebuild `libtidesdb.so`, so changing `--allocator` requires a full install run (omit `--rebuild-plugin`) to take effect. Verify the linkage with:



```
ldd /usr/local/lib/libtidesdb.so | grep -E  
'jemalloc|mimalloc|tcmalloc'
```

After installation, start the server:



```
~/mariadb-tidesdb/bin/mariadb --defaults-file=~/mariadb-  
tidesdb/my.cnf &
```

Connect via socket (faster for local access):



```
~/mariadb-tidesdb/bin/mariadb -S /tmp/mariadb.sock
```

Or via TCP:



```
~/mariadb-tidesdb/bin/mariadb -h 127.0.0.1 -P 3306
```

The installer creates a `my.cnf` that loads the TidesDB plugin automatically. For user-owned prefixes, the server runs as your current user; for system prefixes, it runs as `mysql`.

Tables and Column Families

Every TidesDB table corresponds to one main column family that holds the row data. Each secondary index gets its own separate column family. The naming convention is rather deterministic, a table `test.events` maps to the column family `test__events`, and a secondary index named `idx_ts` on that table maps to `test__events__idx_idx_ts`.

This separation is meaningful. Because each column family has its own LSM-tree, a secondary index has its own memtable, its own set of SSTables files, and its own compaction and flush schedules.

When a table is renamed with `RENAME TABLE` or `ALTER TABLE ... RENAME`, the engine renames all associated column families, both the main data CF and every secondary index CF.

Primary Keys and Row Storage

If you define a `PRIMARY KEY` on a table, TidesDB uses it as the physical ordering key for the data column family. The key bytes are stored in a memcmp-comparable format, integers are encoded big-endian with their sign bit flipped so that negative values sort before positive ones, and strings use their collation's sort key encoding. This means range scans on the primary key are efficient, because physically adjacent keys in the LSM-tree correspond to logically adjacent rows.

If you create a table without an explicit primary key, the engine generates a hidden 8-byte row ID for each row, encoded in big-endian. These hidden IDs are monotonically increasing, assigned from an atomic counter that is recovered on restart by seeking to the last key in the column family.

Inside the column family, every row's key is prefixed with a single namespace byte (`0x01` for data rows, `0x00` for metadata). The value is stored in a packed binary format with a 5-byte header (see “How It Stores Data Internally” for details), followed by the null bitmap and each non-null field serialized using MariaDB's `Field::pack()` method. Fixed-size fields like `INT` and `BIGINT` are stored at their native pack length. `CHAR` fields have trailing spaces stripped. `VARCHAR` fields store only the actual data length rather than padding to the declared maximum. `BLOB` and `TEXT` fields are inlined with a length prefix followed by the data bytes. This packed format is more compact than the raw record buffer and reduces I/O and storage costs, especially for tables with variable-length columns.

Composite primary keys are fully supported. Each key part is encoded in comparable format and concatenated, so a composite key like `(dept_id, emp_id)` sorts first by department, then by employee within each department. The optimizer can use prefix lookups on the leading columns of a composite PK (e.g., `WHERE dept_id = 3` on a `PRIMARY KEY (dept_id, emp_id)`) via an iterator-based prefix scan.

```
-- With explicit PK (comparable key ordering)
CREATE TABLE users (
  id    INT NOT NULL PRIMARY KEY,
  name  VARCHAR(100)
) ENGINE=TIDESDB;

-- Composite PK (multi-column ordering)
CREATE TABLE emp_projects (
  emp_id  INT NOT NULL,
  proj_id INT NOT NULL,
  hours   INT NOT NULL,
  PRIMARY KEY (emp_id, proj_id)
) ENGINE=TIDESDB;

-- Without PK (hidden auto-generated row ID)
CREATE TABLE logs (
  ts      DATETIME,
  message TEXT
) ENGINE=TIDESDB;
```

Secondary Indexes

Secondary indexes are stored in their own column families, separate from the main data. Each index entry consists of a key that concatenates the comparable-format index column bytes with the comparable-format primary key bytes. The value is empty (a single zero byte). This design means that

every secondary index entry is self-contained, given an index key, the engine can extract the primary key from its tail and perform a point lookup into the main data CF to fetch the full row.

When you insert, update, or delete a row, the engine transactionally maintains all secondary indexes within the same transaction. For updates, the engine builds the old and new comparable index key for each secondary index and compares them with `memcmp`; if the indexed columns and PK bytes are identical, that index is skipped entirely, avoiding a redundant delete-then-reinsert round-trip into the library. This is a significant optimization for updates that only touch non-indexed columns. For deletes, the corresponding index entries are removed.

Duplicate key violations on primary keys and unique indexes are properly detected. Inserting a row with a primary key that already exists returns the standard `ER_DUP_ENTRY` error. The same applies to unique secondary indexes. `REPLACE INTO` and `INSERT ... ON DUPLICATE KEY UPDATE` work correctly, `write_row()` returns `HA_ERR_FOUND_DUPP_KEY` with the conflicting row's PK in `dup_ref`, so the server can perform the delete-then-reinsert (REPLACE) or switch to `update_row()` (IODKU), properly cleaning up old secondary index entries in the process.

```
CREATE TABLE products (  
  id          INT NOT NULL PRIMARY KEY,  
  category INT,  
  name        VARCHAR(100),  
  KEY idx_category (category)  
) ENGINE=TIDESDB;  
  
INSERT INTO products VALUES (1, 10, 'Widget'), (2, 20, 'Gadget'), (3, 10,  
'Sprocket');  
  
-- Uses the secondary index for the lookup  
SELECT * FROM products WHERE category = 10;
```

The optimizer is aware of these indexes. The engine reports cost estimates based on the LSM-tree's read amplification factor, which it obtains from TidesDB's internal statistics. Point lookups through a secondary index cost roughly one seek into the index CF plus one point-get into the data CF.

That data-CF point-get is skipped entirely when the read is covering. When every column a query needs is carried by the index key - the indexed columns plus the primary-key columns appended to each entry - and each of those columns is of a type the engine can reconstruct from its comparable bytes (integers, `YEAR`, `DATE`, `DATETIME/TIMESTAMP`, and fixed `CHAR/BINARY` in a binary or latin1 charset), the row is materialised straight from the index bytes with no fetch into the data CF at all. The capability is advertised to the optimizer per key part, so a query that reads only the reconstructable

columns of an index still gets an index-only plan even when the index also carries a column whose sort bytes are not invertible - `VARCHAR`, `DECIMAL`, or a floating-point column - which would otherwise be priced as a covering scan that then pays a hidden per-row primary-key lookup.

Index Condition Pushdown (ICP)

The engine supports Index Condition Pushdown for secondary index scans. When the optimizer pushes a `WHERE` condition down to the storage engine, the engine evaluates it on the index key columns before performing the expensive primary key point-lookup into the data column family. This is the same pattern used by InnoDB - the engine decodes the index key columns into the record buffer and calls MariaDB's `handler_index_cond_check()` evaluator. When the condition columns are of a type the engine can decode from the index bytes - integer types (`TINYINT`, `SMALLINT`, `MEDIUMINT`, `INT`, `BIGINT`), temporal types (`DATE`, `DATETIME`, `TIMESTAMP`, `YEAR`), and fixed-length `CHAR/BINARY` with binary or latin1 charset - an entry that fails the condition is skipped without touching the data CF at all. For a condition column whose type cannot be reconstructed from its comparable bytes - `DECIMAL`, `VARCHAR`, floating-point, or a multi-byte-charset `CHAR` - the engine fetches the full row through the primary key first and then applies the pushed condition, so ICP still filters the result set but the data-CF fetch is not avoided for those columns.

Multi-Range Read (MRR)

The engine implements a custom MRR path for point-lookup batches such as `WHERE col IN (v1, v2, ..., vN)` on a primary or full-key unique index. When every range the optimizer hands the engine is a full-key point equality (`UNIQUE_RANGE | EQ_RANGE`) and there are at least two ranges, the engine buffers them, converts each key into comparable bytes, and sorts by those bytes so the LSM sees a monotone stream of seeks - much friendlier to the block cache and the merge-heap than N scattered seeks in user-supplied order. Primary-key lookups bypass the iterator entirely via `fetch_row_by_pk`; secondary-index lookups reuse a single cached iterator and do one seek per entry. Ranges whose rows have been deleted concurrently are silently skipped.

The engine deliberately declines MRR in three cases, falling back to the base handler's default implementation:

- Single-range scans (`count < 2`) - MRR has no sorting win for one key, and the `eq_ref` path keeps the single-row `fetch_row_by_pk` shape where pessimistic row locking engages naturally.
- Non-point ranges - true `BETWEEN`/`</>` scans stay on `read_range_first`.
- Partitioned tables - `ha_partition` already dispatches MRR across children using its own DS-MRR logic.

Auto-Increment

Auto-increment works in a similar way to InnoDB. The engine calls MariaDB's built-in `update_auto_increment()` mechanism during `write_row()`. Rather than calling `index_last()` on every INSERT (which would create and destroy a TidesDB merge-heap iterator each time), the engine maintains an in-memory atomic counter on the shared table descriptor. The counter is seeded once at table open time by seeking to the last key in the primary key column family - but only when the AUTO_INCREMENT column is the leftmost part of the primary key, where that last key carries the highest assigned value. When the AUTO_INCREMENT column instead lives in a separate unique key, the engine leaves the counter at zero and lets MariaDB seed it on demand. It is atomically incremented via a CAS loop on each INSERT, making auto-increment assignment O(1). When a user inserts an explicit value larger than the current counter, `write_row()` bumps the counter to match.

`TRUNCATE TABLE` and `ALTER TABLE ... AUTO_INCREMENT=N` both reset the counter via the engine's `reset_auto_increment` handler hook - the next generated ID equals `N` (or `1` after a bare `TRUNCATE`). This applies to both user-defined AUTO_INCREMENT columns and hidden-PK tables.

```
CREATE TABLE tickets (  
  id    INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
  desc  VARCHAR(200)  
) ENGINE=TIDESDB;  
  
INSERT INTO tickets (desc) VALUES ('First ticket');  
INSERT INTO tickets (desc) VALUES ('Second ticket');  
INSERT INTO tickets (id, desc) VALUES (100, 'Explicit ID');  
INSERT INTO tickets (desc) VALUES ('Continues from 100');  
  
SELECT * FROM tickets ORDER BY id;  
-- 1, 2, 100, 101
```

Transactions and Concurrency

TidesDB uses MVCC internally. Each statement runs inside a TidesDB transaction. The engine maintains a per-connection transaction context through MariaDB's handler callback interface, following the same pattern as InnoDB. A transaction object is allocated lazily on the first data access and registered with MariaDB's transaction coordinator. After commit or rollback, the transaction object is kept alive and reused via `tidesdb_txn_reset()` on the next statement, which obtains a fresh MVCC snapshot while preserving internal buffers (ops array, arenas, read-set arrays). This avoids the malloc/free overhead of creating a new transaction on every autocommit statement. Autocommit single-statement DML uses `READ_COMMITTED` isolation since there is no concurrent modification

within a single-statement transaction, eliminating unnecessary conflict tracking overhead. Multi-statement transactions (`BEGIN ... COMMIT`) use the session's isolation level for proper write-write conflict detection.

The engine respects the session's isolation level set via `SET TRANSACTION ISOLATION LEVEL`. The mapping from MariaDB isolation levels to TidesDB isolation levels is:

MariaDB	TidesDB
<code>READ UNCOMMITTED</code>	<code>TDB_ISOLATION_READ_UNCOMMITTED</code>
<code>READ COMMITTED</code>	<code>TDB_ISOLATION_READ_COMMITTED</code>
<code>REPEATABLE READ</code>	<code>TDB_ISOLATION_SNAPSHOT</code>
<code>SERIALIZABLE</code>	<code>TDB_ISOLATION_SERIALIZABLE</code>

MariaDB's `REPEATABLE READ` resolves through the per-table `ISOLATION_LEVEL` option. A table that leaves the option at its default (`REPEATABLE_READ`) maps to TidesDB's `SNAPSHOT` isolation, which is the semantic equivalent of InnoDB's repeatable-read, consistent read snapshot with write-write conflict detection only, no read-set tracking. TidesDB's own `REPEATABLE_READ` level is stricter (tracks read-set, detects read-write conflicts at commit) and would cause excessive conflicts under normal OLTP concurrency. A table that sets `ISOLATION_LEVEL='SNAPSHOT'`, `SERIALIZABLE`, `READ_COMMITTED` or `READ_UNCOMMITTED` is honored as written so a workload can pin a tighter or looser level on a per-table basis without changing the session default. Sessions that explicitly request `READ UNCOMMITTED`, `READ COMMITTED`, or `SERIALIZABLE` bypass the table option and use the session level.

DDL operations (`ALTER TABLE`, `CREATE INDEX`, `DROP INDEX`, `TRUNCATE`, `OPTIMIZE`) and autocommit single-statement DML always use `READ_COMMITTED` regardless of the session setting, to avoid unnecessary conflict tracking overhead and unbounded read-set growth during large scans.

TidesDB supports SQL savepoints (`SAVEPOINT`, `ROLLBACK TO SAVEPOINT`, `RELEASE SAVEPOINT`) inside explicit multi-statement transactions. Savepoints are only meaningful inside `BEGIN ... COMMIT` blocks.

Consistent Snapshots

The engine supports `START TRANSACTION WITH CONSISTENT SNAPSHOT`. This eagerly creates a TidesDB transaction and captures the snapshot sequence number immediately, rather than waiting until the first data access. The isolation level is enforced to be at least `SNAPSHOT`, regardless of the session setting, since lower levels like `READ_COMMITTED` would refresh the snapshot on each read, violating the consistent snapshot semantics. Rows committed by other connections after the snapshot are invisible to the transaction. This is useful for cross-engine consistency when TidesDB and InnoDB tables coexist in the same transaction:

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION WITH CONSISTENT SNAPSHOT;
-- snapshot is taken now, not at first SELECT
SELECT * FROM tidesdb_table;  -- sees data as of snapshot time
SELECT * FROM innodb_table;   -- InnoDB also snapshotted at the same time
COMMIT;
```

The engine sets `lock_count()` to zero, which tells MariaDB to bypass its own table-level locking layer entirely. Concurrency control is split between plugin-level row locks and library-level MVCC. By default, `tidesdb_pessimistic_locking` is ON and write-intent statements acquire row locks layered over MVCC. Disabling it (`SET GLOBAL tidesdb_pessimistic_locking = OFF`) drops the engine back to pure optimistic concurrency where readers and writers proceed without blocking each other and write-write conflicts surface at commit time (see Pessimistic Row Locking below).

Write-write conflict detection is a property of the higher isolation levels. At `SNAPSHOT` and `SERIALIZABLE` the library validates each write against the version the transaction actually read and runs a first-committer-wins reservation before the write log, so two transactions that modify the same row cannot both commit. At `READ_COMMITTED` the library performs no write-write checking at all - and every autocommit statement runs at `READ_COMMITTED` regardless of the session level, since a single-statement transaction has no concurrent modification of its own to track. At that level correctness under concurrent writes to the same row comes from the pessimistic row locks described below rather than from conflict detection, which is why the locks are on by default and why turning them off at `READ_COMMITTED` lets concurrent read-modify-write statements lose updates.

The reservation runs inside `tidesdb_txn_commit()`, so a detected conflict fails the losing transaction's `COMMIT`. The engine maps `TDB_ERR_CONFLICT` to `HA_ERR_LOCK_DEADLOCK`, but because MariaDB reports any error raised from the commit callback as `ER_ERROR_DURING_COMMIT` (ERROR 1180) rather than `ER_LOCK_DEADLOCK` (1213), a commit-time conflict surfaces to the application as 1180. The transaction is rolled back cleanly and should be retried; MariaDB does not re-run it automatically. Conflicts therefore only arise at `SNAPSHOT` or higher, and only when the pessimistic locks are off or a pattern the locks do not cover races at commit. Enabling `tidesdb_print_all_conflicts` logs every conflict event to the error log for diagnostics (see System Variables).

This is a fundamental architectural difference between the two engines. InnoDB uses pessimistic row-level locks that serialize access to hot rows - when two transactions want to update the same row, the second one waits until the first commits or rolls back. TidesDB uses optimistic MVCC - both transactions proceed concurrently without blocking each other, and the second one to commit fails with a conflict error. In other words, InnoDB makes concurrent writers wait; TidesDB makes them fail and retry. Neither approach is inherently better. Pessimistic locking guarantees forward progress but

introduces lock waits and potential deadlocks. Optimistic MVCC eliminates all lock waits and deadlocks but requires applications to handle retries. Workloads with low contention (most rows are touched by at most one writer at a time) see virtually no conflicts and benefit from the absence of lock overhead. Workloads with high contention on a small number of hot rows (e.g., a single counter row updated by every transaction) will see higher conflict rates and depend on efficient retry logic.

Pessimistic Row Locking

By default, TidesDB layers a plugin-level row lock manager on top of MVCC so workloads that depend on `SELECT ... FOR UPDATE` semantics, TPC-C-style read-modify-write cycles, or any other pattern that needs forward progress without application retry behave the same way as InnoDB. The lock manager can be disabled by setting `tidesdb_pessimistic_locking = OFF`, at which point the engine falls back to pure optimistic concurrency where write-write conflicts are detected at commit time and the application must retry.

```
SET GLOBAL tidesdb_pessimistic_locking = ON;    -- default, row-level
locks engaged
SET GLOBAL tidesdb_pessimistic_locking = OFF;  -- pure OCC, conflicts
surface at commit
```

The lock manager is a partitioned hash table sized at startup from CPU count, `8 * std::thread::hardware_concurrency()` clamped to the range 128..65,536. Each partition is `alignas(64)` so unrelated stripes never share a cache line, and each is protected by its own mutex. The partition array itself is allocated with `posix_memalign` (or `_aligned_malloc` on Windows) so the 64-byte alignment the struct declares is honoured at runtime. The comparable bytes that identify a row (or a unique secondary key prefix) are hashed (XXH3) into a partition, and each partition holds an active hash chain of lock entries keyed by the full comparable bytes.

When a transaction commits or rolls back and the last grant on a slot is released, the slot is unlinked from the partition's active chain and pushed onto a per-partition freelist. The next acquire that needs a new slot in that partition pops from the freelist before mallocing. The slot's memory is retained across reuse so the deadlock walker can safely dereference cross-partition pointers it captured while holding another partition's mutex, but the chain length tracks the number of locks currently held in the partition rather than the running total of keys ever touched. Chain depth scales with concurrent active locks, not lifetime cardinality, which is what makes the manager safe to drive at OLTP write rates without the per-acquire chain walk degrading over time.

Lock Modes

Each lock supports two modes. Shared (S) is read-intent, exclusive (X) is write-intent, and the compatibility rules are the same as in any classic two-mode lock manager - S and S coexist on the same row, anything else conflicts. To prevent a stream of readers from starving a writer, a new S request blocks when an X is already queued, so writers cannot be indefinitely deferred.

The same transaction may acquire the same lock more than once. A second acquire in the same or weaker mode (X subsumes S, same-mode is identity) returns immediately. A sole holder of S can upgrade to X in place. An upgrade attempt while another transaction also holds S is rejected as `ER_LOCK_DEADLOCK`, since granting it would require the upgrader to block on its own outstanding S grant.

Which Statements Acquire Locks

Acquisition sites split by statement intent. `fetch_row_by_pk` and `iter_read_current` acquire on the materialisation path for `SELECT ... FOR UPDATE` and for plain reads under `REPEATABLE-READ` / `SERIALIZABLE`, so the SQL cursor contract that every cursor-visible row is locked is preserved. `UPDATE` and `DELETE` deliberately skip the lock during their secondary-index scan and ICP filtering pass, then `update_row` and `delete_row` reacquire the X lock on the actual mutation target inside `write_row` / the delete path. This avoids X-locking every row a scan walks past when ICP discards most of them. `write_row` continues to acquire X on its primary-key insert and on every unique secondary index prefix before the duplicate check so two concurrent inserts of the same unique value cannot both pass dup-check and both commit.

Statement	Mode	Notes
<code>INSERT</code>	X	Primary key always; unique secondary prefixes also locked so the dup-check and put are serialised
<code>UPDATE</code>	X	Acquired in <code>update_row</code> on the target row; secondary-index scans under UPDATE do not lock the rows they walk past during ICP
<code>DELETE</code>	X	Acquired in <code>delete_row</code> on the target row; secondary-index scans under DELETE do not lock the rows they walk past during ICP
<code>SELECT ... FOR UPDATE</code>	X	Acquired during materialisation on every cursor-visible row, per the SQL locking-cursor contract
<code>SELECT</code> under session <code>READ UNCOMMITTED</code> or <code>READ COMMITTED</code>	none	MVCC snapshot already provides a consistent view
<code>SELECT</code> under session <code>REPEATABLE READ</code> or <code>SERIALIZABLE</code>	S	Prevents concurrent modification of read rows within the transaction

The read-lock mode is decided from the **SQL session isolation level** (`thd_tx_isolation`), not from the table's effective TidesDB isolation. There is no SQL-level `SNAPSHOT`, so a table created with `ISOLATION_LEVEL='SNAPSHOT'` (which resolves from a default `REPEATABLE READ` session) still takes `S` read locks because the session level is `REPEATABLE READ`; the table option changes the library's MVCC behaviour, not the read-lock mode.

Both explicit transactions (`BEGIN ... COMMIT`) and autocommit single-statement DML participate in locking. Autocommit statements block on locks held by other transactions, which prevents an autocommit `UPDATE` from silently bypassing a lock held by a `SELECT ... FOR UPDATE` in another session.

When pessimistic locking is on, the engine no longer silently downgrades the session's isolation level. Earlier revisions forced `READ_COMMITTED` at the library boundary on the assumption that the lock manager would carry the load alone, but that left phantoms unprotected at `REPEATABLE-READ` and broke higher levels. The lock manager and library optimistic concurrency control now compose - locks serialise write contention on the rows they cover, and OCC continues to detect anything the locks miss (notably new rows that appear in a range a transaction has scanned, since the manager has no range or gap locks).

Locking Non-Existing Rows

Locks are keyed by comparable bytes, not by the existence of a row. A `SELECT ... FOR UPDATE` on a primary key value that does not exist (e.g., `WHERE id = 15` when no such row exists) still acquires a lock on that key. This blocks other transactions from inserting, updating, or deleting that key until the lock is released, which supports the common check-then-insert pattern:

```
BEGIN;
SELECT * FROM t WHERE id = 42 FOR UPDATE; -- empty set, but lock is held
-- no other transaction can INSERT id=42 while this lock is held
INSERT INTO t VALUES (42, 'reserved');
COMMIT;
```

The same applies to UNIQUE secondary keys. The lock taken on the unique prefix during `INSERT` serialises check-then-insert across writers, so two concurrent `INSERT`s of the same unique value cannot both pass the dup-check and both commit.

Deadlock Detection

Before a transaction blocks on a held lock, the engine walks the wait-for graph looking for cycles. The walker is a fixed-capacity DFS over `(lock, mode)` frames. Starting from the target lock and the requested mode, it visits every conflicting holder of that lock, follows each holder's `waiting_on_lock` to the next lock, and pushes those frames onto the frontier. If any holder on the path is the requestor itself, granting the lock would close a cycle and the engine returns `ER_LOCK_DEADLOCK` (ERROR 1213) immediately rather than block. This is the same error code and semantics InnoDB uses for deadlocks.

The DFS frontier and visited set are stack-allocated at `DEADLOCK_MAX_DEPTH * 4` (the depth cap is 100, so capacity is 400 frames). Frontier overflow and depth-cap exceeded both return “deadlock” so a false-positive triggers a cheap retry rather than a stall at `tidesdb_lock_wait_timeout_ms`. Each hop takes the partition mutex of the lock being inspected so the walker can read request fields safely, then releases the mutex before moving on. The cross-partition wait-for edge is followed through atomic `waiting_on_lock` and `waiting_on_mode` fields on the transaction; the walker never dereferences a request struct outside the partition mutex that protects it. Lock entries themselves are never freed at runtime (the recycle path overwrites key bytes in place, leaving the entry's mutex and condition variable intact), so the lock pointers stored in `waiting_on_lock` are always safe to follow.

A waiter that receives `KILL QUERY` during its wait is woken promptly. The `kill_query` handler on callback broadcasts on the waited-on lock's condition variable, the wait loop observes `thd_killed()` on its next iteration, and the statement returns `HA_ERR_LOCK_WAIT_TIMEOUT` instead of hanging until the holder commits.

```

-- Connection A:
BEGIN;
SELECT * FROM t WHERE id = 1 FOR UPDATE;  -- holds X on id=1

-- Connection B:
BEGIN;
SELECT * FROM t WHERE id = 2 FOR UPDATE;  -- holds X on id=2

-- Connection A:
UPDATE t SET v = v + 1 WHERE id = 2;      -- blocks, waiting for B

-- Connection B:
UPDATE t SET v = v + 1 WHERE id = 1;      -- deadlock detected, ERROR 1213

```

Lock Wait Timeout

Waits are bounded by `tidesdb_lock_wait_timeout_ms`, which mirrors `innodb_lock_wait_timeout` and defaults to 50,000 milliseconds (50 seconds). When the deadline expires the acquire returns `HA_ERR_LOCK_WAIT_TIMEOUT` and the engine bumps the `tidesdb_lock_timeouts` status counter. Setting the value to zero disables the bound entirely, leaving `KILL QUERY` as the only way out of a wait. The variable is session-scoped, so per-statement budgets can be set via `SET SESSION` for transactions that need tighter control:

```

SET SESSION tidesdb_lock_wait_timeout_ms = 1000;  -- 1 second budget
UPDATE accounts SET balance = balance + 100 WHERE id = 5;

```

Lock Release

All locks held by a transaction are released atomically when the transaction commits or rolls back. There is no early lock release. Each lock entry is unlinked from the transaction's granted set, any waiters newly compatible with the remaining granted set are promoted from the waiting queue to the granted set, and the lock's condition variable is broadcast so the promoted waiters wake up and complete their acquire. If a connection is closed while holding locks, the engine releases them during connection cleanup.

Observability

The lock manager surfaces eight status variables for monitoring contention and capacity. `Tidesdb_lock_waits` counts the number of times any thread blocked in the wait loop, and `Tidesdb_lock_wait_us` accumulates the total microseconds spent there. `Tidesdb_lock_deadlocks` counts cycle detections, `Tidesdb_lock_timeouts` counts deadline expirations, and

`Tidesdb_lock_held` reports the current count of granted locks across all transactions. `Tidesdb_lock_entries` shows how many lock entries the manager has ever allocated in the hash table (slots are retained across reuse, so this is a lifetime counter rather than a current depth), `Tidesdb_lock_entry_recycles` counts freelist reuse events for memory-hygiene tracking, and `Tidesdb_lock_chain_max` reports the longest hash-chain length observed at acquire time across the run. The last is the canonical signal for “are my partitions degrading”, a healthy run keeps it in the low double digits even under sustained OLTP write contention.

When to Disable It

The default of ON suits any workload that depends on serialization guarantees for correctness - TPC-C-style read-modify-write cycles, banking transfers, inventory deductions, or any pattern that performs `SELECT ... FOR UPDATE` followed by an update on the read value. With pessimistic locking on, a contending transaction blocks rather than reads a stale value and fails at commit, which guarantees forward progress without application-level retry logic.

Turn pessimistic locking off when the workload is low-contention, the application is already prepared to retry, and the extra throughput of pure optimistic concurrency control matters more than the simpler client logic. Bulk ETL and analytics-style writers that touch disjoint key ranges fit this profile. Disable with `SET GLOBAL tidesdb_pessimistic_locking = OFF` or by adding the line to `my.cnf`. Inside the engine’s own MTR test suite, two cases that specifically validate the optimistic conflict path force the variable off through `.opt` files.

The per-table isolation level is configurable at table creation time and defaults to `REPEATABLE_READ`:

```
CREATE TABLE ledger (  
  id INT PRIMARY KEY,  
  amt DECIMAL(10,2)  
) ENGINE=TIDESDB ISOLATION_LEVEL='SERIALIZABLE';
```

The available isolation levels are `READ_UNCOMMITTED`, `READ_COMMITTED`, `REPEATABLE_READ`, `SNAPSHOT`, and `SERIALIZABLE`.

Bulk DML Batching

Statements that touch many rows such as `LOAD DATA INFILE`, multi row `INSERT`, `INSERT ... SELECT`, and `UPDATE` or `DELETE` over a range keep the TidesDB transaction from growing unbounded by committing mid statement in fixed size batches. The engine hooks MariaDB’s `start_bulk_insert`, `start_bulk_update`, and `start_bulk_delete` callbacks, counts the row operations (data write plus secondary index maintenance) against `TIDESDB_BULK_INSERT_BATCH_OPS` (500 ops), and at each threshold commits the current transaction and resets it with `READ_COMMITTED`

for the next batch. This keeps statement memory bounded regardless of statement size. Autocommit semantics are preserved so a failure rolls back only the current batch, and the statement as a whole reports the first error encountered.

The mid commit logic is shared between INSERT, UPDATE, and DELETE via a single `maybe_bulk_commit` helper, so the batching threshold and the iterator plus dup cache invalidation policy are identical across the three paths.

Write Back-Pressure

The TidesDB library signals back-pressure with `TDB_ERR_MEMORY_LIMIT` when the unified memtable, flush queue, or L0 backlog hits a soft cap and writers should pause until flush and compaction free capacity. Earlier revisions surfaced this signal to the SQL layer as `HA_ERR_LOCK_DEADLOCK`, which broke clients that treat 1213 as fatal and do not retry (bulk loaders, schema-build scripts, batch ETL). The engine now absorbs the signal inside the plugin: every write entry point routes through a thin blocking wrapper that sleeps with exponential backoff and retries the operation until the library accepts it, the wait timeout expires, or the connection is killed. After the budget is exhausted, the unmodified `TDB_ERR_MEMORY_LIMIT` bubbles up through the error mapping and surfaces as `HA_ERR_LOCK_WAIT_TIMEOUT`, which is the accurate name since no lock is held.

The wait budget is controlled by `tidesdb_backpressure_wait_timeout_ms`, a session variable defaulting to 60,000 milliseconds. Setting it to zero disables blocking and the back-pressure signal returns immediately. The deadline is anchored once per statement in `external_lock(F_WRLCK)` and consulted from every backpressure call inside that statement, so a multi-row INSERT respects the configured ms end-to-end instead of multiplying it by row count - a bulk INSERT of N rows with K secondary indexes does not multiply the budget by $N * K$. Two status counters expose activity: `Tidesdb_backpressure_waits` counts the number of times a writer blocked, and `Tidesdb_backpressure_wait_us` accumulates total microseconds spent in the wait.

All hot write paths route through the wrappers, including `write_row` (primary and secondary maintenance), `update_row`, `delete_row`, `fts_update_meta`, the inplace ADD INDEX secondary population, and both `hton_commit` and the mid-statement `bulk_commit`. Iterator construction (`tidesdb_iter_new`) is also wrapped, so transient reader-fd starvation backs off and retries instead of immediately surfacing `HA_ERR_LOCK_WAIT_TIMEOUT` or being misreported as `HA_ERR_CRASHED`. Tuning `tidesdb_unified_memtable_write_buffer_size` and the number of flush and compaction threads remains the right knob for sustained-write workloads, but the wrappers ensure that transient back-pressure spikes never present as deadlocks to the SQL layer.

Single-Delete Optimization

DELETE on a TidesDB table writes a tombstone into every column family the row touches: the primary row CF plus one CF per secondary, full-text, or spatial index. Regular tombstones have to be carried forward through every compaction until they reach the largest active level, because any level below could still contain an older put of the same key that the tombstone is masking. Insert-then-delete workloads (event streams, log tables, TTL-style purges, the classic iibench benchmark) pile these tombstones at the low end of the key space where DELETE range scans start, and the scan CPU climbs linearly with the backlog until compaction catches up.

The TidesDB library's single-delete primitive (`tidesdb_txn_single_delete`) lets compaction drop a put and its matching tombstone together the first time both appear in the same merge input, regardless of level. The caller's contract is "at most one put between single-deletes on the same key (or between the start of the key's history and its first single-delete)". For reads, a single-delete behaves exactly like a regular tombstone.

The plugin splits this across two behaviours:

Secondary-index single-delete (automatic)

Every secondary index entry — `(col_values, pk)` for a regular index, `(term, pk)` for a FULLTEXT index, `(hilbert_value, pk)` for a SPATIAL index — is written exactly once per row lifetime and deleted exactly once, across every path the plugin takes: `INSERT`, `UPDATE` (which delete-plus-put when the indexed columns change), `DELETE`, `REPLACE INTO`, `INSERT ... ON DUPLICATE KEY UPDATE`. The same `(composite, pk)` bytes never see a second put without an intervening delete, so the single-delete contract holds by construction of the index key layout.

The plugin therefore uses `tidesdb_txn_single_delete` for every secondary-index delete automatically. No configuration, no user flag, no workload assumption. This alone covers three of the four tombstones per deleted row on a table with three secondary indexes.

Primary-CF single-delete (opt-in per session)

The primary row CF is different. `UPDATE t SET non_pk_col = ...` writes a fresh row at the same `data_key(pk)`, producing a put-over-put. `REPLACE INTO` on a table without secondary indexes takes a short-circuit path that overwrites the primary row silently for the same reason. Under either pattern, dropping a primary-CF put and its later single-delete together at compaction can re-expose an older put — a silent correctness problem the engine cannot detect from the outside.

Primary-CF single-delete is therefore behind the session variable `tidesdb_single_delete_primary`, default OFF. Enabling it is the caller's explicit promise that:

- The session performs no `UPDATE` on non-PK columns of TidesDB tables.

- The session performs no `REPLACE INTO` or `INSERT ... ON DUPLICATE KEY UPDATE` that hits the silent-overwrite path on a table without secondary indexes.
- New rows with a given PK are always preceded by a `DELETE` of that PK (append-only or insert-then-delete).

Enable it only when the workload is known to fit this shape. Typical safe cases:

```
-- classic insert-then-delete (event stream, TTL purge, iibench-shape)
SET SESSION tidesdb_single_delete_primary = 1;
INSERT INTO events (...) VALUES ...;    -- monotonic PK
DELETE FROM events WHERE ts < NOW() - INTERVAL 1 HOUR;
```

Leave it OFF for any session that may issue `UPDATE` on a non-PK column, `REPLACE INTO` on a no-secondary table, or `INSERT ... ON DUPLICATE KEY UPDATE` on a no-secondary table. Setting the variable ON in those scenarios can leak older row versions through reads after a compaction.

When to expect a benefit

The larger the tombstone backlog at the scan head of your `DELETE` statements, the more the single-delete pair-cancellation helps. On iibench-shaped insert-then-delete workloads, with three secondary indexes, turning both automatic secondary-index SD and the primary-CF session variable together typically cuts the `max_d` sawtooth peak by 60 to 95 percent, depending on how long deletes have been running against the same key range without compaction catching up. On workloads with no `DELETE`, no benefit — and no risk either, since the secondary-index path only changes behaviour on `DELETE` and `UPDATE`.

Tombstone Density Trigger

Single-delete + pair-cancellation handles `INSERT+DELETE` workloads where the contract holds. For everything else (`UPDATE`s of indexed columns, `REPLACE INTO` on tables without secondary indexes, mixed read-write OLTP) tombstones still accumulate inside SSTables until a compaction at the largest level reclaims them. Reads over a deleted region pay for every tombstone the merge iterator has to skip, and a column family with infrequent natural compactions can accumulate enough tombstones to seriously degrade range-scan latency.

The library's tombstone-density trigger lets the engine notice and act on this state without waiting for capacity-based or file-count-based triggers. After every flush, the engine inspects level-1 SSTables and asks whether any single SSTable's tombstone count divided by entry count exceeds a configurable ratio while having at least a configurable minimum entry count. A single witness is enough to escalate compaction.

TideSQL exposes the trigger as two per-table options that map onto the underlying column family's `tombstone_density_trigger` and `tombstone_density_min_entries` settings:

```
CREATE TABLE events (  
  id    BIGINT PRIMARY KEY,  
  ts    DATETIME,  
  body  TEXT,  
  KEY (ts)  
) ENGINE=TIDESDB  
  TOMBSTONE_DENSITY_TRIGGER=5000    -- 50% (parts per 10000)  
  TOMBSTONE_DENSITY_MIN_ENTRIES=2048;
```

`TOMBSTONE_DENSITY_TRIGGER` is in parts per 10000, where `5000` means 0.50. A value of `0` (the default) disables the check and preserves the previous structural-trigger-only behavior. `TOMBSTONE_DENSITY_MIN_ENTRIES` (default `1024`) is a floor that prevents a single-entry SSTable that happens to be a tombstone from noisily firing compaction.

Both options are also surfaced as session defaults so a deployment can switch the policy globally without touching every CREATE TABLE statement:

```
SET GLOBAL tidesdb_default_tombstone_density_trigger = 5000;  
SET GLOBAL tidesdb_default_tombstone_density_min_entries = 2048;
```

`ALTER TABLE ... TOMBSTONE_DENSITY_TRIGGER=N` updates the live column family configuration via `tidesdb_cf_update_runtime_config`, so the new ratio takes effect on the next post-flush check without requiring a restart.

The aggregates surface in monitoring as four global status variables which read from a once-per-refresh CF-list walk so the cost is paid by the monitoring caller, not the write path:

Variable	Meaning
<code>Tidesdb_total_tombstones</code>	Sum of tombstone entries across every CF
<code>Tidesdb_tombstone_ratio</code>	Database-wide tombstone count divided by entry count (0.0 to 1.0)
<code>Tidesdb_max_sst_tombstone_density</code>	Worst single-SSTable tombstone density observed
<code>Tidesdb_max_sst_tombstone_density_level</code>	LSM level (1-based) where the worst SSTable lives

`SHOW ENGINE TIDESDB STATUS` includes a `--- Tombstones ---` block with the same four numbers in human-readable form.

Auto Compact After Range Delete

Bulk operations such as sliding-window expiry, tenant eviction, and time-bucketed log rotation share a shape: a multi-row DELETE over a known primary-key range followed by a wait for compaction to physically reclaim the tombstoned space. The tombstone-density trigger handles this eventually, but a caller that already knows the range can skip the wait by asking the engine to compact that range synchronously at end-of-statement.

The session variable `tidesdb_compact_after_range_delete_min_rows` enables this:

```
SET SESSION tidesdb_compact_after_range_delete_min_rows = 100000;  
DELETE FROM events WHERE ts < NOW() - INTERVAL 30 DAY;  
-- end_bulk_delete fires tidesdb_compact_range over the touched PK range
```

The default is `0`, which disables the feature. A non-zero value is both an opt-in and a row-count threshold: only DELETE statements that touch at least that many rows trigger the synchronous compaction, so a one-row primary-key DELETE never pays the cost. The plugin tracks the comparable minimum and maximum primary-key bytes seen during the statement (no additional locking, no scan, just two `std::string` swaps per `delete_row` call) and on `end_bulk_delete` calls `tidesdb_compact_range` over the observed range on the table's primary CF.

Secondary index tombstones are not directly compacted by this feature, since their key shape is `(col_values, pk)` rather than just the PK and a PK range does not bound a secondary-index range. Secondary-index tombstones are reclaimed by the per-CF tombstone density trigger above instead, which is the natural division of labor.

The compaction is synchronous and runs on the caller's thread, so the trade-off is that the DELETE statement returns only after the compaction commits. The threshold should be set high enough that the synchronous compaction time is small relative to the DELETE work that triggered it. Typical values are 10000 to 1000000 depending on row size and disk throughput.

Table Options

TidesDB exposes a rich set of per-table options that control the underlying column family's behavior. These are specified as table-level options in `CREATE TABLE` and are baked into the column family at creation time. They appear in `SHOW CREATE TABLE` output.

Compression

The engine supports several compression algorithms for SSTables. The default is LZ4:

```
CREATE TABLE archive (  
    id    INT PRIMARY KEY,  
    data TEXT  
) ENGINE=TIDESDB COMPRESSION='ZSTD';
```

Available choices are `NONE`, `SNAPPY`, `LZ4`, `ZSTD`, and `LZ4_FAST`. Different tables can use different compression algorithms. ZSTD tends to give the best compression ratio, while LZ4 and LZ4_FAST favor speed.

An encrypted table is the one exception. Its data column family is created with compression forced to `NONE` regardless of this option, because the rows are already ciphertext by the time they reach the library and ciphertext does not compress - running a codec over it on every flush and compaction would spend CPU for no space saving. The table's secondary-index column families hold unencrypted comparable keys and keep whatever algorithm was selected.

Write Buffer Size

The write buffer is the in-memory skip list that absorbs writes before they are flushed to disk as an SSTable. A larger buffer means fewer, larger flushes, which can improve write throughput at the cost of higher memory usage:

```
CREATE TABLE high_write (  
    id  INT PRIMARY KEY,  
    val VARCHAR(200)  
) ENGINE=TIDESDB WRITE_BUFFER_SIZE=16777216;  -- 16 MB
```

The default is 64 MB, matching `TDB_DEFAULT_WRITE_BUFFER_SIZE` in the underlying library. When `tidesdb_unified_memtable=ON` (the default) the per-CF buffer is overridden by `tidesdb_unified_memtable_write_buffer_size` since every CF shares a single memtable, so this option only affects per-CF mode.

Bloom Filters

By default, TidesDB creates bloom filters for each SSTable, which allow point lookups to skip SSTables that definitely do not contain the requested key. The false positive rate is configurable:

```
-- Disable bloom filters entirely
CREATE TABLE no_bloom (id INT PRIMARY KEY, v INT) ENGINE=TIDESDB
BLOOM_FILTER=0;

-- Very low FPR (10 parts per 10,000 = 0.1%)
CREATE TABLE precise (id INT PRIMARY KEY, v INT) ENGINE=TIDESDB
BLOOM_FPR=10;
```

The `BLOOM_FPR` value is specified in parts per 10,000. The default of 100 gives a 1% false positive rate.

Sync Mode

Controls how aggressively the engine flushes writes to durable storage:

```
-- No fsync (fastest, data at risk on crash)
CREATE TABLE fast_writes (id INT PRIMARY KEY, v INT) ENGINE=TIDESDB
SYNC_MODE='NONE';

-- Periodic fsync every 500ms
CREATE TABLE balanced (id INT PRIMARY KEY, v INT)
ENGINE=TIDESDB SYNC_MODE='INTERVAL' SYNC_INTERVAL_US=500000;

-- fsync every write (safest, slowest)
CREATE TABLE durable (id INT PRIMARY KEY, v INT) ENGINE=TIDESDB
SYNC_MODE='FULL';
```

The default is `FULL`. For benchmarking or non-critical data, `NONE` can dramatically improve throughput.

`SYNC_MODE` governs how the engine fsyncs a family's SSTable files (the klog and vlog). Under `tidesdb_unified_memtable=ON` (the default) WAL durability is governed instead by `tidesdb_unified_memtable_sync_mode`, since every table shares one write-ahead log rather than a per-CF WAL; in that mode this per-table option controls only the SSTable sync for the family, not commit durability.

B+tree SSTable Format

TidesDB can optionally use a B+tree layout for the key log within SSTables instead of the default block-based format. The SSTable still consists of a key log and a value log; only the key log's internal organization changes. This is a per-table choice:


```
CREATE TABLE btree_table (id INT PRIMARY KEY, v INT) ENGINE=TIDESDB
USE_BTREE=1;
```

When `ANALYZE TABLE` is run on a B+tree-formatted table, the output includes additional statistics about the tree structure (node counts, heights).

Per-Index USE_BTREE

The B+tree format can also be set on individual secondary indexes, overriding the table-level default. This allows mixing LSM and B+tree indexes on the same table:

```
CREATE TABLE mixed (
  id INT NOT NULL PRIMARY KEY,
  a INT,
  b INT,
  KEY idx_a (a) USE_BTREE=1,      -- B+tree index
  KEY idx_b (b)                  -- inherits table default (LSM)
) ENGINE=TIDESDB;
```

`SHOW KEYS` reflects the per-index type `idx_a` shows `BTREE`, `idx_b` shows `LSM`. The per-index option is also honoured during `ALTER TABLE ... ADD INDEX ... USE_BTREE=1`.

Block Indexes

When using the default block-based key log format (i.e., `USE_BTREE=0`), TidesDB builds block-level indexes inside each SSTable to speed up key lookups. Block indexes are enabled by default. You can disable them, or tune their sampling ratio and prefix length:

```
-- Disable block indexes entirely
CREATE TABLE no_block_idx (id INT PRIMARY KEY, v INT) ENGINE=TIDESDB
BLOCK_INDEXES=0;

-- Tune block index parameters
CREATE TABLE custom_idx (
  id INT PRIMARY KEY,
  v VARCHAR(200)
) ENGINE=TIDESDB
BLOCK_INDEXES=1
INDEX_SAMPLE_RATIO=4
BLOCK_INDEX_PREFIX_LEN=32;
```

`BLOCK_INDEXES` enables or disables block-level indexes within the SSTable key log (default enabled). `INDEX_SAMPLE_RATIO` controls how frequently index entries are sampled from the key log blocks (default 1, meaning every block is indexed). Higher values reduce index size at the cost of more binary search steps during lookups. `BLOCK_INDEX_PREFIX_LEN` sets the byte length of the key prefix stored in each block index entry (default 16). A longer prefix improves point-lookup accuracy but increases index memory usage.

Key Log Value Threshold

Each SSTable consists of a key log and a value log. Small values are stored inline in the key log alongside the key, while larger values are written to the separate value log with only a pointer kept in the key log. The `KLOG_VALUE_THRESHOLD` option controls the cutoff in bytes:

```
-- Store values up to 1 KB inline in the key log
CREATE TABLE inline_vals (
  id INT PRIMARY KEY,
  val VARCHAR(200)
) ENGINE=TIDESDB KLOG_VALUE_THRESHOLD=1024;
```

The default is 512 bytes. Raising the threshold keeps more data inline, which benefits workloads with small rows by avoiding an extra indirection through the value log. Lowering it reduces key log size, which can improve cache efficiency for tables with large row values.

Minimum Disk Space

`MIN_DISK_SPACE` sets the minimum free disk space (in bytes) that TidesDB requires before it will flush memtables or run compaction. If free space drops below this threshold, background operations are paused to prevent filling the disk:

```
CREATE TABLE guarded (  
  id INT PRIMARY KEY,  
  v INT  
) ENGINE=TIDESDB MIN_DISK_SPACE=1073741824; -- 1 GB
```

The default is 100 MB.

LSM-Tree Tuning

Several options let you tune the shape and behavior of the LSM-tree:

```
CREATE TABLE tuned (  
  id INT PRIMARY KEY,  
  v VARCHAR(200)  
) ENGINE=TIDESDB  
  LEVEL_SIZE_RATIO=8  
  MIN_LEVELS=3  
  DIVIDING_LEVEL_OFFSET=1  
  SKIP_LIST_MAX_LEVEL=16  
  SKIP_LIST_PROBABILITY=25  
  L1_FILE_COUNT_TRIGGER=4  
  L0_QUEUE_STALL_THRESHOLD=10;
```

`LEVEL_SIZE_RATIO` controls how much larger each level is compared to the previous one (default 10). `MIN_LEVELS` sets the minimum depth of the LSM-tree (default 1, matching `TDB_DEFAULT_MIN_LEVELS`). `DIVIDING_LEVEL_OFFSET` sets the offset used to compute the dividing level, which serves as the primary compaction target (default 1, matching `TDB_DEFAULT_DIVIDING_LEVEL_OFFSET`). The dividing level is calculated as $\text{num_levels} - 1 - \text{DIVIDING_LEVEL_OFFSET}$. TidesDB does not use traditional selectable compaction policies (like Leveled or Tiered); instead, it employs three complementary merge strategies - full preemptive merge, dividing merge, and partitioned merge - that are automatically selected based on the current state of the LSM-tree relative to the dividing level. The skip list parameters control the in-memory memtable structure. `L1_FILE_COUNT_TRIGGER` determines how many SSTables can accumulate at Level 1 before compaction merges them into deeper levels. `L0_QUEUE_STALL_THRESHOLD` sets how many immutable

memtables can be queued for flush before the engine stalls new writes to allow flushes to catch up (default 10, matching `TDB_DEFAULT_L0_QUEUE_STALL_THRESHOLD`). Note that the flush threshold is adaptive, under idle conditions the active memtable can grow to 150% of `WRITE_BUFFER_SIZE` before flushing, dropping to 100% under pressure. Worst-case memtable memory per column family is approximately $(\text{WRITE_BUFFER_SIZE} \times 1.5) + (\text{WRITE_BUFFER_SIZE} \times \text{L0_QUEUE_STALL_THRESHOLD})$ since the stall threshold itself bounds how many immutable memtables can queue before new writes block.

Object Store Compaction Tuning

When using S3 object store mode, two per-table options control compaction behavior for remote storage:

```
-- Reduce compaction frequency for write-heavy tables in object store
mode
CREATE TABLE remote_logs (
  id BIGINT PRIMARY KEY,
  msg TEXT
) ENGINE=TIDESDB OBJECT_LAZY_COMPACT=1 OBJECT_PREFETCH_COMPACT=1;
```

`OBJECT_LAZY_COMPACT` (default 0) doubles the L1 file count compaction trigger when enabled, reducing the frequency of compaction and thus remote I/O at the cost of higher read amplification. `OBJECT_PREFETCH_COMPACT` (default 1) downloads all input SSTables in parallel before the compaction merge begins, using bounded threads. When disabled, input SSTables are downloaded on demand during merge source creation one at a time. These options are persisted to the column family config and have corresponding session-level defaults `tidesdb_default_object_lazy_compact` and `tidesdb_default_object_prefetch_compact`. In non-object-store deployments, these options have no effect.

Tombstone Density

Two per-table options arm the post-flush tombstone-density compaction trigger described under [Tombstone Density Trigger](#):

```
CREATE TABLE events (
  id    BIGINT PRIMARY KEY,
  ts    DATETIME,
  body  TEXT,
  KEY (ts)
) ENGINE=TIDESDB
TOMBSTONE_DENSITY_TRIGGER=5000      -- 50%, parts per 10000
TOMBSTONE_DENSITY_MIN_ENTRIES=2048;
```

`TOMBSTONE_DENSITY_TRIGGER` is in parts per 10000, where `5000` means 0.50. Default `0` disables the check. `TOMBSTONE_DENSITY_MIN_ENTRIES` (default `1024`) is a floor on the SSTable entry count before the density check considers it. Both options are session-defaultable via `tidesdb_default_tombstone_density_trigger` and `tidesdb_default_tombstone_density_min_entries`. `ALTER TABLE` updates the runtime CF config so changes take effect on the next post-flush check.

Combining Multiple Options

Options can be freely combined:

```
CREATE TABLE optimized (
  id  INT PRIMARY KEY,
  val VARCHAR(100)
) ENGINE=TIDESDB
COMPRESSION='ZSTD'
WRITE_BUFFER_SIZE=8388608
BLOOM_FILTER=1
BLOOM_FPR=50
SYNC_MODE='FULL'
KLOG_VALUE_THRESHOLD=1024
LO_QUEUE_STALL_THRESHOLD=6
ISOLATION_LEVEL='REPEATABLE_READ';
```

TTL (Time-To-Live)

TidesDB supports automatic expiration of rows, at both the table level and the per-row level. Expired rows are silently filtered out during reads and eventually reclaimed during compaction.

Table-Level TTL

Every row inserted into the table expires after the specified number of seconds:

```
CREATE TABLE sessions (  
  id      INT PRIMARY KEY,  
  token   VARCHAR(100)  
) ENGINE=TIDESDB TTL=3600;  -- 1 hour  
  
INSERT INTO sessions VALUES (1, 'abc123');  
-- After 3600 seconds, this row will no longer be returned by queries
```

Per-Row TTL

A column can be designated as the TTL source using the `TTL` field option. The value in that column specifies the row's lifetime in seconds from the time of insertion:

```
CREATE TABLE cache (  
  id      INT PRIMARY KEY,  
  val     VARCHAR(100),  
  ttl_sec INT `TTL`=1  
) ENGINE=TIDESDB;  
  
INSERT INTO cache VALUES (1, 'short-lived', 5);      -- expires in 5  
seconds  
INSERT INTO cache VALUES (2, 'long-lived', 86400);   -- expires in 1 day  
INSERT INTO cache VALUES (3, 'permanent', 0);       -- 0 means no  
expiration
```

When a per-row TTL column is present and has a non-zero value, it takes precedence over the table-level TTL. If the per-row value is zero, the table-level default applies. If neither is set, the row never expires.

When a row is updated, its TTL is recomputed from the new column value, effectively refreshing its expiration.

Session-Level TTL

A per-session TTL can be set with the `tidesdb_ttl` session variable. This applies a TTL to all inserts and updates on any TidesDB table for the duration of the session, even tables that were not created with a TTL option:

```

SET SESSION tidesdb_ttl = 300;                -- 5 minutes
INSERT INTO events (id, data) VALUES (1, 'temporary'); -- expires in 300s
SET SESSION tidesdb_ttl = 0;                  -- back to table
default

```

The `SET STATEMENT` syntax can scope the TTL to a single statement:

```

SET STATEMENT tidesdb_ttl = 60 FOR INSERT INTO events (id, data)
VALUES (2, 'one-minute');

```

The priority order is per-row TTL column > session `tidesdb_ttl` > table-level `TTL` option > no expiration.

Data-at-Rest Encryption

TidesDB can encrypt row data before writing it to the column family. Encryption uses MariaDB's built-in key management infrastructure, so it works with any configured key management plugin (e.g., `file_key_management`).

```

CREATE TABLE secrets (
  id INT NOT NULL PRIMARY KEY,
  val VARCHAR(100)
) ENGINE=TIDESDB `ENCRYPTED`=YES;

```

Each row is encrypted individually. The on-disk format is a 4-byte little-endian key version followed by a 16-byte random IV followed by the ciphertext. The key version prefix lets the engine decrypt rows that were written under an older version of the key after a rotation, since `encryption_key_get(key_id, key_version)` resolves the exact key bytes used at write time. On read the engine decrypts transparently. You can specify which encryption key ID to use:

```

CREATE TABLE classified (
  id INT NOT NULL PRIMARY KEY,
  data TEXT
) ENGINE=TIDESDB `ENCRYPTED`=YES `ENCRYPTION_KEY_ID`=2;

```

Encryption works with all other features, including secondary indexes, BLOB columns, and TTL. The secondary index keys themselves are not encrypted (they need to remain comparable for seeking), but the row data pointed to by those keys is encrypted in the data column family. Because ciphertext is incompressible, the engine forces that data column family's compression to `NONE` regardless of the table's `COMPRESSION` option (see Compression); the index CFs are unaffected.

If `encryption_key_get()` cannot return the requested key (rotation hole, keyring plugin not loaded, version never existed), the encrypt or decrypt call fails closed - the engine logs the failure and returns an error to the SQL layer rather than feeding uninitialised stack bytes into `encryption_crypt`. A row written with a key version no longer in the keyring becomes unreadable until the key is restored; the engine never silently mis-encrypts or returns zeroed plaintext.

Full-Text Search

TidesDB supports FULLTEXT indexes for natural language and boolean mode full-text search, with BM25 relevance ranking. Each FULLTEXT index is stored as a dedicated column family containing an inverted index, where each key-value pair maps a (term, document) pair to its term frequency and document length.

```
CREATE TABLE articles (  
  id    INT NOT NULL PRIMARY KEY,  
  title VARCHAR(200),  
  body  TEXT,  
  FULLTEXT ft_content (title, body)  
) ENGINE=TIDESDB;  
  
INSERT INTO articles VALUES (1, 'MySQL Tutorial', 'DBMS stands for DataBase  
Management System');  
INSERT INTO articles VALUES (2, 'How To Use MySQL', 'After you went through  
a tutorial you can start');  
INSERT INTO articles VALUES (3, 'Optimizing MySQL', 'In this tutorial we  
show optimization techniques');
```

Natural Language Mode

The default search mode. The engine tokenizes the query, scans the inverted index for each term, computes BM25 relevance scores, and returns results sorted by score:


```
SELECT id, title, MATCH(title, body) AGAINST('tutorial') AS score
FROM articles WHERE MATCH(title, body) AGAINST('tutorial')
ORDER BY score DESC;
```

Multi-term queries accumulate BM25 scores across all query terms. Documents matching more terms and with higher term frequencies rank higher, normalized by document length.

Boolean Mode

Boolean mode supports required (+), excluded (-), prefix wildcard (*), and exact phrase ("...") operators:

```
-- Documents must contain 'mysql' AND 'tutorial'
SELECT * FROM articles
WHERE MATCH(title, body) AGAINST('+mysql +tutorial' IN BOOLEAN MODE);

-- Documents must contain 'mysql' but NOT 'tutorial'
SELECT * FROM articles
WHERE MATCH(title, body) AGAINST('+mysql -tutorial' IN BOOLEAN MODE);

-- Prefix wildcard: matches 'optimization', 'optimizing', etc.
SELECT * FROM articles
WHERE MATCH(title, body) AGAINST('optim*' IN BOOLEAN MODE);

-- Exact phrase: words must appear consecutively in this order
SELECT * FROM articles
WHERE MATCH(title, body) AGAINST('"database management system"' IN BOOLEAN
MODE);

-- Phrase with operator: require phrase, exclude a word
SELECT * FROM articles
WHERE MATCH(title, body) AGAINST('+ "management system" -tutorial' IN
BOOLEAN MODE);
```

Phrase queries use the inverted index to find candidate documents containing all phrase words, then verify the exact word sequence by re-tokenizing the document text. This matches the approach used by InnoDB's FTS implementation.

BM25 Ranking

Relevance scores are computed using the Okapi BM25 algorithm:

$$\text{IDF}(t) = \ln(1 + (N - \text{df} + 0.5) / (\text{df} + 0.5))$$
$$\text{score} = \text{IDF} * (\text{tf} * (\text{k1} + 1)) / (\text{tf} + \text{k1} * (1 - \text{b} + \text{b} * |\text{d}| / \text{avgdl}))$$

where N is the total number of documents, df is the document frequency of the term, tf is the term frequency in the document, $|\text{d}|$ is the document length in tokens, and avgdl is the average document length. The parameters k1 and b are configurable via system variables (see below). The IDF formula includes the $+1$ inside the logarithm to guarantee non-negative scores for all terms, matching the Lucene variant that is now the industry standard.

Tokenization

The engine uses a charset-aware tokenizer that correctly handles multi-byte character sets including UTF-8, CJK characters, Cyrillic, Greek, and other Unicode scripts. Text is split on word boundaries using MariaDB's charset classification tables, then lowercased using the charset's case-folding rules. Words shorter than `tidesdb_fts_min_word_len` or longer than `tidesdb_fts_max_word_len` are excluded from the index and search queries.

Stop Words

Common words like “the”, “is”, “a”, “of” are excluded from the full-text index to reduce index bloat and improve search quality. By default, TidesDB uses the same default stop word list as InnoDB (`information_schema.INNODB_FT_DEFAULT_STOPWORD` — 36 entries, 35 distinct words, since InnoDB lists “the” twice). Stop words are filtered during tokenization, so they are never stored in the inverted index and never match in search queries.

The stop word list can be customized via the `tidesdb_ft_stopword_table` system variable. When set to a `db_name/table_name` string, the engine loads stop words from the specified TidesDB table (which must have a `value` VARCHAR column containing one word per row). When set to NULL or empty string, the default stop words are restored:

```
-- Use a custom stop word table
CREATE TABLE mydb.my_stopwords (value VARCHAR(50)) ENGINE=TidesDB;
INSERT INTO mydb.my_stopwords (value) VALUES ('custom'), ('words'),
('here');
SET GLOBAL tidesdb_ft_stopword_table = 'mydb/my_stopwords';

-- Restore defaults
SET GLOBAL tidesdb_ft_stopword_table = NULL;
```

After changing the stop word table, existing FULLTEXT indexes should be rebuilt with `ALTER TABLE ... DROP INDEX ..., ADD FULLTEXT INDEX ...` to reflect the new stop word list.

Blend Characters

Blend characters are treated as both word separators and valid word characters during tokenization. When a blend character appears inside a token, the tokenizer emits both the full blended form and the individual sub-parts. This enables Romance language elision (Italian, French, Catalan) and names with apostrophes to be searchable by any component or the full form.

For example, with `tidesdb_fts_blend_chars = ''`:

Input	Indexed Tokens
L'aria	l'aria, aria
Dell'aria	dell'aria, dell, aria
O'Malley	o'malley, malley

Searching for any of the indexed forms will match the document. The full blended form scores higher than a sub-part match because both the blended token and the sub-part contribute to the BM25 score.

```
-- Enable apostrophe blending for Italian/French
SET GLOBAL tidesdb_fts_blend_chars = '';

CREATE TABLE articoli (
  id    INT PRIMARY KEY,
  testo TEXT,
  FULLTEXT (testo)
) ENGINE=TIDESDB;

INSERT INTO articoli VALUES (1, "L'aria fresca della montagna");
INSERT INTO articoli VALUES (2, "Dell'aria pura si respira bene");

-- All three queries match:
SELECT * FROM articoli WHERE MATCH(testo) AGAINST('aria');           -- sub-
part
SELECT * FROM articoli WHERE MATCH(testo) AGAINST("l'aria");        --
blended
SELECT * FROM articoli WHERE MATCH(testo) AGAINST("dell'aria");     --
blended
```

The default is empty (no blend characters). The setting is global and takes effect for all subsequent FULLTEXT indexing and query operations. After changing blend characters, existing indexes should be rebuilt.

Multi-Column Indexes

A single FULLTEXT index can span multiple columns. The engine concatenates the text from all indexed columns into a single document for tokenization and scoring:

```
CREATE TABLE docs (  
  id      INT NOT NULL PRIMARY KEY,  
  title   VARCHAR(200),  
  summary TEXT,  
  body    TEXT,  
  FULLTEXT (title, summary, body)  
) ENGINE=TIDESDB;
```

Index Maintenance

FULLTEXT index entries are maintained transactionally within the same transaction as row data changes. When a row is inserted, the engine tokenizes the document, counts term frequencies, and writes one inverted index entry per unique term. When a row is deleted, the corresponding index entries are removed. When a row is updated and the indexed columns have changed, the old entries are removed and new entries are inserted. Global document count and average document length statistics are maintained atomically in the data column family for BM25 scoring.

FTS System Variables

Variable	Default	Description
<code>tidesdb_fts_min_word_len</code>	3	Minimum word length in characters for indexing
<code>tidesdb_fts_max_word_len</code>	84	Maximum word length in characters for indexing
<code>tidesdb_fts_bm25_k1</code>	1.2	BM25 k1 parameter (term-frequency saturation)
<code>tidesdb_fts_bm25_b</code>	0.75	BM25 b parameter (document-length normalization, 0-1)
<code>tidesdb_ft_stopword_table</code>	NULL	Custom stop word table (<code>db_name/table_name</code>). NULL uses InnoDB defaults.
<code>tidesdb_fts_blend_chars</code>	empty	Characters treated as both separators and word chars. Set to <code>'</code> for Italian/French.

Vector Search

TidesDB supports approximate nearest neighbor (ANN) vector search through MariaDB's built-in MHNSW (Multi-layer Hierarchical Navigable Small World) vector index. The server handles all graph construction and search logic transparently; TidesDB provides the storage layer for both the main table data and the hidden MHNSW graph structure (hindex).

```
CREATE TABLE embeddings (  
  id    INT NOT NULL PRIMARY KEY,  
  title VARCHAR(200),  
  v     VECTOR(384) NOT NULL,  
  VECTOR INDEX (v)  
) ENGINE=TIDESDB;  
  
INSERT INTO embeddings VALUES (1, 'cat picture', Vec_FromText('[0.1, 0.9,  
...]'));  
INSERT INTO embeddings VALUES (2, 'dog picture', Vec_FromText('[0.2, 0.8,  
...]'));
```

ANN Search

Vector search uses `ORDER BY VEC_DISTANCE_EUCLIDEAN()` or `VEC_DISTANCE_COSINE()` with a `LIMIT` clause. The MHNSW index provides approximate nearest neighbor results without scanning the entire table:

```
-- Find 5 nearest neighbors by Euclidean distance
SELECT id, title,
       VEC_DISTANCE_EUCLIDEAN(v, Vec_FromText('[0.15, 0.85, ...]')) AS dist
FROM embeddings
ORDER BY dist
LIMIT 5;

-- Cosine distance
SELECT id, title,
       VEC_DISTANCE_COSINE(v, Vec_FromText('[0.15, 0.85, ...]')) AS dist
FROM embeddings
ORDER BY dist
LIMIT 5;
```

Index Options

The MHNSW index accepts two optional parameters:

```
CREATE TABLE docs (
  id INT PRIMARY KEY,
  v  VECTOR(128) NOT NULL,
  VECTOR INDEX (v) M=12 DISTANCE='cosine'
) ENGINE=TIDESDB;
```

`M` controls the number of neighbors per node in the MHNSW graph (default 6, range 3 to 200). Higher values improve recall at the cost of slower inserts and more memory. `DISTANCE` selects the distance metric, either `euclidean` (default) or `cosine`.

DML Support

All DML operations are fully supported on vector-indexed tables. INSERT adds the vector to the MHNSW graph, DELETE removes it, and UPDATE on the vector column invalidates the old graph node and inserts a new one with the updated vector. The engine correctly handles the interleaved `record[0]/record[1]` access pattern that the MHNSW graph maintenance requires for BLOB-backed vector data.

Limitations

Partitioned tables do not support vector indexes. Only one vector index per table is currently supported by MariaDB's MHNSW implementation.

Spatial Indexes

TidesDB supports SPATIAL indexes for geographic and geometric data using a Hilbert curve encoding on the LSM tree. Instead of a traditional R-tree (which requires in-place node updates that conflict with LSM append-only semantics), each geometry's MBR center is mapped to a 64-bit Hilbert curve value and stored as a sorted key in a dedicated column family.

```
CREATE TABLE places (  
  id      INT NOT NULL PRIMARY KEY,  
  name    VARCHAR(100),  
  location GEOMETRY NOT NULL,  
  SPATIAL INDEX (location)  
) ENGINE=TIDESDB;  
  
INSERT INTO places VALUES (1, 'NYC', ST_GeomFromText('POINT(40.7128  
-74.0060)'));  
INSERT INTO places VALUES (2, 'LA', ST_GeomFromText('POINT(34.0522  
-118.2437)'));  
INSERT INTO places VALUES (3, 'Chicago', ST_GeomFromText('POINT(41.8781  
-87.6298)'));
```

Spatial Queries

All MBR-based spatial predicates are supported through the standard MariaDB spatial functions:

```

-- Find cities within a bounding box
SELECT name FROM places
WHERE MBRIntersects(location,
    ST_GeomFromText('POLYGON((39 -76, 43 -76, 43 -72, 39 -72, 39 -76))'));

-- Find cities contained within a region
SELECT name FROM places
WHERE MBRContains(
    ST_GeomFromText('POLYGON((25 -125, 45 -125, 45 -70, 25 -70, 25 -125))'),
    location);

-- Find cities within a region (equivalent to MBRContains with swapped
args)
SELECT name FROM places
WHERE MBRWithin(location,
    ST_GeomFromText('POLYGON((25 -125, 45 -125, 45 -70, 25 -70, 25 -125))'));

```

The supported spatial predicates are `MBRIntersects`, `MBRContains`, `MBRWithin`, `MBREquals`, and `MBRDisjoint`. All geometry types are supported (POINT, LINESTRING, POLYGON, MULTIPOINT, MULTILINESTRING, MULTIPOLYGON, GEOMETRYCOLLECTION).

How It Works

The spatial index stores each geometry as a single entry in a dedicated column family. The key is the 64-bit Hilbert curve value of the MBR center point (in big-endian for lexicographic ordering), followed by the primary key suffix. The value stores the full MBR (4 doubles = 32 bytes) for predicate evaluation during scans. The Hilbert curve provides excellent spatial locality - geographically nearby geometries tend to have numerically adjacent Hilbert values, which means they cluster together in the LSM tree and benefit from sequential I/O during range scans.

Spatial queries use Hilbert range decomposition to avoid scanning the entire index. The query bounding box is mapped to a coarse grid on the Hilbert curve, and only the grid cells that overlap the box are included. These cells are sorted and merged into contiguous Hilbert ranges, and the engine seeks directly to each range - skipping over large portions of the curve that fall outside the query box. Each candidate entry within a range undergoes exact MBR predicate filtering to eliminate false positives from curve approximation. For a query box covering 1% of the coordinate space, this typically produces 10-50 targeted seeks instead of a full index scan. The one predicate this does not accelerate is `MBRDisjoint`: a geometry that does not overlap the query box can lie anywhere on the curve, so a disjoint query scans the full index and filters each entry rather than seeking to a bounded set of ranges. INSERT, UPDATE, and DELETE operations maintain the spatial index transactionally alongside the row data, following the same pattern as secondary indexes and full-text indexes.

Generated Columns

The engine supports both `VIRTUAL` and `STORED` (persistent) generated columns. Virtual columns are computed on read and never physically stored. Stored columns are computed on write and persisted as part of the row data, so they can be read back without recomputation.

```
CREATE TABLE orders (  
  id          INT PRIMARY KEY,  
  price       DECIMAL(10,2),  
  qty         INT,  
  total       DECIMAL(10,2) AS (price * qty) VIRTUAL,  
  category    VARCHAR(10) AS (  
    CASE WHEN price >= 100 THEN 'premium' ELSE 'standard' END  
  ) VIRTUAL  
) ENGINE=TIDESDB;  
  
INSERT INTO orders (id, price, qty) VALUES (1, 49.99, 3);  
SELECT * FROM orders;  
-- total = 149.97, category = 'standard'
```

JSON

MariaDB's `JSON` type is an alias for a text type (typically `LONGTEXT`), so JSON storage and JSON querying work normally on TidesDB tables. JSON functions like `JSON_VALUE()`, `JSON_EXTRACT()`, `JSON_SET()`, and `JSON_CONTAINS()` are evaluated by the MariaDB server.

For efficient filtering on JSON paths, the recommended pattern is to use generated columns that extract the JSON paths you care about, then index those generated columns:

```

CREATE TABLE docs (
  id    INT NOT NULL PRIMARY KEY,
  data  LONGTEXT,
  name  VARCHAR(100) AS (JSON_VALUE(data, '$.name')) PERSISTENT,
  age   INT AS (JSON_VALUE(data, '$.age')) PERSISTENT,
  KEY idx_name (name),
  KEY idx_age (age)
) ENGINE=TIDESDB;

INSERT INTO docs (id, data) VALUES
(1, '{"name":"Alice","age":30,"tags":["admin","dev"]}'),
(2, '{"name":"Bob","age":25,"tags":["dev"]}');

-- Uses idx_name
SELECT * FROM docs WHERE name='Alice';

-- Uses idx_age
SELECT * FROM docs WHERE age >= 30;

-- Server-evaluated JSON predicate (typically not indexable unless you
extract it)
SELECT * FROM docs WHERE JSON_CONTAINS(data, '"admin"', '$.tags');

```

This pattern provides engine-native indexing via normal secondary indexes, while keeping JSON manipulation and path extraction in standard SQL.

Online DDL

The engine classifies `ALTER TABLE` operations into three tiers, each with different performance characteristics.

Instant Operations

These complete instantly without rebuilding data. MariaDB rewrites the `.frm` metadata file, and the change takes effect immediately:

- Adding a column (`ADD COLUMN`)
- Dropping a column (`DROP COLUMN`)
- Renaming a column or index
- Changing a column's default value
- Changing table-level options (like `SYNC_MODE`, `COMPRESSION`, `BLOOM_FPR`, etc.)

When table-level options are changed, the engine applies the new configuration to all live column families (data CF and secondary index CFs) via `tidesdb_cf_update_runtime_config()` with `persist_to_disk=1`. The changes take effect immediately for new operations (new SSTables, new memtables, WAL sync mode). Existing SSTables retain their original settings and are read correctly. Share-level cached options such as isolation level, TTL, and encryption settings are also updated in memory.

Adding or dropping columns is instant because the packed row format includes a self-describing header that records the null bitmap size and field count at write time. When reading old rows written before the schema change, the engine adapts automatically, added columns receive their `DEFAULT` value, and dropped columns are silently skipped.

```
ALTER TABLE events ADD COLUMN priority INT NOT NULL DEFAULT 0,
ALGORITHM=INSTANT;
ALTER TABLE events DROP COLUMN priority, ALGORITHM=INSTANT;
ALTER TABLE events ALTER COLUMN data SET DEFAULT 'none', ALGORITHM=INSTANT;
ALTER TABLE events CHANGE kind event_kind VARCHAR(50), ALGORITHM=INSTANT;
ALTER TABLE events SYNC_MODE='NONE', ALGORITHM=INSTANT;
```

Inplace Operations

Adding or dropping non-FULLTEXT, non-SPATIAL secondary indexes is done inplace. When a new index is added, the engine creates a new column family for it, then performs a full table scan to populate all index entries. This runs with no server-level lock blocking (`HA_ALTER_INPLACE_NO_LOCK`), so concurrent reads and writes can proceed during the index build. When adding a `UNIQUE` index, the engine checks for duplicate values during the population scan and aborts with `ER_DUP_ENTRY` if any are found, preserving all existing rows. If any row's index put fails the entire ALTER is rolled back rather than shipping a silently incomplete index.

```
ALTER TABLE events ADD INDEX idx_ts (ts), ALGORITHM=INPLACE;
ALTER TABLE events DROP INDEX idx_ts, ALGORITHM=INPLACE;

-- Add and drop in a single statement
ALTER TABLE events ADD INDEX idx_kind (event_kind), DROP INDEX idx_ts,
ALGORITHM=INPLACE;
```

For large tables, the index population is batched in groups of 100 rows per transaction commit, to avoid unbounded memory growth in the transaction's write buffer.

`ADD FULLTEXT` and `ADD SPATIAL` are not eligible for `ALGORITHM=INPLACE` because the inplace builder cannot populate the FTS or spatial column family from a row-by-row scan - the tokenizer and Hilbert-curve writer paths only run inside `write_row`. `check_if_supported_inplace_alter` returns `HA_ALTER_INPLACE_NOT_SUPPORTED` for these with a clear reason, so MariaDB falls back to `ALGORITHM=COPY` which routes every existing row through `write_row` and back-fills the new index correctly. An explicit `ALTER TABLE ... ADD FULLTEXT(...), ALGORITHM=INPLACE` is rejected with the reason; the default algorithm (or `ALGORITHM=COPY`) succeeds.

Copy Operations

Changing column types or altering the primary key require a full table copy:

```
ALTER TABLE events MODIFY COLUMN data MEDIUMTEXT;  
ALTER TABLE events DROP PRIMARY KEY, ADD PRIMARY KEY (id, ts);
```

The engine explicitly rejects `ALGORITHM=INPLACE` for these operations with a clear error message, so you will never accidentally trigger a slow copy when you expected an instant change.

ANALYZE TABLE

Running `ANALYZE TABLE` on a TidesDB table produces detailed internal statistics as note-level messages in the result set:

```

MariaDB [demo]> ANALYZE TABLE products;
+-----+-----+-----+-----+
| Table          | Op      | Msg_type | Msg_text
|
+-----+-----+-----+-----+
| demo.products | analyze | Note     | [TIDESDB] CF 'demo__products'
total_keys=10  data_size=636 bytes  memtable=0 bytes |
|               |         |          | levels=5  read_amp=2.00
cache_hit=0.0%                                     |
| demo.products | analyze | Note     | [TIDESDB] avg_key=18.8 bytes
avg_value=44.0 bytes                               |
| demo.products | analyze | Note     | [TIDESDB] level 1  sstables=0
size=0 bytes  keys=0                               |
| demo.products | analyze | Note     | [TIDESDB] level 2  sstables=1
size=636 bytes  keys=10                            |
| demo.products | analyze | Note     | [TIDESDB] level 3  sstables=0
size=0 bytes  keys=0                               |
| demo.products | analyze | Note     | [TIDESDB] level 4  sstables=0
size=0 bytes  keys=0                               |
| demo.products | analyze | Note     | [TIDESDB] level 5  sstables=0
size=0 bytes  keys=0                               |
| demo.products | analyze | Note     | [TIDESDB] idx CF
'demo__products__idx_idx_category' keys=10  data_size=449 bytes  |
|               |         |          | levels=5
|
| demo.products | analyze | status   | OK
|
+-----+-----+-----+-----+

```

The output includes the total number of keys, data size, memtable size, number of LSM levels, read amplification factor, cache hit rate, average key and value sizes, and per-level SSTable counts and sizes. For tables with secondary indexes, each index CF's statistics are reported separately.

When the column family has logged any user bytes, a per-CF write-amplification note is also emitted:

```

| demo.products | analyze | Note     | [TIDESDB] WA  user=4096
wal=4096  flush=8192 (1 ssts)  compact_write=12288 (1 ssts)
compact_read=8192  ratio=6.00x |

```

`user` is the logical bytes the plugin wrote through tidesdb's API, `wal` is the per-CF WAL bytes for that family (zero under unified memtable mode, which uses a single shared WAL accounted for separately in `SHOW ENGINE TIDESDB STATUS`), `flush` and `compact_write` are bytes emitted to SSTables by flush jobs and compaction jobs respectively, along with how many SSTables each produced, `compact_read` is bytes pulled in as compaction input, and `ratio` is $(wal + flush + compact_write) / user$ rounded to two decimals. A high ratio with a low SSTable count tends to point at oversized flush memtables, a high ratio with many compactions points at unnecessary L0 churn or an under-sized `L1_FILE_COUNT_TRIGGER`.

This information is useful for understanding the physical layout of your data and diagnosing performance characteristics.

SHOW ENGINE TIDESDB STATUS

The engine supports `SHOW ENGINE TIDESDB STATUS`, which displays database-level statistics and block cache metrics:

```
SHOW ENGINE TIDESDB STATUS\G
```

The output includes the data directory path, number of column families, global sequence number, memory usage (total system memory, resolved memory limit, memory pressure level, memtable bytes, transaction memory bytes), storage metrics (total SSTables, open SSTable handles, total data size, immutable memtable count), background queue sizes (flush pending, flush queue, compaction queue), block cache statistics (enabled, entries, size, hits, misses, hit rate, partitions), and a **Tombstones** block (total tombstones, overall tombstone ratio, and the worst per-SSTable tombstone density together with the level it occurs at).

When `tidesdb_unified_memtable=ON` an additional **Unified Memtable** block reports the active skip-list bytes, the number of immutables queued, whether a flush is currently in flight, the current WAL generation, and the round-robin next-CF index that the library uses when it scans for CF state to fold into the shared memtable.

A **Write Amplification** block reports cumulative user bytes written through the plugin alongside unified-WAL bytes, per-CF WAL bytes, flush bytes written, compaction bytes written and read, and a derived total WA ratio computed as $(unified_wal + per_cf_wal + flush + compaction_write) / user_bytes_written$. The flush and compaction rows also include their SSTable counts, so the ratio between bytes and SSTable count gives a coarse view of average output-SSTable size at each stage.

When an object-store backend is configured an **Object Store** block is included, and the last successfully uploaded unified-WAL generation is reported alongside the local cache footprint, upload queue depth, and lifetime upload counters.

When `tidesdb_print_all_conflicts` is enabled, the last conflict event is also displayed under a Conflicts section.

Status Variables

The engine also exposes machine-readable status variables for monitoring tools (Prometheus mysqld_exporter, PMM, Datadog, etc.):

```
SHOW GLOBAL STATUS LIKE 'tidesdb%';
```

Variable	Description
<code>Tidesdb_version</code>	TideSQL plugin version string (e.g. <code>4.5.8</code>)
<code>Tidesdb_version_hex</code>	TideSQL plugin version as integer (e.g. <code>263432</code> = <code>0x40508</code>)
<code>Tidesdb_library_version</code>	Underlying TidesDB storage library (<code>libtidesdb</code>) version string
<code>Tidesdb_column_families</code>	Number of active column families
<code>Tidesdb_global_sequence</code>	Global MVCC sequence number
<code>Tidesdb_memtable_bytes</code>	Total memtable memory usage in bytes
<code>Tidesdb_txn_memory_bytes</code>	Transaction buffer memory in bytes
<code>Tidesdb_memory_limit</code>	Resolved memory limit in bytes
<code>Tidesdb_memory_pressure</code>	Memory pressure level (0 = normal)
<code>Tidesdb_total_sstables</code>	Total SSTable count across all column families
<code>Tidesdb_open_sstables</code>	Open SSTable file handles
<code>Tidesdb_data_size_bytes</code>	Total data size on disk in bytes
<code>Tidesdb_immutable_memtables</code>	Immutable memtables pending flush
<code>Tidesdb_flush_pending</code>	Flush operations pending
<code>Tidesdb_flush_queue</code>	Flush queue depth
<code>Tidesdb_compaction_queue</code>	Compaction queue depth
<code>Tidesdb_cache_entries</code>	Block cache entry count
<code>Tidesdb_cache_bytes</code>	Block cache memory usage in bytes

Variable	Description
<code>Tidesdb_cache_hits</code>	Block cache hit count
<code>Tidesdb_cache_misses</code>	Block cache miss count
<code>Tidesdb_cache_hit_rate</code>	Block cache hit rate (percentage)
<code>Tidesdb_cache_partitions</code>	Block cache partition count
<code>Tidesdb_total_tombstones</code>	Total tombstones across all SSTables, summed by the post-flush density check
<code>Tidesdb_tombstone_ratio</code>	Aggregate tombstone density across all SSTables (0.0 to 1.0)
<code>Tidesdb_max_sst_tombstone_density</code>	Maximum tombstone density observed on any single SSTable
<code>Tidesdb_max_sst_tombstone_density_level</code>	LSM level of the SSTable with the maximum density
<code>Tidesdb_backpressure_waits</code>	Number of times a writer blocked on TidesDB back-pressure inside the plugin
<code>Tidesdb_backpressure_wait_us</code>	Total microseconds spent blocked on back-pressure
<code>Tidesdb_unified_memtable_enabled</code>	1 when the unified memtable is active, 0 when each column family carries its own memtable and WAL
<code>Tidesdb_unified_memtable_bytes</code>	Live byte usage of the unified memtable's active skip list
<code>Tidesdb_unified_immutable_count</code>	Number of unified-memtable immutables queued for flush
<code>Tidesdb_unified_is_flushing</code>	1 while a unified-memtable flush is in flight, 0 otherwise
<code>Tidesdb_unified_wal_generation</code>	Current unified-WAL generation number; advances each time the active WAL segment rolls over
<code>Tidesdb_object_store_enabled</code>	1 when an object-store backend is configured (S3 or remote), 0 for purely local storage

Variable	Description
<code>Tidesdb_replica_mode_active</code>	1 when this instance is currently a read-only object-store replica. Reflects the runtime role, so it flips back to 1 if a primary is fenced (self-demoted) by a newer one
<code>Tidesdb_primary_epoch</code>	Lease epoch this node holds as primary under single-writer fencing. Advances each time a node is promoted; a controller confirms a promotion took by watching this rise
<code>Tidesdb_seen_epoch</code>	Highest primary lease epoch this node has observed in the object store. A node whose own epoch is below <code>Tidesdb_seen_epoch</code> has been superseded and fences its writes
<code>Tidesdb_local_cache_bytes</code>	Bytes currently held in the local object-store cache directory
<code>Tidesdb_local_cache_files</code>	Number of files in the local object-store cache
<code>Tidesdb_upload_queue_depth</code>	Pending uploads waiting on the object-store uploader threads
<code>Tidesdb_total_uploads</code>	Lifetime count of successful uploads to the object store
<code>Tidesdb_upload_failures</code>	Lifetime count of upload attempts that failed and were retried
<code>Tidesdb_last_upload_generation</code>	Highest unified-WAL generation acknowledged by the object store (lags <code>Tidesdb_unified_wal_generation</code> by the upload queue)
<code>Tidesdb_uwal_bytes_written</code>	Bytes written by the plugin to the unified WAL over the process lifetime
<code>Tidesdb_wal_bytes_written</code>	Bytes written to per-CF WALs (sum across column families when unified memtable is OFF)
<code>Tidesdb_flush_bytes_written</code>	Bytes emitted to L0 SSTables by flush jobs across all column families
<code>Tidesdb_compaction_bytes_written</code>	Bytes written by compaction jobs across all column families
<code>Tidesdb_compaction_bytes_read</code>	Bytes read by compaction jobs as input across all column families
<code>Tidesdb_user_bytes_written</code>	Logical user bytes ingested via the plugin's write path (key + value, before any WAL/SSTable framing)

Variable	Description
<code>Tidesdb_flush_count</code>	Number of flush operations completed across all column families
<code>Tidesdb_compaction_count</code>	Number of compaction operations completed across all column families
<code>Tidesdb_lock_waits</code>	Number of times any thread blocked in the pessimistic lock-wait loop
<code>Tidesdb_lock_wait_us</code>	Total microseconds spent in the pessimistic lock-wait loop
<code>Tidesdb_lock_deadlocks</code>	Number of wait-for-graph cycle detections that returned <code>ER_LOCK_DEADLOCK</code>
<code>Tidesdb_lock_timeout</code>	Number of times <code>tidesdb_lock_wait_timeout_ms</code> expired during an acquire
<code>Tidesdb_lock_held</code>	Currently held lock count across all live transactions
<code>Tidesdb_lock_entries</code>	Lifetime count of allocated entries in the lock hash table (slots are retained across reuse)
<code>Tidesdb_lock_entry_recycles</code>	Number of times a freelisted lock entry was reused for a new key
<code>Tidesdb_lock_chain_max</code>	Longest hash-chain length observed at acquire time. Stays low under healthy contention; a creeping value points at a partition imbalance or chain regression

Values are refreshed about once a second by a dedicated background thread, so a busy engine flushing and compacting never reads as idle even under a pure-write workload that never plans a query. This provides the same monitoring integration that InnoDB offers via `SHOW GLOBAL STATUS LIKE 'innodb%'`.

Online Backup

TidesDB supports online backups triggered through a system variable. Setting `tidesdb_backup_dir` to a directory path initiates a consistent backup of the entire TidesDB data directory:

```
SET GLOBAL tidesdb_backup_dir = '/path/to/backup';
```

The backup runs without blocking normal read and write operations. The target directory must not already exist or must be empty. After the backup completes, the variable reflects the path of the last successful backup. To clear it:

```
SET GLOBAL tidesdb_backup_dir = '';
```

Checkpoint

TidesDB also supports lightweight checkpoints that use hard links instead of copying SSTable data. Checkpoints are near-instant and consume no additional disk space until compaction removes old SSTables from the live database.

```
SET GLOBAL tidesdb_checkpoint_dir = '/path/to/checkpoint';
```

The checkpoint directory must not already exist or must be empty. Unlike a full backup, checkpoints require the target to be on the same filesystem as the data directory (hard link requirement). The checkpoint can be opened as a normal TidesDB database.

For archival or cross-filesystem copies, use `tidesdb_backup_dir` instead.

Partitioning

TidesDB tables can be partitioned using MariaDB's standard partitioning syntax. Each partition becomes a separate TidesDB table (and therefore a separate column family), which means compaction and flushes happen independently per partition.

```
CREATE TABLE metrics (  
  id      INT NOT NULL,  
  ts      DATE NOT NULL,  
  value   DOUBLE,  
  PRIMARY KEY (id, ts)  
) ENGINE=TIDESDB  
PARTITION BY RANGE COLUMNS(ts) (  
  PARTITION p_2024  VALUES LESS THAN ('2025-01-01'),  
  PARTITION p_2025  VALUES LESS THAN ('2026-01-01'),  
  PARTITION p_future VALUES LESS THAN MAXVALUE  
);
```

All partitioning schemes supported by MariaDB work with TidesDB, `HASH`, `KEY`, `RANGE`, `LIST`, and `RANGE COLUMNS`. Secondary indexes on partitioned tables also work correctly, with each partition maintaining its own index column family.

Partitions can be added and dropped with `ALTER TABLE`:

```
ALTER TABLE metrics ADD PARTITION (PARTITION p_2026 VALUES LESS THAN
('2027-01-01'));
ALTER TABLE metrics DROP PARTITION p_2024;  -- removes all data in that
range
```

System Variables

The engine exposes several system variables that control TidesDB's runtime behavior:

Global Variables (read-only, set at startup)

Variable	Default	Description
<code>tidesdb_flush_threads</code>	4	Number of background threads flushing memtables to SSTables
<code>tidesdb_max_concurrent_flushes</code>	0	Global cap on in-flight memtable flushes. 0 (default) aligns the cap with <code>tidesdb_flush_threads</code> so every configured worker can run. Setting a value below <code>tidesdb_flush_threads</code> is honoured but leaves the surplus workers idle and logs a startup warning
<code>tidesdb_compaction_threads</code>	4	Number of background threads performing LSM compaction
<code>tidesdb_log_level</code>	DEBUG	TidesDB internal log level (DEBUG, INFO, WARN, ERROR, FATAL, NONE)
<code>tidesdb_block_cache_size</code>	256 MB	Size of the global block cache shared across all column families
<code>tidesdb_max_open_sstables</code>	256	Maximum number of SSTable file handles cached in the LRU
<code>tidesdb_max_memory_usage</code>	0 (auto)	Global memory limit in bytes; 0 lets the library auto-detect (50% of system RAM; minimum 5%)
<code>tidesdb_data_home_dir</code>	(empty)	Override the TidesDB data directory; defaults to <code><mysql_datadir>/../tidesdb_data</code>
<code>tidesdb_log_to_file</code>	ON	Write TidesDB logs to a LOG file in the data directory instead of stderr
<code>tidesdb_log_truncation_at</code>	24 MB	Log file truncation size in bytes; 0 disables truncation
<code>tidesdb_unified_memtable</code>	ON	Use a single shared WAL and memtable across all column families. Reduces WAL I/O from $O(\text{num_tables})$ to $O(1)$ per commit. Requires all CFs to use the same comparator
<code>tidesdb_unified_memtable_buffer_size</code>	256 MB	Write buffer size for the unified memtable (0 = library default). Only meaningful when <code>tidesdb_unified_memtable=ON</code>

Variable	Default	Description
<code>able_write_buffer_size</code>		
<code>tidesdb_unified_memtable_sync_mode</code>	FULL	Sync mode for the unified WAL (NONE, INTERVAL, FULL). Controls durability of all commits when unified memtable is enabled. Per-CF <code>SYNC_MODE</code> is ignored in unified mode since per-CF WALs do not exist
<code>tidesdb_unified_memtable_sync_interval</code>	128000	Sync interval in microseconds for the unified WAL (only used when <code>unified_memtable_sync_mode=INTERVAL</code>)
<code>tidesdb_object_store_backend</code>	LOCAL	Object store backend. <code>LOCAL</code> disables object store mode, <code>S3</code> uses S3-compatible storage, <code>FS</code> uses a shared local filesystem directory as the bucket stand-in (cross-process single-writer safety via <code>flock</code>), which lets failover and replication be exercised without S3
<code>tidesdb_object_store_fs_path</code>	(empty)	Shared directory used as the bucket when <code>tidesdb_object_store_backend=FS</code> . Every node points at the same path; required for the FS backend and ignored otherwise
<code>tidesdb_s3_endpoint</code>	(empty)	S3 endpoint hostname (e.g. s3.amazonaws.com or minio.local:9000)
<code>tidesdb_s3_bucket</code>	(empty)	S3 bucket name
<code>tidesdb_s3_prefix</code>	(empty)	S3 key prefix for multi-tenant buckets (e.g. production/db1/)
<code>tidesdb_s3_access_key_id</code>	(empty)	S3 access key ID
<code>tidesdb_s3_secret_key</code>	(empty)	S3 secret access key
<code>tidesdb_s3_region</code>	(empty)	S3 region (e.g. us-east-1, empty for MinIO)
<code>tidesdb_s3_use_ssl</code>	ON	Use HTTPS for S3 connections

Variable	Default	Description
<code>tidesdb_s3_path_style</code>	OFF	Use path-style S3 URLs (required for MinIO)
<code>tidesdb_s3_tls_ca_path</code>	(empty)	Custom CA bundle path for the S3 TLS handshake; empty uses the system bundle
<code>tidesdb_s3_tls_insecure_skip_verify</code>	OFF	Disable S3 TLS peer/host verification. INSECURE; intended for trusted test endpoints only. Logs a warning at start when ON
<code>tidesdb_s3_multipart_threshold</code>	0	Object size in bytes that triggers S3 multipart upload; 0 keeps the library default
<code>tidesdb_s3_multipart_part_size</code>	0	S3 multipart chunk size in bytes; 0 keeps the library default
<code>tidesdb_objstore_local_cache_max</code>	0	Maximum local cache size in bytes (0 = unlimited)
<code>tidesdb_objstore_wal_sync_threshold</code>	1 MB	Sync active WAL to object store when it grows by this many bytes (0 = disable)
<code>tidesdb_objstore_wal_sync_on_commit</code>	OFF	Upload WAL after every commit for RPO=0 replication
<code>tidesdb_objstore_cache_on_read</code>	ON	Cache downloaded objects in the local cache
<code>tidesdb_objstore_cache_on_write</code>	ON	Cache uploaded objects in the local cache

Variable	Default	Description
<code>tidesdb_objstore_max_concurrent_uploads</code>	0	Concurrent upload threads; 0 keeps the library default
<code>tidesdb_objstore_max_concurrent_downloads</code>	0	Concurrent download threads; 0 keeps the library default
<code>tidesdb_objstore_multipart_threshold</code>	0	Object size in bytes that triggers multipart upload at the objstore layer; 0 keeps the library default
<code>tidesdb_objstore_multipart_part_size</code>	0	Multipart chunk size in bytes at the objstore layer; 0 keeps the library default
<code>tidesdb_objstore_sync_manifest_to_object</code>	ON	Upload MANIFEST after each compaction
<code>tidesdb_objstore_wal_upload_sync</code>	OFF	Block memtable flush on WAL upload; OFF keeps WAL upload in the background
<code>tidesdb_objstore_replicate_wal</code>	ON	Upload WAL segments for replica recovery
<code>tidesdb_objstore_replica_replay_wal</code>	ON	Replay WAL on replicas for near-real-time visibility
<code>tidesdb_unified_memtable_skip_list_max_level</code>	0	Skip-list max level for the unified memtable; 0 keeps the library default

Variable	Default	Description
<code>tidesdb_unified_memtable_skip_list_probability</code>	0.0	Skip-list level promotion probability for the unified memtable; 0.0 keeps the library default
<code>tidesdb_replica_mode</code>	OFF	Enable read-only replica mode (writes return SQL error)
<code>tidesdb_replica_sync_interval</code>	5000000	MANIFEST poll interval for replica sync in microseconds
<code>tidesdb_finish_compactions_on_close</code>	OFF	Maps to <code>tidesdb_config_t.finish_compactions_on_close</code> , applied at open. Default OFF (the library default) makes <code>tidesdb_close</code> cancel in-flight compactions at their next checkpoint for a fast shutdown; the cancelled merge discards its uncommitted output and leaves inputs intact, so no data is lost and recovery handles a mid-merge state the same way. Set ON to let in-flight compactions run to completion before close returns, which is the safer setting for object-store and replica deployments where a mid-compaction cancel can leave S3 with referenced-but-not-yet-uploaded SSTables that confuse a syncing replica

Global Variables (dynamic)

Variable	Default	Description
<code>tidesdb_backup_dir</code>	(empty)	Set to a path to trigger an online backup; clear with empty string
<code>tidesdb_checkpoint_dir</code>	(empty)	Set to a path to trigger a hard-link checkpoint (near-instant, same filesystem only); clear with empty string
<code>tidesdb_print_all_locks</code>	OFF	Log every <code>TDB_ERR_CONFLICT</code> event to the error log (similar to <code>innodb_print_all_deadlocks</code>). Last conflict info also shown in <code>SHOW ENGINE TIDESDB STATUS</code>
<code>tidesdb_pessimistic_locking</code>	ON	Plugin-level row lock manager with shared (S) and exclusive (X) modes. ON (default) acquires X on every write-intent statement and S on plain reads under <code>REPEATABLE-READ</code> or <code>SERIALIZABLE</code> ; reads under <code>READ-COMMITTED</code> and <code>SNAPSHOT</code> take no lock. Multiple S holders coexist; new S requests block while an X is queued so writers cannot be starved. Wait-for-graph deadlock detection up to 100 hops, bounded by <code>tidesdb_lock_wait_timeout_ms</code> . Locks can be acquired on non-existing PK values and on unique secondary key prefixes during <code>INSERT</code> . Locks are held until <code>COMMIT</code> or <code>ROLLBACK</code> . OFF reverts to pure optimistic MVCC where write-write conflicts surface at commit time and the application must retry. See Pessimistic Row Locking
<code>tidesdb_promote_primary</code>	OFF	Set to ON to promote a replica to primary. Claims a new primary lease epoch in the object store, performs a final MANIFEST sync and WAL replay, creates a local WAL, and starts accepting writes. The promotion can legitimately fail under single-writer fencing if another node already holds a newer lease, in which case <code>SET GLOBAL</code> returns an error and the node stays a replica, so a failover controller must check the return rather than assume success. Resets to OFF after the attempt

Session Variables

Variable	Default	Description
<code>tidesdb_ttl</code>	0	Per-session TTL in seconds applied to INSERT/UPDATE; 0 means use the table-level default. Can be set with <code>SET SESSION</code> or <code>SET STATEMENT</code>
<code>tidesdb_skip_unique_check</code>	OFF	Skip uniqueness checks on primary key and unique secondary indexes during INSERT. Only safe when the application guarantees no duplicates (e.g., bulk loads with monotonic PKs)
<code>tidesdb_lock_wait_timeout_ms</code>	50000	Milliseconds a pessimistic row-lock acquire waits before returning <code>HA_ERR_LOCK_WAIT_TIMEOUT</code> (range 0..3600000). Mirrors <code>innodb_lock_wait_timeout</code> . 0 disables the bound and the wait is broken only by <code>KILL QUERY</code> . See Lock Wait Timeout
<code>tidesdb_backpressure_wait_timeout_ms</code>	60000	Milliseconds the plugin blocks a writer on TidesDB back-pressure (memtable, flush queue, or L0 backlog at soft cap) before surfacing <code>HA_ERR_LOCK_WAIT_TIMEOUT</code> (range 0..3600000). 0 disables blocking and back-pressure returns immediately. See Write Back-Pressure
<code>tidesdb_single_delete_primary</code>	OFF	Use single-delete semantics on the primary row CF for this session's DELETES. See Single-Delete Optimization
<code>tidesdb_compact_after_range_delete_min_rows</code>	0	If non-zero, after a multi-row DELETE that touches at least this many rows, the engine calls <code>tidesdb_compact_range</code> over the touched primary-key range to physically reclaim tombstoned space synchronously. See Auto Compact After Range Delete
<code>tidesdb_default_compression</code>	LZ4	Default compression algorithm (NONE, SNAPPY, LZ4, ZSTD, LZ4_FAST)
<code>tidesdb_default_write_buffer_size</code>	64 MB	Default write buffer size in bytes (matches <code>TDB_DEFAULT_WRITE_BUFFER_SIZE</code>)
<code>tidesdb_default_bloom_filter</code>	ON	Default bloom filter setting

Variable	Default	Description
<code>tidesdb_default_use_btree</code>	OFF	Default USE_BTREE setting (0=LSM, 1=B-tree)
<code>tidesdb_default_block_indexes</code>	ON	Default block indexes setting
<code>tidesdb_default_sync_mode</code>	FULL	Default sync mode (NONE, INTERVAL, FULL)
<code>tidesdb_default_sync_interval_us</code>	128000	Default sync interval in microseconds (for INTERVAL mode)
<code>tidesdb_default_bloom_fpr</code>	100	Default bloom FPR in parts per 10,000 (100 = 1%)
<code>tidesdb_default_klog_value_threshold</code>	512	Default klog value threshold in bytes (values >= this go to vlog)
<code>tidesdb_default_l0_queue_stall_threshold</code>	10	Default L0 queue stall threshold
<code>tidesdb_default_l1_file_count_trigger</code>	4	Default L1 file count compaction trigger
<code>tidesdb_default_level_size_ratio</code>	10	Default level size ratio
<code>tidesdb_default_min_levels</code>	1	Default minimum LSM-tree levels (matches <code>TDB_DEFAULT_MIN_LEVELS</code>)
<code>tidesdb_default_dividing_level_offset</code>	1	Default dividing level offset (matches <code>TDB_DEFAULT_DIVIDING_LEVEL_OFFSET</code>)

Variable	Default	Description
<code>tidesdb_default_skip_list_max_level</code>	12	Default skip list max level
<code>tidesdb_default_skip_list_probability</code>	25	Default skip list probability (percentage; 25 = 0.25)
<code>tidesdb_default_index_sample_ratio</code>	1	Default block index sample ratio
<code>tidesdb_default_block_index_prefix_length</code>	16	Default block index prefix length
<code>tidesdb_default_min_disk_space</code>	100 MB	Default minimum disk space in bytes
<code>tidesdb_default_isolation_level</code>	REPEATABLE_READ	Default isolation level
<code>tidesdb_default_object_lazy_compaction</code>	OFF	Double L1 compaction trigger in object store mode
<code>tidesdb_default_object_prefetch_compaction</code>	ON	Prefetch input SSTables before compaction merge
<code>tidesdb_default_tombstone_density_trigger</code>	0	Default tombstone-density compaction trigger ratio for new tables, in parts per 10000 (5000 = 0.50). Default 0 disables the check
<code>tidesdb_default_tombstone_density_min_entries</code>	1024	Minimum entry count for an SSTable to be considered by the tombstone-density trigger; smaller SSTables are ignored

Every table option has a corresponding `tidesdb_default_*` session variable. When `CREATE TABLE` does not explicitly set an option, the session (or global) default is used. Table-level options in `CREATE TABLE` override the default. This design mirrors InnoDB's approach of having global defaults that individual tables can override, and makes it straightforward to configure all tables uniformly for benchmarking or deployment without repeating options in every DDL statement:

```
# my.cnf -- set defaults for all new TidesDB tables
[mysqld]
plugin-load-add=ha_tidesdb.so
tidesdb_default_sync_mode=NONE
tidesdb_default_compression=NONE
tidesdb_default_bloom_fpr=10
tidesdb_default_klog_value_threshold=1024
tidesdb_default_l0_queue_stall_threshold=10
```

```
-- All new tables inherit the global defaults
CREATE TABLE t1 (id INT PRIMARY KEY) ENGINE=TIDESDB;

-- Override a specific option for one table
CREATE TABLE t2 (id INT PRIMARY KEY) ENGINE=TIDESDB COMPRESSION='ZSTD';

-- Change defaults for the current session only
SET SESSION tidesdb_default_sync_mode = 'FULL';
CREATE TABLE t3 (id INT PRIMARY KEY) ENGINE=TIDESDB; -- uses FULL
```

The block cache is a read cache backed by two independent clock caches, one for raw klog block bytes (used by the default block-based SSTable format) and one for deserialized B+tree nodes (used by column families with `USE_BTREE=1`). Both caches share the configured `tidesdb_block_cache_size` budget. A larger cache reduces read amplification for workloads that repeatedly access the same key ranges. The flush and compaction thread counts should be tuned based on the number of column families in use - only one flush and one compaction can run per column family at a time, so with N column families, up to N threads can be busy simultaneously. The default of 4 threads handles workloads with up to 4 tables (8 column families, data + one secondary index each). When `tidesdb_log_to_file` is enabled (the default), TidesDB writes to a `LOG` file in the data directory with automatic truncation controlled by `tidesdb_log_truncation_at` (default 24 MB, 0 to disable truncation). When disabled, logs are written to stderr.

The `tidesdb_max_memory_usage` variable controls the global memory cap enforced by the library. When set to 0, the library auto-detects available system memory and targets 50% of it (with a minimum floor of 5% of total system RAM). In a shared MariaDB server where InnoDB and other components also consume memory, you may want to set an explicit limit. When `tidesdb_unified_memtable=ON` (the default), all column families share a single memtable with its own write buffer (`tidesdb_unified_memtable_write_buffer_size`, default 256 MB), so per-CF `WRITE_BUFFER_SIZE` does not affect memtable memory. The per-CF `L0_QUEUE_STALL_THRESHOLD` (default 10) still applies for backpressure and bounds the number of immutable memtables that can queue before new writes block. When unified memtable is disabled, each CF has its own memtable and worst-case memory per column family is approximately $(\text{WRITE_BUFFER_SIZE} \times 1.5) + (\text{WRITE_BUFFER_SIZE} \times \text{L0_QUEUE_STALL_THRESHOLD})$.

How It Stores Data Internally

Understanding the physical layout helps when interpreting `ANALYZE TABLE` output or debugging performance.

Each table's data lives in a column family, which is an independent LSM-tree. Writes go to a skip-list memtable. When the memtable reaches the configured write buffer size, it becomes immutable and is flushed to disk as a sorted SSTable in Level 1. Compaction then merges overlapping SSTables from lower levels into higher levels, maintaining the sorted invariant.

Row keys inside the column family use a namespace prefix byte. Data rows use `0x01` and metadata entries use `0x00`. The metadata namespace is what holds FTS aggregate counters (total document count and total word count for BM25 scoring) so the data scan that seeks to `0x01` naturally skips them.

Primary key bytes are encoded in a memcmp-comparable format. For a signed 32-bit integer, the encoding flips the sign bit and stores the result in big-endian byte order. This means that the integer -1 sorts before 0, and 0 sorts before 1, all under a simple byte comparison. The same principle extends to all numeric types and string collations.

Row values are stored in a packed binary format. Each row begins with a 5-byte header, a magic byte (`0xFE`), followed by the null bitmap size (2 bytes LE) and field count (2 bytes LE) at the time the row was written. This header enables instant `ADD COLUMN` and `DROP COLUMN` by letting the deserializer adapt to rows written with any prior schema. After the header, the null bitmap is written, then each non-null field is serialized using `Field::pack()`. On read, `Field::unpack()` restores the fields into MariaDB's record buffer. If a row was written with fewer fields than the current schema (column was added), the missing fields receive their `DEFAULT` value. If a row was written with more fields (column was dropped), the extra data is skipped. This format is more compact than storing the raw `reclength` bytes, particularly for tables with `VARCHAR` or `CHAR` columns.

Secondary index entries are stored in their own column family. The key format is the concatenation of the comparable index-column bytes and the comparable primary key bytes. The value is a single zero byte (effectively empty); all the information lives in the key. To resolve a secondary index lookup, the engine seeks into the index CF, reads the key, splits off the trailing PK bytes, and performs a point-get into the data CF. When the server indicates that only indexed columns are needed (a covering index scan), the engine can decode primary key and index column values directly from the comparable-format index key bytes - integers, temporal types (DATE, DATETIME, TIMESTAMP, YEAR), and fixed-length CHAR/BINARY (binary/latin1) - skipping the data CF point-get entirely.

For tables without an explicit primary key, the engine generates a hidden 8-byte big-endian row ID, assigned from an atomic counter. This counter is recovered on table open by seeking to the last key in the column family.

Statistics and the Optimizer

The engine maintains cached statistics that are refreshed at most every two seconds. These statistics include the total number of keys, total data size, average key and value sizes, and the LSM-tree's read amplification factor. They are stored as atomic variables on the shared table descriptor and feed into MariaDB's cost-based optimizer through `info()`, `scan_time()`, `keyread_time()`, `rnd_pos_time()`, and `records_in_range()`.

The cost model accounts for the fact that LSM-tree reads may need to consult multiple levels. The read amplification factor - obtained from `tidesdb_get_stats()` - scales the cost of point lookups and random-position reads. A higher read amplification nudges the optimizer toward sequential scans; when the data is well-compacted and the amplification is low, index lookups are cheap.

Cost Methods

- `scan_time()` uses `tidesdb_range_cost()` over the full data column family key space, giving the optimizer an LSM-aware full-table-scan cost that accounts for the actual number of levels, SSTables, compression overhead, and merge complexity. The cost is split 90% I/O / 10% CPU. Falls back to the base `handler::scan_time()` when the library cannot produce a meaningful estimate.
- `keyread_time()` models index reads as $\text{rows} \times 0.00003 \times \text{read_amp} + \text{ranges} \times 0.0001$. Each point lookup touches `read_amp` levels; range scans amortize the merge-heap setup cost across rows.
- `rnd_pos_time()` models random-position lookups as $\text{rows} \times 0.00005 \times \text{read_amp}$, reflecting that each random fetch is a point-get through the full LSM stack.

Range-Aware Cardinality Estimation

`records_in_range()` uses a two-path strategy:

1. Equality detection · When both key bounds convert to identical comparable bytes (a point equality like `WHERE k = 5`), the engine returns the `rec_per_key` estimate directly. This avoids the `tidesdb_range_cost()` path, which is an I/O cost metric rather than a cardinality metric - for memtable-only data it cannot distinguish a point range from a full scan, so the proportional estimate would be meaningless.
2. Range estimation · For range predicates the engine calls `tidesdb_range_cost()` for the requested key range and for the full key space, then returns $\text{total_records} \times (\text{range_cost} / \text{full_cost})$. The function examines in-memory metadata - block indexes, SSTable min/max keys, and entry counts - without any disk I/O. A narrow range returns a small estimate while a wide range returns a proportionally larger one, allowing the optimizer to make informed decisions about index selection and join ordering.

OPTIMIZE TABLE

Running `OPTIMIZE TABLE` triggers a synchronous flush and compaction on all column families associated with the table, both the main data CF and every secondary index CF. The engine calls `tidesdb_purge_cf()` for each column family, which rotates the unified memtable (when enabled), flushes the resulting immutable memtable to SSTables, and then runs a full compaction inline, blocking until complete. This means the table is fully compacted when the statement returns. In object store mode, the flushed SSTables are uploaded to S3, making `OPTIMIZE TABLE` the most direct way to push data to the object store. After compaction, the cached statistics are invalidated so the optimizer sees the post-compaction state sooner.

```
OPTIMIZE TABLE products;
```

This is useful after bulk deletes or updates that leave behind a large number of tombstones, or when `ANALYZE TABLE` reports high read amplification.

CHECK TABLE / REPAIR TABLE

`CHECK TABLE` verifies that all column families associated with the table are readable by fetching metadata from all SSTables. This validates manifests, block indexes, bloom filters, and metadata blocks are intact. If any SSTable is unreadable, the command returns an error indicating the table is corrupt.

```
CHECK TABLE orders;
```

`REPAIR TABLE` triggers a full purge (flush + compaction) of all column families, identical to `OPTIMIZE TABLE`. The compaction pass reads and re-checksums every block, dropping corrupt entries, expired TTL data, and tombstones. In unified memtable mode, the first purge call rotates the shared memtable once, and subsequent index CF purges just run compaction.

```
REPAIR TABLE orders;
```

FLUSH TABLES FOR EXPORT

TidesDB supports `FLUSH TABLES ... FOR EXPORT` for online physical table copies. The statement flushes all pending memtable data to SSTables and holds a table lock while the data directory files are consistent and copyable. After copying, release the lock with `UNLOCK TABLES`.

```
FLUSH TABLES orders FOR EXPORT;
-- copy the column family directories from tidesdb_data/
UNLOCK TABLES;
```

Rename and Drop

Renaming a table renames all associated column families, including secondary index CFs. The engine enumerates all column families whose names start with the old table's prefix and renames each one.

Dropping a table drops the main data CF and then enumerates and drops all index CFs that share the table's naming prefix. The operation is idempotent, if a CF does not exist, the engine simply continues.

`DROP DATABASE` is wired through the engine's handler on `drop_database` callback. MariaDB invokes it after removing the `.frm` files for the database; the engine then enumerates every column family whose name starts with `<db_name>_` and drops each one (data CFs plus their `__idx_*` secondary-index CFs), force-removes the on-disk directories, and purges schema-CF entries for the database (object-store mode only). This prevents orphaned column families accumulating on disk when a database is dropped.

```
RENAME TABLE events TO event_log;
TRUNCATE TABLE event_log;
DROP TABLE event_log;
DROP DATABASE mydb;  -- drops every TidesDB CF under mydb__*
```

Server Lifecycle Hooks

The engine wires several handlerton callbacks so that TidesDB cooperates with MariaDB’s lifecycle and durability guarantees:

Callback	Purpose
<code>flush_logs</code>	<code>FLUSH LOGS</code> (and <code>mariadb-backup</code> ’s pre-copy step) syncs the TidesDB WAL so on-disk copies are a consistent snapshot.
<code>panic</code>	On signal-driven shutdown paths MariaDB may call <code>panic(HA_PANIC_CLOSE)</code> instead of the normal deinit; the engine performs an orderly <code>tidesdb_close()</code> there so pending commits are flushed.
<code>pre_shutdown</code>	Lets background threads quiesce before the deinit path begins; syncs the unified WAL so compactions in flight don’t get killed mid-write.
<code>kill_query</code>	<code>KILL QUERY <id></code> wakes any waiter blocked in <code>row_lock_acquire</code> and, in combination with <code>thd_killed()</code> checks scattered through the scan loops (<code>rnd_next</code> , <code>index_next</code> , <code>index_prev</code> , <code>index_next_same</code> , <code>spatial_scan_next</code> , <code>ft_read</code>), promptly terminates long-running statements with <code>HA_ERR_ABORTED_BY_USER</code> .

Object Store Mode

TidesDB supports an optional object store backend that stores SSTables in S3-compatible storage (AWS S3, MinIO, Google Cloud Storage) while using local disk as a cache. This enables cloud-native deployments where storage capacity is virtually unlimited and compute nodes can be replaced without data loss.

Why Use Object Store Mode

Object store mode is designed for deployments where local disk is either limited, ephemeral, or both. In container environments, Kubernetes pods, and serverless setups, local storage is temporary. Object store mode turns local disk into a bounded cache while S3 holds the durable copy of all data. A node with 1TB of NVMe and an S3 bucket can serve a 100TB dataset, with the hot working set at local disk speed and cold data fetched on demand.

The storage engine maintains a four-tier hierarchy automatically. Hot data lives in the in-memory block cache (microsecond reads). Warm data lives on local disk with open file handles (microsecond to millisecond reads). Cold data is on local disk but closed (reopened on access). Frozen data is in the object store only and fetched via HTTP range requests for point lookups or parallel bulk download for scans. Data flows downward through these tiers as it ages and flows back up when accessed.

When to Use It

- Cloud-native deployments with separated compute and storage
- Workloads that benefit from virtually unlimited storage capacity
- Environments where local disk is ephemeral (containers, serverless, spot instances)
- Multi-node setups where cold start recovery from remote storage replaces traditional replication

When Not to Use It

- Single-server deployments with abundant local storage and no cloud requirement
- Latency-critical workloads where every read must be served from local disk
- Air-gapped environments without network access to an object store

Enabling Object Store Mode

Add the following to `my.cnf`:

```
[mariadb]
tidesdb_object_store_backend = S3
tidesdb_s3_endpoint = s3.amazonaws.com
tidesdb_s3_bucket = my-tidesdb-bucket
tidesdb_s3_access_key = AKIAIOSFODNN7EXAMPLE
tidesdb_s3_secret_key = wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
tidesdb_s3_region = us-east-1
tidesdb_objstore_local_cache_max = 536870912
```

For MinIO:

```
tidesdb_object_store_backend = S3
tidesdb_s3_endpoint = minio.local:9000
tidesdb_s3_bucket = tidesdb-data
tidesdb_s3_use_ssl = OFF
tidesdb_s3_path_style = ON
tidesdb_s3_access_key = minioadmin
tidesdb_s3_secret_key = minioadmin
```

Object store mode works with unified memtable mode (enabled by default). Unified memtable is strongly recommended for object store deployments because it reduces WAL I/O to a single file and ensures all column families are flushed together. No other configuration changes are needed. Existing SQL, table options, indexes, transactions, and all engine features work identically.

Schema Discovery

When object store mode is active, the engine creates a reserved column family called `__tidesql_schema` that stores the binary `.frm` image for every TidesDB table. This column family replicates through S3 alongside data, so replicas automatically discover table definitions without any manual schema synchronization.

The schema CF is updated on every DDL operation:

- `CREATE TABLE` stores the `.frm` binary from the in-memory image (MariaDB does not write `.frm` files to disk for engines with discovery callbacks)
- `ALTER TABLE` updates the stored `.frm` after the inplace alter commits
- `DROP TABLE` removes the entry
- `RENAME TABLE` moves the entry to the new key

At startup, the plugin scans the schema CF for all unique database names and creates any missing database directories under the MariaDB data directory. This ensures `SHOW DATABASES` and `SHOW TABLES` work correctly on replicas without manual `CREATE DATABASE` commands.

The engine registers MariaDB's `discover_table`, `discover_table_names`, and `discover_table_existence` handlerton callbacks. When MariaDB opens a table that has no local `.frm` file, these callbacks read the `.frm` from the schema CF and materialize it. The `.frm` is then cached on disk for subsequent opens.

To prevent an infinite retry loop, `discover_table` verifies that the data column family exists locally before returning the `.frm`. If the table definition has been replicated but the data has not been synced yet, the engine returns “table not found” rather than entering a discover -> open-fail -> re-discover cycle.

In local-only mode (no object store), none of this machinery is active. The schema CF is never created, discovery callbacks are not registered, and there is zero overhead.

Read Replicas

A separate MariaDB instance can run as a read-only replica by pointing to the same S3 bucket with replica mode enabled:

```
tidesdb_replica_mode = ON
tidesdb_replica_sync_interval = 1000000
```

The replica periodically polls S3 for MANIFEST updates and discovers new column families that the primary has created. It also replays unified WAL segments for near-real-time reads. Writes are rejected with a clean SQL error. Multiple replicas can run simultaneously, each with its own local cache.

Table definitions are discovered automatically from the `__tidesql_schema` column family. When a client queries a table that the replica has not opened before, the engine reads the `.frm` from the schema CF, writes it to local disk, and opens the table. Database directories are created at startup, so `SHOW DATABASES` and `SHOW TABLES` reflect the primary's schema immediately.

`OPTIMIZE TABLE` on the primary triggers a unified memtable flush, which creates SSTables for all column families (including `__tidesql_schema`) and uploads them to S3. This ensures that both table data and schema definitions reach the object store promptly.

Primary Promotion

When the primary fails, a replica can be promoted to accept writes:

```
SET GLOBAL tidesdb_promote_primary = ON;
```

This claims a new primary lease epoch in the object store, waits for any in-progress replica sync to complete, performs a final MANIFEST sync and WAL replay to catch the last writes from the old primary, creates a local WAL for crash recovery, establishes the schema column family so its own `CREATE TABLE` statements publish `.frm` images that downstream replicas can discover, and starts accepting writes. The promotion is fast because the replica already has metadata in memory from the sync loop.

Promotion is not guaranteed to succeed. The lease epoch is claimed with a conditional write, so if another node has already promoted and holds a newer epoch, the claim loses the race and `SET GLOBAL tidesdb_promote_primary = ON` returns an error rather than silently becoming a second writer. The

node stays a replica. A failover controller must therefore check the statement result (and confirm `Tidesdb_primary_epoch` advanced) instead of assuming the promotion took.

After promotion, tables that were previously opened (and have their `.frm` cached on disk) are accessible immediately. Tables that have not been opened yet will be discovered from the schema CF on first access, provided their data column family has been synced. If a table's schema has been replicated but its data has not arrived yet, the query returns “table not found” rather than blocking.

Single-Writer Fencing

Object store mode enforces a single writer through a lease epoch rather than relying on the orchestrator to fence a failed primary. Each promotion claims the next epoch with a conditional write against the bucket. A primary only stays authoritative while it holds the current epoch, and every manifest publish carries that epoch as the precondition for its conditional write.

The consequence is that a stale primary cannot corrupt the shared store even if it is still alive and still thinks it is the writer. A primary that was network-partitioned from the bucket, or simply slow, finds on its next publish that the precondition fails because a newer primary has claimed a higher epoch. The library surfaces that as `TDB_ERR_PRECONDITION`, which the engine maps to `HA_ERR_READ_ONLY_TRANSACTION` and uses as the signal to self-demote: the node flips its runtime role back to replica, stops accepting writes, and its `Tidesdb_replica_mode_active` returns to 1. Writes it took locally while superseded never become authoritative because they can never win the conditional write. This is why `Tidesdb_replica_mode_active` tracks the runtime role and not just the startup setting, and why the safety of a failover does not depend on the controller killing the old primary in time.

The two epoch status variables make this observable. `Tidesdb_primary_epoch` is the epoch a node holds; `Tidesdb_seen_epoch` is the highest epoch it has observed in the store. A node whose `Tidesdb_primary_epoch` is below the `Tidesdb_seen_epoch` published by another node has been superseded.

WAL Sync Options

The data loss window on crash depends on WAL sync configuration:

Setting	RPO	Tradeoff
<code>objstore_wal_sync_on_commit = ON</code>	Zero	One HTTP round-trip per commit
<code>objstore_wal_sync_threshold = 1048576</code> (default)	~1MB of writes	Periodic sync based on write volume
<code>objstore_wal_sync_threshold = 0</code>	One full flush cycle	No periodic sync

On clean shutdown there is no data loss regardless of configuration.

Monitoring

`SHOW ENGINE TIDESDB STATUS` includes an Object Store section when enabled:

```
--- Object Store ---
Connector: s3
Total uploads: 1234
Upload failures: 0
Upload queue depth: 2
Last uploaded generation: 87
Local cache: 234567890 / 536870912 bytes (45 files)
Replica mode: OFF
```

The block reports the connector, lifetime upload counters, the pending upload queue depth, the highest unified-WAL generation the object store has acknowledged, and the local cache footprint against its cap. The node's primary/replica role and lease epoch are not printed here; they are exposed as the `Tidesdb_replica_mode_active`, `Tidesdb_primary_epoch`, and `Tidesdb_seen_epoch` status variables so a controller can poll them directly.

Kubernetes Deployment

The `k8s/production/` directory contains a production-oriented HA deployment, `tidesql-ha.yaml`, built around a single identical pool of pods rather than a separate primary and replica topology. There is no primary StatefulSet, replica Deployment, or external failover controller to keep in sync, the deployment is one `StatefulSet` of N identical pods that all boot read-only (`tidesdb_replica_mode=ON`), and exactly one of them is elected the writer at any moment.

Election runs inside each pod. A small agent sidecar competes for a Kubernetes `Lease` object (`tidesql-primary`) using optimistic concurrency, it reads the lease, then `kubectl replaces` it carrying the observed `resourceVersion`, so a stale write loses with a 409 and two pods can never both win. The pod that wins the lease promotes its local engine with `SET GLOBAL tidesdb_promote_primary = ON` and labels itself `role=primary`. Two Services route on that label, `tidesql-write` selects the `role=primary` pod and `tidesql-read` selects any pod. When the primary dies its lease expires, another pod wins and promotes, and the failed pod boots as a replica when Kubernetes reschedules it (it no longer holds the lease) and re-syncs from the object store. The topology self-heals without an operator in the loop.

What makes this safe is not the election but the engine's single-writer fence underneath it. Promotion claims a new lease epoch in the object store with a conditional write, and from that point a superseded primary's manifest publish fails its precondition and self-demotes, so its writes can never become

authoritative regardless of how the Kubernetes lease resolves or how quickly the old pod is killed (see Single-Writer Fencing above). The lease is only responsible for availability and routing, the epoch fence is responsible for correctness, which is why the election layer does not have to be a perfect leader election to be safe. The same promote, zombie, and network-partition fencing is exercised end to end by the SQL-level failover suite under `test/ha_replication/`.

The agent and any operator drive entirely off the engine's status variables, a node is primary when `Tidesdb_replica_mode_active = 0` and `Tidesdb_primary_epoch > 0`, a promotion took when `Tidesdb_primary_epoch` advances and `SET GLOBAL tidesdb_promote_primary = ON` returns no error, and a node was fenced when `Tidesdb_replica_mode_active` flips back to 1.

Getting going is a single apply:



```
kubectl apply -f k8s/production/tidesql-ha.yaml
kubectl -n tidesql get pods -L role -w          # watch one pod become
role=primary
```

Applications then connect for writes through `tidesql-write.tidesql.svc` and for reads through `tidesql-read.tidesql.svc`. The pods carry no persistent local state worth preserving, a rescheduled pod starts with a fresh data directory, the plugin's schema discovery recreates the database directories at startup from the `__tidesql_schema` column family, and table definitions are materialized on first access, so no manual `CREATE DATABASE` is ever needed on a replica.

A few things to set for your own cluster, covered in `k8s/production/README.md`, fill the `tidesql-s3` secret with real keys (or drop it and use IRSA / workload identity), change `storageClassName: gp3` (AWS EKS-specific) to your platform's class, wire real engine auth in place of the placeholder `passwordless-root-over-socket` the probes use, decide `tidesdb_objstore_wal_sync_on_commit` (`OFF` for a small RPO and lower commit latency, `ON` for RPO near zero), and tune the lease TTL and the agent's renew interval together (keep renew below half the TTL) to trade failover speed against tolerance for blips. The agent sidecar image needs `kubectl`, `jq`, the `mariadb` client, and GNU `date` (BusyBox `date` cannot parse the RFC3339 lease timestamps), so build it on a glibc/coreutils base.

Build Requirement

The S3 connector requires TidesDB to be built with `-DTIDESDB_WITH_S3=ON`, which needs `libcurl` only. TidesDB uses its own internal crypto, so there is no OpenSSL dependency. The filesystem connector is always available without additional dependencies and is selected with

`tidesdb_object_store_backend = FS` plus `tidesdb_objstore_fs_path = <shared dir>`.

Pointing several nodes at the same directory lets you exercise replication and failover, including the single-writer fence, on one machine without standing up S3.

Limitations

There are a few things to be aware of when using TidesDB:

The engine does not support foreign keys. Cross-table referential integrity must be enforced at the application level. When a `CREATE TABLE` statement includes `FOREIGN KEY` clauses, the constraint is silently ignored and only the supporting index is created. To convert an InnoDB table that has foreign keys to TidesDB, disable FK checks first:

```
SET FOREIGN_KEY_CHECKS = 0;
ALTER TABLE t1 ENGINE = TidesDB;
SET FOREIGN_KEY_CHECKS = 1;
```

Changing the primary key of a table requires a full copy rebuild. The engine does not support inplace primary key changes. Changing a column's type (e.g., `INT` to `BIGINT`) also requires a copy rebuild.

The statistics cache refreshes every two seconds, so immediately after a bulk load, the optimizer may briefly see stale row counts. Running `ANALYZE TABLE` forces an immediate refresh.

Multi-statement transactions at `REPEATABLE_READ` or higher isolation may fail at commit time with `ER_LOCK_DEADLOCK` (ERROR 1213) due to optimistic concurrency control conflicts. Applications using explicit `BEGIN ... COMMIT` blocks should implement retry logic for this error. Autocommit statements are retried automatically by MariaDB. With the default `tidesdb_pessimistic_locking=ON`, hot-row contention is serialized through row locks rather than surfacing as commit-time conflicts, at the cost of introducing lock waits and the possibility of wait-for-graph deadlocks (also ERROR 1213). Setting the variable to `OFF` reverts to pure optimistic concurrency where commit-time conflicts are the only signal and the application is responsible for retry.

—

MariaDB Compatibility

MariaDB Version	TideSQL Version	Full Support
10.x.x	✗	✗
11.4.10	>= 3.4.0	✓
11.8.6	>= 4.0.0	✓
12.2.2	>= 1.0.0	✓
12.3.1	>= 4.2.6	✓
13.0.2	>= 4.5.4	✓

Current TideSQL release is 4.5.9 (hex `0x40509`).

As versions are tested and confirmed working we update this table. Full Support means the system is tested against all known functionality.

—

TideSQL repository: <https://github.com/tidesdb/tidesql>

TidesDB Kafka Streams Plugin Reference

Overview

The TidesDB Kafka Streams plugin is a drop-in replacement for the default RocksDB state stores in [Apache Kafka Streams](#). It provides a `KeyValueStore` implementation backed by TidesDB, giving Kafka Streams applications access to TidesDB's ACID transactions, MVCC concurrency, LSM-tree storage, configurable compression, bloom filters, block indexes, B+tree klog format, TTL-based expiration, commit hooks for change data capture, online backups, and lightweight checkpoints, all through the standard Kafka Streams state store interface.

Switching from RocksDB to TidesDB requires no changes to your stream topology. You replace the store supplier or builder, and the plugin handles the rest. The underlying TidesDB database is managed automatically, the plugin creates and opens the database in Kafka Streams' state directory, manages column families, handles transactions, and closes the database on shutdown.

Getting Started

Prerequisites

- Java 11 or higher
- Maven 3.6+
- TidesDB native C library installed on the system (see [Building](#))
- TidesDB Java bindings installed (`com.tidesdb:tidesdb-java`)

Adding to Your Project

Maven

```
<dependency>
  <groupId>com.tidesdb</groupId>
  <artifactId>tidesdb-kafka</artifactId>
  <version>0.4.3</version>
</dependency>
```

You must also ensure the TidesDB JNI shared library is on the Java library path at runtime:



```
-Djava.library.path=/usr/local/lib
```

Usage

Basic Usage with Materialized

The simplest way to use TidesDB as a state store is through `Materialized.as()` with the store supplier:

```
import com.tidesdb.kafka.store.TidesDBStoreSupplier;

StreamsBuilder builder = new StreamsBuilder();

KStream<String, String> input = builder.stream("input-topic");

KTable<String, Long> counts = input
    .groupByKey()
    .count(Materialized.as(new TidesDBStoreSupplier("my-counts")));

counts.toStream().to("output-topic");
```

This creates a TidesDB-backed state store with default configuration: LZ4 compression, bloom filters enabled at 1% FPR, block indexes enabled, `SYNC_NONE` durability mode, 64 MB write buffer, and 64 MB block cache.

Usage with StoreBuilder

For topology-level state stores, use the builder:

```
import com.tidesdb.kafka.store.TidesDBStoreBuilder;

StoreBuilder<TidesDBStore> storeBuilder = new TidesDBStoreBuilder("my-
store")
    .withLoggingEnabled(Collections.emptyMap());

builder.addStateStore(storeBuilder);

builder.stream("input-topic")
    .process(() -> new MyProcessor(), "my-store");
```

Custom Configuration

Both the supplier and builder accept a `TidesDBStoreConfig` for fine-grained control over every TidesDB database and column family parameter:

```
import com.tidesdb.kafka.store.TidesDBStoreConfig;
import com.tidesdb.kafka.store.TidesDBStoreSupplier;
import com.tidesdb.*;

TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .compressionAlgorithm(CompressionAlgorithm.ZSTD_COMPRESSION)
    .enableBloomFilter(true)
    .bloomFPR(0.001)
    .writeBufferSize(128 * 1024 * 1024) // 128 MB
    .blockCacheSize(128 * 1024 * 1024) // 128 MB
    .syncMode(SyncMode.SYNC_NONE)
    .enableBlockIndexes(true)
    .numFlushThreads(4)
    .numCompactionThreads(4)
    .build();

KTable<String, Long> counts = input
    .groupByKey()
    .count(Materialized.as(new TidesDBStoreSupplier("my-counts", config)));
```

Or with the builder:

```
TidesDBStoreBuilder storeBuilder = TidesDBStoreBuilder.create("my-  
store", config)  
    .withCachingEnabled()  
    .withLoggingEnabled(Collections.emptyMap());
```

Configuration Reference

Database Configuration

These settings control the TidesDB database instance that backs the state store. Each Kafka Streams state store opens its own independent TidesDB database.

Option	Type	Default	Description
<code>numFlushThreads</code>	int	2	Number of background threads flushing memtables to SSTables
<code>numCompactionThreads</code>	int	2	Number of background threads performing LSM compaction
<code>logLevel</code>	LogLevel	INFO	TidesDB internal log level (DEBUG, INFO, WARN, ERROR, FATAL, NONE)
<code>blockCacheSize</code>	long	64 MB	Size of the block cache shared across all column families
<code>maxOpenSSTables</code>	long	256	Maximum number of SSTable file handles cached in the LRU
<code>maxMemoryUsage</code>	long	0 (auto)	Global memory limit in bytes; 0 lets TidesDB auto-detect (50% of system RAM)
<code>logToFile</code>	boolean	false	Write TidesDB logs to a file instead of stderr
<code>logTruncationAt</code>	long	24 MB	Log file truncation size; 0 disables truncation
<code>maxConcurrentFlushes</code>	int	0 (library default)	Global semaphore on in-flight memtable flushes across all column families; 0 inherits the engine default
<code>raiseOpenFileLimit</code>	long	0 (untouched)	Raise the process open-file ceiling toward this many descriptors before open, so the engine can keep more SSTables open; applied at init before the database opens. 0 leaves the limit untouched
<code>cancelBackgroundWorkOnClose</code>	boolean	false	When true, cancel background compaction db-wide right before close for a fast shutdown; flushes are unaffected so durability is preserved
<code>finishCompactionsOnClose</code>	boolean	false	Close behavior for in-flight compactions. false (default) cancels them at their next checkpoint for a fast shutdown (no data lost; recovery handles a mid-merge state); true waits for in-flight compactions to finish before <code>close()</code> returns

Option	Type	Default	Description
<code>objectStoreFsPath</code>	String	null	Filesystem path for object store connector; null disables object store mode
<code>objectStoreS3Config</code>	S3Config	null	S3-compatible object store connector (AWS S3, MinIO, GCS); takes precedence over <code>objectStoreFsPath</code> . Requires a native build with <code>TIDESDB_WITH_S3=ON</code>
<code>objectStoreConfig</code>	ObjectStoreConfig	null	Object store behavior configuration; null uses defaults
<code>unifiedMemtable</code>	boolean	false	Enable unified memtable mode (single memtable + WAL for all CFs)
<code>unifiedMemtableWriteBufferSize</code>	long	0 (auto)	Unified memtable write buffer size; 0 uses 64 MB default
<code>unifiedMemtableSkipListMaxLevel</code>	int	0 (default 12)	Skip list max level for unified memtable
<code>unifiedMemtableSkipListProbability</code>	float	0 (default 0.25)	Skip list probability for unified memtable
<code>unifiedMemtableSyncMode</code>	int	0 (SYNC_NONE)	Sync mode for unified WAL
<code>unifiedMemtableSyncIntervalUs</code>	long	0	Sync interval for unified WAL in microseconds

Column Family Configuration

Each state store maps to a single TidesDB column family. These settings control the column family's storage behavior.

Option	Type	Default	Description
<code>columnFamilyName</code>	String	"default"	Name of the column family
<code>writeBufferSize</code>	long	64 MB	Memtable flush threshold
<code>compressionAlgorithm</code>	CompressionAlgorithm	LZ4_COMPRESSION	Compression algorithm for SSTables
<code>enableBloomFilter</code>	boolean	true	Enable bloom filters for point lookups
<code>bloomFPR</code>	double	0.01	Bloom filter false positive rate (1%)
<code>enableBlockIndexes</code>	boolean	true	Enable compact block indexes for efficient seeking
<code>indexSampleRatio</code>	int	1	Sample every Nth block for the index
<code>blockIndexPrefixLength</code>	int	16	Block index prefix length in bytes
<code>syncMode</code>	SyncMode	SYNC_NONE	Durability mode for WAL writes
<code>syncInterval</code>	long	128000	Sync interval in microseconds (for SYNC_INTERVAL mode)
<code>useBtree</code>	boolean	false	Use B+tree klog format instead of block-based SSTables
<code>minLevels</code>	int	5	Minimum number of LSM levels
<code>levelSizeRatio</code>	long	10	Level size multiplier for LSM compaction
<code>skipListMaxLevel</code>	int	12	Skip list max level for memtables
<code>skipListProbability</code>	float	0.25	Skip list promotion probability
<code>defaultIsolationLevel</code>	IsolationLevel	READ_COMMITTED	Default transaction isolation level

Option	Type	Default	Description
<code>el</code>			
<code>klogValueThreshold</code>	long	512	Values larger than this go to the value log
<code>l0QueueStallThreshold</code>	int	20	Number of L0 immutable memtables before stalling writes
<code>l1FileCountTrigger</code>	int	4	Number of L1 files that triggers compaction
<code>dividingLevelOffset</code>	int	2	Compaction dividing level offset
<code>minDiskSpace</code>	long	100 MB	Minimum free disk space required before writes are rejected
<code>comparatorName</code>	String	"" (memcmp)	Custom comparator name; empty uses default binary comparison
<code>tombstoneDensityTrigger</code>	double	0.0 (disabled)	Per-SSTable tombstone density above which compaction priority escalates; range 0.0 to 1.0
<code>tombstoneDensityMinEntries</code>	long	1024	Minimum entry count for an SSTable to be considered by the tombstone density trigger
<code>objectLazyCompaction</code>	boolean	false	Compact less aggressively in object store mode
<code>objectPrefetchCompaction</code>	boolean	true	Download all inputs before merge in object store mode

Store Behavior

Option	Type	Default	Description
<code>defaultTtlSeconds</code>	long	-1 (disabled)	Default TTL in seconds applied to all put operations; -1 means no expiration

Compression Algorithms

Algorithm	Description
<code>NO_COMPRESSION</code>	No compression
<code>SNAPPY_COMPRESSION</code>	Snappy compression
<code>LZ4_COMPRESSION</code>	LZ4 standard compression (default)
<code>ZSTD_COMPRESSION</code>	Zstandard compression (best ratio)
<code>LZ4_FAST_COMPRESSION</code>	LZ4 fast mode (higher throughput, lower ratio)

Sync Modes

Mode	Description
<code>SYNC_NONE</code>	No explicit sync; relies on OS page cache (fastest, default for Kafka plugin)
<code>SYNC_FULL</code>	Fsync on every write (most durable)
<code>SYNC_INTERVAL</code>	Periodic background syncing at configurable intervals

The default sync mode for the Kafka plugin is `SYNC_NONE`. This is appropriate because Kafka Streams' changelog topics provide durability, and state can always be rebuilt from the changelog. If your application requires local durability guarantees beyond what Kafka provides, set `SYNC_FULL` or `SYNC_INTERVAL`.

Transaction Isolation Levels

Level	Description
<code>READ_UNCOMMITTED</code>	Sees all data including uncommitted changes
<code>READ_COMMITTED</code>	Sees only committed data (default)
<code>REPEATABLE_READ</code>	Consistent snapshot; phantom reads possible
<code>SNAPSHOT</code>	Write-write conflict detection
<code>SERIALIZABLE</code>	Full read-write conflict detection (SSI)

For most Kafka Streams workloads, `READ_COMMITTED` (the default) is sufficient. Higher isolation levels add overhead and are only needed when external threads access the store concurrently with custom logic.

Object Store Configuration

When `objectStoreFsPath` is set, TidesDB operates in object store mode, storing SSTables in an external object store with a local file cache. The `ObjectStoreConfig` controls cache behavior, upload concurrency, WAL replication, and replica mode.

Option	Type	Default	Description
<code>localCachePath</code>	String	null (use db_path)	Local directory for cached SSTable files
<code>localCacheMaxBytes</code>	long	0 (unlimited)	Maximum local cache size in bytes
<code>cacheOnRead</code>	boolean	true	Cache downloaded files locally
<code>cacheOnWrite</code>	boolean	true	Keep local copy after upload
<code>maxConcurrentUploads</code>	int	4	Parallel upload threads
<code>maxConcurrentDownloads</code>	int	8	Parallel download threads
<code>multipartThreshold</code>	long	64 MB	Use multipart upload above this size
<code>multipartPartSize</code>	long	8 MB	Multipart chunk size
<code>syncManifestToObject</code>	boolean	true	Upload MANIFEST after each compaction
<code>replicateWal</code>	boolean	true	Upload closed WAL segments for recovery
<code>walUploadSync</code>	boolean	false	Block flush until WAL is uploaded
<code>walSyncThresholdBytes</code>	long	1 MB	Sync active WAL when it grows by this many bytes; 0 disables
<code>walSyncOnCommit</code>	boolean	false	Upload WAL after every commit for RPO=0 replication
<code>replicaMode</code>	boolean	false	Enable read-only replica mode
<code>replicaSyncIntervalUs</code>	long	5000000	MANIFEST poll interval in microseconds for replicas
<code>replicaReplayWal</code>	boolean	true	Replay WAL for near-real-time reads on replicas

TTL Support

TidesDB supports time-to-live (TTL) on individual key-value pairs. Expired entries are removed during compaction. The plugin exposes TTL in two ways.

Default TTL

Set a default TTL that applies to all `put` operations automatically:

```
TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .defaultTtlSeconds(3600) // 1 hour
    .build();

KTable<String, Long> counts = input
    .groupByKey()
    .count(Materialized.as(new TidesDBStoreSupplier("expiring-counts",
config)));
```

Every entry written through `put`, `putIfAbsent`, and `putAll` will expire after the configured duration.

Per-Key TTL

For fine-grained control, use `putWithTtl` directly on the store:

```
TidesDBStore store = (TidesDBStore) context.getStateStore("my-store");
store.putWithTtl(key, value, 300); // expires in 5 minutes
```

A TTL of -1 means no expiration.

B+tree KLog Format

Column families can optionally use a B+tree structure for the key log instead of the default block-based SSTable format. The B+tree format offers faster point lookups through $O(\log N)$ tree traversal at the cost of slightly higher write amplification.

```
TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .useBtree(true)
    .build();
```

When to use B+tree format

- Read-heavy Kafka Streams workloads with frequent key lookups
- Interactive queries where read latency matters more than write throughput
- Large state stores where block scanning becomes expensive

Tradeoffs

- Slightly higher write amplification during flush
- Larger metadata overhead per node
- Block-based format may be faster for full iteration and range scans

The format is set at column family creation time and cannot be changed afterward. If you need to switch formats, delete the state store directory and let Kafka Streams rebuild from the changelog.

Change Data Capture with Commit Hooks

TidesDB supports commit hooks, callbacks that fire synchronously after every transaction commit. This enables real-time change data capture without WAL parsing.

```
TidesDBStore store = (TidesDBStore) context.getStateStore("my-store");

store.setCommitHook((ops, commitSeq) -> {
    for (CommitOp op : ops) {
        if (op.isDelete()) {
            System.out.println("DELETE key=" + new String(op.getKey()));
        } else {
            System.out.println("PUT key=" + new String(op.getKey()));
        }
    }
    return 0; // 0 = success
});
```

Each `CommitOp` contains `getKey()`, `getValue()` (null for deletes), `getTtl()` (-1 for no expiry), and `isDelete()`. The `commitSeq` is monotonically increasing and can be used as a replication cursor.

To detach the hook:

```
store.clearCommitHook();
```

Hooks execute synchronously on the committing thread. Keep the callback fast to avoid stalling writers.

Operations and Maintenance

Statistics

The store exposes TidesDB statistics at three levels:

Column family statistics

```
TidesDBStore store = (TidesDBStore) context.getStateStore("my-store");
Stats stats = store.getStats();

System.out.println("Total keys: " + stats.getTotalKeys());
System.out.println("Data size: " + stats.getTotalDataSize());
System.out.println("Memtable size: " + stats.getMemtableSize());
System.out.println("Read amplification: " + stats.getReadAmp());
System.out.println("Cache hit rate: " + stats.getHitRate());

// Tombstone observability
System.out.println("Total tombstones: " + stats.getTotalTombstones());
System.out.printf("Tombstone ratio: %.2f%%\n", stats.getTombstoneRatio() *
100.0);
System.out.printf("Worst SSTable density: %.2f%% at level %d\n",
    stats.getMaxSstDensity() * 100.0, stats.getMaxSstDensityLevel());
long[] levelTombstoneCounts = stats.getLevelTombstoneCounts();
```

`getTotalKeys()` includes the active memtable plus every SSTable, so writes are visible to stats without a prior flush. The tombstone observability fields (`getTotalTombstones`, `getTombstoneRatio`, `getMaxSstDensity`, `getMaxSstDensityLevel`, `getLevelTombstoneCounts`) pair with the column family `tombstoneDensityTrigger` and `tombstoneDensityMinEntries` settings to monitor and drive density-based compaction escalation.

Database-level statistics

```
DbStats dbStats = store.getDbStats();

System.out.println("Column families: " + dbStats.getNumColumnFamilies());
System.out.println("Memory pressure: " + dbStats.getMemoryPressureLevel());
System.out.println("Flush queue: " + dbStats.getFlushQueueSize());
System.out.println("Compaction queue: " +
dbStats.getCompactionQueueSize());
System.out.println("Total SSTables: " + dbStats.getTotalSstableCount());

// Object-store single-writer fencing (0 when not in object-store/replica
mode)
System.out.println("Primary lease epoch: " + dbStats.getPrimaryEpoch());
System.out.println("Highest observed epoch: " + dbStats.getSeenEpoch());
```

`getPrimaryEpoch()` is the lease epoch this primary currently holds (0 when it is not a primary or holds no lease); `getSeenEpoch()` is the highest lease epoch this node has observed. In object-store mode a fenced primary sees `isReplicaMode()` flip back to true once a newer epoch is seen, making these two counters the signal for detecting a lost write lease.

Block cache statistics

```
CacheStats cacheStats = store.getCacheStats();

System.out.println("Cache enabled: " + cacheStats.isEnabled());
System.out.println("Hit rate: " + cacheStats.getHitRate());
System.out.println("Entries: " + cacheStats.getTotalEntries());
```

Compaction and Flush

```
// Non-blocking compaction
store.compact();

// Non-blocking flush
store.flush();

// Synchronous flush + aggressive compaction (blocks until complete)
store.purge();

// Purge all column families and drain all queues
store.purgeAll();

// Check background activity
boolean flushing = store.isFlushing();
boolean compacting = store.isCompacting();
```

Use `purge()` before backup, after bulk deletes, or during maintenance windows. Use `compact()` and `flush()` for non-blocking background work.

Targeted Range Compaction

`compactRange` runs a synchronous compaction over a specific key range. Only SSTables whose key range overlaps the bounds participate in the merge, so the work and I/O are bounded to the affected portion of the LSM tree.

```
byte[] start = "tenant_42:".getBytes(StandardCharsets.UTF_8);
byte[] end   = "tenant_42;".getBytes(StandardCharsets.UTF_8);

store.compactRange(start, end);
```

A null or empty endpoint means unbounded on that side; both null/empty is rejected (use `compact()` for full-CF compaction). Use this after a large range delete, tenant eviction, or when you want to reclaim space immediately rather than waiting for the natural tombstone density trigger to fire.

WAL Sync

Force an immediate fsync of the write-ahead log:

```
store.syncWal();
```

This is useful when running with `SYNC_NONE` or `SYNC_INTERVAL` and you need to guarantee durability at a specific point.

Fast Shutdown

`cancelBackgroundWork()` cancels background compaction across the whole database. In-flight merges bail safely at their next checkpoint (discarding uncommitted output, leaving inputs intact) and queued compaction is skipped. Flushes are unaffected, so durability is preserved. The cancellation is sticky for the session and resets on the next open.

```
store.cancelBackgroundWork();  
store.close();
```

This is intended for a fast shutdown when you do not want to wait for long-running compactons to finish. Set `cancelBackgroundWorkOnClose(true)` on the config to have `close()` invoke it automatically — handy when a Kafka Streams task is reassigned or the application is shutting down and you want state stores to release quickly:

```
TidesDBStoreConfig config = TidesDBStoreConfig.builder()  
    .cancelBackgroundWorkOnClose(true)  
    .build();
```

A related, lower-level knob is `finishCompactionsOnClose`, applied to the database at open time. When false (the default), `close()` cancels any in-flight compactons at their next checkpoint — the merge discards its uncommitted output and leaves inputs intact, so no data is lost and recovery handles the mid-merge state. When true, `close()` waits for in-flight compactons to run to completion before returning:

```
TidesDBStoreConfig config = TidesDBStoreConfig.builder()  
    .finishCompactionsOnClose(true)    // wait for compactons to finish on  
close  
    .build();
```

The two settings are independent: `cancelBackgroundWorkOnClose` proactively cancels background work right before close, while `finishCompactionsOnClose` chooses whether close waits for or abandons compactions already running. Leave both at their defaults for the fastest shutdown.

Raising the Open-File Limit

A column family keeps SSTables open as file descriptors. On a busy store the process can hit its open-file ceiling, capping how many SSTables stay cached. `raiseOpenFileLimit(long)` raises this process's ceiling (the POSIX `RLIMIT_NOFILE` soft limit toward the hard limit) so the engine can keep more SSTables open.

It is process-wide, opt-in, and **must be applied before the database opens** — the engine sizes `maxOpenSSTables` to fit the ceiling at open time. The static helper just reports the current ceiling when called with `<= 0`:

```
long ceiling = TidesDBStore.raiseOpenFileLimit(65536); // returns the
effective ceiling
```

The usual way is to set it on the config so it is applied automatically during `init()`, before open:

```
TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .raiseOpenFileLimit(65536)
    .maxOpenSSTables(8192)
    .build();
```

A failed or partial raise is non-fatal — TidesDB simply works within whatever ceiling is in effect.

Backup and Checkpoint

Online backup · copies all data to a new directory without blocking reads or writes:

```
store.backup("/path/to/backup");
```

Lightweight checkpoint · uses hard links for near-instant snapshots (same filesystem only):

```
store.checkpoint("/path/to/checkpoint");
```

	<code>backup()</code>	<code>checkpoint()</code>
Speed	Copies every SSTable byte-by-byte	Near-instant (hard links)
Disk usage	Full independent copy	No extra disk until compaction removes old SSTables
Portability	Can be moved to another filesystem or machine	Same filesystem only
Use case	Archival, disaster recovery	Fast local snapshots

Runtime Configuration Updates

Update runtime-safe column family settings without restarting:

```
ColumnFamilyConfig newConfig = ColumnFamilyConfig.builder()
    .writeBufferSize(256 * 1024 * 1024)
    .bloomFPR(0.001)
    .syncMode(SyncMode.SYNC_INTERVAL)
    .syncIntervalUs(100000)
    .build();

store.updateRuntimeConfig(newConfig, true);
```

Updatable settings include `writeBufferSize`, `skipListMaxLevel`, `skipListProbability`, `bloomFPR`, `indexSampleRatio`, `syncMode`, and `syncIntervalUs`.

Single-Delete

`singleDelete` writes a tombstone with the same read semantics as `delete`, but lets compaction drop the put and tombstone together as soon as both appear in the same merge input - rather than carrying the tombstone forward until it reaches the largest active level.

```
TidesDBStore store = (TidesDBStore) context.getStateStore("my-store");
store.singleDelete(Bytes.wrap("session:42".getBytes()));
```

Caller contract: between any two single-deletes on the same key, the key has been put **at most once**. The engine cannot verify this; violating the contract can leave older puts visible after the single-delete. Use only for insert-once / delete-once workloads (session caches, secondary-index entries that are never updated, log-style state with scheduled purges). When in doubt, prefer `delete`.

Range Cost Estimation

Estimate the cost of iterating between two keys without performing any disk I/O:

```
double costA = store.rangeCost("user:0000".getBytes(),
    "user:0999".getBytes());
double costB = store.rangeCost("user:1000".getBytes(),
    "user:1099".getBytes());
```

This is useful for query planning, load balancing range scan work across threads, and monitoring data distribution changes over time.

Unified Memtable Mode

Enable unified memtable mode to share a single memtable and WAL across all column families. This reduces write amplification for workloads with many small column families.

```
TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .unifiedMemtable(true)
    .unifiedMemtableWriteBufferSize(0)           // 0 = auto (64 MB default)
    .unifiedMemtableSkipListMaxLevel(0)         // 0 = default (12)
    .unifiedMemtableSkipListProbability(0)      // 0 = default (0.25)
    .unifiedMemtableSyncMode(0)                 // 0 = SYNC_NONE
    .unifiedMemtableSyncIntervalUs(0)           // 0 = default
    .build();

KTable<String, Long> counts = input
    .groupByKey()
    .count(Materialized.as(new TidesDBStoreSupplier("unified-counts",
        config)));
```

When unified memtable is enabled, all column families in the database instance share a single memtable and write-ahead log. This is useful when you have many column families with small write volumes, as it avoids per-CF WAL overhead.

Object Store Mode

TidesDB supports storing SSTables in an external object store (filesystem-backed or S3-compatible) with a local file cache. This enables tiered storage, remote replication, and read-only replicas.

Basic Object Store Setup

```
import com.tidesdb.ObjectStoreConfig;

TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .objectStoreFsPath("/mnt/shared/tidesdb-objects")
    .objectStoreConfig(ObjectStoreConfig.builder()
        .localCacheMaxBytes(1024 * 1024 * 1024)    // 1 GB local cache
        .maxConcurrentUploads(8)
        .maxConcurrentDownloads(16)
        .syncManifestToObject(true)
        .replicateWal(true)
        .build()
    ).build();

KTable<String, Long> counts = input
    .groupByKey()
    .count(Materialized.as(new TidesDBStoreSupplier("object-store-counts",
config)));
```

Replica Mode

Open a state store as a read-only replica that polls for updates from the object store:

```
TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .objectStoreFsPath("/mnt/shared/tidesdb-objects")
    .objectStoreConfig(ObjectStoreConfig.builder()
        .replicaMode(true)
        .replicaSyncIntervalUs(5000000)    // poll every 5 seconds
        .replicaReplayWal(true)            // near-real-time reads
        .build()
    ).build();
```

To switch a replica to primary mode at runtime:

```
TidesDBStore store = (TidesDBStore) context.getStateStore("my-store");
store.promoteToPrimary();
```


Column Family Object Store Options

Per-column-family options control SSTable sizing and compaction behavior in object store mode:

```
TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .objectStoreFsPath("/mnt/shared/tidesdb-objects")
    .objectLazyCompaction(true)           // compact less aggressively
    .objectPrefetchCompaction(true)       // download all inputs
before merge
    .build();
```

S3 Object Store

Back a state store with a real S3-compatible service (AWS S3, MinIO, GCS) using `S3Config`. It takes precedence over `objectStoreFsPath` and composes with `objectStoreConfig` for cache and replication tuning. `endpoint`, `bucket`, `accessKey`, and `secretKey` are required; the rest default to secure, AWS-friendly values (HTTPS, virtual-hosted URLs, TLS verification on).

```
import com.tidesdb.S3Config;
import com.tidesdb.ObjectStoreConfig;

S3Config s3 = S3Config.builder()
    .endpoint("s3.amazonaws.com")
    .bucket("my-streams-state")
    .prefix("app1/")           // optional
    .accessKey(System.getenv("AWS_ACCESS_KEY_ID"))
    .secretKey(System.getenv("AWS_SECRET_ACCESS_KEY"))
    .region("us-east-1")
    .build();

TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .objectStoreS3Config(s3)
    .objectStoreConfig(ObjectStoreConfig.builder()
        .localCacheMaxBytes(1024L * 1024 * 1024) // 1 GB local cache
        .maxConcurrentUploads(8)
        .build())
    .build();
```

For MinIO or another self-hosted endpoint, use path-style addressing (and disable SSL for plain HTTP):

```
S3Config minio = S3Config.builder()
    .endpoint("minio.local:9000")
    .bucket("tidesdb")
    .accessKey("minioadmin")
    .secretKey("minioadmin")
    .usePathStyle(true)
    .useSsl(false)
    .build();
```

The S3 connector requires the native TidesDB library to be built with `TIDESDB_WITH_S3=ON`. Probe support before configuring it; opening an S3-backed store against a library without S3 support fails with a clear error:

```
if (!TidesDBStore.isS3Available()) {
    // fall back to objectStoreFsPath, or fail fast with a helpful message
}
```

Column Family Management

The store exposes column family management operations for advanced use cases. The store's own column family is created automatically during `init()`; these operations let you manage additional column families that share the same database instance.

Creating and Fetching a Column Family

Create an extra column family and fetch its handle by name:

```
ColumnFamilyConfig cfConfig = ColumnFamilyConfig.builder()
    .compressionAlgorithm(CompressionAlgorithm.ZSTD_COMPRESSION)
    .build();

store.createColumnFamily("audit_cf", cfConfig);
ColumnFamily cf = store.getColumnFamily("audit_cf");
```

Use the no-argument `getColumnFamily()` for the store's own column family.

Listing Column Families

```
TidesDBStore store = (TidesDBStore) context.getStateStore("my-store");
String[] cfNames = store.listColumnFamilies();
for (String name : cfNames) {
    System.out.println("Column family: " + name);
}
```

Cloning a Column Family

Create a complete, independent copy of an existing column family:

```
store.cloneColumnFamily("default", "snapshot_cf");
```

The clone contains all data from the source at the time of cloning. Modifications to one do not affect the other.

Renaming a Column Family

Atomically rename a column family. Waits for any in-progress flush or compaction to complete before renaming:

```
store.renameColumnFamily("old_name", "new_name");
```

Dropping a Column Family

```
store.dropColumnFamily("old_cf");
```

A handle-based variant accepts a `ColumnFamily` object directly:

```
ColumnFamily cf = store.getDb().getColumnFamily("old_cf");
store.deleteColumnFamily(cf);
```

Custom Comparators

Register a custom comparator for use with column families. Built-in comparators include `"memcmp"` (default), `"lexicographic"`, `"uint64"`, `"int64"`, `"reverse"`, and `"case_insensitive"`.

```
TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .comparatorName("reverse")
    .build();

KTable<String, Long> counts = input
    .groupByKey()
    .count(Materialized.as(new TidesDBStoreSupplier("reverse-sorted",
config)));
```

Comparators are set at column family creation time and cannot be changed afterward.

Replica Promotion

If the database was opened in replica mode, switch to primary mode:

```
TidesDBStore store = (TidesDBStore) context.getStateStore("my-store");
store.promoteToPrimary();
```

Benchmarking

The plugin includes a comprehensive benchmark suite comparing TidesDB against RocksDB across multiple workload types. Benchmarks are fully configurable via system properties.

Running Benchmarks



```
# Run with default settings
mvn test -Dtest=StateStoreBenchmark \
    -DargLine="-Djava.library.path=/usr/local/lib"

# Run with custom data directory (e.g., fast SSD)
mvn test -Dtest=StateStoreBenchmark \
    -DargLine="-Djava.library.path=/usr/local/lib -
Dbenchmark.data.dir=/mnt/ssd/bench"

# Run with custom parameters
mvn test -Dtest=StateStoreBenchmark \
    -DargLine="-Djava.library.path=/usr/local/lib \
        -Dbenchmark.sizes=1000,10000,100000 \
        -Dbenchmark.value.size=256 \
        -Dbenchmark.mixed.ratio=80 \
        -Dbenchmark.percentiles=true"
```

Or use the included runner script:



```
./run.sh -b                # Run benchmarks
./run.sh -b -d /mnt/fast-ssd/bench # Run on specific directory
./run.sh -a                # Run tests, benchmarks, and generate
charts
```

Benchmark Parameters

All parameters are configurable via `-D` system properties:

Property	Default	Description
<code>benchmark.data.dir</code>	(temp)	Data directory for benchmark databases
<code>benchmark.sizes</code>	1000,5000,10000,50000,100000	Comma-separated operation counts for standard benchmarks
<code>benchmark.large.sizes</code>	100000,...,25000000	Sizes for large dataset benchmarks
<code>benchmark.threads</code>	1,2,4,8,16	Thread counts for concurrent access benchmarks
<code>benchmark.value.size</code>	64	Value size in bytes for standard benchmarks
<code>benchmark.large.value.size</code>	10240	Value size in bytes for large-value benchmarks
<code>benchmark.warmup</code>	3	Number of warmup iterations before measurement
<code>benchmark.iterations</code>	5	Number of measurement iterations (for statistical accuracy)
<code>benchmark.compaction.batch</code>	50000	Batch size for compaction pressure test
<code>benchmark.compaction.batches</code>	5	Number of batches for compaction pressure test
<code>benchmark.range.data</code>	50000	Data size for range scan benchmark
<code>benchmark.range.sizes</code>	10,100,1000,5000,10000	Comma-separated range sizes
<code>benchmark.mixed.ratio</code>	50	Read percentage for mixed workload (0-100)
<code>benchmark.seed</code>	42	Random seed for reproducibility
<code>benchmark.percentiles</code>	true	Enable per-operation latency percentile tracking

Benchmark Workloads

The suite runs the following workloads:

- Sequential Writes · ordered key insertion

- Random Writes · random key insertion
- Sequential Reads · ordered key lookups
- Random Reads · random key lookups
- Mixed Workload · configurable read/write ratio
- Range Scans · iterator-based range queries
- Bulk Writes · batched `putAll` operations
- Update Workload · overwriting existing keys
- Large Values · configurable large value sizes
- Full Iteration · complete store scan
- Delete Workload · sequential key deletion
- Large Datasets · up to 25M keys with warmup and statistical analysis (mean, stddev)
- Concurrent Access · multi-threaded mixed workload with throughput and scalability analysis
- Compaction Pressure · accumulated data over multiple batches to stress compaction
- Memory/CPU Metrics · write/read performance with heap memory and CPU usage tracking
- Latency Percentiles · per-operation nanosecond latencies with p50, p90, p95, p99, p99.9, and max

Generating Charts

After running benchmarks, generate visualizations:



```
./run.sh -c
```

This creates PNG charts in a timestamped `charts_*` directory including performance comparisons, speedup charts, throughput comparisons, error bar plots for large datasets, concurrent scalability curves, compaction pressure plots, memory usage comparisons, and a summary table.

Requirements for chart generation, Python 3, pandas, matplotlib, seaborn (installed automatically into a virtual environment by the runner script).

Running Tests



```
# Run unit tests
mvn test -Dtest=TidesDBStoreTest \
    -DargLine="-Djava.library.path=/usr/local/lib"

# Run all tests
mvn test -DargLine="-Djava.library.path=/usr/local/lib"
```

Performance Tuning

Write-Heavy Workloads

For workloads that prioritize write throughput:

```
TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .compressionAlgorithm(CompressionAlgorithm.LZ4_FAST_COMPRESSION)
    .syncMode(SyncMode.SYNC_NONE)
    .writeBufferSize(128 * 1024 * 1024)
    .enableBloomFilter(true)
    .bloomFPR(0.01)
    .numFlushThreads(4)
    .numCompactionThreads(4)
    .build();
```

Read-Heavy Workloads

For workloads that prioritize read latency:


```
TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .useBtree(true)
    .blockCacheSize(256 * 1024 * 1024)
    .enableBloomFilter(true)
    .bloomFPR(0.001)
    .enableBlockIndexes(true)
    .compressionAlgorithm(CompressionAlgorithm.LZ4_COMPRESSION)
    .build();
```

Memory-Constrained Environments

For environments with limited memory:

```
TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .blockCacheSize(16 * 1024 * 1024)
    .writeBufferSize(16 * 1024 * 1024)
    .maxMemoryUsage(256 * 1024 * 1024)
    .maxOpenSSTables(64)
    .build();
```

Fair Benchmarking Against RocksDB

The benchmark suite configures both engines with equivalent settings to ensure a fair comparison:

Setting	TidesDB	RocksDB
Compression	LZ4	LZ4
Bloom filter	1% FPR	10 bits/key (~1% FPR)
Block cache	64 MB	64 MB (LRU)
Write buffer	64 MB	64 MB
Background threads	2 flush + 2 compaction	4 (maxBackgroundJobs)
Sync / durability	SYNC_NONE	sync=false, WAL enabled
Block indexes	Enabled	Binary search
LSM levels	5	5
Bulk writes	Single transaction	WriteBatch

The one structural difference that cannot be eliminated is transaction overhead. TidesDB requires all operations to go through transactions (begin -> op -> commit), while RocksDB supports direct `db.put()/db.get()` calls. The plugin mitigates this with transaction reuse via `reset()`, but there is still a per-operation cost that is inherent to TidesDB's MVCC architecture. This cost is most visible on small-data read benchmarks; at larger dataset sizes, disk I/O dominates and the gap closes.

Architecture

Store Lifecycle

1. Creation · `TidesDBStoreSupplier.get()` or `TidesDBStoreBuilder.build()` creates a `TidesDBStore` instance with the provided configuration.
2. Initialization · `init()` is called by Kafka Streams with the state directory. The plugin opens a TidesDB database at `<stateDir>/<storeName>`, creates or opens the configured column family, and initializes a reusable transaction for the hot path.
3. Operation · `get`, `put`, `delete`, `putAll`, `range`, and `all` map directly to TidesDB transactions. Single-operation calls (`get`, `put`) reuse a pooled transaction via `reset()` to avoid allocation overhead. Multi-operation calls (`putAll`, `putIfAbsent`, `delete`) use fresh transactions for atomicity.
4. Flush · `flush()` triggers a non-blocking memtable flush to disk.
5. Close · `close()` frees the reusable transaction and closes the TidesDB database.

Transaction Reuse

The plugin maintains a single reusable transaction for the `get` and `put` hot path. After each commit, the transaction is reset via `reset()` instead of being freed and reallocated. This retains internal buffers and avoids repeated allocation overhead, which matters in Kafka Streams where every record processed may trigger a state store read or write.

When the reusable transaction is unavailable (e.g., concurrent access from a punctuator), the plugin falls back to creating a new transaction.

Direct Access

For advanced use cases, the store exposes direct access to the underlying TidesDB objects:

```
TidesDBStore store = (TidesDBStore) context.getStateStore("my-store");
TidesDB db = store.getDb();
ColumnFamily cf = store.getColumnFamily();
```

Use with caution · operations on these objects bypass the store's transaction management.

Examples

Word Count

```
import com.tidesdb.kafka.store.TidesDBStoreSupplier;

StreamsBuilder builder = new StreamsBuilder();

KStream<String, String> lines = builder.stream("text-input");

KTable<String, Long> wordCounts = lines
    .flatMapValues(line -> Arrays.asList(line.toLowerCase().split("\\W+")))
    .groupBy((key, word) -> word)
    .count(Materialized.as(new TidesDBStoreSupplier("word-counts")));

wordCounts.toStream()
    .to("word-counts-output", Produced.with(Serdes.String(),
        Serdes.Long()));
```

Windowed Aggregation

```
KStream<String, String> events = builder.stream("events");

events
    .groupByKey()
    .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(5)))
    .count()
    .toStream()
    .to("windowed-output");
```

Custom Aggregation with Config

```
TidesDBStoreConfig config = TidesDBStoreConfig.builder()
    .compressionAlgorithm(CompressionAlgorithm.ZSTD_COMPRESSION)
    .defaultTtlSeconds(86400) // 24-hour expiration
    .useBtree(true)
    .build();

activities
    .groupByKey()
    .aggregate(
        UserStats::new,
        (userId, activity, stats) -> { stats.addActivity(activity); return
stats; },
        Materialized.<String, UserStats>as(new TidesDBStoreSupplier("user-
stats", config))
        .withKeySerde(Serdes.String())
        .withValueSerde(new UserStatsSerde())
    );
```

TidesDB Kafka Streams plugin repository: <https://github.com/tidesdb/tidesdb-kafka>

TidesDB Admintool Reference

Overview

Admintool is a command-line utility for managing and inspecting TidesDB databases. It provides interactive and non-interactive modes for database operations, diagnostics, and maintenance.

Installation

Build from source:



```
git clone https://github.com/tidesdb/admintool.git
cd admintool
mkdir build && cd build
cmake ..
make
```

Usage



```
admintool [options]
```

Options

Option	Description
<code>-h, --help</code>	Show help message
<code>-v, --version</code>	Show TidesDB version
<code>-d, --directory <path></code>	Open database at path on startup
<code>-c, --command <cmd></code>	Execute command and exit

Examples



```
# Interactive mode
admintool

# Open database on startup
admintool -d ./mydb

# Execute single command
admintool -d ./mydb -c "cf-list"
```

Database Operations

open

Open or create a database.

```
open <path> [options]
```

Options

Option	Description
<code>--unified</code>	Enable unified memtable mode
<code>--unified-buffer-size <n></code>	Unified memtable write buffer size (bytes)
<code>--unified-skip-list-max-level <n></code>	Skip list max level for unified memtable (0 = default 12)
<code>--unified-skip-list-probability <f></code>	Skip list probability for unified memtable (0 = default 0.25)
<code>--unified-sync <mode></code>	Unified WAL sync mode (<code>none</code> <code>interval</code> <code>full</code>)
<code>--unified-sync-interval <us></code>	Unified WAL sync interval (microseconds)
<code>--cache-size <n></code>	Block cache size (bytes, 0 to disable)
<code>--max-open-sstables <n></code>	Max cached SSTable structures
<code>--flush-threads <n></code>	Flush thread pool size
<code>--compaction-threads <n></code>	Compaction thread pool size
<code>--max-concurrent-flushes <n></code>	Global cap on in-flight memtable flushes across all CFs (0 = default)
<code>--finish-compactions-on-close</code>	Let in-flight compactions run to completion before <code>close</code> returns (default cancels them at the next checkpoint for a fast shutdown — no data is lost either way)
<code>--max-memory <n></code>	Global memory limit (bytes, 0 = auto)
<code>--log-level <level></code>	Log level: <code>debug</code> <code>info</code> <code>warn</code> <code>error</code> <code>fatal</code> <code>none</code> (default: <code>none</code>)
<code>--log-to-file</code>	Write logs to <code><db_path>/LOG</code> instead of stderr
<code>--log-truncation-at <n></code>	Log truncation size in bytes (0 = no truncation)
<code>--object-store-fs <root_dir></code>	Enable filesystem object-store connector. Auto-enables unified memtable mode

Option	Description
<code>--obj-cache-path <dir></code>	Local cache directory for object store (default: <code><db_path></code>)
<code>--obj-cache-max-bytes <n></code>	Maximum local cache size in bytes (0 = unlimited)
<code>--obj-replica-mode</code>	Open the database in read-only replica mode
<code>--obj-replica-sync-interval <us></code>	Replica MANIFEST poll interval (default: 5000000)
<code>--obj-wal-sync-on-commit</code>	Upload the WAL to the object store after every commit (RPO=0)
<code>--obj-cache-on-read <0 1></code>	Cache downloaded object files locally (default 1)
<code>--obj-cache-on-write <0 1></code>	Keep a local copy after upload (default 1)
<code>--obj-max-uploads <n></code>	Parallel upload threads (default 4)
<code>--obj-max-downloads <n></code>	Parallel download threads (default 8)
<code>--obj-multipart-threshold <n></code>	Use multipart upload at/above this object size (bytes)
<code>--obj-multipart-part-size <n></code>	Multipart chunk size (bytes)
<code>--obj-sync-manifest <0 1></code>	Upload the MANIFEST after each compaction (default 1)
<code>--obj-replicate-wal <0 1></code>	Upload closed WAL segments for node-failure recovery (default 1)
<code>--obj-wal-upload-sync <0 1></code>	Block flush until the WAL is uploaded (default 0 = background)
<code>--obj-wal-sync-threshold <n></code>	Sync the active WAL when it grows by this many bytes (0 = off)
<code>--obj-replica-replay-wal <0 1></code>	Replay the WAL for near-real-time replica reads (default 1)

Option	Description
<code>--object-store-s3</code>	Enable the S3-compatible object-store connector (requires a library built with <code>TIDESDB_WITH_S3=ON</code>)
<code>--s3-endpoint <ep></code>	S3 endpoint (e.g. <code>s3.amazonaws.com</code> or <code>minio.local:9000</code>)
<code>--s3-bucket <name></code>	S3 bucket name
<code>--s3-prefix <p></code>	S3 key prefix (e.g. <code>production/db1/</code>)
<code>--s3-access-key <k></code>	AWS access key ID (falls back to <code>\$AWS_ACCESS_KEY_ID</code>)
<code>--s3-secret-key <k></code>	AWS secret access key (falls back to <code>\$AWS_SECRET_ACCESS_KEY</code>)
<code>--s3-region <r></code>	AWS region (falls back to <code>\$AWS_REGION</code>)
<code>--s3-no-ssl</code>	Use HTTP instead of HTTPS
<code>--s3-path-style</code>	Use path-style URLs (required for MinIO)
<code>--s3-ca-path <path></code>	Custom CA bundle file for TLS verification
<code>--s3-insecure-skip-verify</code>	Disable TLS verification (test endpoints only; insecure)
<code>--s3-multipart-threshold <n></code>	S3 multipart upload threshold (bytes)
<code>--s3-multipart-part-size <n></code>	S3 multipart chunk size (bytes)

Examples

```

admintool> open ./mydb
Opened database at './mydb'

admintool> open ./mydb --unified
Opened database at './mydb' (unified memtable)

admintool> open ./mydb --unified --cache-size 134217728 --flush-threads 4
Opened database at './mydb' (unified memtable)

admintool> open ./mydb --log-level info --log-to-file
Opened database at './mydb'

admintool> open ./mydb --object-store-fs /mnt/objects
Opened database at './mydb' (object-store fs:/mnt/objects)

admintool> open ./mydb --object-store-s3 --s3-endpoint minio.local:9000 \
--s3-bucket tidesdb --s3-path-style --s3-no-ssl \
--s3-access-key minioadmin --s3-secret-key minioadmin
Opened database at './mydb' (object-store s3:minio.local:9000/tidesdb)

```

① Unified Memtable Mode

When `--unified` is set, all column families share a single skip list and WAL instead of each column family maintaining its own. A transaction touching N column families results in 1 WAL write instead of N. On-disk SSTables remain per-column-family. This mode is beneficial for write-heavy multi-CF workloads.

① Filesystem Object Store

`--object-store-fs <root_dir>` attaches a filesystem-backed object-store connector that mirrors objects under the given directory. This is intended for testing and local replication scenarios. Object-store mode automatically enables unified memtable mode. The connector is destroyed when the database is closed. Combine with `--obj-cache-max-bytes`, `--obj-replica-mode`, `--obj-replica-sync-interval`, and `--obj-wal-sync-on-commit` to tune behavior.

S3 Object Store

`--object-store-s3` attaches an S3-compatible connector (AWS S3, MinIO, etc.). It is only available when the linked TidesDB library was built with `TIDESDB_WITH_S3=ON`; otherwise the command reports `This TidesDB library was built without S3 support` and the open is rejected (the connector symbol is resolved at runtime, so the tool still runs against an S3-off library). At a minimum supply `--s3-endpoint` and `--s3-bucket`; credentials may be passed with `--s3-access-key/--s3-secret-key` or left to the standard `AWS_ACCESS_KEY_ID/AWS_SECRET_ACCESS_KEY/AWS_REGION` environment variables. Use `--s3-path-style` for MinIO. The shared `--obj-*` tuning flags apply to both the filesystem and S3 connectors.

close

Close the current database.

```
close
```

info

Show database information including column families, cache stats, and mode.

```
info
```

Output includes

- Database path
- Column family list
- Block cache statistics (entries, size, hits, misses, hit rate, partitions)
- Unified memtable status (enabled/disabled)
- Object store status and connector type (if active)
- Replica mode status (if active)

backup

Create a backup of the open database.

```
backup <destination_path>
```

version

Show TidesDB version.

```
version
```

Column Family Operations

cf-list

List all column families.

```
cf-list
```

cf-create

Create a new column family.

```
cf-create <name> [options]
```

Options

Option	Description
<code>--btree</code>	Use B+tree format for klog (faster point lookups)
<code>--compression <algo></code>	Compression algorithm: <code>none</code> <code>lz4</code> <code>lz4-fast</code> <code>zstd</code> <code>snappy</code>
<code>--bloom-fpr <rate></code>	Bloom filter false positive rate (e.g., <code>0.01</code> for 1%)
<code>--no-bloom</code>	Disable bloom filters
<code>--sync <mode></code>	WAL sync mode: <code>none</code> <code>interval</code> <code>full</code>
<code>--sync-interval <us></code>	Sync interval in microseconds (for <code>interval</code> mode)
<code>--comparator <name></code>	Key comparator: <code>memcmp</code> <code>lexicographic</code> <code>uint64</code> <code>int64</code> <code>reverse</code> <code>case_insensitive</code>
<code>--comparator-ctx <str></code>	Comparator context string passed to the comparator (max 255 chars)
<code>--write-buffer-size <n></code>	Memtable flush threshold (bytes)
<code>--klog-value-threshold <n></code>	Value size threshold for vlog storage (bytes)
<code>--block-index-prefix-len <n></code>	Block index prefix length
<code>--index-sample-ratio <n></code>	Block index sample ratio
<code>--no-block-indexes</code>	Disable block indexes
<code>--skip-list-max-level <n></code>	Skip list max level
<code>--skip-list-probability <f></code>	Skip list probability
<code>--isolation <level></code>	Default transaction isolation level
<code>--min-disk-space <n></code>	Minimum required disk space (bytes)
<code>--l1-trigger <n></code>	L1 file count compaction trigger
<code>--l0-stall <n></code>	L0 queue stall threshold
<code>--level-size-ratio <n></code>	Level size multiplier
<code>--min-levels <n></code>	Minimum LSM levels
<code>--dividing-level-offset <n></code>	Compaction dividing level offset
<code>--tombstone-density-trigger <ratio></code>	Tombstone density above which compaction priority escalates (<code>0.0</code> to <code>1.0</code> , <code>0</code> disables)

Option	Description
<code>--tombstone-density-min-entries <n></code>	Minimum entry count for an SSTable to be considered by the density trigger
<code>--object-lazy-compaction</code> / <code>--no-object-lazy-compaction</code>	Lazy compaction in object-store mode (doubles the L1 trigger, reducing remote I/O)
<code>--object-prefetch-compaction</code> / <code>--no-object-prefetch-compaction</code>	Prefetch all input SSTables in parallel before a compaction merge (default: enabled)

Examples

```
cf-create users
cf-create cache --btree
cf-create logs --compression zstd --sync full --bloom-fpr 0.001
cf-create events --comparator reverse --write-buffer-size 134217728
```

Compression algorithms

Algorithm	Description	Use case
<code>none</code>	No compression	Pre-compressed data, max write throughput
<code>lz4</code>	LZ4 standard (default)	General purpose, balanced
<code>lz4-fast</code>	LZ4 fast mode	Write-heavy, speed over ratio
<code>zstd</code>	Zstandard	Best compression ratio, storage-constrained
<code>snappy</code>	Snappy	Legacy compatibility (not available on SunOS)

Sync modes

Mode	Description	Use case
<code>none</code>	No explicit sync, relies on OS page cache	Caches, temporary data
<code>interval</code>	Periodic background syncing	Most production workloads
<code>full</code>	Fsync on every write	Financial transactions, audit logs

Built-in comparators

Name	Description
<code>memcmp</code>	Binary byte-by-byte comparison (default)
<code>lexicographic</code>	Null-terminated string comparison
<code>uint64</code>	Unsigned 64-bit integer comparison
<code>int64</code>	Signed 64-bit integer comparison
<code>reverse</code>	Reverse binary comparison (descending order)
<code>case_insensitive</code>	Case-insensitive ASCII comparison

cf-drop

Drop a column family.

```
cf-drop <name>
```

cf-rename

Rename a column family.

```
cf-rename <old_name> <new_name>
```

cf-clone

Clone a column family. Creates a complete copy of an existing column family with a new name, including all data.

```
cf-clone <source_name> <destination_name>
```

Behavior

- Flushes the source column family's memtable to ensure all data is on disk
- Waits for any in-progress flush or compaction to complete
- Copies all SSTable files (`.klog` and `.vlog`) to the new directory
- Copies manifest and configuration files
- The clone is completely independent; modifications to one do not affect the other

Use cases

- Testing with a copy of production data
- Creating a snapshot before experimental changes
- Data migration before configuration changes

Example

```
admintool(./mydb)> cf-clone users users_backup  
Cloned column family 'users' to 'users_backup'
```

cf-stats

Show column family statistics and configuration.

```
cf-stats <name>
```

Output includes

- Memtable size, levels, total keys
- Data size, avg key/value sizes
- Read amplification, cache hit rate
- Full configuration (compression, bloom filter, sync mode, dividing level offset, tombstone density trigger and minimum, object-store lazy/prefetch compaction, comparator name and context string, etc.)
- Per-level SSTable counts, sizes, key counts, and tombstone counts
- Tombstone observability: total tombstones, database-wide ratio, and the worst single-SSTable tombstone density (with the level it lives on)
- B+tree statistics (if enabled)
- Write amplification: lifetime on-disk framed bytes written by the WAL, flushes, and compactions; bytes read by compactions; logical user bytes committed; flush and compaction output SSTable counts; and the derived write-amplification ratio

cf-status

Show flush/compaction status.


```
cf-status <name>
```

cf-update

Update runtime-safe configuration settings for a column family. Changes apply to new operations only; existing SSTables and memtables retain their original settings.

```
cf-update <name> [options]
```

Options

All options from `cf-create` are supported except `--btree`, `--comparator`, and `--comparator-ctx` (these cannot be changed after creation). Additionally:

Option	Description
<code>--persist</code>	Save changes to disk (default)
<code>--no-persist</code>	Apply changes in-memory only

Updatable settings

- `--compression` · Compression for new SSTables (existing SSTables retain their original compression)
- `--bloom-fpr`, `--no-bloom` · Bloom filter settings for new SSTables
- `--sync`, `--sync-interval` · Durability mode (also updates the active WAL immediately)
- `--write-buffer-size` · Memtable flush threshold
- `--klog-value-threshold` · Value log threshold for new writes
- `--block-index-prefix-len`, `--index-sample-ratio`, `--no-block-indexes` · Block index settings for new SSTables
- `--skip-list-max-level`, `--skip-list-probability` · Skip list settings for new memtables
- `--isolation` · Default transaction isolation level
- `--level-size-ratio`, `--min-levels`, `--dividing-level-offset` · LSM level sizing
- `--l1-trigger`, `--l0-stall` · Compaction and backpressure thresholds
- `--tombstone-density-trigger`, `--tombstone-density-min-entries` · Tombstone-density compaction escalation
- `--object-lazy-compaction` / `--no-object-lazy-compaction`, `--object-prefetch-compaction` / `--no-object-prefetch-compaction` · Object-store mode tuning

- `--min-disk-space` · Minimum disk space required

Non-updatable settings

- `--comparator`, `--comparator-ctx` · Cannot change sort order after creation (would corrupt key ordering)
- `--btree` · Cannot change klog format after creation (existing SSTables use the original format)

Examples

```
admintool(./mydb)> cf-update users --compression zstd --bloom-fpr
0.001
Configuration updated for 'users' (persisted to disk)

admintool(./mydb)> cf-update cache --write-buffer-size 268435456 --no-
persist
Configuration updated for 'cache' (in-memory only)
```

cf-config-save

Save the current column family configuration to an INI file. The optional `section_name` defaults to the column family name.

```
cf-config-save <cf> <ini_file> [section_name]
```

Example

```
admintool(./mydb)> cf-config-save users ./users.ini
Saved configuration of 'users' to './users.ini' (section [users])
```

cf-config-load

Create a column family by loading its configuration from an INI file section. Useful for replicating a tuned configuration across databases or for templating column families.

```
cf-config-load <ini_file> <section_name> <cf_name>
```

Example

```
admintool(./mydb)> cf-config-load ./users.ini users users_replica
Created column family 'users_replica' from './users.ini' [users]
```

Key-Value Operations

put

Insert or update a key-value pair.

```
put <cf> <key> <value> [--ttl <seconds>]
```

- `--ttl <seconds>` · Set time-to-live in seconds from now. The key expires after the specified duration. If omitted, the key does not expire.

Examples

```
admintool(./mydb)> put users user:1 "John Doe"
OK

admintool(./mydb)> put sessions sess:abc "token123" --ttl 3600
OK (expires at 1711584000)
```

get

Retrieve a value by key.

```
get <cf> <key>
```

delete

Delete a key.

```
delete <cf> <key>
```

single-delete

Tombstone a key with single-delete semantics. Compaction can drop the put and the tombstone together as soon as both appear in the same merge input, rather than carrying the tombstone forward to the largest level.

```
single-delete <cf> <key>
```

When to use

`single-delete` is only safe when each key is put at most once between any two single-deletes (and at most once before the first single-delete on that key). Violating the contract can leave older puts visible after the tombstone. Prefer `delete` for any workload that issues repeated updates to the same key.

Example

```
admintool(./mydb)> single-delete users user:1  
OK
```

scan

Scan all keys in a column family.

```
scan <cf> [limit] [--reverse] [--isolation <level>]
```

Default limit: 100

Options

Option	Description
<code>--reverse</code>	Iterate from the last key to the first (descending order)
<code>--isolation</code> <code><level></code>	Open the read transaction at a specific isolation level: <code>read_uncommitted</code> <code>read_committed</code> <code>repeatable_read</code> <code>snapshot</code> <code>serializable</code>

Examples

```

admintool(./mydb)> scan users 10 --reverse
1) "user:9" -> "..."
2) "user:8" -> "..."
...
(10 entries, reverse)

admintool(./mydb)> scan users --isolation snapshot

```

range

Scan keys within a range.

```
range <cf> <start_key> <end_key> [limit]
```

prefix

Scan keys with a prefix.

```
prefix <cf> <prefix> [limit]
```

SSTable Inspection

sstable-list

List all SSTables in a column family.

```
sstable-list <cf>
```

sstable-info

Show SSTable file information.

```
sstable-info <klog_path>
```

On-disk format handling

The SSTable inspectors parse the klog directly off disk: they read the trailing metadata block to find how many leading blocks are key/value data blocks (the remaining blocks are the block-index, bloom and metadata blobs), skip each data block's header, and decompress data blocks with the column family's compression algorithm before decoding entries. Entries written by `single-delete` are labelled `[SINGLE-DEL]`; plain tombstones are labelled `[DEL]`.

sstable-dump

Dump SSTable entries (keys and values). Tombstones are shown as `[DEL]`, single-deletes as `[SINGLE-DEL]`, expiring keys as `[TTL:<expiry>]`, and vlog-stored values as `[VLOG:<offset>]`.

```
sstable-dump <klog_path> [limit]
```

Default limit: 1000

sstable-dump-full

Dump SSTable entries with vlog value resolution. When `vlog_path` is supplied, values stored in the vlog are read back and decompressed (using the SSTable's compression algorithm) instead of being shown as `(in vlog, N bytes)`.

```
sstable-dump-full <klog_path> [vlog_path] [limit]
```

sstable-stats

Show SSTable statistics, including total entries, tombstones (and how many are single-deletes), TTL entries, vlog references, sequence range, and key/value size distribution.

```
sstable-stats <klog_path>
```

sstable-keys

List only keys from an SSTable. Single-delete and plain tombstones are flagged inline.

```
sstable-keys <klog_path> [limit]
```

sstable-checksum

Verify SSTable block checksums.

```
sstable-checksum <klog_path>
```

bloom-stats

Show bloom filter statistics for an SSTable, including the filter size, hash function count, hash version (1 = legacy, 2 = fmix-finalized), fill ratio, and estimated false-positive rate.

```
bloom-stats <klog_path>
```

WAL Inspection

wal-list

List WAL files in a column family.

```
wal-list <cf>
```

wal-info

Show WAL file information.

```
wal-info <wal_path>
```

wal-dump

Dump WAL entries. Automatically detects per-CF and unified WAL formats.

```
wal-dump <wal_path> [limit]
```

Default limit: 1000

For unified WAL files (created when using `--unified` mode), each entry is prefixed with `[CF:<index>]` showing which column family the entry belongs to. Both WAL formats pack every operation of a committed transaction into a single block, so one block can hold many entries — `wal-dump` decodes them all. Entries are labelled `[PUT]`, `[DELETE]`, or `[SINGLE-DELETE]`.

Example (per-CF WAL)

```
admintool(./mydb)> wal-dump ./mydb/users/wal_1.log 5
WAL Entries (limit: 5):
1) [PUT] seq=1 key="user:1" value="John Doe"
2) [PUT] [TTL:1711584000] seq=2 key="sess:abc" value="token123"
3) [DELETE] seq=3 key="user:2"

(3 WAL entries dumped)
```

Example (unified WAL)


```
admintool(./mydb)> wal-dump ./mydb/uwal_0.log 5
WAL Entries (limit: 5):
1) [CF:0] [PUT] seq=10 key="user:1" value="Alice"
2) [CF:1] [PUT] seq=10 key="order:1" value="item_A"
3) [CF:0] [PUT] seq=11 key="user:2" value="Bob"

(3 WAL entries dumped)
```

wal-verify

Verify WAL integrity. Automatically detects per-CF and unified WAL formats and reports the format type.

```
wal-verify <wal_path>
```

Example

```
admintool(./mydb)> wal-verify ./mydb/users/wal_1.log
Verifying WAL: ./mydb/users/wal_1.log
  File Size: 4096 bytes
  Format: Per-CF WAL
  Valid Entries: 42
  Corrupted Entries: 0
  Sequence Range: 1 - 42
  Last Valid Position: 3840
  Status: OK
```

wal-checksum

Verify WAL block checksums.

```
wal-checksum <wal_path>
```

Maintenance

level-info

Show per-level SSTable details.

```
level-info <cf>
```

verify

Verify column family integrity.

```
verify <cf>
```

compact

Trigger compaction on a column family.

```
compact <cf>
```

compact-range

Run a synchronous compaction over a specific key range. Only SSTables whose minimum and maximum keys overlap the requested range participate in the merge, so the work and I/O are bounded to the affected portion of the LSM tree rather than the whole column family.

```
compact-range <cf> <start_key> <end_key>
```

Behavior

- Synchronous, blocks the caller until the merge commits or fails
- Selects only SSTables whose key range overlaps `[start_key, end_key)`
- Applies the same emit-loop logic as background compactions (tombstone reclamation, single-delete pair cancellation, sequence-based deduplication, value recompression)

- Output SSTables are committed to the manifest atomically and old inputs are marked for deletion

Use cases

- Bulk reclaim after a large range delete, where waiting for natural compaction would leave tombstones on disk
- Tenant eviction or sliding-window expiration that does not fit TTL semantics
- Post-import cleanup of a known key range
- Operational counterpart to the automatic tombstone density trigger when an operator wants reclaim now rather than at the next threshold crossing

Example

```
admintool(./mydb)> compact-range users tenant_42: tenant_42;  
Compacting range 'users' [tenant_42: .. tenant_42;)...  
Range compaction completed for 'users'
```

flush

Flush memtable to disk.

```
flush <cf>
```

purge-cf

Purge a single column family. Forces a synchronous flush and aggressive compaction, blocking until all I/O is complete.

```
purge-cf <name>
```

Behavior

1. Waits for any in-progress flush to complete
2. Force-flushes the active memtable (even if below threshold)
3. Waits for flush I/O to fully complete
4. Waits for any in-progress compaction to complete

5. Triggers synchronous compaction inline (bypasses the compaction queue)
6. Waits for any queued compaction to drain

Use cases

- Before backup or checkpoint to ensure all data is on disk and compacted
- After bulk deletes to reclaim space immediately
- Manual maintenance during a maintenance window
- Pre-shutdown to ensure all pending work is complete

Example

```
admintool(./mydb)> purge-cf users
Purging column family 'users' (synchronous flush + compaction)...
Purge completed for 'users'
```

purge

Purge all column families. Forces a synchronous flush and aggressive compaction for every column family, then drains both the global flush and compaction queues.

```
purge
```

Behavior

- Calls `purge-cf` on each column family
- Drains the global flush queue
- Drains the global compaction queue

Example

```
admintool(./mydb)> purge
Purging all column families (synchronous flush + compaction)...
Database purge completed.
```

sync-wal

Force an immediate fsync of the active write-ahead log for a column family. Useful for explicit durability control when using `TDB_SYNC_NONE` or `TDB_SYNC_INTERVAL` modes.

```
sync-wal <cf>
```

Use cases

- Application-controlled durability at specific points
- Pre-checkpoint to ensure all buffered WAL data is on disk
- Graceful shutdown to flush WAL buffers
- Force durability for specific high-value writes

Example

```
admintool(./mydb)> sync-wal users  
WAL synced for 'users'
```

checkpoint

Create a lightweight, near-instant snapshot of the open database using hard links instead of copying SSTable data.

```
checkpoint <destination_path>
```

Behavior

- Requires destination to be a non-existent or empty directory
- Flushes active memtables so all data is in SSTables
- Halts compactions to ensure a consistent view
- Hard links all SSTable files (`.klog` and `.vlog`) into the checkpoint directory
- Copies small metadata files (manifest, config)
- Resumes compactions
- Falls back to file copy if hard linking fails (e.g., cross-filesystem)

Checkpoint vs Backup

	backup	checkpoint
Speed	Copies every SSTable byte-by-byte	Near-instant (hard links)
Disk usage	Full independent copy	No extra disk until compaction removes old SSTables
Portability	Can be moved to another filesystem or machine	Same filesystem only
Use case	Archival, disaster recovery	Fast local snapshots, point-in-time reads

Example

```

admintool(/mydb)> checkpoint ./mydb_checkpoint
Creating checkpoint at './mydb_checkpoint'...
Checkpoint completed successfully.

```

db-stats

Show aggregate statistics across the entire database instance.

```
db-stats
```

Output includes

- Column family count
- System total and available memory
- Resolved memory limit and memory pressure level (normal/elevated/high/critical)
- Global sequence number
- Flush queue size and pending count
- Compaction queue size
- Total SSTables, total data size
- Open SSTable handles
- In-flight transaction memory
- Immutable memtable count and total memtable bytes
- Unified memtable section (if enabled): memtable bytes, immutable count, flushing status, WAL generation, next CF index

- Object store section (if enabled): connector type, replica mode, local cache usage, upload stats, and single-writer fencing epochs (primary epoch held by this node and the highest lease epoch seen)
- Write amplification section: lifetime on-disk framed bytes for the shared unified WAL, per-CF WALs, flushes, and compactions (summed across all column families); compaction bytes read; logical user bytes committed; flush and compaction output SSTable counts; and the derived database-wide write-amplification ratio

Example

```
admintool(./mydb)> db-stats
Database Statistics:
  Path: ./mydb
  Column Families: 2
  Total Memory: 16777216000 bytes (16000.00 MB)
  Available Memory: 8388608000 bytes (8000.00 MB)
  Resolved Memory Limit: 8388608000 bytes (8000.00 MB)
  Memory Pressure Level: 0 (normal)
  Global Sequence: 1042
  Flush Queue Size: 0
  Flush Pending Count: 0
  Compaction Queue Size: 0
  Total SSTables: 5
  Total Data Size: 1048576 bytes (1.00 MB)
  Open SSTable Handles: 5
  In-Flight Txn Memory: 0 bytes
  Immutable Memtables: 0
  Total Memtable Bytes: 2048
  Unified Memtable:
    Enabled: yes
    Memtable Bytes: 4096
    Immutable Count: 0
    Is Flushing: no
    Next CF Index: 3
    WAL Generation: 7
  Write Amplification (lifetime, on-disk framed bytes):
    Unified WAL Bytes Written: 1048576
    WAL Bytes Written: 0
    Flush Bytes Written: 2097152
    Compaction Bytes Written: 4194304
    Compaction Bytes Read: 4194304
    User Bytes Written: 1572864
    Flush Count: 4
    Compaction Count: 2
    Write Amplification: 4.67x
```

Example with object store


```
admintool(./mydb)> db-stats
...
Object Store:
  Enabled: yes
  Connector: s3
  Replica Mode: no
  Local Cache: 524288000 / 1073741824 bytes (42 files)
  Last Uploaded Generation: 6
  Upload Queue Depth: 0
  Total Uploads: 156
  Total Upload Failures: 0
  Primary Epoch: 3
  Seen Epoch: 3
```

cache-stats

Show detailed block cache statistics.

```
cache-stats
```

Output includes

- Whether cache is enabled
- Total cached entries and size (bytes and MB)
- Cache hits and misses
- Hit rate percentage
- Number of cache partitions

Example

```
admintool(./mydb)> cache-stats
Block Cache Statistics:
  Enabled: yes
  Entries: 128
  Size: 8388608 bytes (8.00 MB)
  Hits: 5432
  Misses: 312
  Hit Rate: 94.57%
  Partitions: 8
```

range-cost

Estimate the computational cost of iterating between two keys in a column family. Uses only in-memory metadata with no disk I/O. The returned value is a relative scalar, meaningful only for comparison with other `range-cost` results.

```
range-cost <cf> <key_a> <key_b>
```

Example

```
admintool(./mydb)> range-cost users user:0000 user:0999
Range cost for 'users' [user:0000 .. user:0999]: 42.500000

admintool(./mydb)> range-cost users user:1000 user:1099
Range cost for 'users' [user:1000 .. user:1099]: 8.250000
```

Use cases

- Query planning: compare candidate key ranges to find the cheapest to scan
- Load balancing: distribute range scan work across threads
- Monitoring: track data distribution changes across key ranges

Cost Values

A cost of 0.0 means no overlapping SSTables or memtable entries were found. Key order does not matter — the function normalizes the range.

promote

Promote a read-only replica to primary. This performs a final MANIFEST sync and WAL replay, creates a local WAL for crash recovery, and atomically switches to primary mode.

```
promote
```

Example

```
admintool(./mydb_replica)> promote  
Promoted to primary successfully.
```

Behavior

- Only valid when the database is in replica mode
- Performs a final MANIFEST sync and WAL replay
- Atomically transitions from replica to primary mode
- After promotion, writes are accepted normally
- Returns an error if the database is already a primary

cancel-background-work

Cancel background compaction across the whole database for a fast shutdown. In-flight merges bail out safely and queued compaction is skipped; flushes are left untouched so durability is preserved. The cancellation is sticky for the session and is reset on the next `open` — typically issued right before `close`.

```
cancel-background-work
```

Example

```
admintool(./mydb)> cancel-background-work  
Background compaction cancelled (flushes preserved). Sticky until next  
open.
```

raise-open-file-limit

Raise this process's open-file ceiling so the engine can keep more SSTables open. Because the engine sizes `max_open_sstables` to fit the ceiling at open time, run this **before** `open`. With no argument (or a non-positive one) it just reports the current ceiling.

```
raise-open-file-limit [n]
```

Example

```
admintool> raise-open-file-limit 100000
Requested open-file ceiling 100000; effective ceiling now 100000.
admintool> open ./mydb
```

Transactions

admintool can hold a single transaction open across interactive prompts, which makes the isolation, savepoint, and reset APIs reachable from the command line. Begin a transaction with `txn-begin`, stage operations with the `txn-*` commands, then finish with `txn-commit` or `txn-rollback`. Closing the database (or leaving admintool) automatically rolls back an open transaction.

txn-begin

Begin a transaction, optionally at a specific isolation level (default `read_committed`).

```
txn-begin
[read_uncommitted|read_committed|repeatable_read|snapshot|serializable]
```

txn-status

Show whether a transaction is active and its isolation level.

```
txn-status
```

txn-put

Stage a put in the active transaction.

```
txn-put <cf> <key> <value> [--ttl <seconds>]
```

txn-get

Read a key through the active transaction (sees the transaction's own uncommitted writes).

```
txn-get <cf> <key>
```

txn-delete

Stage a delete in the active transaction.

```
txn-delete <cf> <key>
```

txn-single-delete

Stage a single-delete in the active transaction (see [single-delete](#) for the contract).

```
txn-single-delete <cf> <key>
```

txn-savepoint

Create a named savepoint within the active transaction.

```
txn-savepoint <name>
```

txn-rollback-to

Roll the transaction back to a previously created savepoint, discarding everything staged after it.

```
txn-rollback-to <name>
```

txn-release

Release (drop) a savepoint without rolling back.

```
txn-release <name>
```

txn-reset

Reset the active transaction for reuse, optionally changing the isolation level.

```
txn-reset  
[read_uncommitted|read_committed|repeatable_read|snapshot|serializable]
```

txn-commit

Commit the active transaction.

```
txn-commit
```

txn-rollback

Roll back and discard the active transaction.

```
txn-rollback
```

Example

```
admintool(./mydb)> txn-begin snapshot
Transaction started (isolation: snapshot).
admintool(./mydb)> txn-put users user:1 "John"
Staged put in transaction.
admintool(./mydb)> txn-savepoint sp1
Savepoint 'sp1' created.
admintool(./mydb)> txn-put users user:2 "Jane"
Staged put in transaction.
admintool(./mydb)> txn-rollback-to sp1
Rolled back to savepoint 'sp1'.
admintool(./mydb)> txn-commit
Transaction committed.
```

Interactive Mode

When run without `-c`, admintool enters interactive mode with a prompt:

```
admintool>
```

When a database is open:

```
admintool(./mydb)>
```

Type `help` for available commands, `quit` or `exit` to leave.

Examples

Create database and column family

```
admintool> open ./mydb
Opened database at './mydb'
admintool(./mydb)> cf-create users
Created column family 'users'
```

Create database with unified memtable

```
admintool> open ./mydb --unified --cache-size 134217728
Opened database at './mydb' (unified memtable)
```

Create column family with custom settings

```
admintool(./mydb)> cf-create logs --compression zstd --sync interval -
-sync-interval 100000 --bloom-fpr 0.001
Created column family 'logs'
```

Update column family configuration at runtime

```
admintool(./mydb)> cf-update users --compression zstd --write-buffer-
size 268435456
Configuration updated for 'users' (persisted to disk)
```

Insert and retrieve data

```
admintool(./mydb)> put users user:1 "John Doe"
OK
admintool(./mydb)> get users user:1
John Doe
```

Insert with TTL

```
admintool(./mydb)> put sessions sess:abc "token123" --ttl 3600
OK (expires at 1711584000)
```

Inspect column family


```
admintool(/mydb)> cf-stats users
Column Family: users
  Memtable Size: 1024 bytes
  Levels: 5
  Total Keys: 1
  ...
```

Scan with prefix

```
admintool(/mydb)> prefix users user: 10
1) "user:1" -> "John Doe"
(1 entries with prefix)
```

Compare range scan costs

```
admintool(/mydb)> range-cost users user:0000 user:0999
Range cost for 'users' [user:0000 .. user:0999]: 42.500000
admintool(/mydb)> range-cost users user:1000 user:1099
Range cost for 'users' [user:1000 .. user:1099]: 8.250000
```

Verify integrity

```
admintool(/mydb)> verify users
Verifying column family 'users'...
Verification passed.
```

TidesDB Tuning Reference

This reference gives small, mid, and large scale configuration presets for the engine and column families configuration, tuned for modern systems. These configurations can be applied to integrations such as FFI libraries and TideSQL. Every number here is derived from the engine knobs, defaults, and clamps in `tidesdb.h` and `tidesdb.c`, not from guesswork. Read the mental model first, then pick the tier that matches your hardware and adjust it with the workload overlays.

The library ships with a 64 MB write buffer, a level ratio of 10, a 1 percent bloom false-positive rate, a 512 byte value threshold, an L1 trigger of 4, an L0 stall of 10, two flush and two compaction threads, a 64 MB block cache, 256 open SSTables, a memory limit of 50 percent of RAM, READ_COMMITTED isolation, and LZ4 compression. The presets below move away from these defaults only where a tier benefits.

What “modern systems” means here

Tier	Profile	RAM for the DB	Cores	Storage	Scale
Small	edge, container, sidecar, embedded	1 to 4 GB	2 to 4	eMMC / SATA SSD / constrained NVMe	up to tens of GB, a handful of CFs
Mid	single production server, typical cloud VM	8 to 32 GB	8 to 16	NVMe SSD	up to ~1 TB, dozens of CFs
Large	storage node, high-end server	64 to 512 GB	32 to 128	fast NVMe (or striped)	multi-TB, many CFs, high concurrency

The mental model

A few formulas in the engine determine almost everything, so it is better to tune with these in mind than knob by knob.

Per-CF write memory is bounded by two mechanisms acting together. The L0 stall throttles writes once the immutable queue reaches `l0_queue_stall_threshold`, and each immutable holds one `write_buffer_size`. The active memtable is hard-capped at `2 x write_buffer_size`. The worst-case steady-state memory for one busy column family is therefore about

```

per_cf_mem  ~=  l0_queue_stall_threshold * write_buffer_size  (queued
immutable)
                + 2 * write_buffer_size                      (active
memtable ceiling)
                + bloom filters + block indexes              (small,
resident)

```

Above the stall sits a last-resort hard cap of `l0_queue_stall_threshold + 6` immutables. It scales with the threshold, so raising the threshold raises the cap in lockstep and there is no hidden ceiling.

When more than one column family shares the database in per-CF mode, the effective L0 stall is reduced to `resolved_memory_limit / (num_cfs * write_buffer_size)`, floored at 2. The more column families you run this way, the sooner each one stalls, so the total never exceeds the memory budget. If you have many column families, either give the database a larger `max_memory_usage` or switch to unified memtable mode, which shares one queue and ignores this split.

A `max_memory_usage` of 0 resolves to 50 percent of system RAM, with a floor of 5 percent. A non-zero value is honored but clamped up to that 5 percent floor. The block cache is clamped separately so that both internal caches together use at most 30 percent of the resolved limit. Set the limit explicitly in containers, where the host RAM the engine measures is not your cgroup limit.

Each open SSTable holds two file descriptors. At open time the engine clamps `max_open_sstables` to `(fd_limit - 64) / 2`, so a target of N open SSTables needs roughly `2N + 64` descriptors. About one eighth of that budget is reserved for the write path, and reads back off with the retryable `TDB_ERR_BUSY` at the cap. To run a large `max_open_sstables`, raise the ceiling before `tidesdb_open()`.

```

tidesdb_raise_open_file_limit(32768); /* opt-in, POSIX RLIMIT_NOFILE
/ Windows CRT cap */

```

The `max_concurrent_flushes` count is pinned one to one to `num_flush_threads` at open, and a mismatch is normalized with a warning. A single hot column family drains its immutable queue at the speed of the flush pool. Compaction runs one round per column family, but each round fans out across the compaction pool through sub-compaction.

Database-level presets

Field	Small	Mid	Large	Notes
<code>num_flush_threads</code>	2	4	8 to 16	match to concurrently hot CFs; pins <code>max_concurrent_flushes</code>
<code>num_compaction_threads</code>	2	4	8 to 16	up to core count on NVMe, 1 to 2 on HDD
<code>block_cache_size</code>	32 MB	1 to 4 GB	16 to 64 GB	shared across all CFs; clamped to 30 percent of mem limit
<code>max_open_stables</code>	64 to 128	512 to 1024	4096 to 16384	needs <code>2N + 64</code> fds; raise ulimit for mid and large
<code>max_memory_usage</code>	512 MB to 1 GB	0 (50 percent RAM) or explicit	explicit (~50 percent RAM)	set explicitly inside containers
<code>log_to_file</code>	1	1	1	keep <code>LOG</code> in the data dir
<code>log_level</code>	<code>TDB_LOG_WARN</code>	<code>TDB_LOG_WARN</code>	<code>TDB_LOG_INFO</code>	drop to WARN in steady state
<code>unified_memtable</code>	0 (1 if many CFs)	per workload	per workload	see the many-column-family overlay

The `max_concurrent_flushes` field is left at its default so it tracks `num_flush_threads`.

Column-family presets

Field	Small	Mid	Large	Notes
<code>write_buffer_size</code>	8 to 16 MB	64 MB	128 to 256 MB	bigger batches, more memory, bigger L1 SSTables
<code>level_size_ratio</code>	10	10	10 (12 to 15 for multi-TB)	higher ratio means fewer levels and more write amp
<code>min_levels</code>	1	1	1 to 2	a deeper floor avoids early level churn on big sets
<code>dividing_level_offset</code>	1	1	1	lower is more aggressive compaction
<code>l0_queue_size_threshold</code>	6 to 8	10	16 to 20	drives per-CF write memory (see model)
<code>l1_file_count_trigger</code>	4	4	4 to 8	higher tolerates burstier flushing
<code>klog_value_threshold</code>	512	512	512 to 1024	values at or above this go to the value log
<code>compression_algorithm</code>	<code>TDB_COMPRESS_LZ4</code>	<code>TDB_COMPRESS_LZ4</code>	<code>TDB_COMPRESS_LZ4</code> (ZSTD for cold)	LZ4 is the throughput default
<code>enable_bloom_filter</code>	1	1	1	almost always worth it
<code>bloom_fpr</code>	0.01	0.01	0.01 (0.005 read-heavy)	lower FPR means more bits per key
<code>enable_block_indexes</code>	1	1	1	required for fast seeks
<code>index_sample_ratio</code>	1	1	1	1 indexes every block, making lookups definitive
<code>block_index_prefix_length</code>	16	16	16 to 32	raise if keys share long prefixes

Field	Small	Mid	Large	Notes
<code>sync_mode</code>	<code>TDB_SYNC_INTERVAL</code>	<code>TDB_SYNC_IN_TERVAL</code> or <code>FULL</code>	<code>TDB_SYNC_INTE_RVAL</code> or <code>FULL</code>	see durability
<code>sync_interval_us</code>	128000	128000	128000	only used with INTERVAL
<code>default_isolation_level</code>	<code>READ_COMMITTED</code>	<code>READ_COMMITTED</code>	per workload	SERIALIZABLE only when you need SSI
<code>use_btree</code>	0	0 (1 for point and seek-heavy)	0 (1 for point and seek-heavy)	B+tree klog favors point lookups and seeks

What each tier costs in memory

Using the per-CF model above, one busy column family bounds to roughly the following.

Tier	buffer	L0 stall	worst-case per-CF write memory
Small	16 MB	8	$8 \times 16 + 2 \times 16 = \sim 160$ MB
Mid	64 MB	10	$10 \times 64 + 2 \times 64 = \sim 768$ MB
Large	128 MB	16	$16 \times 128 + 2 \times 128 = \sim 2.3$ GB

Multiply by the number of concurrently hot column families and keep the total under `max_memory_usage`. In per-CF mode the engine auto-scales the stall down to protect the budget, but sizing it yourself avoids surprise throttling.

What each tier costs in file descriptors

Tier	<code>max_open_sstable</code>	fds needed ($2N + 64$)	recommended <code>ulimit -n</code>
Small	128	~ 320	1024 (default is fine)
Mid	1024	~ 2112	4096 or more
Large	8192	~ 16448	32768 or more (call <code>tidesdb_raise_open_file_limit</code>)

Full examples

Small

```
tidesdb_config_t db = tidesdb_default_config();
db.db_path          = "./data";
db.num_flush_threads = 2;
db.num_compaction_threads = 2;
db.block_cache_size  = 32 * 1024 * 1024;    /* 32 MB */
db.max_open_sstables = 128;                 /* ~320 fds */
db.max_memory_usage  = 768 * 1024 * 1024;    /* explicit cap for a small
box */
db.log_to_file       = 1;
db.log_level         = TDB_LOG_WARN;

tidesdb_column_family_config_t cf = tidesdb_default_column_family_config();
cf.write_buffer_size  = 16 * 1024 * 1024;    /* 16 MB */
cf.l0_queue_stall_threshold = 8;
cf.compression_algorithm = TDB_COMPRESS_LZ4;
cf.sync_mode          = TDB_SYNC_INTERVAL;
```

Mid

```
tidesdb_raise_open_file_limit(8192);          /* before open */

tidesdb_config_t db = tidesdb_default_config();
db.db_path          = "./data";
db.num_flush_threads = 4;
db.num_compaction_threads = 4;
db.block_cache_size  = 2ull * 1024 * 1024 * 1024; /* 2 GB */
db.max_open_sstables = 1024;                      /* ~2112 fds */
db.max_memory_usage   = 0;                          /* 50 percent of
RAM */
db.log_to_file        = 1;
db.log_level          = TDB_LOG_WARN;

tidesdb_column_family_config_t cf = tidesdb_default_column_family_config();
cf.write_buffer_size    = 64 * 1024 * 1024; /* 64 MB (default) */
cf.l0_queue_stall_threshold = 10;
cf.compression_algorithm = TDB_COMPRESS_LZ4;
cf.sync_mode            = TDB_SYNC_INTERVAL; /* TDB_SYNC_FULL for
strict durability */
```


Large

```
tidesdb_raise_open_file_limit(65536);
/* before open */

tidesdb_config_t db = tidesdb_default_config();
db.db_path          = "./data";
db.num_flush_threads = 12;
db.num_compaction_threads = 12;
db.block_cache_size  = 32ull * 1024 * 1024 * 1024;
/* 32 GB */
db.max_open_sstables = 8192;
/* ~16448 fds */
db.max_memory_usage   = 128ull * 1024 * 1024 * 1024;
/* explicit on a 256 GB box */
db.log_to_file         = 1;
db.log_level           = TDB_LOG_INFO;

tidesdb_column_family_config_t cf = tidesdb_default_column_family_config();
cf.write_buffer_size      = 128 * 1024 * 1024;
cf.l0_queue_stall_threshold = 16;
cf.level_size_ratio       = 10;
/* 12 to 15 for very large datasets */
cf.l1_file_count_trigger  = 6;
cf.compression_algorithm  = TDB_COMPRESS_LZ4;
cf.sync_mode              = TDB_SYNC_INTERVAL;
```

Workload overlays

Apply these on top of a tier.

For write-heavy ingest, use a larger `write_buffer_size` for better batching and a higher `l0_queue_stall_threshold` to absorb bursts at the cost of memory, add flush threads, and choose `TDB_SYNC_INTERVAL`, or `TDB_SYNC_NONE` for a rebuildable cache. Keep `level_size_ratio` at 10 to bound write amplification, and raise it only if reads can absorb the extra level depth.

For read-heavy point lookups, lower `bloom_fpr` to 0.005 or 0.002 to cut false positives, give the block cache more room, and consider `use_btree = 1` for $O(\log n)$ klog lookups instead of block scans. Keep `index_sample_ratio` at 1 so block index lookups are definitive and can short-circuit negative reads.

For range scans and iteration, `use_btree = 1` helps seek-then-scan, block indexes must stay enabled, and a larger block cache keeps hot blocks resident. Raise `block_index_prefix_len` if your keys share long common prefixes.

For delete-heavy workloads, arm the tombstone density trigger so delete-dominated column families are compacted before range scans degrade.

```
cf.tombstone_density_trigger      = 0.30;  /* compact when an sstable
is >30% tombstones */
cf.tombstone_density_min_entries = 1024;  /* ignore tiny sstables */
```

Use `tidesdb_txn_single_delete` for keys put at most once between deletes, which lets a put and its delete cancel at the first merge regardless of level, and `tidesdb_compact_range` to reclaim a known range immediately.

For large values, raise `klog_value_threshold` so more of them move to the value log and keep the klog scannable, and lower it if values are small and hot so they stay inline. Raise `multipart_part_size` in object store mode for very large SSTables.

For many column families that map to one logical entity, such as a table plus N secondary index column families, enable unified memtable mode so a transaction touching K column families does one WAL write instead of K.

```
db.unified_memtable              = 1;
db.unified_memtable_write_buffer_size = 0;  /* 0 => 64 MB
(TDB_DEFAULT_WRITE_BUFFER_SIZE) */
db.unified_memtable_sync_mode     = TDB_SYNC_INTERVAL;
```

Unified mode requires the default `memcmp` comparator for every column family, since the shared skip list has a single sort order, and it shares one immutable queue, so the per-CF stall auto-scaling does not apply. This is the right default once you exceed a handful of column families per transaction.

Unified memtable sizing and the L0 and L1 split

Unified mode changes what `write_buffer_size` and the L0 stall mean, and the two buffer knobs drive different things. This is the subtle part.

The `unified_memtable_write_buffer_size` sizes the one shared memtable. All column families write into it, and it rotates as a whole the moment its total size reaches that value. The threshold is fixed, with no adaptive idle headroom, unlike per-CF mode, which rotates at up to 1.5x its buffer. A

value of 0 means 64 MB, the `TDB_DEFAULT_WRITE_BUFFER_SIZE` constant, not any column family's `write_buffer_size`.

Each column family's own `write_buffer_size` no longer sizes a memtable in unified mode, and the per-CF active and immutable memtables stay empty. That value still matters, but only as level geometry. It is the capacity floor for that column family's levels, since DCA never sizes a level below it, so it governs the on-disk level shape rather than the in-memory footprint.

L0 is shared while L1 and everything below it stays per-CF, and this asymmetry is the heart of the model. There is exactly one L0 flush queue, `unified_mt.immutables`. The L0 stall, the hard cap of `stall + 6`, and the graduated L0 delays all measure that single shared queue. Backpressure is still evaluated once per column family in a commit, comparing the shared queue depth against each participating column family's `l0_queue_stall_threshold`, so the smallest configured threshold among the column families in a transaction is the one that actually stalls it. Set `l0_queue_stall_threshold` consistently across column families in unified mode, or expect the minimum to win.

L1 stays per column family. Flush demux writes each column family's slice of the rotated memtable to that column family's own level 1, so every column family keeps its own L1 file count, its own `l1_file_count_trigger`, and its own compaction. L1 never hard-stalls writes in either mode and only applies graduated delays, and compaction stays one round per column family.

Memory in unified mode is database-wide rather than per column family. Because the queue and the active memtable are shared, the worst-case write memory is bounded once for the whole database, regardless of column family count.

```
unified_write_mem  ~=  l0_queue_stall_threshold *
unified_memtable_write_buffer_size
                    + 2 * unified_memtable_write_buffer_size    (shared
active ceiling, 2x)
```

With the defaults of a 64 MB unified buffer and a stall of 10, that is about $(10 + 2) * 64 \text{ MB} = \sim 768 \text{ MB}$ total, no matter how many column families you run. That fixed, count-independent ceiling is the main reason to choose unified mode for many-CF workloads. In per-CF mode the bound is instead per column family, and the engine auto-scales each one's stall down to protect the budget. So in unified mode you tune memory with the unified buffer and a single `l0_queue_stall_threshold`, not with per-CF `write_buffer_size`.

For object store and cloud-native deployments, set `object_store` and an `tidesdb_objstore_config_t`, and unified memtable mode is enabled automatically. Give `local_cache_max_bytes` enough room for the hot working set, keep `replicate_wal = 1`, and set

`wal_sync_on_commit = 1` for an RPO of zero, or rely on the 1 MB `wal_sync_threshold_bytes` for a bounded loss window. The `max_concurrent_uploads` count defaults to 4 and `max_concurrent_downloads` to 8, and both are worth raising on large nodes. Two per-CF knobs trade remote I/O against read amplification. The `object_lazy_compaction` flag, off by default, doubles the L1 file-count compaction trigger when an object store is attached, which cuts compaction frequency and upload churn at the cost of more files to read. The `object_prefetch_compaction` flag, on by default, downloads all evicted merge inputs in parallel before a compaction instead of one at a time.

Durability

Mode	Behavior	Use when
<code>TDB_SYNC_NONE</code>	WAL written, not fsynced per commit, synced at flush	rebuildable cache, max throughput
<code>TDB_SYNC_INTERVAL</code>	background fsync every <code>sync_interval_us</code> (default 128 ms)	general purpose, good throughput with a bounded loss window
<code>TDB_SYNC_FULL</code>	fsync coalesced across concurrent committers	strict durability; unified-mode group commit keeps it fast

In full-sync unified mode the WAL fsync is coalesced into one fsync per batch of concurrent committers, so `TDB_SYNC_FULL` stays performant at high concurrency. In per-CF mode every commit fragments across separate WALs, so `TDB_SYNC_FULL` costs one fsync per commit.

Lower-impact knobs

These rarely need changing from their defaults, but they are real and worth knowing.

The `skip_list_max_level` (default 12) and `skip_list_probability` (default 0.25) are the standard probabilistic skip-list parameters. Keep `skip_list_max_level` below 64, because the memtable write path uses a stack-allocated update array only when the level is under 64, and a higher value forces a per-operation heap allocation. A level of 12 already indexes far more entries than a memtable holds. The unified memtable has its own `unified_memtable_skip_list_max_level` and `unified_memtable_skip_list_probability`, and a 0 for either resolves to the same 12 and 0.25 defaults.

The `min_disk_space` field (default 100 MB) is a safety floor. Flush and compaction are skipped while free space on the column family directory is below it, so the data stays in memory and writes keep flowing until memory pressure intervenes. Raise it if you want a larger headroom before the engine stops writing new files.

The `log_truncation_at` field (default 24 MB) matters only with `log_to_file = 1`. The `LOG` file is truncated once it grows past this size, which bounds log disk use.

The `num_compaction_threads` count sets both the number of compaction worker threads and the shared sub-compaction helper budget, so one round of a single column family can fan out across the whole pool. Size it to your storage parallelism, one to two on HDD and up to the core count on NVMe, not to your column family count.

Things to watch

Set `max_memory_usage` explicitly in containers, because the auto-resolve uses host RAM rather than your cgroup limit.

A high `max_open_sstables` is silently clamped if the file-descriptor limit cannot honor it, so call `tidesdb_raise_open_file_limit()` before `tidesdb_open()`.

The `max_concurrent_flushes` count follows `num_flush_threads`, so set the thread count and do not fight the one-to-one pin.

The comparator is permanent. It cannot change after a column family is created without corrupting key order, and unified mode forces `memcmp`.

Many column families in per-CF mode self-throttle, since the effective L0 stall shrinks with column family count. Grow `max_memory_usage` or switch to unified mode.

The block cache shares the 30 percent memory clamp with the B+tree node cache, so a large `block_cache_size` may be clamped down relative to `max_memory_usage`.

The `min_disk_space` floor gates flush and compaction, not just writes. If free space drops below it, new SSTables stop being written and memory climbs until pressure relief intervenes, so monitor disk free space against this floor.

A `skip_list_max_level` of 64 or more costs a heap allocation on every memtable write, so stay below 64. The default 12 is fine.